VBD

Based on R.T.M. Nagpur University New Syllabus

Based on New Syllabus

THEORY OF COMPUTATION

Prof. Dinesh Gawande, Prof. Vaibhav Tumane

SOLVED QUESTION BANK + UNIVERSITY PAPER SOLUTIONS

B.TECH. IV SEMESTER

COMP. SCI. & ENGG., ARTIFICIAL INTELLIGENCE (CSE, AI)

B.TECH. III SEMESTER

COMP. ENGG., COMP. TECH.(CE, CT)

! Xerox center सावधान !

प्रिय विद्यार्थी, VBD की Xerox करने वाले Xerox center की संपूर्ण जानकारी दीजिए और इनाम पाइए। यदि हमारी जांच में आपके द्वारा दी गई जानकारी सही पाई गई तो आपको 5000/- रुपये का इनाम दिया जाएगा।* उपरोक्त जानकारी WhatsApp No. 9373644557 या Email id : vbdnagpur@gmail.com पर भेजें।

VBD

THEORY OF COMPUTATION

FOR
B.Tech. III SEMESTER
Computer Science & Engineering, Computer Technology
(CE, CT)

FOR
B.Tech. IV SEMESTER
Computer Science & Engineering, Artificial Intelligence
(CSE, AI)

By

Prof. Dinesh Gawande
M.Tech. (CSE)
Assistant Professor

Prof. Vaibhav Tumane
M.E. (SE)
Assistant Professor

Based on R.T.M. Nagpur University New Syllabus

VBD PUBLICATIONS

Distributed by:
VBD **Publications Pvt. Ltd.**
477, Golchha Marg
Sadar Nagpur – 440001.

Sales Office:
Plot No. 236, Near Ram Coolers,
Besides Ganesh Temple,
Singada Market, Navi Shukrawari,
Mahal, Nagpur - 32
☎: 0712 - 2722235, 7620266004, Fax : 0712 - 2722236
Website : www.vbdpublications.com
E-mail : vbdbooks@gmail.com

© Publishers

Price : Rs. 620/-

Edition : 2022

The shelf-life of this book is 6 months.

फोटोकॉपी (झेराक्स) करने से मैटर बहुत छोटा हो जाता है और इसे पढ़ने से आपकी आँखें कमजोर होती हैं।
As per Indian Copyright Act 1957, photocopying of a book is punishable under law.

Books Available At ALL LEADING BOOK SELLERS
OR CONTACT:
VIJAY BOOK DEPOT
GOLCHHA MARG, SADAR, NAGPUR
☎: 0712 - 2520496

For online purchases, please visit WWW.VBDPUBLICATIONS.COM

The text of this publication, or any part thereof, should not be reproduced or transmitted in any form or stored in any computer storage system or device for distribution without the prior written permission of the publisher.

Note: While every care and precautions have been taken regarding the contents and data of this book, the publisher does not hold responsibility for any error or omissions. Please refer prescribed text books. Any dispute will be subject to Nagpur Jurisdiction only.

Published by : ABD Publishers & Printers Pvt. Ltd., Nagpur.

Statistical Analysis

Exam. / Unit No.	W-13 (CS)	S-14 (CS)	W-13 (CT)	S-14 (CT)	W-14 (CS)	S-15 (CS)	W-14 (CT)	S-15 (CT)
Unit - I	15 M	28 M	00 M	26 M	28 M	26 M	26 M	26 M
Unit-II	16 M	26 M	27 M	28 M	26 M	26 M	28 M	26 M
Unit-III	41 M	26 M	32 M	41 M	28 M	28 M	26 M	28 M
Unit-IV	13 M	28 M	07 M	13 M	26 M	28 M	28 M	26 M
Unit-V	28 M	23 M	26 M	26 M	30 M	26 M	26 M	26 M
Unit-VI	17 M	29 M	40 M	26 M	22 M	26 M	26 M	28 M
Questions covered in VBD	130 M (100%)	160 M (100 %)	132 M (100 %)	160 M (100 %)	160 M (100 %)	160 M (100 %)	160 M (100 %)	160 M (100 %)

Exam. / Unit No.	W-15 (CS)	S-16 (CS)	W-15 (CT)	S-16 (CT)	W-16 (CS)	S-17 (CS)	W-16 (CT)	S-17 (CT)	W-17 (CS)
Unit - I	26 M	26 M	28 M	26 M	26 M	26 M	26 M	26 M	26 M
Unit-II	26 M	28 M	26 M	28 M	28 M	28 M	26 M	26 M	26 M
Unit-III	28 M	26 M	26 M	26 M	26 M	26 M	28 M	28 M	28 M
Unit-IV	28 M	28 M	26 M	26 M	26 M	28 M	28 M	28 M	28 M
Unit-V	28 M	26 M	26 M	33 M	28 M	26 M	26 M	26 M	26 M
Unit-VI	24 M	26 M	28 M	21 M	26 M	26 M	26 M	26 M	26 M
Questions covered in VBD	160 M (100%)	160 M (100 %)	160 M (100 %)	160 M (100 %)	160 M (100%)	160 M (100 %)	160 M (100 %)	160 M (100 %)	160 M (100%)

Exam. / Unit No.	S-18 (CS)	W-17 (CT)	S-18 (CT)	W-18 (CS)	S-19 (CS)	W-18 (CT)	S-19 (CT)	W-19 (CS)	W-19 (CT)
Unit - I	26 M	28 M	26 M	26 M	26 M	26 M	26 M	26 M	26 M
Unit-II	26 M	26 M	26 M	28 M	26 M	26 M	28 M	28 M	28 M
Unit-III	28 M	26 M	28 M	26 M	28 M	28 M	26 M	26 M	26 M
Unit-IV	26 M	14 M	28 M	28 M	26 M	28 M	26 M	26 M	28 M
Unit-V	28 M	31 M	26 M	30 M	34 M	26 M	28 M	28 M	26 M
Unit-VI	26 M	35 M	26 M	22 M	20 M	26 M	26 M	26 M	26 M
Questions covered in VBD	160 M (100 %)	160 M (100 %)	160 M (100 %)	160 M (100%)	160 M (100 %)	160 M (100 %)	160 M (100 %)	160 M (100 %)	160 M (100 %)

Note:- Due to Covid - 19 Summer - 2020 to Winter - 2021 exams were conducted online by the respective Colleges.

PREFACE

This *VBD* on **Theory of Computation** has been prepared for the students of ***Third Semester Bachlor of Technology (CBCS)- Computer Engineering, Computer Technology and*** **Fourth Semester Bachelor of** ***Technology (CBCS)- Computer Science and Engineering and Artificial Intelligence*** as per the New Syllabus prescribed by the RTM-Nagpur University.

The subject matter has been thoroughly dealt with, covering 100 % syllabus.

Each unit begins with its syllabus. The previously asked University questions are fully covered in the order as per the syllabus.

In each unit, sufficient theory is given then followed by sample Assembly language programs.

Every type of Assembly language program is included alongwith previously asked.

Fully solved University Question Papers are included at the end with solutions reference inside.

We are grateful to the publishers and the staff of *VBD* Publications, Nagpur, who have taken considerable pains and shown extreme co-operation during the preparation of this book.

We will appreciate suggestions from teachers and students for the improvement of the book in the future editions.

Authors

SYLLABUS

[B.Tech. THIRD SEMESTER CE]

UNIT- I **See Unit-I and II of this book and appendix**

Introduction to Theory of Automaton, Strings, Alphabet, language, Chomsky hierarchy of languages, Finite state machine definitions, Finite automation model acceptance of strings and language, Non deterministic finite automaton, Deterministic finite automaton.

UNIT- II **See Unit-II of this book**

Equivalence between NFA and DFA. Conversion of NFA into DFA, NFA with e-moves and e-closure with examples, Conversion of NFA with e moves to without e-moves Minimization of FSM, Equivalence between two FSM's, Mealy Machine and Moore Machine and its conversion.

UNIT- III **See Unit-III of this book and appendix**

Introduction to Regular languages and Regular expression, Regular set, Regular Expression and Regular languages sample examples, Regular expression examples, Equivalence and Inter Conversion between Regular Expression and FA, Conversion from RE to RG

UNIT- IV **See Unit-III of this book**

Introduction to Context Free Grammar (CFG) and Derivation Trees, Left Derivation Tree and Right Derivation Tree, CFG minimization (Reduction of CFG), Chomsky normal form(CNF), Greibach normal form(GNF)

UNIT- V **See Unit-V and VI of this book**

Introduction to Push Down Automaton (PDA) Definition and working principle, Implementation of Push Down Automaton. Introduction to Turing machines, Definition, Model of TM, Types of TM, Designing of TM. Decidability and Undecidability of problems, Properties of recursive & recursively enumerable languages, Post correspondence problem, Ackermann function.

CONTENTS

[B.Tech. THIRD SEMESTER CSE]

UNIT -I:

- Introduction to Theory of Automaton — A-1
- Strings, Alphabet, language — P.1-3, 4, 10
- Chomsky hierarchy of languages — P.1-11
- Finite state machine definitions — P.2-9
- Finite automation model acceptance of strings and language — P.2-8 to 2-12
- Non deterministic finite automaton — P.2-11
- Deterministic finite automaton — P.2-11

UNIT- II:

- Equivalence between NFA and DFA — P.2-29
- Conversion of NFA into DFA — P.2-13
- NFA with e-moves and e-closure with examples — P.2-21
- Conversion of NFA with e moves to without e-moves — P.2-21
- Minimization of FSM — P.2-33
- Equivalence between two FSM's — P.2-47
- Mealy Machine and Moore Machine and its conversion — P.2-48

UNIT- III:

- Introduction to Regular languages and Regular expression — P.3-11
- Regular set — A-2
- Regular Expression — P.3-11
- Regular languages sample examples — P.3-11
- Regular expression examples — 3-12
- Equivalence and Inter Conversion between Regular Expression and FA — P.3-18
- Conversion from RE to RG — P.3-39 to 3-46

UNIT- IV:

- Introduction to Context Free Grammar (CFG) and Derivation Trees — P.3-60, 3-65
- Left Derivation Tree and Right Derivation Tree — P.3-69
- CFG minimization (Reduction of CFG) — P.3-73
- Chomsky normal form(CNF) — P.3-80
- Greibach normal form(GNF) — P.3-85

UNIT- V:

- Introduction to Push Down Automaton (PDA) P.4-4
- Definition and working principle P.4-4
- Implementation of Push Down Automaton. P.4-8
- Introduction to Turing machines P.5-7
- Definition P.5-7
- Model of TM, Types of TM P.5-8, 21
- Designing of TM. P.5-10
- Decidability P.6-13
- Undecidability of problems P.6-14
- Properties of recursive and recursively enumerable languages P.6-7
- Post correspondence problem P.6-15
- Ackermann function. P.6-38

SYLLABUS

(B.Tech. THIRD SEMESTER CT)

UNIT-I **See Unit-I and II of this book**

Strings, Alphabet, Language operations, Finite state machine definitions, Finite automation model, Acceptance of strings and language, Non-deterministic finite automation, Deterministic finite automation, Equivalence between NFA and DFA, Conversion of NFA into DFA, Minimization of FSM, Equivalence between two FSM's.

UNIT-II **See Unit-III of this book and appendix**

Regular sets, Regular expressions, Identity rules, Manipulation rules, Manipulation of regular expressions, Equivalence between RE and FA, Inter conversion, Pumping lemma, Closure properties of regular sets (proofs not required), Chomsky hierarchy of languages, Regular grammars, Right linear and left linear grammars, Equivalence between regular and FA, Inter conversion between RE and RG.

UNIT-III **See Unit-III, IV of this book and appendix**

Context free grammar, Derivation trees, Chomsky normal form, Greibach normal form, Push down automata, Definition, Model acceptance of CFL, Closure properties of CFL (Proofs omitted), Pumping Lemma of CFL, Introduction of DCFL and DPDA.

UNIT-IV **See Unit-V, VI of this book and appendix**

Turing Machine: Definition, Model of TM, Design of TM, Universal Turing Machine, Computable function, Recursive enumerable language, Types of TM's (proofs not required), Linear bounded automata and Context sensitive language, Counter machine.

UNIT-V **See Unit-V and VI of this book**

Decidability and Undecidability of problems, Properties of recursive and recursively enumerable languages, Halting problems, Post correspondence problem, Ackermann function and Church's hypothesis. Bounded Minimalization, Unbounded Minimalization.

CONTENTS

(B.Tech. THIRD SEMESTER CT)

UNIT- I:

- Strings P.1-4
- Alphabet P.1-4
- Language operations P.1-3
- Finite state machine definitions P.2-8
- Finite automation model P.2-9
- Acceptance of strings and language P.2-12
- Non deterministic finite automation P.2-11
- Deterministic finite automation P.2-11
- Equivalence between NFA and DFA P.2-29
- Conversion of NFA into DFA P.2-13
- Minimization of FSM P.2-33
- Equivalence between two FSM's P.2-47

UNIT-II:

- Regular sets A-2
- Regular expressions P.3-11
- Identity rules P.3-36
- Manipulation rules P.3-12, 3-14
- Manipulation of regular expressions P.3-16 to 3-35
- Equivalence between RE and FA, Inter conversion, P.3-16
- Pumping lemma P.3-52
- Closure properties of regular sets (proofs not required) P.3-59
- Chomsky hierarchy of languages P.1-11
- Regular grammars, Right linear and left linear grammars P.3-38 to 3-45
- Equivalence between regular and FA, P.3-16
- Inter conversion between RE and RG P.3-39 to 3-46

UNIT-III:

- Context free grammar P.3-60
- Derivation trees P.3-65
- Chomsky normal form P.3-80

- Greibach normal form — P.3-85
- Push down automata — P.4-4
- Definition, Model acceptance of CFL — P.3-62 to 3-65
- Closure properties of CFL (Proofs omitted) — P.4-26
- Pumping Lemma of CFL — P.PS-192
- Introduction of DCFL and DPDA — A-2

UNIT-IV:

- Turing Machine — P.5-7
- Definition — P.5-7
- Model of TM — P.5-8
- Design of TM — P.5-10
- Universal Turing Machine — P.5-27
- Computable function — P.5-36
- Recursive enumerable language — P.6-7
- Types of TM's (proofs not required) — P.5-21
- Linear bounded automata and Context sensitive language — P.5-34
- Counter machine. — A-6

UNIT-V:

- Decidability and Undecidability of problems — P.6-13, 14
- Properties of recursive and recursively enumerable languages — P.6-7
- Halting problems — P.6-19
- Post correspondence problem — P.6-15
- Ackermann function, and Church's hypothesjs — P.6-38, 5-30
- Bounded Minimalization — P.6-25
- Unbounded Minimalization — P.6-25

SYLLABUS

[B.Tech. FOURTH SEMESTER CSE]

UNIT - I

See Unit-I, II and III of this book

FINITE AUTOMATA (FA)

Basic Terminology and Definitions, Chomsky hierarchy, Deterministic Finite Automata , Language of a DFA. Non-deterministic Finite Automata, Equivalence of Deterministic and Non-deterministic Finite Automata, Applications of Finite Automata, Finite Automata with Epsilon Transitions, Eliminating Epsilon transitions, Minimization of Deterministic Finite Automata, Finite automata with output (Moore and Mealy machines) and Inter conversion.

UNIT - II

See Unit-III of this book

REGULAR GRAMMARS (RG)

Definition, Regular grammars and FA, Conversion. Proving languages to be non-regular, Pumping lemma, applications, Closure properties of regular languages.

Regular Expressions (RE):

Introduction, Identities of Regular Expressions, Finite Automata and Regular Expressions, Converting from DFA's to Regular Expressions, Converting Regular Expressions to Automata, applications of Regular Expressions.

UNIT - III

See Unit-III of this book

CONTEXT FREE GRAMMAR (CFG)

Definition, Parse Tress, Derivation Trees, Rightmost and Leftmost derivations of Strings and Conversions. Ambiguity in CFGs, Minimization of CFGs, Normal forms for CFG, Pumping Lemma for CFLs.

UNIT - IV

See Unit-IV of this book

PUSH DOWN AUTOMATA (PDA)

Definition, Model, Non-determinism, Acceptance by two methods and their equivalence, Conversion of PDA to CFG, CFG to PDAs, Closure and decision properties of CFLs.

UNIT - V

See Unit-V and VI of this book

TURING MACHINES (TM)

Formal definition and behavior, Languages of a TM, TM as acceptor, TM as transducers, Variations of TM, Linear Bounded Automata, TM as computer of function. Properties of recursive and recursively enumerable languages, Recursively enumerable set, Undecidability, Decidability and solvability, Post correspondence Problem, Primitive recursive functions, Ackerman function.

CONTENTS

[B.Tech. FOURTH SEMESTER CSE]

UNIT - I

FINITE AUTOMATA (FA)

- Basic Terminology and Definitions P.1-3
- Chomsky hierarchy P.1-11
- Deterministic Finite Automata, language of a DFA P.2-11
- Non-deterministic Finite Automata P.2-11
- Equivalence of Deterministic and Non-deterministic Finite Automata P.2-29
- Applications of Finite Automata P.2-10
- Finite Automata with Epsilon Transitions P.3-75
- Eliminating Epsilon Transitions P.3-75
- Minimization of Deterministic Finite Automata P.2-33
- Finite automata with output (Moore and Mealy machines) and Inter conversion P.2-50, 48

UNIT - II

REGULAR GRAMMARS (RG)

- Definition, Regular grammars and FA Conversion P.3-36
- Proving languages to be non-regular P.3-53
- Pumping lemma, Applications P.3-52, 53
- Closure properties of regular languages P.3-59

Regular Expressions (RE): P.3-11

- Introduction P.3-11
- Identities of Regular Expressions P.3-36
- Finite Automata and Regular Expressions P.3-16
- Converting from DFA's to Regular Expression P.3-46
- Converting Regular Expressions to Automata P.3-15
- Applications of Regular Expressions P.3-15

UNIT - III

CONTEXT FREE GRAMMAR (CFG)

- Definition P.3-60
- Parse Tress, Derivation Trees P.3-65
- Rightmost and Leftmost derivations of Strings and Conversions P.3-70
- Ambiguity in CFGs P.3-66
- Minimization of CFGs P.3-73

- Normal forms for CFG P.3-80
- Pumping Lemma for CFLs P.PS-192

UNIT - IV

PUSH DOWN AUTOMATA (PDA)

- Definition P.4-4
- Model P.4-5
- Non-determinism P.4-6
- Acceptance by two methods and their equivalence P.4-8
- Conversion of PDA to CFG P.4-18
- CFG to PDAs P.4-24
- Closure and decision properties of CFLs P.4-26

UNIT - V

TURING MACHINES (TM)

- Formal definition and behavior P.5-7
- Languages of a TM P.5-7
- TM as acceptor P.5-14
- TM as transducers P.5-20
- Variations of TM P.5-21
- Linear Bounded Automata P.5-34
- TM as computer of function P.5-36
- Properties of recursive and recursively enumerable languages P.6-7
- Recursively enumerable set P.6-7
- Undecidability P.6-14
- Decidability and solvability P.6-13
- Post correspondence Problem P.6-15
- Primitive recursive functions P.6-28
- Ackermann function P.6-38

SYLLABUS

[B.Tech. FOURTH SEMESTER AI]

UNIT – I

See Unit-I and II of this book

Strings, Alphabet, Language operations, Finite state machine definitions, Finite automation model, Acceptance of strings and language, Non-deterministic finite automation, Deterministic finite automation, Equivalence between NFA and DFA, Conversion of NFA into DFA, Moore and Mealy machines

UNIT – II

See Unit-I, III of this book and appendix

Regular sets, Regular expressions, Identity rules, Manipulation rules, Manipulation of regular expressions, Pumping lemma, Closure properties of regular sets(proofs not required), Chomsky hierarchy of languages, Regular grammars, Right linear and left linear grammars, Equivalence between regular linear programming and FA.

UNIT - III

See Unit-III, IV of this book and appendix

Context free grammar, Derivation trees, Chomsky normal form, Greibach normal form, Push down automata, Definition, Model acceptance of CFL, Equivalence of CFL and PDA, Inter conversion, Closure properties of CFL(Proofs omitted),Pumping Lemma of CFL .

UNIT - IV

See Unit-V, VI of this book

Turing Machine: Definition, Model of TM, Design of TM, Universal Turing Machine, Computable function, Recursive enumerable language, Types of TM"s (proofs not required), Linear bounded automata and Context sensitive language.

UNIT - V

See Unit-VI of this book

Decidability and Undecidability of problems, Properties of recursive and recursively enumerable languages, Halting problems, Post correspondence problem, Ackermann function, and Churchs hypothesis, Recursive and Primitive Functions, Bounded Minimization, Unbounded Minimization.

CONTENTS

[B.Tech. FOURTH SEMESTER AI]

UNIT - I

- Strings P.1-4
- Alphabet P.1-4
- Language operations P.1-3
- Finite state machine definitions P.2-8
- Finite automation model P.2-9
- Acceptance of strings and language P.2-12
- Non deterministic finite automation P.2-11
- Deterministic finite automation P.2-11
- Equivalence between NFA and DFA P.2-29
- Conversion of NFA into DFA P.2-13
- Moore and Mealy machines P.2-48

UNIT - II

- Regular sets A-2
- Regular expressions P.3-11
- Identity rules P.3-36
- Manipulation rules P.3-12, 13, 14
- Manipulation of regular expressions P.3-16 to 3-35
- Pumping lemma P.3-52
- Closure properties of regular sets(proofs not required) P.3-59
- Chomsky hierarchy of languages P.1-11
- Regular grammars P.3-36
- Right linear and left linear grammars P.3-38
- Equivalence between regular linear programming and FA P.3-37

UNIT - III

- Context free grammar P.3-60
- Derivation trees P.3-65
- Chomsky normal form P.3-80
- Greibach normal form P.3-85
- Push down automata, Definition P.4-4

- Model acceptance of CFL — P.3-62 to 65
- Equivalence of CFL and PDA — A-6
- Inter conversion — A-12
- Closure properties of CFL(Proofs omitted) — P.4-26
- Pumping Lemma of CFL — P.PS-192

UNIT - IV

- Turing Machine: Definition — P.5-7
- Model of TM — P.5-8
- Design of TM — P.5-10
- Universal Turing Machine — P.5-27
- Computable function — P.5-36
- Recursive enumerable language — P.6-7
- Types of TMs (proofs not required) — P.5-21
- Linear bounded automata and Context sensitive language — P.5-34

UNIT - V

- Decidability and Undecidability of problems — P.6-13, 14
- Properties of recursive and recursively enumerable languages — P.6-7
- Halting problems — P.6-19
- Post correspondence problem — P.6-15
- Ackerman function — P.6-38
- Church hypothesis — P.5-30
- Recursive — P.6-21
- Primitive Functions — P.6-28
- Bounded Minimization — P.6-25
- Unbounded Minimization — P.6-25

SOLVED UNIVERSITY QUESTION PAPERS

WINTER - 13 to SUMMER - 14 (CS)	PS-1 to PS-18
WINTER - 13 to SUMMER - 14 (CT)	PS-19 to PS-36
WINTER - 14 to SUMMER - 15 (CS)	PS-37 to PS-56
WINTER - 14 to SUMMER - 15 (CT)	PS-57 to PS-72
WINTER - 15 to SUMMER - 16 (CS)	PS-73 to PS-90
WINTER - 15 to SUMMER - 16 (CT)	PS-91 to PS-108
WINTER - 16 to SUMMER - 17 (CS)	PS-109 to PS-128
WINTER - 16 to SUMMER - 17 (CT)	PS-129 to PS-150
WINTER - 17 to SUMMER - 18 (CS)	PS-151 to PS-166
WINTER - 17 to SUMMER - 18 (CT)	PS-167 to PS-178
WINTER - 18 to SUMMER - 19 (CS)	PS-179 to PS-194
WINTER - 18 to SUMMER - 19 (CT)	PS-195 to PS-208
WINTER - 19 (CS)	PS-209 to PS-214
WINTER - 19 (CT)	PS-215 to PS-220
APPENDIX	A-1 to A-14

Note:- Due to Covid - 19 Summer - 2020 to Winter - 2021 exams were conducted online by the respective Colleges.

UNIT - I

UNIT - I
MATHEMATICAL PRELIMINARIES

SYLLABUS

Mathematical preliminaries : Sets, Operations, Relations, Strings, Closure of relation, Countability and diagonalization, Induction and proof methods: Pigeon-hole principle, Concept of language, Formal grammars, Chomsky hierarchy.

UNIVERSITY PAPER SOLUTIONS
SINCE SUMMER - 2006

Note to the students :
- The questions given below are from previous Nagpur University examination papers.
- Though these questions are from old university papers, they have been included in the new syllabus.
- Questions which are out of new syllabus are not included here.

SUMMER - 06 (CT)

Q.1. Define the following terms with the help of examples: **6M**
(i) Transitive closure
(ii) Reflexive Transitive closure
(iii) Equivalence Relation.

Ans. P.1-5, Q.4

WINTER - 06 (CT)

No question asked in University Exam.

SUMMER - 07 (CT)

Q.1. What do you mean by the following: **5M**
(a) Prefix of a string.
(b) Suffix of a string.
(c) Proper prefixes and suffixes of a string.
(d) Star closure of a language.
(e) Positive closure of a language.

Ans. P.1-4, 11, Q.2, Q.20

WINTER - 07 (CT)

No question asked in University Exam.

SUMMER - 08 (CT)

Q.1. Write short note on Chomsky Hierarchy of languages. **5M**

Ans. P.1-11, Q.22

WINTER - 08 (CT)

No question asked in University Exam.

SUMMER - 09 (CS)

Q.1. Write short notes on pigeon hole principle. **3M**

Ans. P.1-10, Q.16

SUMMER - 09 (CT)

Q.1. Explain the terms Symbol, Alphabet, String, Prefix and Suffix of a string, Proper prefix and Proper Suffix of a string, Power set. **8M**

Ans. P.1-4, Q.2

Q.2. Discuss briefly the Chomsky Hierarchy of languages with suitable example of each. **6M**

Ans. P.1-11, Q.22

WINTER - 09 (CS)

No question asked in University Exam.

WINTER – 09 (CT)

Q.1. Write a short note on Chomsky Hierarchy of lang -uages. **4M**

Ans. P.1-11, Q.22

SUMMER - 10 (CS)

Q.1. Prove that:

$P(n) : 1.1! + 2.2! + 3.3! + ... + n.n!$

$= (n + 1)! - 1, n > 1$ **6M**

Ans. P.1-8, Q.12

Q.2. Explain Generalized Pigeonhole principle. **3M**

Ans. P.1-10, Q.17

Q.3. Write a short note on Chomsky Hierarchy of languages. Give an example of grammar of type 2 but which is not of type 3. **6M**

Ans. P.1-11, Q.22

SUMMER - 10 (CT)

Q.1. Write a short note on Chomsky Hierarchy of languages. **3M**

Ans. P.1-11, Q.22

WINTER - 10 (CS)

No question asked in University Exam.

WINTER - 10 (CT)

No question asked in University Exam.

SUMMER - 11 (CS)

Q.1. Explain the Chomsky Hierarchy of languages. **4M**

Ans. P.1-11, Q.22

SUMMER - 11 (CT)

Q.1. Write a short note on: Chomsky hierarchy. **4M**

Ans. P.1-11, Q.22

WINTER - 11 (CS)

Q.1. Prove that by principle of induction

$1.2.3 + 2.3.4 + ... + n(n+1)(n+2)$

$= \frac{n.(n+1)(n+2)(n+3)}{4}$. **5M**

Ans. P.1-9, Q.15

Q.2. Write short notes on Chomsky Hierarchy. **4M**

Ans. P.1-11, Q.22

WINTER – 11 (CT)

No question asked in University Exam.

SUMMER - 12 (CS)

Q.1. Define: Relation, Homomorphism, Countability. **6M**

Ans. P.1-4, 5, 6, Q.3, Q.5, Q.6

Q.2. State pigeonhole principle. **1M**

Ans. P.1-10, Q.16

Q.3. Prove that the set of all real numbers is not countable using diagonalization. **6M**

Ans. P.1-6, Q.7

SUMMER - 12 (CT)

Q.1. Explain the Chomsky hierarchy of phrase structure grammar with suitable example. **6M**

Ans. P.1-14, Q.23

WINTER - 12 (CS)

Q.1. With the help of Mathematical Induction prove that :

$1^2 + 2^2 + 3^2 + ... + n^2 = \frac{n(n+1)(2n+1)}{6}$. **4M**

Ans. P.1-8, Q.11

Q.2. What is diagonalization? Explain with an example. **5M**

Ans. P.1-5, Q.5

WINTER - 12 (CT)

Q.1. Write a short notes on: Chomsky hierarchy of languages. **5M**

Ans. P.1-11, Q.22

SUMMER - 13 (CS)

Q.1. Define: Language, Closure and Homomorphism. **6M**

Ans. P.1-3, 6, 10, Q.1, Q.6, Q.18

Q.2. Comment on countability of set of all real number. Use diagonalization. **5M**

Ans. P.1-6, Q.7

SUMMER - 13 (CT)

Q.1. Define:
Diagonalization, countability, decidability, relation and language. **5M**

Ans. P.1-4, 5, 10, Q.3, Q.5, Q.18

Q.2. Discuss different proof methods with example. **4M**

Ans. P.1-7, Q.9

Q.3. Explain Chomsky Hierarchy of formal languages. **4M**

Ans. P.1-11, Q.22.

SOLVED QUESTION BANK

[Sequence given as per syllabus]

INTRODUCTION

In this chapter we will take quick revision of the graph and set theory concept and introduce with the basic concept which are required for the remaining topics as the base knowledge.

SETS, OPERATIONS, STRINGS

Q.1. Define set and explain operation on sets.
OR Define closure. ***CS: S-13(2M)***

Ans. Set:

- A collection of objects without repetition is known as set. These objects or entities in the set are called as members of the set.
- If every member of set A is a member of set B. then we write $A \subseteq B$, i.e. A is contained in B.
- Set A and set B are equal if they have the same members
  i.e. A = B iff (if and only if)
  $A \subseteq B$ and $B \subseteq A$.
- If $A \subseteq B$ and $A \neq B$ i.e. if every member of A is in B and there is at least one member of B which is not there in A, then we write, $A \subseteq B$ or $A \subset B$. A is called as proper subset of B.
- **Operation on Sets :**
  Usual operations on sets are :

(1) Union : If A and B are two sets then union of A and B denoted as
$A \cup B = \{x/x \text{ is in A or x is in B }\}$

(2) Intersection : If A and B are two sets then the intersection of A and B, denoted as
$A \cap B = \{x/x \text{ is in A and x is in B}\}$

(3) Set difference : If A and B are the two sets then the differences of A and B, denoted as
$A - B = \{x/x \text{ is in A but not in B}\}$

(4) Cartesian product : If A and B are the two sets then the cartesian product of A and B, denoted as
$A \times B = \{ (a, b) / a \text{ is in A and b is in B}\}$

(5) **Concatenation :** If A and B are two sets then the concatenation of A and B, denoted as
A.B = { ab/a is in A and b is in B}
For example: If A = {0, 1} and B = {1, 2} then A.B = {01, 02, 11, 12}.

(6) **Closure :** If A is a set then closure of A, denoted as $A^* = A^0 \cup A^1 \cup A^2 \cup ..... \cup A^\infty$
where, A^i is i^{th} power of set A, defined as A^i = A. A. A ...i times.

(i) $A^0 = \{\epsilon\}$, i.e., set of all possible combinations of members of A of length 0.

(ii) A^1 = A. A, i.e., set of all possible combination of members of A of length 1.

Therefore, A * is set of all possible combinations of the members of A.

Example: If $\Sigma = \{0, 1\}$ then Σ^* will be set of all possible combinations of 0's and 1's, which is one of the languages defined over Σ.

Q.2. Explain the terms Symbol, Alphabet, String, Prefix and Suffix of a string, Proper Prefix and Proper Suffix of a string power set.

CT : S-09 (8M)

OR What do you mean by:
(i) Prefix of string
(ii) Suffix of string
(iii) Proper prefixes and suffixes of string.

CT : S-07 (3M)

Ans.

(i) Symbol :

- Symbol is an abstract or a user defined entity. It cannot be formally defined (like 'Point' in geometry).

Example: Letter, digits or any symbol that we want to consider as a part of the language we are going to define.

(ii) Alphabet (Σ) :

- A finite set of symbols is known as alphabet.

Example : D = {0, 1, 2, 3, ... , 9}
Y = { +, –, *, /, %, ++, - - }
Z = { a, b, D, $, @.}

- It must noted that Z is containing heterogeneous (not of the same type) entities, though it is an alphabet.

(iii) String :

- A string over an alphabet Σ is defined as any finite sequence of symbols from Σ.
- Word length = number of symbols in the word
  = | × |
- Conventionally strings are denoted by small case letters x, y, z, u, v, or w.

(iv) Prefix :

- Any number of leading symbols of the string is known as prefix of a string.

Example: Let, x = abc

- Prefixes of x are, ϵ, a, ab, abc.

where, ϵ (epsilon) is an empty string with length zero i.e. $|\epsilon| = 0$.

(v) Proper prefix:

- The prefix other than the string itself is called as proper prefix.
- In above example, ϵ, a, ab these three prefixes are proper prefixes of abc.

(vi) Suffix:

- Any number of trailing symbols of the string is known as suffix of a string.
- For above example, suffixes of x are: ϵ, c, bc, abc.

(vii) Proper suffix:

The suffix other than the string itself is called as proper suffix.

- In above example, ϵ, c, bc are proper suffixes of abc.

(viii) Power set :

- If A is the set then the power set of A, denoted as 2^A = {P/P is a subset of A} i.e. set of all possible subsets of A.

Example: A = {0, 1}
2^A = [ϕ, {0}, {1}, {0, 1}] where, ϕ = { } null set.

RELATION

Q.3. Define Relation. **CS: S-12(2M), CT: S-13(1M)**

OR Explain term Relations and properties of relations.

Ans. Relations:

- A set of ordered pairs is known as relation. The

first component of the pair is from the set called domain and second component is from the set called range.

- In a relation, if domain and range are the same set 's', then we say the relation is on S.
- If R is a relation and (a, b) is a pair in R, then we write aRb.

Properties of relations :

If R is a relation on set S, it is said to be

(1) Reflexive, if aRa exists $\forall$ (for all) $a \subseteq S$.

(2) Transitive, if aRb and bRc imply aRc.

(3) Symmetric, if aRb imply bRa.

CLOSURE OF RELATIONS

Q.4. Explain closure properties of Relation.

OR Define the following term with the help of example: ***CT: S-06(6M)***

(1) Transitive closure

(2) Reflexive closure

(3) Equivalence relation.

Ans. Closure properties of Relations:

If P is a set of properties of relation R, then the P-closure of relation R is the smallest relation R' that includes all the pairs of R and possesses the properties in P.

(1) Transitive closure:

Transitive closure of R (R^+) is defined as:

(i) If $(a,b) \subseteq R$ then (a, b) is in R^+.

(ii) If $(a,b) \subseteq R^+$ and $(b,c) \subseteq R$ then (a, c) is in R^+.

Example: Let, S = {1, 2, 3} and

$$R = \{(1, 2), (2, 2), (2, 3)\}$$

Then, $R^+ = \{(1, 2) (2, 2) (2, 3) (1, 3)\}$

(2) Reflexive closure:

Reflexive and Transitive closure of R(R^*) is defined with the help of above definition as

$$R^* = R^+ \cup \{(a, a)/a \subseteq S\}$$

Example: For the above set S and relation R

$R^* = \{(1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)\}$

(3) Equivalence Relation:

If a relation is reflexive, transitive as well as symmetric it is said to be Equivalence Relation.

Example: Relation '=' on set of integers.

(i) it is reflexive because a = a.

(ii) it is transitive because a = b and b = c.

(iii) it is symmetric because a = b imply b = a.

Therefore, it is an equivalence relation.

COUNTABILITY AND DIAGONALIZATON

Q.5. Define:

Diagonalization, countability, decidability.

CS: S-12(2M), CT: S-13(3M)

OR Explain diagonalization. Explain with an example. ***CS: W-12(5M)***

Ans.

(1) Diagonalization:

- If R is a binary relation on some set A, then R can be represented by a table, with rows and columns labled with elements of A, and 1 in a box with row index a and column index b indicates that (a, b) is in R, whereas 0 in a box with row index a and column index b indicates that (a, b) is not in R, such a representation is shown below:

R	a	b	c	d
a	1	0	1	0
b	0	0	1	1
c	0	1	1	0
d	1	1	1	0

- If we define diagonal set D of R as follows:
  D = {a/a is in A, and (a, a) = 0}
- And for each a in A, if we define set R_a as follows:

  $R_{a'} = \{b / b \text{ is in A, and } (a, b) = 1\}$

  Then D is distinct from each R_a.

(2) Countability:

- Infinite sets that can be placed in one-to-one correspondence with the set of natural numbers (N) are said to be countably infinite or just countable.
- Finite sets are always countable.
- Some infinite sets are uncountable, e.g. set of real numbers.

- A set is said to be countable if it is finite or countably infinite.

(3) **Decidability:**

- In computability theory, a set of natural number is called recursive, computable or decidable.
- If there is an algorithm which terminates after a finite amount of time and correctly decides whether or not a given number belongs to the set is called Decidability.
- A set which is not computable is called non-computable or undecidable.

Q.6. Define Homomorphism. **CS: S-12,13(2M)**

Ans. Homomorphism:

- A homomorphism is a structure preserving map between two algebric structures (such as groups, rings).
- Let, A and A′ be two relational structures of the same type Δ. A mapping $f : A \to A'$ is a homomorphism from A to A′, if for every $i \in I$ and for every $u_1, u_2, \ldots u_{\delta i} \in A$, the following implication holds :

  $(u_1, u_2, \ldots, u_{\delta_i}) \in R_i(R) \Rightarrow (f(u_1), f(u_2) \ldots$

  $f(u_{\delta_i})) \in R_i(A')$.
- If there exists a homomorphism from A to A′ we say that A is homomorphism to A′ and write $A \to A'$

  otherwise we write $A \not\to A'$
- The fact that f is a homomorphism A to A′, is denoted by $f : A \to A'$

Q.7. Comment on Countability of set of all real number by using diagonalization. **CS: S-12(6M), S-13(5M)**

Ans. Countability using diagonalization :

- If we are programming in a language having the following restrictions:

(1) There are only finitely many characters in the language.

(2) Every program is finite.

- Then the set of all programs is countable, as it is a subset of all the finite strings in the language which itself is countable.
- If "printing out a real number" has an infinite decimal expansion (e.g. an irrational number) then printing it never halts.
- If this is not a legal behavior for our program then certainly we cannot write a program which prints every real number.
- If we are allowed to print an infinite length output and our program is finite then we have to calculate the number somehow, but there is only a countable number of numbers which we can compute their values. So yet again, we cannot print all the real numbers.
- Let R denote the set of all real numbers and suppose that R is countable. Then by definition it is either finite or countably infinite. Clearly, it is not finite, therefore it must be countably infinite.
- By definition, since it is countably infinite it has the same cardinality as N (the natural numbers) and by definition, there is a bijection from N to R.
- $f : N \Rightarrow R$

  $0 \Rightarrow r_0, 1 \Rightarrow r_1, 2 \Rightarrow r_2, \ldots$

  *f is "onto" so every real number is in this list.

  Consider the following table:

	0	1	2	3	⋯
r_0	d_0^0	d_1^0	d_2^0	d_3^0	
r_1	d_0^1	d_1^1	d_2^1	d_3^1	
r_2	d_0^2	d_1^2	d_2^2	d_3^2	
⋮					

  where, $r_i = x_i . d_0^i d_2^i \ldots . d_m^i$ .... (padded with zeros to the right)
- The table is a 2 dimensional vector of digits.
- Consider/define the real number:

  $y = 0 . y_0 y_1 y_2 \ldots .$ (infinite)

  where, $y_i = (d_i^i + 1) \bmod 10$ for all i>=0 (*)

Observations:

(i) y is a real number

(ii) Since $r_0, r_1, r_2, \ldots$ is a list of all real numbers, it follows that y must be in this list, i.e. $y = r_j$, for some j>=0.

(iii) This means that $y = r_j = 0 . d_0^j d_1^j d_2^j \ldots d_{j-1}^j d_j^j d_{j+1}^j \ldots$ (from the table).

(iv) This also means that $y_i = d_i^j$ for all i>=0 and in particular, that: $y_j = d_j^j$

But by * :

$$y_j = (d_j^j + 1) \bmod 10$$

a contradiction. Therefore, no such one-to-one and onto function exists, and therefore the real numbers are uncountable.

- So, real numbers with 0 as the integer part is uncountably infinite. Similarly, reals with 1 as the integer part is uncountably and so on.

INDUCTION AND PROOF METHODS

Q.8. Explain Principle of Mathematical Induction.

Ans.

- Induction is used to prove difficulty of the steps of mathematical induction.
- For all natural number n, property p is true.

Steps of mathematical induction:

(i) In the basic step we show that $0 \in A$ i.e. P is true for 0 or 1.

(ii) Then we make an assumption that for some fixed but arbitrary $n \geq 0$, P is true for each natural number 0, 1, ... n. This is called Induction Hypothesis.

(iii) Then using Induction Hypothesis that P for all natural numbers upto and including n for some arbitrary n, we prove that P is true for n + 1. This is called induction step.

Thereby proving that P holds of every natural numbers.

Q.9. Discuss different proofs Method.

CT: S-13(4M)

Ans. Types of proofs:

Suppose we wish to prove an implication p ! q, there are some strategies we have available to try.

(1) **Trivial proof:** If we know q is true then p ! q is true regardless of the truth value of p.

(2) **Vacuous proof:** If p is a conjunction of other hypotheses and we know one or more of these hypotheses is false, then p is false and so p ! q is vacuously true regardless of the truth value of q.

(3) **Direct proof:** Assume p and then use the rules of inference, axioms, definitions and logical equivalences to prove q.

(4) **Indirect proof or proof by contradiction:**

Assume p and q and derive a contradiction r^ : r

(5) **Proof by Contrapositive:** (Special case of proof by Contradiction).

Give a direct proof of :q ! :p. Assume :q and then use the rules of inference, axioms, definitions and logical equivalences to prove :p. (Can be thought of as a proof by contradiction in which we assume p and :q and arrive at the contradiction p^ :p).

(6) **Proof by Cases:** If the hypothesis p can be separated into cases p1_p2_ _ pk, prove each of the propositions, p1!q, p2! q, ... , pk! q, separately. (We may use different methods of proof for different cases).

Q.10. Show with the help of mathematical induction: for any $n \geq 0$,

$$1 + 2 + ... + n = \frac{(n^2 + n)}{2}$$

Ans.

(i) **Basic step:**

for $n = 0$

L.H.S. = 0

$$\text{R.H.S.} = \frac{(0^2 + 0)}{2} = 0$$

$\therefore$ L.H.S. = R.H.S

(ii) **Induction Hypothesis:**

Assume that for some $n \geq 0$

$$1 + 2 + ... + n = \frac{(n^2 + n)}{2}$$

for every natural number upto and including n.

(iii) **Induction step :**

For n + 1, the L.H.S. of a given equation is:

$1 + 2 + ... + (n + 1)$

$= (1 + 2 + 3 + ... + n) + (n + 1)$

$$= \frac{(n^2+n)}{2} + (n+1)$$

$$= \frac{(n^2+n+2n+2)}{2}$$

$$= \frac{(n^2+3n+2)}{2}$$

$$= \frac{((n^2+2n+1)+(n+1))}{2}$$

$$= \frac{((n+1)^2+(n+1))}{2}$$

which we want to be the R.H.S. for n + 1.

Q.11. Prove the following by principle of mathematical induction.

$$1^2 + 2^2 + 3^2 + \ldots\ldots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

CS: W-12(4M)

Ans.

(i) Basic step:

For n = 0

L.H.S. = 0

$$\text{R.H.S.} = \frac{n(n+1)(2n+1)}{6}$$

R.H.S. = 0

∴ L.H.S. = R.H.S

(ii) Induction Hypothesis:

Assume that for some $n \geq 0$

$$1^2 + 2^2 + \ldots\ldots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

(iii) Induction step :

For n = n + 1,

$$\text{L.H.S.} = \sum_{k=1}^{n} K^2$$

$$= \sum_{k=1}^{n+1} K^2$$

$$= 1^2 + 2^2 + \ldots + n^2 + (n+1)^2$$

$$= \frac{n(n+1)(2n+1)}{6} + (n+1)^2$$

$$= \frac{n(n+1)(2n+1) + 6(n+1)^2}{6}$$

$$= \frac{(n+1)[2n^2+n+6n+6]}{6}$$

$$= \frac{(n+1)[2n^2+7n+6]}{6}$$

$$= \frac{(n+1)(n+2)(2n+3)}{6}$$

$$\text{R.H.S.} = \frac{n(n+1)(2n+1)}{6}$$

$$= \frac{(n+1)(n+1+1)[2(n+1)+1]}{6}$$

$$= \frac{(n+1)(n+2)(2n+2+1)}{6}$$

$$= \frac{(n+1)(n+2)(2n+3)}{6} = \text{R.H.S.}$$

∴ L.H.S. = R.H.S.

Q.12. Prove by Induction:

1.1 ! + 2.2 ! + ... + n.n ! = (n + 1) ! – 1

CS: S-10(6M)

Ans.

(i) Basic step:

For n = 0

L.H.S. = 0

R.H.S. = (n + 1)! – 1

= 0

∴ L.H.S. = R.H.S.

(ii) Induction Hypothesis:

Assume that for some $n \geq 0$

1.1 ! + 2.2 ! + ... + n.n ! = (n + 1) ! – 1

For every natural number upto and including n.

(iii) Induction step:

For n = n + 1

L.H.S. = 1.1 ! + 2.2 ! + ... + n.n !

= 1.1! + 2.2! + ... + n.n! + (n + 1) (n + 1)!

= (n + 1)! – 1 + (n + 1) (n + 1)!

= (n + 1)! [1 + (n + 1)] – 1

= (n + 1+ 1)! – 1

= R.H.S.

∴ L.H.S. = R.H.S.

Q.13. Prove that: $1 + 3 + 5 + \ldots + r = n^2$, for all $n > 0$, r = odd integer, $r = 2n - 1$, n = no of terms in the sum.

Ans.

(i) Basic step:

For n = 1

L.H.S. = 1

R.H.S. $= n^2 = 1^2 = 1$

$\therefore$ L.H.S. = R.H.S.

(ii) Induction Hypothesis:

We have, $1 + 3 + 5 + \ldots + r = n^2$

Since $r = 2n - 1$

L.H.S. $= 1 + 3 + 5 + \ldots + (2n - 1) = n^2$

for every natural number upto and including n.

(iii) Induction step:

For n = n + 1, we have to prove that

$1 + 3 + 5 + \ldots + r + r + 2 = (n + 1)^2$

L.H.S. $= (1 + 3 + 5 + \ldots + r + (r + 2))$

$= n^2 + r + 2$

$= n^2 + 2n - 1 + 2$

$= (n + 1)^2$

= R.H.S.

$\therefore$ L.H.S. = R.H.S.

Q.14. Prove by induction:

$$1 + x + x^2 + \ldots + x^n = \frac{1 - x^{n+1}}{1 - x}$$

Ans.

(i) Basic step:

For n = 1,

L.H.S. = 1 + x

$$\text{R.H.S.} = \frac{1 - x^{n+1}}{1 - x} = \frac{1 - x^2}{1 - x}$$

$$= \frac{(1)^2 - x^2}{1 - x} = \frac{(1 - x)(1 + x)}{(1 - x)}$$

$= (1 + x)$

$\therefore$ L.H.S. = R.H.S.

(ii) Induction Hypothesis:

We have, $1 + x + x^2 + \ldots + x^n = \frac{1 - x^{n+1}}{1 - x}$

for every natural number upto and including n.

(iii) Induction step:

For n = n + 1, we have to prove

$$1 + x + x^2 + \ldots + x^{n+1} = \frac{1 - x^{n+1+1}}{1 - x}$$

L.H.S. $= 1 + x + x^2 + \ldots + x^n + x^{n+1}$

$$= \frac{1 - x^{n+1}}{1 - x} + x^{n+1}$$

$$= \frac{(1 - x^{n+1}) + (1 - x)(x^{n+1})}{(1 - x)}$$

$$= \frac{(1 - x^{n+1}) + x^{n+1} - x^{n+1+1}}{(1 - x)}$$

$$= \frac{1 - x^{n+1+1}}{1 - x} = \text{R.H.S.}$$

$\therefore$ L.H.S. = R.H.S.

Q.15. Prove that by using principle of induction

$$1.2.3 + 2.3.4 + \ldots + n(n + 1)(n + 2) = \frac{n(n+1)(n+2)(n+3)}{4}.$$

CS:W-11(5M)

Ans. Let the given statement be P(n), i.e.

$$P(n): 1.2.3 + 2.3.4 + \ldots + n(n + 1)(n + 2) = \frac{n(n + 1)(n + 2)(n + 3)}{4}$$

(i) Basic step:

For n = 1, we have

L.H.S. = P(1) : 1.2.3 = 6

$$\text{R.H.S.} = \frac{1(1 + 1)(1 + 2)(1 + 3)}{4}$$

$$= \frac{1.2.3.4}{4} = 6$$

$\therefore$ L.H.S. = R.H.S.

(ii) Induction Hypothesis:

Let, P(k) be true for some positive integer k, i.e.,

$$1.2.3 + 2.3.4 + \ldots + k(k + 1)(k + 2) = \frac{k(k+1)(k+2)(k+3)}{4}$$

(iii) Induction step:

We shall now prove that P(k + 1) is true.

Consider

$1.2.3 + 2.3.4 + \ldots + k(k + 1)(k + 2)$

$+ (k + 1)(k + 2)(k + 3)$

$$= \{1.2.3 + 2.3.4 + \ldots + k(k+1)(k+2)\} + (k+1)(k+2)(k+3)$$

$$= \frac{k(k+1)(k+2)(k+3)}{4} + (k+1)(k+2)(k+3)$$

$$= (k+1)(k+2)(k+3)\left(\frac{k}{4}+1\right)$$

$$= \frac{(k+1)(k+2)(k+3)(k+4)}{4}$$

$$= \frac{(k+1)(k+1+1)(k+1+2)(k+1+3)}{4}$$

Thus, P (k + 1) is true whenever P(k) is true.

Hence, by the principle of mathematical induction, statement P(n) is true for all natural numbers i.e., n.

PIGEON-HOLE PRINCIPLE

Q.16. Write short note on pigeon hole principle. *CT: S-09(3M)*

OR State pigeon hole principle. *CS: S-12(1M)*

Ans. Pigeon hole principle:

- If A and B are two non empty finite sets and $|A| > |B|$, then there is no one to one function from A to B.
- If we attend to pair of the elements of A i.e.(pigeons) with element of B i.e (the pigeonholes), sooner or later we will have to put more than one pigeon in the pigeonhole.

For example:

- Suppose a postman distributes 51 letters (pigeon) in 50 mailboxes (pigeonhole), then it is evident that some mailbox will contain at least two letters.
- This is referred as a mathematical principle called the pigeonhole principle.
- If n object are distributed over m places and n>m, then some place receives atleast two objects.

Q.17. Explain Generalized Pigeonhole principle. *CS: S-10(3M)*

Ans. Generalised pigeonhole principle: If n pigeonhole is occupied by Kn + 1 pigeons, where K is a positive integers, at least 1 pigeonhole is occupied by atleast K + 1 pigeons.

Example:

There are many red socks, many white socks and many blue socks in a box. What is the least number of socks that one should grab from box to be sure of getting 3 pairs of one colour?

Assume colours n = 3 (pigeon hole)

Hence, K + 1 = 6 socks (pigeons)

$\therefore$ K = 5

Hence,

Kn + 1 = 5 × 3 + 1 = 16 socks (pigeons) should be grabbed.

CONCEPT OF LANGUAGE

Q.18.Define language. *CS : S-13(2M), CT: S-13(1M)*

Ans. Language:

- A Language is defined as a set of strings of symbols from some one alphabet ϕ (null set) and the set consisting of empty string i.e. $\{\in\}$ are also languages.
- Set of all strings over a fixed alphabet Σ is language denoted by Σ^*.

Example: (1) Let $\Sigma = \{a\}$ then,

$\Sigma^* = \{\in, a, aa, aaa, \ldots.\}$

(2) Let $\Sigma = \{0, 1\}$ then

$\Sigma^* = \{\in, 0, 1, 00, 01, 10, 11, 000, \ldots\}$

From the above language Σ^* if we exclude $\in$, the empty word, the remaining set is a language denoted as Σ^+.

$\Sigma^+ = \{0, 1, 00, 01, 10, 11, 000, \ldots.\}$

Q.19. Define context -sensitive languages.

Ans. Context-sensitive languages:

- A grammar G = (V, Σ, P, S) having the production in the following form

$$\alpha \rightarrow \beta$$

where, $\alpha, \beta \in (V \cup \Sigma)^*$

$|\beta| \geq |\alpha|$

α contains variable

if β is at least as long as α, then the resulting grammar is called context-sensitive grammar and its language as context sensitive language (CSG and CSL respectively).

- Almost every language is context sensitive.

Q.20. What do you mean by:

(i) Star closure of a language.

(ii) Positive closure of a language.

CT: S-07(2M)

Ans.

(i) Star-closure (Kleene*) :

Star closure is defined as

$L^* = L^0 \cup L^1 \cup L^2 \ldots\ldots$

Example:

$$\{a,\ bb\}^* = \begin{Bmatrix} \lambda, \\ a,\ bb, \\ aa,\ abb,\ bba,\ bbbb, \\ aaa,\ aabb,\ abba,\ abbbb,\ \ldots. \end{Bmatrix}$$

(ii) Positive closure :

Positive closure is defined as:

$L^+ = L^1 \cup L^2 \cup \ldots..$

$= L^* - \{\lambda\}$

Example:

$$\{a,\ bb\}^+ = \begin{Bmatrix} a,\ bb, \\ aa,\ abb,\ bba,\ bbbb, \\ aaa,\ aabb,\ abba,\ abbbb,\ \ldots.. \end{Bmatrix}$$

Q.21. What is formal grammar? Explain with example.

Ans. Formal Grammar:

- A formal grammar is used to specify a language that has following four components denoted by G = (V, T, P, S).

(1) A set of tokens, known as terminal symbols denoted by T.

(2) A set of nonterminals denoted by V.

(3) A set of productions where each production consists of a nonterminal, called the left side of the production, an arrow and a sequence of tokens and/or nonterminals, called the right side of the production denoted by P.

(4) A designation of one of the nonterminals as the start symbol denoted by S.

Example:

- Consider an expression consisting of digits and plus and minus signs, e.g. 2 – 1 + 5, 6 – 3 , and 7.
- Since a plus or minus sign must appear between two digits, we refer to such expressions as "lists of digits separated by plus or minus signs."
- The following grammar describes the syntax of these expressions. The productions are:

list → list + digit ..... (1)

list → list – digit ..... (2)

list → digit ..... (3)

digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 ..... (4)

- The right sides of the three productions with nonterminal list on the left side can equivalently be grouped as:

list → list + digit | list – digit | digit

- The tokens of the grammar are the symbols

+ – 0 1 2 3 4 5 6 7 8 9

- The nonterminals are the italicized names list and digit, with list being the starting nonterminal because its productions are given first.
- A grammar derives strings by beginning with the start symbol and repeatedly replacing a nonterminal by the right side of a production for that nonterminal.
- The token strings that can be derived from the start symbol form the language defined by the grammar.

CHOMSKY HIERARCHY

Q.22. Explain the Chomsky hierarchy of languages.

CS: S-11, CT: S-13(4M)

OR What are the different types of grammar specified in Chomsky hierarchy? Explain each with example.

OR Write short note on Chomsky hierarchy.

CT: S-08(5M), S-10,11(3M), CS: W-11(4M); CT:W-09(4M),W-12(5M)

OR Discuss briefly the Chomsky Hierarchy of languages with suitable example of each.

CT: S-09(6M)

OR Write short note on Chomsky Hierarchy of languages. Give an example of grammar of type 2 but which is not of type 3. CS: S-10(6M)

Ans. Chomsky Hierarchy:

- The four classes of languages regular, context-free, context-sensitive and recursively enumerable are often referred to as the Chomsky hierarchy.

- Chomsky himself, in his papers of 1956 and 1959, designated the four types as type 3, type 2, type 1 and type 0, from most restrictive to most general.
- Each level of hierarchy can be characterized by a class of grammars and by a certain type of abstract machine, or model of computation.
- This hierarchy is summarized in table (1).

Type	Languages (grammars)	Form of Productions In grammar	Accepting Device	Example
3	Regular	$A \to aB$, $A \to a$ ($A, B, \in V$, $a \in \Sigma$)	Finite Automation	$L = \{0^n 1^m \mid n \geq 1, m \geq 1\}$
2	Context-free	$A \to \alpha$ $A \in V$, $\alpha \in (V \cup \Sigma)^*$	Push down Automation	$L = \{WCW^R \mid w \in (a,b)^*\}$
1	Context-sensitive	$\alpha \to \beta$ $(\alpha, \beta) \in (V \cup \Sigma)^*$ $\lvert\beta\rvert \geq \lvert\alpha\rvert$, α contains a variable	Linear-bounded automation	$L = \{a^i b^n c^n \mid i \geq 0, n \geq 1\}$
0	Recursively enumerable (unrestricted or phrase-structure)	$\alpha \to \beta$ $(\alpha, \beta \in (V \cup \Sigma)^* \alpha$ contains a variable)	Turing Machine	$L = \{1^n 2^n 3^n \mid n \geq 1\}$

Table : The Chomsky Hierarchy

- The characterizations of these languages in terms of grammar make it obvious that for $1 \leq i \leq 3$, every language of type 1 is of type i-1 except that a CFL is context - sensitive only if it does not contain the null string. (For any CFL, the set of non null strings in the language is a CSL).
- In first two classes, we have also seen that inclusion is strict. There are CFLs that are not regular (even if the null string is omitted) and there are CSLs that are not context free.

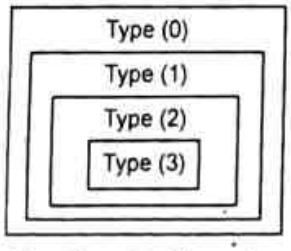


Fig. Chomsky Hierarchy

- Several other language families can also be assembled that can be fitted into this relationship. Including the families of deterministic context-free languages and recursive languages. The modified Chomsky Hierarchy is shown in following figure.

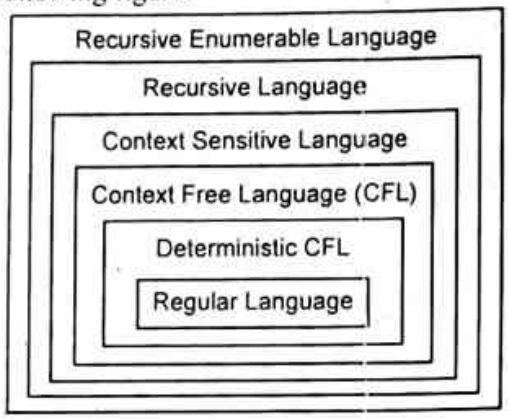


Fig. Modified Chomsky Hierarchy

(1) (Type - 0) : Unrestricted Grammars :

The unrestricted grammar is defined as

$G = (V_N, V_T, P, S)$

where,

V_N = a finite set of non-terminals

V_T = a finite set of terminals

S = starting non-terminal, $S \in V_N$ and

P is set of productions.

- In this grammar, every production rule is of the form: $\alpha \to \beta$, where α and β are arbitrary string of grammar symbols with $\alpha \neq \varepsilon$. i.e. they can be any strings of terminals and non-terminals.
- The languages specifiable using type (0) grammar are called as recursively enumerable languages.

Example:

Write a regular grammar generating

$\{a^i \mid i$ is a positive power of 2$\}$

Solution:

Let, $G = (V_N, V_T, P, S)$

where,

V_N = finite set of non-terminals

$= \{S, A, B, C, D, E\}$

V_T = finite set of terminals = $\{a, \in\}$

S = starting non-terminal, $S \in V_N = \{S\}$

and P is set of productions given by

$S \rightarrow ACaB$

$Ca \rightarrow aaC$

$CB \rightarrow DB$

$CB \rightarrow E$

$aD \rightarrow Da$

$AD \rightarrow AC$

$aE \rightarrow Ea$

$AE \rightarrow \epsilon$

- The non-terminals A and B act as left and right end markers for sentential forms; C is a marker that moves through the string of a's between A and B, doubling their number by productions ($Ca \rightarrow aaC$).
- When C hits the right end marker B, it becomes a D or E by production ($CB \rightarrow DB$), or ($CB \rightarrow E$). If a D is choosen, that D migrates left by production ($aD \rightarrow Da$) until the left end marker A is reached.
- When D becomes a C again by production ($AD \rightarrow AC$), and the process starts over.
- If an E is chosen, the right end marker is consumed. The E migrates left by production ($aE \rightarrow Ea$) and consumer the left end marker, leaving a stirng of 2^i a's for some $i > 0$.

(2) (Type - 1) : Context Sensitive Grammar:

The context sensitive grammar is defined as

$$G = (V_N, V_T, P, S)$$

where,

V_N = a finite set of non-terminals

V_T = a finite set of terminals

S = starting non-terminal, $S \in V_N$

and P is set of productions.

- In this grammar, every production rule is of the form: $\alpha \rightarrow \beta$, where α and β are arbitrary string of grammar symbols with $\alpha \neq \epsilon$ and $|\beta| \geq |\alpha|$ i.e., they can be any strings of terminals and non-terminals and the length of the string on the right side must be greater than or equal to the length of the string on the left side.
- As there is a restriction that the right hand side must be at least as long as that of the left hand side.
- The number of languages that it can specify will be less than the number of languages type(0) grammar can specify, because there may exist a language, which can be specified by using a grammar which must have at least one production rule with $|\alpha| > |\beta|$, then it can be specified using type (0) grammar, but cannot be specified using type (1) grammar.
- By default the requirement of type (1) grammar fulfills the requirement of type (0) grammar, thus every language that can be specified using type (1) grammar, is always possible to specify using type (0) grammar, but vice-versa is not true.

Example:

The language $L = \{a^n b^n c^n \mid n > 0\}$ is context sensitive. We can show this exhibiting a context sensitive grammar for the language L_1 grammar as

$S \rightarrow abc \mid aAbc$

$Ab \rightarrow bA$

$Ac \rightarrow Bbcc$

$bB \rightarrow Bb$

$aB \rightarrow aa \mid aaA$

(3) (Type-2) : Context Free Grammar:

- The context sensitive grammar is defined as

$$G = (V_N, V_T, P, S)$$

where,

V_N = a finite set of non-terminals

V_T = a finite set of terminals

S = starting non-terminal, $S \in V_N$

and P is set of productions.

- In this grammar, every production rule is of the form: $A \rightarrow \alpha$, where α, is arbitrary string of grammar symbols i.e., the left side of the production rule is a single non-terminal or variable and the right side can be any string of terminals and non-terminals.

Example:

Let the grammar G defined as,

$G = (\{S\}, \{a, b\}, P, S)$

where, P consists of following productions,

$S \rightarrow aSa \mid bSb \mid a \mid b$

(4) (Type - 3) : Regular Grammar:

- The context sensitive grammar is defined as

  $G = (V_N, V_T, P, S)$

  where,

  V_N = a finite set of non-terminals

  V_T = a finite set of terminals

  S = starting non-terminal, $S \in V_N$

  and P is set of productions.
- In this grammar, every production rule is of the form: $A \rightarrow aB$, or $A \rightarrow a$, where A and B are non-terminals and a is terminal, i.e., the left side of the production rule is a single non-terminal or variable and the right side can a terminal followed by a non-terminal or only a single terminal.

Example:

Consider grammar

$G = (\{S, B, C\}, \{a, b\}, P, S)$

where, P consists of

$S \rightarrow Ca \mid Bb$

$C \rightarrow Bb$

$B \rightarrow Ba|b$

Q.23. Explain the Chomsky hierarchy of phrase structure grammar with suitable example.

CT: S-12(6M)

Ans. Phrase-structure grammars:

A phrase-structure grammar is a collection of three things:

(1) A finite alphabet Σ of letters called terminals.

(2) A finite set of symbols called non-terminals that includes the start symbol S.

(3) A finite set of production of the form

String 1 $\rightarrow$ String 2

where string 1 can be any string of terminals and non-terminals that contains at least one non-terminals and where string 2 is any string of terminals and non-terminals what so ever.

- A derivation in a phrase-structure grammar is a series of working string beginning with the start symbol S, which by making substitution according to the productions, arrives at a string of all terminals, at which point generation must stop.
- The language generated by a phrase structure grammar is the set of all strings of terminals that can be derived starting at S.

Example:

The following is a phrase structure grammar over $\Sigma = (a, b)$ with non-terminals A an B:

$A \rightarrow BA/\in$ (P_1, say)

$B \rightarrow aB/a$ (P_2, say)

$aaaB \rightarrow ba$ (P_3, say)

Solution:

The first production P_1 says that we can start with A and derive number of symbols of the type B, for example

$A \Rightarrow BA$

$\Rightarrow BBA$

$\Rightarrow BBBA$

$\Rightarrow BBBBA$

$\Rightarrow BBBB$

The second production shows us that each B can be any string of a's (with at least one a):

$B \Rightarrow aB$

$\Rightarrow aaB$

$\Rightarrow aaaB$

$\Rightarrow aaaaB$

$\Rightarrow aaaaa$

The third production (P_3) says that any time we find three a's and B, we can replace these four symbols with the two-terminal string ba.

The following is a summary of one possible derivation in this grammar:

$A \Rightarrow BBBBBB$

$\Rightarrow aaaaBBBBB$

$\Rightarrow aabaBBBB$

$\Rightarrow aabaaaBB$

$\Rightarrow aabbaBB$

$\Rightarrow aabbaaaBB$

$\Rightarrow aabbba$

- All CFGs are phrase-structure grammars in which we restrict ourselves as to what we put on the left side of productions.
- So, all CFLs can be generated by phrase-structure grammars.

POINTS TO REMEMBER:

(1) *Set is a collection of objects without repetition.*

(2) *Operation on sets are union, intersection, set difference, Cartesian product and concatenation.*

(3) *A string is any finite sequence of symbols.*

(4) *There are three closure relations:*

(i) Transitive closure

(ii) Reflexive closure

(iii) Equivalence closure.

(5) *A language is set of strings of symbols from some alphabet.*

(6) *A formal grammar is used to specify a language.*

(7) *The four classes of languages-regular, context-free, context-sensitive and recursively enumerable are referred as Chomsky hierarchy.*

(8) *Pigeonhole principle states:*

"If A and B are two nonempty final sets, then there exist no one-to-one function from A to B, if $|A| > |B|$ *".*

Notes :-

UNIT - II
FINITE AUTOMATON

SYLLABUS

Finite Automaton, Regular languages, Deterministic and non deterministic finite automata, ϵ-closures, Minimization of automata, Equivalence, Moore and Mealy machine.

UNIVERSITY PAPER SOLUTIONS
SINCE SUMMER - 2006

Note to the students :

- The questions given below are from previous Nagpur University examination papers.
- Though these questions are from old university papers, they have been included in the new syllabus.
- Questions which are out of new syllabus are not included here.

SUMMER - 06 (CT)

Q.1. Construct deterministic finite automata, which is equivalent to N.F.A.

$M = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_3\})$ **7M**

where, transition δ is,

Σ / ϕ	0	1
q_0	q_0, q_1	q_0
q_1	q_2	q_1
q_2	q_3	q_3
q_3	ϕ	q_2

Ans. P.2-19, Q.22

WINTER - 06 (CT)

Q.1. Design a finite state machine which determines the residue mod 3 for each binary string treated as binary integer. **5M**

Ans. P.2-32, Q.47

Q.2. Construct deterministic finite automata equivalent the following :

(i) $M = (\{p, q, r, s\}, \{0, 1\}, \delta, p, \{q, s\})$

where δ is given as,

Q	Input	
	0	1
p	q, s	q
q	r	q, r
r	s	p
s	-	p

8M

Ans. P.2-17, Q.20

Q.3. Give an algorithm to construct minimum automata and use it to minimize the automata whose transition table is given below: **8M**

Σ / ϕ	a	b
$\rightarrow q_0$	q_0	q_3
q_1	q_2	q_5
q_2	q_3	q_4
q_3	q_0	q_5
q_4	q_0	q_6
q_5	q_1	q_4
$*q_6$	q_1	q_3

Ans. P.2-39, Q.57

SUMMER - 07(CT)

Q.1. Differentiate between deterministic and non-deterministic finite automata. **4M**

Ans. P.2-13, Q.12

Q.2. Construct the transition system (DFA) for the following language :

(i) $L_1 = \{ a^m b^n : m, n \text{ are +ve} \}$

(ii) $L_2 = \{a^m b a^n : m, n \text{ are +ve}\}$. **4M**

Ans. P.2-33, Q.50

Q.3. Construct finite automata which accepts all those binary strings in which the number of 1's and 0's are respectivley divisible by 3 and 2. **5M**

Ans. P.2-33, Q.51

Q.4. Construct Moore Machine equivalent to the Mealy Machine represented by the transition table given below : **8M**

Pre-sent state \ Σ	Next state			
	a = 0		a = 1	
	State	Output	State	Output
q_1	q_1	1	q_2	0
q_2	q_4	1	q_4	0
q_3	q_2	0	q_3	1
q_4	q_3	0	q_1	0

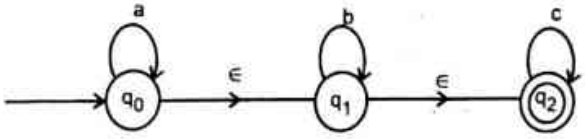
Ans. P.2-53, Q.71

WINTER - 07 (CT)

Q.1. Differentiate clearly betwen NFA and DFA. **3M**

Ans. P.2-13, Q.12

Q.2. Give the steps required to convert a transition system with ∈-moves into an equivalent transition system with out ∈-moves. Obtain an equivalent automata without ∈ -moves for the transition system shown in figure. **8M**



Ans. P.2-22, Q.27

Q.3. Construct a Moore machine equivalent to the Mealy machine M given in the table : **7M**

Present State	Next State			
	a = 0		a = 1	
	State	Output	State	Output
→ q_1	q_1	1	q_2	0
q_2	q_4	1	q_4	1
q_3	q_2	1	q_3	1
q_4	q_3	0	q_1	1

Ans. P.2-51, Q.69

Q.4. Explain the application of finite automata. **3M**

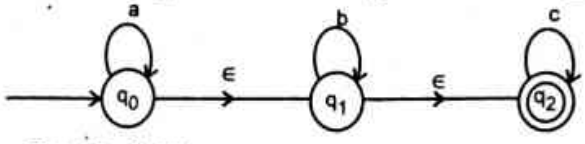
Ans. P.2-10, Q.4

SUMMER - 08 (CT)

Q.1. Differentiate clearly between NFA and DFA with suitable example. **4M**

Ans. P.2-13, Q.12

Q.2. Give the steps required to convert a transition system with ∈-moves into an equivalent transition system without ∈-moves. Obtain an equivalent automata without ∈ -moves for the transition system shown in figure. **7M**



Ans. P.2-22, Q.27

Q.3. Explain clearly what you understand by the term "∈ closure of a state". **3M**

Ans. P.2-21, Q.25

Q.4. Construct-Mealy machine which is equivalent to the Moore machine given in table: **6M**

Present state	Next State		Output
	a = 0	a = 1	
→ q_0	q_1	q_2	1
q_1	q_3	q_2	0
q_2	q_2	q_1	1
q_3	q_0	q_3	1

Ans. P.2-57, Q.80

WINTER - 08 (CT)

Q.1. Explain the structure of finite state machine (F.S.M.) Design for two bit binary adder. **8M**

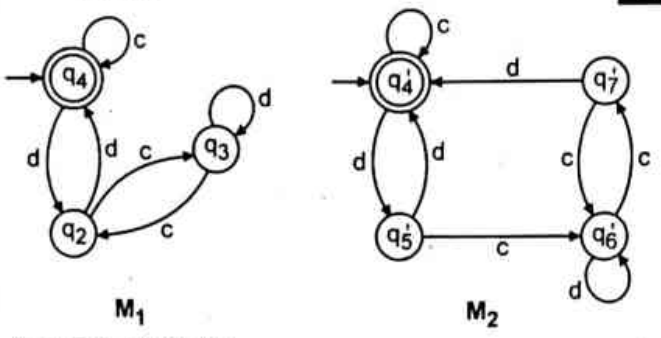
Ans. P.2-9, Q.2

Q.2. Construct Moore Machine equivalent to the Mealy Machine as given in the transition table. Transition table for Mealy Machine **6M**

Present state	Next state			
	a = 0		a = 1	
	State	O/P	State	O/P
→ q_1	q_1	1	q_2	0
q_2	q_4	1	q_4	1
q_3	q_2	1	q_3	1
q_4	q_3	0	q_1	1

Ans. P.2-51, Q.69

Q.3. Show that the automata M_1 & M_2 are equivalent or not. **4M**



M_1 M_2

Ans. P.2-30, Q.37

SUMMER - 09 (CS)

Q.1. Differentiate between NFA and DFA. **3M**

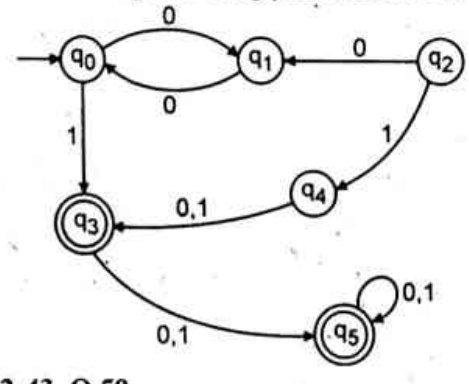
Ans. P.2-13, Q.12

Q.2. Convert the following NFA to its equivalent DFA. **10M**

Q \ Σ	0	1
→ p	p	{p, q}
* q	r	r
r	-	s
* s	s	s

Ans. P.2-18, Q.21

Q.3. Construct the minimized DFA for the following transition diagram using pair table method. **7M**



Ans. P.2-43, Q.59

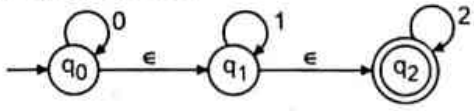
SUMMER - 09 (CT)

Q.1. Construct a DFA equivalent to $M = (\{q_0, q_1, q_2, q_3\}, \{a, b\}, \delta, q_0, \{q_3\}, \delta,$ is given as. **5M**

State \ Σ	a	b
→ q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	$\{q_1\}$
q_2	–	$\{q_3\}$
(q_3)	–	–

Ans. P.2-19, Q.23

Q.2. Convert the given NFA with ∈-moves to NFA without ∈ – moves. **6M**



Ans. P.2-21, Q.27

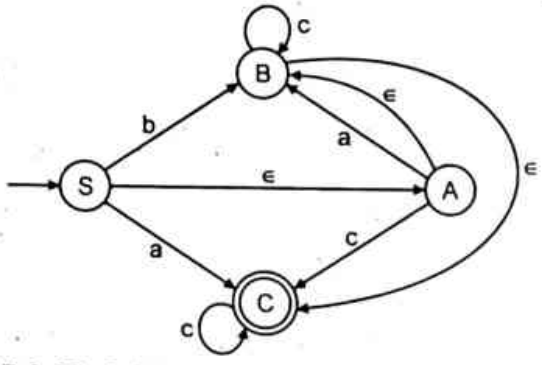
Q.3. Construct a minimum state automaton equivalent to the finite automaton. **8M**

ϕ \ Σ	0	1
→ q_0	q_1	q_5
q_1	q_6	q_2
(q_2)	q_0	q_2
q_3	q_2	q_6
q_4	q_7	q_5
q_5	q_2	q_6
q_6	q_6	q_4
q_7	q_6	q_2

Ans. P.2-44, Q.60

WINTER - 09 (CS)

Q.1. Convert the following NFA with ∈-transited into DFA. **7M**



Ans. P.2-25, Q.31

Q.2. Convert the following Mealy machine to Moore machine. **7M**

	a = 0		a = 1	
	State	O/P	State	O/P
→ q_1	q_3	0	q_2	0
q_2	q_1	1	q_4	0
q_3	q_2	1	q_1	1
q_4	q_4	1	q_3	0

Ans. P.2-52, Q.70

Q.3. Design a DFA to accept all natural number divisible by '3'. Also explain the logic of designing DFA. **5M**

Ans. P.2-32, Q.47

Q.4. Define 'ϵ' closure. **2M**

Ans. P.2-21, Q.25

WINTER - 09 (CT)

Q.1. Construct a Finite Automata accepting strings of a's and b's containing the number of b's is divisible by 3. **3M**

Ans. P.2-32, Q.46

Q.2. Construct a F.A. accepting strings of 0's and 1's ending in 010 or 0010. **2M**

Ans. P.2-31, Q.41

Q.3. Construct a Mealy m/c which is equivalent to the m/c given in table : **7M**

P.S.	N.S.		O/P
	a = 0	a = 1	
→ q_0	q_1	q_2	1
q_1	q_3	q_2	0
q_2	q_2	q_1	1
q_3	q_0	q_3	1

Ans. P.2-57, Q.80

SUMMER - 10 (CS)

Q.1. Design a Moore and Mealy machine that gives a | as its output if and only if the last three digit as all is assume that $\Sigma = \{0,1\}$. **7M**

Ans. P.2-58, Q.82

Q.2. Construct a minimum state automaton equivalent to a given automaton M whose transition table is given below: **7M**

Input

States	a	b
→ q_0	q_0	q_3
(q_1)	q_2	q_5
q_2	q_3	q_4
q_3	q_0	q_5
q_4	q_0	q_6
q_5	q_1	q_4
q_6	q_1	q_3

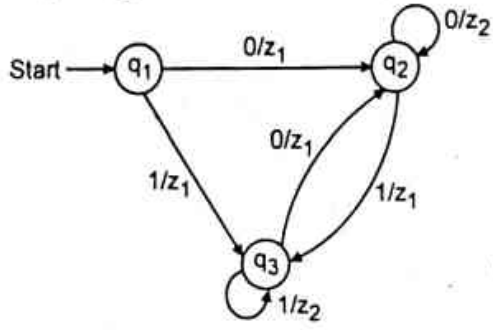
Ans. P.2-46, Q.61

SUMMER - 10 (CT)

Q.1. Construct DFA that recognize the language $L = \{W \in \{0,1\}^* | w|_0 =$ even and $|W_1| =$ odd$\}$. **6M**

Ans. P.2-31, Q.43

Q.2. Construct Moore m/c which is equivalent to the Mealy m/c given in transition diagram. **7M**



Ans. P.2-56, Q.78

WINTER - 10 (CS)

Q.1. $M = \{(q_1, q_2, q_3), (0,1), \delta, q_1, (q_3)\}$ is a non deterministic finite automata where 'd' is given by:

$\delta(q_1,0) = (q_2,q_3)$

$\delta(q_1,1) = (q_1)$

$\delta(q_2,0) = (q_1,q_2)$

$\delta(q_2,1) = (\phi)$

$\delta(q_3,0) = (q_2)$

$\delta(q_3,0) = (q_2)$

$\delta(q_3,1) = (q_1,q_2)$

Construct an equivalent DFA. **6M**

Ans. P.2-14, Q.15

Q.2. Construct Mealy m/c which can output even, odd according as total number of 1's encountered is even or odd. The input symbol are 0,1 and '1'. **5M**

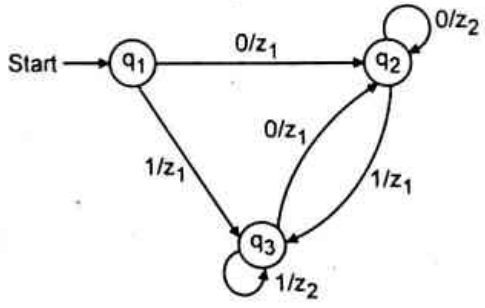
Ans. P.2-58, Q.83

WINTER - 10 (CT)

Q.1. Find minimal state DFA to accept strings over (0, 1) which contains both 11 and 010 as substring. **8M**

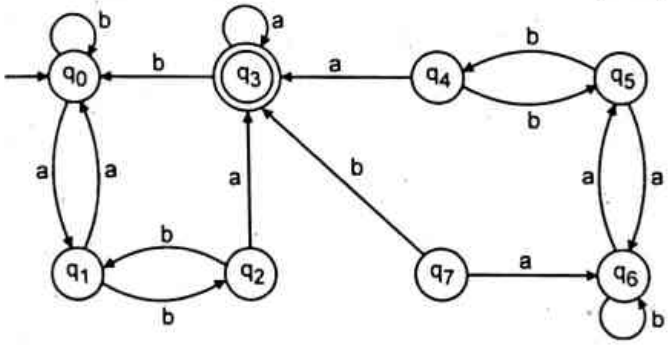
Ans. P.2-20, Q.24

Q.2. Consider a Mealy machine in Fig. construct a Moore machine equivalent to this Mealy machine. **5M**



Ans. P.2-56, Q.78

Q.3. Construct the minimum state automaton equivalent to the transition diagram given below. **6M**



Ans. P.2-41, Q.58

Q.4. Give the definition of ϵ-closure of state. **3M**

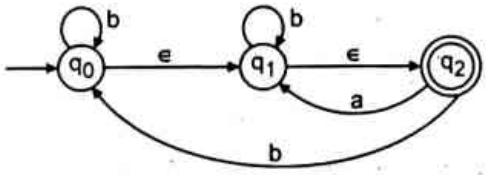
Ans. P.2-21, Q.25

Q.5. Construct a FA accepting a decimal digit which is divisible by 3. **4M**

Ans. P.2-32, Q. 47

SUMMER - 11 (CS)

Q.1. Define ϵ-closure. Find ϵ-closure of all the states of given NFA. Remove epsilon without changing the acceptance. Also explain the method. **10M**



Ans. P.2-21, 26, Q.25, Q.32

Q.2. Give difference between Moore and Mealy machine. Give the steps of conversion from Mealy to Moore Machine. **3M**

Ans. P.2-49, 50, Q.64, Q.67

SUMMER - 11 (CT)

Q.1. Explain the importance of term "non determinism" in finite automata theory. **3M**

Ans. P.2-12, Q.9

WINTER - 11 (CS)

No question asked in RTMNU exam.

WINTER - 11 (CT)

Q.1. Give the difference between NFA and DFA. Construct a minimum state automata equivalent to FA. **10M**

State \ Σ	0	1
→ q0	q1	q5
q1	q6	q2
(q2)	q0	q2
q3	q2	q6
q4	q7	q5
q5	q2	q6
q6	q6	q4
q7	q6	q2

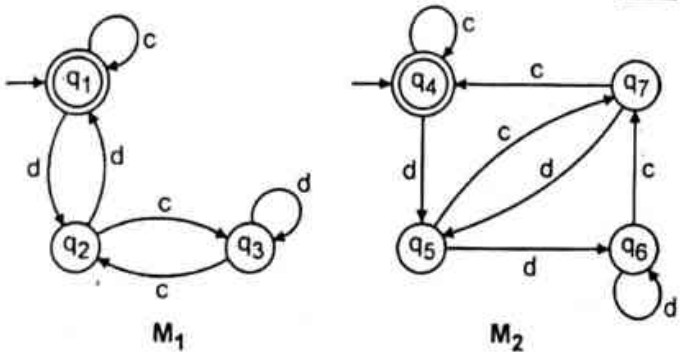
Ans. P.2-13, 44, Q.12, 60

Q.2. Construct a Moore machine equivalent to Mealy machine. **4M**

Present State	Next State			
	Input a = 0		Input a = 1	
	State	Output	State	Output
→ q_1	q_3	0	q_2	0
q_2	q_1	1	q_4	0
q_3	q_2	1	q_1	1
q_4	q_4	1	q_3	0

Ans. P.2-50, Q.67

Q.3. Are the automatas M_1 and M_2 defined by following transition diagrams equivalent? **6M**



M_1 M_2

Ans. P.2-30, Q.36

Q.4. Construct FA accepting all strings over {0, 1} ending in 010 or 0010. **5M**

Ans. P.2-31, Q.41

SUMMER - 12 (CS)

Q.1. Design DFA to accept:

(i) All strings of {0, 1} with even number of 0's and odd number of 1's. **2M**

Ans. P.2-31, Q.43

Q.2. Construct a Mealy machine which outputs remainder generated after dividing an integer number by five. **7M**

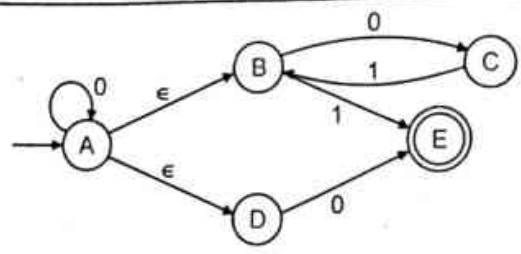
Ans. P.2-57, Q.81

SUMMER - 12 (CT)

Q.1. Design DFA to check whether the given binary number is divisible by 5. **7M**

Ans. P.2-31, Q.42

Q.2. Construct minimized DFA from the NFA given below. **6M**



Ans. P.2-28, Q.34

Q.3. Explain the importance of term "Non determinism" in finite automata theory. **3M**

Ans. P.2-12, Q.9

Q.4. Design Moore machine for a binary input sequence such that if it has a substring 101, the machine outputs A. If it has a substring 110, the machine outputs B. Otherwise it outputs C. **7M**

Ans. P.2-55, Q.77

WINTER - 12 (CS)

Q.1. Answer following:

(i) An FSM can be used to add two given integer.

(a) true

(b) false

(c) may be true. **2M**

(ii) Mealy machine.

(a) generates no language as such

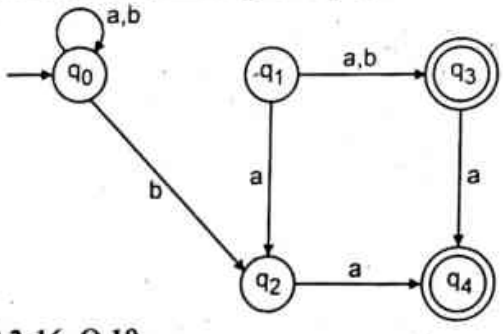
(b) has no terminal state

(c) has equivalent Moore machine

(d) all of these.

Ans. P.2-10, Q.5

Q.2. Construct DFA equivalent to NDFA M whose transition diagram is given by a, b. **5M**

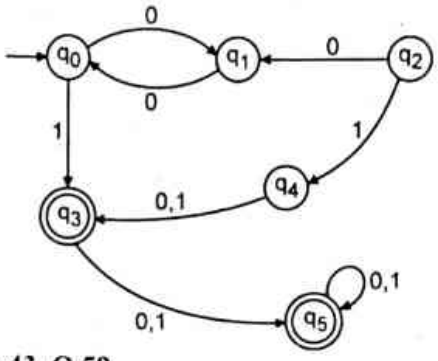


Ans. P.2-16, Q.19

Q.3. Design a Mealy machine that recognize the double occurrence of symbol 'a' in input string $S \in \Sigma^*$, where $\Sigma = \{a, b\}$. **5M**

Ans. P.2-56, Q.79

Q.4. Construct minimum state automata for the transition diagram given below using pair table method. **6M**



Ans. P.2-43, Q.59

Q.5. What do you mean by $\in$- closure of a state? Explain with suitable example. **2M**

Ans. P.2-21, Q.25

WINTER - 12 (CT)

Q.1. Give the analytical definition of Non-deterministic and Deterministic Finite Automata. Differentiate clearly between NFA and DFA. **5M**

Ans. P.2-13, 11, Q.12, Q.8

Q.2. Construct a minimum state DFA equivalent to the NFA given below :

$M = (\{p,q,r,s\},\{0,1\}, \delta, p, \{q,s\})$

where, δ is defined by following transition table: **5M**

Q \ Σ	0	1
p	{q, s}	{q}
q	{r}	{q, r}
r	{s}	{p}
s	ϕ	{p}

Ans. P.2-17, Q.20

Q.3. Construct Finite Automata for accepting the strings of a's and b's containing number of a's divisible by 3. **6M**

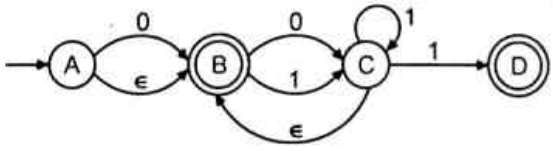
Ans. P.2-32, Q.46

Q.4. Design a transition diagram of Moore machine for accepting the sequence 01100 or 10011, $\Sigma = (0, 1)$ and output x or y respectively for sequence but z for otherwise. **8M**

Ans. P.2-55, Q.77

SUMMER - 13 (CS)

Q.1. Get NFA without $\in$ for following. **5M**



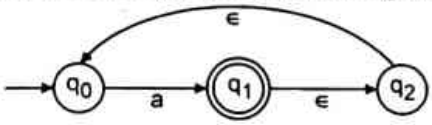
Ans. P.2-27, Q.33

Q.2. Design DFA accepting the set of all string that, when interpreted in reverse as a binary integer is divisible by 5. Example 0, 10011, 0101. **3M**

Ans. P.2-31, Q.42

SUMMER - 13 (CT)

Q.1. Obtain NFA without $\in$ for following NFA. **4M**



Ans. P.2-24, Q.30

Q.3. Optimize following DFA using marking algorithm: **7M**

δ	0	1
→a	b	f
b	g	c
* c	a	c
d	c	g
e	h	f
f	c	g
g	g	e
h	g	c

Ans. P.2-37, Q.54

SOLVED QUESTION BANK

[Sequence given as per syllabus]

INTRODUCTION

The finite automata is a mathematical model of a system, with discrete inputs and outputs. The system can be in any one of a finite number of internal configurations or "states". The state of system summarizes the information concerning past inputs that is needed to determine the behaviour of the system on subsequent inputs. The control mechanism of an elevator is good example of a finite state system. That mechanism does not remember all previous requests for service but only the current floor, the direction of motion (up or down), and the collection commonly used programs such as text editors and the lexical analyzers found in most compilers are often designed as finite state systems.

Finite automata is called "finite" because number of possible states and number of letter in the alphabet are both finite, and "automata" because the change of the state is totally governed by the input. It is deterministic, since, which state is next is automatic not will-full, just as the motion of the hands of the clock is automatic, while the motion of hands of a human is presumably the result of desire and thought.

FINITE AUTOMATA

Q.1. Define and explain finite Automata.

Ans. Finite Automata :

- Finite Automata is a basic model of computation, which will translate the I/P into O/P without any human interaction.
- It is a mathematical model of finite state machine. It consists of finite sets of states, I/P symbols and set of transitions.
- In finite state machine the internal state of machine alters when the required machine receives an I/P and generates the O/P.
- It depends upon present state and I/P and generates the O/P.
- It depends upon present state and I/P applied, the O/P generated is the result generated after the process satisfying certain characteristics.

Formal Definition :

Finite Automata is denoted by five tuple:

$M = (Q, \Sigma, \delta, q_0, F)$

where,

$Q \rightarrow$ Set of states, $(q_0, ...q_n)$.

$\Sigma \rightarrow$ Set of I/P symbols or alphabet.

$\delta \rightarrow$ Set of transitions, $Q \times \Sigma \rightarrow Q$.

$q_0 \rightarrow$ Initial state, $q_0 \subseteq Q$.

$F \rightarrow$ Set of final state, $F \subseteq Q$.

Capability of Finite Automata:

- It is used to decide whether the input is accepted or not accepted.
- If the machine terminates with final states the I/P is accepted.
- There are two types of Finite Automata (FA):

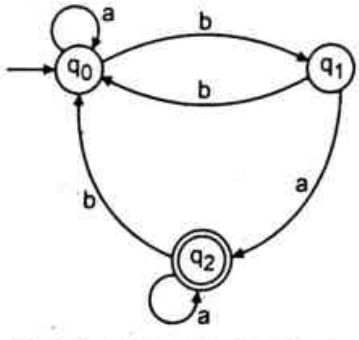
(1) Deterministic Finite Automata (DFA).

(2) Non Deterministic Finite Automata (NFA).

(1) Deterministic Finite Automata (DFA):

- If there is a single (unique) transition from one state to other with a single I/P symbol, then that automata is called as Deterministic finite Automata.

Example:



- A deterministic finite automata is a quintuple.

$M = \{Q, \Sigma, \delta, q_0, F\}$

$M = \{(q_0, q_1, q_2), (a, b), \delta, (q_0), (q_2)\}$

Here, every state (q_0, q_1, q_2) for Input symbol (a,b), performs unique transition, like transitions are:

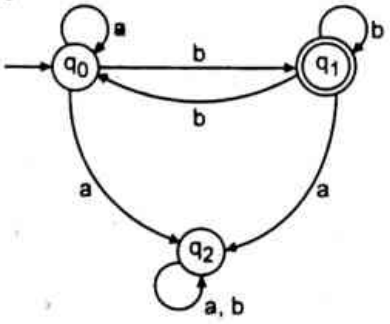
(1) $\delta(q_0,a)\rightarrow q_0$
(2) $\delta(q_0,b)\rightarrow q_1$
(3) $\delta(q_1,a)\rightarrow q_2$
(4) $\delta(q_1,b)\rightarrow q_0$
(5) $\delta(q_2,a)\rightarrow q_2$
(6) $\delta(q_2,b)\rightarrow q_0$

} Unique transitions

(2) Non Deterministic Finite Automata (NFA) :

If there are more than one transitions from one state to another state with a single I/P symbol, then that automata is called as Non-Deterministic Finite Automata.

Example:



$M = \{(q_0,q_1,q_2), (a,b), \delta, (q_0), (q_1)\}$

Here, every state (q_0, q_1, q_2) for Input symbol (a, b), performs more than one transition shown as below:

(i) $\delta(q_0,a)\rightarrow \{q_0,q_2\}$
$\delta(q_0,b)\rightarrow \{q_1\}$

(ii) $\delta(q_1,a)\rightarrow \{q_2\}$
$\delta(q_1,b)\rightarrow \{q_0,q_1\}$

(iii) $\delta(q_2,a)\rightarrow \{q_2\}$
$\delta(q_2,b)\rightarrow \{q_2\}$.

Q.2. Explain the structure of finite state machine with block diagram. Design for two bit binary adder. ***CT: W-08(8M)***

Ans. Structure of finite state machine :

- Finite state machine is a mathematical model which consist of I/P symbol, finite state and transition function.
- It consist of two functions, namely.
  (1) Machine Function (MAF) : I × S = O
  (2) State Function (STF) : I × S = S
  where,
  I : I/P symbol
  O : O/P symbol
  S : State of machine.

Structure of FA / FSM :

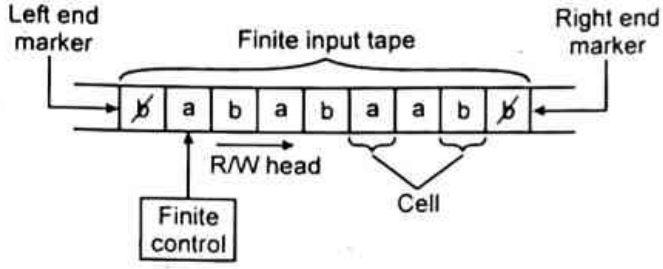


Fig. A finite automata

It consist of three components:
(1) Finite I/P Tape
(2) Read/write head
(3) Finite Control.

(1) Finite I/P Tape:

- It is used to store all the given I/P symbol, at Leftend and Rightend and two end marker are placed.
- End Markers are denoted by three symbol [¢, $, ;] some times '$' is used to show end marker.

(2) Read/write head :

It checks one I/P symbol at a time and move only from left to right direction.

(3) Finite Control:

It work as a store room, it stores one I/P symbol at a time, then read/write head check as scan that symbol and goes towards next symbol. If it will get end number to scan, then say that scanning will be completed.

Q.3. Explain transition diagram of a finite automaton.

Ans.

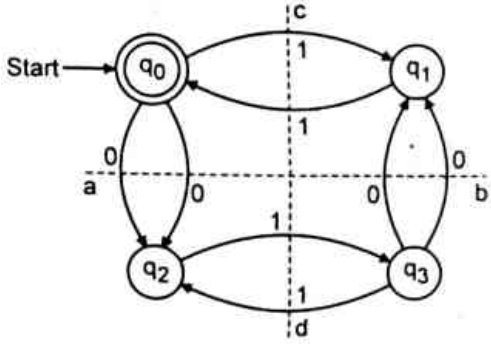


Fig. Transition diagram of a finite automaton

Transition diagram of a finite automaton :

- Above figure shows transition diagram of a finite automaton. The initial state q_0, is indicated by the arrow labeled "start". There is a one final state, q_0 in this case, indicated by double circle. The FA accepts all strings of 0's and 1's.
- To see this, visualize "control" as travelling from state to state in the diagram. Control starts at q_0 and must finish at q_0 if input sequence is to be accepted. Each 0-input causes control to cross the horizontal line a-b, which a 1-input does not. Thus control is at a state above the line a - b if and only if the input seen so far contains an even number of 0's.
- Similarly, control is a state to the left of the vertical line c-d if and only if the input contains an even number of 1's. This control is at q_0 if and only if there are both an even number of 0's and even number of 1's in the input. The FA uses its state to record only the parity of the number of 0's and number of 1's, not the actual numbers, which would require an infinite number of states.

Q.4. Give various application of finite automata. **CT: W-07(3M)**

OR Explain briefly application of finite automata.

OR Explain with suitable example the application of finite automata.

Ans. Application of Finite Automata are :

(1) Lexical Analyzers.

(2) Text Editors.

(3) In switching circuit design.

(4) Text processing program.

(1) Lexical Analyzers : The tokens of programming languages can be expressed as regular sets.

Example : ALGOL identifiers which are upper or lower - case letters followed by any sequence of letters and digits with no limit on length, may be expressed as :

(letter) (letter/digit)*

- A number of lexical-analyzer generators take as input, a sequence of regular expressions describing the tokens and produce a single finite automaton recognizing any token.
- Usually, they convert the regular expression to an NFA with transitions and then construct subsets of states to produce a DFA directly, rather than first eliminating ϵ-transitions.
- Each final state indicates the particular token found, so the automaton is really a Moore machine.

(2) Text Editors :

- Certain text editors and similar programs permit the substitution of a string for any string matching a given regular expression.

  Example:

  The UNIX text editor allows a command such as : s/ bbb */b/ that substitutes a single blank for the first string of two or more blanks found in a given line.
- Let "any" denote the expression $a_1 + a_2 + ... + a_n$ where a_i's are all of a computers characters except the "new line" character.
- We could convert a regular expression r to a DFA that accepts any*r.
- The UNIX text editor converts the regular expression any* r is converted to an NFA with transitions and the NFA is then simulated directly.

Q.5. Answer following: **CS : W-12(2M)**

(i) An FSM can be used to add two given integer.

(a) true

(b) false

(c) may be true

Ans. (b) False, A finite state machine is a mathematical model which takes some input to produce the relevant output.

(ii) Mealy Machine.

(a) generates no language as such

(b) has no terminal state

(c) has equivalent Moore machine

(d) all of these

Ans. (d) All of these.

REGULAR LANGUAGES

Q.6. Give formal definition of Regular language.

Ans. Regular language:

- The collection of regular languages over an alphabet Σ is defined recursively as follows:

(i) The empty language ϕ is a regular language.

(ii) For each $a \in \Sigma$, the singleton language {a} is a regular language.

(iii) If A and B are regular languages, then $A \cup B$ (union), A . B (concatenation) and A* are regular languages.

(iv) No other languages over Σ are regular.

Example: All finite languages are regular, in particular the empty string language $\{\in\} = \phi^*$ is regular.

- A simple example of a language that is not regular is the set of strings $\{a^n b^n \mid n \geq 0\}$.
- It cannot be recognized with a finite automata, since a finite automata has finite memory and it cannot remember the exact number of a's.

Q.7. Mention the properties of regular language.

Ans. In theoretical computer science, a regular language is a formal language that satisfies the following equivalent properties:

(i) It can be accepted by a deterministic finite state machine.

(ii) It can be accepted by a non-deterministic finite state machine.

(iii) It can be accepted by an alternating finite automation.

(iv) It can be described by a formal regular expression.

(v) It can be generated by regular grammar and prefix grammar.

(vi) It can be accepted by a read-only Turing machine.

DETERMINISTIC AND NON-DETERMINISTIC FINITE AUTOMATA

Q.8. Give the analytical definition of non-deterministic finite automata (NFA) and deterministic finite automata (DFA). **CT: W-12(2M)**

OR What do you understand by determinism and Non-determinism?

Ans. Non Deterministic Finite Automata (NFA) :

- If there are more than one transition from one state to another state with a single I/P symbol, then that automata is called as NFA.
- Therefore NFA is finite automata in which there may exists more than one path corresponding to a Σ.
- NFA is denoted by a 5 tuple.

$M = (Q, \Sigma, \delta, q_0, F)$

where,

Q is finite set of states.

Σ is finite alphabet or input symbol.

$q_0 \in Q$ is initial state.

$F \subseteq Q$ is set of final state.

δ is transition function mapping from $Q \times \Sigma$ to 2^Q (to take care of zero or more transition).

Example :

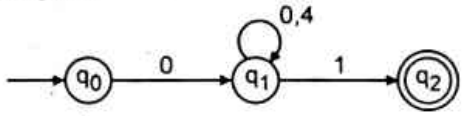


Fig. (a)

$NFA = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, \{q_0\}\{q_2\})$

$\delta(q_0, 0) = q_1$, $\delta(q_1, 0) = q_1$, $\delta(q_1, 1) = \{q_1, q_2\}$

$\delta(q_1, 1) = q_1$, $\delta(q_1, 1) = q_2$

Deterministic Finite Automata (DFA) :

- The finite automata in which there exists only one transition from one state with a single input symbol to the another state is called as Deterministic Finite Automata.
- DFA is denoted by 5 tuple $M = (Q, \Sigma, \delta, q_0, F)$.

where,

$Q \rightarrow$ is set of finite states.

$\Sigma \rightarrow$ is set of finite input symbol.

$q_0 \rightarrow q_0 \in Q$ is start state.

$F \rightarrow F \subseteq Q$ is set of final states.

$\delta \rightarrow$ transition function mapping from $Q \times \Sigma$ to Q.

Example:

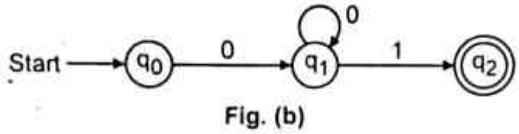


Fig. (b)

DFA = $(\{q_0, q_1, q_2\}, \{0, 1\}, \delta, \{q_0\}\{q_2\})$

$\delta(q_0,0) = q_1$, $\delta\ (q_1,0) = q_1$

$\delta(q_1, 1) = q_2$

Q.9. Explain the importance of term "non-determinism" in finite automata theory.

CT :S-11, 12(3M)

Ans.

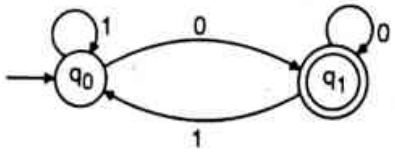
- A "non-deterministic" finite automata (NFA) has the power to be in several states at once.
- Each NFA accepts a language that is also accepted by some DFA.
- We can permit several possible "next states" for a given combination of current state and input symbol.
- Non-determinism is an essential feature of FA, every NFA is equivalent to FA.
- Thus, we get profit from the greater flexibility of non-determinism, when we wish to design language recognizers without actually extending the class of language we are able to describe.

Q.10. Explain acceptance of string by Deterministic Finite Automata.

Ans. Acceptance of string by DFA :

- If x is a string made of symbols belonging to Σ of the finite automata, then x is accepted by the finite automata, if a path corresponding to x in a finite automata starts in an initial state and ends in one of the final states of the automata, i.e. there exists a sequence of moves for x in the finite automata which takes transitions from initial state to one of the final states of the automata.

Transition :



Let, M = $(\{q_0, q_1,\}, \{0,1\}, \delta, q_0, \{q_1\})$,

where,

$\delta(q_0,0) = q_1, \delta(q_0,1) = q_0$.

$\delta(q_1,0) = q_1, \delta(q_1,1) = q_0$.

Let, x be 010, to find x is accepted by automata or not, we proceed as follows :

Method I :

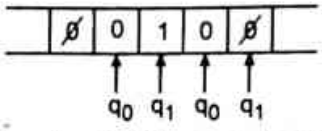
$\delta_1(q_0,0) = \delta(q_0,1) = q_1$

$\therefore\ \delta_1(q_0,01) = \delta(\delta_1(q_0,0),1) = q_0$

$\therefore\ \delta_1(q_0,010) = \delta(\delta_1(q_0,01),0) = q_1$

[$\because$ q_1 is a member of F, x = 010 is accepted by the automata.]

Method II:



Here, (i) 'q_0' gets '0' goes to 'q_1'

(ii) 'q_1' gets '1' goes to 'q_0'

(iii) 'q_0' gets '0' goes to 'q_1'

- In this method, if we found Final state below Right end Marker then we can say that string is accepted by DFA.
- Here, 'q_1' is a Final state and we caught under Right end Marker hence string is accepted by FA.

Method III:

String {0 | 0}

Present state	Current I/P symbol	Next State
→ q_0	0	q_1
* q_1	1	q_0
q_0	0	q_1

- In a machine or in a state function two states are there present as well as next state but, no machine take a pause on next state, as if reaches to the next state again, it will be converted into present state.
- In the above method if we get final state as a last next state then say that string is accepted by DFA.

Q.11. Explain acceptance of string by NFA.

Ans. Acceptance of string by NFA :

- If x is a string made of symbols in Σ of the NFA, i.e. x is in Σ^*, then x is accepted by the NFA, if there exists at least one path corresponding to x in a NFA which starts in an initial state and ends in one of the final states of the NFA.

- If $\delta_1(\{q_0\}, x) = P$

  where P is a set containing at least one member of F, then x is accepted by the NFA.

  Transition :

- Consider FA given below:

  $M = (\{q_0, q_1, q_2, q_3\}, \{0,1\} \delta, q_0, \{q_3\})$

  where $\delta(q_0,0) = \{q_1\}$, $\delta(q_0,1) = \phi$

  $\delta(q_1,0) = \{q_1\}$, $\delta(q_1,1) = \{q_1, q_2\}$

  $\delta(q_2,0) = \phi$, $\delta(q_2,1) = \{q_3\}$

  $\delta(q_3,0) = \{q_3\}$, $\delta(q_3,1) = \{q_3\}$

- If x = 0111, then to find, whether x is accepted by the NFA or not we proceed as follows:

  $\delta_1(\{q_0\},0) = \{q_1\}$

  $\therefore \delta_1(\{q_0\},01) = \delta_1(\delta(\{q_0\},0), 1) = \delta_1(\{q_1\},1)$

  $= \delta(q_1,1) = \{q_1,q_2\}$

  $\therefore \delta_1(\{q_0\},011) = \delta_1(\delta(\{q_0\},01), 1)$

  $= \delta_1(\{q_1,q_2\},1)$

  $= \delta(q_1,1) \cup \delta(q_2,1)$

  $= \{q_1,q_2\} \cup \{q_3\} = \{q_1,q_2,q_3\}$

  Since, $\delta_1(\{q_0\},0111) = \{q_1,q_2,q_3\}$,

  which contains q_3, a member of F of NFA,

  hence x = 0111, is accepted by the NFA.

Q.12. Differentiate clearly between NFA and DFA.

CS : S-09(3M), CT: S-07,08,(4M), W-07,11,12(3M)

OR Explain the point of differences between the determinism and non-determinism.

Ans.

Sr. No.	Non-deterministic finite automata (NFA)	Deterministic finite automata (DFA)
(1)	If the basic finite automata model is modified to allow zero, one, or more transitions from one state to another state on the same input symbol. The new finite automata is called NFA	DFA consist of a finite set of states and a set of transitions from state to state occur on single input symbols choosen from an alphabet Σ.
(2)	NFA is denoted by 5 tuple. $(Q, \Sigma, \delta, q_0, F)$ where, Q = finite set of states. Σ = set of input symbol. q_0 = start states. F = final states. δ = transition function from $Q \times \Sigma$ to 2^Q.	DFA is denoted by 5 tuple $(Q, \Sigma, \delta, q_0, F)$ where, Q = set of states. Σ = set of input symbol. q_0 = start states. F = final states. δ = transition function from $Q \times \Sigma$ to Q.
(3)	In NFA there exists one or more transition from state to state on same input symbol.	In DFA, there exists exactly one transition from state to state on same input symbol.
(4)	In NFA state has an ϵ-transition.	In DFA no state has an ϵ-transition i.e. a transition on input ϵ.
(5)	NFA can recognize the regular set slower than DFA.	DFA can recognize the regular set faster than NFA.

Q.13. Explain steps of conversion NFA to DFA.

Ans. Step for conversion of NFA to DFA are as follows:

(1) DFA (Σ) = NFA (Σ) is input symbol.

(2) DFA (q_0) = NFA (q_0) is initial state.

(3) The states permitted in DFA will be represented as, Q' which is a power set of NFA.

Example: $Q' = 2^Q$ is power set.

(4) δ' is equal to transitions of DFA:

The transitions will start from initial state in the given NFA. The resultant states will be considered as valid if it is available in the power set. The transition generation process is assumed to be completed when no new state is left as unprocessed.

(5) Defining final states of DFA:

Final state of DFA will be all the states i.e. q_0 and will operate all the input symbol present in the transition (step 4) containing as a component, of final state of NFA.

Another Method:

(1) The initial state of DFA and NFA is same.

(2) Final state of DFD are those states in which final state of NFA will appear.

(3) If two states are equivalent, Equivalent states or definition :

Those, state having same transition with same characteristic means both are either final state or Non-final state], then replace second state by first one, and delete second one.

Q.14. Convert following NFA to DFA:

$M = \{(q_0, q_1), (0,1), \delta, (q_0), (q_1)\}$

where, ' δ ' is

Q \ Σ	0	1
→ $\{q_0\}$	$\{q_0, q_1\}$	$\{q_1\}$
* $\{q_1\}$	–	$\{q_0, q_1\}$

Ans. Formal definition:

$M = \{Q, \Sigma, \delta, q_0, F\}$

$M = \{\{q_0, q_1\}, \{0, 1\}, \delta, \{q_0\}, \{q_1\}\}$

Step 1: Transition table of DFA:

Q \ Σ	0	1
→ $\{q_0\}$	$\{q_0, q_1\}$	$\{q_1\}$
*$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1\}$
*$\{q_1\}$	ϕ	$\{q_0, q_1\}$

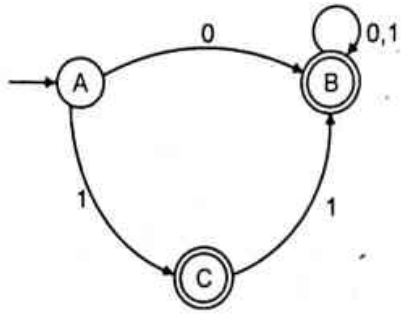
In step 1 initial state of NFA and DFA is same and their transitions are also same and use all transitions as a state then we can say given NFA is converted into DFA.

Step 2: In step 2 rename all new states with any alphabet.

Here, $\{q_0\} = A$, $\{q_0, q_1\} = B$, $\{q_1\} = C$

Q \ Σ	0	1
→ A	B	C
*B	B	B
*C	ϕ	B

Step 3 : Transition diagram:



Q.15. $M = \{(q_1, q_2, q_3), (0,1), \delta, \{q_1\}, (q_3)\}$ is a non deterministic finite automata where d is given by :

$\delta(q_1, 0) = \{q_2, q_3\}$

$\delta(q_1, 1) = \{q_1\}$

$\delta(q_2, 0) = \{q_1, q_2\}$

$\delta(q_2, 1) = \{\phi\}$

$\delta(q_3, 0) = \{q_2\}$

$\delta(q_3, 1) = \{q_1, q_2\}$.

CS : W-10(6M)

Construct equivalent DFA of a given NFA.

Ans. Formal definition:

$M = (\{q_1, q_2, q_3\}, \{0, 1\}, \delta, \{q_1\}, \{q_3\})$

Transition Table :

Q \ Σ	0	1
→ $\{q_1\}$	$\{q_2, q_3\}$	$\{q_1\}$
$\{q_2\}$	$\{q_1, q_2\}$	ϕ
* $\{q_3\}$	$\{q_2\}$	$\{q_1, q_2\}$

Step 1: Transition table of DFA:

Q \ Σ	0	1
→ q_1	$\{q_2, q_3\}$	$\{q_1\}$
*$\{q_2, q_3\}$	$\{q_1, q_2\}$	$\{q_1, q_2\}$
$\{q_1, q_2\}$	$\{q_1, q_2, q_3\}$	$\{q_1\}$
*$\{q_1, q_2, q_3\}$	$\{q_1, q_2, q_3\}$	$\{q_1, q_2\}$

Note:

(1) All ways try to write states names in sequence so that mistakes will be avoided.

(2) Also have sequence during converting transition into states like here, transition $\{q_2, q_3\}$ is use as a state use as a state then $\{q_1, q_2\}$ is converted into state.

Step 2: Rename all states:

(i) $\{q_1\} = A$

(ii) $\{q_2, q_3\} = *B$

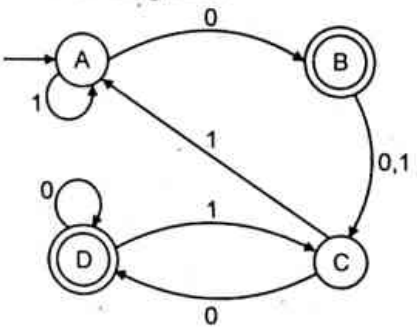
(iii) $\{q_1, q_2\} = C$

(iv) $\{q_1, q_2, q_3\} = *D$

Transition table:

Q \ Σ	0	1
→A	B	A
* B	C	C
C	D	A
* D	D	C

Step 3: Transition diagram:



Q.16. State rules for optimization.

Ans. Rules for optimization:

(1) Initial state cannot be optimized.

(2) Optimization is possible either between final states or between non final states. Final states and non final states is not possible

(3) If the entries for two states in the transition table are same then they can be optimize by removing one of the state and its entries from the table and replace the existing state at the respective places where the remove state is present.

Q.17. Convert NFA to DFA

NFA = ({p, q, r, s}, {0, 1}, δ, {p}, {s})

State \ Σ	0	1
→ p	p, q	p
q	r	r
r	s	ϕ
* s	s	s

Step 1: DFA (Σ) = NFA (Σ)

DFA (p) = NFA (p)

Step 2: $Q' = 2^Q$

Here Q = 4

$\therefore Q' = 2^4 = 16$

$\therefore Q' = \{\phi, p, q, r, s, pq, rs, pr, qs, qr, ps, pqr, qrs, rsp, pqs, pqrs\}$

Step 3:

(1) $\delta'(p, 0) = (p, q)$, $\delta'(p, 1) = (p)$ } A = p

(2) $\delta'(p\ q, 0) = (p\ q\ r)$, $\delta'(p\ q, 1) = (p\ r)$ } B = pq

(3) $\delta'(p\ q\ r, 0) = (p\ q\ r\ s)$, $\delta'(p\ q\ r, 1) = (p\ r)$ } C = pqr

(4) $\delta'(p\ r, 0) = (p\ q\ s)$, $\delta'(p\ r, 1) = (p)$ } D = pr

(5) $\delta'(p\ q\ r\ s, 0) = (p\ q\ r\ s)$, $\delta'(p\ q\ r\ s, 1) = (r\ p\ s)$ } * E = pqrs

(6) $\delta'(p\ q\ s, 0) = (p\ q\ r\ s)$, $\delta'(p\ q\ s, 1) = (p\ r\ s)$ } * F = pqs

(7) $\delta'(r\ p\ s, 0) = (p\ q\ s)$, $\delta'(r\ p\ s, 1) = (p\ s)$ } * G = prs

(8) $\delta'(p\ s, 0) = (p\ q\ s)$, $\delta'(p\ s, 1) = (p\ s)$ } * H = ps

Step 4: Transition Table:

State \ Σ	0	1
→ A	B	A
B	C	D
C	E	D
D	F	A
* E	E	G
~~* F~~	~~E~~	~~G~~
* G	F	H
~~* H~~	~~F~~	~~H~~

Here, Entries of state E & F are same.

And entries of G & H also same. So, after optimization the transition table is :

Transition Table:

State \ Σ	0	1
→ A	B	A
B	C	D
C	E	D
D	E	A
* E	E	G
~~* G~~	~~E~~	~~G~~

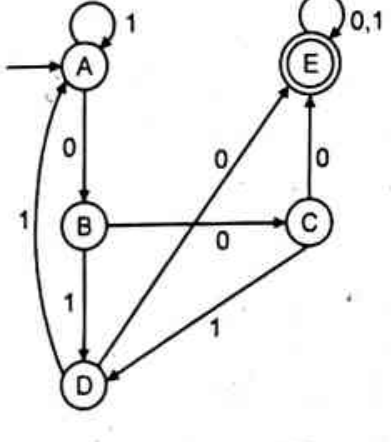
∴ E = G

Here, F is replace by E and H is replace by G.
Now, entries of E & G are same and both are final state so remove one of them.
So, after optimization:

State \ Σ	0	1
A	B	A
B	C	D
C	E	D
D	E	A
* E	E	E

Here, G is replace with E.

Transition diagram :



Q.18. Convert following NFA to DFA.

NFA = ({q_1,q_2 q_3}, {0, 1}, δ, {q_1}, {q_3})

State \ Σ	0	1
→ q_1	q_2 q_3	q_1
q_2	q_1 q_2	–
* q_3	q_3	q_1 q_2

Ans.

Step 1: DFA(Σ) = NFA(Σ)

DFA(q_1) = NFA(q_1)

Step 2:

$Q' = 2^Q$, $Q = 3$ ∴ $2^3 = 8$

∴ $Q' = 8$

Step 3:

(i) $\delta'(q_1, 0) = (q_2q_3)$, $\delta'(q_1, 1) = (q_1)$ } $A = q_1$

(ii) $\delta'(q_2q_3, 0) = (q_1q_2q_3)$, $\delta'(q_2q_3, 1) = (q_1q_2)$ } $*B = q_2q_3$

(iii) $\delta'(q_1q_2q_3, 0) = (q_1q_2q_3)$, $\delta'(q_1q_2q_3, 1) = (q_1q_2)$ } $*C = q_1q_2q_3$

(iv) $\delta'(q_1q_2, 0) = (q_1q_2q_3)$, $\delta'(q_1q_2, 1) = (q_1)$ } $D = q_1q_2$

Step 4: Final states are B & C

Transition Table:

State \ Σ	0	1
→ A	B	A
* B	C	D
* C	C	D
D	C	A

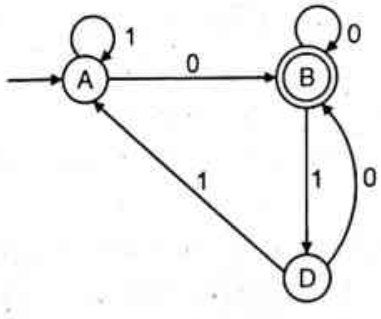
Here B = C

So after optimization, the transition table is

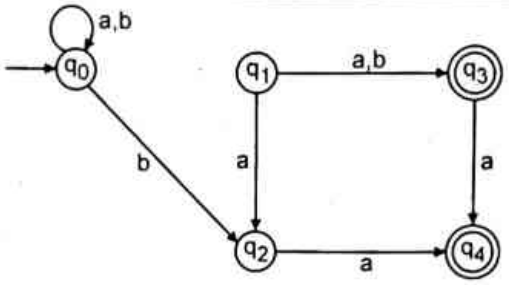
Transition table :

State \ Σ	0	1
A	B	A
*B	B	D
D	B	A

Transition diagram:



Q.19. Construct DFA equivalent to the transition diagram given below. CS : W-12(5M)



Ans. Transition table of NFA :

Q \ Σ	a	b
→ {q0}	{q0}	{q0, q2}
{q1}	{q2, q3}	{q3}
{q2}	{q4}	–
* {q3}	{q4}	–
* {q4}	–	–

Transition table of DFA :

Q \ Σ	a	b
→ {q0}	{q0}	{q0, q2}
{q0, q2}	{q0, q4}	{q0, q2}
* {q0, q4}	{q0}	{q0, q2}

Rename states:

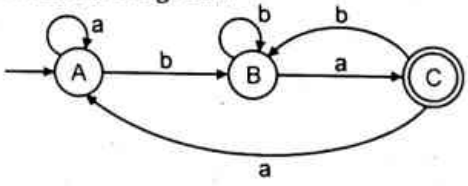
(1) $\{q_0\} = A$.

(2) $\{q_0, q_2\} = B$.

(3) $\{q_0, q_4\} = C$.

Q \ Σ	a	b
→ A	A	B
B	C	B
* C	A	B

Transition Diagram:



Q.20. Construct DFA equivalent to following :

M = {(p, q, r, s), (0, 1), δ, p, (q, s)} its equivalent A δ is given as :

CT:W-06(8M), W-12(5M)

Q \ Σ	0	1
p	{q, s}	{q}
q	{r}	{q, r}
r	{s}	{p}
s	φ	{p}

Ans. Transition table for DFA:

Q \ Σ	0	1
→ {p}	{q, s}	{q}
* {q, s}	{r}	{p, q, r}
* {q}	{r}	{q, r}
{r}	{s}	{p}
*{p, q, r}	{q, r, s}	{p, q, r}
* {q, r}	{r, s}	{p, q, r}
*{s}	{φ}	{p}
* {q, r, s}	{r. s}	{p, q, r}
* {r, s}	{s}	{p}

Rename states:

(1) {p} = A **(2)** {q, s} = B **(3)** {q} = C
(4) {r} = D **(5)** {p, q, r} = E **(6)** {q, r} = F
(7) {s} = G **(8)** {q, r, s} = H **(9)** {r, s} = I

Q \ Σ	0	1
→ A	B	C
* B	D	E
* C	D	F
D	G	A
* E	H	E
* F	I	E
* G	φ	A
* H	I	E
* I	G	A

Here states F & H are equivalent. Hence, replace H by F.

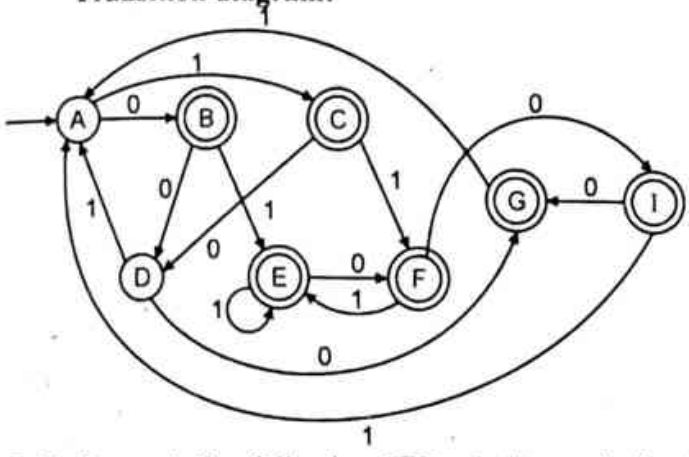
Transition table of DFA:

Q \ Σ	0	1
→ A	B	C
* B	D	E
* C	D	F
D	G	A
* E	F	E

* F	I	E
* G	ϕ	A
* I	G	A

Here state 'D' and 'I' has same transition but not having same characteristics, one is final other non final.

Transition diagram:



Q.21.Convert the following NFA to its equivalent DFA.

Q \ Σ	0	1
→ p	p	{p, q}
* q	r	r
r	-	s
* s	s	s

CS : S-09(10M)

Ans. Conversion of NFA to DFA :

Step 1 : DFA (Σ) = NFA (Σ)

DFA(P) = NFA(P)

Step 2 : $Q' = 2^Q$, $Q = 4$ ∴ $2^4 = 16$

$Q' = 16$

Step 3 :

(i) $\left.\begin{matrix}\delta'(p,0)=(p)\\ \delta'(p,1)=(p,q)\end{matrix}\right\} A = P$

(ii) $\left.\begin{matrix}\delta'(pq,0)=(pr)\\ \delta'(pq,1)=(pqr)\end{matrix}\right\} * B = pq$

(iii) $\left.\begin{matrix}\delta'(pr,0)=(p)\\ \delta'(pr,1)=(pqs)\end{matrix}\right\} C = pr$

(iv) $\left.\begin{matrix}\delta'(pqr,0)=(pr)\\ \delta'(pqr,1)=(pqrs)\end{matrix}\right\} * D = pqr$

(v) $\left.\begin{matrix}\delta'(pqs,0)=(prs)\\ \delta'(pqs,1)=(pqrs)\end{matrix}\right\} * E = pqs$

(vi) $\left.\begin{matrix}\delta'(prs,0)=(ps)\\ \delta'(prs,1)=(pqs)\end{matrix}\right\} * F = prs$

(vii) $\left.\begin{matrix}\delta'(ps,0)=(ps)\\ \delta'(ps,1)=(pqs)\end{matrix}\right\} * G = ps$

(viii) $\left.\begin{matrix}\delta'(pqrs,0)=(prs)\\ \delta'(pqrs,1)=(pqrs)\end{matrix}\right\} * H = pqrs$

Step 4 : Final states are B, D, E, F, G, H

Transition table :

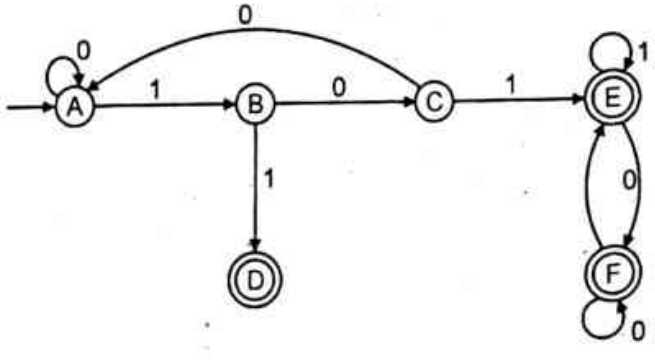
State \ Σ	0	1
A	A	B
* B	C	D
C	A	E
* D	C	H
* E	F	H
*F	G	E
*G	G	E
*H	F	H

Here state E = H and F = G

∴ After optimization the transition table is

State \ Σ	0	1
→A	A	B
* B	C	D
C	A	E
* D	C	E
* E	F	E
*F	F	E

Transition diagram :



Q.22. Construct deterministic finite automata, which is equivalent to NFA.

$M = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_3\})$

where, transition δ is,

Σ / φ	0	1
$\rightarrow q_0$	q_0, q_1	q_0
q_1	q_2	q_1
q_2	q_3	q_3
$*q_3$	φ	q_2

CT : S-06(7M)

Ans. Convertion of NFA to DFA :

Step 1 : $DFA(\Sigma) = NFA(\Sigma)$

$DFA(q_0) = NFA(q_0)$

Step 2 : $Q' = 2^Q$, $Q = 4$ $\therefore$ $2^4 = 16$

$\therefore$ $Q' = 16$

Step 3 :

(i) $\left.\begin{array}{l}\delta'(q_0, 0) = (q_0\ q_1)\\ \delta'(q_0, 1) = (q_0)\end{array}\right\} A = q_0$

(ii) $\left.\begin{array}{l}\delta'(q_0\ q_1, 0) = (q_0\ q_1 q_2)\\ \delta'(q_0\ q_1, 1) = (q_0\ q_1)\end{array}\right\} B = q_0 q_1$

(iii) $\left.\begin{array}{l}\delta'(q_0\ q_1 q_2, 0) = (q_0\ q_1 q_2 q_3)\\ \delta'(q_0 q_1 q_2, 1) = (q_0 q_1 q_3)\end{array}\right\} C = q_0 q_1 q_2$

(iv) $\left.\begin{array}{l}\delta'(q_0\ q_1 q_3, 0) = (q_0\ q_1 q_2)\\ \delta'(q_0 q_1 q_3, 1) = (q_0 q_1 q_2)\end{array}\right\} * D = q_0 q_1 q_3$

(iv) $\left.\begin{array}{l}\delta'(q_0\ q_1 q_2 q_3, 0) = (q_0\ q_1 q_2 q_3)\\ \delta'(q_0 q_1 q_2 q_3, 1) = (q_0 q_1 q_2 q_3)\end{array}\right\} * E = q_0 q_1 q_2 q_3$

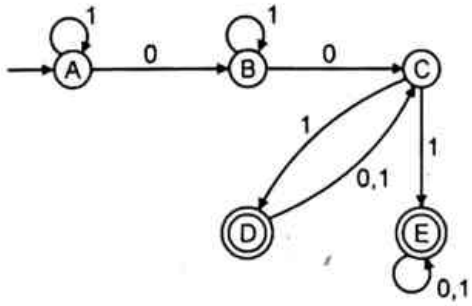
Step 4 : Final states are D and E.

Transition table :

State / Σ	0	1
$\rightarrow$A	B	A
B	C	B
C	E	D
* D	C	C
* E	E	E

Here, optimization of states are not possible, because all states have the different transitions.

Transition diagram :



Q.23. Construct a DFA equivalent to

$M = (\{q_0, q_1, q_2, q_3\}, \{a, b\}, \delta, q_0, \{q_3\},$ **is given as.**

State / Σ	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	$\{q_1\}$
q_2	–	$\{q_3\}$
$*q_3$	–	–

CT: S-09(5M)

Ans. Conversion of NFA to DFA

Step 1 : $DFA(\Sigma) = NFA(\Sigma)$

$DFA(q_0) = NFA(q_0)$

Step 2 : $Q' = 2^Q$, $Q = 4$ $\therefore$ $2^4 = 16$

$Q' = 16$

Step 3 :

(i) $\left.\begin{array}{l}\delta'(q_0, a) = (q_0\ q_1)\\ \delta'(q_0, b) = (q_0)\end{array}\right\} A = q_0$

(ii) $\left.\begin{array}{l}\delta'(q_0\ q_1, a) = (q_0 q_1 q_2)\\ \delta'(q_0\ q_1, b) = (q_0 q_1)\end{array}\right\} B = q_0 q_1$

(iii) $\left.\begin{array}{l}\delta'(q_0\ q_1 q_2, a) = (q_0\ q_1 q_2)\\ \delta'(q_0 q_1 q_2, b) = (q_0 q_1 q_3)\end{array}\right\} C = q_0 q_1 q_2$

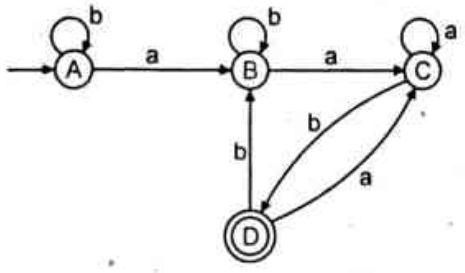
(iv) $\left.\begin{array}{l}\delta'(q_0\ q_1 q_3, a) = (q_0\ q_1 q_2)\\ \delta'(q_0 q_1 q_3, b) = (q_0 q_1)\end{array}\right\} * D = q_0 q_1 q_3$

Step 4 : Final state is D.

Transition table :

State \ Σ	a	b
→A	B	A
B	C	B
C	C	D
* D	C	B

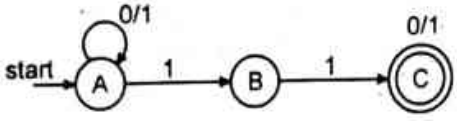
Here, state B and D have the same transition but B is non final state and D is final state. So, optimization is not possible.



Q.24. Find minimal state deterministic finite automata to accept the strings over {0, 1} which contain both 11 and 010 as substring.

CT:W-10(8M)

Ans.(1) NFA which accept string which contain 11 as substring.



Transition table for DFA :

δ	0	1
q_0 = {A}	q_0 = {A}	q_1 = {A B}
q_1 = {A B}	q_0 = {A}	q_2 = {A B C}
q_2 = {A B C}	q_3 = {A C}	q_2 = {A B C}
q_3 = {A C}	q_3 = {A C}	q_2 = {A B C}

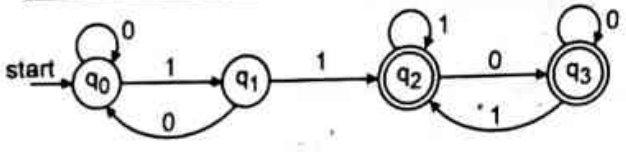


Fig. DFA

Since state { q_2 } and { q_3 } are final state they are non distinguishable state because their transition on input symbol 0/1 goes on same states. Hence we merge these two states to form single group { q_2 }. States {A} & {AB} are distinguishable.

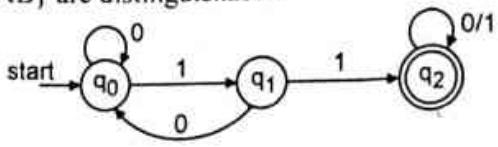
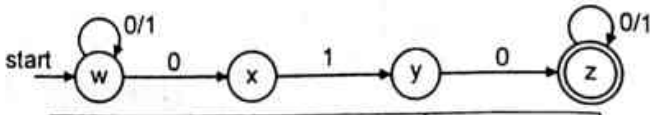


Fig. Minimum DFA

(2) NFA accepting string which contain 010 as substring.



δ	0	1
1 = {w}	2 = {w x}	1 = {w}
2 = {w, x}	2 = {w x}	3 = {w y}
3 = {w y}	4 = {w x z}	1 = {w}
4 = {w x z}*	4 = {w x z}	5 = {wyz}
5 = {wyz}*	4 = {w x z}	6 = {wz}
6 = {wz}*	4 = {w x z}	6 = {wz}

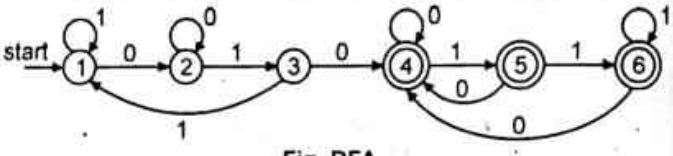


Fig. DFA

Since state {4}, {5}, {6} are non distinguishable so we merge them into single group as {4}

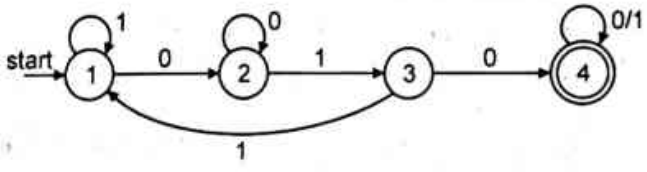


Fig. Minimum DFA

Consider the minimum DFA (1) and minimum DFA (2). From these two DFA, group of the initial state of minimum DFA (1) and initial state of minimum DFA (2) respectively form the initial state of required DFA accepting string which contain 11 and 010 substring.

Transition table for required DFA.

δ	0	1
D = {q_0 , 1}	E = {q_0, 2}	F = {q_1 , 1}
E = {q_0, 2}	E = {q_0, 2}	G = {q_1, 3}
F = {q_1 , 1}	E = {q_0, 2}	H = {q_2 , 1}
G = {q_1, 3}	I = {q_0 , 4}	H = {q_2 , 1}
H = {q_2 , 1}	J = {q_2, 2}	H = {q_2 , 1}
I = {q_0 , 4}	I = {q_0 , 4}	K = {q_1 , 4}
J = {q_2, 2}	J = {q_2, 2}	L = {q_2 , 3}
K = {q_1 , 4}	I = {q_0 , 4}	M = {q_2, 4}
L = {q_2 , 3}	M = {q_2, 4}	H = {q_2 , 1}
*M = {q_2, 4}	M = {q_2, 4}	M = {q_2, 4}

Note : Only {q_2, 4} become a final state for required DFA.

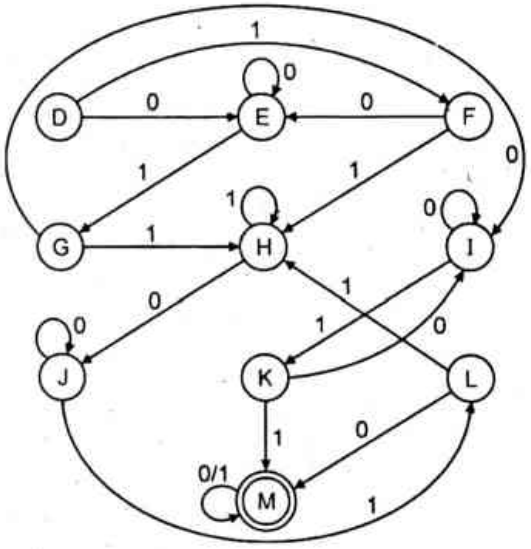


Fig. Required DFA

∈-CLOSURES

Q.25. Explain clearly what do you understand by the term "∈-closure of a state"? ***CT: S-08(3M)***

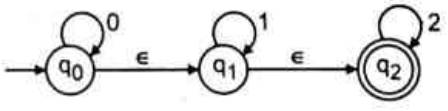
OR What do you mean by ∈closure of state? Explain with suitable example. ***CS : W-12(2M)***

OR Define '∈' closure. ***CS : W-09(2M), S-11(2M)***

OR Give the definition of ∈closure ob state. ***CT : W-10(3M)***

Ans.

- Consider NFA given by following diagram shown in fig.



- A string x in Σ* will be accepted by NFA with ∈-moves, if there exist at least one path corresponding to x, which starts in an initial state, ends in one of the final states, but since this path may be formed by ∈-transition as well as non ∈-transitions.
- To find out whether x is accepted or not by the NFA with ∈-moves, we required to define a function ∈-closure (q), where q is a state of the automata.
- The function ∈-closure (q) is defined as follows : ∈- closure (q) ⇒ set of all those states of the automata which can be reached from q on a path labeled by ∈,

  Example: In the NFA with ∈-moves given above

  ∈–closure (q_0) = {q_0, q_1, q_2}

  ∈–closure (q_1) = {q_1, q_2)

  ∈–closure (q_2) = {q_2}
- Closure is nothing but, to do some thing but considering all its reachable points.
- ∈–closure (q) will never be an empty set, because q is always reachable from itself without consuming any input symbol, i.e. on a path labeled by ∈, hence q will always be there in ∈– closure (q).
- If P is set of states, then ∈–closure function can be extended to find out ∈–closure (P) as follows : ∈–closure (P) = U for every q in P ∈–closure (q).

Q.26. Give steps for conversion of NFA with ∈ to DFA.

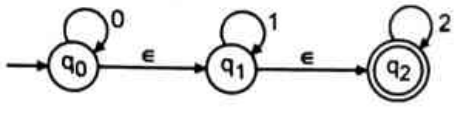
Ans. ∈ is null input and it permits state transition .

Steps:

(1) To remove ∈-transition it is necessary to add the effect of ∈ transition on the existing states.

(2) ∈-closure of a state is a state itself and all the states which are reachable using ∈-transition.

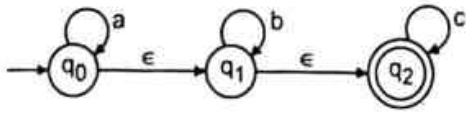
Q.27. Convert following to DFA. ***CT: S-09(6M)***



OR Give the steps required to convert a transition

system with ∈ moves into an equivalent transition system without ∈ moves obtain an equivalent atomata without ∈-moves for transition system shown.

CT:S-08(7M), W-07(8M)



Ans.

Step 1: Transition table:

State \ Σ	0	1	2	∈
→ {q0}	{q0}	–	–	{q1}
{q1}	–	{q1}	–	{q2}
*{q2}	–	–	{q2}	–

Find out ∈-closure:

From definition of ∈- closure,

$\hat{\delta}(q_0, \epsilon) = \epsilon$ - closure $(q_0) = \{q_0, q_1, q_2\}$

$\hat{\delta}(q_1, \epsilon) = \epsilon$ - closure $(q_1) = \{q_1, q_2\}$

$\hat{\delta}(q_2, \epsilon) = \epsilon$ - closure $(q_2) = \{q_2\}$

Transition of NFA without ∈-moves:

(1) $\delta'(q_0, 0) = \epsilon$ - closure $(\delta(\hat{\delta}(q_0, \epsilon), 0))$

$\delta'(q_0, 0) = \epsilon$ - closure $(\delta(q_0, q_1, q_2, 0))$

$= \epsilon$ - closure $(\delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0))$

$= \epsilon$ - closure (q_0)

$= \{q_0 q_1 q_2\}$

$\delta'(q_0, 1) = \epsilon$ -closure $(\delta(\hat{\delta}(q_0, \epsilon), 1))$

$= \epsilon$ - closure $(\delta(q_0, q_1, q_2, 1))$

$= \epsilon$ - closure $(\delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1))$

$= \epsilon$ - closure (q_1)

$= \{q_1 q_2\}$

$\delta'(q_0, 2) = \epsilon$ - closure $(\delta(\hat{\delta}(q_0, \epsilon), 2))$

$= \epsilon$ - closure $(\delta(q_0, q_1, q_2, 2))$

$= \epsilon$ - closure $(\delta(q_0, 2) \cup \delta(q_1, 2) \cup \delta(q_2, 2))$

$= \epsilon$ - closure (q_2)

$= \{q_2\}$

(2) $\delta'(q_1, 0) = \epsilon$ - closure $(\delta(\hat{\delta}(q_1, \epsilon), 0))$

$= \epsilon$ - closure $(\delta(q_1, q_2, 0))$

$= \epsilon$ - closure (ϕ)

$= \phi$

$\delta'(q_1, 1) = \epsilon$ - closure $(\delta 9\hat{\delta}(q_1, \epsilon), 1))$

$= \epsilon$ - closure $(\delta(q_1 q_2, 1))$

$= \epsilon$ - closure (q_1)

$= \{q_1 q_2\}$

$\delta'(q_1, 2) = \epsilon$ - closure $(\delta(\hat{\delta}(q_1, \epsilon), 2))$

$= \epsilon$ - closure $(\delta(q_1 q_2, 2))$

$= \epsilon$ - closure (q_2)

$= \{q_2\}$

(3) $\delta'(q_2, 0) = \epsilon$ - closure $(\delta(\hat{\delta}(q_2, \epsilon), 1))$

$= \epsilon$ - closure $(\delta(q_2, 0))$

$= \epsilon$ - closure (ϕ)

$= \phi$

$= \epsilon$ - closure $(\delta(\hat{\delta}(q_2, \epsilon), 1))$

$\delta'(q_2, 1) = \epsilon$ - closure $(\delta(q_2, 1))$

$= \epsilon$ - closure (ϕ)

$= \phi$

$\delta'(q_2, 0) = \epsilon$ closure $(\delta(\hat{\delta}(q_2, \epsilon), 0))$

$= \epsilon$ - closure $(\delta(q_2, 2))$

$= \epsilon$ - closure (q_2)

$= \{q_2\}$

Now, transition table:

State \ Σ	0	1	2
→ q0	{q0q1q2}	{q1q2}	{q2}
q1	ϕ	{q1q2}	{q2}
*q2	ϕ	ϕ	{q2}

Note: Final states are all those states whose ∈-closure contain, Final state in its transition like in step 1.

Step 1 :

$\hat{\delta}(q_0, \epsilon) = \{q_0, q_1, q_2\} \rightarrow$ Final state

$\hat{\delta}(q_1, \epsilon) = \{q_1, q_2\} \rightarrow$ Final state

Now, convert this NFA to DFA:

Step 2: DFA (Σ) = NFA (Σ)

DFA (q_0) = NFA (q_0)

Step 3: $Q' = 2^Q$, Here $Q = 3$

$\therefore Q' = 8$

$\therefore Q' = \{\phi, q_0, q_1, q_2, q_0q_1, q_0q_2, q_1q_2, q_0q_1q_2\}$

Step 4:

(1) $\delta'(q_0, 0) = (q_0 q_1 q_2)$
$\delta'(q_0, 1) = (q_1 q_2)$
$\delta'(q_0, 2) = (q_2)$
$A = q_0$

(2) $\delta'(q_0 q_1 q_2, 0) = (q_0 q_1 q_2)$
$\delta'(q_0 q_1 q_2, 1) = (q_1 q_2)$
$\delta'(q_0 q_1 q_2, 2) = (q_2)$
$*B = q_0 q_1 q_2$

(3) $\delta'(q_1 q_2, 0) = (\phi)$
$\delta'(q_1 q_2, 1) = (q_1 q_2)$
$\delta'(q_1 q_2, 2) = (q_2)$
$*C = q_1 q_2$

(4) $\delta'(q_2, 0) = \phi$
$\delta'(q_2, 1) = \phi$
$\delta'(q_2, 2) = (q_2)$
$*D = q_2$

No new state combination is left to processes so stop here.

Transition Table:

State \ Σ	0	1	2
→ A	B	C	D
* B	B	C	D
* C	-	C	D
* D	-	-	D

Here, C = B

So, after optimization

Transition Table:

State \ Σ	0	1	2
→ A	B	B	D
* B	B	B	D
* D	-	-	D

Transition diagram :

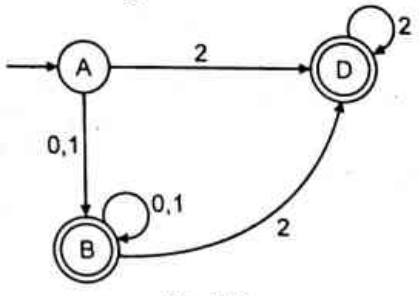


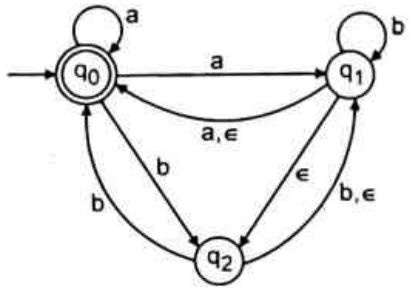
Fig. DFA

Q.28. Convert the following NFA with ∈-transition to NFA without ∈-transition.

NFA=($\{q_0, q_1, q_2\}$, {a,b}, δ, $\{q_0\}$, $\{q_0\}$) and mapping from δ is defined as follows:

State \ Σ	a	b	∈
q_0	$q_0 q_1$	q_2	ϕ
q_1	q_0	q_1	$q_0 q_2$
q_2	ϕ	$q_0 q_1$	q_1

Ans. Transition graph for NFA with ∈-moves:



Now,

∈ - closure $(q_0) = \hat{\delta}(q_0, \in) = (q_0)$

∈ - closure $(q_1) = \hat{\delta}(q_1, \in) = (q_0, q_1, q_2)$

∈ - closure $(q_2) = \hat{\delta}(q_2, \in) = (q_0 q_1 q_2)$

(1) $\delta'(q_0, a) = \in$ - closure $(\delta(\hat{\delta}(q_0, \in), a))$
$= \in$ - closure $(\delta(q_0, a))$
$= \in$ - closure $(q_0 q_1)$
$= \{q_0, q_1, q_2\}$

$\delta'(q_0, b) = \in$ - closure $(\delta(\hat{\delta}(q_0, \in), b))$
$= \in$ - closure $(\delta(q_0, b))$
$= \in$ - closure (q_2)
$= \{q_0, q_1, q_2\}$

(2) $\delta'(q_1, a) = \in$ - closure $(\delta(\hat{\delta}(q_1, \in), a))$
$= \in$ - closure $(\delta(q_0, q_1, q_2, a))$
$= \in$ - closure $(q_0 q_1)$
$= \{q_0, q_1, q_2\}$

$\delta'(q_1, b) = \in$ - closure $(\delta(\hat{\delta}(q_1, \in), b))$
$= \in$ - closure $(\delta(q_0, q_1, q_2, b))$
$= \in$ - closure $(q_0 q_1)$
$= \{q_0, q_1, q_2\}$

(3) $\delta'(q_2, a) = \in$ - closure $(\delta(\hat{\delta}(q_2, \in), a))$
$= \in$ - closure $(\delta(q_0, q_1, q_2, a))$
$= \in$ - closure $(q_0 q_1)$

$= \{q_0, q_1, q_2\}$

$\delta'(q_1, b) = \epsilon$ - closure $(\delta(\hat{\delta}(q_2, \epsilon), b))$

$= \epsilon$ - closure $(\delta(q_0, q_1, q_2, b))$

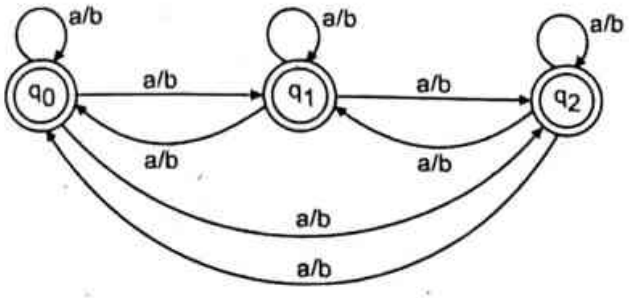
$= \epsilon$ - closure $(q_0\, q_1\, q_2)$

$= \{q_0, q_1, q_2\}$

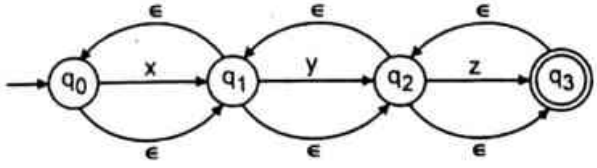
Transition Table for NFA without ϵ-moves:

State \ Σ	a	b
→ q_0	$\{q_0\, q_1\, q_2\}$	$\{q_0 q_1 q_2\}$
* q_1	$\{q_0\, q_1\, q_2\}$	$\{q_0 q_1 q_2\}$
* q_2	$\{q_0\, q_1\, q_2\}$	$\{q_0 q_1 q_2\}$

Transition graph for NFA without ϵ-moves:



Q.29. Construct a DFA equivalent to the transition diagram given in figure.



Ans. Transition Table:

State \ Σ	x	y	z	ϵ
q_0	q_1	-	-	q_1
q_1	-	q_2	-	$q_0\, q_2$
q_2	-	-	q_3	$q_1\, q_3$
q_3	-	-	-	q_2

Now, ϵ-closure:

$\hat{\delta}(q_0, \epsilon) = \epsilon$ - closure $(q_0) = \{q_0, q_1, q_2, q_3\}$

$\hat{\delta}(q_1, \epsilon) = \epsilon$ - closure $(q_1) = \{q_0, q_1, q_2, q_3\}$

$\hat{\delta}(q_2, \epsilon) = \epsilon$ - closure $(q_2) = \{q_0, q_1, q_2, q_3\}$

$\hat{\delta}(q_3, \epsilon) = \epsilon$ - closure $(q_3) = \{q_0, q_1, q_2, q_3\}$

Transition of NFA without ϵ moves :

Initial state of DFA = ϵ-closure (Initial state of NFA)

$= \epsilon$ - closure (q_0)

$= \{q_0, q_1, q_2, q_3\}$

$\therefore \delta(\{q_0, q_1, q_2, q_3\}, x) = \epsilon$ - closure $(\delta(q_0, q_1, q_2, q_3), x)$

$= \epsilon$ - closure (q_1)

$= \{q_0, q_1, q_2, q_3\}$

$\therefore \delta(\{q_0, q_1, q_2, q_3\}, y) = \epsilon$ - closure $(\delta(q_0, q_1, q_2, q_3), y)$

$= \epsilon$ - closure (q_2)

$= \{q_0, q_1, q_2, q_3\}$

$\therefore \delta(\{q_0, q_1, q_2, q_3\}, z) = \epsilon$ - closure $(\delta(q_0, q_1, q_2, q_3), z)$

$= \epsilon$ - closure (q_3)

$= \{q_0, q_1, q_2, q_3\}$

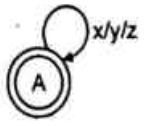
Transition table for DFA :

Q \ Σ	x	y	z
* $\{q_0, q_1, q_2, q_3\}$	$q_0\, q_1\, q_2\, q_3$	$q_0\, q_1\, q_2\, q_3$	$q_0\, q_1\, q_2\, q_3$

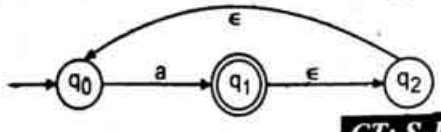
Renaming: $\{q_0\, q_1\, q_2\, q_3\} = A$

Q \ Σ	x	y	z
→*A	A	A	A

Transition graph of DFA:



Q.30. Obtain NFA without ϵ for following NFA.



CT: S-13(4M)

Ans. Transition table :

Q \ Σ	a	ϵ
→ q_0	q_1	-
* q_1	-	q_2
q_2	-	q_0

ϵ - closure of states :

ϵ - closure $(q_0) = \hat{\delta}(q_0, \epsilon) = \{q_0\}$

ϵ - closure $(q_1) = \hat{\delta}(q_1, \epsilon) = \{q_1, q_2\}$

ϵ - closure $(q_2) = \hat{\delta}(q_2, \epsilon) = \{q_0, q_1, q_2\}$

Transition of NFA without ϵ - moves :

$\delta'(q_0, a) = \epsilon$ - closure $(\delta(\hat{\delta}(q_0, \epsilon), a))$

$= \epsilon$ - closure $(\delta(q_0, a))$

$= \epsilon$ - closure (q_1)

$= \{q_1, q_2\}$

$\delta'(q_1, a) = \epsilon$ - closure $(\delta(\hat{\delta}(q_1, a))$

$= \epsilon$ - closure $(\delta(q_1, q_2)\, a)$

$= \epsilon$ - closure $(\delta(q_1, a) \cup \delta(q_2, a))$

$= \epsilon$ - closure (ϕ)

$= \phi$

$\delta'(q_2, a) = \epsilon$ - closure $(\delta(\hat{\delta}(q_2, \epsilon)\, a))$

$= \epsilon$ - closure $(\delta(q_0, q_1, q_2)\, a)$

$= \epsilon$ - closure $(\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a)$

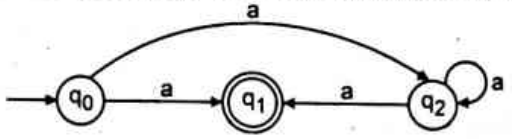
$= \epsilon$ - closure (q_1)

$= \{q_1, q_2\}$

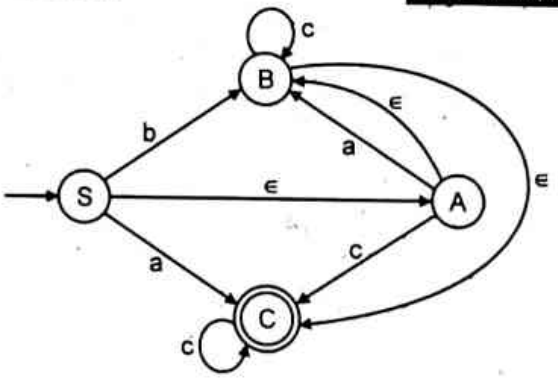
Transition table for NFA without ϵ moves:

Q \ Σ	a
→ q_0	$\{q_1, q_2\}$
* q_1	-
q_2	$\{q_1, q_2\}$

Transition graph for NFA without ϵ moves :



Q.31. Convert the following NFA with ϵ-transition to DFA. *CS : W-09(7M)*



Ans. Transition table :

Q \ Σ	a	b	c	ϵ
→ S	C	B	-	A
B	-	-	B	C
* C	-	-	C	-
A	B	-	C	B

Step 1: ϵ-closure of state:

ϵ-closure of (S) = $\hat{\delta}(S, \epsilon) = \{S, A, B, C\}$

ϵ-closure of (B) = $\hat{\delta}(B, \epsilon) = \{B, C\}$

ϵ-closure of (C) = $\hat{\delta}(C, \epsilon) = \{C\}$

ϵ-closure of (A) = $\hat{\delta}(A, \epsilon) = \{A, B, C\}$

Step 2: Transition of NFA without ϵ-moves:

(1) $\delta'(S, a) = \epsilon$ - closure $(\delta(\hat{\delta}(S, \epsilon), a)$

$= \epsilon$-closure $(\delta(S, A, B, C), a)$

$= \epsilon$-closure $(\delta(C, B))$

$= \{B, C\}$

$\delta'(S, b) = \epsilon$ - closure $(\delta(\hat{\delta}(S, \epsilon), b)$

$= \epsilon$-closure $(\delta(S, A, B, C), b)$

$= \epsilon$-closure (B)

$= \{B, C\}$

$\delta(S, c) = \epsilon$ –closure $(\delta(\hat{\delta}(S, \epsilon), c)$

$= \epsilon$-closure $(\delta(\delta, A, B, C), c)$

$= \epsilon$-closure (B, C)

$= \{B, C\}$

(2) $\delta(B, a) = \epsilon$ –closure $(\delta(\hat{\delta}(B, \epsilon), a)$

$= \epsilon$-closure $(\delta(B, C), a)$

$= \epsilon$-closure (ϕ)

$= (\phi)$

$\delta(B, b) = \epsilon$ –closure $(\delta(\delta(B, \epsilon), b)$

$= \epsilon$-closure $(\delta(B, C), b)$

$= \epsilon$-closure (ϕ)

$= \phi$

$\delta(B, c) = \epsilon$ –closure $(\delta(\hat{\delta}(B, \epsilon), b)$

$= \epsilon$-closure $(\delta(B, C), c)$

$= \epsilon$-closure (B, C)

$= \{B, C\}$

(3) $\delta(C,a) = \epsilon$ –closure $(\delta(\hat{\delta}(C,\epsilon), a)$
$= \epsilon$-closure $(\delta(C,a)$
$= \epsilon$-closure (ϕ)
$= \phi$

$\delta(C,b) = \epsilon$ –closure $(\delta(\hat{\delta}(C,\epsilon), b)$
$= \epsilon$-closure $(\delta(C, b))$
$= \epsilon$-closure (ϕ)
$= \phi$

$\delta(C,c) = \epsilon$ –closure $(\delta(\hat{\delta}(C,\epsilon), c)$
$= \epsilon$-closure $(\delta(C,c)$
$= \epsilon$-closure (C)
$= \{C\}$

(4) $\delta(A,a) = \epsilon$ –closure $(\delta(\hat{\delta}(A,\epsilon), a)$
$= \epsilon$-closure $(\delta(A, B, C), a)$
$= \epsilon$-closure (B)
$= \{B, C\}$

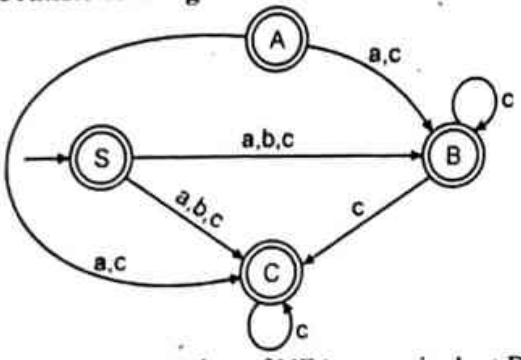
$\delta(A,b) = \epsilon$ –closure $(\delta(\hat{\delta}(A,\epsilon), b)$
$= \epsilon$-closure $(\delta(A,B,C), b)$
$= \phi$

$\delta(A,c) = \epsilon$ –closure $(\delta(\hat{\delta}(A,\epsilon), c)$
$= \epsilon$-closure $(\delta(A, B, C), c)$
$= \epsilon$-closure (B, C)
$= \{B, C\}$

Transition table:

Q \ Σ	a	b	c
→ * S	{B, C}	{B, C}	{B, C}
* A	{B, C}	-	{B, C}
* B	ϕ	ϕ	{B, C}
* C	ϕ	ϕ	{C}

Transition Diagram:



Now, conversion of NFA to equivalent DFA :

Step 1 : DFA (Σ) = NFA (Σ)
DFA(s) = NFA(s)

Step 2 : $Q' = 2^Q$, Here Q = 4
$Q' = 16$

Step 3 :

(1) $\left.\begin{array}{l}\delta'(S,a) = (BC)\\ \delta'(S,b) = (BC)\\ \delta'(S,c) = (BC)\end{array}\right\}$ * $q_0 = S$

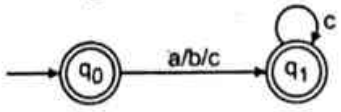
(2) $\left.\begin{array}{l}\delta'(BC,a) = \phi\\ \delta'(BC,b) = \phi\\ \delta'(BC,c) = (BC)\end{array}\right\}$ * $q_1 = BC$

Step 4 :

∴ **Transition table :**

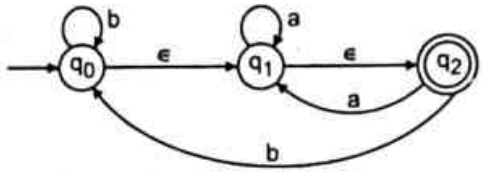
Q \ Σ	a	b	c
→ * q_0	q_1	q_1	q_1
* q_1	-		q_1

Transition diagram :



Q.32. Find ϵ-closure of all the states of given NFA. Remove epsilon without changing the acceptance. Also explain the method.

CS : S-11(8M)



Ans. Transition table:

Q \ Σ	a	b	ϵ
→ q_0	-	$\{q_0\}$	$\{q_1\}$
q_1	$\{q_1\}$	-	q_2
* q_2	$\{q_1\}$	$\{q_0\}$	-

Step 1: ϵ - Closure of states:

ϵ- Closure $\{q_0\} = \hat{\delta}(q_0, \epsilon) = \{q_0, q_1, q_2\}$

ϵ- Closure $\{q_1\} = \hat{\delta}(q_1, \epsilon) = \{q_1, q_2\}$

ϵ- Closure $\{q_2\} = \hat{\delta}(q_2, \epsilon) = \{q_2\}$.

Step 2: Transitions of NFA without ϵ - moves :

(1) $\delta'(q_0, a) = \epsilon$ - closure $(\hat{\delta}(q_0, \epsilon), a))$

$= \epsilon$ - closure $(\delta(q_0, q_1, q_2), a)$

$= \epsilon$ - closure $(\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a))$

$= \epsilon$ - closure (q_1)

$= \{q_1, q_2\}$

$\delta'(q_0, b) = \epsilon$ - closure $\delta((\hat{\delta}(q_0, \epsilon), b))$

$= \epsilon$ - closure $(\delta(q_0, q_1, q_2), b)$

$= \epsilon$ - closure (q_0)

$= \{q_0, q_1, q_2\}$

(2) $\delta'(q_1, a) = \epsilon$ - closure $\delta((\hat{\delta}(q_1, \epsilon), a))$

$= \epsilon$ - closure $(\delta(q_1, q_2), a)$

$= \epsilon$ - clsoure (q_1)

$= \{q_1, q_2\}$

$\delta(q_1, b) = \epsilon$ - closure $(\delta(\hat{\delta}(q_1, \epsilon), b))$

$= \epsilon$ - closure $(\delta(q_1, q_2), b)$

$= \epsilon$ - closure (q_0)

$= \{q_0, q_1, q_2\}$

(3) $\delta(q_2, a) = \epsilon$ - closure $\delta(\hat{\delta}(q_2, \epsilon), a)$

$= \epsilon$ - closure $(\delta(q_2, a))$

$= \epsilon$ - closure (q_1)

$= \{q_1, q_2\}$

$\delta(q_2, b) = \epsilon$ - closure $\delta(\hat{\delta}(q_2, \epsilon), b)$

$= \epsilon$ - closure $(\delta(q_2, b))$

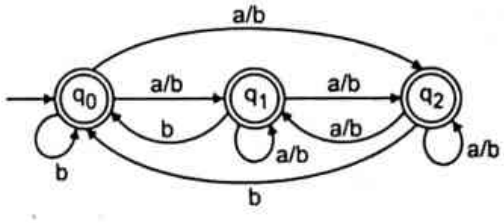
$= \epsilon$ - closure (q_0)

$= \{q_0, q_1, q_2\}$

Step 3: Transition table:

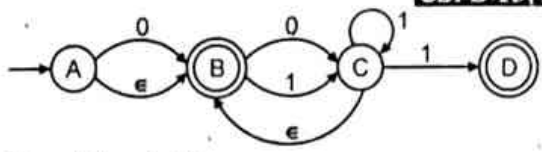
Q \ Σ	a	b
→ *q_0	$\{q_1\ q_2\}$	$\{q_0\ q_1\ q_2\}$
*q_1	$\{q_1\ q_2\}$	$\{q_0\ q_1\ q_2\}$
*q_2	$\{q_1\ q_2\}$	$\{q_0, q_1, q_2\}$

Step 4: Transition diagram:



Q.33. Get NFA without ϵ - transition for following.

CS: S-13(4M)



Ans. Transition table:

Q \ Σ	0	1	ϵ
→ A	B	ϕ	B
* B	C	C	ϕ
C	ϕ	C, D	B
* D	ϕ	ϕ	ϕ

Step 1 : ϵ-Closure of states :

ϵ-Closure (A) = δ (A, ϵ) = {A, B}

ϵ-Closure (B) = δ (B, ϵ) = {B}

ϵ-Closure (C) = δ (C, ϵ) = {B, C}

ϵ-Closure (D) = δ (D, ϵ) = {D}

Step 2 : Transitions of NFA without ϵ - moves:

(1) $\delta(A, 0) = \epsilon$ - closure $(\delta(\hat{\delta}(A, \epsilon), 0)$

$= \epsilon$ - closure $(\delta(A, B), 0)$

$= \epsilon$ - closure(B, C)

$= \{B, C\}$

$\delta(A, 1) = \epsilon$ - closure $\delta((\hat{\delta}(A, \epsilon), 1))$

$= \epsilon$ - closure $\{\delta(A, B), 1)$

$= \epsilon$ - closure(C)

$= \{B, C\}$

(2) $\delta(B, 0) = \epsilon$ - closure $(\delta(\delta(B, \epsilon), 0))$

$= \epsilon$ - closure $\{\delta(B, 0))$

$= \{B, C\}$

$\delta'(B, 1) = \epsilon$ - closure $(\delta(\{\delta(B, \epsilon), 1)$

$= \epsilon$ - closure $(\delta(B, 1))$

$= \{B, C\}$

(3) $\delta'(C, 0) = \epsilon$ - closure $(\delta\{\delta(C, \epsilon), 0))$

$= \epsilon$ - closure $\{\delta(B, C), 0)$

$= \{B, C\}$

$\delta'(C, 1) = \epsilon$ - closure $(\delta(\delta(C, \epsilon), 1)$

$= \epsilon$ - closure $\{\delta(B, C)\ 1)$

$= \epsilon$ - closure (C, D)

$= \{B, C, D\}$

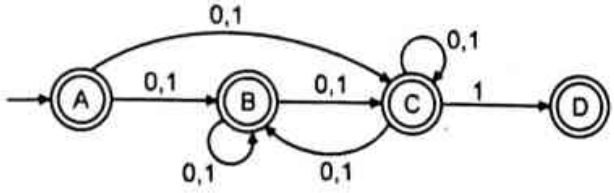
(4) $\delta'(D, 0) = \epsilon$ - closure $(\delta(\hat{\delta}(D, \epsilon), 0))$
$= \epsilon$ - closure $(\delta(D, 0))$
$= \phi$

$\delta'(D, 1) = \epsilon$ - closure $(\delta(\hat{\delta}(D, \epsilon), 1))$
$= \epsilon$ - closure $(\delta(D, 1)) = \phi$

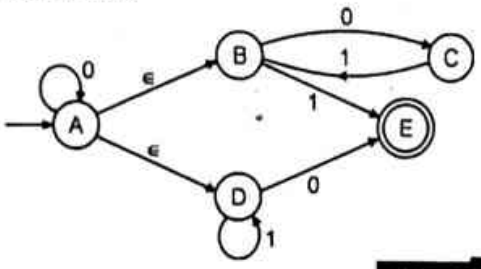
Step 3 : Transition table:

Q \ Σ	0	1
→ *A	B, C	B, C
*B	B, C	B, C
*C	B, C	B, C, D
*D	ϕ	ϕ

Step 4 : Transition diagram:



Q.34. Construct minimized DFA from the NFA given below.



CT : S-12(6M)

Ans. Transition table :

State \ Σ	0	1	ϵ
→ A	A	–	BD
B	C	E	–
C	–	B	–
D	E	D	–
*E	–	–	–

Now, ϵ closure :

$\hat{\delta}(A, \epsilon) = \epsilon$ - closure $(A) = \{A\ B\ D\}$

$\hat{\delta}(B, \epsilon) = \epsilon$ - closure $(B) = \{B\}$

$\hat{\delta}(C, \epsilon) = \epsilon$ - closure $(C) = \{C\}$

$\hat{\delta}(D, \epsilon) = \epsilon$ - closure $(D) = \{D\}$

$\hat{\delta}(E, \epsilon) = \epsilon$ - closure $(E) = \{E\}$

Transition of NFA without ϵ moves :

(i) Initial state of NFA = ϵ - closure $(\delta(\hat{\delta}(A, \epsilon), 0))$
$= \epsilon$ - closure$(\delta(ABD), 0)$
$= \epsilon$ - closure(AEC)
$= \{ABCDE\}$
$\delta'(A, 1) = \epsilon$ - closure $(\delta(\hat{\delta}(A, \epsilon), 1)$
$= \epsilon$ - closure $(\delta(ABD), 1)$
$= \epsilon$ - closure(ED)
$= \{E, D\}$

(ii) $\delta'(B, 0) = \epsilon$ - closure $(\delta(\hat{\delta}(B, \epsilon), 0))$
$= \epsilon$ - closure $(\delta(B, 0))$
$= \epsilon$ - closure(C)
$= \{C\}$
$\delta'(B, 1) = \epsilon$ - closure $(\delta(\hat{\delta}(B, \epsilon), 1))$
$= \epsilon$ - closure $(\delta(B), 1)$
$= \epsilon$ - closure(E)
$= \{E\}$

(iii) $\delta'(C, 0) = \epsilon$ - closure $(\delta(\hat{\delta}(C, \epsilon), 0))$
$= \epsilon$ - closure $(\delta(C), 0)$
$= \epsilon$ - closure(ϕ)
$= \{\phi\}$
$\delta'(C, 1) = \epsilon$ - closure $(\delta(\hat{\delta}(C, \epsilon), 1))$
$= \epsilon$ - closure $(\delta(C), 1)$
$= \epsilon$ - closure(B)
$= \{B\}$

(iv) $\delta'(D, 0) = \epsilon$ - closure $(\delta(\hat{\delta}(D, \epsilon), 0))$
$= \epsilon$ - closure $(\delta(D), 0))$
$= \epsilon$ - closure(E)
$= \{E\}$
$\delta'(D, 1) = \epsilon$ - closure $((\delta(\hat{\delta}(D, \epsilon), 1))$
$= \epsilon$ - closure$(\delta(D), 1)$
$= \epsilon$ - closure(D)
$= \{D\}$

(v) $\delta'(E, 0) = \epsilon$ - closure $\delta(\hat{\delta}(E, \epsilon), 0))$
$= \epsilon$ - closure $(\delta(E, 0)$
$= \epsilon$ - closure (ϕ)
$= \{\phi\}$
$\delta'(E, 1) = \epsilon$ - closure $((\delta(\hat{\delta}(E, \epsilon), 1))$
$= \epsilon$ - closure $(\delta(E), 1)$

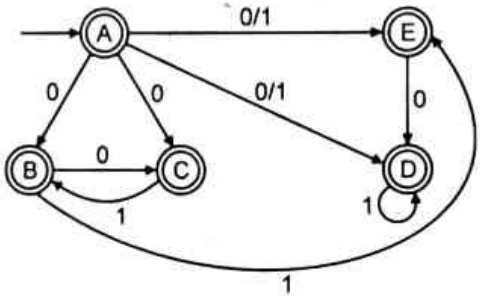
$= \epsilon$ - closure(ϕ)

$= \{\phi\}$

Transition table :

Q \ Σ	0	1
→ A	{ABCDE}	{ED}
*B	{C}	{E}
C	ϕ	{B}
*D	{E}	{D}
*E	$\{\phi\}$	$\{\phi\}$

Transition diagram :



Conversion of NFA to DFA :

Step 1 : DFA (Σ) = NFA (Σ)

DFA (A) = NFA (A)

Step 2 : $Q' = 2^Q$ Here Q = 4

$\therefore Q' = 16$

Step 3 :

(i) $\left.\begin{matrix}\delta'(A,0) = (ABCDE) \\ \delta'(A,1) = (ED)\end{matrix}\right\} *q_0 = A$

(ii) $\left.\begin{matrix}\delta'(ED) = (E) \\ \delta'(ED,1) = (D)\end{matrix}\right\} *q_1 = ED$

(iii) $\left.\begin{matrix}\delta'(E, D) = (\phi) \\ \delta'(E,1) = (\phi)\end{matrix}\right\} *q_2 = E$

(iv) $\left.\begin{matrix}\delta'(D,0) = (E) \\ \delta'(D,1) = (D)\end{matrix}\right\} *q_3 = D$

(v) $\left.\begin{matrix}\delta'(ABCDE,0) = (ABCDE) \\ \delta'(ABCDE,1) = (BDE)\end{matrix}\right\} ABCDE = *q_4$

(vi) $\left.\begin{matrix}\delta'(BDE) = (CE) \\ \delta'(BDE,1) = (ED)\end{matrix}\right\} *q_5 = BDE$

(vii) $\left.\begin{matrix}\delta'(CE,0) = (\phi) \\ \delta'(CE,1) = (B)\end{matrix}\right\} *q_6 = CE$

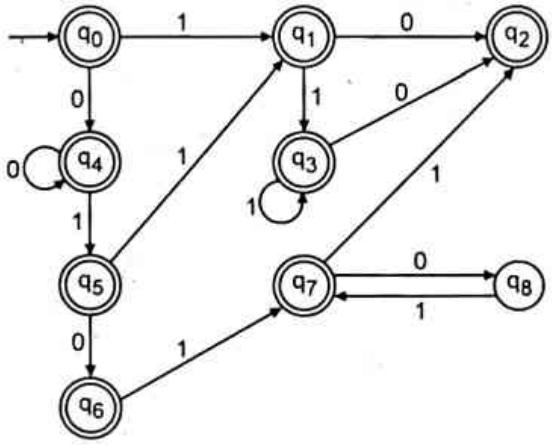
(viii) $\left.\begin{matrix}\delta'(B, 0) = (C) \\ \delta'(B,1) = (E)\end{matrix}\right\} *q_7 = B$

(ix) $\left.\begin{matrix}\delta'(C, 0) = (\phi) \\ \delta'(C,1) = (B)\end{matrix}\right\} q_8 = C$

Step 4 : Transition table :

Q \ Σ	0	1
→ *q_0	q_4	q_1
*q_1	q_2	q_3
*q_2	ϕ	ϕ
*q_3	q_2	q_3
*q_4	q_4	q_5
*q_5	q_6	q_1
*q_6	ϕ	q_7
*q_7	q_8	q_2
q_8	ϕ	q_7

Transition diagram :



EQUIVALENCE BETWEEN NFA AND DFA

Q.35. Explain equivalence between NFA and DFA.
OR "Every NFA has its equivalent DFA but vice versa is not true". Comment on the statement.

Ans. Equivalence between NFA and DFA :

- For every NFA, there exists an equivalent finite automata and the equivalence is defined in terms of language acceptance.
- Since every DFA is in NFA, it is clear that the class of languages accepted by NFAs includes

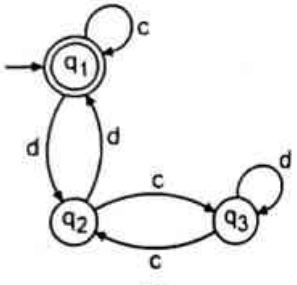
regular sets that is the languages accepted by DFA also.

- NFA is nothing but finite automata in which zero, one or more transitions on single input symbol is permitted. Therefore if we construct a finite automata which will simulate all the moves of NFA on a particular input symbol in parallel then we get a finite automata in which there will be exactly one transition on input symbol and hence it will be deterministic finite automata (DFA). This show that DFA can simulated by NFA.
- That is for every NFA we can construct an equivalent DFA one which accepts the same language.
- The way of DFA simulates as NFA is to allow the states of DFA to correspond to sets of state of NFA.
- The constructed DFA keeps track in its finite control of all states that the NFA could be in after reading the same input as the DFA has read.

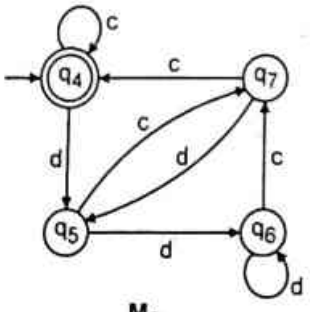
The transformation of NFA to DFA is called as subset construction.

Q.36. Are automata M_1 and M_2 defined by following transition diagram equivalent.

CT: W-11(6M)



M_1



M_2

Ans. Transition Table:

M_1

Q \ Σ	c	d
* q_1	q_1	q_2
q_2	q_3	q_1
q_3	q_2	q_3

M_2

Q \ Σ	c	d
→ * q_4	q_4	q_5
q_5	q_7	q_6
q_6	q_7	q_6
q_7	q_4	q_5

Transition table for equivalency:

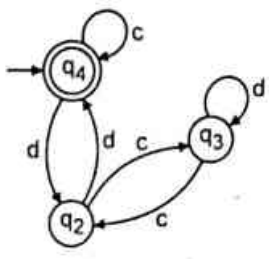
Q \ Σ	c	d
→ * $\{q_1, q_4\}$	$\{q_1, q_4\}$	$\{q_2, q_5\}$
$\{q_2, q_5\}$	$\{q_3, q_7\}$	$\{q_1, q_6\}$

Here pair consist of final q_1 and Non-final q_6. Hence given FA's are Not equivalent.

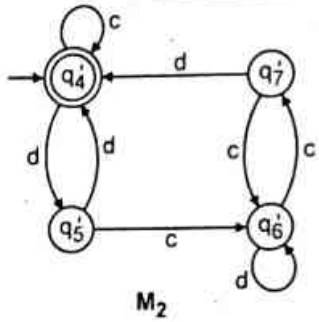
Note: Two FA's are equivalent only when their pair of transition are consist of either. Final state or Non-Final state.

Q.37. Show that automata M_1 and M_2 are equivalent or not.

CT: W-08(4M)



M_1



M_2

Ans. Transition Table:

M_1

Q \ Σ	c	d
* q_1	q_1	q_2
q_2	q_3	q_1
q_3	q_2	q_3

M_2

Q \ Σ	c	d
→ * q_4'	q_4'	q_5'
q_5'	q_6'	q_4'
q_6'	q_7'	q_6'
q_7'	q_6'	q_4'

Transition table for equivalency:

Q \ Σ	c	d
→ * $\{q_1, q_4'\}$	$\{q_1, q_4'\}$	$\{q_2, q_5'\}$
$\{q_2, q_5'\}$	$\{q_3, q_6'\}$	$\{q_1, q_4'\}$
$\{q_3, q_6'\}$	$\{q_2, q_7'\}$	$\{q_3, q_6'\}$
$\{q_2, q_7'\}$	$\{q_3, q_6'\}$	$\{q_1, q_4'\}$

Here every pair transition consist of either. Final or Non Final set. Hence FA's are equivalent.

Q.38. Given an NFA without ∈- moves, what is the maximum number its corresponding DFA can have ? Why?

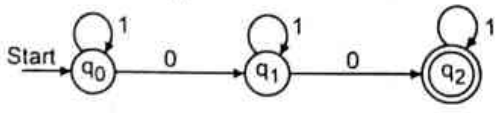
OR Comment on statement of "NFA without ∈-moves contain N number of states then its equivalent DFA contains maximum of 2^N states".

Ans.

- Since the DFA equivalent of NFA is to simulate the moves of NFA in parallel every stable of a DFA will be a combination of one or more states of NFA, hence every state of a DFA will be represented by some subset of set of states of NFA and therefore the transformation of NFA to DFA is normally called as subset construction.
- Therefore if a given NFA has n states, then the number of states of the equivalent DFA will be 2n, with initial state corresponding to the subset {q0}.

Q.39. Design a finite state machine which generates exactly two zeros anywhere in string.

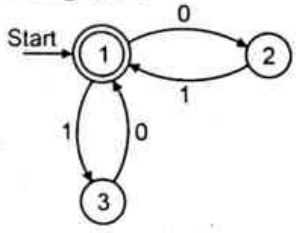
Ans. Transition diagram :



Q.40. Draw transition diagram to represent the finite automata for following language.

Accepting those strings of Zero's and one's which contains equal no. of Zero's and one's and no prefix of the string contains two more zeros than ones, more it should contain no more one's than Zero's.

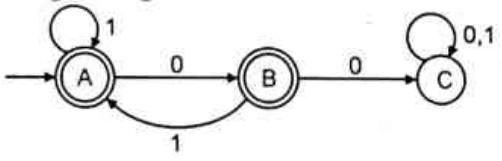
Ans. Transition diagram :



Q.41. Construct a FA accepting all string over {0, 1} ending in 010 or 0010.

CT: W-09(2M),W-11(5M)

Ans. String ending 010:



Q.42. Design DFA to check whether given binary number is divisible by '5'. **CT: S-12(7M)**

OR Design DFA accepting the set of all string that when interpreted in reverse as a binary integer is divisible by 5.

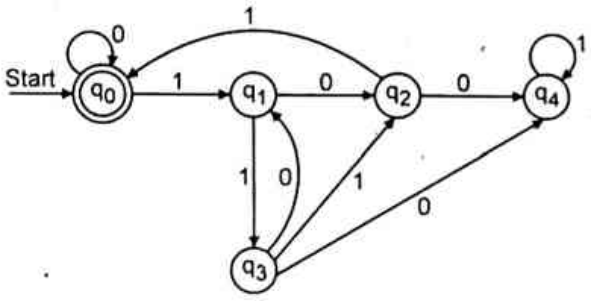
Examples : 10011, 0101. **CS: S-13(3M)**

Ans.

Q \ Σ	0	1
→*q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_4	q_0
q_3	q_1	q_2
q_4	q_3	q_4

Here, $\delta = \{q_0, q_1, q_2, q_3, q_4\}$ which are depends on remainder like for '5' remainder is 0, 1, 2, 3, 4.

Transition diagram:



Q.43. Construct a finite automata for accepting those strings of 0's and 1's in which number of 0's is even number and number of 1's is odd.

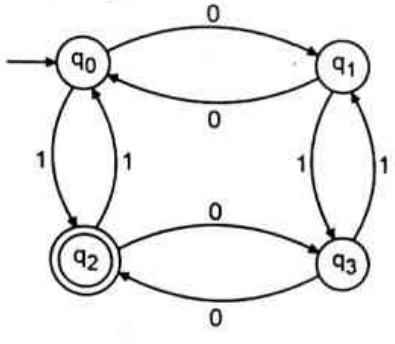
CS: S-12(2M)

OR Construct DFA that recognize the language

$L = \{\omega \in \{0,1\}^* \mid |\omega_0| = \text{even and } |\omega_1| = \text{odd}\}$

CT: S-10(6M)

Ans. Transition diagram :



Q.44. Design FA which accepts Binary digits which when interpreted as digits as integer are divisible by 3.

Ans. Consider, states as = $\{s_0, s_1, s_2\}$ and remainder are output for the machine.

In this case formula for solving given equation is 2 × odd remainder + next bit.

For initial state q_0 :

(i) $2 \times 0 + 0 = 0$ = Remainder is zero that is why output is = q_0.

(ii) $2 \times 0 + 1 = 1 = q_1$

For next state q_1 :

(i) $2 \times 1 + 0 = 2 = q_2$

(ii) $2 \times 1 + 1 = 3$, But 3 is divisible by 3 hence o/p will be '0' = q_0

For last state q_2 :

(i) $2 \times 2 + 0 = 4$

Here remainder is 1 i.e. q_1 state.

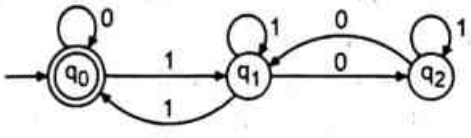
(ii) $2 \times 2 + 1 = 5$

Here remainder is 2 i.e. q_2 state.

Transition Table:

Σ / Q	0	1
→* q_0	q_0	q_1
q_1	q_2	q_0
q_2	q_1	q_2

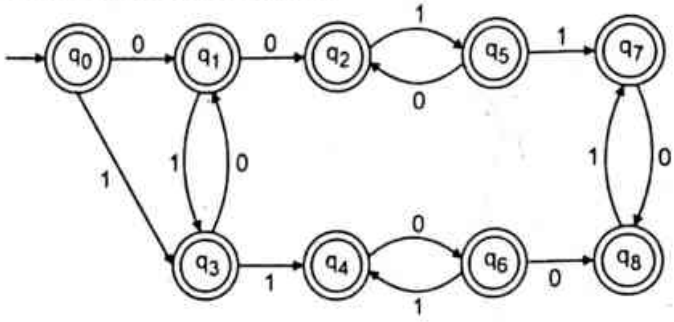
Transition diagram :



Q.45. Construct a finite automata which accepts those strings of 0's and 1's which contains at most one pair of 0's and at most pair of 1's.

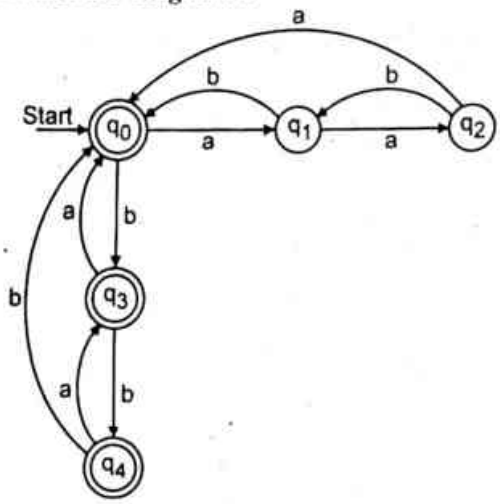
OR A string may contain one pair of 1's. String may contain one pair of 0's. Strings may contain one pair of 1's and 0's.

Ans. Transition diagram :



Q.46. Construct a finite automata for accepting those strings of a's & b's such that difference between strings of a's & strings of b's is multiple of 3. ***CT: W-09(3M), W-12(6M)***

Ans. Transition diagram :



Q.47. Design a finite state machine to check whether a given number is divisible by 3 and by 4.

CS: W-09(5M), CT: W-10(4M)

OR Design a finite state machine which determines the residue mod 3 for each binary string treated as binary integer. ***CT: W-06(5M)***

Ans. (i) Transition diagram for divisible by 3.

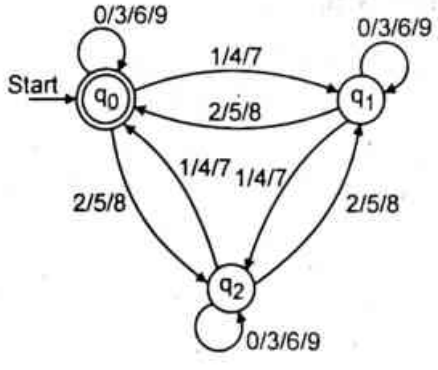


Fig. FSM Divisible by 3

(ii) Transition diagram for divisible by 4.

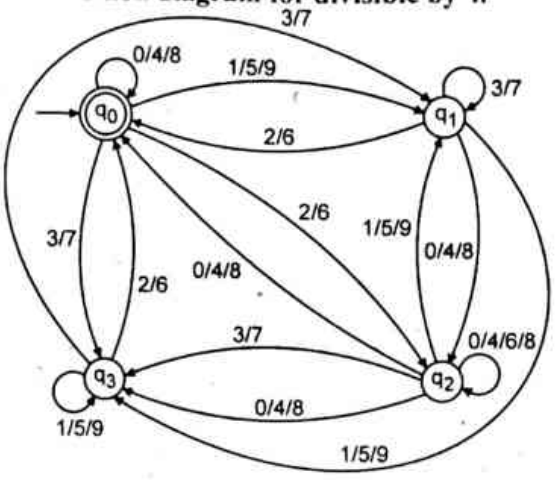


Fig. FSM Divisible by 4

Q.48. Construct a DFA for the language
L = {w / w ∈ (a + b) *}
where, n_a (w) mod 3 > n_b(b) mod 3

Ans. Transition diagram :

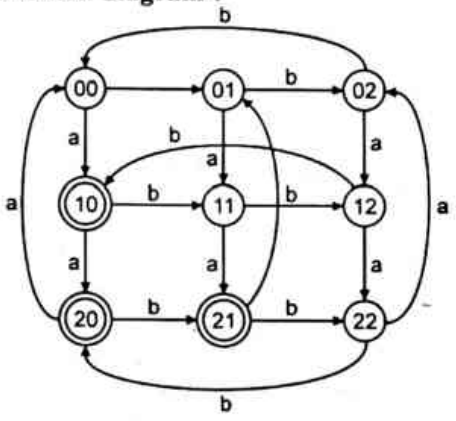


Fig. DFA

Q.49. Construct DFA for the language L {w / w ∈ (a+b)* and n_a (w) mod 3 > n_b (w) mod 2, where n_a (w) indicates number of as in w and n_b (w) indicates number of bs in w.

Ans. Transition diagram :

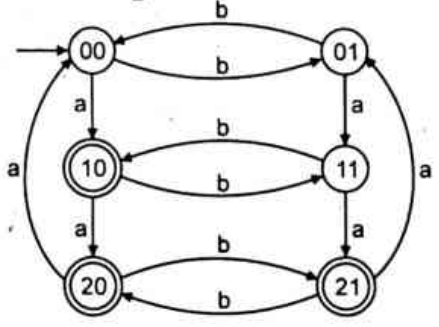


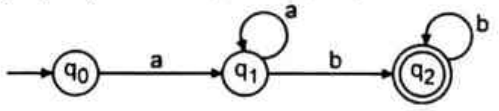
Fig. DFA

Q.50. Construct the transition system (DFA) for the following language. **CT: S-07(4M)**

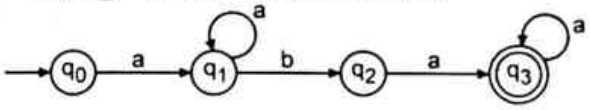
(i) L_1 = {$a^m b^n$: m, n are + ve}

(ii) L_2 = {$a^m ba^n$: m, n are + ve}

Ans. (i) L_1 = {$a^m b^n$: m, n are + ve}



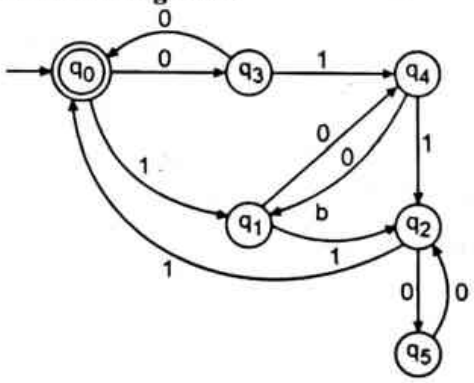
(ii) L_2 = {$a^m ba^n$: m, n are + ve}



Q.51. Construct finite automata which accepts all those binary strings in which the number of 1's and 0's are respectively divisible by 3 and 2. **CT:S-07(5M)**

Ans. The DFA accepts binary strings in which the number of 1's and 0's are divisible by 3 and 2

Transition diagram :



MINIMIZATION OF AUTOMATA

Q.52. Write the procedure for optimization or minimization for DFA. Apply procedure by taking an example for DFA.

Ans. Minimization/ optimization of DFA :

Minimization / optimization of a deterministic finite automata refers to detecting those states of a DFA whose presence or absence in a DFA does not affect the language accepted by the automata, hence these states can be eliminated from the automata without affecting the language accepted by the automata. Such states are:

(i) Unreachable states :

The states of a DFA which are not reachable from

the initial state of DFA on any possible input sequence are unreachable states.

(ii) **Dead states :**

A dead state is that non final state of a DFA whose transitions on every input symbol terminates on itself i.e q is a dead state if q is in Final state and $\delta(q.a) = q$ for every a in Σ.

(iii) **Non distinguishable states:**

These are those states of DFA for which there exist no distinguishing string hence they can not be distinguished from one another.

Therefore optimization refers to :

(1) Detection of unreachable states and eliminating them from DFA.

(2) Identification of non distinguishing states and merging them together.

(3) Detecting dead states and eliminating them from DFA.

Algorithm to detect unreachable states :

Input : $M = (Q, \Sigma, \delta, q_0, F)$

Output : Set $\cup$(which is set of unreachable states).

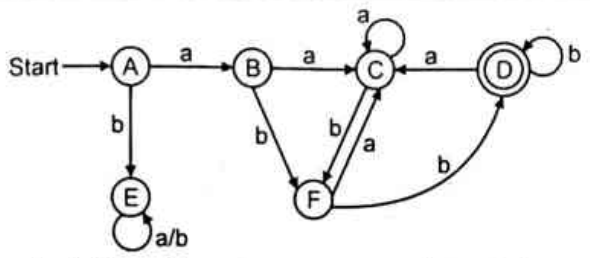
- Let R be the set of reachable states of DFA we take two R's, R_{new} and R_{old}. So that we will be able to perform iteration in the process of detecting unreachable states.

```
Begin
R_old = ϕ
R_new = {q0}
while (R_old ≠ R_new) do
Begin
    temp1 = R_new − R_old
    R_old = R_new
    temp2 = ϕ
    for every a in Σ do
    temp2 = temp2 ∪ δ (temp1.a)
    R_new = R_new ∪ temp2
end
U = Q − R_new
end
```

- If p and q are the two states of a DFA then p and q are said to be distinguishable states if there exist a distinguishing string w which distinguishes p and q.
- A string w is a distinguishing string for states p and q, if transitions from p on w goes to a non final state whereas from q on w goes to a final state or vice versa.
- Therefore to find non distinguishable states of a DFA it is required to find out whether there exist some distinguishing string w, which distinguishes the states, if no such string exists then the states are non distinguishable and can be merged together.
- The technique that we use to find non distinguishable state is the method of successive partitioning in which we start with two groups / partitions, one containing all non final states and other containing all the final state this is because every final state is know to be distinguishable from a non final state, then we find transitions from members of a partition on every input symbol.
- If on a particular input symbol, we find that transitions from some of the members of a partition goes to one place whereas transitions from other members of a partition goes to some other place, then we conclude that the members whose transitions goes to one place are distinguishable from members whose transitions goes to other place.
- Therefore we divide the partition into two parts, we continue this partitioning till we get partitions which can not be partitioned further this happens when, either a partition contains only one state, or a partition contains more than one state, but they are not distinguishable from one another, if we get such a partitions we merge all the states of this partition into a single state.

Example:

Consider the following transition diagram :



- Initially we have two groups as shown below :

A.B.C.E.F	D
Group 1	Group 2

Since $\delta(A, a) = B$

$\delta(B, a) = C$

$\delta(C, a) = C$

$\delta(E, a) = E$

$\delta(F, a) = C$

Partitioning of group 1 is not possible, because the transitions from all the members of group 1 goes to group 1 only, but since

$\delta(A, b) = E$

$\delta(B, b) = F$

$\delta(C, b) = F$

$\delta(E, b) = E$

$\delta(F, b) = D$

State F is distinguishable from rest of the members of group 1. Hence we divide group 1 into two groups one containing A, B, C, E and other containing F as shown below.

- Now, consider three group as shown below :

A.B.C.E	F	D
Group 1	Group 2	Group 3

Since $\delta(A, a) = B$

$\delta(B, a) = C$

$\delta(C, a) = C$

$\delta(E, a) = E$

Partitioning of group 1 is not possible, because the transitions from all the members of group 1 goes to group 1 only, but since

$\delta(A, b)$ E

$\delta(B, b) = F$

$\delta(C, b) = F$

$\delta(E, b) = E$

States A and E are distinguishable from states B and C. Hence we divide group I into two groups one containing A,E and other containing B, C as shown below.

- Now consider, four groups shown below.

A, E	B. C	F	D
Group 1	Group 2	Group 3	Group 4

Since $\delta(A, a) = B$

$\delta(E, a) = E$

State A is distinguishable from state E. Hence we divide group I into two groups one containing A and other containing E as shown below.

- Now consider groups shown below.

A.	E	B.C	F	D
Group 1	Group 2	Group 3	Group 4	Group 5

Since $\delta(B, a) = C$

$\delta(C, a) = C$

Partitioning of group 3 is not possible, because the transitions from all the members of group 3 on a , goes to group 3 only, similarly

$\delta(B, b) = F$

$\delta(C, b) = F$

Partitioning of group III is not possible, because the transitions from all the members of group III on b, also goes to group IV only.

Hence B and C are nondistinguishable states, therefore we merge B and C to form a single state B_1 as shown in figure below.

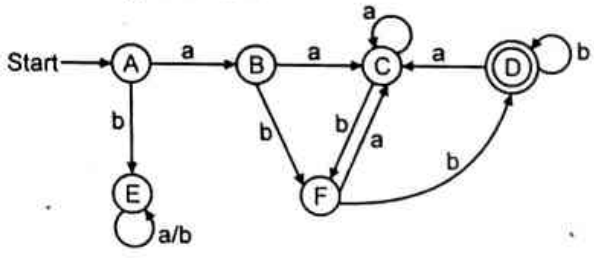


Fig. Transition diagram

Algorithm for detection of dead states

Input : $M = (Q, \Sigma, \delta, q_0, F)$.

Output : Set X (which is set of dead states).

```
Start :
X = ϕ
for every q in (Q – F) do
Begin :
Set  flag = true
     for every a in Σ do
     if (δ (q, a) ≠ q) then
          flag = false
          break
          if flag = true then
X = X ⋃ {q}
END.
END.
```

Q.53. Write an algorithm for optimisation of DFA using pair table method. Apply algorithm by taking an example DFA.

Ans. Marking Algorithm : Marking algorithm is used to optimize DFA.

Step 1 : Marking feasible state pair:

- Pair of states for which identical transition exists are feasible state pair i.e. (p, q) is set to feasible if δ (p, Σ) is not null then δ (q, Σ) should again be not null & if δ (p, Σ) is null then δ (q, Σ) should again be null.
- In short identical transition on input character should be there.
- Feasible state can be from nonfinal - nonfinal or final - final group.

Step 2 : Get feasible state pair table:

- Columns of the table will be Σ and rows will be feasible state pair.
- Fill up the table with feasible state pair table by getting individual transition.

Step 3 : Marking of feasible states:

- Marking algorithms will mark only those states which cannot be merge and therefore at the end of marking algorithms all unmarked state will be equivalent states.
- Initially all feasible state are unmarked pair. These marking is to be repeated till no further marking is possible.
- Row of feasible state pair will be marked if entry for any i/p character satisfy following conditions.

(i) It is not among feasible state pair.

(ii) It is already marked.

Example : Take the given DFA as and Apply marking algorithm.

δ'	a	b	c
A	B	E	F
B	C	-	H
C	A	-	H
D	F	-	E
*E	E	F	G
F	D	-	E
G	B	H	D
*H	H	D	A

Step 1 : Getting feasible state pair :

Final -Final : (E, H)

Non final : Non final - {(A, G), (B, C), (B, D), (B, F) , (C, D), (C, F), (D, F)}

Step 2 : Feasible state pair table :

The feasible state pair table is :

δ	a	b	c
EH	EH	FD	GA
AG	BB	EH	FD
BC	CA	-	HH
BD	CF	-	HE
BF	CD	-	HE
CD	AF	-	HE
CF	AD	-	HE
DF	FD	-	EE

Step 3 : Marking of feasible states :

Now, mark the states which cannot be merge

δ	a	b	c
EH	EH	FD	GA
AG	BB	EH	FD
√BC	CA	-	HH

√BD	CF	-	HE
√BF	CD	-	HE
√CD	AF	-	HE
√CF	AD	-	HE
DF	FD	-	EE

Since EH, AG & DF are unmarked state therefore, E = H, A = G, D = F. Optimized DFA will be,

δ	a	b	c
AG	B	EH	DF
B	C	-	EH
X	AG	-	EH
DF	DF	-	EH
*EH	EH	DF	AG

Q.54. FA is defined by { a, b, c, d, e, f, g, h } , {0, 1 } , δ, {a} , {c}) where δ is

δ	0	1
a	b	f
b	g	c
c	a	c
d	c	g
e	h	f
f	c	g
g	g	e
h	g	c

Optimize it by pair-table method only. *CT: S-13(7M)*

Ans. Applying marking algorithm on given DFA.

Step 1 : Feasible state pair are:

{(a,b), (a, d), (a,e), (a, f), (a, g), (a, h), (b, d), (b, e), (b, f), (b, g), (b, h), (d, e), (d, f), (d, g), (d, h), (e, f), (e, g), (e, h), (f, g), (f, h) (g, h)}

Step 2 : Feasible state pair table is :

δ	0	1
(ab)	bg	fc
(ad)	bc	fg
ae	bh	ff
(af)	bc	fg
(ag)	bg	fe
(ah)	bg	fc
(bd)	gc	cg
(be)	gh	cf
(bf)	gc	cg
(bg)	gg	ce
bh	gg	cc
(dc)	ch	gf
df	cc	gg
(dg)	cg	ge
(dh)	cg	gc
(ef)	hc	fg
(eg)	hg	fe
(eh)	hg	fc
(fg)	cg	ge
(dh)	cg	gc
(gh)	gg	ec

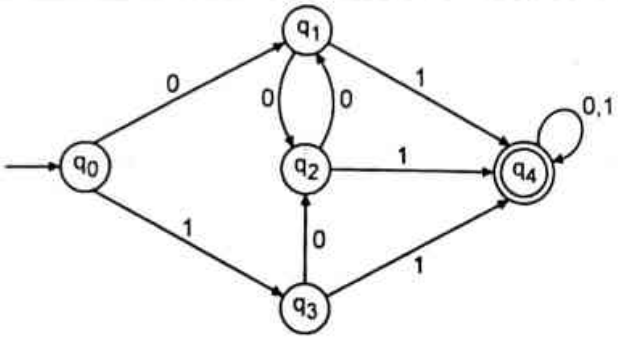
Since ae, bh and df are unmarked

∴ a = e, b = h and d = f

∴ Optimized DFA will be,

δ	0	1
ae	bh	df
bh	g	c
*c	ae	c
df	c	g
g0	g	ae

Q.55. Construct the minimum state automata equivalent to the transition diagram in the fig.



Ans. Transition table :

δ	0	1
→q0	q1	q3
q1	q2	q4
q2	q1	q4
q3	q2	q4
* q4	q4	q4

- Initially we have two groups as shown below.

Non-final **final**

q_0, q_1, q_2, q_3 q_4

Group 1 **Group 2**

Since $\delta(q_0,0) = q_1$,

$\delta(q_1,0) = q_2$,

$\delta(q_2,0) = q_1$,

$\delta(q_3,0) = q_2$,

Partitions of group 1 is not possible, because the transitions from all the member of group 1 goes to group 1 only, but since

$\delta(q_0,1) = q_3$,

$\delta(q_1,1) = q_4$,

$\delta(q_2,1) = q_4$,

$\delta(q_3,1) = q_4$,

States q_1, q_2, q_3 are distinguishable from another q_0 of group 1. Hence we divide group 1 into two group, one containing q_0 and other containing q_1, q_2, q_3 as shown below.

q_0 q_1, q_2, q_3 q_4

Group 1 **Group 2** **Group 3**

- Since $\delta(q_1,0) = q_2$,

$\delta(q_2,0) = q_1$,

$\delta(q_3,0) = q_3$,

Partitioning of group 2 is not possible, because the transitions from all members of group 1 goes to group 2 only.

Here, q_1, q_2, q_3 are non distinguishable, therefore we merge q_1, q_2, q_3 to form single group. There are no dead state.

Transition table for minimum DFA :

δ	0	1
→{q0}	{q0, q1, q3}	{q0, q1, q3}
{q0, q1, q3}	{q0, q1, q3}	{q4}
{q4}	{q4}	{q4}

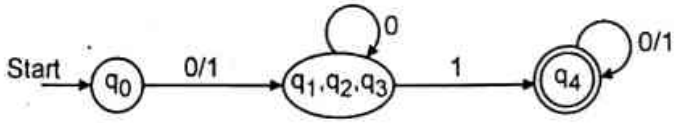


Fig. Minimum DFA

Q.56. Construct an optimised DFA equivalent to NFA

{p, q, r, s }, {0, 1} , δ, p, { q, s}

δ (p, 0) = q **δ (r, 0) = s**

δ (p, 1) = (q, s) **δ (r, 1) = ϕ**

δ (q, 0) = r **δ (s, 0) = p**

δ (q, 1) = (p, r) **δ (s, 1) = q**

Ans. Transition group for NFA:

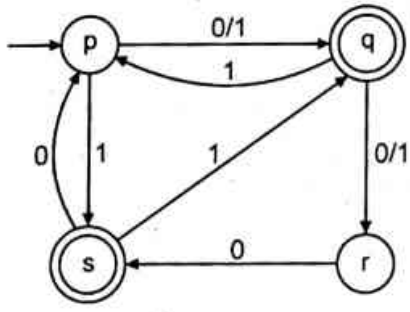


Fig.

Transition table for DFA:

δ	0	1
A = {p}	B = {q}	C = {q, s }
*B = {q}	D = {r}	E = {p, r}
*C = {q, s}	E = {p, r}	F= {p, q, r}
D = {r}	G = {s}	ϕ
E = {p, r}	H = {q, s}	H = {q, s}
* F = {p, q, r}	I = {q, r, s}	J = {p, q, r, s}
* G = {s}	A = {p}	B = {q}
* H = {q, s}	E = {p, r}	F = {p, q, r}
*I = {q, r, s}	K = {p, r, s}	F = {p, q, r}
*J = (p, q, r, s}	J = {p, q, r, s}	J = {p, q, r, s}
*K = {p, r, s}	L = {p, q, s}	J = {p, q, r, s}
*L = {p, q, s}	F = {p, q, r}	J = {p, q, r, s}

Transition graph of DFA :

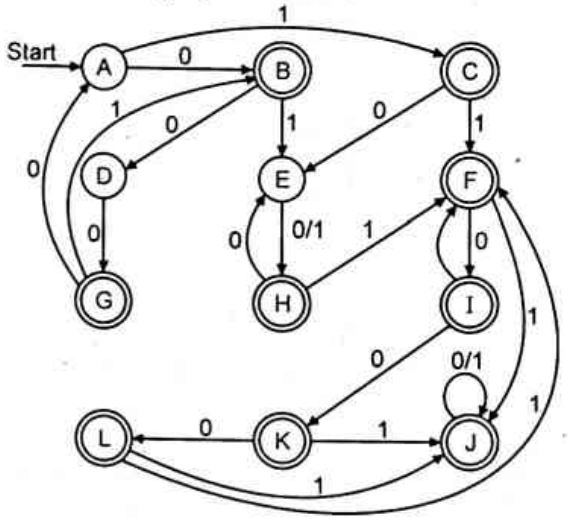


Fig. DFA

Optimization of DFA :

Final **Non Final**

BCFGHIJKL ADE

BCGH FIJKL ADE

BG CH FIJKL ADE

B G CH FIJKL A D E

B G* CH FIJKL A D E

Transition table for minimum DFA:

δ	0	1
A	B	CH
B	D	E
CH	E	FIJKL
D	G	ϕ
E	CH	CH
FIJKL	FIJKL	FIJKL
G	A	B

Transition diagram :

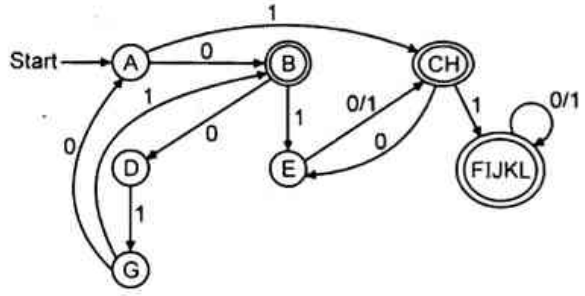


Fig. Optimize DFA

Q.57. Construct a minimum state automata equivalent to a given automata M whose transition table is given. ***CT: W-06(8M)***

State	Input	
δ	0	1
→ q_1	q_0	q_3
q_1	q_2	q_5
q_2	q_3	q_4
q_3	q_0	q_5
q_4	q_0	q_6
q_5	q_1	q_4
* q_6	q_1	q_3

Ans. From the given transition table

Group 1 : $\{q_6\}$ Set of final state

Group 2 : $\{q_0, q_1, q_2, q_3, q_4, q_5\}$

Set of final state

The pair of states are equivalent if their transition on all input symbols are same column wise.

- Here, we will consider the pair of states from group 2 as :

  (i) $\{q_0, q_1\}$

  (ii) $\{q_0, q_2\}$

(iii) $\{q_0, q_3\}$

(iv) $\{q_0, q_4\}$

(v) $\{q_0, q_5\}$

Now,

Step 1 :

(i) $\{q_0, q_1\}$

$\therefore q_0 = q_1$

	0	1
q_0	q_0	q_3
q_1	q_2	q_5

Both from same group that is group 2, so q_0 and q are equal.

(ii) $\{q_0, q_2\}$

$\therefore q_0 = q_2$

	0	1
q_0	q_0	q_3
q_1	q_3	q_4

From same group

(iii) $\{q_0, q_3\}$

$\therefore q_0 = q_3$

	0	1
q_0	q_0	q_3
q_3	q_0	q_5

(iv) $\{q_0, q_4\}$

$\therefore q_0 \neq q_4$

	0	1
q_0	q_0	q_3
q_4	q_0	q_6

From different set

(v) $\{q_0, q_5\}$

$\therefore q_0 = q_5$

	0	1
q_0	q_0	q_3
q_3	q_1	q_4

$\therefore$ So, new groups are

$\{q_6\}, \{q_4\}. \{q_0, q_1, q_2, q_3, q_5\}$

Step 2:

(i) $\{q_0, q_1\}$

(ii) $\{q_0, q_2\}$

(iii) $\{q_0, q_3\}$

(iv) $\{q_0, q_5\}$

(i) $\{q_0, q_1\}$

$\therefore q_0 = q_1$

	0	1
q_0	q_0	q_3
q_1	q_2	q_5

(ii) $\{q_0, q_2\}$

$\therefore q_0 \neq q_2$

	0	1
q_0	q_0	q_3
q_2	q_3	q_4

From different set or group

(iii) $\{q_0, q_3\}$

$\therefore q_0 = q_3$

	0	1
q_0	q_0	q_3
q_3	q_0	q_5

(iv) $\{q_0, q_5\}$

$\therefore q_0 \neq q_5$

	0	1
q_0	q_0	q_3
q_5	q_1	q_4

From different set

$\therefore$ New combination are:

$\{q_6\}, \{q_4\}, \{q_2, q_5\}, \{q_0, q_1, q_3\}$

Step 3:

(i) $\{q_0, q_1\}$

(ii) $\{q_0, q_3\}$

(i) $\{q_0, q_1\}$

$\therefore q_0 \neq q_1$

	0	1
q_0	q_0	q_3
q_3	q_1	q_4

From different group

*** Note:** Always check in new combination:

(ii) $\{q_0, q_3\}$

$\therefore q_0 \neq q_3$

	0	1
q_0	q_0	q_3
q_3	q_0	q_5

From different group

$\therefore$ New combination are:

$\{q_6\}, \{q_4\}, \{q_0\}, \{q_1, q_3\}, \{q_2, q_5\}$

Step 4 :

(i) $\{q_1, q_3\}$

$\therefore q_1 \neq q_3$

	0	1
q_1	q_2	q_5
q_3	q_0	q_5

From different group

$\therefore$ So, new combination

$\{q_6\}, \{q_4\}, \{q_0\}, \{q_1\}, \{q_3\}, \{q_2, q_5\}$

Step 5 :

(1) $\{q_2, q_5\}$

$\therefore q_2 \neq q_5$

	0	1
q_2	q_3	q_4
q_5	q_1	q_4

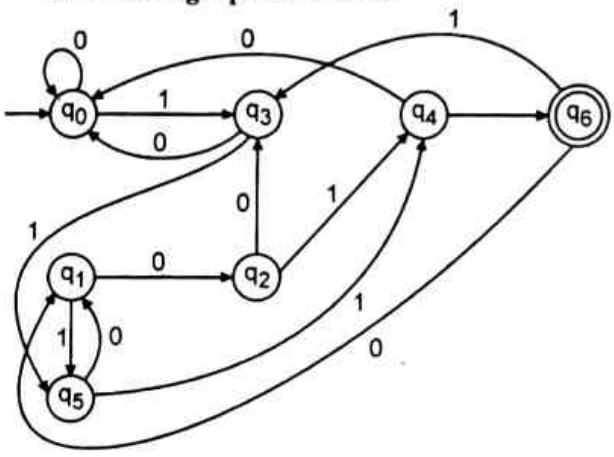
From different group

$\therefore$ So, new combination

$\{q_6\}, \{q_4\}, \{q_0\}, \{q_1\}, \{q_3\}, \{q_2\}, \{q_5\}$

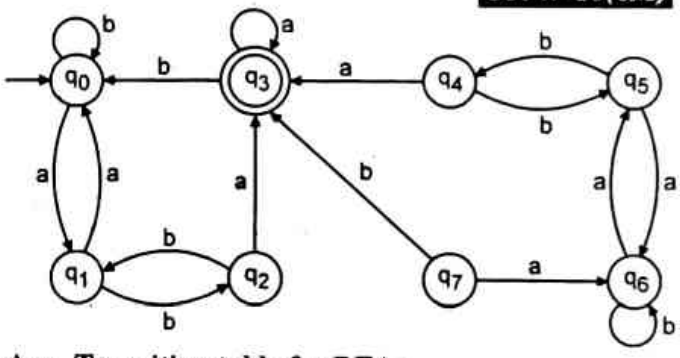
So, the given FA is already minimized.

Transition graph and DFA:



Q.58.Construct minimum state automation equivalent to the transition diagram given in fig.

CT: W-10(6M)



Ans. Transition table for DFA :

Σ / State	a	b
→ q_0	q_1	q_0
q_1	q_0	q_2
q_2	q_3	q_1
* q_3	q_3	q_0
q_4	q_3	q_5
q_5	q_6	q_4
q_6	q_5	q_6
q_7	q_6	q_3

group 1 = $\{q_3\}$

group 2 = $\{q_0, q_1, q_2, q_4, q_5, q_6, q_7\}$

Step 1:

(i) $\{q_0, q_1\}$

$q_0 = q_1$

	a	b
q_0	q_1	q_0
q_1	q_0	q_2

(ii) $\{q_0, q_2\}$

$\therefore q_0 \neq q_2$

	a	b
q_0	q_1	q_0
q_2	q_3	q_1

Different group

(iii) $\{q_0, q_4\}$

$\therefore q_0 \neq q_4$

	a	b
q_0	q_1	q_0
q_4	q_3	q_5

Different group

(iv) $\{q_0, q_5\}$

$\therefore q_0 = q_5$

	a	b
q_0	q_1	q_0
q_5	q_6	q_4

(v) $\{q_0, q_6\}$

$\therefore q_0 = q_6$

	a	b
q_0	q_1	q_0
q_6	q_5	q_6

(vi) $\{q_0, q_7\}$

$\therefore q_0 \neq q_7$

	a	b
q_0	q_1	q_0
q_7	q_6	q_3

Different group

$\therefore$ New combination of groups are:

$\{q_3\}, \{q_0, q_1, q_5, q_6\}, \{q_2, q_4, q_7\}$

Step 2:

(i) $\{q_0, q_1\}$

(ii) $\{q_0, q_5\}$

(iii) $\{q_0, q_6\}$

(i) $\{q_0, q_1\}$

$\therefore q_0 \neq q_1$

	a	b
q_0	q_1	q_0
q_1	q_0	q_2

Different group

(ii) $\{q_0, q_5\}$

$\therefore q_0 \neq q_5$

	a	b
q_0	q_1	q_0
q_5	q_6	q_4

Different group

(iii) $\{q_0, q_6\}$

$\therefore q_0 = q_6$

	a	b
q_0	q_1	q_0
q_6	q_5	q_6

$\therefore$ New combination of groups are:

$\{q_3\}, \{q_0, q_6\}, \{q_1, q_5,\}, \{q_2, q_4, q_7\}$

Step 3:

(i) $\{q_2, q_4\}$

(ii) $\{q_2, q_7\}$

(i) $\{q_2, q_4\}$

$\therefore q_2 = q_4$

	a	b
q_2	q_3	q_1
q_4	q_3	q_5

(ii) $\{q_2, q_7\}$

$\therefore q_2 \neq q_7$

	a	b
q_2	q_3	q_1
q_7	q_6	q_3

Different group

$\therefore$ New combination

$\{q_3\}, \{q_0, q_6\}, \{q_1, q_5\}, \{q_2, q_4\}, \{q_7\}$

Step 4:

(i) $\{q_0, q_6\}$

(ii) $\{q_1, q_5\}$

(iii) $\{q_2, q_4\}$

(i) $\{q_0, q_6\}$

$\therefore q_0 = q_6$

	a	b
q_0	q_1	q_0
q_6	q_5	q_6

(ii) $\{q_2, q_4\}$

$\therefore q_2 = q_4$

	a	b
q_2	q_3	q_1
q_4	q_3	q_5

(iii) $\{q_1, q_5\}$

$\therefore q_1 = q_5$

	a	b
q_1	q_0	q_2
q_5	q_6	q_4

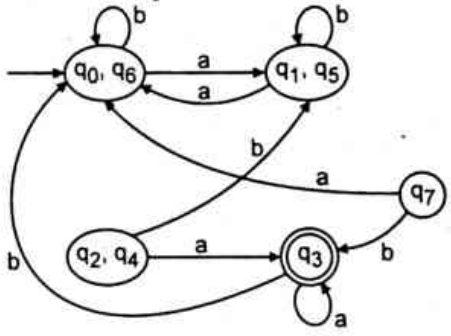
Final combination of groups are:

$\{q_3\}, \{q_0, q_6\}, \{q_1, q_5\}, \{q_2, q_4\}, \{q_7\}$

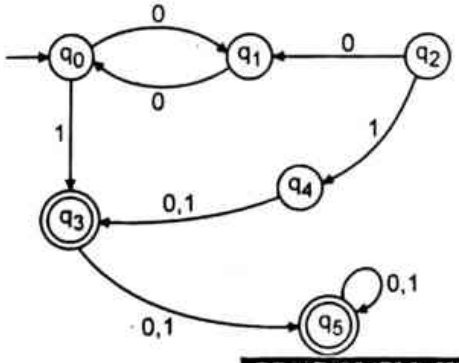
∴ **Transition table:**

Σ / States	a	b
→ $\{q_0, q_6\}$	$\{q_1, q_5\}$	$\{q_0, q_6\}$
$\{q_1, q_5\}$	$\{q_0, q_6\}$	$\{q_1, q_5\}$
$\{q_2, q_4\}$	$\{q_3\}$	$\{q_1, q_5\}$
*$\{q_3\}$	$\{q_3\}$	$\{q_0, q_6\}$
$\{q_7\}$	$\{q_0, q_6\}$	$\{q_3\}$

Transition Graph:



Q.59. Construct minimized DFA for the following transition diagram using pair table method:



CS : S-09(7M), W-12(6M)

Ans. Transition Table:

Σ / Q	0	1
→ q_0	q_1	q_3
q_1	q_0	ϕ
q_2	q_1	q_4
*q_3	q_5	q_5
q_4	q_3	q_3
*q_5	q_5	q_5

Step 1:

Set of final state = $\{q_3, q_5\}$ ... (A)

Set of Non-final state = $\{q_0, q_1, q_2, q_4\}$ ...(B)

Step 2:

(i) $\{q_0, q_1\}$

$q_0 = q_1$

	0	1
q_0	q_1	q_3
q_2	q_0	ϕ
	B	A

(ii) $\{q_0, q_2\}$

$q_0 \neq q_2$

	0	1
q_0	q_1	q_3
q_2	q_4	q_4
	B	A & B

(iii) $\{q_0, q_4\}$

$q_0 \neq q_4$

	0	1
q_0	q_1	q_3
q_4	q_3	q_3
	A & B	A

(iv) $\{q_3, q_5\}$

$\{q_3 = q_5\}$

	0	1
q_3	q_5	q_5
q_5	q_5	q_5
	A	A

Now

$\{q_3, q_5\} \rightarrow A$

$\{q_0, q_1\} \rightarrow B$

$\{q_2, q_4\} \rightarrow C$

Step 3 :

(i) $\{q_0, q_1\}$

$q_0 = q_1$

	0	1
q_0	q_1	q_3
q_1	q_0	ϕ
	B	A

(ii) $\{q_2, q_4\}$

$q_2 \neq q_4$

	0	1
q_2	q_1	q_4
q_4	q_3	q_3
	A&B	A&C

New sets are:

$\{q_3, q_5\}$

$\{q_0, q_1\}$

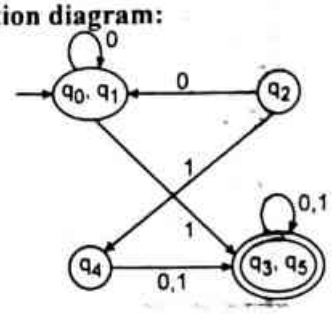
$\{q_2\}$

$\{q_4\}$

Transition Table:

Q \ Σ	0	1
→ $\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_3, q_5\}$
$\{q_2\}$	$\{q_0, q_1\}$	$\{q_4\}$
* $\{q_3, q_5\}$	$\{q_3, q_5\}$	$\{q_3, q_5\}$
$\{q_4\}$	$\{q_3, q_5$	$\{q_3, q_5$

Transition diagram:



Q.60. Construct a minimum state automation equivalent to finite automation.

CT : S-09(8M), W-11(7M)

Q \ Σ	0	1
→ q_0	q_1	q_5
q_1	q_6	q_2
(q_2)	q_0	q_2
q_3	q_2	q_6
q_4	q_7	q_5
q_5	q_2	q_6
q_6	q_6	q_4
q_7	q_6	q_2

Ans.

Final state = $\{q_2\}$ ..(A)

Non-Final state = $\{q_0, q_1, q_3, q_4, q_5, q_6, q_7\}$ ..(B)

Step 1 :

(i) $\{q_0, q_1\}$

$q_0 \neq q_1$

	0	1
q_0	q_1	q_5
q_1	q_6	q_2
	B	A&B

(ii) $\{q_0, q_3\}$

$q_0 \neq q_3$

	0	1
q_0	q_1	q_5
q_3	q_2	q_6
	A&B	B

(iii) $\{q_0, q_4\}$

$q_0 = q_4$

	0	1
q_0	q_1	q_5
q_4	q_7	q_5
	B	B

(iv) $\{q_0, q_5\}$

$q_0 \neq q_5$

	0	1
q_0	q_1	q_5
q_5	q_2	q_6
	A & B	B

(v) $\{q_0, q_6\}$

$q_0 = q_6$

	0	1
q_0	q_1	q_5
q_6	q_6	q_4
	B	B

(vi) $\{q_0, q_7\}$

$q_0 \neq q_7$

	0	1
q_0	q_1	q_5
q_7	q_6	q_2
	B	A & B

Now, new states are:

$\{q_2\}$ → A

$\{q_0, q_4, q_6\}$ → B

$\{q_1, q_3, q_5, q_7\}$ → C

Step 2:

(i) $\{q_1, q_3\}$

$q_1 \neq q_3$

	0	1
q_1	q_6	q_2
q_3	q_2	q_6
	B	A & B

(ii) $\{q_1, q_5\}$

$q_1 \neq q_5$

	0	1
q_1	q_6	q_2
q_5	q_2	q_6
	A & B	A & B

(iii) $\{q_1, q_7\}$

$q_1 = q_7$

	0	1
q_1	q_6	q_2
q_7	q_6	q_2
	A	B

Now, new states are:

$\{q_2\}$ → A

$\{q_0, q_4, q_6\}$ → B

$\{q_1, q_7\}$ → C

$\{q_3, q_5\}$ → D

Step 3 :

(i) $\{q_0, q_4\}$

$q_0 = q_4$

	0	1
q_0	q_1	q_5
q_4	q_7	q_5
	C	D

(ii) $\{q_0, q_6\}$

$q_0 \neq q_6$

	0	1
q_0	q_1	q_5
q_6	q_6	q_4
	B & C	B & D

Now new states are:

$\{q_2\}$ → A

$\{q_0, q_4\}$ → B

$\{q_6\}$ → C

$\{q_1, q_7\}$ → D

$\{q_3, q_5\}$ → E

Step 4 :

(i) $\{q_0, q_4\}$

$q_0 = q_4$

	0	1
q_0	q_1	q_5
q_4	q_7	q_5
	C	D

(ii) $\{q_1, q_7\}$

$q_1 = q_7$

	0	1
q_1	q_6	q_2
q_7	q_6	q_2
	B	A

(iii) $\{q_3, q_5\}$

$q_3 = q_5$

	0	1
q_3	q_2	q_6
q_5	q_2	q_6
	A	B

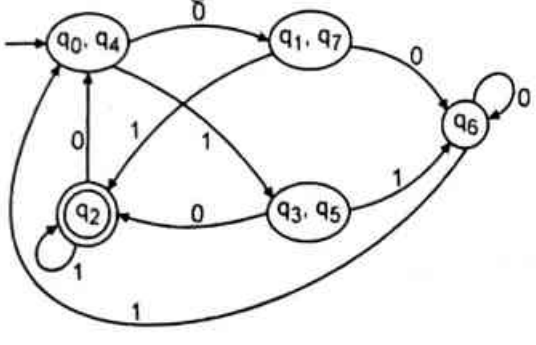
New states are:

$\{q_2\}, \{q_0, q_4\}, \{q_6\}, \{q_1, q_7\}, \{q_3, q_5\}$

Transition table for minimized DFA :

Σ / Q	0	1
→ $\{q_0, q_4\}$	$\{q_1, q_7\}$	$\{q_3, q_5\}$
$\{q_1, q_7\}$	$\{q_6\}$	$\{q_2\}$
*$\{q_2\}$	$\{q_0, q_4\}$	$\{q_2\}$
$\{q_3, q_5\}$	$\{q_2\}$	$\{q_6\}$
$\{q_6\}$	$\{q_6\}$	$\{q_0, q_4\}$

Transition Diagram:



Q.61. Construct a minimum state automaton equivalent to a given automaton M whose transition table is given below: **CS:S-10(7M)**

States	a	b
→ q_0	q_0	q_3
(q_1)	q_2	q_5
q_2	q_3	q_4
q_3	q_0	q_5
q_4	q_0	q_6
q_5	q_1	q_4
q_6	q_1	q_3

Ans. Final state = $\{q_1\} \rightarrow A$

Non final state = $\{q_0 q_2\ q_3\ q_4\ q_5\ q_6\} \rightarrow B$

Step 1 :

(i) $\{q_0\ q_2\}$

$q_0 = q_2$

	a	b
q_0	q_1	q_3
q_2	q_3	q_4
	B	B

(ii) $\{q_0, q_3\}$

	a	b
q_0	q_0	q_3
q_3	q_0	q_5
	B	B

$\therefore q_0 = q_3$

(iii) $\{q_0, q_4\}$

	a	b
q_0	q_0	q_3
q_4	q_0	q_6
	B	B

$\therefore q_0 = q_4$

(iv) $\{q_0, q_5\}$

	a	b
q_0	q_0	q_3
q_5	q_1	q_4
	A & B	B

$\therefore q_0 \neq q_5$

(v) $\{q_0, q_6\}$

	a	b
q_0	q_0	q_3
q_6	q_1	q_3
	A & B	B

$\therefore q_0 \neq q_6$

$\therefore$ So the new states are :

$\{q_1\}\{q_0, q_2, q_3, q_4\}\{q_5, q_6\}$

Step 2 :

Consider $\{q_1\} \rightarrow A$

$\{q_0, q_2, q_3, q_4\} \rightarrow B$

$\{q_5, q_6\} \rightarrow C$

(i) $\{q_0, q_2\}$

	a	b
q_0	q_0	q_3
q_2	q_3	q_4
	B	B

$\therefore q_0 = q_2$

(ii) $\{q_0, q_3\}$

	a	b
q_0	q_0	q_3
q_3	q_0	q_5
	B	B & C

$\therefore q_0 \neq q_3$

(iii) $\{q_0, q_4\}$

	a	b
q_0	q_0	q_3
q_4	q_0	q_6
	B	B & C

$\therefore q_0 \neq q_4$

$\therefore$ The new states are

$\{q_1\} \rightarrow A$, $\{q_5, q_6\} \rightarrow B$, $\{q_0, q_2\} \rightarrow C$, $\{q_3, q_4\} \rightarrow D$

Step 3 :

(i) $\{q_5, q_6\}$

	a	b
q_5	q_1	q_4
q_6	q_1	q_3
	A	D

$\therefore q_5 = q_6$

(ii) $\{q_0, q_2\}$

	a	b
q_0	q_0	q_3
q_2	q_3	q_4
	C & D	D

$\therefore q_0 \neq q_2$

(iii) $\{q_3, q_4\}$

	a	b
q_3	q_0	q_5
q_4	q_0	q_6
	C	B

$\therefore q_3 = q_4$

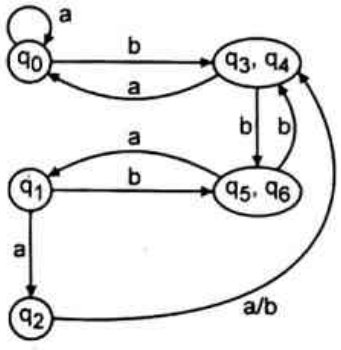
$\therefore$ The new states are

$\{q_0\}\ \{q_1\}\ \{q_2\}\ \{q_3, q_4\}\ \{q_5, q_6\}$

Transition table :

Σ / Q	0	1
$\{q_0\}$	$\{q_0\}$	$\{q_3, q_4\}$
$\{q_1\}$	$\{q_2\}$	$\{q_5, q_6\}$
$\{q_2\}$	$\{q_3, q_4\}$	$\{q_3, q_4\}$
$\{q_3, q_4\}$	$\{q_0\}$	$\{q_5, q_6\}$
$\{q_5, q_6\}$	$\{q_1\}$	$\{q_3, q_4\}$

Transition Diagram:



Q.62. What do you mean by equivalent of two finite state machine? Justify your answer with suitable example.

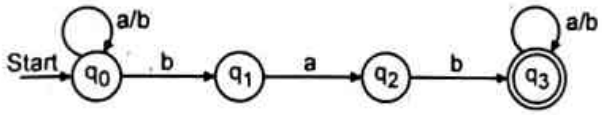
Ans.

- Finite state machine M_1 and M_2 are said to be equivalent, if they accept the same language i.e. $L(M_1) = L(M_2)$.

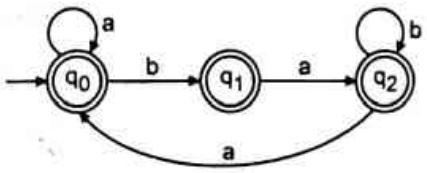
- It is possible to test whether the automatas of finite state machine M_1 and M_2 are accepting the same language and hence are equivalent or not.
- One method of doing this is to minimize M_1 and M_2 both and if the minimal state automata obtained from M_1 and minimal state automata obtained from M_2 comes out to be identical then M_1 is equivalent to M_2.

Example :

Non deterministic finite automata for language having bab as substring.



Deterministic finite automata for language which have bab as substring.



- Another method to test whether M_1 is equivalent to M_2 is find out whether.

$$(L(M_1) \cap L(\overline{M_2})) \cup (L(\overline{M_1}) \cap L(M_2) = \phi$$

- Construct a finite state machine accepting intersection of language accepted by M_1 and complement of M_2, if this machine accepts as empty set, then it means that there is no string acceptable to M_1 which is not acceptable to M_2.
- Similarly construct a finite state machine accepting intersection of language accepted by M_2 and complement of M_1, if this machine accept as empty set, then it means that there is no string acceptable to M_2 which is not acceptable to M_1.
- Thus language accepted by M_1 is same as language accepted by M_2. Hence machine M_1 is equivalent to M_2.

MOORE AND MEALY MACHINE

Q.63. What is limitation of finite automata? Explain Moore and Mealy machines to overcome this limitation.

Ans. Limitation of finite automata :

- One limitation of the finite automaton is that its output is limited to a binary signal "accept"/ "don't accept".
- Models in which the output is chosen from some other alphabet have been considered. There are two distinct approaches such as the output may be associated with the state called a Moore machine or with the transition called a Mealy machine.

Moore machines:

- A Moore machine is having six-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where Q, Σ, δ and q_0 are as in the DFA. Δ is the output alphabet and λ is a mapping from Q to Δ giving the output associated with each state.
- The output of M is response to input $a_1\ a_2\ .....a_n$, $n \geq 0$, is $\lambda(q_0)\ \lambda(q_1)\ ....\ \lambda(q_n)$, where q_0, q_1-$....q_n$ is the sequence of states such that $\delta(q_{i-1}, a_i) = q_i$ for 1 i n.

  Note : Any Moore machine gives output $\lambda(q_0)$ in response to input $\in$.
- The DFA may be viewed as a special case of a Moore machine where the output alphabet is (0, 1) and state q is "accepting" if and only if $\lambda(q) = 1$

Example :

Suppose we wish to determine the residue mode 3 for each binary string treated as a binary integer. If i written in binary is followed by a 0, the resulting string has value 2i, and if i in binary is followed by a 1, the resulting string has value 2i +1. If the remainder of i/3 is p, then the remainder of 2i/3 is 2p mod 3. If p = 0, 1 or 2, then 2p mod 3 is 0, 2, or 1, respectively. Similarly, the remainder of (2i + 1)/3 is 1, 0, or 2, respectively.

Therefore Moore machine consist of three states q_0, q_1, q_2 where q_i entered if and only if the input seen so far has residue . We define $\lambda(q_i) = j$ for $j = 0$, 1 and 2.

Figure shows transition diagram of a Moore machine.

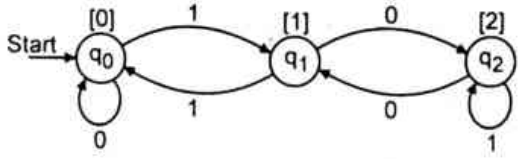


Fig. A Moore machine calculating residues

- On input 1010 the sequence of states entered is q_0, q_1, q_2, q_2, q_1, giving output sequence 01221. That is, $\in$ (which has "value" 0) has residue 0, 1 has residue 1, 2 (in decimal) has residue 2, 5 has residue 2, and 10 (in decimal) has residue 1.

Mealy machines:

- A Mealy machine is also a six-tuple $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where all ae as in the Moore machine, except that λ maps $Q\ \Sigma$ to Δ. This is, $\lambda(q, a)$ gives the output associated with the transition from state q on input a. The output of M in response to input $a_1\ a_2 \ldots a_n$ is $\lambda(q_0, a_1)$ $\lambda(q_1, a_2)$-----$\lambda(q_{n-1}, a_n)$, where $q_0, q_1 \ldots\ldots q_n$ is the sequence of states such that $\delta(q_{i-1}, a_1) = q_i$ for $1 \le i \le n$. This sequence has length n +1. On an input $\in$, a Mealy machine gives output $\in$.

Example :

The Mealy machine $M = (\{q_0, p_0, p_1\}, \{0, 1\}, \{y, n\}, d, \lambda, q_0)$ is shown in fig. Label a/b on an arc from state p to state q indicate that $\delta(p, u) = q$ and $\lambda(p, a) = b$.

- The response of M to input 01100 is nnyny, with the sequence of states entered being q_0, p_0, p_1p_1, p_0p_0. Note how p_0 remembers a zero and p_1 remembers a one. State q_0 is initial and "remembers" that no input has yet been received.

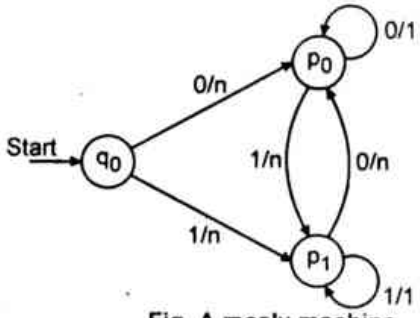
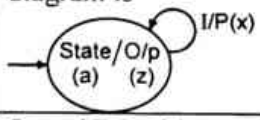
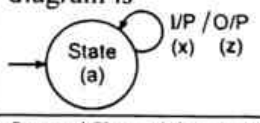
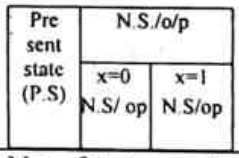
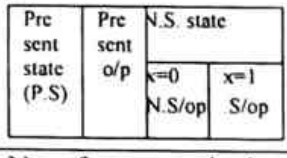


Fig. A mealy machine

Q.64. What is difference between Mealy and Moore machine? **CS: S-11(1 ½ M)**

Ans.

Sr. No.	Moore Machine	Mealy Machine
(1)	In Moore machine output may be associated with the state.	In Mealy machine output may be associated with the transition.
(2)	In Moore circuit output depends only on present state.	Output not only depends on present state but also depends on present input.
(3)	The flow /state diagram is 	The flow /state diagram is 
(4)	State / Transition table : 	State / Transition table : 
(5)	No. of state required in Moore is greater than no. of state required in Mealy.	No. of state required in Mealy is less than the no. of state required in Moore,

Q.65. What are the advantages of Moore and Mealy machine over finite state machine?

Ans. Advantages of Moore & Mealy machine :

(1) The finite state machine have binary output i.e. they accept the string or do not accept the string. This acceptability is decided on the basis of reachability of the final state from the initial state. This limitation can be removed by considering

the model where the output can be chosen from some other alphabets.

(2) There are two distinct approaches, the first in which the output is associated with the state, called Moore machine and the second in which the output is associated with transition, called Mealy machine.

(3) Even if the output alphabet has two symbols. The Mealy machine model can save states when compared with a finite automaton.

Example :

Consider the language (0+1) * (00+11) of all strings of 0's and 1's whose last symbols are the same. This language is accepted by no DFA with fewer than five states but can be accepted by Mealy machine.

- The states of Mealy machine corresponds to the sequence of accepting and no accepting states entered by DFA on the same input ; however, the Mealy machine does not make an output prior to input, while the DFA rejects the string ϵ, as its initial state is nonfinal.

Q.66. Explain Finite Automata with output.

Ans. Finite Automata with output :

- The DFA and NFA are only capable of deciding whether the i/p is rejected or accepted the mealy and moore machine are called as finite state machine with o/p, because they are capable of translating the i/p to o/p is a string of symbols belonging to some particularly chosen alphabets.
- They are represented by 6 tuples:

$\{Q\ \Sigma, \delta, q_0, \Delta, \lambda\}$

Q : Finite set of state.

Σ : Set of i/p symbol.

δ : Transition function, $Q \times \Sigma \rightarrow Q$.

q_0 : Initial state.

Here,

Δ = Set of output alphabets.

λ = Output mapping function.

Moore machine: In this machine o/p depend on present state only.

$$\lambda \Rightarrow Q \Rightarrow \Delta$$

Mealy machine: The o/p depends on present state & symbol which gives output.

$$\lambda \Rightarrow Q \times \Sigma \Rightarrow \Delta$$

Q.67. Write conversion steps of mealy machine to moore machine. *CS: S-11(1½ M)* *CT : W-11(4M)*

Ans. Transition table and mealy machine:

Present State	a = 0		a = 1	
	State	Output	State	Output
$\rightarrow q_1$	q_1	0	q_2	0
q_2	q_2	1	q_4	0
q_3	q_3	1	q_1	1
q_4	q_4	1	q_3	0

Step 1 : From the given table, it is clear that state q_1 is associated with one output = 0, at a = 0, a = 1 and q_3 is associated with output = 1 for a = 0 and a = 1.

But states q_2 and q_4 are associated with different output as for a = 0 the output is 1 and for a = 1 the output is 0. Similar for the state q_4.

$\therefore$ New table can be form as

Present State	a = 0		a = 1	
	State	Output	State	Output
q_1	q_3	0	q_{20}	0
q_{20}	q_1	1	q_{40}	0
q_{21}	q_1	1	q_{40}	0
q_3	q_{21}	1	q_1	1
q_{40}	q_{41}	1	q_3	0
q_{41}	q_{41}	1	q_3	0

Step 2 :

Now, from the previous step we found that the output for a = 0 and a = 1, $q_1 = 1$ and $q_3 = 0$

Present State	Next state		Output
	a = 0	a = 1	
$\rightarrow q_1$	q_3	q_{20}	1
q_{20}	q_1	q_{40}	0
q_{21}	q_1	q_{40}	1
q_3	q_{21}	q_1	0
q_{40}	q_{41}	q_3	0
q_{41}	q_{41}	q_3	1

Step 3 :

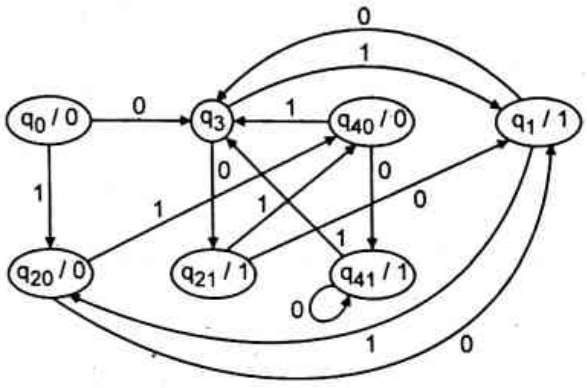
- We observe that the initial state q_1 is associated with output. This Moore machine accepts a zero length sequence which is not accepted by mealy machine.

- To overcome this situation, either we neglect the response of Moore machine or we must add a new starting state q_0, whose transitions are identical with q_1 but whose output is 0.

∴ Equivalent Moore machine table is

Present State	Next state		Output
	a = 0	a = 1	
q_0	q_3	q_{20}	0
q_1	q_3	q_{20}	1
q_{20}	q_1	q_{40}	0
q_{21}	q_1	q_{40}	1
q_3	q_{21}	q_1	0
q_{40}	q_{41}	q_3	0
q_{41}	q_{41}	q_3	1

The transition diagram :



Q.68. Convert the following Mealy m/c to Moore m/c.

Present State	a = 0		a = 1	
	State	o/p	State	o/p
q_1	q_2	1	q_3	1
q_2	q_2	2	q_3	1
q_3	q_2	1	q_3	2

Ans. Output table of Mealy machine :

State	Output
q_1	ϕ
q_2	{1,2} → q_{21}, q_{22}
q_4	{1,2} → q_{31}, q_{32}

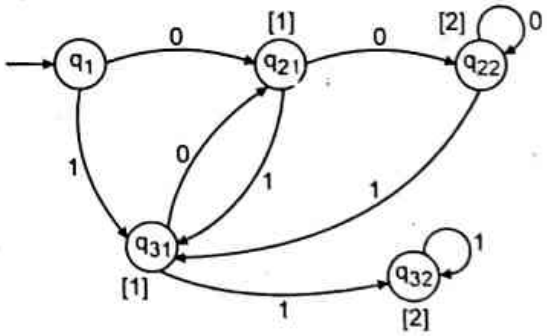
Transition table for Mealy machine :

Present State	a = 0		a = 1	
	State	o/p	State	o/p
→ q_1	q_{21}	1	q_{31}	1
q_{21}	q_{22}	2	q_{31}	1
q_{22}	q_{22}	2	q_{31}	1
q_{31}	q_{21}	1	q_{32}	2
q_{32}	q_{21}	1	q_{32}	2

Transition table for Moore machine :

Present State	Next state		Output
	a = 0	a = 1	
→ q_1	q_{21}	q_{31}	ϕ
q_{21}	q_{22}	q_{31}	1
q_{22}	q_{22}	q_{31}	2
q_{31}	q_{21}	q_{32}	1
q_{32}	q_{21}	q_{32}	2

Transition diagram:



Note: If for a particular string the o/p is not defined then its o/p should be considered as null.

Q.69. Construct Moore m/c equivalent to Mealy m/c 'M' is given in table. *CT:W-07(7M),W-08(6M)*

Present State	Next state			
	a = 0		a = 1	
	State	o/p	State	o/p
→ q_1	q_1	1	q_2	0
q_2	q_4	1	q_4	1
q_3	q_2	1	q_3	1
q_4	q_3	0	q_1	1

Ans. Output associated states :

S	O/P
q_0	0
q_2	{0,1} q_2 → 0: q_{20}, 1: q_{21}
q_3	{0,1} q_3 → 0: q_{30}, 1: q_{31}
q_4	1

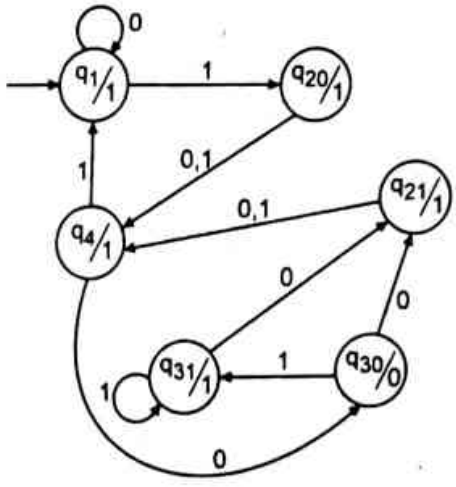
Transition table for Mealy Machine:

Present State	Next state			
	a = 0		a = 1	
	State	o/p	State	o/p
→ q_1	q_1	1	q_{20}	0
q_{20}	q_4	1	q_4	1
q_{21}	q_4	1	q_4	1
q_{30}	q_{21}	1	q_{31}	1
q_{31}	q_{21}	1	q_{31}	1
q_4	q_{30}	0	q_1	1

Transition table for Moore Machine:

Present State	Next state		Output
	a = 0	a = 1	
→ q_1	q_1	q_{20}	1
q_{20}	q_4	q_4	0
q_{21}	q_4	q_4	1
q_{30}	q_{21}	q_{31}	0
q_{31}	q_{21}	q_{31}	1
q_4	q_{30}	q_1	1

Transition diagram:



Q.70. Convert the following Mealy machine to Moore machine.

Present state	a = 0		a = 1	
	State	O/P	State	O/P
→ q_1	q_3	0	q_2	0
q_2	q_1	1	q_4	0
q_3	q_2	1	q_1	1
q_4	q_4	1	q_3	0

CS : W-09(7M)

Ans. Conversion of Mealy to Moore machine :

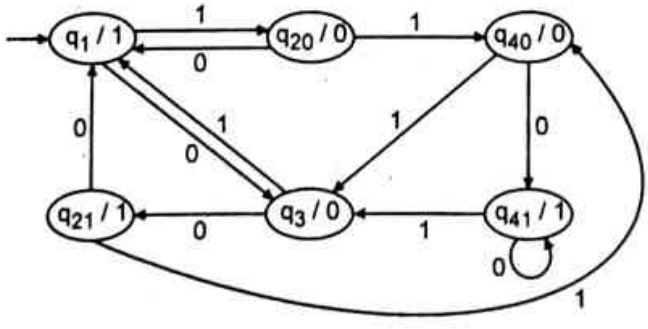
Step 1 : Output table :

State	Associated O/P
q_1	1
q_2	(0, 1) → q_{20}, q_{21}
q_3	0
q_4	{0, 1} → q_{40}, q_{41}

Step 2 : Transition table for Moore machine :

Present State	Next state		Output
	a = 0	a = 1	
q_1	q_3	q_{20}	1
q_{20}	q_1	q_{40}	0
q_{21}	q_1	q_{40}	1
q_3	q_{21}	q_1	0
q_{40}	q_{41}	q_3	0
q_{41}	q_{41}	q_3	1

Transition diagram for Moore machine :



Q.71. Construct Moore m/c to equivalent to mealy m/c represent by transition table given below.

CT: S-07(8M)

Present State	Next state			
	a = 0		a = 1	
	State	o/p	State	o/p
→ q_1	q_1	1	q_2	0
q_2	q_4	1	q_4	0
q_3	q_2	0	q_3	1
q_4	q_3	0	q_1	0

Ans.

State	Associated O/P
q_1	{0, 1} q_1 → q_{10}, q_{11}
q_2	0
q_3	{0, 1} q_3 → q_{30}, q_{31}
q_4	{0, 1} q_4 → q_{40}, q_{41}

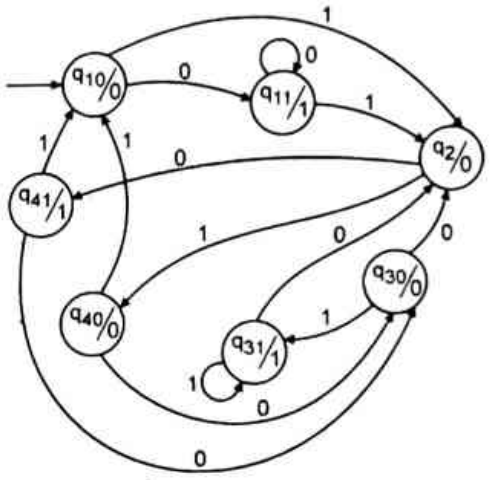
Transition table for mealy machine:

Present State	Next state			
	a = 0		a = 1	
	State	o/p	State	o/p
→ q_{10}	q_{11}	1	q_2	0
q_{11}	q_{11}	1	q_2	0
q_2	q_{41}	1	q_{40}	0
q_{30}	q_2	0	q_{31}	1
q_{31}	q_2	0	q_{31}	1
q_{40}	q_{30}	0	q_{10}	0
q_{41}	q_{30}	0	q_{10}	0

Transition table for Moore Machine:

Present State	Next state		Output
	a = 0	a = 1	
→ q_{10}	q_{11}	q_2	0
q_{11}	q_{11}	q_2	1
q_2	q_{41}	q_{40}	0
q_{30}	q_2	q_{31}	0
q_{31}	q_2	q_{31}	1
q_{40}	q_{30}	q_{10}	0
q_{41}	q_{30}	q_{10}	1

Transition diagram:



Q.72. Write down steps of conversion of Moore m/c to Mealy m/c.

Ans. To convert Moore machine to Mealy, design transition table and with each stage of the mealy machine, the output present in the Moore form.

Step 1: Transition table for Moore machine :

Present state	a = 0	a = 1	Output
→ q_1	q_1	q_2	0
q_2	q_1	q_3	1
q_3	q_1	q_3	0

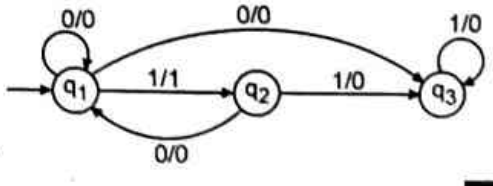
Step 2: From the above table the output of the states are :

State	Output (Δ)
→ q_1	0
q_2	1
q_3	0

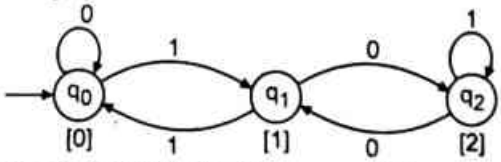
Step 3: Now, depending on the output the states can be generated as :

Present State	a = 0		a = 1	
	State	o/p	State	o/p
q_1	q_1	0	q_2	1
q_2	q_1	0	q_3	0
q_3	q_1	0	q_3	0

Step 4: Transition diagram:



Q.73. Convert following Moore m/c to equivalent mealy machine.



Ans. Transition table for Moore machine:

Present state	a = 0	a = 1	Output
→ q_0	q_0	q_1	0
q_1	q_2	q_0	1
q_2	q_1	q_2	2

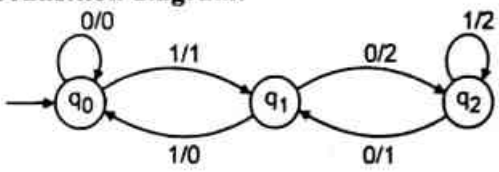
Output table :

State	Output (Δ)
q_0	0
q_1	1
q_2	2

Transition table for Mealy machine :

Present State	a = 0		a = 1	
	State	o/p	State	o/p
→ q_0	q_0	0	q_1	1
q_1	q_2	2	q_0	0
q_2	q_1	1	q_2	2

Transition diagram:



Q.74. Design a Mealy machine which will operate on binary no. system and will generate the remainder for any i/p, where division is 3.

Ans. $\delta(q_0, 0) = 2\times0+0 = 0 = q_0$

$\delta(q_0, 1) = 2\times0+1 = 1 = q_1$

$\delta(q_1, 0) = 2\times1+0 = 2 = q_2$

$\delta(q_1, 1) = 2\times1+1 = 3/3 = 0 = q_0$

$\delta(q_2, 0) = 2\times2+0 = 4/3 = 1 = q_1$

$\delta(q_2, 1) = 2\times2+1 = 5/3 = 2 = q_2$

Transition table for Moore machine :

Present state	a = 0	a = 1	Output
→ q_0	q_0	q_1	0
q_1	q_2	q_0	1
q_2	q_1	q_2	2

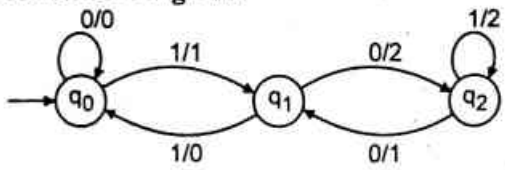
Output table :

State	Output (Δ)
q_0	0
q_1	1
q_2	2

Transition table for Mealy machine :

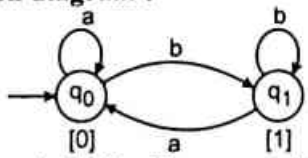
Present State	a = 0		a = 1	
	State	o/p	State	o/p
q_0	q_0	0	q_1	1
q_1	q_2	2	q_0	0
q_2	q_1	1	q_2	2

Transition diagram:



Q.75. Design a mealy machine which will operate on A, B and convert the string of (a, b) into string of (0, 1).

Ans. Transition diagram :



Transition table for Moore machine :

Present state	M = a	M = b	Output
→ q_0	q_0	q_1	0
q_1	q_0	q_1	1

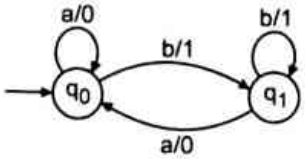
Output table :

State	Output (Δ)
q_0	0
q_1	1

Transition table for Mealy machine :

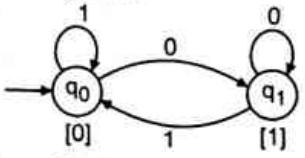
Present State	M = a		M = b	
	State	o/p	State	o/p
q_0	q_0	0	q_1	1
q_1	q_0	0	q_1	1

Transition diagram:



Q.76. Design a mealy machine to perform 1's compliment.

Ans. Transition diagram :



Transition table for Moore machine :

Present state	a = 0	b = 1	Output
→ q_0	q_1	q_0	0
q_1	q_1	q_0	1

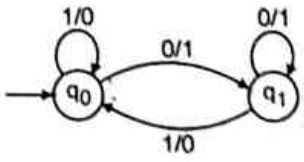
Output table :

State	Output (Δ)
q_0	0
q_1	1

Transition table for Mealy machine :

Present State	a = 0		b= 1	
	State	o/p	State	o/p
→ q_0	q_1	1	q_0	0
q_1	q_1	1	q_0	0

Transition diagram :



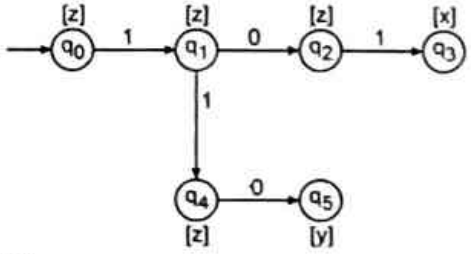
Q.77. Construct Mealy and Moore machines for the input from Σ = {0, 1} if the input ends in 101, output should be x, if the input ends in 110, the output should be y, otherwise output be z.

CT : W-12(8M)

OR. Design Moore machine for a binary input sequence such that if it has a substring 101, the machine outputs A. If it has a substring 110, the machine outputs B. Otherwise it outputs C.

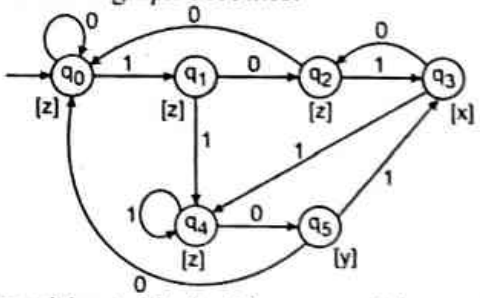
CT : S-12(7M)

Ans. Consider a Moore machine in which if input ends in 101, output should be x, if the input ends in 110 the output should be y and for other strings output should be z.



Now,

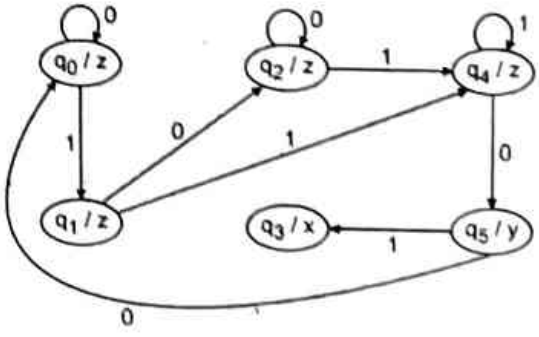
Insert the possibility of 1's and 0's for each state then above graph becomes.



Transition table for Moore machine:

Present State	Next state		Output
	a = 0	a = 1	
→ q_0	q_0	q_1	z
q_1	q_2	q_4	z
q_2	q_0	q_3	z
q_3	q_2	q_4	x
q_4	q_5	q_4	z
q_5	q_0	q_3	y

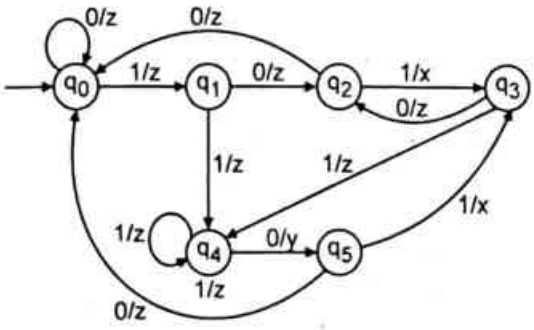
Transition diagram for Moore machine :



Transition table for Mealy Machine:

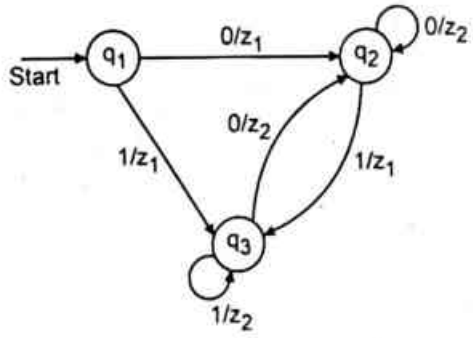
Present State	Next state			
	a = 0	o/p	a = 1	o/p
q_0	q_0	z	q_1	z
q_1	q_2	z	q_4	z
q_2	q_0	z	q_3	x
q_3	q_2	z	q_4	z
q_4	q_5	z	q_4	z
q_5	q_0	y	q_4	z

Transition diagram for Mealy machine:



Q.78. Consider a mealy machine represented by following figure. Construct a Moore machine equivalent to this mealy machine.

CT: S-10(7M), W-10(5M)



Ans. Transition table for mealy machine:

Present State	Next state			
	a = 0	o/p	a = 1	o/p
q_1	q_2	z_1	q_3	z_1
q_2	q_2	z_2	q_3	z_1
q_3	q_2	z_2	q_3	z_2

Output symbol associated with state $q_1 = \phi$

output symbol associated with state $q_2 = z_1, z_2$

output symbol associated with state $q_3 = z_1, z_2$

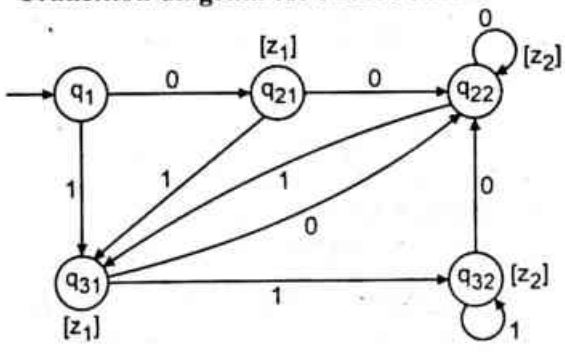
Created new states are : q_{21}, q_{22}, q_{31} & q_{32}

Present State	Next state			
	a = 0	o/p	a = 1	o/p
$\rightarrow q_1$	q_{21}	z_1	q_{31}	z_1
q_{21}	q_{22}	z_2	q_{31}	z_1
q_{22}	q_{22}	z_2	q_{31}	z_1
q_{31}	q_{22}	z_2	q_{32}	z_2
q_{32}	q_{22}	z_2	q_{32}	z_2

Transition table for Moore machine:

Present state	Next State		
	a = 0	a = 1	o/p
$\rightarrow q_1$	q_{21}	q_{31}	ϕ
q_{21}	q_{22}	q_{31}	z_1
q_{22}	q_{22}	q_{31}	z_2
q_{31}	q_{22}	q_{32}	z_1
q_{32}	q_{22}	q_{32}	z_2

Transition diagram for Moore machine :



Q.79. Design a mealy machine that recognizes the double occurrence of symbol 'a' in input string $S \in \Sigma^*$ where $\Sigma = \{a, b\}$. **CS: W-12(5M)**

Ans.

- Let us assume that the output alphabet $\Delta = \{y, n\}$.
  'Y' is 'yes' will be the output if string is accepted and 'n' is 'no' will be the output if string is not accepted i.e. rejected, by the m/c.
- The initial state is q_0, from q_0 there will be two transitions. One on 'a' and other on 'b' as

$\Sigma = \{a, b\}$.

- On 'a' we should make the transition to some other state, say q_1, which will look for second concretive 'a' to get double occurrence of symbol 'a'.

Transition diagram for Mealy machine :

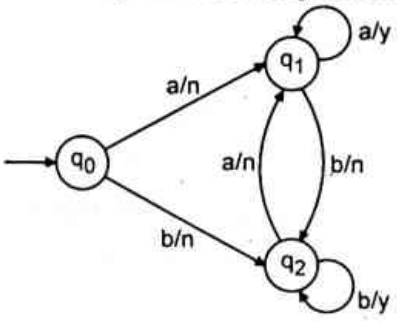


Fig. Mealy machine

Q.80. Construct a Mealy machine which is equivalent to Moore machine given in table. ***CT: S-08(6M), W-09(7M)***

Present State	Next state		Output
	a = 0	a = 1	
→ q_0	q_1	q_2	1
q_1	q_3	q_2	0
q_2	q_2	q_1	1
q_3	q_0	q_3	1

Ans. The Moore machine constructed from transition diagram given in above table is as follows:

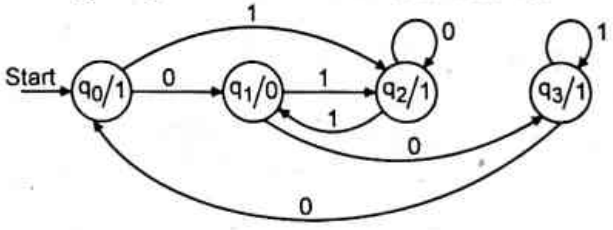


Fig. Moore Machine

Transition table for Mealy machine:

Present state	Nest state			
	a = 0		a = 1	
	State	Output	State	Output
→ q_0	q_1	0	q_2	1
q_1	q_3	1	q_2	1
q_2	q_2	1	q_1	0
q_3	q_0	1	q_3	1

The Mealy machine constructed from transition table is as follows :

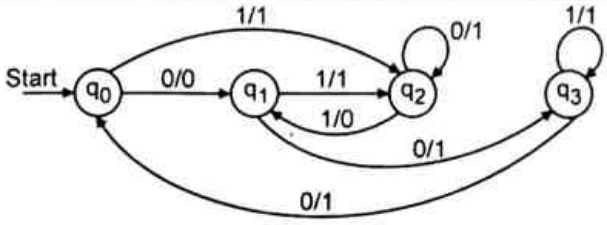


Fig. Mealy Machine

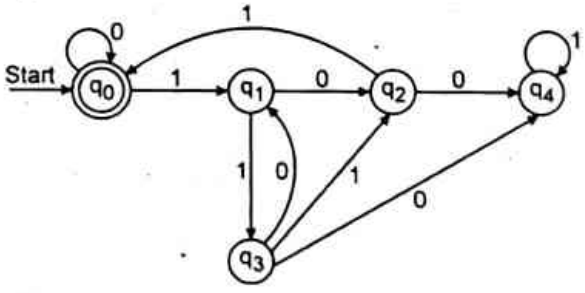
Q.81. Construct a Mealy machine which of remainder generated after dividing an integer number by five. ***CS: S-12(7M)***

Ans. Here, $\delta = \{q_0, q_1, q_2, q_4\}$ which are depends on remainder 0, 1, 2, 3, 4.

Transition table:

Q \ Σ	0	1
→ q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_4	q_0
q_3	q_1	q_2
q_4	q_3	q_4

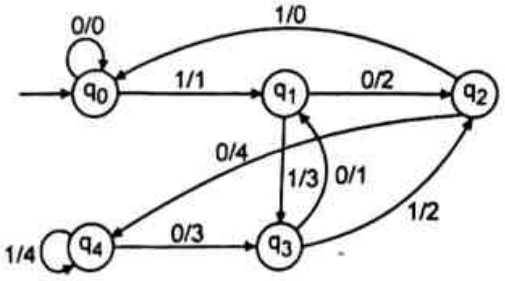
The transition diagram from the transition table given as :



Now, Transition table for Mealy machine:

Present state	Nest state			
	a = 0		a = 1	
	State	Output	State	Output
→ q_0	q_0	0	q_1	1
q_1	q_2	2	q_3	3
q_2	q_4	4	q_0	0
q_3	q_1	1	q_2	2
q_4	q_3	3	q_4	4

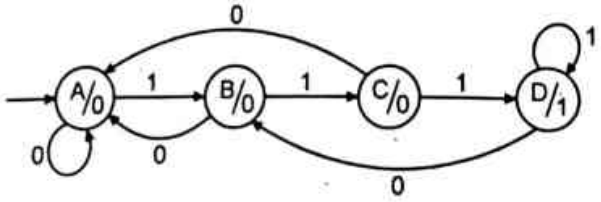
Transition diagram:



Q.82. Design a Moore and Mealy machine that gives a '1' as its output if and only if the last three digit are all 1's assume that $\Sigma = \{0, 1\}$.

CS : S-10(7M)

Ans.



Transition table for Moore machine :

Present State	Next state		Output
	a = 0	a = 1	
→A	A	B	0
B	A	C	0
C	A	D	0
D	B	D	1

Step I : Output associated with states

Output associated with A = 0

Output associated with B = 0

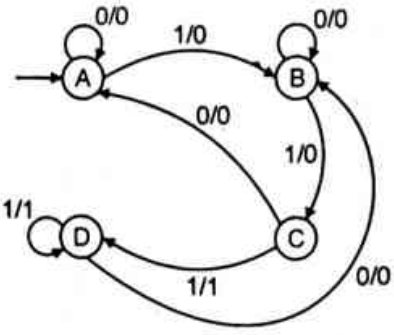
Output associated with C = 0

Output associated with D = 1

Step II : Transition table for Mealy Machine :

Present state	Next state			
	a = 0	O/P	a = 1	O/P
→A	A	0	B	0
B	A	0	C	0
C	A	0	D	1
D	B	0	D	1

Step III : Transition diagram :



Q.83. Construct Mealy machine which can output even, odd according as total number of 1's encountered is even or odd. The input symbol are 0'1 and '1'. **CS : W-10 (5M)**

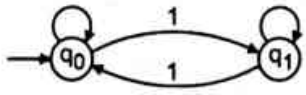
Ans. We require two state q_0 & q_1.

q_0' denotes even number of ones and q_1 denoted odd number of ones.

First we construct FSM for even number of 1odd number of 1

Q \ Σ	0	1
→q_0	q_0	q_1
q_1	q_1	q_0

Transition diagram :



Now, we will add output which is shown in transition diagram.

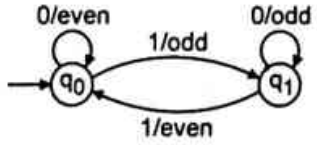


Fig. Mealy machine :

POINTS TO REMEMBER:

(1) A finite automata is a restricted model of computer having finite and fixed storage capability.

(2) For every non-deterministic finite automata there exists an equivalent deterministic finite automata.

(3) The class of languages recognized by finite automata is called as regular sets.

(4) If the number of states of a non-deterministic finite automata is n, then equivalent deterministic finite automata will have 2^n states.

(5) Minimization or optimization of a deterministic finite automata, refers to elimination of unreachable states combining non distinguishable states and elimination of dead state.

(6) Minimal state deterministic finite automata accepting a given language is unique.

(7) There exists an algorithm to answer the question, whether two finite automatons accepts the same language.

(8) A Mealy machine is a DFA in which output is associated with the transition.

(9) A Moore machine is a DFA in which output is associated with the state.

(10) There exists no exact equivalence between a Mealy and Moore machine.

Notes :-

UNIT - III
REGULAR EXPRESSION

SYLLABUS

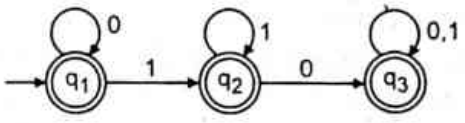
Regular expression, Identities, Regular grammar, Right linear, Left linear, Arden theorem, Pumping lemma for regular sets, Closure & decision properties for regular sets, Context free languages, Parse trees and ambiguity, Reduction of CFGS, Normal forms for CFG .

UNIVERSITY PAPER SOLUTIONS
SINCE SUMMER - 2006

Note to the students :
- The questions given below are from previous Nagpur University examination papers.
- Though these questions are from old university papers, they have been included in the new syllabus.
- Questions which are out of new syllabus are not included here.

SUMMER - 06 (CT)

Q.1. Find out regular expression for transition diagram : **6M**



Ans. P.3-50, Q.74.

Q.2. Prove that the following languages are not regular. **8M**

(i) $L = \{WW \mid W \in (a,b)^*\}$

Ans. P.3-55, Q.87.

(ii) $L = \{0^i 1^i \mid i \geq 1\}$

Ans. P.3-54, Q.84.

Q.3. Convert the following regular expression to DFA, $(0 \mid 1)^* 01$. **9M**

Ans. P.3-21, Q.26.

Q.4. Given a grammar G = (V, T, P, S) which is in CNF and a string w, whose length is n. How long the derivation of w is ? Why ? **4M**

Ans. P.3-81, Q.163

Q.5. Prove that using pumping lemma, the language L is not context free language,

$L = \{a^i b^i c^i \mid i \geq 1\}$ **7M**

Ans. P.3-58, Q.97

Q.6. Define Linear grammar and comment on "Every regular grammar is linear but every linear grammar is not necessarily regular" **4M**

Ans. P.3-37, Q.42

Q.7. Eliminate left recursion and reduce the following grammar :

$S \rightarrow aBDh$

$B \rightarrow Bbc \mid b$

$D \rightarrow dD \mid \in$ **4M**

Ans. P.3-79, Q.155

WINTER - 06 (CT)

Q.1. Obtain an equivalent left linear grammar from given right linear grammar.

$S \rightarrow bA$

$A \rightarrow aA \mid bB \mid b$

$H \rightarrow bA$ **5M**

Ans. P.3-41, Q.53.

Q.2. Give regular expression for representing the set L of strings in which every zero is immediately followed by at least two ones. **5M**

Ans. P.3-12, Q.5.

Q.3. Explain the steps used to construct finite automata equivalent to the given regular expression. Use these steps to construct F.A. equivalent to the regular expression. **8M**

$(0 + 1)^* (00 + 11) (0 + 1)^*$

Ans. P.3-17, 46, Q.21(a), 65.

Q.4. Generate context free grammar for the language.

$L = \{a^m b^n \mid m, n > 0 \text{ and } m > n\}$ **5M**

Ans. P.3-62, Q.107.

Q.5. Find the Greibach normal form grammar equivalent to the following CFG.

$S \rightarrow AA \mid 0$

$A \rightarrow SS \mid 1$ **8M**

Ans. P.3-89, Q.178.

SUMMER - 07 (CT)

Q.1. Show that the grammar.

$S \rightarrow 0 \mid 01S1 \mid 0A1,$

$A \rightarrow 1S \mid 0AA \mid$ is ambiguous **4M**

Ans. P.3-69, Q.128

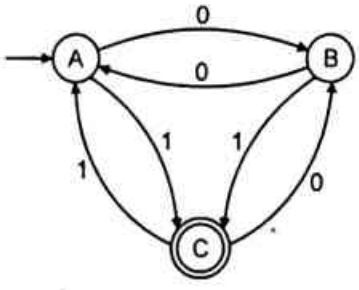
Q.2. Prove or disprove the following for regular expressions :

(a) $(rs + r)^* r = r (sr + r)^*$

(b) $(r + s)^* = r^* + s^*$ **4M**

Ans. P.3-15, Q.15.

Q.3. Construct the regular expression for the following state diagram. **4M**



Ans. P.3-50, Q.75.

Q.5. Obtain the CNF and GNF grammar for the following :

$E \rightarrow E + T \mid T$

$T \rightarrow a \mid (E)$ **8M**

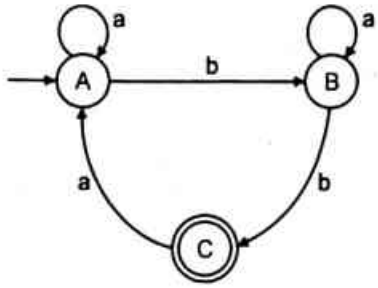
Ans. P.3-95, Q.186

Q.6. Construct the left linear and right linear grammar for the language represented by the regular expression. $(((01 + 10)^* 00)^* 11)^*$ **10M**

Ans. P.3-44, Q.61

WINTER - 07 (CT)

Q.1. Find the regular expression for the automata shown below: **6M**



Ans. P.3-49, Q.73.

Q.2. Show that the grammar

$S \rightarrow aB \mid ab$

$A \rightarrow aAB \mid a$

$B \rightarrow AB\ b \mid b$ is ambiguous. **3M**

Ans. P.3-67, Q.123.

Q.3. Find a CFG with no useless symbols equivalent to the following grammar.

$S \rightarrow AB \mid CA$

$B \rightarrow BC \mid AB$

$A \rightarrow a$

$C \rightarrow aB \mid b$ **5M**

Ans. P.3-73, Q.139.

Q.4. Find a grammar in Chomsky Normal Form (CNF) equivalent to

$S \rightarrow aAbB$

$A \rightarrow qA \mid q$

$B \rightarrow bB \mid b$ **5M**

Ans. P.3-82, Q.164.

Q.5. What are the steps for converting right linear grammar to equivalent left-linear grammar? Convert the following right-linear grammar to its equivalent left linear grammar.

$S \rightarrow 1\ B$

$B \rightarrow 1\ C$

$B \rightarrow 0\ B$

$B \rightarrow 1$

$C \rightarrow 0$ **7M**

Ans. P.3-39, 42, Q.48 & 55

Q.6. Bring the following grammar in GNF

$S \rightarrow SS$

$S \rightarrow 0\,S1 \mid 01$ **6M**

Ans. P.3-90, Q.179.

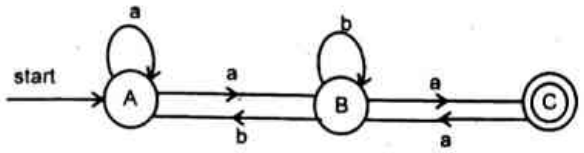
Q.7. Construct a regular grammar G generating the regular set represented by $P = a.b\,(a + b)^*$ **5M**

Ans. P.3-37, Q.43.

SUMMER - 08 (CT)

Q.1. Show that the given automata recognize the regular expression

$(a + a\,(b + aa)^*\,b)^*\,a\,(b + aa)^*\,a$ **7M**

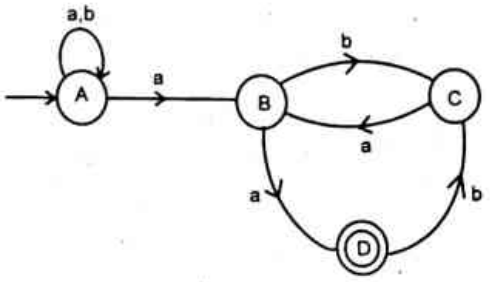


Ans. P.3-47, Q.69.

Q.2. Explain the closure properties of regular languages with suitable example. **5M**

Ans. P.3-59, Q.98.

Q.3. Write a regular expression for the language accepted by the given finite automata. **8M**



Ans. P.3-49, Q.72.

Q.4. Construct a regular grammar G generating the regular set represented by

$P = a^*b\,(a + b)^*$ **5M**

Ans. P.3-37, Q.43.

Q.5. Obtain an equivalent left linear grammar from the given right linear grammar

$S \rightarrow bA$

$A \rightarrow aA \mid bB \mid b$

$B \rightarrow bA$ **5M**

Ans. P.3-41, Q.53.

Q.6. Write a regular expression containing 011 preceded and followed by any number of zero's and 1's over $\Sigma = \{0, 1\}$. **3M**

Ans. P.3-12, Q.2.

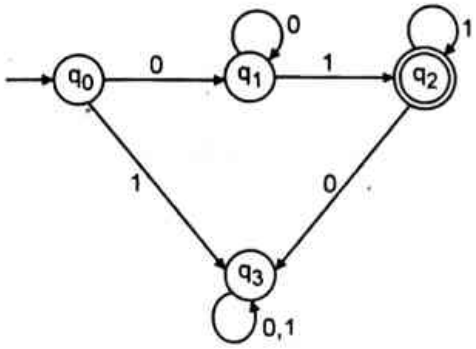
WINTER - 08 (CT)

Q.1. Construct a D.F.A. with reduced state equivalent to the regular expression.

R.E. = $10 + (00 + 1)\,0^*\,1$. **7M**

Ans. P.3-18, Q.22.

Q.2. Find the regular expression for the given transition diagram. **7M**



Ans. P.3-51, Q.78.

Q.3. Give the statement of Pumping Lemma. Give its applications. Prove that $L = \{a^P \mid P \text{ is a Prime}\}$ is not regular. **6M**

Ans. P.3-53, Q.81, 82 & 83.

Q.4. Remove all unit-productions, all useless productions and all ϵ –productions from the grammar :

$S \rightarrow aA \mid aBB$

$A \rightarrow aaA \mid \epsilon$

$B \rightarrow bB \mid bbC$

$C \rightarrow B$

What language does this grammar generate ? **7M**

Ans. P.3-80, Q.156

Q.5. Convert the following grammar to GNF :

$S \rightarrow ABA \mid AB \mid BA \mid AA \mid A \mid B$

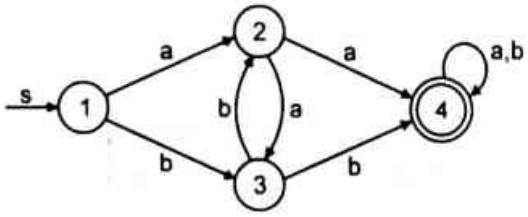
$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$ **6M**

Ans. P.3-92, Q.181.

SUMMER - 09 (CS)

Q.1. Find the regular expression for the following DFA. **8M**



Ans. P.3-50, Q.76.

Q.2. Construct a NFA with ∈ - transition equivalent to the regular expression :

$R_1 = 10 + (0 + 11)\,0^*1$ **5M**

Ans. P.3-18, Q.22.

Q.3. Show by using pumping lemma, language

$L = \{a^n b^{n+1} : n > 0\}$ is not regular. **7M**

Ans. P.3-55, Q.86.

Q.4. Design DFA for L = {a, b} to accept all strings containing "aba" as substring and not containing "bba" as substring. **10M**

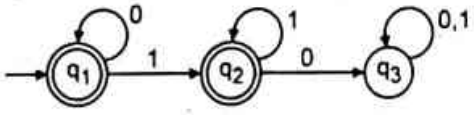
Ans. P.3-28, Q.33

SUMMER - 09 (CT)

Q.1. State and prove the theorem of Pumping lemma. Give its applications. **7M**

Ans. P.3-52, 53, Q.80 & 82.

Q.2. Convert the given transition graph into the regular expression. **6M**



Ans. P.3-47, Q.67.

Q.3. Construct an equivalent left linear grammar for the right linear grammar given as :

$S \rightarrow 0A \mid 1B$

$A \rightarrow 0C \mid 1A \mid 0$

$B \rightarrow 1B \mid 1A \mid 1$

$C \rightarrow 0 \mid 0A$ **5M**

Ans. P.3-41, Q.51.

Q.4. Construct DFA accepting the regular language generated by the left-linear grammar given below

$S \rightarrow Ca \mid Bb$

$C \rightarrow Bb$

$B \rightarrow Ba \mid b$ **5M**

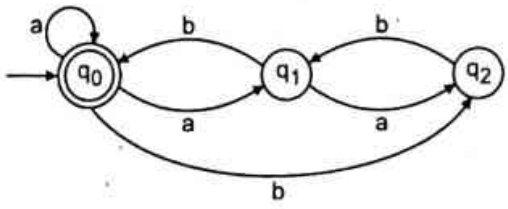
Ans. P.3-39, Q.49.

Q.5. Explain the closure properties of regular sets. **3M**

Ans. P.3-59, Q.98.

WINTER - 09 (CS)

Q.1. For the following FSM, design regular expression. Also explain the indentity used for expression. **4M**



Ans. P.3-52, Q.79(b)

Q.2. Reduce the following grammar :

$S \rightarrow aAa$

$A \rightarrow Sb \mid bCC \mid DaA$

$C \rightarrow abb \mid DD$

$D \rightarrow aDA$

$E \rightarrow aC$ **5M**

Ans. P.3-73, Q.138

Q.3. Find the equivalent CNF for following grammar :

$S \rightarrow aB \mid ab$

$A \rightarrow aAB \mid a$

$B \rightarrow ABb \mid b$ **4M**

Ans. P.3-85, Q.172

Q.4. Using principle of pumping lemma prove

$L = \{a^n b^m c^{m+n} \mid m, n >= 1\}$ is not regular. **5M**

Ans. P.3-56, Q.88

Q.5. Design CFG for $L = \{a^n b^{n+1} \mid n >= 1\}$. **5M**

Ans. P.3-65, Q.115

WINTER - 09 (CT)

Q.1. Justify the following statements :

(i) Union of two Regular languages is a Regular language.

(ii) Intersection of Regular language and CFL is a CFL. **6M**

Ans. P.3-60, Q.101

Q.2. Show that

$L = \{a^p / p \text{ is a prime.}\}$

is not regular. **6M**

Ans. P.3-53, Q.83.

Q.3. Obtain an equivalent right linear grammar for the left linear grammar given below :

$S \rightarrow Abc \mid c$

$A \rightarrow Acc \mid bc$ **7M**

Ans. P.3-42, Q.56.

Q.4. Find CFG with no useless symbol equivalent to the following grammar :

$S \rightarrow AB \mid CA$

$B \rightarrow BC \mid AB$

$A \rightarrow a$

$C \rightarrow aB \mid b$ **6M**

Ans. P.3-73, Q.139.

Q.5. Construct CFG for the language :

(i) $L = \{a^n b^n c^m d^m \mid n \geq 1, m \geq 1\}$

(ii) $L = \{0^n 1^{2n} \mid n \geq 1\}$ **6M**

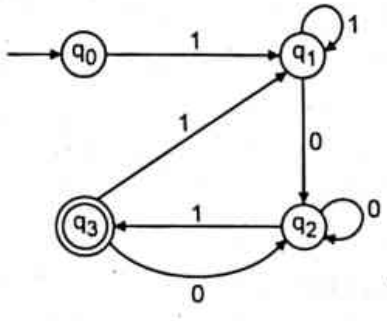
Ans. P.3-61, 63, Q.102,110.

Q.6. Construct a CFG for the language $L = \{a^{3n} \mid n \geq 1\}$ and convert the CFG into CNF. **7M**

Ans. P.3-84, Q.171

SUMMER - 10 (CS)

Q.1. Find the regular expression for the finite automata M given below : **6M**



Ans. P.3-48, Q.70.

Q.2. Convert the following grammar into GNF :

(i) $S \rightarrow XY1 \mid 0, \quad X \rightarrow 00X \mid Y, \quad Y \rightarrow 1X1$

(ii) $S \rightarrow 01 \mid 0S \mid 00S$ **7M**

Ans. P.3-93, 94, Q.183, 184

Q.3. Find a grammar generating :

$L = \{a^n b^n c^i \mid n \geq 1, i \geq 0\}$. **6M**

Ans. P.3-63, Q.110

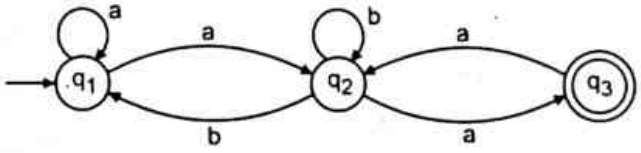
Q.4. Show that the set $L = \{a^{i^2} \mid i \geq 1\}$ is not regular. **7M**

Ans. P.3-54, Q.85.

SUMMER - 10 (CT)

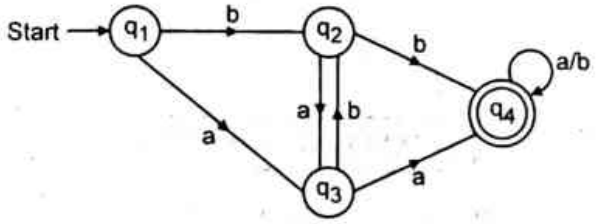
Q.1. Show that the given automata recognizes the regular expression.

$(a + a(b + aa)^* b)^* a(b + aa)^* a$ **7M**



Ans. P.3-47, Q.69.

Q.2. Write a regular expression for the language accepted by the given finite automata. **7M**



Ans. P.3-50, Q.76.

Q.3. Construct CFG for the language

$L = \{a^{3n} \mid n \geq 1\}$

and convert that CFG into CNF. **7M**

Ans. P.3-84, Q.171

Q.4. Get the optimized DFA for the regular expression defined by the grammar.

$S \rightarrow a\,A \mid b\,A$

$A \rightarrow 0\,A \mid 1\,A \mid b\,A \mid \epsilon$ **6M**

Ans. P.3-38, Q.45.

Q.5. Obtain an equivalent left linear grammar from the given right linear grammar.

$S \rightarrow 01A$

$A \rightarrow 10A \mid 1$ **7M**

Ans. P.3-43, Q.57

Q.6. Find the useless symbols in the following grammar and modify the grammar to obtain the grammar that has no useless symbol.

$S \rightarrow 0 \mid A$

$A \rightarrow AB$

$B \rightarrow 1$ **6M**

Ans. P.3-75, Q. 145

WINTER - 10 (CS)

No question asked in University exam.

WINTER - 10 (CT)

Q.1. Show that $L = \{a^P \mid P \text{ is a prime}\}$ is not regular. **5M**

Ans. P.3-53, Q.83.

Q.2. Construct a regular grammar G generating the regular set represented by regular expression.

$a*b(a+b)*$ **5M**

Ans. P.3-37, Q.43.

Q.3. Give the properties of regular set. **3M**

Ans. P.3-59, Q.98.

Q.4. Give regular expressions and transition diagram for finite automata for the following regular sets of string over $\Sigma = \{a, b\}$

(i) All strings beginning and ending with a.

(ii) All strings with two consecutive a.

(iii) All strings containing the substring aba. **6M**

Ans. P.3-19, Q.23.

Q.5. Reduce the given grammar by eliminating useless symbols.

$S \rightarrow aAa$

$A \rightarrow Sb \mid bCC \mid DaA \mid \epsilon$

$C \rightarrow DD \mid CC \mid abb$

$E \rightarrow aC$

$D \rightarrow aDA$ **4M**

Ans. P.3-73, Q.138.

Q.6. Construct context free grammar for the language.

(i) $L = \{a^n b^n c^m d^m \mid n \geq 1, m \geq 1\}$

(ii) $L = \{a^n b^m c^m d^n \mid n \geq 1, m \geq 1\}$ **4M**

Ans. P.3-63, Q.110.

Q.7. Construct a grammar in GNF equivalent to the grammar.

$S \rightarrow AA \mid a$

$A \rightarrow SS \mid b$ **6M**

Ans. P.3-89, Q.178.

SUMMER - 11 (CS)

Q.1. Simplify the regular expression :

$R = \epsilon + 1*(011)*(1*(011)*)*$ **2M**

Ans. P.3-36, Q.39

Q.2. Construct regular grammar generating regular set

$a*b(a+b)*$ **4M**

Ans. P.3-37, Q.43.

Q.3. Reduce the following grammar :

$S \rightarrow AaA$

$A \rightarrow Sb \mid bCC \mid \epsilon$

$C \rightarrow CC \mid abb$ **6M**

Ans. P.3-77, Q.150.

Q.4. What is meant by ambiguous and unambiguous grammar? Explain with example. **2M**

Ans. P.3-66, Q.120.

Q.5. Convert the following grammar to GNF :

$S \rightarrow AB$

$A \rightarrow BS \mid b$

$B \rightarrow SA \mid a$ **8M**

Ans. P.3-88, Q.177.

Q.6. Describe language generated by grammar :

$S \rightarrow aAB$

$A \rightarrow bBb$

$B \rightarrow A \mid \epsilon$

Give the definition for a derivation tree and give an example. **4M**

Ans. P.3-65, 80, Q.119, 157.

Q.7. Convert the following grammar into CNF :

$S \rightarrow aA \mid a \mid b \mid c$

$A \rightarrow aB \mid e$

$B \rightarrow aA$

$C \rightarrow cCD$

$D \rightarrow abd$ **7M**

Ans. P.3-84, Q.169

Q.8. Explain the closure properties of regular set. **2M**

Ans. P.3-59, Q.98.

Q.9. Construct minimum state automata for the regular expression given by $10 + (0+11)0*1$. **8M**

Ans. P.3-18, Q.22.

Q.10. Explain the significance of pumping lemma. Using pumping lemma prove that following language is not regular $\alpha = [a^m b^n \mid m \neq n]$. **5M**

Ans. P.3-53, 57, Q.82 & 90.

SUMMER - 11 (CT)

Q.1. Construct NFA for the following regular expression : $(a + b)^*$ abb.

Convert the resultant NFA into its equivalent DFA and then obtain optimized DFA. **8M**

Ans. P.3-22, Q.28

Q.2. Prove $(a + b)^* = a^* (ba^*)^*$ by showing the automatas for L.H.S. r.e. and R.H.S. r.e. are equivalent. **6M**

Ans. P.3-14, Q.12

Q.3. State the principles of pumping lemma. Show $\{a^n b^n \mid n \geq 1\}$ is not regular by using pumping lemma. **7M**

Ans. P.3-52, 54, Q.80, 84.

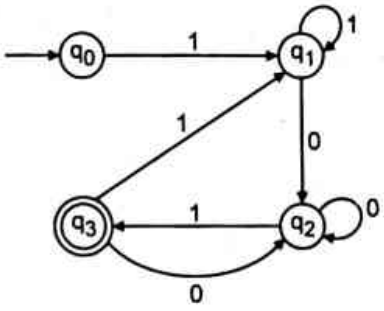
Q.4. Construct a finite automaton recognizing L(G). where G is the grammar :

$S \rightarrow aA \mid bA \mid b$

$A \rightarrow aA \mid bS \mid a$. **5M**

Ans. P.3-38, Q.46.

Q.5 Find the regular expression corresponding to the automaton given in following figure. **7M**



Ans. P.3-48, Q.70.

Q.6. Show that the grammar :

$S \rightarrow a \mid abSb \mid aAb$

$A \rightarrow bS \mid aAAb$ is ambiguous. **4M**

Ans. P.3-66, Q.121(i).

Q.7. Give CFG to accept the following languages :

(i) $L = \{WCW^R \mid W \in \{a, b\}^*\}$

(ii) $L = \{a^m b^n c^{m+n} \mid m, n \geq 1\}$. **6M**

Ans. P.3-64, 65, Q.114, 118

Q.8. Construct a grammar in Greibach normal form equivalent to the grammar :

$S \rightarrow AA \mid a$

$A \rightarrow SS \mid b$. **7M**

Ans. P.3-89, Q.178.

WINTER - 11 (CS)

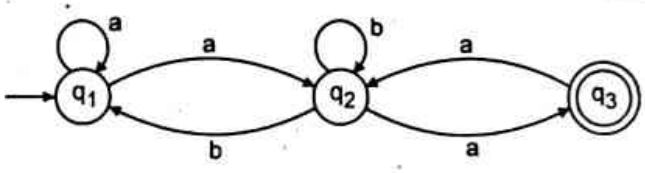
Q.1. Represent the following by regular expression :

(i) $\{1^{2n+1} \mid n > 0\}$

(ii) The set of all strings over {0, 1} which has at most two zeros. **6M**

Ans. P.3-20, Q.24

Q.2. Find the regular expression for the following automaton. **7M**



Ans. P.3-47, Q.69

Q.3. Show that $L = \{a^{n!}, n \geq 0\}$ is not regular. **5M**

Ans. P.3-57, Q.92

Q.4. Construct a grammar in Greibach normal form equivalent to the grammar $G = (\{S, A\}, \{a, b\}, P, S)$ where P is.

$S \rightarrow AA \mid a$

$A \rightarrow SS \mid b$ **7M**

Ans. P.3-89, Q.178.

Q.5. Show that the following grammar is ambiguous :

$S \rightarrow SbS \mid a$ **3M**

Ans. P.3-72, Q.135

Q.6. For the following right linear grammar, obtain equivalent left linear grammar :

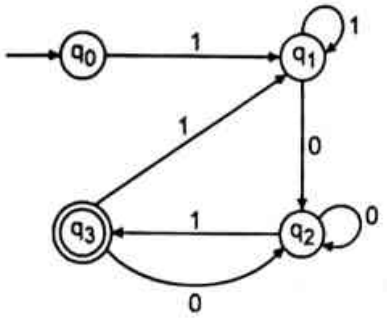
$S \rightarrow 01B \mid 0$

$B \rightarrow 1B \mid 11$ **4M**

Ans. P.3-45, Q.63

WINTER - 11 (CT)

Q.1. Find regular expression corresponding to automata given in figure. **7M**



Ans. P.3-48, Q.70.

Q.2. Using the Pumping Lemma show that the following language is not regular.

$\alpha = \{a^n b^{2n} \mid n > 0\}$ **5M**

Ans. P.3-56, Q.89.

Q.3. Explain the properties of Regular Set. **3M**

Ans. P.3-59, Q.98.

Q.4. Construct a reduced grammar equivalent to grammar.

$S \rightarrow aAa$

$A \rightarrow Sb \mid bCC \mid DaA$

$C \rightarrow abb \mid DD$

$E \rightarrow aC$

$D \rightarrow aDA$. **5M**

Ans. P.3-73, Q.138.

Q.5. Convert the following grammar into GNF.

$S \rightarrow AB$

$A \rightarrow BS \mid b$

$B \rightarrow SA \mid a$. **8M**

Ans. P.3-88, Q.177.

SUMMER - 12 (CS)

Q.1. Write regular expression for

(i) All strings of 0's and 1's with at most one pair of consecutive 0's and at most one pair of consecutive 1's.

(ii) All strings of 0's and 1's not containing 101 as a substring. **6M**

Ans. P.3-14, Q.14.

Q.2. Construct finite automata equivalent to the following regular expression :

$10 + (0 + 11)\,0^* 1$ **7M**

Ans. P.3-18, Q.22.

Q.3. Write context tree grammar for following languages:

(i) $L = \{0^m 1^n 0^{m+n} / m, n \geq 1\}$

(ii) $L = \{a^n b^n c^k : k \geq 3, n \geq 1\}$

(iii) $L = \{0^a 1^b 2^c : |a - b| = c; a, b, c > 0\}$ **6M**

Ans. P.3-65, Q.116

Q.4. Simplify following CFG :

$S \rightarrow aA \mid aBB$

$A \rightarrow aaA \mid \in$

$B \rightarrow bB \mid bbC$

$C \rightarrow B$ **4M**

Ans. P.3-80, Q.156

Q.5. Show that the following grammar is ambiguous :

$S \rightarrow aSbS \mid bSaS \mid \in$ **3M**

Ans. P.3-69, Q.127.

Q.6. Convert the grammar :

$S \rightarrow ABb \mid a$

$A \rightarrow aaA \mid B$

$B \rightarrow bAb$ into GNF and CNF. **4M**

Ans. P.3-95, Q.187

Q.7. Prove that $\{0^i 1^m 2^m / i \geq 1, m \geq 1\}$ is not regular using pumping lemma. **5M**

Ans. P.3-58, Q.94

Q.8. Obtain equivalent left linear grammar for following :

$S \rightarrow abA \mid aa$

$A \rightarrow aA \mid bb$ **5M**

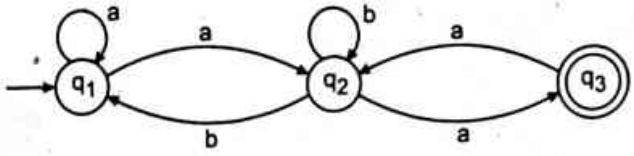
Ans. P.3-43, Q.59

SUMMER - 12 (CT)

Q.1. State the principles of Pumping Lemma for Regular Language. Prove that $L = \{a^i.b^{2i} \mid i \geq 1\}$ is non regular by using Pumping Lemma. **7M**

Ans. P.3-52, 56, Q.80 & 89.

Q.2. Find the equivalent Regular expression for the automaton given below : **6M**



Ans. P.3-47, Q.69.

Q.3. Check whether the following grammar is ambiguous or not. If found ambiguous, remove the ambiguity and write an equivalent unambiguous grammar :

E → E + E

E → E * E

E → (E)

E → id. **7M**

Ans. P.3-68, Q.125.

Q.4. Obtain Right linear grammar equivalent to Left linear grammar given below :

S → B1 | A00 | C0

A → C0 | A1 | B1 | 0

B → B11 | 1

C → A0. **6M**

Ans. P.3-45, Q.62

Q.5. Construct a grammar in GNF equivalent to the grammar given below :

S → AA | a

A → SS | b. **7M**

Ans. P.3-89, Q.178.

WINTER - 12 (CS)

Q.1. Answer the following **2M**

(a) The regular expression for language with at least one 1, 2 and 3 is

(i) 1 + 2 + 3

(ii) 11 * 22 * 33 *

(iii) 1 * 2 * 3

(iv) None of above.

Ans. P.3-16, Q.18

(b) Regular expression (a/b) (a/b) denotes the set.

(i) a/b

(ii) (ab)*

(iii) (a/b)*

(iv) none of above.

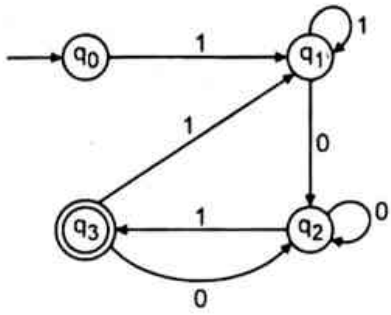
Ans. P.3-16, Q.18

Q.2. Construct grammar for language :

$L\{a^n b^n c^n d^n / n \geq 0\}$ and then determine the type of this language. **5M**

Ans. P.3-65, Q.117

Q.3. Find the regular expression corresponding to automaton given in fig. **5M**



Ans. P.3-48, Q.70

Q.4. Find regular expression for the language on {a, b} defined as :

L = {W : | W | mode 3 = 0} **3M**

Ans. P.3-14, Q.13

Q.5. Construct a reduced grammar equivalent to the grammar :

S → aAa

A → Sb | bCC | DaA

C → abb | DD

E → aC

D → aDa **4M**

Ans. P.3-73, Q.138

Q.6. Using pumping lemma, prove that the language :

$L = \{a^{i^3} / i \geq 1\}$ is not regular. **5M**

Ans. P.3-54, Q.85

Q.7. What is an ambiguous grammar? Check ambiguity of following grammar.

S → a SSb | bSSa | ∈ **4M**

Ans. P.3-66, 68, Q.120, 127.

Q.8. Find the grammar in GNF equivalent to the grammar :

E → E + T / T

T → T * F / F

F → (E) / a **3M**

Ans. P.3-93, Q.182.

Q.9. Obtain right linear grammar equivalent to following left linear grammar :

S → Sab | Aa

A → Abb | bb **6M**

Ans. P.3-41, Q.52

WINTER - 12 (CT)

Q.1. Prove or disprove : $(a + b)^* = a^* + b^*$. **3M**

Ans. P.3-14, Q.10.

Q.2. What are the steps for converting Left Linear Grammar (LLG) into its equivalent Right Linear Grammar (RLG). Convert the following LLG into RLG :

$S \rightarrow Bl$

$B \rightarrow Cl / B0 / l$

$C \rightarrow l$ **7M**

Ans. P.3-39, 43, Q.48 & 60.

Q.3. Show that the given language is not regular using Pumping-Lemma

$L = \{0^i 1^j 0^k / k > i + j\}$. **6M**

Ans. P.3-57, Q.93.

Q.4. What is Leftmost and Rightmost Derivation? Explain each with an example. **6M**

Ans. P.3-70, Q.131.

Q.5. Convert the following Grammar into GNF :

$S \rightarrow ABa/b$

$A \rightarrow bbA/B$

$B \rightarrow aAa$ **7M**

Ans. P.3-84, Q.170

Q.6. Find whether the given CFG is ambiguous or not?

$S \rightarrow aSbS / bSaS / \in$ **5M**

Ans. P.3-10, Q.127.

SUMMER - 13 (CS)

Q.1. Obtain deterministic finite automata for following regular expression :

$(0 + 1)^* 10 (0 + 1)^* + (0 + 1)^* 11 (0 + 1)^*$ **8M**

Ans. P.3-22, Q.29

Q.2. Write regular expression to represent all string of 0 and 1 not containing 101 as a substring. **3M**

Ans. P.3-12, Q.3.

Q.3. Check validity of string "(((a, a), ^, a))" for context free grammar as :

$S \rightarrow (L) | a | \wedge$

$L \rightarrow L, S | S$

Also draw parse tree. **3M**

Ans. P.3-70, Q.130

Q.4. Obtain right linear grammar for following :

$S \rightarrow aa\ A | bc$

$A \rightarrow bb\ B | b$

$B \rightarrow cA | c$ **8M**

Ans. P.3-43, Q.58

Q.5. Eliminate left recursion from the following grammar.

$L \rightarrow L, A | A$

$A \rightarrow a$ **2M**

Ans. P.3-79, Q.153

Q.6. Obtain equivalent CNF and GNF for following CFG :

$S \rightarrow AB | 0C2$

$A \rightarrow 0A1 | 01$

$B \rightarrow 1B2 | 12$

$C \rightarrow 0C2 | 0D1 | 01$

$D \rightarrow 0D1 | 01$ **6M**

Ans. P.3-87, Q.176

Q.7. Eliminate $\in$- production for following :

$S \rightarrow aSB | aA | bB$

$A \rightarrow aA | \in$

$B \rightarrow bB | \in$ **2M**

Ans. P.3-77, Q.149

Q.8. Check whether the given grammar is ambiguous?

$S \rightarrow i\ CtS | iCtSeS | a$

$C \rightarrow b$ **2M**

Ans. P.3-67, Q.122(ii)

Q.9. State and prove pumping lemma for regular sets. **6M**

Ans. P.3-52, Q.80

SUMMER – 13 (CT)

Q.1. Obtain DFA for following regular expression:

$RE = \in + 1^* (011)^* (1^*(011)^*)^*$. **6M**

Ans. P.3-18, Q.21(b)

Q.2. Design CFG for following languages :

(i) $L = \{a^m b a^m : m \geq 0\}$

(ii) $L = \{a^n b^n c^k : n \geq 0, k \geq 3\}$

(iii) $L = \{a^n b^m c^k : k = |n - m|\}$ **6M**

Ans. P.3-65, Q.116

Q.3. Derive the string "aabbbb" using grammar :

$S \rightarrow AB \mid \epsilon$

$A \rightarrow aB$

$B \rightarrow Sb$

Draw parse tree using leftmost and rightmost derivation. **4M**

Ans. P.3-69, Q.129

Q.4. Check whether following grammar is ambiguous:

$S \rightarrow icScS \mid icS \mid a$ **3M**

Ans. P.3-66, Q.121(iii).

Q.5. Obtain CNF and GNF for grammar :

$S \rightarrow ABb \mid a$

$A \rightarrow aaA \mid B$

$B \rightarrow bAb$ **5M**

Ans. P.3-84, Q.170

Q.6. Using pumping lemma prove that :

$L = \{a^n b^m c^n \mid m, n \geq 1\}$ is not regular. **5M**

Ans. P.3-58, Q.95.

SOLVED QUESTION BANK

[Sequence given as per syllabus]

INTRODUCTION

The languages accepted by finite automata are easily described by simple expressions called regular expressions. In this section, we introduce the operations of concatenation and closure on sets of strings, define regular expressions, and prove that the class of languages accepted by finite automata is precisely the class of languages describable by regular expressions.

The construction of automata from regular expressions is an application of closure properties. Also the equivalence of DFA's and regular expressions are also considered. Every regular expression will represent a regular language and the automaton formed corresponding to a regular expression is a deterministic finite automation. Pumping lemma is necessary condition for the language to be regular.

REGULAR EXPRESSION

Q.1. What is regular expression?

Ans. Regular Expression :

The language accepted by finite automata can be represented by some expression called as regular expression.

Formal Definition :

The class of language over Σ can be defined recursively as

1) ϕ (Null set) and ϵ (empty string) are always regular expression.
2) If $\Sigma = \{a\}$, then a is also regular expression.
3) If R_1 and R_2 are regular expression over Σ, then the combination $(R_1 + R_2)$ (union), $(R_1 . R_2)$ (series) and R_1^* or R_2^* (closure) are also regular expression.
4) Any expression formed using above points (1), (2) and (3) are also regular expression.

Q.2. Write a regular expression containing 011, preceded and followed by any number of zero's and 1's over $\Sigma = \{0, 1\}$. ***CT : S-08(3M)***

Ans. Any number of zeros and one's = $(0 + 1)^*$

Required Regular Expression = $(0 + 1)^* . 011 . (0 + 1)^*$

Q.3. Represent the following sets by regular expressions :

(i) The set of all strings over {0, 1} which has atmost two 1's.

(ii) The set of all strings over {a, b} beginning and ending with b.

(iii) The set of all strings not containing 101 as a substring. ***CS : S-13(3M)***

Ans.

(i) At most two 1's means the string contains maximum of two 1's or one 1 or zero one's

$\therefore$ Regular Expression = $0^* 1 . 0^* . 1 . 0^* + 0^* 10^* + 0^*$

(ii) Regular Expression = b . (a + b) * . b.

(iii) Regular Expression = 0 . 1 *1 *. 0 + 1 *. 0.01 * 0.

Q.4. Give regular expression for representing the set of all words begin and end with 'Z' and in between any word using 'Y'.

Ans. Regular Expression = Z (A + B + C + ---- + Z)* . Y (A + B + C + ------- + Z)* . Z

- Since every word begin and end with Z, there will be Z in beginning and ending with Z.
- In between these two Z, the word may contain any combination of A to Z using Y as given above.

Q.5. Give regular expression for representing the set L of string in which every zero is immediately followed by at least two ones. ***CT : W-06(5M)***

Ans.

- At least two 1's means two or more one's given by 11.1^*
- String consist of every zero that is immediately followed by at least two ones.

Hence,

Regular Expression = $(0111^* + 1011)^*$

Q.6. Write regular expression for identifier and signed decimal constant in 'C'.

Ans.

- Valid identifiers in 'C' are the sequence of a 'letter' followed by any number of 'letters' or 'digits'.
- The regular expression for identifiers can be written as.

  Regular expression = (Letter) . (Letter / digit) *

  where,

  Letter = A, B, --------- z, a, b, --------- z

  and digit = 0, 1, -------- 9
- Signed decimal constant is represented by sign followed any numbers of decimal numbers.

  The regular expression = (sign) . $(\text{decimal})^+$

  where,

  sign = + | –

  decimal = 0, 1, 2, --------- 9

  $(\text{decimal})^+ = (\text{decimal}) . (\text{decimal})^+$

Q.7. State true or false with justification.

(i) $(a + b)^* = (a + b) + (a + b)^*$.

(ii) $(a^* b^*) = (a + b)^*$.

Ans.

(i) **$(a + b)^* = (a + b) + (a + b)^*$:**

True

Justification :

Let $R_1 = (a + b)^*$

$L(R_1) = \{\in, a, b, aa, bb, ab, ba, ------\}$ ....(1)

Let $R_2 = \underset{\downarrow}{(a + b)} + \underset{\downarrow}{(a + b)^*}$

p q

$L(p) = \{a, b\}$

$L(q) = \{\in, a, b, aa, bb, ab, ba, ----\}$

$L(R_2) = L(p) \cup L(q)$

$= \{\in, a, b, aa, bb, ab, ba, -------\}$ ...(2)

From (1) & (2)

$(a + b)^* = (a + b) + (a + b)^*$

(ii) **$(a^* b^*) = (a + b)^*$:**

False

Justification :

Let $R_1 = (a^* . b^*)$

$= \{\in, a, b, aa, bb, abb, bbb, ---\}$ ....(1)

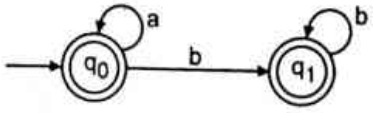


Fig. DFA

Let $R_2 = (a + b)^*$

$= \{\in. a, b, ab, aa, bb, ba, ------\}$ ....(2)

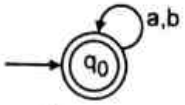


Fig. DFA

From (1) and (2),

$(a^* . b^*) \neq (a + b)^*$

Q.8. State true or false with justification

(i) $(a + b)^* = (a + b)^* + (a + b)^*$.

(ii) $(a^* b^*)^* = (a + b)^*$.

Ans.

(i) $(a + b)^* = (a + b)^* + (a + b)^*$:

True

Justification :

Let $R_1 = (a + b)^*$

$L(R_1) = \{\in, a, b, aa, bb, ab, ba, ------\}$ ....(1)

$R_2 = \underbrace{(a+b)^*}_{P} + \underbrace{(a+b)^*}_{Q}$

$L(P) = \{\in, a, b, ab, ba, aa, bb, -------\}$

$L(Q) = \{\in, a, b, ab, ba, aa, bb, ------\}$

$\therefore L(R) = L(P) \cup L(Q)$

$= \{\in, a, b, ab, ba, aa, bb, -----\} \cup$

$\{\in, a, b, ab, ba, aa, ba, ------\}$

$= \{\in, a, b, ab, ba, aa, ba, -------\}$ ....(2)

From (1) and (2)

$(a + b)^* = (a + b)^* + (a + b)^*$

(ii) $(a^* b^*)^* = (a + b)^*$:

True

Justification :

Let $R_1 = (a^* . b^*)^*$ &

$R_2 = (a + b)^*$

$\therefore L(R_1) = (\{\in, a, aa, -----\} \cup \{\in, b, bb, -----\})^*$

$= (\{\in, a, b, aa, bb, -------\})^*$

$= (\{\in, a, b, aa, bb, ab, ba, -----\})$ ....(1)

$L(R_2) = (\{\in, a, b, aa, bb, ab, ba, -----\})$ ....(2)

From (1) and (2),

$(a^* . b^*)^* = (a + b)^*$

Q.9. State truth/falsity of the following. Justify your answer.

(i) $(a . b)^* = a^* . b^*$ (ii) $(ab)^*\ a = a\ (ba)^*$

Ans.

(i) $(a . b)^* = a^* . b^*$:

False

Justification :

Let $R_1 = (a . b)^*$

$L(R_1) = \{\in, ab, abab, ababab, ------\}$ ....(1)

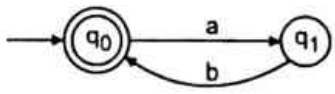


Fig. DFA

$R_2 = a^* . b^*$

$L(R_2) = \{\in, a, aa, aaa, ----\} . \{\in, b, bb, bbb----\}$

$= \{\in, a, b, aa, bb, aaa, bbb, ab, abb------\}$

....(2)

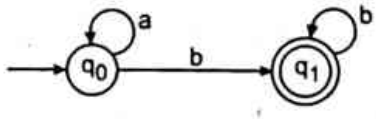


Fig. DFA

From (1) and (2)

$(a . b)^* \neq a^* . b^*$

(ii) $(ab)^*\ a = a\ (ba)^*$:

True

Justification :

Let $R_1 = (ab)^* . a$

$L(R_1) = \{a, aba, ababa, abababa, ------\}$ ....(1)

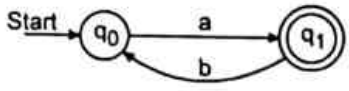


Fig. DFA

Let $R_2 = \{a. (ba)^*$

$L(R_2) = \{a, aba, ababa, ababa ------\}$ ....(2)

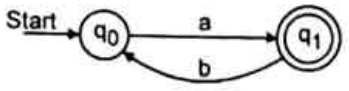


Fig. DFA

From (1) and (2)

$(ab)^* . a = a . (ab)^*$

Q.10. Check whether the following language is identical.

$(a^* + b^*)$ and $(a + b)^*$ **CT : W-12(3M)**

Ans. Let $R_1 = (a^* + b^*)$

$\therefore L(R_1) = \{\in, a, aa, bb, aaa, bbb, ------\}$ ....(1)

$R_2 = (a + b)^*$

$L(R_2) = (\{\in, a, b, aa, bb, ab, ba, -----\})$ ....(2)

From (1) and (2),

$(a^* + b^*)^* \neq (a + b)^*$

i.e. Not Identical.

Q.11. Prove the following identities for regular expression a and b :

(i) $(a^* b^*)^* = (a + b)^*$

(ii) $(a^* ab + ba)^* a^* = (a + ab + ba)^*$

Ans.

(i) $(a^* b^*)^* = (a + b)^*$:

True

Justification :

Let $R_1 = (a^* . b^*)^*$ &

$R_2 = (a + b)^*$

$\therefore L(R_1) = (\{\in, a, aa, -----\} \cup \{\in, b, bb, -----\})^*$

$= (\{\in, a, b, aa, bb, -------\})^*$

$= (\{\in, a\ b, aa, bb, ab, ba, -----\})$ ....(1)

$L(R_2) = (\{\in, a, b, aa, bb, ab, ba, -----\})$ ....(2)

From (1) and (2),

$(a^* . b^*)^* = (a + b)^*$

(ii)

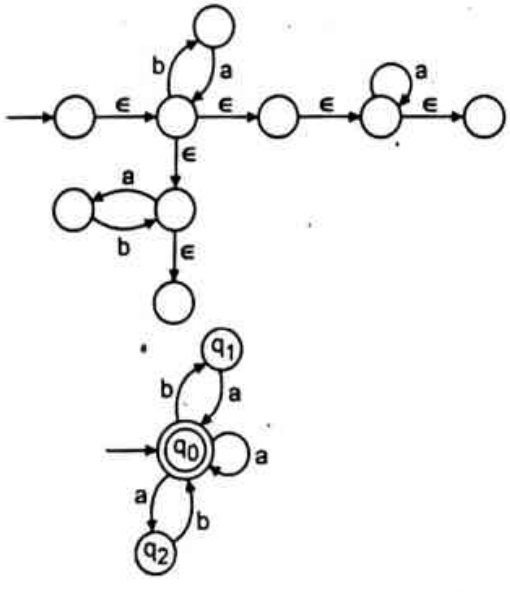


Fig.1 Finite automata for $(a^* ab + ba)^* . a^*$

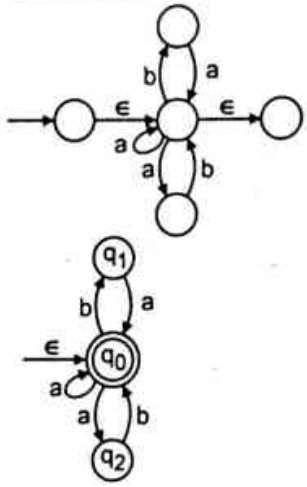


Fig.2 Finite automata for $(a + ab + ba)^*$

From Fig. (1) and (2),

Finite automata for both regular expressions are equivalent.

Hence,

$(a^* ab + ba)^* . a^* = (a + ab + ba)^*$.

Q.12. Prove $(a + b)^* = a^* (ba^*)^*$ by showing the automata for L.H.S. regular expression and R.H.S. regular expression are equivalent.

CT : S-11(6M)

Ans. Let $R_1 = (a + b)^*$

$L(R_1) = \{\in, a, b, aa, bb, ab, ba, aaa, bbb, ---\}$ ....(1)

$R_2 = a^* (ba^*)^*$

$L(R_2) = \{\in, a, b, aa, bb, ab, ba, aaa, bbb, --\}$ ....(2)

$\therefore$ From equation (1) and (2)

$(a + b)^* = a^* (ba^*)^*$

Q.13. Find regular expression for language on (a, b) defined as.

$L = \{W : |W| \bmod 3 = 0\}$. **CS : W-12(3M)**

Ans. The regular expression for the language on {a, b} containing all string in which length that are multiple of three is given by

Regular Expression $= (aaa + bbb)^*$

Q.14. Write regular expression for.

(a) All string of 0's and 1's with at most one pair of consecutive 0's and at most one pair of consecutive 1's

(b) All string of 0's and 1's not containing 101 as a substring. **CS : S-12(6M)**

Ans.

(a) Regular Expression = (000* 111* + 111* 000*)*

(b) Regular Expression = 01* 1*0 + 1*0.0*

Q.15. Prove or disprove the following for regular expressions :

(i) (rs + r)* r = r (sr + r)*

(ii) (r + s)* = r* + s* **CT : S-07(4M)**

Ans.

(i) (rs + r)* r = r (sr + r)* :

Let $R_1 = (rs + r)^* . r$

$\therefore L(R_1) = \{r, rsr, rr, rsrr, rrsr, rsrsr, ...\}$ ....(1)

Let $R_2 = r(sr + r)^*$

$L(R_2) = \{r, rsr, rr, rsrr, rsrsr, ...\}$ ....(2)

From (1) and (2)

$(rs + r)^* . r = r(sr + r)^*$

(ii) (r + s)* = r* + s* :

Let $R_1 = (r + s)^*$

$\therefore L(R_1) = \{\epsilon, r, s, rr, ss, rs, sr, rrr, ...\}$ ....(1)

and $R_2 = r^* + s^*$

$L(R_2) = \{\epsilon, r, rr, rrr, ..\} \cup \{\epsilon, s, ss, sss, ...\}$

$= \{\epsilon, r, s, rr, ss, rrrr, sss, ...\}$ ....(2)

From (1) and (2)

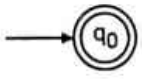
$(r + s)^* \neq r^* + s^*$.

Q.16. Explain how to convert regular expression to finite automata, give example.

Ans.

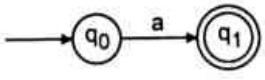
Step 1 : Construct a transition graph of finite automata equivalent to the given regular expression using ϵ-moves. This is done as follows.

(1) R. E. = ϵ

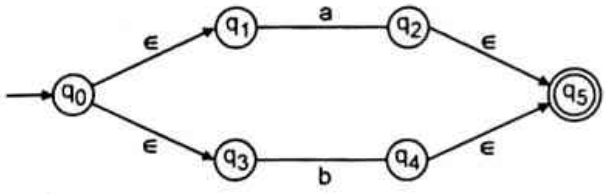


(2) R. E. = ϕ

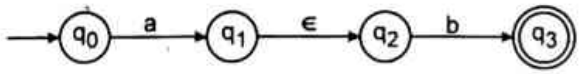
(3) R. E. = a



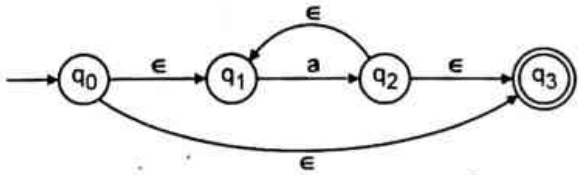
(4) R. E. = a + b



(5) R. E. = a.b



(6) R. E. = a*



Step 2 : Construct the transition table for the transition graph obtained in step 1. That is convert the NFA with ϵ-moves to DFA. Then minimize the DFA if possible.

Q.17. Write short note on Application of Regular expression.

Ans. Application of Regular expression :

(1) Lexical Analyzer :

- The function of a compiler is to convert a source program written in a high-level language to a machine code.
- The first phase of the complier is lexical analyzer.
- Its job is to read the source file character by character and output the valid words called as tokens.
- To recognize a token from a text file or source program the lexical analyzer uses regular expression.

(2) Text Editors :

- Certain text editors and similar programs have some provision to substitute other string for any string matching a particular regular expression.
- "Find and replace" is the common name used for such substitutions which can replace every occurance of the given string.

- This searching of every occurance through given file is simpler with the help of regular expression.
- Unix text editor uses this method. Text editor does not use regular expression directly but converts it to an NFA with ∈-transitions and the NFA is then simulated directly.

Q.18. Answer the following: **CS : W-12(2M)**

(a) The regular expression for language with atleast one 1, 2, and 3 is

(1) 1 + 2 + 3
(2) 11* 22* 33*
(3) 1* 2* 3
(4) None of above

Ans. (2) 11*22*33*

(b) Regular expression (a/b) (a/b) denotes the set.

(1) a/b
(2) (ab)*
(3) (a/b)*
(4) None of above.

Ans. (3) (a/b)*

EQUIVALENCE BETWEEN REGULAR EXPRESSION AND FINITE AUTOMATA

Q.19. Construct a transition system corresponding to the regular expressions given below:

(i) (ab + c*)*b (ii) (11 + 0)* (00 + 1)*

Ans.

(i) Regular expression = (ab + c*)* . b

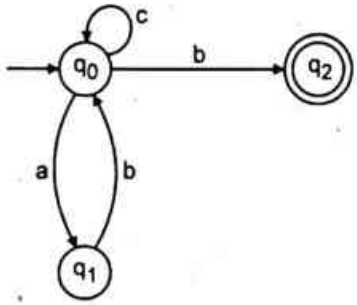


Fig. Transition system

(ii) Regular expression = (11 + 0)* (00 + 1)*.

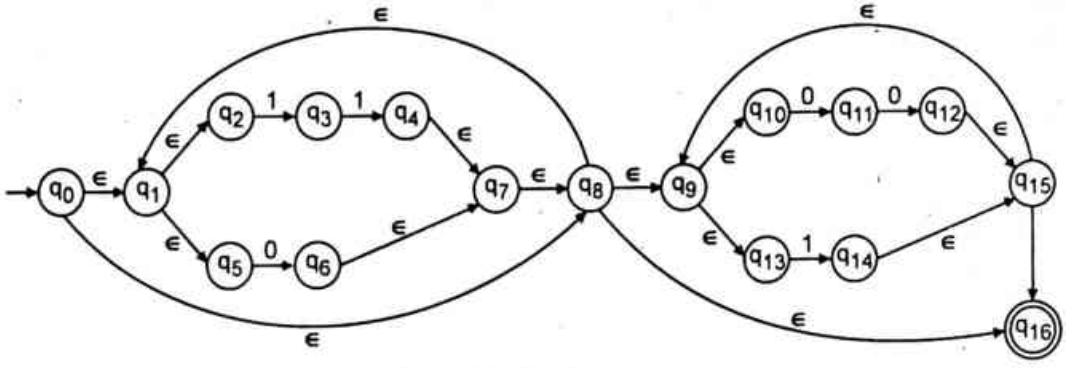


Fig. NFA with ∈-moves

Q.20. Find the minimum state DFA for the regular expression.

((1 + 0) (1 + 0) *) + ((1 + 0) (1 + 0) *)*

Ans. NFA with ∈- moves for given regular expression is

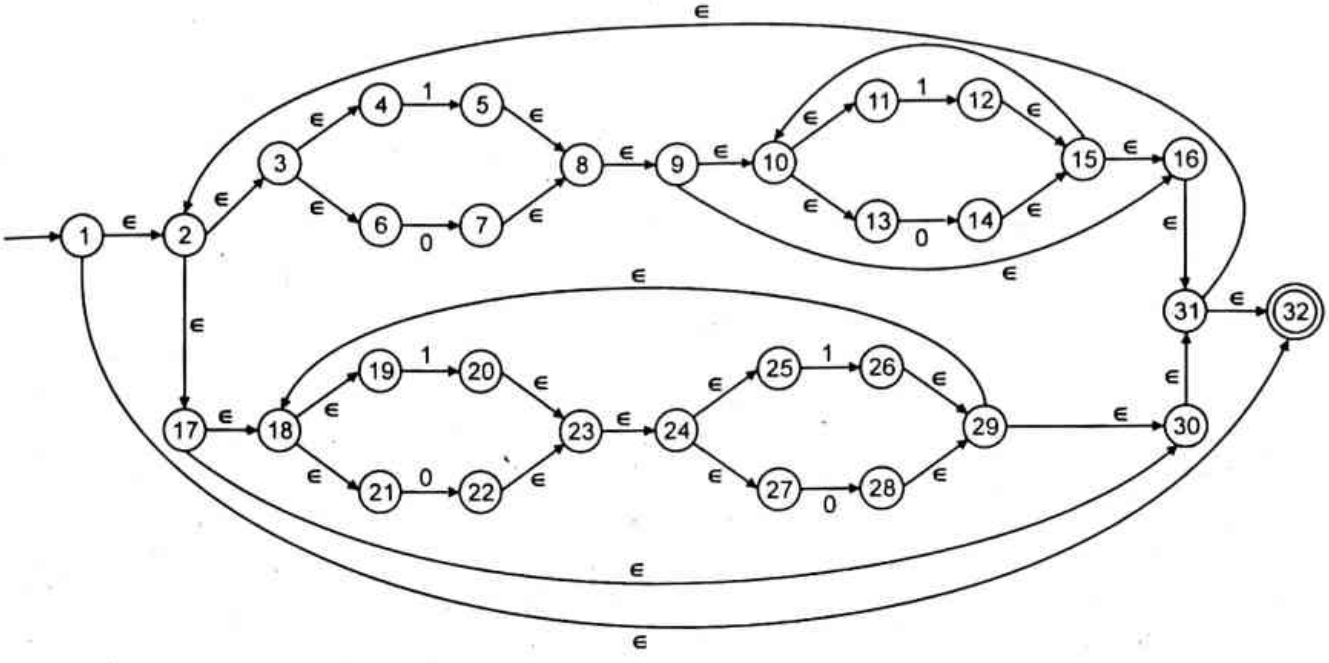


Fig. NFA with ϵ - moves

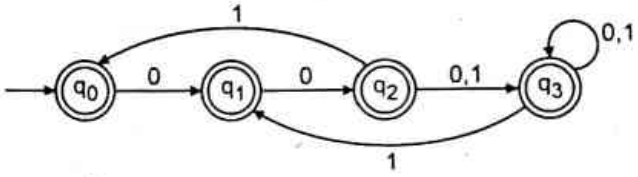


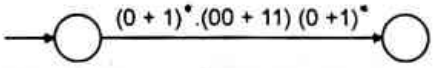
Fig. Minimized state DFA

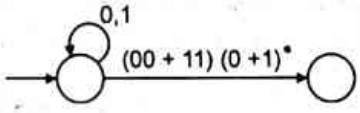
Q.21.(a) Construct DFA equivalent to the regular expression :

(0 + 1)* (00 + 11) (0 + 1)*. ***CT : W-06(4M)***

Ans. Regular Expression (R.E.)

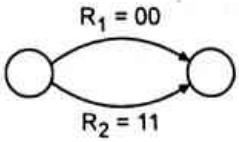
= (0 + 1)* (00 + 11) (0 + 1)*



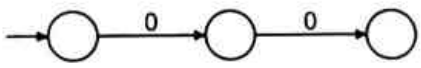


-------- (1)

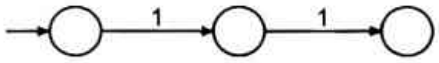
(00 + 11)



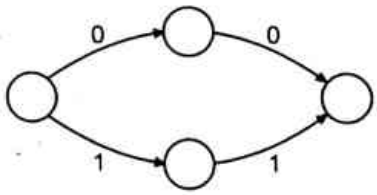
$R_1 = 00$



$R_2 = 11$



00 + 11



-------- (2)

From (1) and (2)

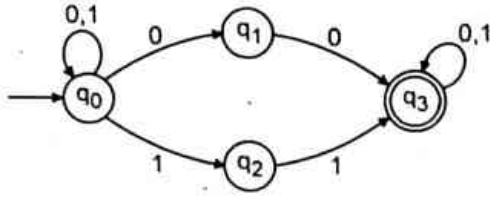


Fig. NFA for given R.E.

Transition table for NFA :

Q \ Σ	0	1
→ q_0	$\{q_0, q_1\}$	$\{q_0, q_2\}$
q_1	$\{q_3\}$	-
q_2	-	$\{q_3\}$
* q_3	$\{q_3\}$	$\{q_3\}$

Transition table for DFA :

Q \ Σ	0	1
→$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_1\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2, q_3\}$
* $\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$
* $\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$

Renaming the states as,

$\{q_0\}$ = A, $\{q_0, q_1\}$ = B,

$\{q_0, q_2\}$ = C, $\{q_0, q_1, q_3\}$ = D,

$\{q_0, q_2, q_3\}$ = E

Transition table for DFA :

Q \ Σ	0	1
→A	B	C
B	D	C
C	B	E
* D	D	E
* E	D	E

Here state D and state E both are final and equivalent. Hence replace D by E

Minimized transition table for DFA :

Q \ Σ	0	1
→A	B	C
B	E	C
C	B	E
* E	E	E

Transition Graph for DFA :

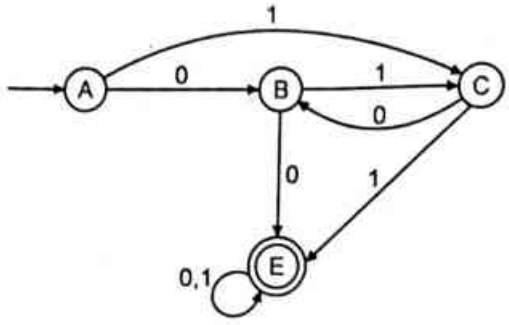


Fig. DFA

Q.21.(b) Obtain DFA for following regular expression. RE = ∈ + 1* (011)* (1*(011)*)*.

CT : S-13(6M)

Ans. RE = ∈ + 1* (011)* (1*(011)*)*

The given regular expression is in following format

R = ∈ + r r*

Considering R = 0* (011)*

This format is simplified according to identity rule

∈ + rr* = ∈ + r* r = r*

r = (0* (011)*)*

again this can be simplified using identity rule

$(r_1^* r_2^*)^*$ *[∵ $r_1 = a, r_2 = abb$]*

Using (p +q)* = (p*q*)* = (p* + q*)*

r = (0 + 011)*

Q.22. Find minimum state DFA for the regular expression

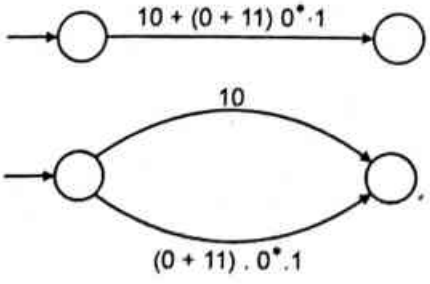
10 + (0 + 11) 0 * 1.

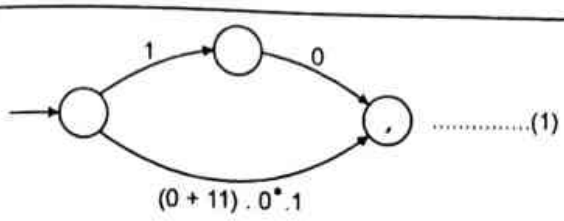
CS : S-11(8M), S-12(7M), CT : W-08(7M)

OR Construct NFA equivalent to the regular expression :

R_1 = 10 + (0 + 11) 0 * 1 **CS : S-09(5M)**

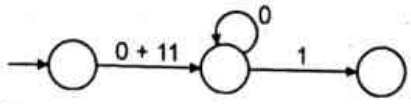
Ans. Regular Expression = 10 + (0 + 11) 0* . 1



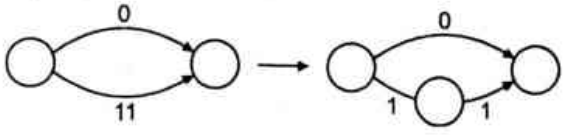


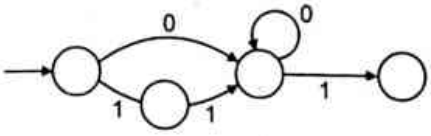
.....(1)

(0 + 11) 0* . 1



0 + 11.





..... (2)

From (1) and (2)

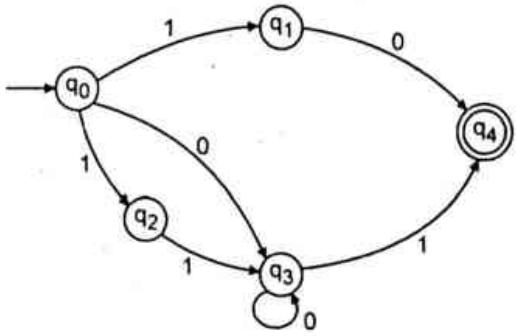


Fig. NFA

Transition table for NFA :

Q \ Σ	0	1
→ q_0	$\{q_3\}$	$\{q_1, q_2\}$
q_1	$\{q_4\}$	-
q_2	-	$\{q_3\}$
q_3	$\{q_3\}$	$\{q_4\}$
* q_4	-	-

Transition table for DFA :

Q \ Σ	0	1
→ q_0	$\{q_3\}$	$\{q_1, q_2\}$
q_3	$\{q_3\}$	$\{q_4\}$
q_1, q_2	$\{q_4\}$	$\{q_3\}$
* q_4	-	-

Renaming states as,

$\{q_0\} = A$, $\{q_3\} = B$,

$\{q_1, q_3\} = C$, $\{q_4\} = D$,

Transition table for DFA :

Q \ Σ	0	1
→A	B	C
B	B	D
C	D	B
*D	-	-

Transition graph for DFA :

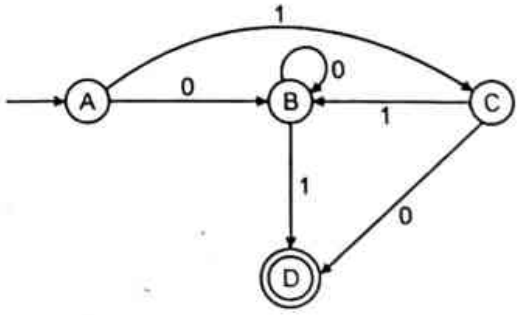


Fig. Transition graph for DFA

Q.23. Give regular expressions and transition diagram for finite automata for the following regular sets of string over alphabet {a, b}.

(i) All strings beginning and ending with a .

(ii) All strings without two consecutive a's.

(iii) All strings with odd numbers of a's and even numbers of b's.

(iv) All strings containing the substring aba.

CT : W-10(6M)

Ans.

(i) Regular Expression :

a. (a + b) * a

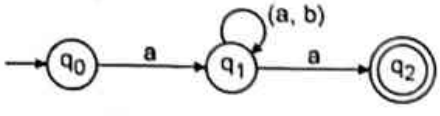


Fig. Transition diagram

(ii) Regular Expression :

(b + ba)* + (a + ba)*

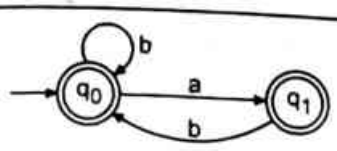


Fig. Transition diagram

(iii) Odd Number of a's = $(aa)^*$. a

Even Number of b's = $(bb)^*$

∴ **Regular Expression :**

$(aa)^*$. a $(bb)^*$

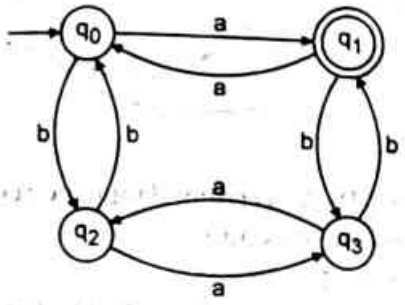


Fig. Transition diagram

(iv) Regular Expression :

$(a + b)^*$. aba . $(a + b)^*$

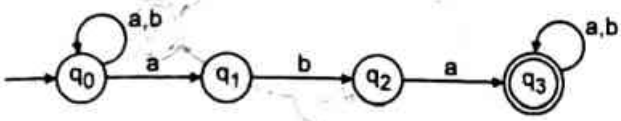


Fig. Transition diagram

Q.24. Represent the following by regular expression.

(1) $\left\{1^{2n+1} \middle| n > 0\right\}$

(2) The set of all strings over {0, 1} which has at most two zeros. ***CS : W-11(6M)***

Ans.

(1) $0^* = \{\in, 0, 00, 000 .....\}$

(2) $1^* = \{\in, 1, 11, 111 ....\}$

(3) $01^* = \{0, 01, 011, 0111, ...\}$

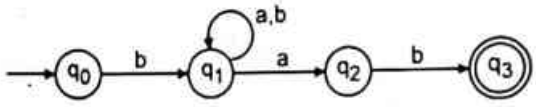
(4) $10^* = \{1, 10, 100, 1000.....\}$

- Now, the regular expression that does not contain a pair of '0's means '00'.
  Therefore, these set are
  1^* and 01^*
- Hence the R.E. that do not contain pair of 0's is $(1 + 01)^*$. Now, we can find R.E. that contain exactly one pair of 0's. For that we write the R.E. which does not contain a pair of 0's on both sides of '00'.
  Thus, the R.E. that contain at most one pair of 0's
  $(0 + 01)^* 00 (1 + 01)^*$
- Similarly the R.E. which does not contain pair of 1's i.e. '11'.
  Here the set are
  0^* and 10^*
- Hence, the R.E. that contain at most are pair of b's can be found out by placing $(0 + 10)^*$ at both sides of '11'.
  The R.E. which contain at most one pair of 1's is given by $(0 + 10)^* 11 (0 + 10)^*$.

Q.25. Construct DFA that accepts the languages over a's and b's where the strings do not start with a but ends with ab.

Ans. Regular Expression = b . $(a + b)^*$ ab

Transition graph for NFA :



Transition table for NFA :

Q \ Σ	a	b
→ q_0	-	$\{q_1\}$
q_1	$\{q_1, q_2\}$	$\{q_1\}$
q_2	-	$\{q_3\}$
* q_3	-	-

Transition table for DFA :

Q \ Σ	a	b
→ q_0	-	$\{q_1\}$
q_1	$\{q_1, q_2\}$	$\{q_1\}$
q_1, q_2	$\{q_1, q_2\}$	$\{q_1, q_3\}$
* q_1, q_3	$\{q_1, q_2\}$	$\{q_1\}$

Renaming states as,

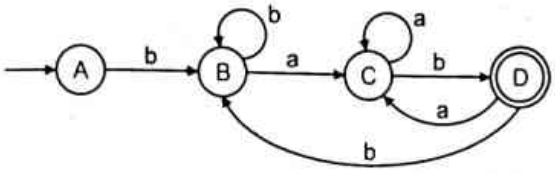
$\{q_0\} = A$, $\{q_1\} = B$

$\{q_1, q_2\} = C$, $\{q_1, q_3\} = D$

Optimized transition table for DFA :

Q \ Σ	a	b
→A	-	B
B	C	B
C	C	D
*D	C	B

Transition diagram for DFA :

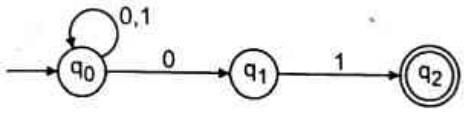


Q.26. Convert the following regular expression to DFA.

(0 | 1)* 01 ***CT : S-06(9M)***

Ans. Regular Expression = (0 | 1)* . 01

Transition graph for NFA :



Transition table for NFA :

Q \ Σ	0	1
→ q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	-	$\{q_2\}$
* q_2	-	-

Transition table for DFA :

Q \ Σ	0	1
→ $\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
* $\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$

Renaming states as,

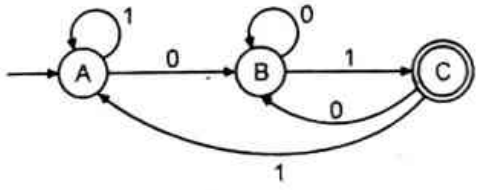
$\{q_0\}$ = A, $\{q_0, q_1\}$ = B

$\{q_0, q_2\}$ = C

Optimized transition table for DFA :

Q \ Σ	0	1
→A	B	A
B	B	C
*C	B	A

Transition diagram for DFA :

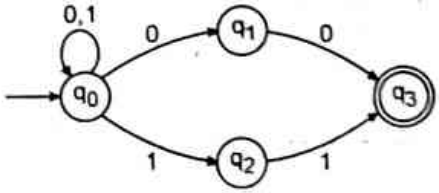


Q.27. Construct DFA accepting language represented by regular expression :

(0 / 1) * (00 / 11)

Ans. Regular Expression = (0 / 1)* (00 / 11)

Transition graph for NFA



Transition table for NFA :

Q \ Σ	0	1
→ q_0	$\{q_0, q_1\}$	$\{q_0, q_2\}$
q_1	$\{q_3\}$	-
q_2	-	$\{q_3\}$
* q_3	-	-

Transition table for DFA :

Q \ Σ	0	1
→ $\{q_0\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_1\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2, q_3\}$
* $\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_2\}$
* $\{q_0, q_2, q_3\}$	$\{q_0, q_1\}$	$\{q_0, q_2, q_3\}$

Renaming states as,

$\{q_0\}$ = A, $\{q_0, q_1\}$ = B

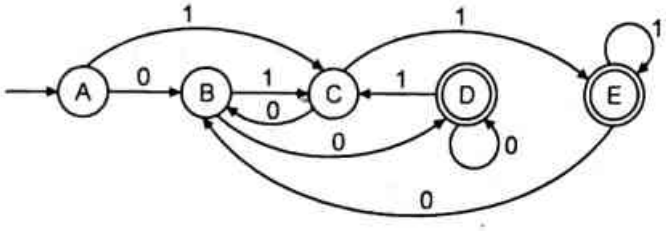
$\{q_0, q_2\} = C$ $\{q_0, q_1, q_3\} = D$

$\{q_0, q_2, q_3\} = E$

Optimized transition table for DFA :

Q \ Σ	0	1
→A	B	C
B	D	C
C	B	E
*D	D	C
*E	B	E

Transition diagram for DFA:



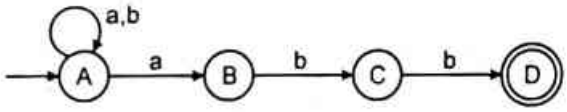
Q.28. Obtain optimized DFA for (a + b)* abb where $\Sigma = \{a, b\}$.

OR Construct NFA for following regular expression : (a + b) * abb

Convert the resultant NFA into its equivalent DFA and then obtain optimized DFA

CT : S-11(8M)

Ans. Regular Expression = (a + b)* abb



Transition table for NFA :

Q \ Σ	a	b
→A	{A, B}	{A}
B	-	{C}
C	-	{D}
*D	-	-

Transition table for DFA :

Q \ Σ	a	b
→{A}	{A, B}	{A}
{A, B}	{A, B}	{A, C}
{A, C}	{A, B}	{A, D}
*{A, D}	{A, B}	{A}

Rename all states as,

A = P,

{A, B} = Q,

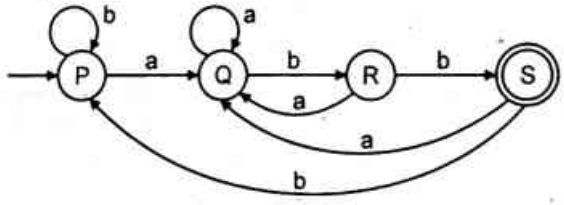
{A, C} = R,

{A, D} = S

Optimized transition table for DFA :

Q \ Σ	a	b
→P	Q	P
Q	Q	R
R	Q	S
*S	Q	P

Transition diagram :

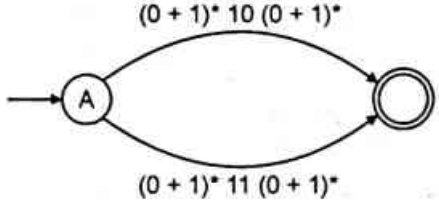


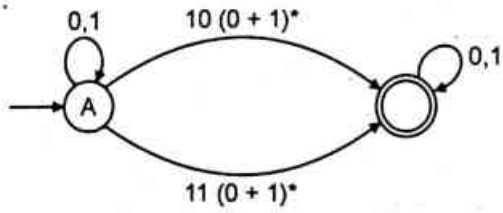
Q.29. Obtain deterministic finite automata for following regular expression.

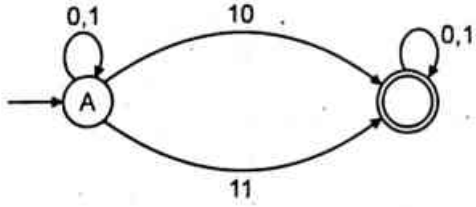
(0 + 1)* 10 (0 + 1)* + (0 + 1)* 11 (0 + 1)*.

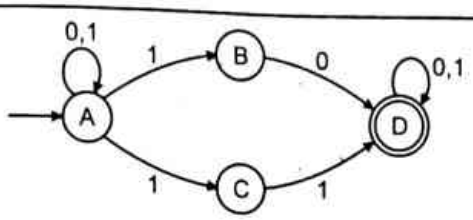
CS : S-13(8M)

Ans. Regular Expression = (0 + 1)* 10 (0 + 1)*
+ (0 + 1)* 11 (0 + 1)*









Transition table for NFA :

Q \ Σ	0	1
→A	{A}	{A, B, C}
B	{D}	-
C	-	{D}
*D	{D}	{D}

Transition table for DFA :

Q \ Σ	0	1
→{A}	{A}	{A, B, C}
{A, B,C}	{A, D}	{A, B, C,D}
*{A, D}	{A, D}	{A, B, C,D}
*{A, B, C, D}	{A, D}	{A, B, C, D}

Rename all states as,

A = P, {A, B, C} = Q,
{A, D} = R, {A, B, C, D} = S

Transition table for DFA :

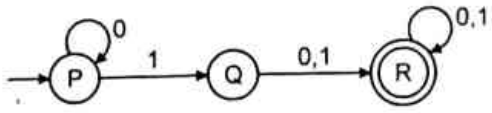
Q \ Σ	0	1
→P	P	Q
Q	R	S
*R	R	S
*S	R	S

Here R and S are equivalent.
So replace S by R

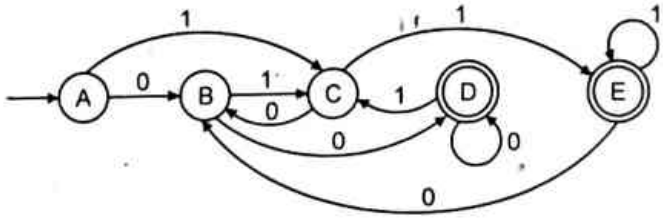
Transition table for DFA :

Q \ Σ	0	1
→P	P	Q
Q	R	R
* R	R	R

Transition diagram :



Transition graph for DFA with original states :



Q.30. Construct a finite automata (DFA) that accepts the language over 0's and 1's which do not contain 101 as a subset.

Ans. Condition : Not containing 101 as substring

Compliment of Condition : Atleast containing 101 as substring.

Regular Expression $= (0 + 1)^* . 101 . (0 + 1)^*$

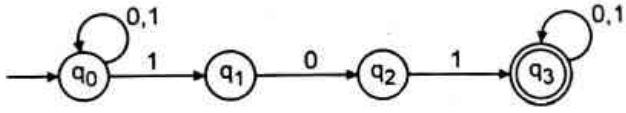


Fig. NFA

Transition table for NFA :

Q \ Σ	0	1
→ q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2\}$	-
q_2	-	$\{q_3\}$
* q_3	$\{q_3\}$	$\{q_3\}$

Transition table for DFA :

Q \ Σ	0	1
→$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1\}$
$\{q_0, q_2\}$	$\{q_0\}$	$\{q_0, q_1, q_3\}$
* $\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$
* $\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$
*$\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$

Renaming states as,

$\{q_0\} = A$, $\{q_0, q_1\} = B$

$\{q_0, q_2\} = C$ $\{q_0, q_1, q_3\} = D$

$\{q_0, q_2, q_3\} = E$ $\{q_0, q_3\} = F$

Transition table for DFA :

Q \ Σ	0	1
→A	A	B
B	C	B
C	A	D
* D	E	D
* E	F	D
* F	F	D

Here state E and state F both are final and equivalent.

Therefore Replace F by E

Transition table for DFA :

Q \ Σ	0	1
→A	A	B
B	C	B
C	A	D
* D	E	D
* E	E	D

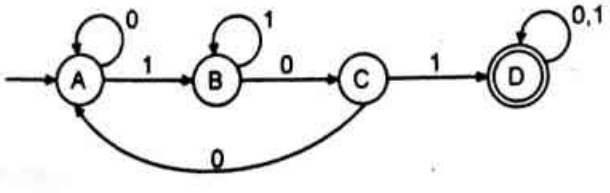
Here state D and state E both are final and equivalent.

Hence Replace E by D

Minimized Transition table for DFA :

Q \ Σ	0	1
→A	A	B
B	C	B
C	A	D
* D	D	D

Transition diagram for DFA :

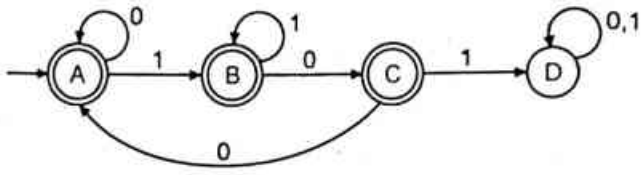


The DFA generated represents compliment of given condition.

Hence it should be converted into DFA for original condition.

This will be performed by simply interchanging the final and non-final states.

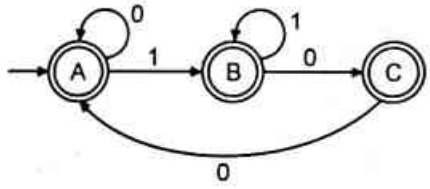
∴ DFA for the original language :



Dead States : Dead states are the non-final states whose transition on all the input symbols terminates on itself.

Here D is a dead state and it can be removed.

∴ Transition graph for the original language :



Transition table for the original language :

Q \ Σ	0	1
* →A	A	B
* B	C	B
* C	A	-

Q.31. Construct an optimal DFA, which accept all strings of zeros and ones, that do not contain 011 as a subset.

Ans. Condition : Not containing substring as 011.

Compliment of Condition : Containing atleast 011 as substring.

Regular Expression = $(0 + 1)^* . 011 . (0 + 1)^*$

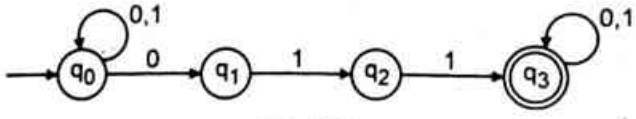


Fig. NFA

Transition table for NFA :

Q \ Σ	0	1
→ q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	-	$\{q_2\}$
q_2	-	$\{q_3\}$
* q_3	$\{q_3\}$	$\{q_3\}$

Transition table for DFA :

Q \ Σ	0	1
→ $\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_3\}$
* $\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_3\}$
* $\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$
* $\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_3\}$

Renaming states as,

$\{q_0\} = A$, $\{q_0, q_1\} = B$

$\{q_0, q_2\} = C$ $\{q_0, q_3\} = D$

$\{q_0, q_1, q_3\} = E$ $\{q_0, q_2, q_3\} = F$

Transition table for DFA :

Q \ Σ	0	1
→ A	B	A
B	B	C
C	B	D
* D	E	D
* E	E	F
* F	E	D

Here state D and state F both are final and equivalent.

Hence replace F by D.

Transition table for DFA :

Q \ Σ	0	1
→ A	B	A
B	B	C
C	B	D
* D	E	D
* E	E	D

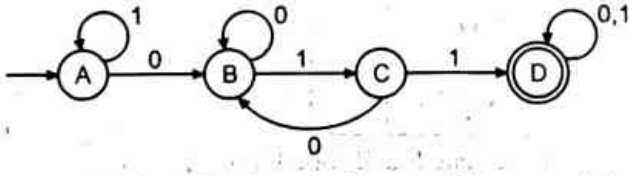
Here state D and state E both are final and equivalent.

Hence Replace E by D

Minimized transition table for DFA :

Q \ Σ	0	1
→ A	B	A
B	B	C
C	B	D
* D	D	D

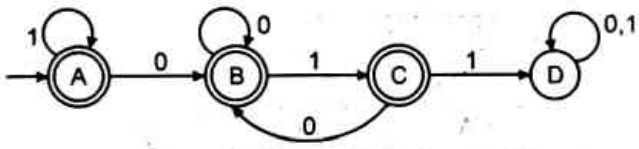
Transition graph for DFA :



This is the DFA for compliment of given condition.

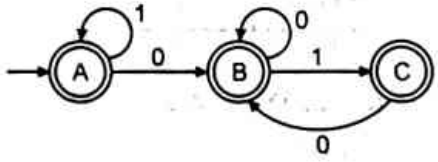
Hence convert it to original condition.

DFA for the original language :



Here D is dead state. So remove it.

Transition graph for the original language :



Transition table for the original language :

Q \ Σ	0	1
* → A	B	A
* B	B	C
* C	B	-

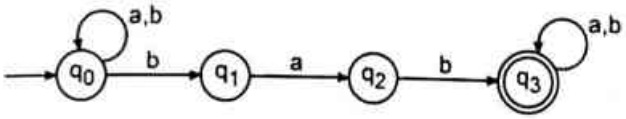
Q.32. Design DFA for language of (a, b) containing 'aba' as substring and not containing 'bab' as substring.

Ans. Part 1 : Infinite condition

Condition : Not containing substring bab

Compliment of condition : Containing atleast one bab

Regular Expression = (a + b)* bab. (a + b)*.



Transition table for NFA :

Q \ Σ	a	b
→ q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2\}$	-
q_2	-	$\{q_3\}$
* q_3	$\{q_3\}$	$\{q_3\}$

Transition Table for DFA :

Q \ Σ	a	b
→ $\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1\}$
$\{q_0, q_2\}$	$\{q_0\}$	$\{q_0, q_1, q_3\}$
* $\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$
* $\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$
* $\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$

Renaming states as,

$\{q_0\}$ = A, $\{q_0, q_1\}$ = B

$\{q_0, q_2\}$ = C $\{q_0, q_1, q_3\}$ = D

$\{q_0, q_2, q_3\}$ = E $\{q_0, q_3\}$ = F

Transition table for DFA :

Q \ Σ	a	b
→ A	A	B
B	C	B
C	A	D
*D	E	D
*E	F	D
*F	F	D

Here state F and state E both are final and equivalent.

∴ Hence replace F by E.

Transition table for DFA :

Q \ Σ	a	b
→ A	A	B
B	C	B
C	A	D
* D	E	D
* E	E	D

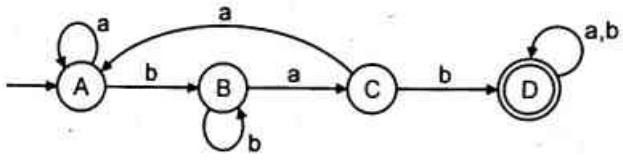
Here state E and state D both are final and equivalent.

Hence replace E by D

Minimized transition table For DFA :

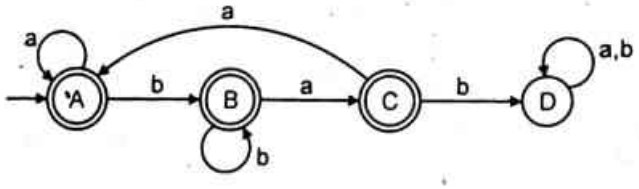
Q \ Σ	a	b
→ A	A	B
B	C	B
C	A	D
* D	D	D

Transition graph for DFA :



This is the DFA for compliment of condition. Hence it is converted to original condition.

DFA for the original language :



Here D is dead state. So remove it.

Transition graph for original language:

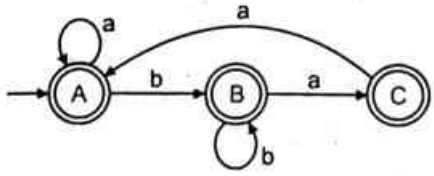


Fig. DFA - 1

Transition table for original DFA :

Q \ Σ	a	b
* → A	A	B
* B	C	B
* C	A	-

Part II : Finite Condition :

Condition : Containing atleast one aba

Regular Expression = (a + b)* .aba. (a + b)*.

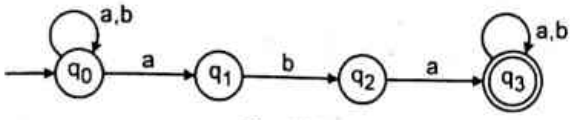


Fig. NFA

Transition table for NFA :

Q \ Σ	a	b
→ q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	-	$\{q_2\}$
q_2	$\{q_3\}$	-
* q_3	$\{q_3\}$	$\{q_3\}$

Transition table for DFA :

Q \ Σ	a	b
→ $\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1, q_3\}$	$\{q_0\}$
* $\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$
* $\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_3\}$
* $\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_3\}$

Renaming states as,

$\{q_0\}$ = A', $\{q_0, q_1\}$ = B'

$\{q_0, q_2\}$ = C' $\{q_0, q_1, q_3\}$ = D'

$\{q_0, q_2, q_3\}$ = E' $\{q_0, q_3\}$ = F'

Transition table for DFA :

Q \ Σ	a	b
→ A'	B'	A'
B'	B'	C'
C'	D'	A'
* D'	D'	E'
* E'	D'	F'
* F'	D'	F'

Here state F' and state E' both are final and equivalent.

Hence replace E' by F'.

Transition table for DFA :

Q \ Σ	a	b
→ A'	B'	A'
B'	B'	C'
C'	D'	A'
* D'	D'	F'
* F'	D'	F'

Again states D' and F' are both final and equivalent.

Hence Replace F' by D'

Transition table for DFA :

Q \ Σ	a	b
→ A'	B'	A'
B'	B'	C'
C'	D'	A'
* D'	D'	D'

Transition graph for original language :

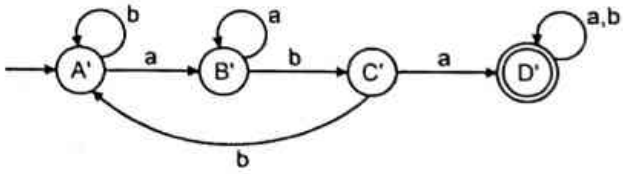


Fig. DFA-2

Transition table for final DFA :

Q \ Σ	a	b
→ [AA']	[AB']	[BA']
[AB']	[AB']	[BC']
[BA']	[AB']	[CA']
[BC']	[AD']	[CA']
[CA']	[AB']	[A']
[AD']	[AD']	[BD']
[BD']	[AD']	[CD']
[CD']	[AD']	[D']

Note : The final states of the final DFA will be all the states consisting of the final state of the finite condition.

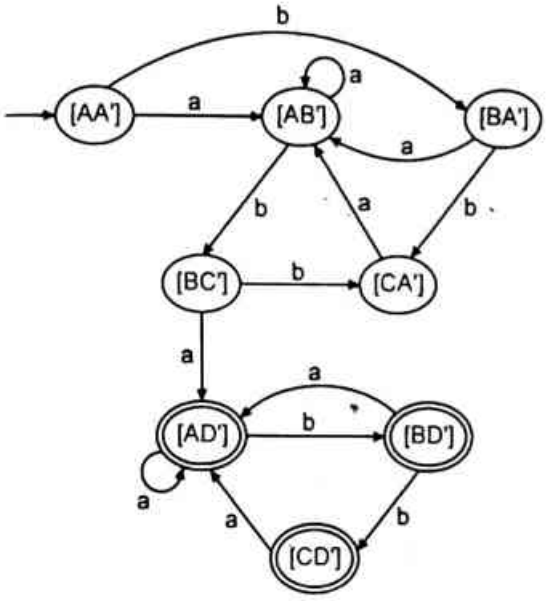


Fig. DFA for language over [a,b]

Q.33. Design DFA for L = {a, b} to accept all strings containing "aba" as substring and do not contain "bba" as substring. ***CS : S-09(10M)***

Ans. Part 1 : Finite condition :

Condition : Containing aba as substring.

Regular Expression = (a + b)* aba (a + b)*

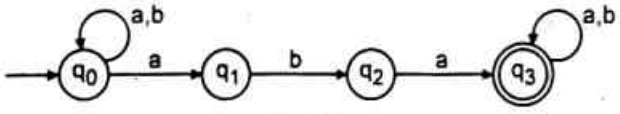


Fig. NFA

Transition table for NFA :

Q \ Σ	a	b
→ q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	-	$\{q_2\}$
q_2	$\{q_3\}$	-
* q_3	$\{q_3\}$	$\{q_3\}$

Transition table for DFA :

Q \ Σ	a	b
→ $\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1, q_3\}$	$\{q_0\}$
* $\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$
* $\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_3\}$
* $\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_3\}$

Renaming all states as,

$\{q_0\} = A$, $\{q_0, q_1\} = B$

$\{q_0, q_2\} = C$ $\{q_0, q_1, q_3\} = D$

$\{q_0, q_2, q_3\} = E$ $\{q_0, q_3\} = F$

Transition table for DFA :

Q \ Σ	a	b
→ A	B	A
B	B	C
C	D	A
*D	D	E
*E	D	F
*F	D	F

Here state E and F both are final and equivalent.

Hence replace E by F

Transition table for DFA :

Q \ Σ	a	b
→ A	B	A
B	B	C
C	D	A
* D	D	F
* F	D	F

Also state D and state F are equivalent.

Hence replace F by D

Transition table for DFA :

Q \ Σ	a	b
→ A	B	A
B	B	C
C	D	A
* D	D	D

Transition graph for DFA :

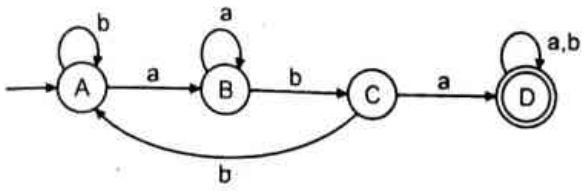


Fig. DFA-1

Part 2 : Infinite condition :

Condition : Do not contain 'bba' as substring.

Complement of condition : Containing bba as substring.

∴ **Regular Expression** = (a+b)* . bba.(a+b)*

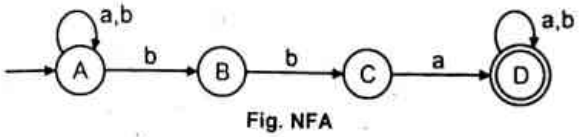


Fig. NFA

Transition table for NFA :

Q \ Σ	a	b
→A	A	A, B
B	-	C
C	D	-
*D	D	D

Transition table for DFA :

Q \ Σ	a	b
→{A}	{A}	{A, B}
{A, B}	{A}	{A, B, C}
{A, B, C}	{A , D}	{A, B, C}
* {A, D}	{A, D}	{A, B, D}
*{A, B, D}	{A, D}	{A, B, C, D}
* {A, B, C, D}	{A, D}	{A, B, C, D}

Renaming all states as,

{A} = P, {A,B} = Q

{A,B, C} = R {A,D} = S

{A,B,D} = T {A,B, C, D} = U

Transition table for DFA :

Q \ Σ	a	b
→P	P	Q
Q	P	R
R	S	R
* S	S	T
* T	S	T
* U	S	U

Here state U and T both are equivalent.

Hence replace U by T

Transition table for DFA :

Q \ Σ	a	b
→P	P	Q
Q	P	R
R	S	R
* S	S	T
* T	S	T

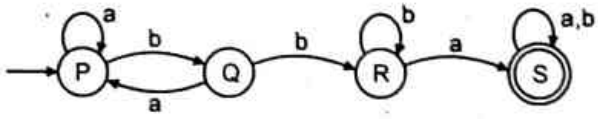
Also here state T and state S are equivalent.

Hence replace T by S

Transition table for DFA :

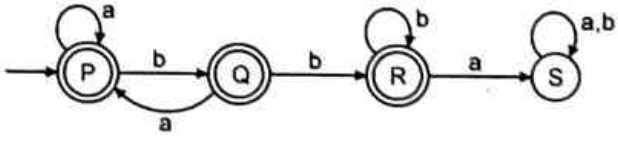
Q \ Σ	a	b
→P	P	Q
Q	P	R
R	S	R
* S	S	S

Transition graph for DFA :



Now, take complement condition.

Transition graph for original language :



Here 'S' is a dead state.

Transition graph for original language :

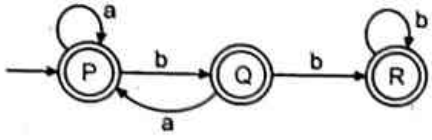


Fig. DFA -2

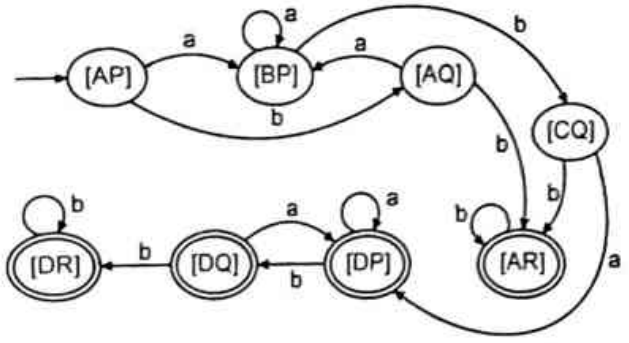
Transition table for DFA :

Q \ Σ	a	b
→* P	P	Q
* Q	P	R
* R	-	R

Transition table for final DFA :

Q \ Σ	a	b
→ [AP]	[BP]	[AQ]
[BP]	[BP]	[CQ]
[AQ]	[BP]	[AR]
[CQ]	[DP]	[AR]
[AR]	[B]	[AR]
* [DP]	[DP]	[DQ]
* [DQ]	[DP]	[DR]
* [DB]	[D]	[DR]

Transition graph for final DFA :



Q.34. Construct a minimal Finite state machine to accept strings over Σ containing a, b which does not contain 'bab' as substring but contains at least 'bbb'. Also obtain regular expression for the same. Make use of the fact that regular set are closed under intersection.

Ans. Part 1: Finite condition :

Condition : Strings contain atleast one substring bbb.

Regular expression = (a + b)* bbb (a + b)*

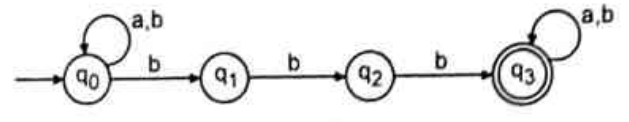


Fig. NFA

Transition table for NFA :

Q \ Σ	a	b
→ q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	-	$\{q_2\}$
q_2	-	$\{q_3\}$
* q_3	$\{q_3\}$	$\{q_3\}$

Transition table for DFA :

Q \ Σ	a	b
→ $\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$
*$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
*$\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$
* $\{q_0, q_1, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$

Renaming all states as,

$\{q_0\}$ = A, $\{q_0, q_1\}$ = B

$\{q_0, q_1, q_2\}$ = C, $\{q_0, q_1, q_2, q_3\}$ = D

$\{q_0, q_3\}$ = E $\{q_0, q_1, q_3\}$ = F

Transition table for DFA :

Q \ Σ	a	b
→A	A	B
B	A	C
C	A	D
*D	E	D
*E	E	F
*F	E	D

States D and F are final and equivalent.

Hence replace F by D.

Transition table for DFA :

Q \ Σ	a	b
→A	A	B
B	A	C
C	A	D
* D	E	D
* E	E	D

Again states D and E are equivalent.

Replace state E by state D.

Minimized transition table for DFA :

Q \ Σ	a	b
→ A	A	B
B	A	C
C	A	D
* D	D	D

Transition graph for original language :

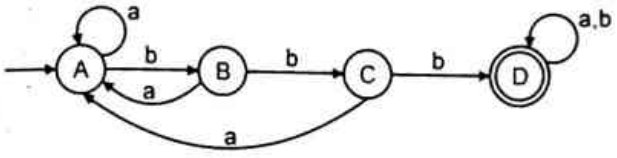


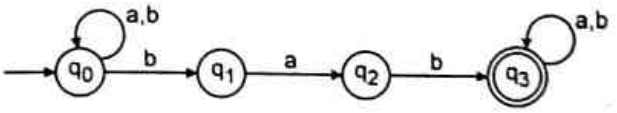
Fig. DFA 1

Part 2 : Infinite Condition :

Condition : String does not contain bab as substing.

Complement of condition : Strings contain atleast bab as substring.

∴ **Regular Expression** = $(a + b)^*$ bab. $(a + b)^*$



Transition table for NFA :

Q \ Σ	a	b
→ q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2\}$	-
q_2	-	$\{q_3\}$
* q_3	$\{q_3\}$	$\{q_3\}$

Transition table for DFA :

Q \ Σ	a	b
→ $\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1\}$
$\{q_0, q_2\}$	$\{q_0\}$	$\{q_0, q_1, q_3\}$
* $\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$
* $\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$
* $\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$

Renaming states as,

$\{q_0\}$ = A', $\{q_0, q_1\}$ = B'

$\{q_0, q_2\}$ = C', $\{q_0, q_1, q_3\}$ = D'

$\{q_0, q_2, q_3\}$ = E', $\{q_0, q_3\}$ = F'

Transition table for DFA :

Q \ Σ	a	b
→ A'	A'	B'
B'	C	B'
C'	A'	D'
* D'	E'	D'
* E'	F'	D'
* F'	F'	D'

Here state E' and F' are final and equivalent.

So replace F' by E'.

Transition table for DFA :

Q \ Σ	a	b
→ A'	A'	B'
B'	C	B'
C'	A'	D'
* D'	E'	D'
* E'	E'	D'

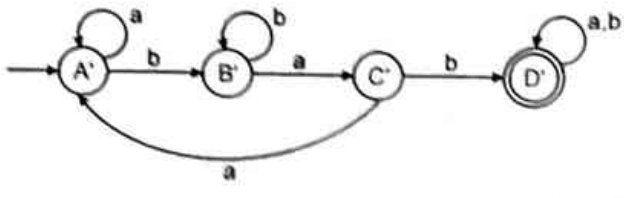
States E' and D' are equivalent and final

So replace E' by D'

Minimized transition table for DFA :

Q \ Σ	a	b
→ A'	A'	B'
B'	C'	B'
C'	A'	D'
* D'	D'	D'

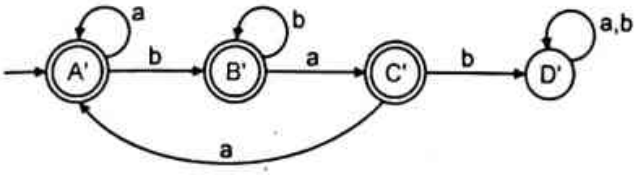
Transition graph for original DFA :



But above DFA is for the complement of given condition. Hence we will convert it to the original condition i.e. strings does not contain 'bab' as substring.

To convert it to original condition, convert final state to non-final state and vice-versa.

Transition graph for original language :



Here D is called as dead state.

So Remove state D.

Transition graph for original language :

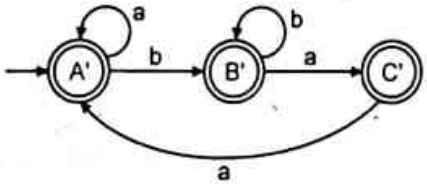


Fig. DFA - 2

Transition table for DFA-2 :

Σ / Q	a	b
→ A'	A'	B'
B'	C'	B'
C'	A'	-

Transition table for final DFA :

Σ / Q	a	b
→ [AA']	[AA']	[BB']
[BB']	[AC']	[CB']
[AC']	[AA']	[B']
[CB']	[AC']	[DB']
[DB']	[DC']	[DB']
[DA']	[DA']	[DB']
[DC']	[DA']	[D']

Note : Final states will be those states that contain final state of DFA for finite condition

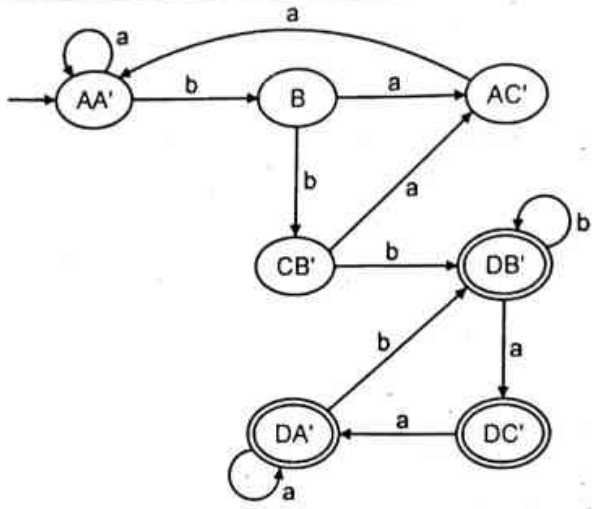


Fig. Final DFA

Q.35. Show that the following regular expressions are equivalent, by showing that their minimum state DFA's are the same :

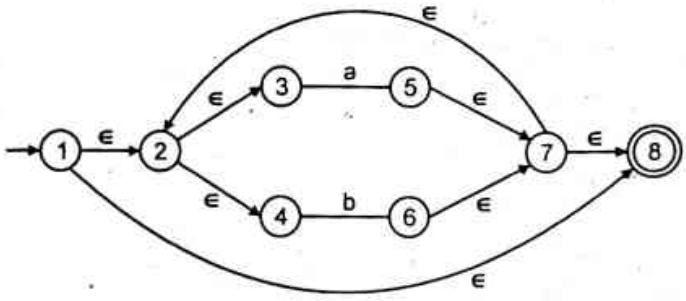
(i) (a/b)*

(ii) (a*/b*)*

(iii) ((∈ /a) b*)*

Ans. First we construct NFA with ∈-moves for given regular expression. Then these NFA with ∈-moves be converted to DFA.

(i) (a/b)* = (a + b)*



Transition table for DFA :

Σ / States	a	b
→ * {1, 2, 3, 4, 8}	{2, 3, 4, 5, 7, 8}	{2,3,4,6,7, 8}
* {2, 3, 4, 5, 7, 8}	{2, 3, 4, 5, 7, 8}	{2,3,4,6,7, 8}
* {2, 3, 4, 6, 7, 8}	{2, 3, 4, 5, 7, 8}	{2,3,4,6,7, 8}

Renaming states as,

{1, 2, 3, 4, 8,} = A, {2, 3, 4, 5, 7, 8} = B

{2, 3, 4, 6, 7, 8} = C

Transition table for DFA :

Σ / States	a	b
→ A *	B	C
B *	B	C
C *	B	C

Replacing C by B and then in turn replace B by A

[∵ all states are equivalent and final]

Minimized DFA :

Σ / States	a	b
→ A *	A	A

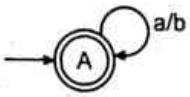


Fig. (1) DFA

(ii) (a* | b*) = (a* + b*)*

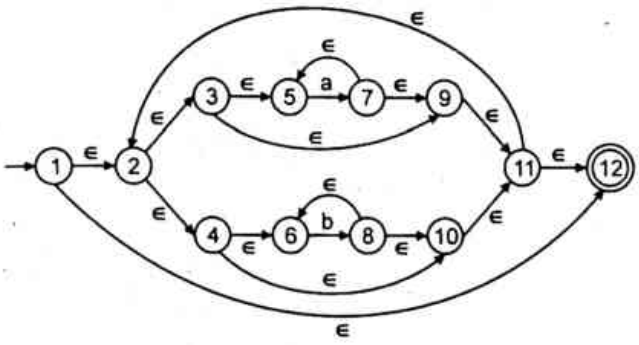


Fig. NFA with ϵ-moves

Transition table for DFA :

Σ / States	a	b
→ {1,2,3,4,5,6, 9,10,11,12}*	{2,3,4,5,6,7, 9,10,11,12}	{2,3,4,5,6,8, 9,10,11,12}
{2,3,4,5,6,7, 9,10,11,12}*	{2,3,4,5,6,7, 9,10,11,12}	{2,3,4,5,6,8, 9,10,11,12}
{2,3,4,5,6,8, 9,10,11,12}*	{2,3,4,5,6,7, 9,10,11,12}	{2,3,4,5,6,8, 9,10,11,12}

Renaming states as,

{1, 2, 3, 4, 5, 6, 9, 10, 11, 12} = A {2, 3, 4, 5, 6, 7, 9, 10, 11, 12} = B {2, 3, 4, 5, 6, 8, 9, 10, 11, 12} = C

Transition table for DFA :

Σ / States	a	b
→ *A	B	C
* B	B	C
* C	B	C

States A, B and C are all final and equivalent. So replace C by B and in turn replace B by A.

∴ Minimized DFA :

Σ / States	a	b
→ * A	A	A

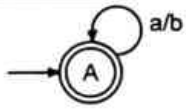


Fig. (2) DFA

(iii) ((ϵ/a) b*)* = ((ϵ + a) . b*)*

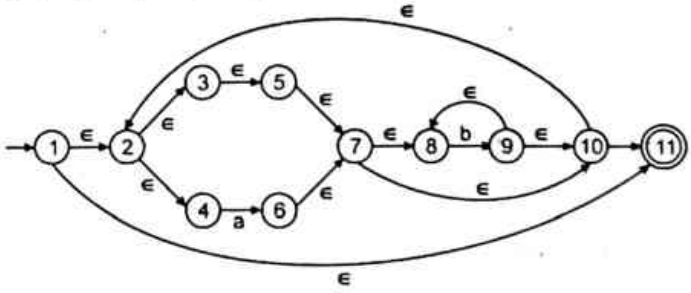


Fig. NFA with ϵ-moves

Transition table for DFA :

Σ / States	a	b
→* {1,2,3,4,5, 7,8,10,11}	{2,3,4,5,6, 7,8,10,11}	{2,3,4,5,7, 8,9,10,11}
* {2,3,4,5,6, 7,8,10,11}	{2,3,4,5,6, 7,8,10,11}	{2,3,4,5,7, 8,9,10,11}
* {2,3,4,5, 7,8,9,10,11}	{2,3,4,5,6, 7,8,10,11}	{2,3,4,5,7, 8,9,10,11}

Renaming states as,

{1, 2, 3, 4, 5, 7, 8, 10, 11} = A {2, 3, 4, 5, 6, 7, 8, 10, 11} = B {2, 3, 4, 5, 7, 8, 9, 10, 11} = C

Transition table for DFA :

Σ / States	a	b
→ *A	B	C
* B	B	C
* C	B	C

States A, B and C are equivalent and final.
So replace C by B and in turn replace B by A.

∴ **Minimized transition table :**

Q \ Σ	a	b
→ * A	A	A

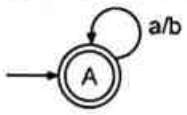


Fig. (3) DFA

From figure (1), (2) and (3)
The DFA's for all regular expression are same.
Hence given regular expressions are equivalent.

Q.36. Construct DFA machine for accepting language over {0, 1}*, L = {x | x begin with 0 and ends in double 1}.

Ans. Regular Expression = 0.(0 + 1)*. 11

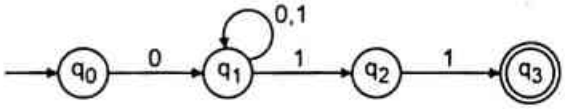


Fig. NFA

Transition table for NFA :

Q \ Σ	0	1
→ q_0	$\{q_1\}$	-
q_1	$\{q_1\}$	$\{q_1, q_2\}$
q_2	-	$\{q_3\}$
* q_3	-	-

Transition table for DFA :

Q \ Σ	0	1
→ $\{q_0\}$	$\{q_1\}$	-
$\{q_1\}$	$\{q_1\}$	$\{q_1, q_2\}$
$\{q_1, q_2\}$	$\{q_1\}$	$\{q_1, q_2, q_3\}$
* $\{q_1, q_2, q_3\}$	$\{q_1,\}$	$\{q_1, q_2, q_3\}$

Renaming all states as,

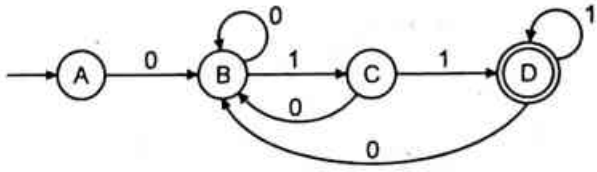
q_0 = A, q_1 = B

q_1, q_2 = C, q_1, q_2, q_3 = D

Transition table for DFA :

Q \ Σ	0	1
→ A	B	-
B	B	C
C	B	D
*D	B	D

Transition graph for DFA :



Q.37. Draw a finite automata that recognizes the language L_1 and L_2 which are the subset of {0, 1}*

L_1 = {X | 00 is not a substring of X}

L_2 = {X ends with 01}

Ans. Part I : Infinite Condition (L_1)

Condition : 00 is not substring of X.

Compliment of Condition : 00 is substring of X.

∴ **Regular Expression** = (0 + 1)* . 00. (0 + 1)*.

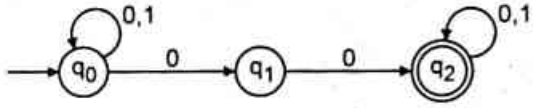


Fig. NFA

Transition table for NFA :

Q \ Σ	0	1
→ q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	-
* q_2	$\{q_2\}$	$\{q_2\}$

Transition table for DFA :

Q \ Σ	0	1
→ $\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_0\}$
* $\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$
* $\{q_0, q_2\}$	$\{q_0, q_1, q_2,\}$	$\{q_0, q_2\}$

Renaming all states as,

$\{q_0\} = A$, $\{q_0, q_1\} = B$

$\{q_0, q_1, q_2\} = C$ $\{q_0, q_2\} = D$

Transition table for DFA :

Q \ Σ	0	1
→A	B	A
B	C	A
*C	C	D
*D	C	D

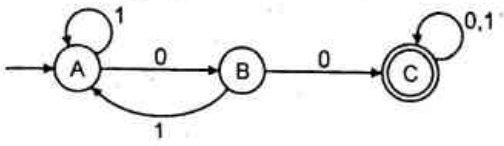
Here states C and D are equivalent and final.

Replace D by C.

Transition table for DFA :

Q \ Σ	0	1
→A	B	A
B	C	A
*C	C	C

Transition graph for DFA :



This is the DFA for compliment of given condition, hence convert it to original condition by interchanging final and non-final state.

Transition graph for original language :

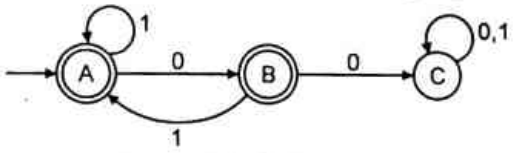


Fig. DFA

Here, C is dead state. So Remove C.

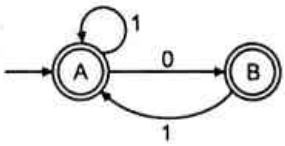


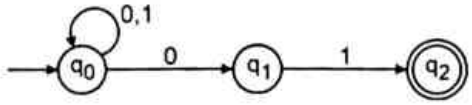
Fig. DFA-1

Transition Table for DFA-1 :

Q \ Σ	0	1
→*A	B	A
*B	-	A

Part II : Finite Condition : X ends with 01.

Regular Expression $= (0 + 1)^* . 01$.



Transition table for NFA :

Q \ Σ	0	1
→ q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	-	$\{q_2\}$
* q_2	-	-

Transition table for DFA :

Q \ Σ	0	1
→$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
*$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$

Renaming all states as,

$q_0 = A'$ $q_0, q_1 = B'$

$q_0, q_2 = C'$

Transition table for DFA :

Q \ Σ	0	1
→ A′	B′	A′
B′	B′	C′
* C′	B′	A′

Transition graph for original language :

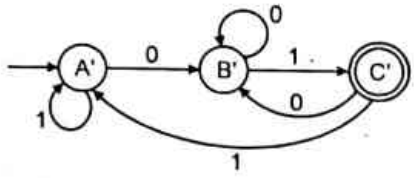


Fig. DFA-2

Transition table for final DFA :

Q \ Σ	0	1
→[AA′]	[BB′]	[AA′]
[BB′]	[BB′]	[AA′]
* [AC′]	[BB′]	[AA′]

Transition graph for final DFA :

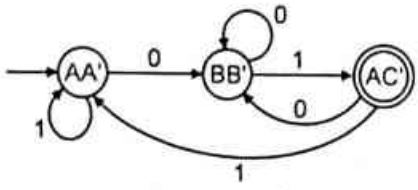


Fig. DFA

REGULAR EXPRESSION IDENTITIES

Q.38. Give various regular expressions identities.

Ans. Regular expressions identities :

- Two regular expressions P and Q are equivalent if P and Q represent the same set of strings.
- Following are the identities for regular expressions.

$I_1 : \phi + R = R$

$I_2 : \phi R = R\phi = 0$

$I_3 : \in R = R \in = R$

$I_4 : \in^* = \in$ and $\phi^* = \in$

$I_5 : R + R = R$

$I_6 : R^* R^* = R^*$

$I_7 : RR^* = R^* R$

$I_8 : (R^*)^* = R^*$

$I_9 : \in + RR^* = R^* = \in + R^* R$

$I_{10} : (PQ)^* P = P(QP)^*$

$I_{11} : (P + Q)^* = (P^* Q^*) = (P^* + Q^*)^*$

$I_{12} : (P + Q)R = PR + QR$ and

$R(P + Q) = RP + RQ$

Q.39. Simplify following regular expression.

$R = \in + 1^* (011)^* (1^* (011)^*)^*$ **CS : S-11(2M)**

Ans. $R = \in + P_1 P_1^*$ *[$\because P_1 = 1^* (011)^*$]*

$= P_1^*$ *[$\because \in + R R^* = R^*$]*

$= (1^* (011)^*)^*$

$= (P_2^* P_3^*)^*$ *[$\because P_2 = 1, P_3 = 011$]*

$= (P_2 + P_3)^*$

[$\because (P + Q)^* = (P^* Q^*)^*$

$= (P^* + Q^*)^*$]

$= (1 + 011)^*$

Q.40. Using identity rule prove the following :

$(1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)$

$= 0^*1 \ (0 + 10^* 1)^*$.

Ans. L.H.S. $= (1 + 00^* 1) + (1 + 00^* 1)(0 + 10^* 1)^*$

$(0 + 10^* 1)$

$= (1 + 00^* 1)(\in + (0 + 10^* 1)^* (0 + 10^* 1)$

[$\because (P + Q)R = PR + QR$]

$= (1 + 00^* 1)(0 + 10^* 1)^*$

[$\because \in + RR^* = R^*$]

$= (\in + 00^*) 1 (0 + 10^* 1)^*$

[$\because (P + Q)R = PR + QR$]

$= 0^* 1 (0 + 10^* 1)^*$ *[$\because \in + RR^* = R^*$]*

= R.H.S.

Hence proved.

REGULAR GRAMMAR

Q.41. What is regular grammar? Give example.

Ans. Regular Grammar :

A regular grammar is defined as

G = (V, T, P, S)

where V is set of symbols called non terminals which are used to define the rules.

T is set of symbols called terminals.

P is set of production rules.

S is a start symbol which $\in$ V.

The production rules P are of the form

- $A \rightarrow aB$
- $A \rightarrow a$

where A and B are non terminal symbols and 'a' is terminal symbol.

Example :

If G = (V, T, P, S) with

V = (S, A)

T = (0, 1)

S is a start symbol and production rules are as given below.

$S \rightarrow 0S \mid 1B$

$B \rightarrow \in$

This is called a regular language and can be represented by some DFA.

Thus, when a grammar can be represented by some finite automata then it is a regular grammar.

Q.42. Define linear grammar and comment on "Every regular grammar is linear but every linear grammar is not necessarily regular.

CT : S-06(4M)

Ans. Linear grammar :

The set of rules used to define the language is called linear grammar.

Comment :

The regular grammar is context free grammar, in which every production is restricted to have one of the following form.

(a) $A \rightarrow aB$
(b) $A \rightarrow w$ } ....(1)

where A, B = non terminals symbol
w = terminal symbol

- Linear grammar is a right linear or left linear grammar.
- Right linear grammar is a CFG, in which every production is restricted to have one of the following form :

(a) $A \rightarrow wB$
(b) $A \rightarrow w$ } ....(2)

where, A, B = non-terminal symbol
w = terminal symbol

- Left linear grammar is a CFG in which every production is restricted to have one of following form :

(a) $A \rightarrow Bw$
(b) $A \rightarrow w$ } ....(3)

where A, B = non-terminal symbol
w = terminal symbol

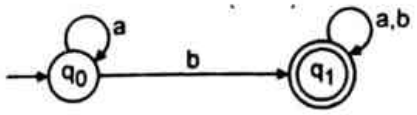
- From 1 and 2 it is clear that every regular grammar is a right linear grammar. Hence every regular grammar is a linear grammar.
- But left linear grammar can not be a regular grammar. Hence every linear grammar is not regular grammar.

Q.43. Construct a regular grammar G generating the regular set represented by P = a*b(a + b)*.

CS : S-11(4M), CT : W-07,10, S-08(5M)

Ans. $P = a^*b(a+b)^*$

Transition graph :



Let $q_0 = Aq_1 = A_1$

$G = (\{A_0, A_1\}) \{a, b\}, P, A_0\})$

$A_0 \rightarrow aA_0$, $A_0 \rightarrow bA_1$,

$A_0 \rightarrow b$, $A_1 \rightarrow aA_1$

$A_1 \rightarrow bA_1$ $A_1 \rightarrow a$

$A_1 \rightarrow b$

Q.44. Find regular expression for the given grammer :

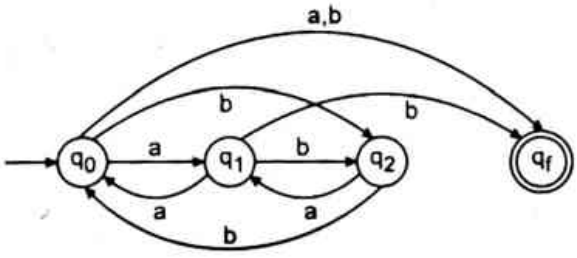
$A \rightarrow ac \mid bD \mid a \mid b$

$C \rightarrow aA \mid bD \mid b$

$D \rightarrow aC \mid bA.$

Ans. Let states q_0, q_1, q_2 correspond to A, C and D.

The Finite automata represented by given grammer is,



$q_0 = q_1a + q_2b + \epsilon$ ....(1)

$q_1 = q_0a + q_2a$ ...(2)

$q_2 = q_0b + q_1b$ ...(3)

$q_f = q_0 (a + b) + q_1b$ ....(4)

From equation (2) and (3)

$q_1 = q_0a + (q_0b + q_1b) a$

$= q_0a + q_0ba + q_1ba$

$q_1 = q_0 (a + ba) + q_1 ba$

R Q P R

By Arden's Theorem

$\therefore q1 = q_0 (a + ba) (ba)^*$ ....(5)

From equation (4) and (5)

$q_f = q_0 (a + b) + q_0 (a + ba) (ba)^* . b$

$= q_0 ((a + b) + (a + ba) (ba)^* . b)$ ....(6)

From equation (1) and (3)

$q_0 = q_1a + q_0bb + q_1bb + \epsilon$

$= q_0\ bb + q_1(a + bb) + \epsilon$

From (5)

$q_0 = q_0\ bb + q_0(a + ba)(ba)^*(a + bb) + \epsilon$

$\underbrace{q_0}_{R} = \underbrace{q_0}_{R}\ \underbrace{(bb + (a + ba)(ba)^*(a + bb)}_{P}\ +\underbrace{\epsilon}_{Q}$

By Arden's Theorem

$q_0 = \epsilon\,(bb + (a + ba)(ba)^*.(a + bb))^*$

$[\because \epsilon R^* = R^*]$

$\therefore\ q_0 = (bb + (a + ba)(ba)^*.(a + bb))^*$ ....(7)

From equation (6) and (7)

Regular Expression :

$q_f = (bb + (a + ba)(ba)^*$

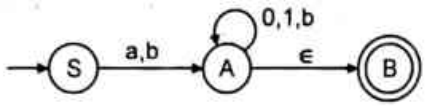
$.(a + bb)^*\ ((a + b) + (a + ba).(ba)^*.b)$

Q.45. Get the optimized DFA for the regular expression defined by the grammar.

S → aA | bA

A → 0A | 1A | bA | ∈. **CT : S-10(6M)**

Ans. Transition Graph for given regular grammar :



State equations are,

$S = \epsilon$ ....(1)

$A = S(a + b) + A(0 + 1 + b)$ ....(2)

$B = A\epsilon = A$ $[\because R\epsilon = R]$ ....(3)

From equation (1) and (2)

$A = \epsilon\,(a + b) + A(0 + 1 + b)$

$= (a + b) + A(0 + 1 + b)$

By Arden's Theorem

$\therefore\ A = (a + b)(0 + 1 + b)^*$ ....(4)

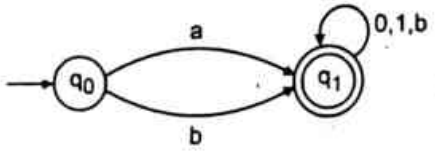
From equation (3) and (4)

$B = A = (a + b)(0 + 1 + b)^*$

Regular Expression :

$(a + b)(0 + 1 + b)^*$

The DFA is,



Q.46. Construct finite automata recognizing L (G), where G is the grammar.

S → aA | bA | b

A → aA | bS | a **CT : S-11(5M)**

Ans. Transitions :

- S → aA makes a transition from S to A with input a
- S → bA makes a transition from S to A with input b
- S → b makes a transition from S to F with input b
- A → aA makes a transition from A to A with input a
- A → bS makes a transition from A to S with input b
- A → a makes a transition from A to F with input a

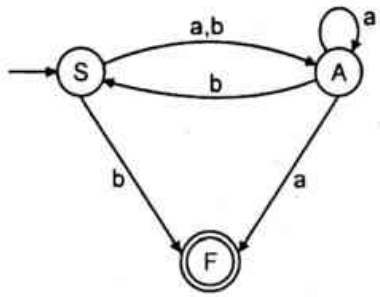


Fig. Finite automata

RIGHT LINEAR GRAMMAR AND LEFT LINEAR GRAMMAR

Q.47. What are right linear and left linear grammar? Explain with example.

Ans. Regular grammar :

- This is a type of context free grammar in which there are certain restriction on non-terminal.

(1) The left hand side of production should consist of only single non-terminal symbol.

(2) The right hand side consist of single non-terminal symbol which is allowed to appear as right most non-terminal or left most non-terminal.

Example :

A → aB

B → aaB

A → Aab

- Depending on the position of non-terminal on right hand side there are two types of regular grammar.

(1) Left linear grammar

(2) Right linear grammar

(1) Left linear grammar :

- The regular grammar is said to be left linear grammar if it has the production of the form

  $A \rightarrow a$ .....

  $A \rightarrow Bw$ .....

  $A \rightarrow \in$

  Where, w is any number of terminal symbols.
- The restriction in this type of grammar is, the right hand side of production consist of only one non-terminal which is also leftmost non-terminal.

Example :

Left linear Grammar :

$A \rightarrow a$

$A \rightarrow Ba$

$A \rightarrow Aab$

$C \rightarrow Aa$

(2) Right linear grammar :

- The regular grammar is said to be right linear if it has the production of the form.

  $A \rightarrow a$

  $A \rightarrow wB$

  $A \rightarrow \in$
- The restriction in this type of grammar is, the right hand side of grammar consist of single non terminal symbol which is also rightmost non terminal.

Example :

Right linear grammar :

$A \rightarrow a$

$A \rightarrow aB$

$A \rightarrow abA$

$C \rightarrow aA$

Q.48. What are the steps for converting right linear grammar to equivalent left-linear grammar.

CT : W-07,12(3M)

Ans. Steps for converting right linear grammar to equivalent left-linear grammar :

(1) Construct transition graph for given right linear grammar.

(2) Exchange the position of final state and initial state and reverse the direction of all transitions.

(3) This is the transition graph for Left linear grammar.

(4) Write the grammar in left linear grammar.

Steps for converting left linear grammar to right linear grammar :

(1) Construct transition graph for given left linear grammar.

(2) Exchange the position of final state and initial state and reverse the direction of all transitions.

(3) This is the transition graph for Right linear grammar.

(4) Write the grammar in Right linear grammar.

Q.49. Construct DFA accepting the regular language generated by the left linear grammar given below :

$S \rightarrow Ca \mid Bb$

$C \rightarrow Bb$

$B \rightarrow Ba \mid b$ ***CT : S-09(5M)***

Ans. To construct DFA following steps should be performed

(a) Step 1 : Reversing the right side of the production of grammar.

$S \rightarrow a\,C \mid b\,B$

$C \rightarrow b\,B$

$B \rightarrow a\,B \mid b$

Step 2 : Obtain the regular expression by replacing $\rightarrow$ by '=' symbol

$S = a\,C \mid b\,B$ ----- (1)

$C = a\,B$ ----- (2)

$B = a\,B \mid b$ ----- (3)

From (3)

$B = a^* . b$

Put value of B in equation (2)

$C = b\,a^* . b$

Put value of C and B in equation (1)

$S = a\,ba^* b \mid b\,a^* . b$

The regular expression is

$ab\,a^* b \mid b\,a^* b$

Step 3 : Reverse the regular expression obtained in step 2.

∴ b a * | b a * b a

Step 4 : Construct DFA for the language accepted by regular expression obtained in step 3.

∴ Now NFA for ba * b | ba * ba is as follows.

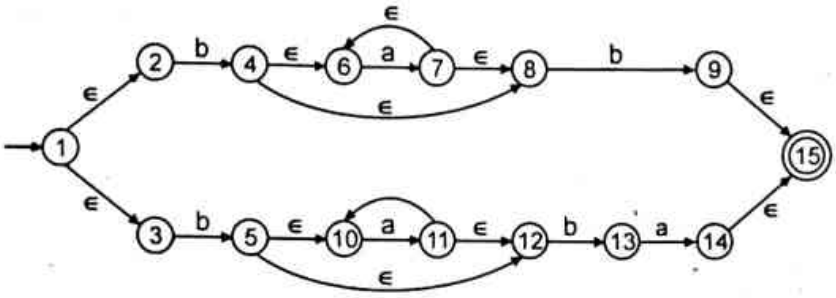


Fig. NFA

Transition table:

State	a	b
A = {1, 2, 3}	ϕ	B = {4, 5, 6, 8, 10,12 }
B = {4, 5, 6, 8, 10, 12 }	C = { 6, 7, 8,10,11,12 }	D ={9, 13, 15}
C = {6, 7, 8, 10, 11, 12}	C = {6, 7, 8, 10, 11, 12}	D ={9, 13, 15}
D = {9, 13, 15} *	F = { 14, 15}	ϕ
E = { 14, 15}*	ϕ	ϕ

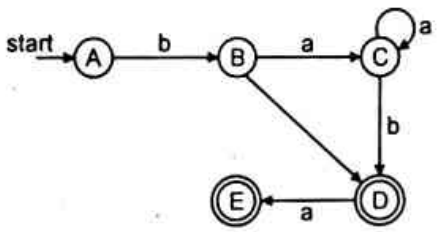


Fig. DFA accepting the regular language generated by the given left linear grammar

Q.50. Convert the following right linear grammar to its equivalent left-linear grammar :

S → bB,

B → bC

B → aB

B → b

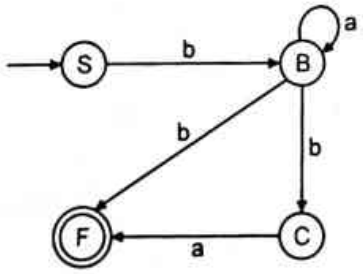
C → a .

Ans. S → bB

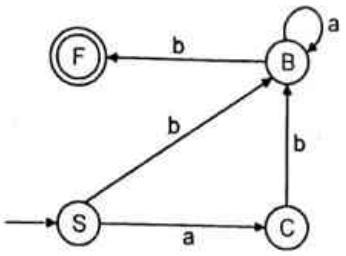
B → bC / a B / b

C → a

Transition graph for right linear grammar :



Transition graph for left linear grammar :



Production for left linear grammar :

$S \rightarrow Bb / Ca$

$B \rightarrow Ba / b$

$C \rightarrow Bb$

Q.51. Construct equivalent left linear grammar for the right linear grammar given as,

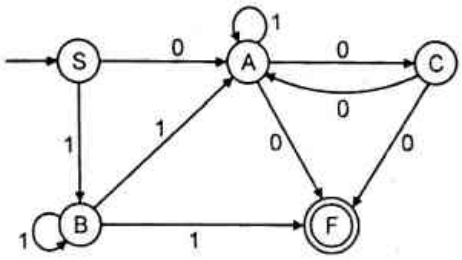
$S \rightarrow 0A \mid 1B$

$A \rightarrow 0C \mid 1A \mid 0$

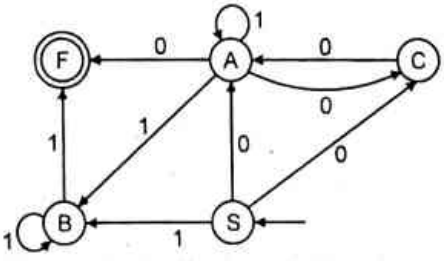
$B \rightarrow 1B \mid 1A \mid 1$

$C \rightarrow 0 \mid 0A$ ***CT : S-09(5M)***

Ans. Transition graph for right linear grammar :



Transition graph for left linear grammar :



Production for left linear grammar :

$S \rightarrow A0 / B1 / C0$

$A \rightarrow A1 / B1 / 0 / C0$

$B \rightarrow B1 / 1$

$C \rightarrow A0$

Q.52. Obtain right linear grammar equivalent to following left linear grammar.

$S \rightarrow Sab \mid Aa$

$A \rightarrow Abb \mid bb$ ***CS : W-12(6M)***

Ans. The given left linear grammar is,

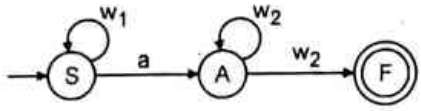
$S \rightarrow Sab \mid Aa$

$A \rightarrow Abb \mid bb$

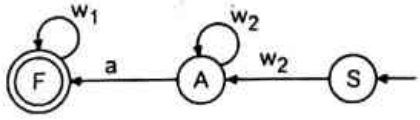
Let $w_1 = ab$

$w_2 = bb$

Transition graph for left linear grammar :



Transition graph for right Linear grammar :



Right Linear grammar :

$S \rightarrow w_2A$ $S \rightarrow bbA$

$A \rightarrow w_2A / a$ $A \rightarrow bbA / a$

$F \rightarrow w_1F / w_1$ $F \rightarrow abF / ab$

Q.53. Obtain an equivalent left linear grammar from the given right linear grammar.

$S \rightarrow bA$

$A \rightarrow aA \mid bB \mid b$

$B \rightarrow bA$ ***CT : W-06, S-08(5M)***

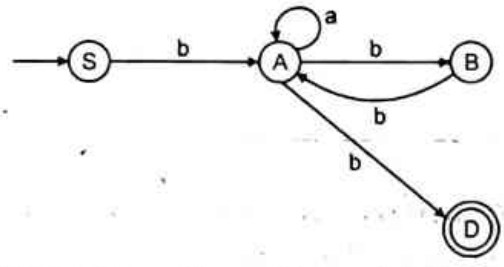
Ans. Given Right linear grammar is,

$S \rightarrow bA$

$A \rightarrow aA \mid bB \mid b$

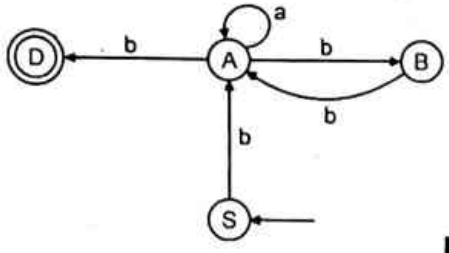
$B \rightarrow bA$

Transition graph for right linear grammar :



Now exchange the positions of initial and final states and reverse the directions of all transitions.

Transition graph for left linear grammar :



This is the finite automata for left linear grammar.

Production for left linear grammar:

$S \rightarrow Ab$

$A \rightarrow Aa \mid Bb \mid b$

$B \rightarrow Ab$

Q.54. Consider the following grammar :

$S \rightarrow A1$

$A \rightarrow A\ 10 \mid 01$

Obtain an equivalent Right Linear Grammar.

Ans. Given grammar is,

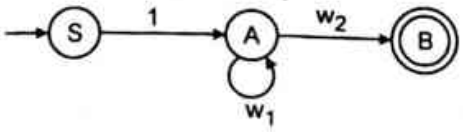
$S \rightarrow A1$

$A \rightarrow A10 \mid 01$

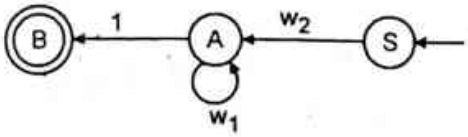
Let $w_1 = 10$ and

$w_2 = 01$

Transition graph for left linear grammar :



Transition graph for right Linear grammar :



Production for right linear grammar is :

$S \rightarrow w_2A \rightarrow 01A$

$A \rightarrow w_1A \rightarrow 10\ A \mid 1$

Q.55. Convert the following right linear grammar to its equivalent left-linear grammar :

$S \rightarrow 1B$

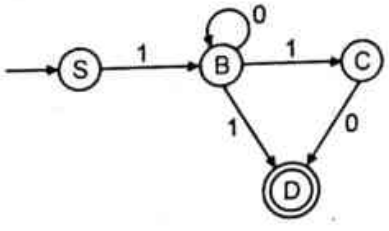
$B \rightarrow 1C$

$B \rightarrow 0B$

$B \rightarrow 1$

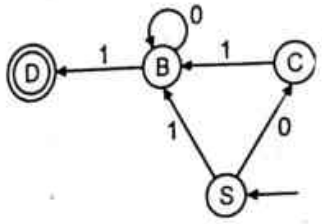
$C \rightarrow 0.$ ***CT : W-07(4M)***

Ans. The transition graph for given right linear grammar :



Exchange the position of initial state and final state. Also reverse the directions of transitions.

Transition graph for left linear grammar :



This is transition graph of left linear grammar.

Production for left linear grammar :

$S \rightarrow B1 \mid C0$

$B \rightarrow B0 \mid 1$

$C \rightarrow B1$

Q.56. Obtain an equivalent right linear grammar for the left linear grammar given below.

$S \rightarrow Abc / c$

$A \rightarrow Acc / bc.$ ***CT : W-09(7M)***

Ans. Given grammar is,

$S \rightarrow Abc / c$

$A \rightarrow Acc / bc$

Reversing the side of productions

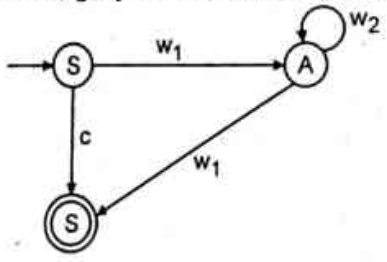
$S \rightarrow cbA / c$

$A \rightarrow ccA / cb$

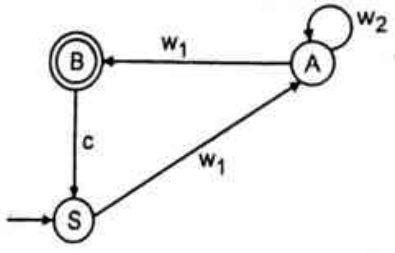
Replace '/' by + and $\rightarrow$ by '='

Let $w_1 = bc$ and $w_2 = cc$

Transition graph for left linear grammar :



Transition graph for right linear grammar :



Production of right Linear grammar :

$S \rightarrow w_1A \rightarrow bcA \mid c$

$A \rightarrow ccA \mid bc.$

Q.57. Obtain an equivalent left linear grammar from the given right linear grammar.

S → 01A

A → 10A / 1 **CT : S-10(7M)**

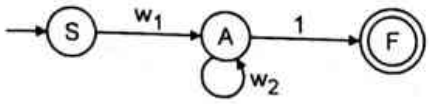
Ans. Let $w_1 = 01$

and $w_2 = 10$

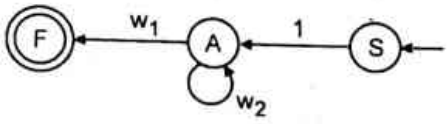
$S \rightarrow w_1 A$

$A \rightarrow w_2 A / 1$

Transition graph for right linear grammar :



Transition graph for left Linear grammar is,



Left Linear grammar :

$S \rightarrow A1$

$A \rightarrow Aw_2 / w_1$

$\therefore S \rightarrow A1$

$A \rightarrow A10 / 01$

Q.58. Obtain right linear grammar for following

S → aaA / bc

A → bbB / b

B → cA / c **CS : S-13(8M)**

Ans. Given grammar :

$S \rightarrow aaA / bc$

$A \rightarrow bbB / b$

$B \rightarrow cA / c$

is already in right linear grammar.

Q.59. Obtain an equivalent left linear grammar for following

S → abA | aa

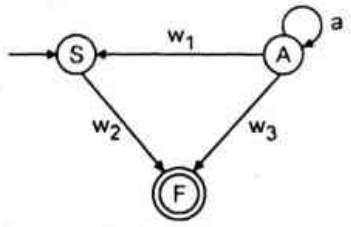
A → aA | bb **CS : S-12(5M)**

Ans. Let , $w_1 = ab$

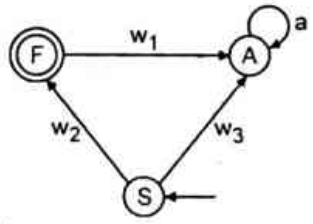
$w_2 = aa$

$w_3 = bb$

Transition graph for right linear grammar is,



Transition graph for left linear grammar is,



Equivalent left linear grammar :

$S \rightarrow Aw_3 / w_2$

$A \rightarrow Aa / w_1$

$\therefore S \rightarrow Abb / aa$

$A \rightarrow Aa / ab$

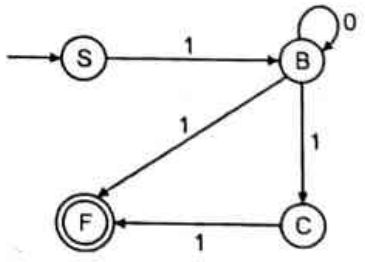
Q.60. Convert the following left linear grammar to right linear grammar.

S → B1

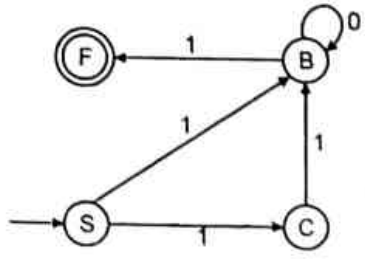
B → C1 | B0 | 1

C → 1 **CT : W-12(5M)**

Ans. Transition graph for left linear grammar :



Transition graph for right linear grammar :



Production for right linear grammar :

$S \rightarrow 1B / 1C$

$B \rightarrow 0B / 1$

$C \rightarrow 1B$

Q.61. Construct the left linear and right linear grammar for the language represented by regular expression (((01 + 10)* 00) * 11) *. **CT : S-07(10M)**

Ans.

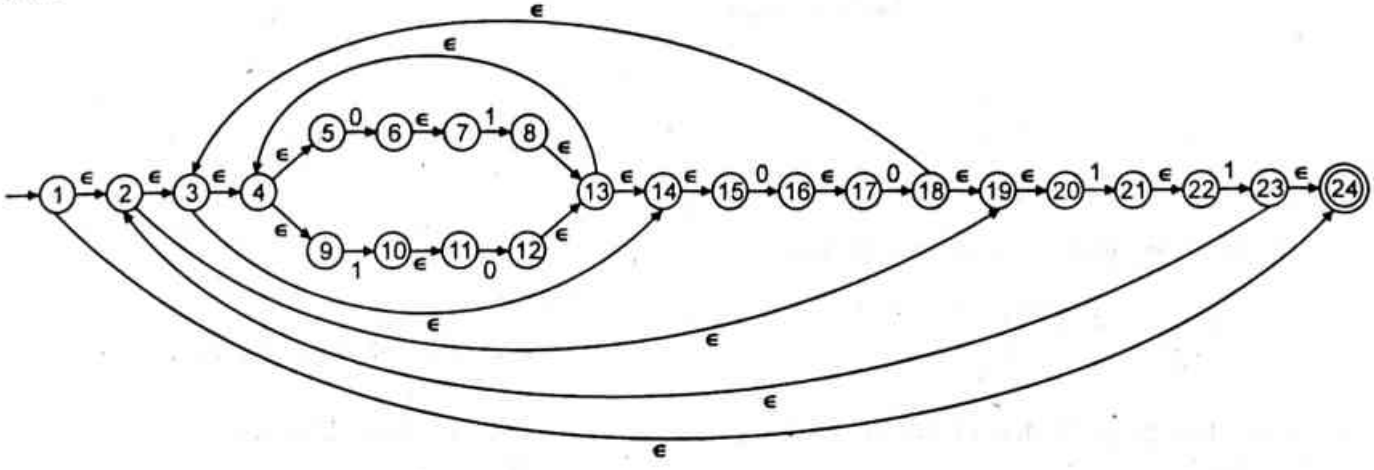


Fig. DFA

Transition table for the DFA :

Sr. N0.	State / Σ	0	1
1.	A* = (1,2,3,4,5,9,19,20,24,14,15)	B = (6,7, 16,17)	C = (10, 21,11,22)
2.	B = (6,7,16,17)	D =(18,19,20,3,4,5,14,15)	E = (8,13,14,15,4,5,9)
3.	C = (10,11, 21,22)	F = (12,13,14,15,4,5,9)	G = (23,2,3,4,5,9,14,15,19,20)
4.	D = (18,19,20, 3, 4, 5,14,15)	B = (6,16,7,17)	H = (10, 11)
5.	E = (8,13,14,15, 4,5,9)	B = (6,16,7,17)	H = (10, 11)
6.	F = (2,3,4,5,9,14,15,19,20,23)	B = (6,16,7,17)	H = (10, 11)
7.	G = (23,2,3,4,5,9,14, 15,19,20)	B = (6,16,7,17)	H = (10, 11)
8.	H = (10,11)	B = (6,16,7,17)	H = (10, 11)

Transition graph for DFA :

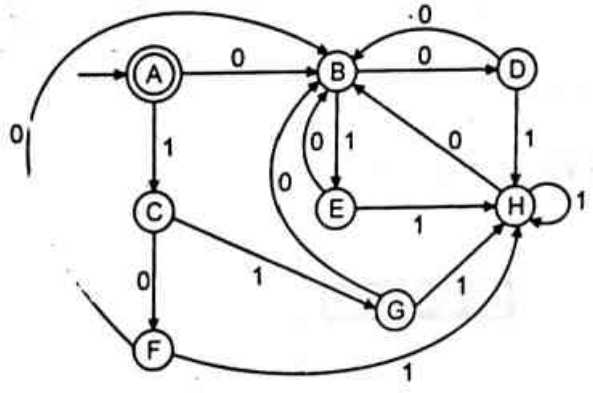


Fig. Transition graph for DFA

∴ **Right linear grammar is**

A → 0B / 1C
B → 0D / 1E
C → 0F / 1G
D → 0B / 1H
E → 0B / 1H
F → 0B / 1H
G → 0B / 1H
H → 0B / 1H

∴ **Left linear grammar is**

A → B0 / C1
B → D0 / E1
C → F0 /G1
D → B0 / H1
E → B0 / H1
F → B0 / H1
G → B0 / H1
H → B0 / H1

Q.62. Obtain right linear grammar equivalent to left linear grammar given below :

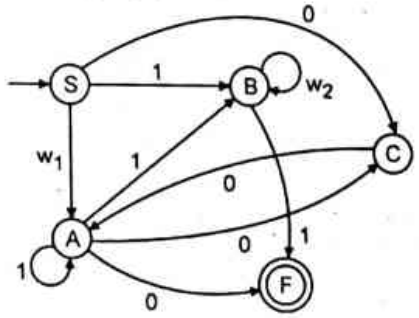
S → B1 / A00 / C0
A → C0 / A1 / B1 / 0
B → B11 / 1
D → A0

CT : S-12(6M)

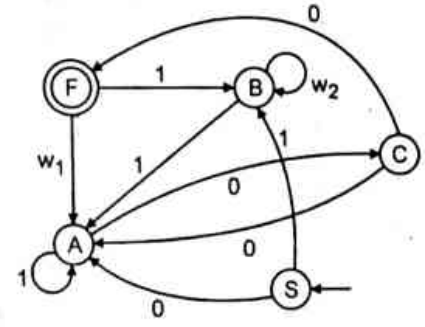
Ans. Let, 00 = w_1

11 = w_2

Transition graph for left linear Grammar :



Now, Transition graph for right linear grammar :



∴ **Production for right linear grammar :**

S → 1B / 0A
A → 00 / 1A / 0C
B → 1 / 11B / 1A
D → 0A / 0

Q.63. For the following right linear grammar, obtain equivalent left linear grammar

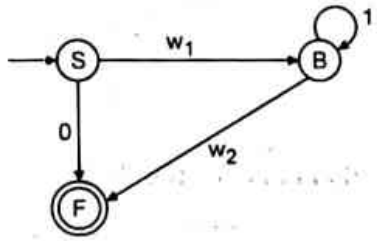
S → 01B / 0
A → 1B / 11

CS : W-11(4M)

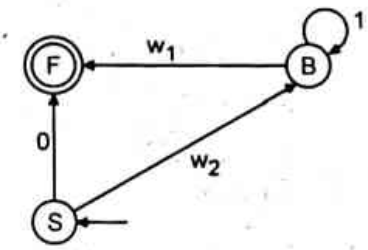
Ans. Let , $w_1 = 01$

$w_2 = 11$

Transition graph for right linear grammar :



Now, Transition graph for left linear grammar :



∴ **Left Linear grammar :**

S → Bw_2 / 0
A → B_1 / w_1
∴ S → B_{11} / 0 B → B_1 / 01

ARDEN THEORM

Q.64. State and explain Arden's theorm.

Ans. Arden's theorm : Let P and Q be two regular expressions over Σ. If P does not contains ϵ, then the equation in R, i.e. R = Q + RP has a unique solution given by R = QP*.

Proof :

R = Q + RP
= Q + (QP*) P *[∵ R = QP*]*
= Q (ϵ + P*P)
= QP*

Thus $R = QP^*$ is a solution of equation (1). To prove uniqueness, consider equation (I). Here, replacing R by Q + RP on the R.H. S., we get the equation

$$Q + RP = Q + (Q + RP)P$$
$$= Q + QP + RPP$$
$$= Q + QP + RP^2$$
$$= Q + QP + QP^2 + \ldots\ldots + QP^i + RP^{i+1}$$
$$= Q(\epsilon + P + P^2 + \ldots\ldots + P^i) + RP^{i+1}$$

From equation (1),

$R = Q(\epsilon + P + P^2 + \ldots\ldots + P^i) + RP^{i+1}$ for $i \geq 0$

- We now prove that any solution (1) is equivalent to QP^*. If R satisfies equation (1), then it satisfies equation (2).
- Let w be a string of length z in the set R. Then w belongs to the set $Q(\epsilon + P + P^2 + \ldots\ldots + P^i) + RP^{i+1}$.
- As P does not contain ϵ, RP^{i+1} has no string of length less then i + 1 and so w is not in the set RP^{i+1}. This means that w belongs to the set $Q(\epsilon + P + P^2 + \ldots\ldots + P^i)$, and so to QP^*.
- Consider a string w in the set QP^*. Then w is in the set QP^k for some $k \geq 0$, and thus in $Q(\epsilon + P + P^2 + \ldots\ldots + P^k)$.
- So w is on the R.H.S. of equation (2).
- Thus, w is in R (L.H.S. of equation (2)). Thus R and QP^* represent the same set.
- This proves the uniqueness of the solution of equation (1).

Q.65. Explain steps used to construct finite automata equivalent to given regular expression.

CT : W-06(4M)

Ans. Steps for converting finite automata to regular expression :

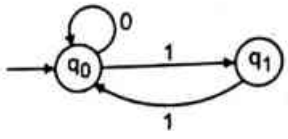
(i) Always add ϵ to equation of initial state.

(ii) The regular expression is obtained from the equation of final state.

(iii) If there are more than one final state, then final regular expression is obtained by adding all the final states.

(iv) The equations for states can be written for all incoming transitions to states.

Example : Consider diagram

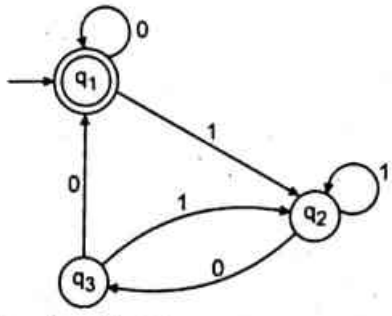


Equations for states :

$q_0 = q_0 0 + q_1 1 + \epsilon$ *[∵ q_0 is initial]*

$q_1 = q_0 1$

Q.66. Construct a regular expression corresponding to the state diagram given below :



Ans. Equation for all state :

$q_1 = q_1 0 + q_3 0 + \epsilon$ ....(1)

$q_2 = q_1 1 + q_2 1 + q_3 1$ ....(2)

$q_3 = q_2 0$ .... (3)

from 1 and 3

$q_1 = q_1 0 + q_2 00 + \epsilon$ ....(4)

$q_2 = q_1 1 + q_2 1 + q_2 01$

$$q_2 = \underbrace{q_1 1}_{R\ \ Q} + \underbrace{q_2}_{R} \underbrace{(1 + 01)}_{P}$$

By Arden's theorem

$q_2 = q_1 1 (1 + 01)^*$ ...(5)

From equation (4) and (5)

$q_1 = q_1 0 + q_1 1 (1 + 01)^* 0 0 + \epsilon$ ....(4)

$$q_1 = \underset{R}{q_1} \underbrace{(0 + 1(1 + 01)^* 00}_{P} + \underbrace{\epsilon}_{Q}$$

By Arden's theorem

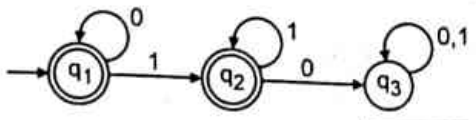
$q_1 = \epsilon (0 + 1 (1 + 01)^* 00)^*$

∴ Regular Expression :

$q_1 = (0 + 1 (1 + 01)^* 00)^*$

Q.67. Describe in English the set accepted by finite automata whose transition diagram is as shown in the fig.

OR Convert given transition graph into regular expression.



CT : S-09(6M)

Ans. Equation for all states :

$q_1 = q_1 0 + \epsilon$ ....(1)

$q_2 = q_1 1 + q_2 1$ ....(2)

$q_3 = q_2 0 + q_3 0 + q_3 1$ .... (3)

$\underbrace{q_1}_{R} = \underbrace{q_1}_{R}\underbrace{0}_{P} + \underbrace{\epsilon}_{Q}$

By Arden's theorem

$q_1 = \epsilon 0^*$

$q_1 = 0^*$ ....(4)

$q_2 = q_1 1 + q_2 1$

$\underbrace{q_2}_{R} = \underbrace{0^* 1}_{Q} + \underbrace{q_2}_{R}\underbrace{1}_{P}$ *[∵ From (4)]*

By Arden's theorem

$q_2 = 0^* 1 1^*$ ....(5)

Add equation (4) and (5)

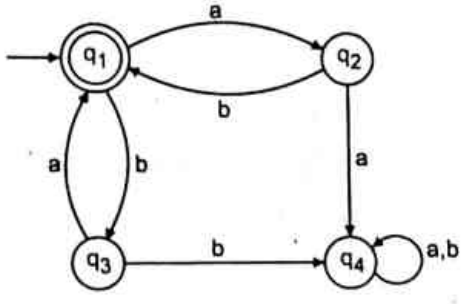
$q_1 + q_2 = 0^* + 0^* 1 1^*$

$= 0^* (\epsilon + 1 1^*)$

Regular Expression = $0^* 1^*$

[∵ $\epsilon + R.R^* = R^*$]

Q.68. Prove that the finite automata whose transition diagram is given in the fig accepts the set of all strings over {a, b} with an equal number of a's and b's.



Ans. Equation for all states :

$q_1 = q_2 b + q_3 a + \epsilon$ ....(1)

$q_2 = q_1 a$ ....(2)

$q_3 = q_1 b$ .... (3)

$q_4 = q_2 a + q_3 b + q_4 (a + b)$ ....(4)

$q_1 = q_1 ab + q_1 ba + \epsilon$

[∵ From (2) and (3)]

By Arden's theorem

$q_1 = \epsilon (ab + ba)^*$

$q_1 = (ab + ba)^*$

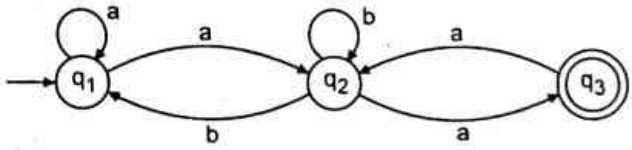
Regular Expression = $(ab + ba)^*$.

Q.69. Transition system given in figure. Prove that the strings recognized are :

$(a + a (b + aa)^* b)^* a (b + aa)^* a$.

CT : S-08,10(7M)

OR Find the regular expression for the following automata. **CS : W-11(7M), CT : S-12(6M)**



Ans. Equation for all states :

$q_1 = q_1 a + q_2 b + \epsilon$ ....(1)

$q_2 = q_1 a + q_2 b + q_3 a$ ....(2)

$q_3 = q_2 a$ .... (3)

$q_2 = q_1 a + q_2 b + q_2 aa$ *[∵ From (3)]*

$\underbrace{q_2}_{R} = \underbrace{q_1 a}_{Q} + \underbrace{q_2}_{R}\underbrace{(b + aa)}_{P}$

By Arden's theorem

$q_2 = q_1 a (b + aa)^*$ ....(A)

$q_1 = q_1 a + q_1 a (b + aa)^* b + \epsilon$ *[∵ From (1)]*

$\underbrace{q_1}_{R} = \underbrace{q_1}_{R}\underbrace{(a + a (b + aa)^* b)}_{P} + \underbrace{\epsilon}_{Q}$

By Arden's theorem

$q_1 = \epsilon (a + a (b + aa)^* b)^*$

$q_1 = (a + a (b + aa)^* b)^*$

$q_2 = (a + a(b + aa)^* b)^* a (b + aa)^*$

[∵ From (A)]

$q_2 = (a + a(b + aa)^* b)^* a (b + aa)^*$

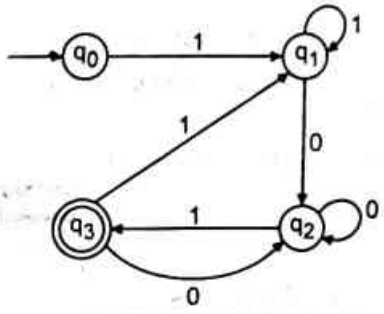
Regular Expression :

$q_2 = (a + a(b + aa)^* b)\, a (b + aa)^* a$

..... *[∵ From (3)]*

Q.70. Find the regular expression corresponding to the automation given in fig. below.

CS : S-10(6M), W-12(5M), CT : S-11, W-11(7M)



Ans. Equation for all states are :

$q_0 = \epsilon$ ....(1)

$q_1 = q_0 1 + q_1 1 + q_3 1$ ....(2)

$q_2 = q_1 0 + q_2 0 + q_3 0$ .... (3)

$q_3 = q_2 1$ ....(4)

From equation (3)

$q_2 = q_1 0 + q_2 0 + q_2 10$

$\underbrace{q_2}_{Q} = \underbrace{q_1 0}_{Q} + \underbrace{q_2}_{R}\ \underbrace{(0 + 10)}_{P}$

By Arden's theorem

$q_2 = q_1 0 (0 + 10)^*$ .....(A)

From equation (2),

$q_1 = 1 + q_1 1 + q_2 11$ *[∵ $q_0 = \epsilon$]*

$= 1 + q_1 1 + q_1 0 (0 + 10)^* 11$ *[∵ From (A)]*

$\underbrace{q_1}_{R} = \underbrace{q_1}_{R}\ \underbrace{(1 + 0 (0 + 10)^* 11)}_{P} + \underbrace{1}_{Q}$

By Arden's Theorem

$q_1 = 1 (1 + 0 (0 + 10)^* 11)$ ....(B)

Since $q_3 = q_2 1$

$q_3 = q_1 0 (0 + 10)^* 1$

R.E. $= 1 (1 + 0 (0 + 10)^* 11)\ 0 (0 + 10)^* 1$

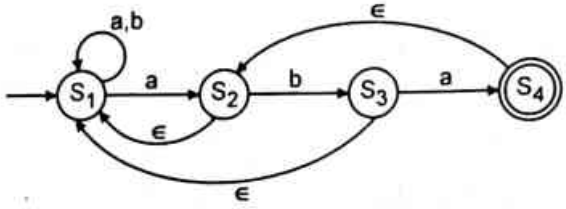
Q.71. Prove that the set (a + b)* aba is accepted by non-deterministic finite automaton:

$M = (\{S_1, S_2, S_3, S_4,\}, \{a, b\}, \delta, S_1, \{S_4\})$

where δ state transitions is as follows :

δ	a	b	∈
S_1	$\{S_1, S_2\}$	$\{S_1\}$	-
S_2	-	$\{S_3\}$	$\{S_1\}$
S_3	$\{S_4\}$	-	$\{S_1\}$
S_4	-	-	$\{S_2\}$

Ans. The Transition graph for NFA with ∈ - moves:



The equations of states are

$S_1 = (a + b) S_1 + \epsilon S_2 + \epsilon S_3$ ....(1)

$S_2 = bS_3 + aS_1$

$= bS_3 + S_1$ *[∵ ∈ . R = R]* ....(2)

$S_3 = aS_4 + \epsilon S_1$ ....(3)

$= aS_4 + S_1$

$S_4 = \epsilon S_2 + \epsilon$

$= S_2 + \epsilon$ ....(4)

From equation (3) and (4)

$S_3 = aS_4 + S_1$

$S_3 = a(S_2 + \epsilon) + S_1$

$S_3 = aS_2 + a + S_1$ ....(5)

From equation (2) and (5)

$S_2 = bS_3 + S_1$

$= b(aS_2 + a + S_1) + S_1$

$= baS_2 + ba + bS_1 + S_1$

$\therefore S_2 = (ba)^* . (ba + bS_1 + S_1)$ ....(6)

From equation (1) and (6)

$S_1 = (a + b) S_1 + aS_2$

$= (a + b) S_1 + a ((ba)^* (ba + bS_1 + S_1))$

$= (a + b) S_1 + a ((ba)^* . ba + abS_1 + aS_1$

$= ((a + b) + ab + b) S_1 + a (ba)^* . ba$

$S_1 = (a + b) S_1 + a(ba)^* . ba$

$[\because ((a + b) + ab + a) = a + b]$

$= (a + b) S_1 + aba (ba)^*$

$[\because R^* . R = R . R^*]$

$S_1 = (a + b)^* . aba (ba)^*$

But (ba)* means zero or occurances of ba. When ba will occur more than once in the string then the string will end with aba.

Hence we can write

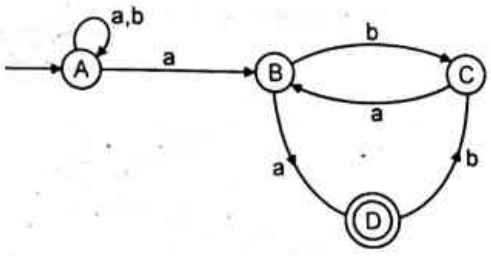
$(ba)^* = aba$

$\therefore S_1 = (a + b)^* . aba .$

Q.72. Given the finite automation in figure, determine the regular expression that represents the language accepted by the automaton.

OR Write a regular expression for the language accepted by given finite automata.

CT : S-08(8M)



Ans. Let rename the states as,

$A = q_0, B = q_1, C = q_2, D = q_3$

The state equations are,

$q_0 = q_0 (a + b) + \epsilon$ ....(1)

$q_1 = q_0 a + q_2 a$ ....(2)

$q_2 = q_1 b + q_3 b$ ....(3)

$q_3 = q_1 a$ ....(4)

The regular expression will be obtained from final states q_3. Hence first find equation of state q_1.

From equation (1)

By Arden's Theorem

$q_0 = \epsilon (a + b)^*$

$q_0 = (a + b)^*$ $[\because \epsilon R^* = R^*]$ ....(5)

From equation (2) and (5)

$q_1 = (a + b)^* . a + q_2 a$ ....(6)

From equation (3) and (4)

$q_2 = q_1 b + q_1 ab$

$q_1 = q_1 (b + ab)$ ....(7)

From equation (6) and (7)

$q_1 = (a + b)^* . a + q1 (b + ab) a$

By Ardens Theorem

$q_1 = (a + b)^* . a ((b + ab)a)^*$ ....(8)

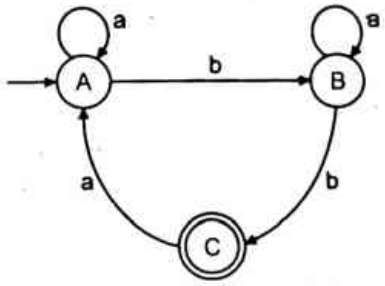
From equation (4) and (8)

Regular expression :

$q_3 = (a + b)^* . a (b + ab) . a)^* . a$

This is the regular expression that represents the languages accepted by finite automaton.

Q.73. Find the regular expression for the automata shown below : **CT : W-07(6M)**



Ans. Equations for states are,

$A = Aa + Ca + \epsilon$ ....(1)

$B = Ab + Ba$ ....(2)

$C = Bb$ ....(3)

From equation (2),

By Arden Theorem

$B = Aba^*$ ....(4)

From equation (1), (3) and (4)

$A = Aa + Bba + \epsilon$ $[\because C = Bb]$

$= Aa + Aba^* ba + \epsilon$

$= A (a + ba^* ba) + \epsilon$

By Ardens Theorem

$\therefore A = \epsilon (a + ba^* ba)^*$ $[\because \epsilon R^* = R^*]$

$\therefore A = (a + ba^* ba)^*$ ....(5)

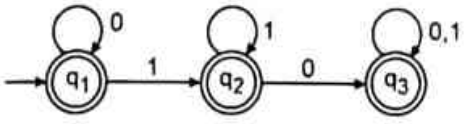
From equation (3), (4) and (5)

$C = Aba^*b$

Regular Expression :

$C = (a + ba^* ba)^* ba^* b .$

Q.74. Find out regular expression for transition diagram. **CT : S-06(6M)**



Ans. Equations for states are

$q_1 = q_1 0 + \epsilon$ ....(1)

$q_2 = q_1 1 + q_2 1$ ....(2)

$q_3 = q_2 0 + q_3 (0+1)$ ....(3)

From (1),

By Arden's Theorem

$q_1 = \epsilon 0^* = 0^*$ ....(4)

From equation (2) and (4)

$q_2 = 0^* 1 + q_2 1$

By Arden's Theorem

$q_2 = 0^* 11^*$ ....(5)

From equation (3) and (5)

$q_3 = 0^* 11^* 0 + q_3 (0+1)$

By Arden's Theorem

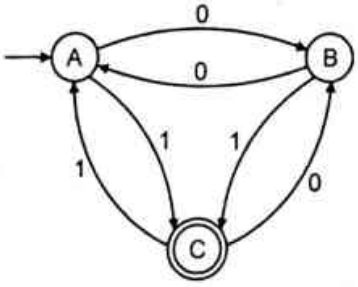
$q_3 = 0^* 11^* 0 (0+1)^*$ ....(6)

From equation (4), (5) and (6)

Regular expression :

$q_1 + q_2 + q_3 = 0^* + 0^* 11^* + 0^* 11^* 0 (0+1)^*$

Q.75. Construct the regular expression for the following state diagram. **CT : S-07(4M)**



Ans. The equations for states are,

$A = B0 + C1 + \epsilon$ ....(1)

$B = A0 + C0$ ....(2)

$C = A0 + B1$ ....(3)

From equation (2) and (3)

$B = A0 + A00 + B10$

$B = A (0 + 00) + B10$

By Arden's Theorem

$B = A (0 + 00) (10)^*$ ....(4)

From equation (1) and (3)

$A = B0 + A01 + B11 + \epsilon$

$= A01 + B (0 + 11) + \epsilon$

$= A01 + A (0 + 00) (10)^* (0 + 11) + \epsilon$

$A = A (01 + (0 + 00) (10)^* (0+11)) + \epsilon$

By Arden's Theorem

$A = \epsilon (01(0 + 00) (10)^* (0 + 11))^*$

$A = (01 (0 + 00) (10)^* (0 + 11))^*$ .....(5)

[$\because \epsilon R^* = R^*$]

From equation (3), (4) and (5)

$C = A0 + B1$

$C = (01 (0 + 00) (10)^* (0+11))^* 0 + A (0+00) (10)^*$

$= (01 (0 + 00) (10)^* (0+11)^* 0 (01(0+00)$

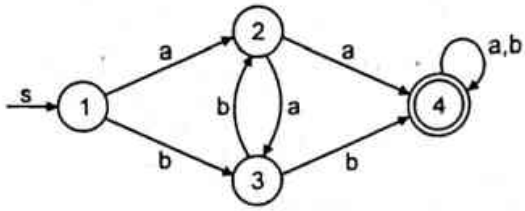
$(10)^* (0 + 11))^* 1$

$= (01(0 + 00) (10)^* (0 + 11))^* (0 + 1)$

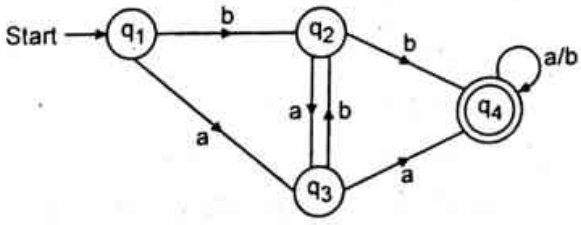
Regular Expression :

$(01(0 + 00) (10)^* (0 + 11))^* (0 + 1)$

Q.76. For the DFA shown in Fig. find the regular expression. **CS: S-09(8M)**



OR Write a regular expression for language accepted given finite automata. **CT: S-10(7M)**



Ans. The equations for states are

$q_1 = \epsilon$ ....(1)

$q_2 = q_1 a + q_3 b$ ....(2)

$q_3 = q_1 b + q_2 a$ ....(3)

$q_4 = q_2 a + q_3 b + q_4 (a + b)$ ....(4)

From equation (2) and (3)

$q_2 = q_1 a + q_1 bb + q_2 ab$

$q_2 = q_1 (a + bb) + q_2 ab$

By Arden's Theorem

$q_2 = q_1 (a + bb) (ab)^*$

$q_2 = \in (a+bb) (ab)^*$

$q_2 = (a + bb) (ab)^*$ *[$\because \in R = R$]*

Now, From equation (4)

$q_4 = (a + bb)(ab)^* a + (q_1 b + q_2 a) b + q_4 (a + b)$

$q_4 = (a + bb) (ab)^* a + (bb + (a +bb) (ab)^* ab) + q_4 (a + b)$

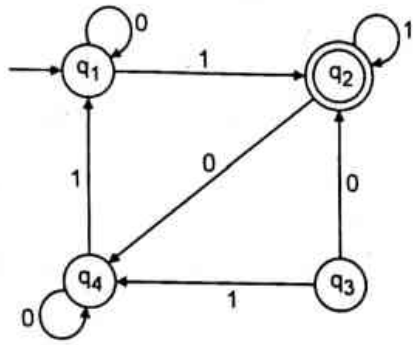
By Arden's Theorem

$q_4 = (a + bb) (ab)^* a + (bb + (a + bb) (ab)^* ab)$ $(a + b)^*$

$\therefore$ Regular Expression

$= (a + bb) (ab)^* a + (bb + (a + bb) (ab)^*ab)$ $(a + b)^*$

Q.77. Convert following FSM into regular expression :



Ans. The equations for states are,

$q_1 = q_1 0 + q_4 1 + \in$ ....(1)

$q_2 = q_2 1 + q_1 1 + q_3 0$ ....(2)

$q_3 = \in$ ....(3)

$q_4 = q_2 0 + q_4 0 + q_3 1$ ....(4)

[$\because q_3$ is not reachable state hence equation (3)]

$q_4 = q_2 0 + q_4 0$

By Arden's Theorem

$q_4 = q_2 00^*$ ....(5)

From equation (1)

$q_1 = q_1 0 + q_2 00^* 1 + \in$ .....(6)

From equation (2)

$q_2 = q_2 1 + q_1 1$

By Arden's Theorem

$q_2 = q_1 11^*$ ....(7)

From equation (6) and (7)

$q_1 = q_1 0 + q_1 11^* 00^* 1 + \in$

$q_1 = q_1 (0 + 11^* 00^* 1) + \in$

By Arden's Theorem

$q_1 = \in (0 + 11^* 00^* 1)^*$

$q_1 = (0 + 11^* 00^* 1)^*$ ....(8)

[$\because \in R^* = R^*$]

From equation (2),

$q_2 = q_1 11^* 1 + (0 + 11^* 00^* 1)^* 1$

[$\because$ from (7) and (8)]

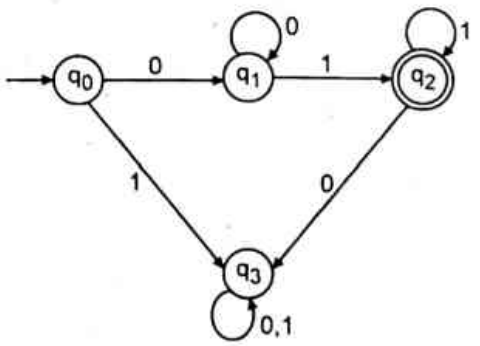
$= (0 + 11^*00^*1)^* 11^*1 + (0 + 11^*00^*1)^*1$

$= (0 + 11^*0^*1)^*1 (1^*1 + \in)$

$= (0 + 11^*0^*1)^*11^*$ *[$\because \in + RR^* = R^*$]*

Regular expression : $(0 + 11^* 0^* 1)^* 11^*$

Q.78. Find the regular expression for the given transition diagram. **CT-W-08(7M)**



Ans. The equations for states are,

$q_0 = \in$ ....(1)

$q_1 = q_0 0 + q_1 0$ .....(2)

$q_2 = q_1 1 + q_2 1$ ....(3)

$q_3 = q_2 0 + q_3 (0 + 1)$ ....(4)

From equation (1) and (2)

$q_1 = \in 0 + q_1 0$

$= 0 + q_1 0$ *[$\because \in 0 = 0$]*

By Arden's Theorem

$q_1 = 00^*$ ....(5)

From equation (3) and (5)

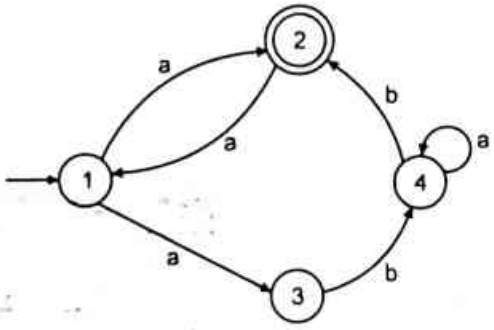
$q_2 = 00^* 1 + q_2 1$

By Arden's Theorem

$q_2 = 00^* 11^*$

Regular expression : $00^* 11^*$

Q.79.(a) Find regular expression for the FA shown in figure.



Ans. Let rename the states

$1 = q_1, 2 = q_2, 3 = q_3, 4 = q_4$

$q_1 = q_2a + \in$ ....(1)

$q_2 = q_1a + q_4b$ ....(2)

$q_3 = q_1a$ ....(3)

$q_4 = q_3b + q_4a$ ....(4)

From equation (3) and (4)

$q_4 = q_1ab + q_4a$

By Arden's Theorem

$q_4 = q_1aba^*$ ....(5)

From equation (2) and (5)

$q_2 = q_1a + q_1aba^*b$

$q_2 = q_1 (a + aba^*b)$ ....(6)

From (1) and (6)

$q_1 = q_1 (a + aba^*b) . a + \in$

By Arden's Theorem

$q_1 = \in (aa + aba^*ba)^*$

$q_1 = (aa + aba^*ba)^*$ *[$\because \in R^* = R^*$]* ....(7)

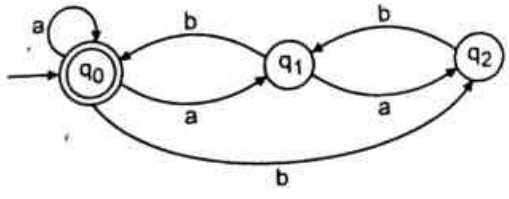
From equation (2), (5) and (7)

$q_2 = (aa + aba^*ba)^*a + q_1aba^*b$

$= (aa + aba^*ba)^*a + (aa + aba^*ba)^* . aba^*b$

Regular expression : $(aa + aba^*ba)^* (a + aba^*b)$

Q.79.(b) Design RE for the following FSM. Explain identity used for expression. ***CS : W-09(6M)***



Ans. Equations for all states.

(1) $q_0 = q_0a + q_1b + \in$

(2) $q_1 = q_0a + q_2b.$

(3) $q_2 = q_1a + q_0b .$

Subtract equation (3) in (2)

$q_1 = q_0a + (q_0b + q_1a) b$

$= q_0a + q_0bb + q_1ab$

$q_1 = q_0(a + bb) + q_1ab$

By Arden Theorem

$q_1 = q_0(a + bb) (ab)^*$ ....(4)

Now substitute value of equation (4) in equation (1)

$q_0 = q_0a + q_0 [(a + bb) (ab)^*] b + \in$

$= q_0a + q_0(a + bb) (ab)^* b + \in$

$q_0 = q_0 (a + [(a + bb) (ab)^* b] + \in$

By Arden theorem

$q_0 = \in [a + [(a + bb) (ab)^* b]^*$ *[$\because \in R = R$]*

$q_0 = [a + [(a + bb) (ab)^* b]^*$

PUMPING LEMMA FOR REGULAR SETS

Q.80. Explain pumping lemma in detail.

OR State principle of pumping lemma. ***CT : S-11,12(3M)***

OR State and prove the theorem of pumping lemma. ***CT : S-09(5M),CS : S-13(6M)***

Ans. Pumping lemma :

- Pumping lemma is a powerful tool for proving certain language are non-regular.
- It's also useful in the development of algorithm to answer certain questions concerning finite automata, such as whether the language accepted by a given FA is finite or infinite.
- It states that given any sufficiently long string accepted by an FSM, we can find a substring near the beginning of the string that may be repeated or pumped as many times as we like and the resulting string will still be accepted by, same FSM.

Theorem : Let $M = \{Q, \Sigma, \delta, q_0, F\}$ be a finite automata with n state. L be the regular set accepted by M. Let W belongs to L ($W \in L$) and $|W| \geq m$.

Proof :

- If $m \geq n$ then their exist, x, y, z such that $W = xyz$, $y \neq \in$ and $x y^i z \in L$ for each $i \geq 0$.

  Let $W = a_1, a_2 \ldots\ldots a_m \quad m \geq n$

  $\delta(q_0, a_1, a_2 \ldots\ldots a_m) = q_i, i = 1, 2 - - - - - m$

  $Q = \{q_0, q_1, \ldots\ldots q_m\}$

  where, Q is sequence of states in path with path valuc $W = a_1, a_2 \ldots a_m$.
- But there are only n state hence atleast two states in Q must coincide.

  Let take them as state q_j, q_k $(q_j = q_k)$
- The string W can be decomposed into 3-substring as

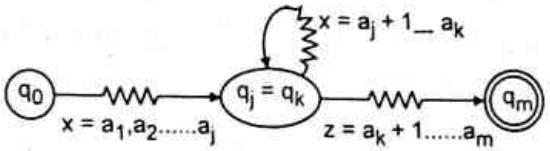
  $x = a_1, a_2 \ldots\ldots a_j$

  $y = a_j + 1 \ldots\ldots a_k$ and

  $z = a_k + 1 \ldots\ldots a_m$

  since, $k \leq n$

  $|xy| \leq n$ and $W = xyz$
- The path with path value W in the transition diagram of M is shown.



- The automata M start from the initial state q_0.

  On applying the string x, it reaches $q_j = q_k$.

  On applying the string y, it comes to $q_j = q_k$.
- So after application of y^i for each $i \geq 0$ the automata is in the same state q_j.
- On applying z it reaches q_m which is a final state.

  So XY^iZ is accepted by FAM. Hence, $xy^iz \in L$, since every state in Q is obtained by applying an input symbol and $y \neq \in$.

Q.81. Give formal statement of pumping Lemma for regular sets. ***CT : W-08(2M)***

Ans. Formal Statement of Pumping Lemma :

Let L be a regular set then there is a constant n such that if W is a string in L and $|W| \geq n$, we may write $W = xyz$ in such a way that $|xy| \leq n$ and $y \neq \in$ $(|y| \geq 1)$, $y = 1, 2, 3, \ldots\ldots n$

Hence, $1 \leq |y| \leq n$

And for all $i \geq 0$,

$xy^iz \in L$

Q.82. State some applications of Pumping Lemma. ***CT : W-08, S-09(2M)***

OR Explain the significance of pumping lemma. ***CS : S-11(2M)***

Ans. Applications of Pumping Lemma :

Pumping lemma can be used to prove that certain sets are not regular. Following are the steps needed for proving that a given set is not regular.

Step 1 : Assume that language L is regular. Let n be the number of states or constant of pumping lemma in the corresponding finite automaton.

Step 2 : Choose a string w such that $|w| \geq n$. Use pumping lemma to write $w = xyz$ with $|xy| \leq n$ and $|y| > 0$.

Step 3 : Find a suitable integer i such that $xy^iz \notin L$. This contradicts our assumption. Hence L is not regular.

Q.83. Prove that the language

$L = \{a^P / P$ is prime no.$\}$ is not regular. ***CT : W-08(2M)***

OR Shown that $L = \{a^P / P$ is a prime$\}$ is not regular. ***CT : W-09(6M), W-10(5M)***

Ans.

Step 1:

$P = 2$, length $= a^2 = aa = 2$

$P = 3$, length $= a^3 = aaa = 3$

$P = 5$, length $= a^5 = aaaaa = 5$

Hence the length of the string in L is always prime number.

Step 2:

Assume L is regular

Let n is called as constant of pumping lemma.

Let $W \in L$ and $W = xyz$

$\therefore W = a^n$

$\therefore |W| = n = |xyz|$

Step 3 :

For i = 1

$xyz \in L$

for i = 2

$\leq |y| \leq n$

$\leq |xy^2z| \leq n$ ....(1)

But xy^2z is the combination of y and xyz

Hence we will add the value of xyz i.e.

$|xyz| = n$ on both side of equation (1)

$n + 1 \leq |xy^2z| \leq n + n$

subtract 1and add 1

$n < |xy^2z| < 2n + 1$

For n = 1

$1 < |xy^2z| < 3$

Hence length = 2 (prime number)

For n = 2

$2 < |xy^2z| < 5$

Hence length = 3, 4 (Not prime number)

Hence the language L consists of a strings whose length are not always prime.

Thus our assumption that L is regular is wrong.

Hence by applying pumping lemma we prove that, L is not regular.

Q.84. Prove that the following language is not regular. ***CT : S-06(4M)***

$L = \{0^i1^i \mid i \geq 1\}$

OR Show $\{a^nb^n \mid n \geq 1\}$ is not regular by using pumping lemma. ***CT : S-11(4M)***

Ans.

Step 1:

i = 1, length = $0^11^1 = 01$ = 2

i = 2, length = $0^21^2 = 0011$ = 4

i = 3, length = $0^31^3 = 000111$ = 6

Hence length of string in L is always even number.

Step 2:

Assume L is regular

Let n is called as constant of pumping lemma.

Let $W \in L$ and $W = xyz$

$\therefore W = 0^n1^n$

$\therefore |W| = 2n = |xyx|$

Step 3 :

For i = 1

$xyz \in L$

For i = 2

$\leq |y| \leq n$

$\leq |xy^2z| \leq n$ ....(1)

But xy^2z is a combination of y and xyz

Hence we will add the value of xyz i.e.$|xyz| = 2n$ on both side of equation (1)

$2n + 1 \leq |xy^2z| \leq n + 2n$

$2n + 1 \leq |xy^2z| \leq 3n$

Subtract 1 and Add 1

$2n < |xy^2z| < 3n + 1$

For n = 1

$2 < |xy^2z| < 4$

Length =3 (Not even number)

Thus, the language L consists of a strings whose length are not always even number.

So, our assumption that L is regular is wrong.

Hence by applying pumping lemma we prove that, L is not regular.

Q.85. Show that the set $L = \{a^{i^2} \mid i \geq 1\}$ is not regular. ***CS : S-10(7M)***

OR Using pumping lemma, prove that the language : $L = \{a^{i^3} / i \geq 1\}$ is not regular. ***CS : W-12(5M)***

Ans.

Step 1:

i = 1, length = $a^{1^2} = a$ = 1

i = 2, length = $a^{2^2} = aaaa$ = 4

i = 3, length = $a^{3^2} = aaaaaaaaa$ = 9

i = 4, length = $a^{4^2} = aaaaaaaaaaaaaaaa$ = 16

Hence length of string in L is always perfect square.

Step 2:

Assume L is regular

Let n is called as constant of pumping lemma.

Let $W \in L$ and $W = xyz$

$\therefore \quad W = a^{n^2}$

$\therefore |W| = n^2 = |xyz|$

Step 3 :

For i = 1

$xyz \in L$

for i = 2

$1 \le |y| \le n$

$1 \le |xy^2z| \le n$ ....(1)

But xy^2z is a combination of y and xyz

Hence we will add the value of $|xyz| = n^2$ on both side of equation (1)

$n^2 + 1 \le |xy^2z| \le n + n^2$

Subtract 1 and Add 1

$n^2 < |xy^2z| < n + n^2 + 1$

For n = 1

$1 < |xy^2z| < 3$

Length =2 (Not a perfect square)

Hence the length of strings in L is not always a perfect square.

Hence our assumption that L is regular is wrong.

Hence by applying pumping lemma we prove that L is not regular.

Q.86. Prove that the following language is non-regular using pumping lemma $L = \{a^n b^{n+1} \mid n > 0\}$.

CS : S-09(7M)

Ans.

Step 1:

n = 1, length = a^1b^2 = abb = 3

n = 2, length = a^2b^3 = aabbb = 5

n = 3, length = a^3b^4 = aaabbbb = 7

Hence length of string in L is always odd.

Step 2:

Assume L is regular

Let m be a constant of pumping lemma.

Let $W \in L$ and $W = xyz$

$W = a^m b^{m+1}$

Hence $|W| = m + m + 1 = |xyz|$

$|W| = 2m + 1 = |xyz|$

Step 3 :

For i = 1

$yz \in L$

For i = 2

$\le |y| \le m$

$\le |xy^2z| \le m$ ....(1)

But xy^2z is a combination of y and xyz.

Hence we will add the value of $|xyz| = 2m + 1$ on both side of equation (1)

$2m + 1 + 1 \le |xy^2z| \le m + 2m + 1$

$2m + 2 \le |xy^2z| \le 3m + 1$

Subtract 1 and Add 1

$2m + 1 < |xy^2z| < 3m + 2$

For m = 1

$3 < |xy^2z| < 5$

Length =4 (Not a odd number)

Hence the length of string in L is not always odd.

Thus, our assumption that L is regular is wrong.

Hence by applying pumping lemma we prove that, L is not regular.

Q.87. Prove that the following language are not regular. $L = \{WW \mid W \in (a, b)^*\}$

CT : S-06(4M)

Ans. We prove the result by method of contradiction.

Assume L is regular.

Let n be the number of states in the automaton M accepting 'L'

Let $WW = a^n b a^n b$ is in L

$\therefore L = \{abab, aabaab, aaabaaab \ldots..\}$

Let

$y = ab,$

$x = ab$ and

$Z = \epsilon$

$\therefore w = XY^iZ$

$= ab\,(ab)^i \in$

For i = 1,

$W = ab\,(ab)^1$

= abab is in L

For i = 2,

$W = ab(ab)^2$

= ababab which is not in L

Hence

$L = \{WW \mid W \in (a, b)^*\}$ is not regular.

Q.88. Show that by using Pumping Lemma the language given below is not regular.

$L = \{0^n 1^m 0^{n+m} \mid n, m \geq 1\}$.

OR Using pumping lemma, prove that the language : Prove $\{a^n b^m c^{m+n} \mid m, n >= 1\}$ is not regular.

CS : W-09,12(5M)

Ans.

Step 1 :

Let m = 1, n = 1, length = $0^1 1^1 0^{1+1} = 0100 = 4$

Let m = 1, n = 2, length = $0^2 1^1 0^{1+2} = 001000 = 6$

Let m = 2, n = 2, length = $0^2 1^2 0^{2+2} = 00110000 = 8$

From above, length of every string in language L starts from 4 and is always even.

Step 2 :

Assume L is regular.

Let l be the constant of pumping lemma.

Let $w = 0^l 1^l 0^{l+l}$ and w = xyz

$|w| = 4l = |xyz|$ (l = 1, 2, 3.....)

Step 3 :

We have $1 \leq |y| \leq l$

for i = 1,

$xy^i z \in L$

for i = 2

$1 \leq |xy^2z| \leq l$ ....(1)

But xy^2z is combination of xyz and y.

So add |xyz| on both sides of equation (1)

$4l + 1 \leq |xy^2| \leq l + 4l$

$4l + 1 \leq |xy^2z| \leq 5l$

Subtract 1 and Add1

$4l < |x\ y^2z| < 5l + 1$

For l = 1, $4 < |x\ y^2z| < 6$

Length = 5 (Not even)

For l = 2, $8 < |x\ y^2| < 11$

Length = 9, 10 (Not always even)

Thus, length of string in L is not always even.

So, our assumption that L is regular is wrong.

Hence by pumping lemma L is not regular.

Q.89. Use pumping lemma to prove that the language. $L = \{0^n 1^{n+n} \mid n > 0\}$ is non-regular.

OR Using the pumping lemma show that the following language is not regular.

$\alpha = \{a^n b^{2n} \mid n > 0\}$ **CT : W-11(5M)**

OR Prove that $L = \{a^i b^{2i} \mid i \geq 1\}$ is non regulator by using lemma. **CT : S-12(4M)**

Ans.

Step 1 :

For n = 1, length = $0^1 1^2 = 011 = 3$

For n = 2, length = $0^2 1^4 = 001111 = 6$

For n = 3, length = $0^3 1^6 = 000111111 = 9$

Hence length of the strings in L is multiple of 3.

Step 2 :

Assume L is regular.

Let l be constant of pumping lemma and $w = 0^l 1^{l+l}$ and w = xyz

$\therefore |w| = 3l = |xyz|$

Step 3 :

We have, $1 \leq |y| \leq l$

for i = 1, $xyz \in L$

for i = 2,

$1 \leq |xy^2z| \leq l$ ....(1)

But xy^2z is combination of xyz and y

Hence we add |xyz| to equation (1)

$\therefore 3l + 1 \leq |x\ y^2| \leq 3l + l$

$3l + 1 \leq |xy^2z| \leq 4l$

Subtract 1 and Add1

$3l < |xy^2z| \leq 4l + 1$

When l = 1, $3 < |xy^2z| < 5$

Length = 4 (Not multiple of 3)

Thus, length of string in L is not always multiple of 3.

So, our assumption that L is regular is wrong.

Hence by pumping lemma L is not regular.

VBD

Q.90. Using pumping lemma prove that following language is not regular.

$\alpha = \{a^m b^n \mid m \neq n\}$ **CS : S-11(3M)**

Ans. Given : $m \neq n$

Let L be a regular language and n be a pumping lemma constant.

Let $x = a^m b^n$

where $l \neq n$

By pumping lemma theorem, w can be written as,

$w = xyz$ (given), $|xy| \leq n$ and $|y| \geq 1$

Hence $1 \leq |y| \leq n$ and $xy^i z \in L$

consider, $y = a^i b^i$

Then $xyz = a^{m-i} b^{n-j}$

If $m - i = n - j$ this will violate the condition of $m \neq n$ in L and we do not want equal no of a and b.

Hence by pumping lemma language is not regular.

Q.91. Prove $L = \{a^n b^n c^m \mid m \neq n\}$ is not regular.

Ans. $L = \{a^n b^n c^m \mid m \neq n\}$

$L = \{abcc, aabbc, aaabbbcc, \ldots\ldots\}$

Let $x = \epsilon$

$y = ab$

$z = cc$

$w = xy^i z$

$= \epsilon (ab)^i cc$

For $i = 1$, $w = (ab)^1 cc = abcc \in L$

For $i = 2$, $w = (ab)^2 cc = aabbcc \notin L$

Therefore $L = \{a^n b^n c^m \mid m \neq n\}$ is not regular.

Q.92. Show that $L = \{a^{n!} ; n \geq 0\}$ is not regular.

CS : W-11(5M)

Ans.

Step 1 :

$n = 0$, length $= a^{0!} = a = 1$

$n = 1$, length $= a^{1!} = a = 1$

$n = 2$, length $= a^{2!} = aa = 2$

$n = 3$, length $= a^{3!} = aaaaaa = 6$

Hence length of the strings in L is multiple of 3.

Step 2 :

Assume L is regular.

Let 'm' is called as constant of pumping lemma

Let $W \in L$ and $W = xyz$

$W = a^m$

$|W| = m = |xyz|$

Step 3 :

For $i = 1$

$xyz \in L$

For $i = 2$

$1 \leq |y| \leq m$

$1 \leq |xy^2 z| \leq m \ldots (1)$

But $xy^2 z$ is combination of y and xyz

Hence, we will add value xyz i.e.

$|xyz| = n!$ on both side of equation (1)

$n! + 1 \leq |xy^2| \leq n + n!$

Subtract 1 and Add 1

$n! < |xy^2 z| < n + 1 + n!$

For $n = 1$,

$1 < |xy^2 z| < 1 + 1 + 1!$

$1 \leq |xy^2 z| \leq 3$

For $n = 2$

$2 \leq |x\ y^2 z| \leq 2 + 1 + 2$

$2 \leq |x\ y^2 z| \leq 5$

Length is not factorial of any number

Hence, length of string in L is not always factorial.

So, our assumption that 'L' is regular is wrong.

Hence by pumping lemma 'L' is not regular.

Q.93. Show that given language is not regular using pumping lemma.

$L = \{0^i 1^j 0^k / k > i + j\}$ **CT : W-12(6M)**

Ans. Let given $i = l$ and $j = l$ and $i + j = k$

Put $w = 0^l 1^l 0^{2l}$ The string y must then be a^k and the pumped string will be

$w_i = 0^{l+(i-1)k} b^l a^{2l}$

If we take $i \geq 2$ then $l + (i-1) > l$ then w_i is not regular.

$\therefore$ $L = \{0^i 1^j 0^k / k > i + j\}$ is not regular.

Q.94. Prove that $\{0^i1^m2^m / i \geq 1, m \geq 1\}$ is not regular using pumping lemma. **CS : S-12(5M)**

Ans. We prove this by contradiction.

Assume L = T(m) for some DFA with n states.

Choose $w = 0^n1^{2n}2^{2n}$ in L.

Using the pumping lemma, we write w = xyz with $|xy| \leq n$ and $|y| > 0$.

As $w = 0^n1^{2n}2^{2n}$, $xy = 0^i$ for some $i \leq n$.

This means that $y = 0m$ for some m, $i \leq m \leq n$.

Then $xy^{k+1} <= 0^{n+mk}1^{2n}2^{3n}$ choosing k large enough,

So that $n + mk > 2n$, we can make

$n + mk + n > 2n$. So $xy^{m+1}z \notin L$.

Hence, L is not regular.

Q.95. Use pumping lemma to prove that

$L = \{a^ib^jc^i \mid i, j \geq 1\}$ is not regular.

OR Using pumping lemma prove that $L = \{a^nb^mc^n \mid m, n \geq 1\}$ is not regular.

CT : S-13(5M)

Ans. We assume that L is regular and try to obtain a contradiction.

Suppose L is a regular.

Let 'n' be constant of pumping lemma and

Let $w = a^ib^jc^i$ and w = xyz.

The string is in L and length is atleast 2n + m.

so $w = xyz \mid xy \mid \leq n, y \neq \in$

Clearly xy will contain only a's since $w = a^nb^n$ and in this first 'n' placed are occupied by the a's only so y will also made of a's only.

According to pumping lemma xy^iz is also in L for i > 0.

Let us choose i = 1 then the resulting string xyz is not in L since it has fewer a's than b's . So it contradicts the pumping lemma i.e. L is not regular.

Q.96. Consider the set L as a regular set :

$L_{set} = \{a_2, a_1, a_4, a_3 \ldots\ldots\ldots\ldots a_{2n}, a_{2n-1}$

$\mid a_1, a_2, \ldots\ldots\ldots\ldots a_2n$ is in L}.

Identify whether the given set is regular or not. Justify your answer.

Ans. We prove it by Induction Hypothesis.

Let $M = (Q_1, \Sigma, \delta, q_0, F)$ be a DFA accepting given L. Construct a DFA $M' = \{Q', \Sigma, \delta', q_0, F'\}$ be a DFA accepting L' M' will process tape symbols in pairs, on reading first symbol a in a pair, M' stores a in its finite control. Then on reading the second symbol on tape M^1 behaves like M on the input.

To prove that M' accepts L', we show by induction that

$\delta'(q, a_2\ a_1\ a_4\ a_3 \ldots\ldots\ldots a_i\ ai-1)$

$= \delta(q, a_1, a_2 \ldots ai)$

Because $i = 0$, $\delta'(T, \in) = q = \delta(q, \in)$

Assume the hypothesis is true for all i,

By Induction hypothesis.

$\delta'(q_1, a_2, a_1 \ldots\ldots\ldots ai-2,\ ai-3)$

$= \delta(T_1, a_{11}, a_2 \ldots\ldots\ldots ai-2)$

$= p$ for some q

Thus,

$\delta'(q, a_2\ a_1 \ldots\ldots\ldots a_i, ai-3)$

$= \delta'(P, a_i, ai-1)$

$= \delta'([P, ai], ai-1)$

$= \delta(P, ai-1, ai)$

$= \delta(q, a_1\ a_2 \ldots\ldots\ldots ai)$

Therefore string $a_2\ a_1\ a_4\ a_3 \ldots\ldots\ldots ai\ ai-1$ is in L(M') if and only if $a_1\ a_2 \ldots\ldots ai$ is in L(M) and Thus L(M') = L(M)

Hence, the language L is regular.

Q.97. Prove that using pumping lemma, the language L is not context free language,

$L = \{a^ib^ic^i / i \geq 1\}$ **CT : S-06(7M)**

Ans. $L = \{a^ib^ic^i / i \geq 1\}$

$L = \{abc, aabbcc, aaabbbccc, \ldots\ldots\}$

Let $w = uv^ixy^iz$ and w = aabbcc

$u \rightarrow \in$ and $y \rightarrow \in$

$w = v^i x z^i$

$= (aa)^i bbcc$

For i = 1,

w = aabbcc is in L

For i = 2,

$W = (aa)^2$ bbcc

= aaaabbcc which is not in L

Therefore $L = \{a^i b^i c^j \mid i \geq 1\}$ is not CFL.

CLOSURE AND DECISION PROPERTIES FOR REGULAR SETS

Q.98. Explain the closure properties of regular sets.

CS : S-11(2M)

CT : S-08(5M), S-09, W-10,11(3M)

Ans. Following are the properties of regular languages /set.

(1) The union of two regular languages / set is regular.
(2) The intersection of two regular language is regular.
(3) The complement of a regular language is regular.
(4) The difference of two regular languages is regular.
(5) The reversal of a regular languages is regular.
(6) The closure (star) of a regular languages is regular.
(7) The concatenation of regular language is regular.
(8) A homomorphism of a regular language is regular.
(9) The inverse homomorphism of a regular language is regular.

Q.99. Explain decision properties of regular languages.

Ans. Decision Properties of Regular Languages :

- Decision problems of regular languages are computational problems.
- Regular languages are the languages of the finite automations.
- So, finite automaton provides computational procedure to solve the decisions problems.

Following are the decision problems :

(a) Emptiness problem
(b) Finiteness problem
(c) Membership problem
(d) Equivalence and Minimization problem

(a) Testing Emptiness of Regular Languages :

- It L is regular language then to know L is empty or not we have to decide ϕ is empty, and all other regular languages are not.
- If our representation is any kind of finite automation, the emptiness question is whether there is any path whatsoever from the start state to some accepting state, if so, language is nonempty, while if the accepting states are all separated from the start state, then the language is empty.
- The start state is clearly reachable from the start state, if there is an edge from q to p with any label (an input symbol, or ϵ if the automation is an ϵ NFA) then p is reachable. In that manner we can compute the set of reachable states.
- If a regular expression is given representing the language L, rather than an automation, we could convert the expression to an ϵ-NFA and proceed as above.
- Since the automation that results from a regular expression of length n has at most O(n) states and transitions, the algorithm takes O(n) time.

(b) Testing Finiteness in a Regular Languages

- Let L is regular language accepted by some finite automaton M. If M be a n states DFA then language L(M) is infinite if and only if M accepts a string x such than $n \leq |x| < 2n$.

Theorem : Whether a regular set is finite is solvable.

Proof :

- If a finite automation accepts any strings at all then some will be of length less than the size of the state set.
- If we were to find a string accepted by finite automation M which was longer than or equal to the size of it's state set, we could use the inflation aspect of the pumping lemma to show that machine m must accept on infinite number of strings. This means that :

"A finite automation accepts only strings of length less than the size of its set of states, if and only if it accepts a finite set".

- Hence, to solve the finiteness problem for $M = (Q, \Sigma, \delta, q_0, F)$, we need to determine whether or not :

L (M) – | strings of length < | Q | | = ϕ

- But how many inputs we must examine in order to tell whether M will accept on input longer than or equal to the size of its state set.
- The answer is that we only need to consider input strings upto twice the size of the state set.

(c) Testing Membership in a Regular Language

- Another important question is, given a string w and a regular language L, is w in L. While w is represented explicitly, L is represented by an automation or regular expression.
- The algorithm is simple if L is represented by a DFA. Process the DFA processing the string of input symbol w, beginning in the start state.
- If the DFA ends in an accepting state, the answer is "yes"; otherwise the answer is "no".
- If | w | = n, and the DFA is represented by a suitable data structure, such as two-dimensional array that is the transition table, then each transition requires constant time, and entire test takes O(n) time.

(d) Equivalence Problem :

- Given two sets of regular languages, then the question arises; whether these sets are equivalent or they define same set of regular languages? Or, Given two finite automatons M_1 and M_2 they accept the same languages. i.e.

  $L_1 = L_2$
- If difference of two regular languages is empty then both are equivalent.
- Another consequence of equivalence problem is, two regular languages are equivalent if and only if they are the languages of some common DFA.

CONTEXT FREE LANGUAGES

Q.100. Define context free grammar.

Ans. Context free grammar :

- A context free grammar is a finite set of variables (also called nonterminals or syntactic categories) each of which represents a language.
- The languages represented by the variables are described recursively in terms of each other and primitive symbols called terminals. The rules relating the variables are called productions.
- A typical production states that the language associated with a given variable contains strings that are formed by concatenating strings from the languages of certain strings that are formed along with some terminals.
- Therefore, CFG is a four tuple denoted as :

  G = (V, T, P, S), where

(1) V is a finite set of symbol called as non terminals or variables

(2) T is a set of symbols called as terminals

(3) P is a set of productions

(4) S is a member of V, called as start symbol.

Example :

G = ({S}, {a, b}, P, S) where P contains :

P = {S $\rightarrow$ aSa,

S $\rightarrow$ bSb,

S $\rightarrow \in$

}

Q.101. Justify the following Statements.

(i) Union of two Regular language is a Regular language.

(ii) Intersection of Regular language and CFL is a CFL. ***CT : W-09(6M)***

Ans.

(i) If L_1 and L_2 are regular then there exist regular expressions r_1 and r_2 such that $L = L(r_1)$ and $L = L(r_2)$ (By definition).

$r_1 + r_2$ is a regular expression denoting the languages $L_1 \cup L_2$. Thus union of two regular languages is a regular languages.

(ii) Let $M_1 = (Q, \Sigma, \delta_1, q_0, z, F_1)$ be an NPDA that accepts L_1 that is a CFL.

$M_2 = (P, \Sigma, \delta_2, p_0, z, F_2)$ be a DFA that accepts L_2 that is a regular languages.

We construct a push down automation $\hat{M} = \{\hat{Q}, \Sigma, \Gamma, \hat{\delta}, \hat{q_0}, z, \hat{F}\}$ which simulates the

parallel action of M_1 and M_2: whenever a symbol is read from the i/p string, M simultaneously executes the moves of M_1 and M_2.

Let $\hat{Q} = Q \times P$

$\hat{q}_0 = (q_0, p)$

$\hat{F} = F_1 \times F_2$

and define $\hat{\delta}$ such that

$((q_k, p_l), x) \in \hat{\delta}$ $((q_i, q_j), a, b)$

if and only if $((q_k, x), \in S_1 (q_i, q_j), a, b)$

and $\delta_2 (p_j, a) = p_l$

In this, we also require that if $a = \in$ the $p_j = p_i$.

In other word, the states of $\hat{M}$ are labeled with pairs (p_i, p_j), representing the respective state in which M_1 and M_2 can be after reading a certain input string. It is a string forward induction argument to show that $((q_0, p_0), w, \beta) \vdash_{\hat{M}}^{*} ((q_r, p_s), x)$ with $q_r \in F_1$ and $Ps \in F_2$ if and only if,

$(q_0, w, \beta) \vdash M_1 (q_r, x)$, and

$S^* (p_0, w) = ps$

Therefore, a string is accepted by $\hat{M}$ if and only if it is accepted by M_1 and M_2 that is, if it is in $L(M_1) \cap L(M_2) = L_1 \cap L_2$.

Q.102. Construct CFG for the language :

$L = \{0^n 1^{2n} \mid n \geq 1\}$ ***CT : W-09(3M)***

Ans. If n = 1 then input string = 011

If n = 2 then input string = 001111

From above we can conclude that the only strings are accepted in which count of 1's are twice than count of 0's.

$\therefore S \rightarrow ABB$

$A \rightarrow AABB \mid 0$

$B \rightarrow ABBB \mid 1$

Hence, above grammar accepts all strings generated by language L.

Q.103. Prove the following :

Define an A-production to be a production with variable A on the left. Let G = (V, T, P, S) be a CFG.

Let $A \rightarrow \alpha_1 B\alpha_2$ be a production in P and $B \rightarrow \beta_1 \mid \beta_2 \mid \ldots \mid \beta_r \mid$ be the set of all B-productions.

Let G_1 = (V, T, P, S) be obtained from G by deleting the production $A \rightarrow \alpha_1 B\alpha_2$ from P and adding the productions $A \rightarrow \alpha_1\beta_1\alpha_2 \mid \alpha_1\beta_2\alpha_2 \mid \ldots \mid \alpha_1\beta_r\alpha_2$ then L(G) = L (G_1).

Ans.

- Obviously $L(G_1) \subseteq L(G)$, since if $A \rightarrow \alpha_1\beta_i\alpha_2$, is used in a derivation of G_1, then $A \rightarrow \alpha_1 B\alpha_2 \rightarrow \alpha_1 \beta_i\alpha_2$ can be used in G.
- To show that $L(G) \subseteq L(G_1)$, one simply notes that $A \rightarrow \alpha_1 B\alpha_2$ is the only production in G not in G_1 whenever $A \rightarrow \alpha_1 B\alpha_2$ is used in a derivation by G, the variable B must be rewritten at some later step using a production of the form $B \rightarrow \beta_i$ These two steps can be replaced by the single step $A \rightarrow \alpha_1\beta_i\alpha_2$,

Q.104. Prove the following :

Let G = (V, T, P, S) be a CFG. Let $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \ldots A\alpha_r$ be the set of A productions for which A is the leftmost symbol of the right-hand side. Let $A \rightarrow \beta_1 \mid \beta_2 \mid \ldots B_s$ be the remaining A-productions. Let $G_1 = (V \cup \{B\}, T, P_1, S)$ be the CFG formed by adding the variable B to V and replacing all the A-productions by the productions.

1) $A \rightarrow \beta_i$

   $A \rightarrow \beta_i B \quad 1 \leq i \leq S$

2) $B \rightarrow \alpha_i$

   $B \rightarrow \alpha_i B \quad 1 \leq i \leq r$

Then $L(G_1) = L(G)$

Ans. In a leftmost derivation, a sequence of productions of the form $A \rightarrow A\alpha_i$ must eventually end with a production $A \rightarrow \beta_j$

The sequence of replacements.

$A \to A\,\alpha_{i1} \to A\,\alpha_{i2}\alpha_{i1} \to \ldots$

$\to A\,\alpha_{ip}\alpha_{ip-1}\ldots\alpha_{i1}$

$\to \beta_j\,\alpha_{ip}\alpha_{ip-1}\ldots\alpha_{i1}$

in G can be replaced in G_1 by

$A \to \beta_j\,B \to \beta_j\,\alpha_{ip}B \to \beta_j\alpha_{ip}\alpha_{ip-1}\,B$

$\to \ldots\ldots \to \beta_j\,\alpha_{ip}\alpha_{ip-1}\ldots\alpha_{i2}B$

$\to \beta_j\,\alpha_{ip}\alpha_{ip-1}\ldots\alpha_{i1}$

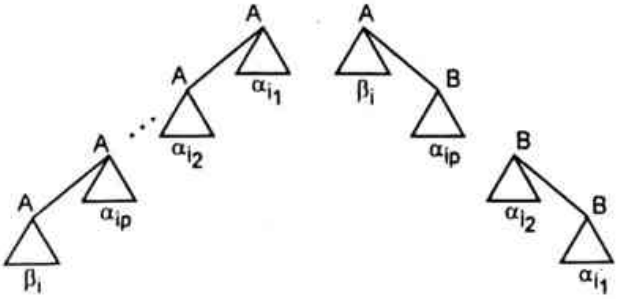


Fig. Transformation on portion of a derivation tree

Q.105. Prove that the following is not CFL

$\{a^n b^n d^i \mid i \# n\}$

Ans. L = {aabbd, abdd, aaabbbdd, aabbddd……..}

Let w = aabbd and

$w = uv^i xy^i z$

Let $y = \epsilon$ and $z = \epsilon$

$\therefore\ w = uv^i x$

$= (aa)\,(bb)^i d$

For i = 1,

w = aa(bb)d which is in L

For i = 2,

w = aa(bbbb)d which is not in L

Hence

$L = \{a^n b^n d^i \mid i \neq n\}$ is not CFL.

Q.106. Prove that the following language is not a CFL.

$L = \{a^i b^j c^k \mid i < j < k\}$

Ans. $L = \{a^i b^j c^k / i < j < k\}$

L = {abbccc, aabbbcccc, aaabbbbccccc……. }

Let $w = uv^i xy^i z$ and w = abbccc

Again $u \to \epsilon$, $Z \to \epsilon$

$w = v^i xy^i$

$w = (a)^i\,(bb)\,(ccc)^i$

For i = 1,

w = abbccc is in L

For i = 2,

w = aabbccc is not in L

Therefore $L = \{a^i b^j c^k / i < j < k\}$ is not context tree language.

Q.107. Generate context free grammar for the language $L = \{a^m b^n \mid m, n > 0 \text{ and } m > n\}$

CT : W-06(5M)

Ans.

- Writing L as a concatenation $L_1 L_2$, where these three languages contain strings of a's, strings of b's respectively - is doomed to failure.
- The language L contains both a^2b^1 and a^3b^2, however, if we allowed L_1 to contain a^2, L_2 to contain b^1, then L_1L_2 would also contain a^2b^1 and this string is not an element of L.
- If we look for another way to express L as a concatenation, we might note that $a^n b^n$.
- The only difference between this and a string x in L is that x has at least one extra a in the front :

  $x = a^m a^n b^n$ (for given $m > 0$)

  A correct formula L is therefore $L = L_1 L_2$,

  where $L_1 = \{a^m \mid m \geq 0\}$,

  $L_2 = \{a^n\, b^n \mid n > 0\}$,
- A recursive definition for L_1 is in the following :

  $A \to aA \mid a$

  $\wedge \in L_2$ for any $x \in L_1$, $axb \in L_1$ nothing else is in L_1 and a corresponding CFG is one with the productions.

  $B \to aBb \mid \wedge$
- The final CFG G = {V, Σ, S, P) incorporating these pieces is shown below.

$V = \{S, A, B, C\}$, $\Sigma = \{a, b\}$

$P = \{S \rightarrow AB)$

$A \rightarrow aA / a$

$B \rightarrow aBb / \wedge\}$

A derivation of $a^2.b$ (a) (ab)

$S \rightarrow AB \rightarrow Aa\ Bb \rightarrow Aa \wedge b \rightarrow aab$

Q.108. Find CFG for each of the languages defined by the following

(i) ab* (ii) a*b* (iii) (baa+abb)*

(iv) The language over $\Sigma = \{a, b\}$ containing words that have different first and last letter.

Ans.

(i) ab*

The productions are

$S \rightarrow AB \mid a$

$A \rightarrow a$

$B \rightarrow CB \mid b \mid \in$

$C \rightarrow b$

(ii) a*b*

The productions for a* are

$A \rightarrow BA \mid \in$

$B \rightarrow a$

The productions for b* are

$C \rightarrow DC \mid \in$

$D \rightarrow b$

Now combining both the productions we get

$S \rightarrow AC \mid \in$

$A \rightarrow BA \mid \in$

$B \rightarrow a$

$C \rightarrow DC \mid \in$

$D \rightarrow b$

(iii) (baa + abb)*

$S \rightarrow AS \mid BS \mid \in$

$A \rightarrow baa$

$B \rightarrow abb$

(iv) The language over $\Sigma = \{a, b\}$ containing words that have different first and last letter.

We start with production D. The only requirement for string dorivable from D is that its first and last symbols be different. The substring between the first and last symbols is {a, b}*

$D \rightarrow aAb \mid bAa \mid$

Where a stands for any combination of a and b. Thus production for aA can be written as

$A \rightarrow aA \mid bA \mid \in$

∴ We may describe CFG for the given language as

$S \rightarrow D$

$D \rightarrow aAb \mid bAa \mid$

$A \rightarrow aA \mid bA \mid \in$

Q.109. Give context free grammar generating the set of palindromes strings that reads the same forward as backward, over alphabets {a, b}.

Ans. A palindrome can be defined as a string that reads the same forward and backward or by the following definition.

1) $\in$ is palindrome
2) If a is any symbol, then string a is palindrome.
3) If a is any symbol and x is palindrome, then axa is a palindrome.
4) Nothing is a palindrome unless it follows from (1) through (3).

We may define palindromes over {a, b} recursively as follows.

1) $\in$, a, b are palindromes
2) If w is a palindrome, so awa and bwb are palindromes.
3) Nothing else is palindromes.

A CFG for palindromes now follows immediately from (1) and (2). It is.

$S \rightarrow a \mid b \mid \in$ (From 1)

$S \rightarrow aSa \mid bSb \mid$ (From 2)

Q.110. Construct CFG for the language

(i) $L = \{a^n b^n c^m d^m \mid n \geq 1, m \geq 1\}$

(ii) $L = \{a^n b^m c^m d^n \mid n \geq 1, m \geq 1\}$

(iii) $L = \{a^i b^n c^n \mid i \geq 0, n \geq 1\}$

(iv) $L = \{a^n b^n c^i \mid n \geq 1, i \geq 0\}$

CS : S-10(6M), CT : W-09(3M), W-10(4M)

Ans.

(i) $L = \{a^n b^n c^m d^m \mid n \geq 1, m \geq 1\}$

It is generated by the grammar.

$S \to AB$

$A \to aAb \mid ab$

$B \to cBd \mid cd$

(ii) $L = \{a^n b^m c^m d^n \mid n \geq 1, m \geq 1\}$

It is generated by CFG

$S \to aSd \mid aAd$

$A \to bAc \mid bc$

(iii) $L = \{a^i b^n c^n \mid i \geq 0, n \geq 1\}$

$S \to AB$

$A \to aA \mid \in$

$B \to bBc \mid bc \mid \in$

(iv) $L = \{a^n b^n c^i \mid n \geq 1, i \geq 0\}$

$S \to AB$

$A \to aAb \mid ab \mid \in$

$B \to CB \mid \in$

Q.111. Find the CFL generated by the following grammar G

$G = (\{S\}, \{a, b, \in\}, P, S)$

Where P consist of

$S \to aSa \mid bSb \mid a \mid b \mid \in$

Ans. In given grammar G = (V, T, P, S) where V = {S}.

$T = \{a, b, \in\}$, $P = \{S \to aSa \mid bSb \mid a \mid b \mid \in\}$.

Here S is only variable, a, b, $\in$ are terminals.

There are six production. By applying the production we have

$S \to aSa \to a\,b\,S\,b\,a \to a\,b\,S\,b\,a \to w\,S\,w^R$

where $w = ab$

$w^R = ba$

Therefore the given grammar recognize the language of all even and odd length palindromes over {a, b}

$L = \text{Palindromes} = \{ww^R \mid w \in (a, b)^*\}$

Q.112. Generate CFG for the language

$L = \{a^m b^n \mid m \leq n \leq 2m\}$

Ans. Consider the language

$L = \{a^m b^n \mid m \leq n \leq 2m\}$

$L = L_1 \cup L_2$

where

$L_1 = \{a^m b^n \mid m \leq n\}$

$L_2 = \{a^m b^n \mid n \leq 2m\}$

We construct CFG for L_1 first. The set of production for language L_1 are as follows.

$S \to A \mid B$

$A \to aAb \mid \in$

$B \to a\,B\,P \mid b$

$P \to b$

Now we construct CFG for language L_2. The set of production for language L_2 are as follows.

$S \to C$

$C \to D\,E\,E$

$D \to F\,D\,E\,E \mid a$

$F \to a$

$E \to b$

Now, combining the set of production for language L_1 and L_2 we get the CFG for the given language L. The set of production for language L are as follows.

$S \to A \mid B \mid C$ $\quad C \to D\,E\,E$

$A \to aAb \mid \in$ $\quad D \to F\,D\,E\,E \mid a$

$B \to a\,B\,P \mid b$ $\quad F \to a$

$P \to b$ $\quad E \to b$

Q.113. Find grammar generating

$L = \{a^n b^n c^i \mid n \geq 1, i > 0\}$

Ans. L = {abc, aabbc, aabbcc, ...}

Hence CFG for L is

$S \to AB$

$A \to aAb \mid ab \mid \in$

$B \to cB \mid \in$

Q.114. Give CFG to accept the language

$L = \{a^m b^n c^{m+n} \mid m, n \geq 1\}$

OR $L = \{0^m 1^n 0^{m+n} \mid m, n \geq 1\}$ **CT : S-11(3M)**

Ans. The production of grammar are :

$S \rightarrow 0A0$

$A \rightarrow 0A0 / 1B0$

$B \rightarrow 1B0 / 10$

$G = \{(S, A, B), (0, 1), P, S\}$

Q.115. Design CFG for $L = \{a^n b^{n+1} \mid n >= 1\}$.

CS : W-09(5M)

Ans. $L = \{abb, aabbb, aaabbbb \ldots\ldots\}$

$\therefore S \rightarrow ABb$

$A \rightarrow bB \mid a$

$B \rightarrow aA \mid a \mid b \mid \in$

Q.116. Write context tree grammar for following language :

(i) $L = \{a^n b^n c^k \mid k \geq 3, n \geq 1\}$

(ii) $L = \{0^a 1^b 2^c \mid a - b \mid = c ; a, b, c > 0\}$

CS : S-12(6M)

OR Design CFG for following languages :

(i) $L = \{a^m b a^m : m \geq 0\}$

(ii) $L = \{a^n b^n c^k \mid n \geq 0, k \geq 3\}$

(iii) $L = \{a^n b^m c^k : k = \mid n - m \mid\}$

CT : S-13(6M)

Ans. $L = \{0100, 001000, \ldots..\}$

(1) $S \rightarrow AB$

$A \rightarrow 0A / 01 / \in$

$B \rightarrow 0B / 0$

(2) $L = \{abccc, aabbccc, aabbcccc \ldots\ldots\}$

$\therefore S \rightarrow AB$

$A \rightarrow aA / ab / \in$

$B \rightarrow bB / cB / c$

(3) $L = \{0112, 0012, 000112, \ldots\ldots\}$

$\therefore S \rightarrow AB$

$A \rightarrow 0A / \in$

$B \rightarrow 1B / 2 / \in$

Q.117. Construct grammar for language $L = \{a^n b^n c^n d^n \mid n \geq 0\}$ and then determine the type of this language. **CS : W-12(5M)**

Ans. $L = \{\in, abcd, aabbccdd, \ldots..\}$

$S \rightarrow AB \mid \in$

$A \rightarrow aAb \mid ab$

$B \rightarrow cBd \mid cd$

The language of above grammar is context free language.

Q.118. Write CFG to specify all strings are {a, b} that are palindromes.

OR $L = \{WCW^R \mid W \in \{a, b\}^*\}$ **CT : S-11(3M)**

Ans. Productions of grammar are

$S \rightarrow aSa \mid bSb \mid a \mid b \mid \in$

Because of palindrome is a string whose first and the last symbol is same, second and the last but one symbol is same, and so on.

The CFG is

$G = \{(S), (a,b), P,S\}$

PARSE TREE AND AMBIGUITY

Q.119. Define Derivation trees or Parse tree.

CS : S-11(2M)

OR Explain parse tree with example.

Ans. Parse tree or Derivation tree :

- The derivations in a CFG can be represented using trees. Such trees representing derivations are called derivation trees.
- While deriving a string w from S, if every derivation is considered to be a step in the tree construction, then we get a graphical display of derivation of a string w as a tree, called as derivation tree or parse tree of a string w.
- Therefore, a derivation tree or parse tree is display of derivations as a tree.
- A tree is a derivation tree, provided it satisfies the following requirement.

(a) All the leaf nodes of the tree are labelled by terminals of the grammar.

(b) The root node of the tree is labelled by the start symbol of the grammar.

(c) The interior node are labelled by the non-terminals.

(d) If an interior node has a label A, and has n descendents with labels $X_1, X_2, \ldots\ldots X_n$ from left to right, then there must exist production rule $A \rightarrow X_1, X_2, \ldots\ldots X_n$ in the grammar.

(c) A vertex n is a leaf if its label is $a \in \Sigma$ or $\in$; n is the only son of its father if its label is $\in$.

Example: for a grammar whose list of productions is

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$

The tree shown in figure is a derivation tree for a string id + id* id.

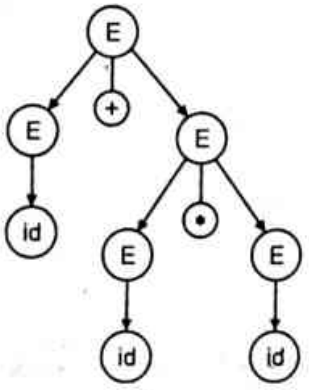


Fig.

Q.120. What is ambiguous grammar and unambiguous grammar ? Explain with suitable examples.

CS : S-11, W-12(2M)

Ans. Ambiguous grammar : A grammar G is said to be ambiguous if there exists at least one string. In L(G) having more than one parse trees or derivation trees generating the string.

Unambiguous grammar : A grammar G is said to be unambiguous if there exists any string in L (G) having only one parse tree or derivation tree generating the string.

Example : $S \rightarrow a \mid ab\,Sb \mid a\,A\,b$

$A \rightarrow bS \mid a\,AA\,b$

Consider input string abab Sbb

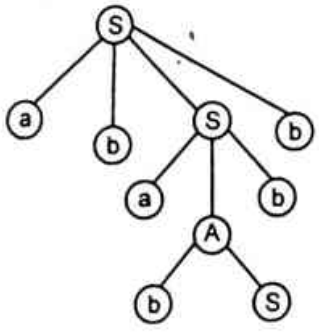


Fig. Parse tree

Since there exist only one parse tree hence given grammar is unambiguous.

Q.121. Find whether the following grammar are ambiguous or not.

(i) $S \rightarrow a \mid ab\,Sb \mid aAb$

$A \rightarrow bS \mid a\,AA\,b$ **CT : S-11(4M)**

(ii) $S \rightarrow ab \mid ab$

$A \rightarrow aAB \mid a$

$B \rightarrow ABb \mid b$

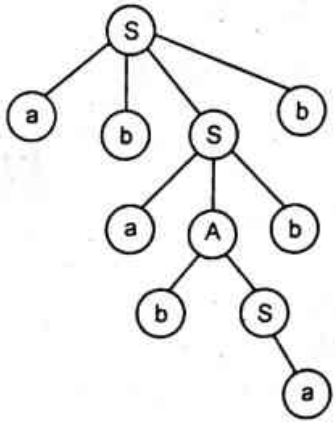
(iii) $S \rightarrow icScS \mid icS \mid a$ **CT : S-13(3M)**

Ans.

(i) $S \rightarrow a \mid ab\,Sb \mid aAb$

$A \rightarrow bS \mid a\,AA\,b$

- Since there is no such string for which there exists a more than one parse tree.
- Hence the given grammar is not ambiguous.
- Consider input string. aba ba bb.



(ii) $S \rightarrow ab \mid ab$

$A \rightarrow aAB \mid a$

$B \rightarrow ABb \mid b$

- Since according to reachable condition, from start state we can't reach to state A or B.
- So there is only one string i.e. ab can be derived from above grammar.
- Hence there exists only one parse tree. Therefore above grammar is not ambiguous.

(iii) $S \rightarrow icScS \mid icS \mid a$

Consider input using icicacica

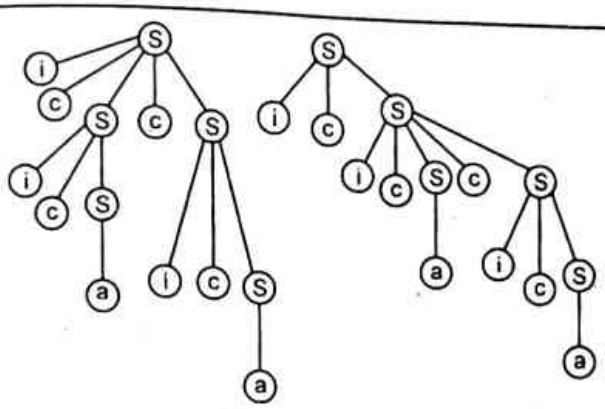


Fig. Parse trees

We have two parse tree for same input.

∴ Given grammar is ambiguous.

Q.122. Check whether the following grammars are ambiguous or not.

(i) $S \rightarrow aSb \mid SS \mid \in$

(ii) $S \rightarrow iCtS \mid iCt\,SeS \mid a$

$C \rightarrow b$. ***CS : S-13(2M)***

Ans.

(i) $S \rightarrow aSb \mid SS \mid \in$:

This grammar is an ambiguous grammar because there exists more than one parse trees for string ab in L(G) of this grammar, as shown in fig.below.

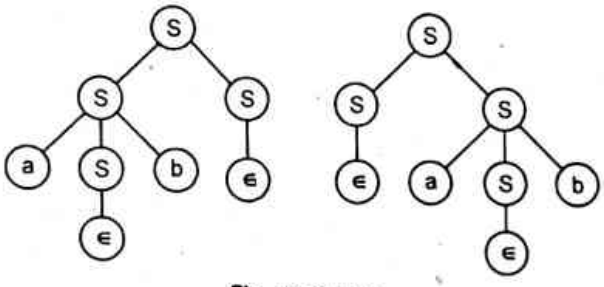


Fig. parse trees

(ii) $S \rightarrow iC\,t\,S \mid iC\,t\,Se\,S \mid a$:

$C \rightarrow b$

Consider an input string ib t ib t a e a.

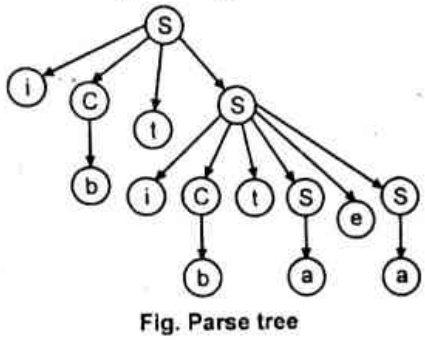


Fig. Parse tree

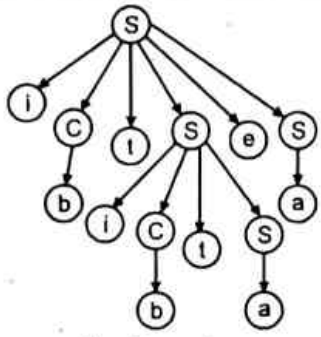


Fig. Parse tree

Since there exists two parse tree generating the same string ib t ib t a e a. Hence the given grammar is an ambiguous grammar.

Q.123. Show that grammar

$S \rightarrow aB \mid ab$

$A \rightarrow aAB \mid a$

$B \rightarrow A\,Bb \mid b$ is ambiguous. ***CT : W-07(3M)***

Ans. Consider the input string a a a b b b

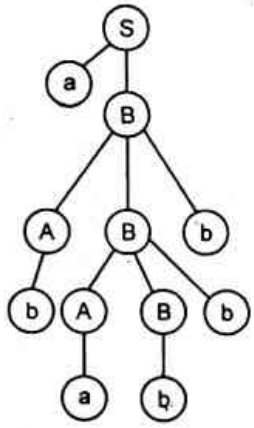


Fig. Parse tree

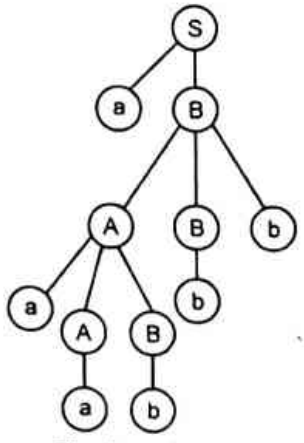


Fig. Parse tree

Since we derive string aaabbb by two ways. Hence there exists two parse tree for string aaabbb. Therefore given grammar is ambiguous.

Q.124. Show that CFG with productions
$S \rightarrow a \mid Sa \mid bSS \mid SSb \mid SbS$ is ambiguous.

Ans. Consider an input string baa baa

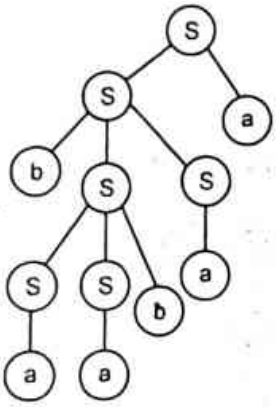


Fig. Parse tree

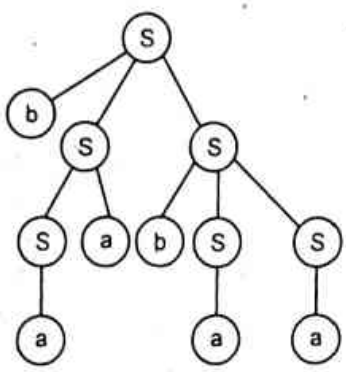


Fig. Parse tree

Since for an input string baabaa, there exists two parse tree generating the string. Hence the given grammar is ambiguous .

Q.125. Check whether the following grammar is ambiguous or not. If found ambiguous remove the ambiguity and write an equivalent unambiguous grammar :
$E \rightarrow E + E \mid E * E \mid (E) \mid id$ ***CT : S-12(7M)***

Ans. Consider an input string id + id * (id)

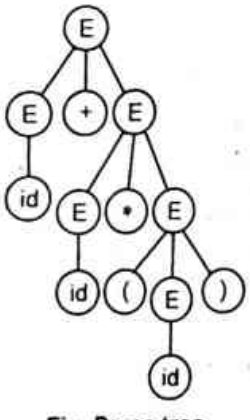


Fig. Parse tree

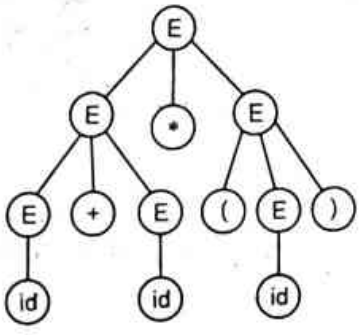


Fig. Parse tree

Since for an input string id + id * (id) there exists two parse tree as shown in fig. Hence the given grammar is ambiguous.

Removal of ambiguity :

- The production $E \rightarrow E + E$ by itself is enough to produce ambiguity and we will need to eliminate this type of production.
- Instead of $E \rightarrow E + E$ we write $E \rightarrow E + T$. Since we have production $E \rightarrow E * E$ in given grammar. So we write $T \rightarrow T * F$ instead of it. So the unambiguous grammar is given by -

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Q.126. Show that the grammar
$E \rightarrow E + E$
$E \rightarrow E * E$
$E \rightarrow id$ is ambiguous.
Also remove ambiguity from the given grammar.

Ans. Consider an input string id + id * id

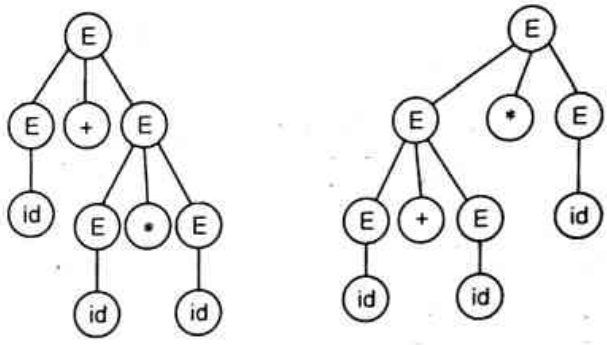


Fig. Parse tree

Since for an string id + id * id, there exists two parse trees generating the string. Hence the given grammar is ambiguous.

The unambiguous grammar after removing ambiguity from given grammar is :

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

Q.127. Show whether the grammar is ambiguous or not.

(i) $S \rightarrow aSSb \mid bSSa \mid \in$

(ii) $S \rightarrow aSbS \mid bSaS \mid \in$

(iii) $S \rightarrow aSbS \mid aS \mid \in$

CS : W-12(2M), S-12(3M), CT : W-12(5M)

Ans.

(i) $S \rightarrow aSSb \mid bSSa \mid \in$

Consider input string abab.

The parse tree are

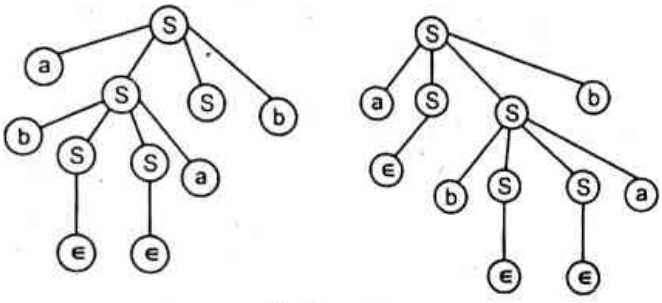


Fig. Parse tree

We derive string in two ways (both are leftmost)

∴ The grammar is ambiguous

(ii) $S \rightarrow aSbS \mid bSaS \mid \in$

Let the input string be abab

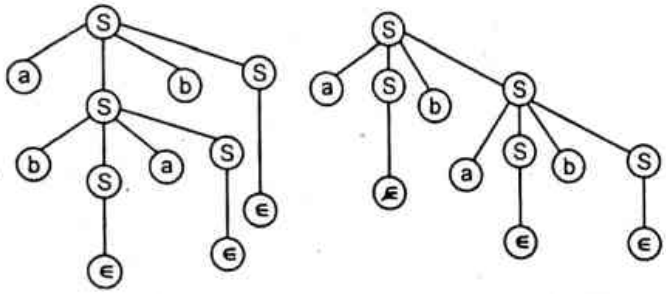


Fig. Parse tree

Since we drive the string abab by two ways. The grammar is ambiguous.

(iii) $S \rightarrow aSbS \mid aS \mid \in$

Let the input string be aacbc

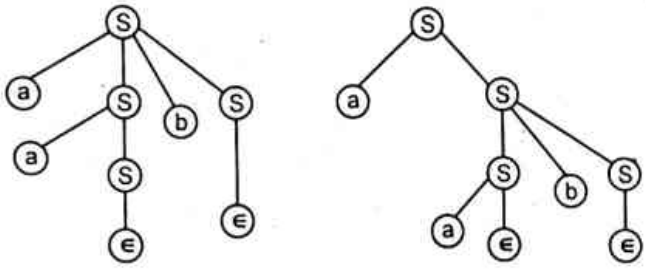


Fig. Parse tree

Since, we drive the string aab by two ways.

The grammar is ambiguous.

Q.128. Show that the grammar.

$S \rightarrow 0 / 01S1/0A1$ **CT : S-07(4M)**

$A \rightarrow 1S/0AA$ **is ambiguous.**

Ans. Left most Derivation :

$S \rightarrow 01S1$

$\rightarrow 010A11$

$\rightarrow 0101S11$

$\rightarrow 0101011$

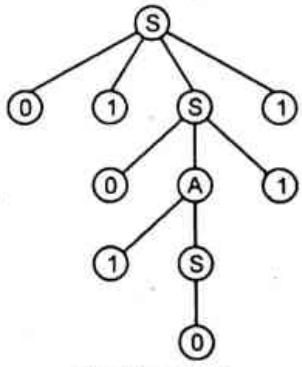


Fig. Parse tree

Also,

By left most derivation :

$S \rightarrow 0A1$

$\rightarrow 01S1$

$\rightarrow 0101S11$

$\rightarrow 0101011$

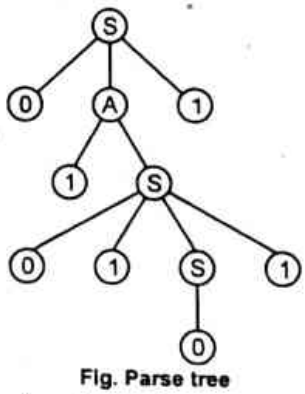


Fig. Parse tree

Hence, the string 01011 has more than one parse tree. Hence, grammar is ambiguous.

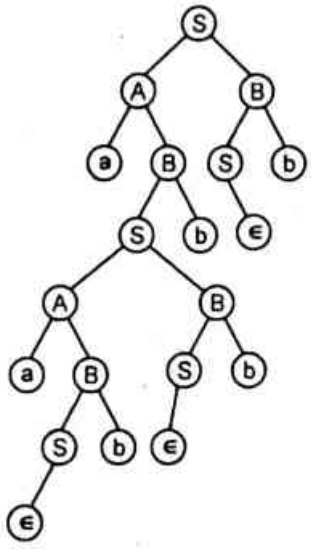
Q.129. Derive the string "aabbbb" using grammar :

$S \rightarrow AB \mid \in$

$A \rightarrow aB$

$B \rightarrow Sb$ **Draw parse tree using left most and right most derivation.** **CT : S-13(4M)**

Ans. Left most derivation :



Now,

Right most derivation :

"aabbbb" string is not derive using right most derivation.

Q.130. Check validity of string "(((a, a), ^, a))" for context tree grammar.

S → (L) / a / ^

A → L, S/S

Also draw parse tree ***CS : S-13(3M)***

Ans. Left Most Derivation :

S → (L)
⇒ (L, S)
⇒ (S, S)
⇒ ((L), S)
⇒ ((S), S)
⇒ (((L)), S)
⇒ (((L, S)), S)
⇒ (((S, S)), S)
⇒ (((a, S)), S)
⇒ (((a, a)), S)
⇒ (((a, a)), (L))
⇒ (((a, a)), (L, S))
⇒ (((a, a)), (S,S))

Hence, the string (((a, a), ^, a)) can be generated by given production.

Q.131. Explain the orders of derivation. Consider a grammar having the production listed below.

E → E + E

E → E * E

E → id

Construct the leftmost and rightmost derivation for the string id + id * id and also draw the parse tree.

OR What is leftmost and rightmost Derivation ? Explain each with an example.

CT : W-12(6M)

Ans. Two important orders of derivation are

(a) Leftmost order of derivation

(b) Rightmost order of derivation

(a) Leftmost order of derivation :

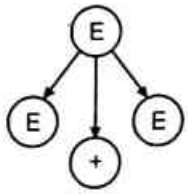
It is that order of derivation in which leftmost non-terminal is considered first for derivation at every stage in derivation process.

Example : one of the leftmost order of derivation of a string id + id * id is

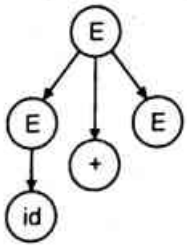
E → E + E
E → id + E
E → id + E * E
E → id + id * E
E → id + id * id

Parse Tree for Leftmost derivation of string id+ id * id.

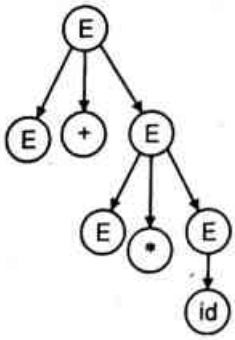
Step 1 : E → E + E



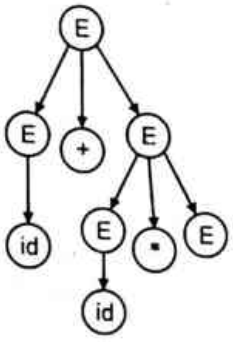
Step 2 : E → E + E → id + E



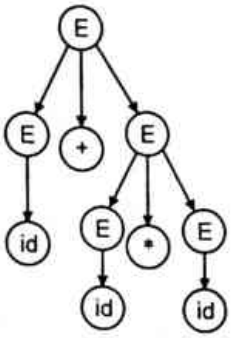
Step 3 : E → E + E → id + E → id + E * E



Step 4 : $E \rightarrow E + E \rightarrow id + E \rightarrow id + E * E \rightarrow id + id * E$



Step 5: $E \rightarrow E + E \rightarrow id + E \rightarrow id + E * E \rightarrow id + id * id.$



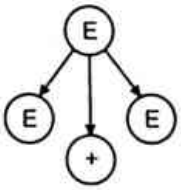
(b) Rightmost order of derivation : The rightmost order of derivation is one in which rightmost non-terminal is considered first.

Example : one of the rightmost order of derivation of id + id* id is

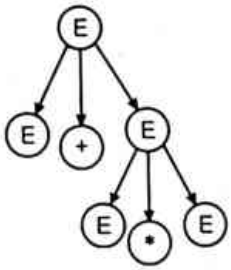
$E \rightarrow E + E \rightarrow E + E * E \rightarrow E + E * id \rightarrow E + id * id \rightarrow id + id * id.$

Parse tree for rightmost derivation of string id + id * id :

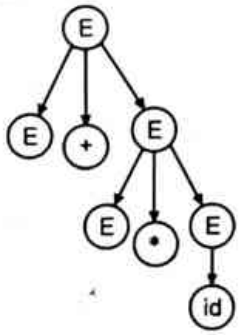
Step 1 : $E \rightarrow E + E$



Step 2 : $E \rightarrow E + E \rightarrow E + E * E$

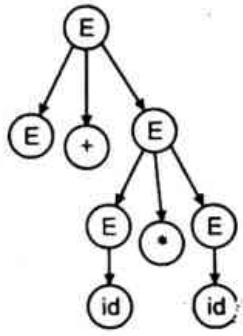


Step 3 : $E \rightarrow E + E \rightarrow E + E * E \rightarrow E + E * id$

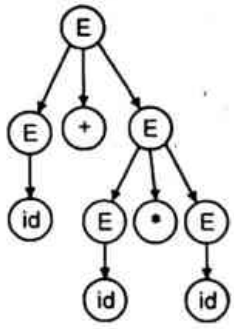


Step 4 :

$E \rightarrow E + E \rightarrow E + E * E \rightarrow E + E * id \rightarrow E + id * id$



Step 5 : $E \rightarrow E + E \rightarrow E + E * E \rightarrow E + E * id \rightarrow E + id * id \rightarrow id + id * id$



The parse tree that gets generated using the leftmost order of derivation of id + id * id, given above, as well the parse tree that gets generated, using the rightmost order of derivation of id + id * id, given above is same, hence these orders are equivalent.

Q.132. Consider the following grammar

$S \rightarrow b\,B$

$B \rightarrow b\,C$

$B \rightarrow a\,B$

$C \rightarrow a$

$B \rightarrow b$

Draw leftmost derivation tree of the above given grammar.

Ans. Consider an input string bab

$\therefore$ Leftmost order of derivation of a string bab is

$S \to b\,B \to ba\,B \to b\,a\,b$

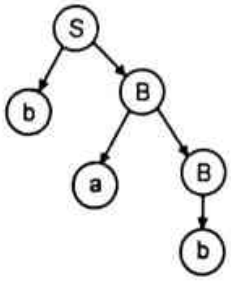


Fig. Leftmost derivation tree

Also consider an input string bba

∴ Leftmost order of derivation of string bba is

$S \to bB \to b\,b\,C \to b\,b\,a$

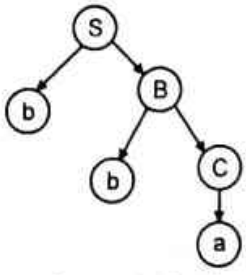


Fig. Leftmost derivation tree

Q.133. Let G be the grammar for string abaabb

$G : S \to Xbaa\,X \mid a\,X$

$X \to Xa \mid Xb \mid \in$

Find (i) Leftmost derivation

(ii) Rightmost derivation.

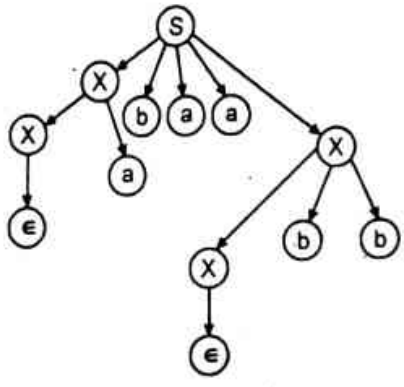
Ans.

(i) Leftmost derivation :

$S \to X\,baa\;X \to Xabaa\,X \to \in abaaX$

$\to \in abaaX \to \in abaaXb \to \in aba\,a\,X\,bb$

$\to \in aba\,a \in bb \to abaabb$

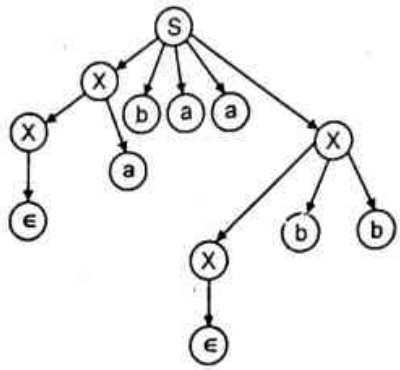


(ii) Rightmost derivation :

$S \to X\,baaX \to X\,baaXb \to X\,baaXbb$

$\to X\,baa \in bb \to Xabaa \in bb$

$\to \in aba\,a \in bb \to abaabb$



Q.134. Consider the grammar with productions

$S \to aAB$,

$A \to bBb$,

$B \to A \mid \in$

Draw leftmost derivation tree of above given grammar.

Ans. Leftmost Derivation :

$S \to a\,AB \to abBbB \to abbBbbB \to abb \in bbB$

$\to abb \in bb \in$

$S \to abbbb.$

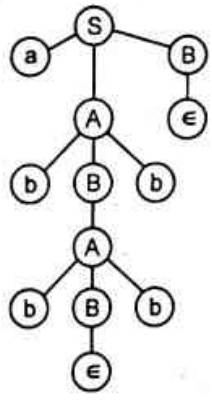


Fig. Leftmost Derivation Tree

Q.135. Show that the following grammar is ambiguous $S \to SbS \mid a$ **CS : W-11(3M)**

Ans. Since there is no such string for which there exists a more than one parse tree.

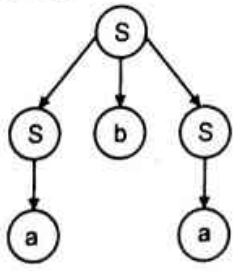


Fig. Parse tree

Hence the given grammar is not ambiguous.

REDUCTION OF CFGS

Q.136. Explain how reduction of context free grammar achieved.

Ans. Reduction of context free grammar :

- Reduction of grammar refers to identification of those grammar symbols (called as useless grammar symbols) and hence those productions, which does not play any roles in the derivation of any w in L(G), and eliminating them from the grammar.
- Reduction of grammar is achieved using following methods .

(i) Eliminate useless symbols, those variables or terminals that do not appear in any derivation of a terminal string from the start symbol.

(ii) Eliminate unit production, those of the form $X \rightarrow Y$ for variables X and Y.

(iii) Eliminate $\in$-production, those of the form $X \rightarrow \in$ for some variable X.

(iv) Eliminate left-recursion, those of the form A $\cdot A \rightarrow A\ \alpha \mid \beta$ for some variable A.

ELIMINATION OF USELESS SYMOBLS

Q.137. State rules for elimination of useless symbols.

Ans. For elimination of useless symbols, there are two rules.

Rule 1 : The grammar that contain only terminal or $\in$ is useful grammar (symbol)

Rule 2 : The grammar that contain combination of useful non-terminal and terminal is also a useful grammar.

Q.138. Reduce the given grammar by eliminating useless symbols.

$S \rightarrow aAa$

$A \rightarrow Sb \mid bCC \mid DaA \mid \in$

$C \rightarrow DD \mid CC \mid abb$

$E \rightarrow aC$

$D \rightarrow aDA$

CS : W-09(5M), W-12(4M)

CT : W-10(4M), W-11(5M)

Ans.

$\therefore$ Let ϕ represents the useful symbols. Initially it is empty

$\phi = \{\ \}$

By Rule 1,

We get A & C as useful symbol

$\therefore \phi = \{A, C\}$

By Rule 2,

We get S & E as useful symbol

$\therefore \phi = \{A, C, S, E\}$

$\therefore$ The grammar becomes

$S \rightarrow aAa$

$A \rightarrow Sb \mid bCC \mid \in$

$C \rightarrow CC \mid abb$

$E \rightarrow aC$

Now, reachable condition :

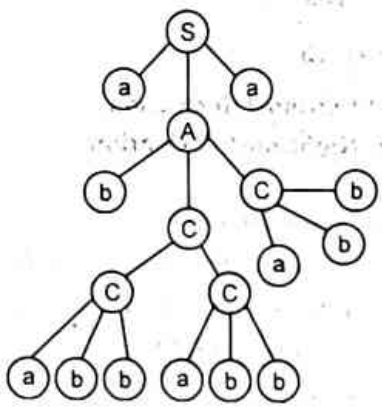
By reachable condition, E also becomes useless.

$\therefore$ Reduced grammar is

$S \rightarrow aAa$

$A \rightarrow Sb \mid bCC \mid \in$

$C \rightarrow CC \mid abb$



Q.139. Find CFG with no useless symbols equivalent to the following grammar.

$S \rightarrow AB \mid CA$

$B \rightarrow BC \mid AB$

$A \rightarrow a$

$C \rightarrow aB \mid b$

CT:W-07(5M),W-09(6M)

Ans.

- Since $A \rightarrow a$ and $C \rightarrow b$ are the production of the form $P \rightarrow$ terminal symbol. Hence they are capable of deriving string.

- There is production B → BC | AB. The right side of these production contain nonterminals A, B, C.
- Even though A and C are known to be capable of deriving to terminal symbol, non terminal B is not capable of deriving any terminal symbol.
- Therefore by eliminating the productions containing B we get,

  S → CA

  A → a

  C → b

Q.140. Find the useless symbol in the following context - free grammar and modify the grammar that has no useless symbol.

S → aB | SB

A → BAc | bSC | a

B → aSB | bBC

C → SBc | aBC | ac

Ans. Let ϕ represents the set of useful symbols.

Initially it is empty

∴ ϕ = { }

By rule 1,

We get A & C as useful grammar,

∴ ϕ = {A, C }

By Rule 2,

There is no non-terminal which is useful

∴ ϕ = {A, C }

It does not contain the start symbol 'S'.

∴ It will not generate any language.

Therefore the whole CFG becomes useless.

Q.141. Eliminate all the useless symbol.

S → aB | bX

A → BAd | bSX | a

B → aSB | bBX

X → SBD | aBX | ad

Ans. Let ϕ be the set of useful symbols. Initially it is empty.

∴ ϕ = { }

From rule 1 & 2, we get ,

∴ ϕ = {S, A, X }

B is not in ϕ

∴ It is useless symbol

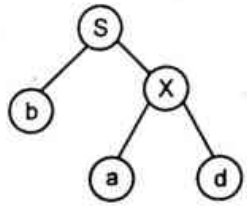
By removing the production of B, the grammar becomes,

S → bX

A → bSX | a

X → ad

By the reachable condition,



'A' is unreachable

∴ It is useless.

Therefore the useful grammar is

S → bX

X → ad

Q.142. Obtain an equivalent grammar containing no useless symbols from the following grammar.

A → xyz | Xyzz

X → Xy | xYx

Y → yYy | XZ

Z → Zy | z

Ans.

- Since A → xyz and Z → z, are the productions of the form A → w, where w is in 1 non-terminals A and Z are capable of deriving to w in T*.
- There are two X-productions X → Xy and X → xYx, the right side of these productions contain non-terminals X and Y respectively.
- Similarly there are two Y-productions Y → yYy and Y → XZ, the right side of these productions contain non-terminals Y and X respectively, hence both X and Y are not capable of deriving to w in T*. Therefore by eliminating the productions containing X and Y, we get :

  A → xyz

  Z → Zy

- Since A is a start symbol, it will always be used in derivation of every w in L (G) and A → xyz being a production in the grammar, the terminals x, y, and z will also be used in the derivation, but since a non-terminal Z occurs nowhere on the right side of the A-production, it will not be used in the derivation of any w in L (G). Hence by eliminating the productions containing non-terminal Z, we get

  A → xyz

  which is a grammar containing no useless grammar symbols.

Q.143. Minimize the following grammar

S → aC | SB

A → b C Ca

B → a S B | b B C

C → a B C | a d

Ans.

- We will minimize the grammar by eliminating useless symbols.
- Since A is not reachable state from any state, hence eliminating production of A we set minimized grammar as follows.

  S → aC | SB

  B → a S B | b BC

  C → a B C | ad
- Also the right side of both the B-productions contain B. Hence the non terminal B is not derivable to any terminal symbol. Therefore B can be eliminated from the grammar.

  ∴ Minimized grammar is

  S → a C

  C → ad

Q.144. Simplify the following grammar

S → AB

A → a

B → C

B → b

C → D

D → E

Check whether the grammar is in reduced form.

Ans.

- In a given grammar, A → a and B → b are the productions which are capable of deriving terminal symbols. Hence they are useful.
- But the productions AB → C, C → D and D → E are not capable of deriving terminal symbols. Therefore they can be eliminate.
- By eliminating them we get following grammar in a reduced form as.

  S → AB

  A → a

  B → b

Q.145. Find the useless symbols in the following grammar and modify the grammar to obtain the grammar that has no useless symbol.

S → 0/ A

A → AB

B → 1 ***CT : S-10(6M)***

Ans.

- In given grammar symbol A derives only non-terminals which are useless.
- Also B is unreachable symbol.

  Hence Symbol A and B are useless symbol.

  ∴ Grammar after removing useless symbols becomes.

  ∴ $\boxed{S \rightarrow 0}$

ELIMINATION OF ∈ - CLOSURE

Q.146. Explain how to eliminate ∈-production from a grammar.

Ans.

- Given a grammar G, containing ∈-productions. If L(G) contains ∈ then elimination of all productions from G gives a grammar G_1 such that

  $L(G_1) = L(G) - \in$
- To eliminate ∈-productions from a grammar we use the following technique.

  If A → ∈ , is an ∈-production to be eliminated, then we look for all those production in the

grammar, whose right side contain A, and replace each occurrence of A in each of these productions to obtain the non ∈-productions, to be added to the grammar to keep the language generated same.

Example :

Consider the following grammar

$S \rightarrow aA$

$A \rightarrow b \mid \in$

- To eliminate $A \rightarrow \in$ form the above grammar, we replace A on the right side of the production $S \rightarrow aA$, to obtain a non ∈-productions $S \rightarrow a$, to be added to the grammar as a substitute, to keep the language generated by the grammar same.
- Therefore the ∈ free grammar equivalent to the given grammar is

$S \rightarrow aA$

$A \rightarrow b$

Q.147. Consider the following grammar.

$S \rightarrow ABAC$

$A \rightarrow aA \mid \in$

$B \rightarrow bB \mid \in$

$C \rightarrow c$

Eliminate all the ∈-production from the grammar, without changing the language generated by the grammar.

Ans.

- To eliminate $A \rightarrow \in$ from the grammar, the non ∈-productions to be added are obtained as follows.

  List of the productions containing A on right side is

  $S \rightarrow ABAC$

  $A \rightarrow aA$
- Replace each occurrence of A in each of these productions to obtain the non ∈-productions, to be added to the grammar, the list of these productions is

  $S \rightarrow BAC \mid ABC \mid BC$

  $A \rightarrow a$
- Add these productions to the grammar and eliminate $A \rightarrow \in$ from the grammar to obtain the following grammar.

  $S \rightarrow ABAC \mid BAC \mid ABC \mid BC$

  $A \rightarrow aA \mid a$

  $B \rightarrow bB \mid \in$

  $C \rightarrow c$
- To eliminate $B \rightarrow \in$ from the grammar, the non ∈-productions, to be added are obtained as follows:

  List of the productions containing B on right is :

  $S \rightarrow ABAC \mid BAC \mid ABC \mid BC$

  $B \rightarrow bB$
- Replace each occurrence of B in each of these productions to obtain the non ∈-productions, to be added to the grammar, the list of these productions are

  $S \rightarrow AAC$

  $S \rightarrow AC$

  $S \rightarrow C$

  $B \rightarrow b$
- Add these productions to the grammar and eliminate $A \rightarrow \in$ from the grammar to obtain the following grammar.

  $S \rightarrow ABAC \mid BAC \mid ABC \mid BC \mid AAC \mid AC \mid C$

  $A \rightarrow aA \mid a$

  $B \rightarrow bB \mid b$

  $C \rightarrow c$

Q.148. Simplify following CFG :

$S \rightarrow aA \mid aBB$

$A \rightarrow aaA \mid \in$

$B \rightarrow bB \mid bbC$

$C \rightarrow B$

Ans. Here non-terminal C is useless since it is not in the right hand side production of S. Hence remove production of C.

Hence, grammar is,

$S \rightarrow aA \mid aBB$

$A \rightarrow aaA \mid \in$

$B \rightarrow bB$

Remove ∈ production

$\therefore S \rightarrow aA \mid aBB \mid a$

$A \rightarrow aaA \mid aa$

$B \rightarrow bB$

Non-terminal B is also useless since, it does not have production containing only terminal symbol. Remove B. Hence grammar is

$\therefore S \rightarrow aA / a$

$A \rightarrow aaA / aa$

Q.149. Eliminate $\in$ - production for following

$S \rightarrow aSB \mid aA \mid bB$

$A \rightarrow aA \mid \in$

$C \rightarrow bB \mid \in$ **CS : S-13(2M)**

Ans. Here, $A \rightarrow \in$

$\therefore S \rightarrow aSB \mid aA \mid a \mid bB$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid \in$

Now, $B \rightarrow \in$

$\therefore S \rightarrow aSB \mid as \mid aA \mid a \mid bB \mid b$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

Q.150. Consider the following grammar.

$S \rightarrow Aa\ A$

$A \rightarrow Sb \mid bCC \mid \in$

$C \rightarrow CC \mid abb$

Eliminate all the $\in$-productions

CS : S-11(6M)

Ans.

- To eliminate $A \rightarrow \in$ from the grammar, the non $\in$-productions to be added are obtained as follows :
  List of the productions containing A on right is
  $S \rightarrow AaA$
- Replace each occurrence of A in this production to obtain the non $\in$-productions to be added to the grammar, the list of these productions is.
  $S \rightarrow aA \mid Aa \mid a$
- Add these productions to the grammar and eliminate $A \rightarrow \in$ from the grammar to obtain the following grammar.
  $S \rightarrow AaA \mid aA \mid Aa \mid a$
  $S \rightarrow Sb \mid bCC$
  $C \rightarrow CC \mid abb$

ELIMINATION OF UNIT PRODUCTION

Q.151. Explain how to eliminate unit productions from the grammar with example.

Ans.

- A production of the form $A \rightarrow B$, where A and B both are the non terminals is called as unit production.
- Unit productions in the grammar increases the cost of derivations.
- Following algorithm can be used to eliminate unit productions from the grammar.

Algorithm :

While there exist a unit production $A \rightarrow B$ in the grammar do

{

select a unit production $A \rightarrow B$, such that there exist at least one non unit production $B \rightarrow a$

for every non unit production $B \rightarrow a$ do

add production $A \rightarrow a$ to the grammar

eliminate $A \rightarrow B$ from the grammar

}

Example :

Given the grammar

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow C \mid b$

$C \rightarrow D$

$D \rightarrow E$

$E \rightarrow a$

- Eliminate all the unit productions from the grammar.
  The given grammar contains productions
  $B \rightarrow C$
  $C \rightarrow D$
  $D \rightarrow E$
  which are the unit productions.
- To eliminate these productions from the given grammar, we select a unit production $B \rightarrow C$, but since there exist no non unit C-production in the grammar, we select $C \rightarrow D$, but since there exist no non unit D-production, we next select $D \rightarrow E$,

there exist a non unit E-production , $E \rightarrow a$, hence we add $D \rightarrow a$, to the grammar and eliminate $D \rightarrow E$ from the grammar.

- But since $B \rightarrow C$ and $C \rightarrow D$ are still there in the grammar, we once again select a unit production $B \rightarrow C$, but since there exist no non unit C-production in the grammar, we select $C \rightarrow D$, now there exist a non unit production $D \rightarrow a$ in the grammar, hence we add $C \rightarrow a$ to the grammar and eliminate $C \rightarrow D$ from the grammar.
- But since $B \rightarrow C$ is still there in the grammar, we once again select a unit production $B \rightarrow C$ now there exist a non unit production $C \rightarrow a$ in the grammar, hence we add $B \rightarrow a$ to the grammar and eliminate $B \rightarrow C$ from the grammar.
- Now there exist no unit productions in the grammar. Therefore the grammar that we get, which does not contain unit productions is :

  $S \rightarrow AB$

  $A \rightarrow a$

  $B \rightarrow a \mid b$

  $C \rightarrow a$

  $D \rightarrow a$

  $E \rightarrow a$
- But we see that the grammar symbols C, D and E, becomes useless as result of elimination of unit productions, because they will not be used in the derivation of any w in L (G). Hence we can eliminate them from the grammar to obtain a grammar.

  $S \rightarrow AB$

  $A \rightarrow a$

  $B \rightarrow a \mid b$
- Therefore, we conclude that to obtain the grammar in the most simplified form, we have to eliminate unit productions first, and then useless grammar symbols.

ELIMINATION OF LEFT RECURSION

Q.152. What do you mean by left recursion? Explain how can it be eliminated from the grammar with example.

Ans. Left Recursion :

- If a grammar contain a pair of productions of the form

  $A \rightarrow A\alpha \mid \beta$ then the grammar is a left recursive grammar.
- Left recursion can be eliminated from the grammar, by replacing a pair of productions $A \rightarrow A\alpha \mid \beta$, using the productions $A \rightarrow \beta B$ and $B \rightarrow \alpha B \mid \in$.
- In general if a grammar contain productions.

  $A \rightarrow A\alpha_1 \mid A\alpha_2 \ldots. \mid A\alpha_m \mid \beta_1 \mid\mid \beta_2 \ldots. \mid \beta_n \mid$,

  then the left recursion can be eliminated by adding the following productions in place of the above productions,

  $A \rightarrow \beta_1 B \mid \beta_2 B \mid \ldots. \mid \beta_n B$

  $B \rightarrow \alpha_1 B \mid \alpha_2 B \mid \ldots. \mid \alpha_m B$

Example :

Consider the following grammar :

$S \rightarrow aBDh$

$B \rightarrow Bb \mid c$

$D \rightarrow EF$

$E \rightarrow g \mid \in$

$F \rightarrow f \mid \in$

- The grammar is left recursive because it contains a pair of productions $B \rightarrow Bb \mid c$, to eliminate the left recursion from the grammar, replace this pair of productions by the following productions.

  $B \rightarrow cC$

  $C \rightarrow bC \mid \in$
- Therefore the grammar that we get after the elimination of left recursion is

  $S \rightarrow aBDh$

  $B \rightarrow cC$

  $C \rightarrow bC \mid \in$

  $D \rightarrow EF$

  $E \rightarrow g \mid \in$

  $F \rightarrow f \mid \in$

Q.153. Eliminate left recursion from the following grammar

$L \rightarrow L, A / A$

$A \rightarrow a$

CS : S-13(2M)

Ans. Here, $A \rightarrow A\ \alpha / \beta$

Here left recursion occurs

$\therefore$ Remove left recursion.

$\therefore\ L \rightarrow AL'$

$L' \rightarrow , AL' | \in$

$A \rightarrow a$

Now, Remove $\in$ because $L' \rightarrow \in$

$\therefore\ L \rightarrow AL' / A$

$L' \rightarrow , AL' |, A$

$A \rightarrow a$

Q.154. Consider the following grammar

(i) $S \rightarrow A$

$A \rightarrow Ad \mid Ac \mid aB \mid aC$

$B \rightarrow bBC \mid f$

(ii) $S \rightarrow (L) \mid a$

$L \rightarrow L.S \mid S$

Eliminate the left recursion from the above grammar.

Ans.

(i) $S \rightarrow A$

$A \rightarrow Ad \mid Ac \mid aB \mid aC$

$B \rightarrow bBC \mid f$

- The grammar is left recursive because it contains productions $A \rightarrow Ad \mid Ae \mid aB \mid aC$, to eliminate the left recursion from the grammar, replace these productions by the following productions.

  $A \rightarrow aBD \mid aCD$

  $D \rightarrow dD \mid cD \mid \in$

- Therefore the grammar that we get after the elimination of left recursion is :

  $S \rightarrow A$

  $A \rightarrow aBD \mid aCD$

  $D \rightarrow dD \mid cD \mid \in$

  $B \rightarrow bBC \mid f$

  $C \rightarrow g$

(ii) $S \rightarrow (L) \mid a$

$L \rightarrow L.S \mid S$

- The grammar is left recursive because it contains productions $L \rightarrow L.S \mid S$ to eliminate the left recursion from the grammar, replace these productions by the following productions :

  $L \rightarrow SA$

  $A \rightarrow SA \mid \in$

- Therefore the grammar that we get after the elimination of left recursion is :

  $S \rightarrow (L) \mid a$

  $L \rightarrow SA$

  $A \rightarrow SA \mid \in$

Q.155. For the given context free grammar remove left recursion and reduce it further.

$S \rightarrow aBDh$

$B \rightarrow Bbc \mid b$

$D \rightarrow dD \mid \in$

CT : S-06(4M)

Ans.

- The grammar is left recursive because it contains a pair of production $B \rightarrow Bbc \mid b$.
- To eliminate the left recursion from the grammar, replace this pair of productions by the following productions.

  $B \rightarrow bE$

  $E \rightarrow bcE \mid \in$

- Therefore, the grammar that we get after the elimination of left recursion is

  $S \rightarrow aBDh$

  $B \rightarrow bE$

  $E \rightarrow bcE \mid \in$

  $D \rightarrow dD \mid \in$

- Since every non terminal symbol B, E and D are reachable from start state.
- Also they are capable of deriving to w in T^*. Therefore the above CFG cannot be further reduced.

Q.156. Remove all unit production, all useless production and all $\in$-productions from the grammar.

OR Simplify following CFG

S → aA | aBB

A → aaA | ∈

B → bB | bbC

C → B What language does this grammar generate ? ***CS : S-12(4M), CT : W-08(7M)***

Ans. S → aA | aBB

A → aaA | ∈

B → bB | bbC

C → B

Removing unit production, C → B.

S → aA | aBB

A → aaA | ∈

B → bB | bbC

C → bB | bbC

Removing ∈ - production, A → ∈

S → aA | aBB | a

A → aaA | aa

B → bB | bbC

C → bB | bbC

Removing useless symbol C

S → aA | aBB | a

A → aaA | aa

B → bB

B is also useless, so remove it

The grammar is,

S → aA | a

A → aaA | aa

Q.157. Describe language generated by grammar :

S → aAB

A → bBb

B → A | ∈ ***CS : S-11(2M)***

Ans. Elimination of ∈ - production :

Here B → ∈ is ∈-production, so remove it from the grammar

Now, production are

S → aAB | aA

A → bBb | bb

B → A

Also,

B → A is unit production.

Hence remove it.

S → aA

A → bb

Here replace ' → ' by '='

S = aA

A = bb

∴ S = abb

NORMAL FORMS FOR CFG

Q.158. State the normal forms for CFG.

Ans. There are two normal forms for CFG :

(i) Chomsky-Normal Form (CNF)

(ii) Greibach Normal Form (GNF)

Q.159. Give Chomsky Normal Form.

Ans. Chomsky Normal Form : A grammar is said to be in Chomsky Normal Form (CNF) if it is ∈-free and each non ∈-production is of the form A → BC or of the form A → a.

Here A, B and C are non terminals and a is a terminal.

Q.160. How to convert the CFG into Chomsky normal form (CNF) explain with example?

Ans. In order to convert the CFG into CNF the following steps should be followed.

(1) Eliminate ∈-productions from the grammar.

(2) Eliminate unit productions from grammar.

(3) Eliminate useless symbols from the grammar.

(4) After that obtain the grammar in one of the following form

A → BC

A → a

where A, B, C are non-terminals and a is a terminal symbols.

Let G be the grammar with productions

S → AACD

A → aAb | ∈

C → aC | a

D → aDa | bDb| ∈

Let us go through each step of the conversion to CNF.

1. Eliminating ∈-productions :

The nullable variables are A and D and The grammar with productions is,

$S \rightarrow AACD \mid ACD \mid AAC \mid CD \mid AC \mid C$

$A \rightarrow aAb \mid ab$

$C \rightarrow aC \mid a$

$D \rightarrow aDa \mid bDb \mid aa \mid bb$

2. **Eliminating unit productions :**

Here we may simply add the productions

$S \rightarrow aC \mid a$

and delete $S \rightarrow C$

3. **Restricting the right sides of productions to single terminals or strings of two or more variables :**

This step yields the productions.

$S \rightarrow AACD \mid ACD \mid AAC \mid CD \mid AC \mid X_aC \mid a$

$A \rightarrow X_aAX_b \mid X_aX_b$

$C \rightarrow X_aC \mid a$

$D \rightarrow X_aDX_a \mid X_bDX_b \mid X_aX_a \mid X_bX_b$

$X_a \rightarrow a$

$X_b \rightarrow b$

4. **The final step to CNF :**

There are six productions whose right side are too long. Applying our algorithm produces the grammar with productions.

$S \rightarrow AT_1$ $T_1 \rightarrow AT_2$ $T_2 \rightarrow CD$

$S \rightarrow AU_1$ $U_1 \rightarrow CD$

$S \rightarrow AV_1$ $V_1 \rightarrow AC$

$S \rightarrow CD \mid AC \mid X_aC \mid a$

$A \rightarrow X_aW_1$ $W_1 \rightarrow AX_b$

$A \rightarrow X_aX_b$

$C \rightarrow X_aC \mid a$

$D \rightarrow X_aY_1$ $Y_1 \rightarrow DX_a$

$D \rightarrow X_bZ_1$ $Z_1 \rightarrow DX_b$

$D \rightarrow X_aX_b \mid X_bX_b$

$X_a \rightarrow a$ $X_b \rightarrow b$

Q.161. Convert the grammar with production :

$S \rightarrow ABa$

$A \rightarrow aab$

$B \rightarrow Ac$, to Chomsky normal form.

Ans. $S \rightarrow Aba$ is replaced by $S \rightarrow ABC$ and $C \rightarrow a$

Similarly $A \rightarrow aab$ is replaced by $A \rightarrow CCD$ and $D \rightarrow b$

$B \rightarrow Ac$ is replaced by $B \rightarrow AE$ and $E \rightarrow c$.

In the next stage, the production $S \rightarrow ABC$ is replaced by $S \rightarrow AF$ and $F \rightarrow BC$ and the production $A \rightarrow CCD$ is replaced by $A \rightarrow CG$ and $G \rightarrow CD$.

The production for grammar in CNF are shown below :

$S \rightarrow AF$ $B \rightarrow AE$

$F \rightarrow BC$ $C \rightarrow a$

$A \rightarrow CG$ $D \rightarrow b$

$G \rightarrow CD$ $E \rightarrow c$.

Q.162. Reduce following grammar to CNF :

$S \rightarrow aAD$

$A \rightarrow aB$

$B \rightarrow b$

$D \rightarrow b$

Ans. $S \rightarrow aAD$ is replaced by $S \rightarrow CAD$ and $C \rightarrow a$. Similarly $A \rightarrow aB$ is replaced by $A \rightarrow CB$.

$A \rightarrow bAB$ is replaced by $A \rightarrow BAB$ and $B \rightarrow b$.

In the next stage, the production $S \rightarrow CAD$ is replaced by $S \rightarrow CE$ and $E \rightarrow AD$ and the production $A \rightarrow BAC$ is replaced by $A \rightarrow BF$ and $F \rightarrow AB$.

The production for grammar in CNF are shown below.

$S \rightarrow CE$

$A \rightarrow CB \mid BF$

$E \rightarrow AD$

$F \rightarrow AB$

$C \rightarrow a$

$B \rightarrow b$

$D \rightarrow d$

Q.163. Given a grammar G = (V, T, P, S) which is in CNF and a string w, whose length is n. How long the derivation of w is? Why?

CT : S-06(4M)

Ans. Let $w \in L(G)$ and $|w| = k$.

- When G is in GNF, the number of steps in the derivation of w is k, where $k = |w|$.

- The number of terminals increase by 1 for each application and a production to a sentential form. Hence the number of steps in the derivation of w is k.
- In the CNF, each production yields one terminal or two variables but nothing else.
- For getting the terminals in w, we have to apply production of the form A → a (k times).
- The corresponding string of variables. (Non terminals) which is of length k can be obtained by k – 1 steps. (Each production A → increases the number of variables by one). So the total no. of steps is 2k–1.

Q.164. Find a grammar in Chomsky normal form equivalent to

S → aAbB

A → qA | q

B → bB | b ***CT : W-07(5M)***

Ans. Productions that are already in proper form are A → q, B → b.

There are no unit productions, so we may begin by replacing terminals on the right side by non terminals, except in the case of the productions A → q and B → b.

S → aAbB is replaced by S → DAEB, D → a and E → b.

Similarly A → q A is replaced by A → FA and F → a.

Also B → bB is replaced by B → EB and E → b.

We get following set of productions.

S → DAEB

A → FA | q

B → EB | b

D → a

E → b.

F → q

Now the production S → DAEB is replaced by S → GB and G → DAE.

At the next stage G → DAE is replaced by G → HE and H → DA.

The production for grammar in CNF are shown below.

S → GB

G → HE

H → DA

A → FA|q

B → EB|b

D → a

E → b.

F → q

Q.165. Find a grammar in CNF equivalent to the grammar

S → –S | [S ↑ S] | a | b

OR

S → –S

S → [S ↑ S]

S → a

S → b

Ans.

- Productions that are already in proper form are S → a , S → b .
- There are no unit productions, so we may begin by replacing terminals on the right side by non terminals except in case of productions S → a and S → b .

  There are 4 terminals –, [, ↑,]

  S → – S | [S ↑ S] | a | b is replaced by

  S → AS | B S C S D | a | b,

  A → –, B → [, C → ↑, D →]
- The set of productions are ,

  S → AS | BSCSD | a | b

  A → –

  B → [

  C → ↑

  D →]

  Now S → BSCSD is replaced by S → BE and E → SCSD. At next stage E → SCSD is replaced by E → SF and F → CSD.

  Further, F → CSD is replaced by F → CG and G → S D.

  Thus we obtain productions for grammar as follows.

  S → AS | BE | a | b

  E → S F

  F → C G

  G → S D

A → –
B → [
C → ↑
D →]
which is grammar in CNF equivalent to the given grammar.

Q.166. Given a grammar ({S, A, B}, {a, b}, P, S) where P is
S → bA | aB
A → bAA | aS | a
B → aBB | bS | b
Find an equivalent grammar in CNF.

Ans. **S → bA | aB**
A → bAA | aS | a
B → aBB | bS | b

- Productions that are already in proper form are A→a, B→b.
- There are no unit productions, so we may begin by replacing terminals on the right by variables, except in the case of the productions A→a and B→b . S→ bA is replaced by S→ CA and C→ b. Similarly,
- A→aS is replaced by A →DS and D →a. A →bAA is replaced by A →CAA, S→ aB is replaced by S →DB, B→ bS is replaced by B→ CS and B →aBB is replaced by B →DBB.
- In the next stage, the production A →CAA is replaced by A→CH and H→AA, and the production B →DBB is replaced by B →DE & E→BB.
- The productions for grammar in CNF are shown below.

S → CA | DB H → AA
A → DS | CH | a E → BB
B → CS | DE | b C → b
D → a

Q.167. Reduce the following grammar to CNF
S → 1A | 0B
A → 1AA | 0S | 0
B → 0BB | 1S | 1

Ans.

- S→1A is replaced by S→DA and D→1. Similarly S→0B is replaced by S→EB and E→0.
- Therefore A →1AA|0S is replaced by A→DAA | ES and B → 0BB | 1SS is replaced by B → EBB | DSS .
- Following set of productions is obtained
  S → DA | EB
  A → DAA | ES | 0
  B → EBB | DS | 1
  D → 1
  E → 0
- Now, A→ DAA is replaced by A→ DF and F→ AA. B→EBB is replaced by B→EG and G→ BB.
- The productions for grammar in CNF are shown below.
  S → DA | EB
  A → DF | ES | 0
  B → EG | DS | 1
  F→ AA
  G →BB
  D →1
  E →0

Q.168. Convert the following CFG to CNF
S → aSa | bSb | a | b | aa | bb

Ans.

- Replace S → aSa | bSb| a | b | aa | bb by S→ ASA | BSB | a | b | AA | BB, A→a and B →b.
- At next stage, S → ASA is replaced by S → AD and D → SA. S → BSB is replaced by S→BE and E→ SB.
- The productions for grammar in CNF are shown below.
  S → AD | BE | a | b | AA | BB
  D → SA
  E → SB
  A → a
  B → b

Q.169. Convert the following into CNF

$S \rightarrow aA \mid a \mid b \mid c$

$A \rightarrow aB \mid e$

$B \rightarrow aA$

$C \rightarrow cCD$

$D \rightarrow abd$ **CS : S-11(7M)**

Ans. Here, $S \rightarrow a$, $S \rightarrow b$, $S \rightarrow c$, $A \rightarrow e$ } Already in CNF form

(1) $S \rightarrow aA$

$\therefore S \rightarrow SA$ *[$\because S \rightarrow a$]*

(2) $A \rightarrow aB$

$\therefore A \rightarrow SB$ *[$\because S \rightarrow a$]*

(3) $B \rightarrow aA$

$\therefore B \rightarrow SA$ *[$\because S \rightarrow a$]*

(4) $C \rightarrow cCD$

$\therefore C \rightarrow SCD$ *[$\because S \rightarrow c$]*

$C \rightarrow SA$ *[$\because A_1 \rightarrow CD$]*

(5) Consider

$A_2 \rightarrow d$

(6) $D \rightarrow abd$

$D \rightarrow SS\,A_2$ *[$\because S \rightarrow a, S \rightarrow b, A_2 \rightarrow d$]*

$D \rightarrow S_1\,A_2$ *[$\because S_1 \rightarrow SS$]*

Equivalent grammar in CNF is

$S \rightarrow SA \mid a \mid b \mid c$

$A \rightarrow SB \mid e$

$A_1 \rightarrow CD$

$B \rightarrow SA$

$C \rightarrow SA_1$

$D \rightarrow S_1\,A_2$

$S_1 \rightarrow SS$

Q.170. convert the following in CNF :

$S \rightarrow ABa \mid b$

$A \rightarrow bbA \mid B$

$B \rightarrow aAa$ **CT : W-12(7M)**

OR $S \rightarrow ABb \mid a$

$A \rightarrow aaA \mid B$

$B \rightarrow bAb$ **CT : S-13(5M)**

Ans. Removal of unit production

$S \rightarrow ABa \mid b$

$A \rightarrow bbA \mid aAa$

$B \rightarrow aAa$

Now,

Here $S \rightarrow b$ is already in CNF :

Consider :

(1) $S \rightarrow ABa$

Let $A_1 \rightarrow a$

$S \rightarrow ABA$

Let $A_2 \rightarrow AB$

$S \rightarrow A_2\,A_1$

(2) $A \rightarrow bbA$

$A \rightarrow SSA$

Let $S_1 \rightarrow SS$

$\boxed{A \rightarrow S_1\,A}$

(3) $A \rightarrow aAa$

$A \rightarrow A_1\,AA_1$

Let $\boxed{A_3 \rightarrow AA_1}$

$\boxed{A \rightarrow A_1A_3}$

(4) $\boxed{B \rightarrow A_1A_3}$

Required CNF :

$S \rightarrow A_2\,A_1 \mid b$

$S_1 \rightarrow SS$

$A \rightarrow S_1\,A \mid A_1\,A_3$

$A_1 \rightarrow a$

$A_2 \rightarrow AB$

$A_3 \rightarrow AA_1$

$B \rightarrow A_1\,A_3$

Q.171. Construct CFG for the language :

$L = \{a^{3n} \mid n \geq 1\}$ and convert the CFG into CNF. **CT : W-09, S-10(7M)**

Ans. $L = \{a^{3n} \mid n \geq 1\}$

$\therefore L = \{aaa, aaaaaa, \ldots\ldots\}$

$\therefore S \rightarrow aaa \mid aA$

$A \rightarrow aa \mid aaS$ is CFG.

Convertion of CFG to CNF :

Let $R_1 \rightarrow a$

$\therefore S \rightarrow R_1 R_1 R_1 / R_1 A$

$A \rightarrow R_1 R_1 / R_1 R_1 S$

For,

$S \rightarrow R_1 R_1 R_1$

$\therefore R_2 \rightarrow R_1 R_1$

$\therefore S \rightarrow R_2 R_1$

For $A \rightarrow R_1 R_1 S$

$\therefore A \rightarrow R_2 S$

CNF is

$R_1 \rightarrow a$

$S \rightarrow R_2 R_1 / R_1 A$

$R_2 \rightarrow R_1 R_1$

$A \rightarrow R_2 S$

Q.172. Find the equivalent CNF for following grammar :

$S \rightarrow aB \mid ab$

$A \rightarrow aAB \mid a$

$B \rightarrow ABb \mid b$ ***CS : W-09(4M)***

Ans. $A \rightarrow a$, $B \rightarrow b$ } Already in CNF from

Consider production :

(1) $S \rightarrow aB \mid ab$

Since, $A \rightarrow a$ and $B \rightarrow b$

$S \rightarrow AB / AB$

$\therefore S \rightarrow AB$

(2) Consider

$A \rightarrow aAB / a$

$\therefore A \rightarrow AAB / a$

Since,

$S \rightarrow AB$

$\therefore A \rightarrow AS / a$

(3) $B \rightarrow ABb / b$

$\therefore B \rightarrow ABB / b$

Since,

$S \rightarrow AB$

$\therefore B \rightarrow SB / b$

Hence, grammar in CNF is

$S \rightarrow AB$

$A \rightarrow AS / a$

$B \rightarrow SB / b$

Q.173. Define GNF (Greibach normal form)

OR When do you say, context free grammar (CFG) is in Greibach Normal Form (GNF) ?

Ans. GNF :

Every context free language L without ϵ can be generated by grammar for which every production is of the form $Aa \rightarrow \alpha$,

where A is a non-terminal, a is a terminal and α is a set of zero or more non terminal symbol.

Q.174. Prove the following ;

Every context free language L without ϵ can be generated by a grammar for which every production is of the form $A \rightarrow a\alpha$, where A is a variable, a is a terminal and α is a (possibly empty) string of variables.

Ans.

- Let G = (V, T, P, S) be a Chomsky Normal Form grammar generating the context free language L. Assume that $V = \{A_1, A_2, \ldots\ldots, A_m\}$.

  The first step in the construction is to modify the productions so that if $A_i \rightarrow A_j \gamma$ is a production, then $j > i$.
- Starting with A_1 and proceeding to A_m, we do this as follows.
- We assume that the productions have been modified so that for $1 \leq i < k$, $A_i \rightarrow A_j \gamma$ is a production only if $j > i$. We now modify the A_k productions.
- If $A_k \rightarrow A_j \gamma$ is a production with $j < k$, we generate a new set of productions by substituting for A_j the right hand side of each A_j-production.
- By repeating the process $k - 1$ times at most, we obtain productions of the form $A_k \rightarrow A_l \gamma\ l \geq k$.
- The productions with $l = k$ are then replaced, introducing a new variable B_k. The precise algorithm is given in fig. below.

```
begin
1) for k : = 1 to m do
   begin
2) for j : = 1 to k – 1 do
3) for each production of the form
   A_k → A_j α do
   begin
4) for all productions A_j → β do
5) add production A_k → βα
6) remove production A_k → A_jα
   end ;
7) for each production of the form
   A_k → A_k α do
   begin
8) add production B_k → α and B_k → αB_k ;
9) remove production A_k → A_kα
   end ;
10) for each production A_k → β,
   where β does not begin with A_k do
11) add production A_k → βB_k
   end
. end
```

Fig. Step 1 In the Grei bach normal -form algorithm.

- By repeating the above process for each original variable, we have only production of the forms :

(1) $A_i \rightarrow A_j \gamma$, $j > 1$

(2) $A_i \rightarrow a\gamma$, a in T

(3) $B_i \rightarrow \gamma$; γ in $(V \cup \{B_1, B_2, \ldots\ldots B_{i-1}\})^*$

- The leftmost symbol on the right-hand side of any production for A_m must be a terminal, since A_m is the highest- numbered variable.
- The leftmost symbol on the right-hand side of any production for A_{m-1} must be either A_m or a terminal symbol.
- When it is A_m, we can generate new productions by replacing A_m by the right-hand side of the productions for A_m.
- These productions must have right sides that start with a terminal symbol. We then proceed to the productions for $A_{m-2} \ldots A_2, A_1$ until the right side of each production for an A_i starts with a terminal symbol.
- As the last step we examine the productions for new variables, $B_1, B_2, \ldots\ldots B_m$. Since we begin with grammar in Chomsky normal form, it is easy to prove by induction on the number of applications that the right-hand side of every A_i production, $1 \leq i \leq n$, begins with a terminal or A_jA_k for some j and k. Thus α in line (7) of fig. can never be empty or begin with some B_j, so no B_i-production can start with another B_j.
- Therefore all B_i-productions have right hand sides beginning with terminals or A_i's for each B_i production completes the construction.

Q.175. Write the steps for converting any context free grammar to GNF. (Greibach Normal Form)

Ans. Steps for converting any context free grammars to GNF :

(1) First of all, we simply convert the given CFG to CNF. If the given CFG is already in CNF form then move to second step, as follows.

(2) Now, we rename all the variables of the grammar by the names $A_1, A_2, A_3, A_4, \ldots\ldots$ respectively. It should be noted that the start symbol of the grammar is always renamed by A_1.

Example :

Let a grammar has the non-terminals S, A, B and C. We have to rename these non-terminals of a grammar as under -

Variable	New names
S (Start symbol)	A_1
A	A_2
B	A_3
C	A_4

If the variables of the grammar is already in the required names (i.e. in the form $A_1, A_2, \ldots\ldots$), then we simply move to the third step, as follows.

(3) Now we distinguish the productions of the grammar into two parts -

(a) One, which is already in the form of GNF, i.e. in the form $A \to a\alpha$ or $A \to a$, where A is any variable, a is any terminal and α is string of variable.

(b) Another, which is not in the form of GNF.

(4) Now, from the productions which is not in the form of GNF, find out the productions which is not in the proper form of A_i production, where $i = 1, 2 .....$

Proper form of A_i (i.e. A_1) production.

The production which is in the form of $A_i = A_i A_1 .....$, where $i < j$ is in the proper form of A_1-production. Thus, when $i \geq j$, the production is not in the proper form of A_1-production. Thus there are three cases-

Case 1 : $i < j$ Production is in proper form, so presently left it as it is.

Case 2 : $i > j$ Production is not in proper form. In this case use the productions of the grammar which is in the proper form of A_1 production to get $i = j$.

Case 3 : $i = j$ Also, not in the proper form but in this case we have not to reduce it to get $i = j$ as it is already in this form.

- Let us assume that we have the production in the given grammar-

  $A_1 \to A_1A_1 \mid A_2A_3 \mid A_4A_3$

- Thus, here, it is clear that $A_1 \to A_1A_1$ is not in the proper form from case 3. Thus we have to replace this production from the productions which is in the proper form.

(a) Consider the complete production, and process it as under-

$A_1 \to A_1A_1\ \alpha_i\ \beta_i \mid A_2A_3 \mid A_4A_3.....$

Thus we get, -

$\alpha_i = A_1$

$\beta_i = A_2A_3 \mid A_4A_3.....$

- Now we introduce a new symbol with different name and with same i as in the L.H.S of this production.

- Here in the L.H.S., we have, A_i . Thus, we can introduce a new symbol B_1 which has different name from A and same i as of A.

(b) Now for replacing the production $A_1 \to A_1A_1$, we simply use the following formulas -

$A_1 \to \beta_i \quad A \to \beta_iB \quad B \to \alpha_1 \quad B \to \alpha_i B$

Thus, using these formulas we can replace the production

$A_1 \to A_1A_1$ by the productions -

$A_1 \to A_2A_3 \mid A_4A_3$

$A_1 \to A_2A_3B_1 \mid A_4A_3B_1$

$B_1 \to A_1$

$B_1 \to A_1 B$

(c) Apply these process for all the productions which is not in the proper form, to get the productions in the proper A_1 form.

(5) Now, in all productions which is in the proper form of A_1-productions, put the values of the productions which is already in the proper form of GNF.

- Doing this process, we will get all the productions in the GNF form.

These are the required steps for converting CFG to GNF.

Q.176. Obtain equivalent GNF for following CFG :

S → AB | 0C2
A → 0A1 | 01
B → 1B2 | 12
C → 0C2 | 0D1 | 01
D → 0D1 | 01

CS : S-13(6M)

Ans. S → AB is already in CNF

Now for other production add new 3 productions

P → 0
Q → 1
R → 2 then we get

S → PCR | AB
A → PAQ | PQ
B → QBR | QR
C → PCR | PDQ | PQ
D → PDQ | PQ
P → 0
Q → 1
R → 2

Now

$S \rightarrow AB$

$A \rightarrow PQ$

$B \rightarrow QR$

$C \rightarrow PQ$

$D \rightarrow PQ$ are in CNF

For $A \rightarrow PAQ$

$S \rightarrow AQ$

then $A \rightarrow PS$

For $B \rightarrow QBR$

$T \rightarrow BR$

then $B \rightarrow QT$

For $C \rightarrow PCR$

$R_1 \rightarrow CR$

$\boxed{C \rightarrow PR_1}$

For $C \rightarrow PDQ$ & $D \rightarrow PDQ$

$R_2 \rightarrow DQ$

$C \rightarrow PR_2$ & $D \rightarrow PR_2$

Final grammar is

$S \rightarrow AB \mid AQ$

$A \rightarrow PQ \mid PS$

$B \rightarrow QR \mid QT$

$C \rightarrow PR_1 \mid PR_2 \mid PQ$

$D \rightarrow PR_2 \mid PQ$

$P \rightarrow 0$

$Q \rightarrow 1$

$R \rightarrow 2$

$S \rightarrow AQ$

$T \rightarrow BR$

$R_1 \rightarrow CR$

$R_2 \rightarrow DQ$

Q.177. Convert the following grammar into Greibach normal form.

$G = (\{A_1, A_2, A_3\}, \{a, b,\} P, A_1)$.

where P consists of the following

$A_1 \rightarrow A_2A_3$

$A_2 \rightarrow A_3A_1 \mid b$

$A_3 \rightarrow A_1A_2 \mid a$

OR $S \rightarrow AB$

$A \rightarrow BS \mid b$

$B \rightarrow SA \mid a$ **CS : S-11 (8M), CT : W-11(8M)**

Ans.

Step 1 : Since the RHS of the productions for A_1 and A_2 start with terminals or high-numbered variables, therefore, in the production $A_3 \rightarrow A_1 \rightarrow A_2$, we substitute the string $A_2\ A_3$ for A_1. Note that $A_1\ A_2\ A_3$ is the only production with A_1 on the left.

The resulting set of productions is

$A_1 \rightarrow A_2A_3$

$A_2 \rightarrow A_3A_1 \mid b$

$A_3 \rightarrow A_2A_3A_2 \mid a$

Since, the right side of the production $A_3 \rightarrow A_2A_3A_2$, begins with a lower-numbered variable, we substitute for the first occurrence of A_2 both $A_3\ A_1$ and b. Thus $A_3 \rightarrow A_2A_3A_2$ is replaced by $A_3 \rightarrow A_3A_1A_3A_2$ and $A_3 \rightarrow bA_3A_2$.

The new set is

$A_1 \rightarrow A_2A_3$

$A_2 \rightarrow A_3A_1 \mid b$

$A_3 \rightarrow A_3A_1A_3A_2 \mid bA_3A_2 \mid a$

In the production $A_3 \rightarrow A_3A_1A_3A_2 \mid bA_3A_2 \mid a$. Symbol B_3 is introduced and the production $A_3 \rightarrow A_3A_1A_3A_2$ is replaced by $A_3 \rightarrow b\ A_3\ A_2\ B_3$, $A_3 \rightarrow A\ B_3$, $B_3 \rightarrow A_1A_3A_2$, and $B_3 \rightarrow A_1\ A_3\ A_2\ B_3$

The resulting set is

$A_1 \rightarrow A_2\ A_3$

$A_2 \rightarrow A_3A_1 \mid b$

$A_3 \rightarrow bA_3A_2B_3 \mid aB_3\ \ bA_3A_2 \mid a$

$B_3 \rightarrow A_1A_3A_2 \mid A_1A_3A_2B_3$

Step 2 : All the productions with A_3 on the left have right hand sides that start with terminals. These are used to replace A_3 in the production $A_2 \rightarrow A_3\ A_1$ and then the productions with

$A_2 \rightarrow A_3\ A_1$ and then the productions with A_2 on the left are used to replace A_2 in the production $A_1 \rightarrow A_2\ A_3$.

The result is shown below.

$A_3 \rightarrow b\ A_3A_2B_3$ $\quad$ $A_3 \rightarrow b\ A_3A_2$

$A_3 \rightarrow a\ B_3$ $\quad$ $A_3 \rightarrow a$

$A_2 \rightarrow bA_3A_2B_3A_1$ $\quad$ $A_2 \rightarrow bA_3A_2A_1$

$A_2 \rightarrow aB_3A_1$ $\quad$ $A_2 \rightarrow aA_1$

$A_2 \rightarrow b$

$A_1 \rightarrow b\ A_3A_2A_1A_3$ $\quad$ $A_1 \rightarrow bA_3A_2A_1A_3$

$A_1 \rightarrow a\ B_3A_1A_3$ $\quad$ $A_1 \rightarrow a\ a_3A_1A_3$

$A_1 \rightarrow b\ A_3$

$B_3 \rightarrow A_1A_3A_2$ $\quad$ $B_3 \rightarrow A_1A_3A_2B_3$

Step 3 : The two B_3 productions are converted to proper form, resulting in 10 more productions. That is, the productions $B_3 \rightarrow A_1A_3A_2$ and $B_3 \rightarrow A_1A_3A_2B_3$ are altered by substituting the right side of each of the five productions with A_1 on the left for the first occurrence of A_1.

Thus $B_3 \rightarrow A_1A_3A_2$ becomes

$B_3 \rightarrow b\ A_3\ A_2\ B_3\ A_1\ A_3\ A_3\ A_2$

$B_3 \rightarrow aB_3\ A_1\ A_3\ A_3\ A_2$

$B_3 \rightarrow bA_3\ A_3\ A_2$,

$B_3 \rightarrow bA_3\ A_2\ A_1\ A_3\ A_3\ A_2$,

$B_3 \rightarrow aA_1\ A_3\ A_3\ A_2$

The other production for B_3 is replaced similarly.

The final set of productions is

$A_3 \rightarrow b\ A_3A_2B_3$ $\quad$ $A_3 \rightarrow b\ A_3A_2$

$A_3 \rightarrow a\ B_3$ $\quad$ $A_3 \rightarrow a$

$A_2 \rightarrow b\ A_3A_2B_3A_1$ $\quad$ $A_2 \rightarrow b\ A_3A_2A_1$

$A_2 \rightarrow a\ B_3A_1$ $\quad$ $A_2 \rightarrow a\ A_1$

$A_2 \rightarrow b$

$A_1 \rightarrow b\ A_3A_2B_3A_1A_3$

$A_1 \rightarrow b\ A_3A_2A_1A_3$

$A_1 \rightarrow a\ B_3A_1A_3$ $\quad$ $A_1 \rightarrow a\ A_1A_3$

$A_1 \rightarrow b\ A_3$

$B_3 \rightarrow bA_3A_2B_3A_1A_3A_3A_2B_3$

$B_3 \rightarrow bA_3A_2B_3A_1A_3A_3A_2$

$B_3 \rightarrow aB_3A_1A_3A_3A_2B_3$

$B_3 \rightarrow aB_3A_1A_3A_3A_2$

$B_3 \rightarrow bA_3A_3A_2B_3$ $\quad$ $B_3 \rightarrow bA_3A_3A_2$

$B_3 \rightarrow bA_3\ A_2\ A_1\ A_3\ A_3\ A_2\ B_3$

$B_3 \rightarrow bA_3\ A_2\ A_1\ A_3\ A_3\ A_2$

$B_3 \rightarrow aA_1\ A_3\ A_3\ A_2\ B_3$

$B_3 \rightarrow a\ A_1\ A_3\ A_3\ A_2$

Q.178. Construct a grammar in Greibach normal form equivalent to the grammar.

$S \rightarrow AA \mid a$

$A \rightarrow SS \mid b$

OR $S \rightarrow AA \mid 0$

$A \rightarrow SS \mid 1$

CS : W-11(7M),CT : W-06(8M), W-10(6M) S-11,12(7M)

Ans.

- The given grammar is already in the form of CNF. Thus to convert it into GNF, we rename its non-terminal as follows.

Non terminal	Name
S	A_1
A	A_2

- Therefore given grammar becomes

$$\left.\begin{array}{l} A_1 \rightarrow A_2A_2 \mid a \\ A_2 \rightarrow A_1A_1 \mid b \end{array}\right\} \quad \ldots(1)$$

- In the resultant grammar following is the productions which is already in the form of A_1 production.

$A_1 \rightarrow A_2A_2$

$A_1 \rightarrow a$

$A_2 \rightarrow b$

Also in the resultant grammar (1) following is the production which is not in the form of A_1 production.

$A_2 \rightarrow A_1A_1$

Thus,

(1) Considering the productions

$A_2 \to A_1A_1$

$\therefore A_2 \to A_2\ A_2\ A_1$ [$\because A_1 \to A_2A_2$]

$\therefore A_2 \to aA_1$ [$\because A_1 \to a$]

$\therefore$ The resultant grammar becomes

$$\left.\begin{array}{l} A_1 \to A_2A_2 \mid a \\ A_2 \to A_2A_2A_1 \mid aA_1 \mid b \end{array}\right\} \quad \ldots\ldots (2)$$

From above, we have to consider production

$A_2 \to A_2A_2A_1 \mid aA_1 \mid b$

Here

$\alpha_i = A_2\ A_1$

$\beta = aA_1 \mid b$

- Now introduce a new symbol B_2. At this stage, we have to replace the production $A_2 \to A_2A_2A_1$ of the resultant grammar using the following formulas.

$A \to \beta_i$, $A \to \beta_iB_2$, $B_2 \to \alpha_i$ and $B_2 \to \alpha_i\ B_2$

Thus

$A_2 \to aA_1$

$A_2 \to b$

$A_2 \to aA_1B_2$

$A_2 \to bB_2$

$B_2 \to A_2\ A_1$

$B_2 \to A_2\ A_1B_2$

- Therefore the resultant grammar becomes

$A_1 \to A_2\ A_2 \mid a$

$A_2 \to aA_1 \mid b \mid aA_1B_2 \mid bB_2 \mid aA_1 \mid b$

$B_2 \to A_2A_1 \mid A_2A_1B_2$

- Now removing duplicates from the above resultant grammars.

$$\left.\begin{array}{l} A_1 \to A_2A_2 \mid a \\ A_2 \to aA_1 \mid b \mid aA_1B_2 \mid bB_2 \\ B_2 \to A_2A_1 \mid A_2A_1B_2 \end{array}\right\} \quad \ldots.(3)$$

Since we have production in (3)

$A_1 \to A_2A_2$

Writing this production as

$A_1 \to aA_1A_2$ [$\because A_2 \to aA_1$]

and, $A_1 \to bA_2$ [$\because A_2 \to b$]

and, $A_1 \to aA_1B_2A_2$ [$\because A_2 \to a\ A_1B_2$]

and, $A_1 \to b\ B_2A_2$ [$\because A_2 \to bB_2$]

Therefore resultant grammar becomes

$$\left.\begin{array}{l} A_1 \to a\ A_1A_2 \mid bA_2 \mid aA_1B_2A_2 \mid b\ B_2A_2 \mid a \\ A_2 \to a\ A_1 \mid b \mid aA_1B_2 \mid bB_2 \\ B_2 \to A_2A_1 \mid A_2A_1B_2 \end{array}\right\} (4)$$

The resultant grammar has the productions

$B_2 \to A_2A_1$ and $B_2 \to A_2A_1B_2$

(1) Considering the productions

$B_2 \to A_2A_1$

We can write this production as

$B_2 \to a\ A_1A_1$ [$\because A_2 \to a\ A_1$]

and, $B_2 \to b\ A_1$ [$\because A_2 \to b$]

and, $B_2 \to a\ A_1B_2A_1$ [$\because A_2 \to a\ A_1B_2$]

and, $B_2 \to b\ B_2A_1$ [$\because A_2 \to b\ B_2$]

(2) Considering the production

$B_2 \to A_2A_1B_2$

We can write this production as

$B_2 \to a\ A_1A_1B_2$ [$\because A_2 \to a\ A_1$]

and, $B_2 \to b\ A_1B_2$ [$\because A_2 \to b$]

and, $B_2 \to a\ A_1B_2A_1B_2$ [$\because A_2 \to a\ A_1B_2$]

and, $B_2 \to b\ B_2A_1B_2$ [$\because A_2 \to b\ B_2$]

$\therefore$ Therefore the resultant grammar becomes

$$\left.\begin{array}{l} A_1 \to a\ A_1A_2 \mid bA_2 \mid aA_1B_2A_2 \mid b\ B_2A_2 \mid a \\ A_2 \to a\ A_1 \mid b \mid aA_1B_2 \mid bB_2 \\ B_2 \to a\ A_1A_1 \mid bA_1 \mid aA_1B_2A_1 \\ \qquad \mid bB_2A_1 \mid aA_1A_1B_2 \\ b\ A_1B_2 \mid a\ A_1B_2A_1B_2 \mid b\ B_2A_1B_2 \end{array}\right\} ..(5)$$

The resultant grammar (5) is in the form of GNF.

Q.179. Reduce the following grammar to GNF

$S \to SS$

$S \to 0\,S1 \mid 01$ **CT :W-07(6M)**

Ans.

- As the given grammar is not in CNF therefore we have to convert it into CNF

∴ Given grammar

$S \to SS$

$S \to 0S1 \mid 01$

- Replace $S \to 0S1 \mid 01$ by $S \to A\,SB \mid AB$, $A \to 0$ and $B \to 1$.
- We get the following set of production

$S \to SS$

$S \to A\,SB \mid AB$

$A \to 0$

$B \to 1$

- Replace $S \to ASB$ by $S \to AC$ and $C \to SB$.
- The productions for grammar in CNF are shown below.

$S \to SS$

$S \to A\,C \mid AB$

$C \to SB$

$A \to 0$

$B \to 1$

Conversion of CNF to GNF

First, rename all non terminal symbol

Non terminal	Name
S	A_1
C	A_2
A	A_3
B	A_4

- The grammar becomes

$$\left.\begin{array}{l} A_1 \to A_1A_1 \\ A_1 \to A_3A_2 \mid A_3A_4 \\ A_2 \to A_1A_4 \\ A_3 \to 0 \\ A_4 \to 1 \end{array}\right\} \quad \ldots\ldots(1)$$

- In above grammar the production $A_1 \to A_1A_1$ is not in the proper form

∴ Therefore from grammar (1), consider the production

$A_1 \to A_1A_1 \mid A_3A_2 \mid A_3A_4$

Here $\alpha_i = A_1$

$\beta_i = A_3A_2 \mid A_3A_4$

- Let us consider a new symbol B. Now, we have to replace the production $A_1 \to A_1A_1$ of the grammar by using the following formulas.

$A \to \beta_i$, $A \to \beta_i B_1$, $B_1 \to \alpha_i$ and $B_1 \to \alpha_i\, B_1$

Therefore

$A_1 \to A_3A_2 \mid A_3A_4$

$A_1 \to A_3A_2B \mid A_3A_4B$

$B \to A_1$

$B \to A_1B$

$\therefore A_1 \to A_3A_2 \mid A_3A_4 \mid A_3A_2B \mid A_3A_4B$

$B_1 \to A_1$

$B_1 \to A_1B_1$

- Now, replace $A_1 \to A_1A_1$ in grammar (1) by value of A_1 in above grammar. Therefore the resultant grammar becomes

$A_1 \to A_3A_2 \mid A_3A_4 \mid A_3A_2B \mid A_3A_4B$

$A_1 \to A_2A_3 \mid A_4A_3$

$A_2 \to A_4A_1$

$A_3 \to 0$

$A_4 \to 1$

$B_1 \to A_1 \mid A_1B$

Combining A_1

$A_1 \to A_3A_2 \mid A_3A_4 \mid A_3A_2B \mid A_2A_3 \mid A_4A_3$

$A_2 \to A_4A_1$

$A_3 \to 0$

$A_4 \to 1$

$B_1 \to A_1 \mid A_1B_1$

- After removing duplicate we get

$$\left.\begin{array}{l} A_1 \to A_3A_2 \mid A_3A_4 \mid A_3A_2B \mid A_3A_4B \\ A_2 \to A_4A_1 \\ A_3 \to 0 \\ A_4 \to 1 \\ B_1 \to A_1 \mid A_1B_1 \end{array}\right\} \quad \ldots(2)$$

- Now writing production of grammar (2), we have

$A_1 \to A_3A_2$

$\therefore A_1 \to 0\,A_2$ *[$\because A_3 \to 0$)]*

Also

$A_1 \to A_3A_4$

$\therefore A_1 \to 0\,A_4$ *[$\because A_3 \to 0$]*

Also

$A_1 \to A_3A_2B_1$

$A_1 \to 0\,A_2B$ *[$\because A_3 \to 0$]*

Also

$A_1 \to A_3A_4B$

$A_1 \to 0\,A_4B$ *[$\because A_3 \to 0$]*

Also

$A_2 \to A_4A_1$

$A_2 \to 1\,A_1$ *[$\because A_4 \to 1$]*

$\therefore$ The resultant grammar becomes

$A_1 \to 0A_2 \mid 0A_4 \mid 0A_2B \mid 0A_4B$

$A_2 \to 1A_1$

$A_3 \to 0$ ....(3)

$A_4 \to 1$

$B \to A_1 \mid A_1B_1$

$\therefore$ From resultant grammar consider the production

$B_1 \to A_1$ and $B_1 \to A_1B_1$

(a) Consider the production

$B_1 \to A_1$

So by substituting value of A_1

$\therefore B_1 \to 0A_2 \mid 0A_4 \mid 0A_2B \mid 0A_4B$

(b) Consider the production

$B_1 \to A_1B_1$

Substitue the value of A_1 from grammar (3)

$B_1 \to 0A_2B_1 \mid 0A_4B_1 \mid 0A_2BB_1 \mid 0A_4BB_1$

$\therefore$ The resultant grammar becomes

$A_1 \to 0A_2 \mid 0A_4 \mid 0A_2B \mid 0A_4B$

$A_2 \to 1A_1$

$A_3 \to 0$

$A_4 \to 1$

$B_1 \to 0A_2 \mid 0A_4 \mid 0A_2B \mid 0A_4B \mid 0A_2B_1$
$\mid 0A_4B_1 \mid 0A_2BB_1 \mid 0A_4BB_1$

- Removing duplicate we get the resultant grammar in the form of GNF as follows.

$A_1 \to 0A_2 \mid 0A_4 \mid 0A_2B_1 \mid 0A_4B_1$

$A_2 \to 1\,A_1$

$A_3 \to 0$

$A_4 \to 1$

$B_1 \to 0A_2 \mid 0A_4 \mid 0A_2B_1 \mid 0A_4B_1$
$\mid 0A_2BB_1 \mid 0A_4BB_1$

Q.180. Convert the grammar with productions

$S \to AB$

$A \to aA \mid bB \mid b$

$B \to b$ in Greibach Nomal Form.

Ans. Given grammar

$S \to AB$ ....(1)

$A \to aA \mid bB \mid b$ ....(2)

$B \to b$ ....(3)

Substitute the value of A in equation (1)

$S \to (aA \mid bB \mid b)\,B$

$B \to b$

$\therefore S \to aAB \mid bBB \mid bB$

$B \to b$

$A \to aA \mid bB \mid b$

Thus the above grammar is in GNF in which every production is in the form of

$A \to a\alpha$

a = terminal

α = zero or more set of non terminal

Q.181. Convert the following grammar to GNF

$S \to ABA \mid AB \mid BA \mid AA \mid A \mid B$

$A \to aA \mid a$

$B \to bB \mid b$ **CT : W-08(6M)**

Ans. Given grammar

$S \to ABA \mid AB \mid BA \mid AA \mid A \mid B$ ....(1)

$A \to aA \mid a$ ....(2)

$B \to bB \mid b$ ....(3)

Put value of A in equation (1)

$S \to (aA \mid a)\, BA \mid (aA \mid a)\, B \mid BA \mid$

$(aA \mid a)\, A \mid (aA \mid a) \mid B$

$A \to aA \mid a$

$B \to bB \mid b$

$S \to aABA \mid aBA \mid aAB \mid aB$

$\mid BA \mid aAA \mid aA \mid aA \mid a \mid B$ ....(4)

$A \to aA \mid a$

$B \to bB \mid b$

Substitute the value of B in equation (4)

$S \to aABA \mid aBA \mid aAB \mid aB \mid$

$(bB \mid b)\, A \mid aAA \mid aA \mid a \mid bB \mid b$

$A \to aA \mid a$

$B \to bB \mid b$

Simplifying it we get

$S \to aABA \mid aBA \mid aAB \mid aB \mid bBA$

$\mid bA \mid aAA \mid aA \mid a \mid bB \mid b$

$A \to aA \mid a$

$B \to bB \mid a$

This is a grammar in GNF

Q.182. Find the grammar in GNF equivalent to the grammar :

$E \to E + T / T$

$T \to T * F / F$

$F \to (E) / a$ **CS : W-12(3M)**

Ans. Remove unit production,

$E \to T$ and $T \to F$

$\therefore E \to E + T / T$

$T \to T * F / (E) / a$

Now, $E \to E + T / T * F / (E) / a$

$T \to T * F / (E) / a$

$F \to (E) / a$

$E \to a$, $T \to a$, $F \to a$ } are already in GNF

$\therefore E \to a + T / a * F / (E) / a$

$T \to a * F / (E) / a$

$F \to (E) / a$

Let, $R_1 \to +$

$R_2 \to *$

$R_3 \to)$

$\therefore$ GNF :

$E \to a R_1 T / a R_2 F / (E R_3 / a$

$T \to a R_2 F / (E R_3 / a$

$F \to (E R_3 / a$

$R_1 \to +$

$R_2 \to *$

$R_3 \to)$

Q.183. Convert the following grammar into GNF

$S \to XY1 / 0$

$X \to 00X / Y$

$Y \to 1X1$ **CS : S-10(4M)**

Ans. $S \to XY1/0$

$X \to 00X /Y$

$Y \to 1X1$

Now,

Here, $S \to 0$ is already in GNF

Consider,

(1) $S \to XY1$

Let $S_1 \to 1$

$S \to XYS_1$

$S \to 00\ XYS_1$ *[$\because X \to 00X$]*

$S \to 0S\ XYS_1$ *[$\because S \to 0$]*

Also,

$S \to XYS_1$

$S \to 1\ X1\ YS_1$

$\boxed{S \to 1\ XS_1\ YS_1}$ *[$\because S_1 \to 1$]*

(2) $X \to 00X$

$X \to 0SX$ *[$\because S \to 0$]*

Also,

$X \to 0SX$

$X \to 0XY1X$ *[$\because S \to XY1$]*

$\boxed{X \to 0XYS_1X}$ *[$\because S_1 \to 1$]*

(3) $X \to 1\,X\,1$

$\boxed{X \to 1\,X\,S_1}$

(4) $Y \to 1\,X\,1$

$\boxed{Y \to 1\,X\,S_1}$

Required GNF :

$S \to 0S\,X\,YS_1 / 1\,X\,S_1\;YS_1 / 0$

$S_1 \to 1$

$X \to 0SX / 0\,XYS_1X / 1\,X\,S_1$

$Y \to 1\,X\;S_1$

Q.184 Convert grammar into GNF

$S \to 01 \mid 0S \mid 00S.$ **CS : S-10(3M)**

Ans. Assume,

$A \to 0$

$B \to 1$

$\therefore S \to 0B \mid 0S \mid 0AS$

$\therefore$ **GNF :**

$S \to 0B$

$S \to 0S$

$S \to 0AS$

$A \to 0$

$B \to 1$

$\therefore S \to 0B \mid 0S \mid 0AS$

$A \to 0$

$B \to 1$

Q.185 How long will be the derivation of w if $|w| = \ell$ and $w \in L(G)$ whereas G is in (i) CNF (ii) GNF ?

Ans.

(i) Now consider a production in P, of the form $A \to X_1\,X_2 \ldots\ldots X_m$, where $m \geq 2$. If X_i is a terminal a, introduce a new variable C_a and a production $C_a \to a$, which is of an allowable form.

- Then replace X_i by C_a. Let the new set of variables be V′ and the new set of productions be P′. Consider the grammar $G_2 = (V', T, P', S)$.
- If $\alpha \to \beta$, then $\alpha \to \beta$. Thus $L(G_1) \subseteq L(G_2)$.
- Now we show by induction on the number of steps in a derivation that if $A \to w$, for A in V and w in T^* then $A \to w$. The result is trivial for one-step derivations. Suppose that it is true for derivations of up to k steps. Let $A \to w$ be a $(k+1)$ - step derivation.
- The first step must be of the for m $A \to B_1\,B_2 \ldots B_m$ $m \geq 2$.
- We can write $w_1w_2\ldots\ldots w_m$ where $B_i \to w_i$, $1 \leq i \leq m$.
- If B_i is C_i, for some terminal a_i, then w_i must be a_i. By the construction of P′, there is a production $A \to X_1\,X_2 \ldots\ldots X_m$ of P where $X_i = B_i$ if B_i is in V and $X_i = a_i$ if B_i is in $V' - V$. For those B_i in V, we know that the derivation $B_i \to w_1$ takes no more than k steps, so by the inductive hypothesis $X_i = w_i$ Hence $A \to w$.
- We have now proved the intermediate result that any context free language can be generated by a grammar for which every production is either of the form $A \to a$ or $A \to B_1, B_2 \ldots\ldots B_m$, for $m \geq 2$. Here A and $B_1, B_2 \ldots\ldots B_m$ are variables, and a is a terminal.
- Consider such a grammar $G_2 = (V', T, P', S)$. We modify G_2 by adding some additional symbols to V′ and replacing some productions of P′. For each production $A \to B_1, B_2 \ldots\ldots B_m$ of P′, where $m \geq 3$, we create new variables $D_1, D_2, \ldots D_m$ and replace $A \to B_1, B_2 \ldots B_m$ by the set of productions.

$\{A \to B_1D_1, D_1 \to B_2D_2 \ldots, D_{m-3} \to B_{m-2}D_{m-2}D_{m-2} \to B_{m-1}B_m\}$

- Let V″ be the new nonterminal vocabulary and P″ the new set of productions.
- Let $G_3 = (V'', T, P'', S)$. G_3 is in CNF. It is clear that if $A \to \beta$ then $A \to \beta$, so $L(G_2) \subseteq L(G_3)$. But it is also true that $L(G_3) \subseteq L(G_2)$. as can be

shown is essentially the same manner as it was shown that $L(G_2) \subseteq L(G_1)$.

(ii) The length of derivation of w when G is in GNF is equal to length of derivation of w when G is a context free grammar.

Q.186. Obtained CNF and GNF grammar for the following

$E \rightarrow E + T / T$

$T \rightarrow a / (E)$ ***CT : S-07(8M)***

Ans. CNF :

Removing unit production,

$E \rightarrow E + T / a / (E)$

$T \rightarrow a / (E)$

Production :

(1) $E \rightarrow a$
(2) $T \rightarrow a$ } already in CNF

Consider :

$E \rightarrow E + T$

Let $R_1 \rightarrow +$ to CNF

$E \rightarrow T\ R_1\ T$

Let $R_2 \rightarrow ER_1$ to CNF

$E \rightarrow R_2\ T$

Consider :

$T \rightarrow (E)$

Let $R_3 \rightarrow ($ and $R_4 \rightarrow)$

$T \rightarrow R_3\ E\ R_4$

Let $R_5 \rightarrow R_3\ E$

$T \rightarrow R_5\ R_4$

For $E \rightarrow (E)$

$E \rightarrow R_5\ R_4$

$\therefore$ Grammar in CNF form is

(1) $E \rightarrow R_2\ T$ (2) $R_1 \Rightarrow T$
(3) $R_2 \rightarrow ER_1$ (4) $R_3 \rightarrow ($
(5) $R_4 \rightarrow)$ (6) $R_5 \rightarrow R_3\ E$
(7) $T \rightarrow R_5\ R_4$ (8) $E \rightarrow R_5\ R_4$
(9) $E \rightarrow a$ (10) $T \rightarrow a$

GNF : The productions are

$E \rightarrow a$
$T \rightarrow a$ } already in GNF

$E \rightarrow E + T / a / (E)$

$T \rightarrow a / (E)$

$E \rightarrow a + T / (E) + T / a / (E)$

Let $R_1 \rightarrow +$

$R_4 \rightarrow)$

GNF Grammar :

$E \rightarrow aR_1T / (ER_4\ R_1\ T / a / (E\ R_4$

$T \rightarrow a / (E\ R_4$

Q.187. Convert the grammar :

$S \rightarrow ABb \mid a$

$A \rightarrow aaA \mid B$

$B \rightarrow bAb$ into GNF and CNF ***CS : S-12(4M)***

Ans. CNF :

$S \rightarrow a$ already in CNF

Let $B \rightarrow b$

$S \rightarrow ABB \mid a$

$A \rightarrow SSA \mid b$

$B \rightarrow BAB \mid b$

Now,

$R_1 \rightarrow BB$

and $R_2 \rightarrow SA$

Grammar in CNF is :

$S \rightarrow A\ R_1 \mid a$

$A \rightarrow S\ R_2 \mid b$

$B \rightarrow B\ R_1 \mid b$

$R_1 \rightarrow AB$

$R_2 \rightarrow SA$

GNF :

$S \rightarrow a$
$A \rightarrow b$ } already in GNF
$B \rightarrow b$

Grammar in the GNF is :

$\therefore S \rightarrow b\ R_1 \mid a$

$A \rightarrow a\ R_2 \mid b$

$B \rightarrow b\ R_1 \mid b$

$R_1 \rightarrow bB$

$R_2 \rightarrow aA$

POINTS TO REMEMBER :

(1) Regular expression is a notation to specify regular set.

(2) There are three basic regular expressions, $\phi, \in, a$, for every a in Σ, specifying, $\{\}$, $\{\in\}$, $\{a\}$ respectively.

(3) There are three operators + or |, ., and *, for union, concatenation and closure operations respectively.

(4) Pumping lemma is a theorem, which tells us about a characteristic property of regular sets, that if L is regular set, then for every sufficiently longer string in L, there exist a substring near its beginning that can be pumped or repeated as many times as we want the resulting strings will be in L.

(5) Pumping lemma is used to prove that certain languages are non-regular.

(6) Every finite set is a regular set.

(7) Every infinite set need not necessarily be regular.

(8) A context-free grammar is 4-tuple denoted as :

$G = (V, T, P, S)$ where :

(i) V is a finite set of symbols called as non-terminals or variables.

(ii) T is a set a symbols called as terminals.

(iii) P is a set of productions.

(iv) S is a member of V, called as start symbol.

(9) The class of languages generated by context-free grammars is called as context-free languages.

(10) Context-free grammar is used to specify syntactic structure of programming languages.

(11) A CFG G is said to be ambiguous if there exists a w in L(G), for which more than one parse trees can be generated.

(12) Ambiguity of a CFG G, does not necessarily imply the ambiguity of language generated by G, (i.e., L(G)).

(13) An inherently ambiguous CFL is the one for which there exists no unambiguous CFG.

(14) A grammar symbol is useless if and only if either it does not derive to any string of terminals, or it is not used in the derivation of any w in L(G).

(15) Elimination of useless grammar symbols, gives a minimal grammar specifying a language.

(16) Every CFL not containing $\in$ can be specified by a CFG, which does not contain $\in$-production.

(17) An unit production is a production of the form $A \rightarrow B$, where A and B both are non-terminals.

(18) Presence of unit productions in a grammar increases the cost of deriving w in L(G) from the start symbol S of the grammar.

UNIT - IV
PUSHDOWN AUTOMATA

SYLLABUS

Push down Automata (PDA), non-determinism, acceptance by two methods and their equivalence, conversion of PDA to CFG, CFG to PDAs, closure and decision properties of CFLs, pumping lemma for CFL.

UNIVERSITY PAPER SOLUTIONS
SINCE SUMMER - 2006

Note to the students :
- The questions given below are from previous Nagpur University examination papers.
- Though these questions are from old university papers, they have been included in the new syllabus.
- Questions which are out of new syllabus are not included here.

SUMMER - 06 (CT)

Q.1. Construct Push down Automata for the language.

$L = \{0^{m+n}1^m0^n \mid m,n \geq 0\}$ **6M**

Ans. P.4-14, Q.19.

Q.2. Construct PDA for accepting language **5M**

$L = \{WW^R \mid W \in (011)^*\}$

Ans. P.4-16, Q.23.

Q.3. Prove that using pumping lemma, the language L is not context free language, $L = \{a^i b^i c^i \mid i \geq 1\}$ **7M**

Ans. P.4-30, Q.42.

WINTER - 06 (CT)

Q.1. Design a pushdown automata for accepting the languages.

(i) $L_1 = \{0^n 1^m 0^{m+n} \mid m, n \geq 1\}$

(ii) $L_2 = \{b^n \subset a^n \mid n \geq 1\}$ **8M**

Ans. P.4-11 & 14, Q.14 & 20.

Q.2. Explain closure properties of context free language. **6M**

Ans. P.4-26, Q.40.

SUMMER - 07 (CT)

Q.1. Construct a PDA A equivalent to the following context- free grammar :

$S \rightarrow 0BB$

$B \rightarrow 0S \mid 1S \mid 0$

Test whether " 010000" is in N (A). **5M**

Ans. P.4-26, Q.37.

Q.2. Explain the closure properties of context free languages. **5M**

Ans. P.4-26, Q.40.

WINTER - 07 (CT)

Q.1. Construct a non-deterministic push down automata for the following language

$L = \{WCW^R : W \in (a, b)^*\}$ **8M**

Ans. P.4-15, Q.22.

SUMMER - 08 (CT)

Q.1. Design a PDA to accept the language

$L = \{W C W^R \mid W \in \{a, b\}^*\}$ **8M**

Ans. P.4-15, Q.22.

Q.2. What is Push Down Automata? Explain the model of push down automata. Give its applications also. **5M**

Ans. P.4-4, 5 & 6, Q.1, 2 & 4.

WINTER - 08 (CT)

Q.1. Design PDA for the language : **13M**

$L = \{a^n b^m c^{|n-m|} \mid m, n >= 0\}$.

Ans. P.4-9, Q.13.

SUMMER - 09 (CS)

Q.1. Design PDA for

$L = \{W\, CW^R \mid W \in (a,b)^*\}$

W^R = Reverse of W.

Also explain the stack execution with valid string. **6M**

Ans. P.4-15, Q.22.

Q.2. Deign PDA to accept all string over

$L = \{a^n b^m c^{|m-n|} \mid m, n >= 1\}$. Explain all cases. **8M**

Ans. P.4-9, Q.13.

SUMMER - 09 (CT)

Q.1. Design PDA for the language given below. Give the algorithm and transition diagram for the same

$L = \{a^n\, b^{2n} \mid n \geq 1\}$ **7M**

Ans. P.4-9, Q.12.

WINTER - 09 (CS)

Q.1. Design PDA for : $L = \{a^n\, b^m c^{|m-n|} \mid m, n >= 1\}$

Give solution for various possibilities of 'L'. **8M**

Ans. P.4-9, Q.13.

Q.2. Design PDA to accept following context free grammar.

$S \rightarrow XY$

$X \rightarrow AX \mid BX \mid a$

$Y \rightarrow YA \mid YB \mid a$

$A \rightarrow a$

$B \rightarrow a$ **6M**

Ans. P.4-25, Q.36.

WINTER - 09 (CT)

Q.1. Explain model of PDA and justify the statement PDA is more powerful than FA. **6M**

Ans. P.4-5 & 6, Q.2 & 5.

Q2. Construct a PDA for the language :

$L = \{a^m\, b^m\, c^n \mid m, n \geq 1\}$. **7M**

Ans. P.4-12, Q.15.

Q.3. Construct PDA for the following language :

$L = \{a^n\, b^{2n} \mid n > 0\}$ **7M**

Ans. P.4-9, Q.12.

SUMMER - 10 (CS)

Q.1. Construct a PDA accepting the set of all strings over {a, b} with equal number of a's and b's. **6M**

Ans. P.4-8, Q.10.

Q.2. Construct a PDA A equivalent to the following context free grammar. $S \rightarrow 0BB$, $B \rightarrow 0S \mid 1S \mid 0$. Test whether 010^4 is in N (A). **7M**

Ans. P.4-26, Q.37.

SUMMER - 10 (CT)

Q.1. Construct PDA for the language **7M**

$L = \{a^n b^{2n} \mid n \geq 1\}$

Ans. P.4-9, Q.12.

Q.2. Construct a PDA for the CFG and show the acceptance / rejection of string w = a a b a a

$S \rightarrow aSa \mid b\,S\,b \mid a \mid b$ **7M**

Ans. P.4-25, Q.35.

WINTER - 10 (CS)

Q.1. Explain acceptance of string by PDA in terms of final state and in terms of empty push down stack. **6M**

Ans. P.4-18, Q.28.

Q.2. Construct a PDA equivalent to the following CFG

$S \rightarrow 0BB$, $B \rightarrow 0S|1S|\,0$. Test whether 010^4 is in N(A). **5M**

Ans. P.4-26, Q.37.

Q.3. Construct a context free grammar G which accepts following PDA given by.

$A = \{(q_0, q_1), (a, b), (Z_0, Z), \delta, q_0, Z_0\}$ and δ given by

$\delta(q_0, b, Z_0) = (q_0, Z_0, Z_0)$

$\delta(q_0, \in, Z_0) = (q_0, \in)$

$\delta(q_0, b, Z) = (q_0, Z, Z_0)$

$\delta(q_0, a, Z) = (q_1, Z)$

$\delta(q_1, b, Z) = (q_1, \in)$

$\delta(q_1, a, Z_0) = (q_1, Z_0)$ **8M**

Ans. P.4-20, Q.31.

WINTER - 10 (CT)

Q.1. Design PDA for accepting the language.

$L = \{WW^R \mid W \in \{a, b\}^*\}$ **7M**

Ans. P.4-15, Q.23.

SUMMER - 11 (CS)

Q.1. Design PDA for the language. **7M**

$L = \{W \mid W \in (a, b)^* \text{ and } n_a(W) > n_b(W)\}$

Ans. P.4-13, Q.17.

SUMMER - 11 (CT)

Q.1. Construct a PDA accepting the language :

$L = \{a^m c^{m-n} b^n \mid m, n \geq 1\}$ by empty stack. **8M**

Ans. P.4-9, Q.13.

Q.2. Give the applications of PDA. **2M**

Ans. P.4-6, Q.4.

Q.3. Comment on the following statement :

" Non deterministic push down automata is more powerful than deterministic push down automata." **3M**

Ans. P.4-7, Q.8.

WINTER - 11 (CS)

Q.1. Construct a PDA accepting the following language $L = \{a^n b^m c^m d^n \mid m, n \geq 1\}$. **7M**

Ans. P.4-17, Q.25.

Q.2. Comment on the following statements :

(i) "Non-deterministic PDA is more powerful than deterministic PDA".

(ii) "The class of language accepted by PDA's are closed under complementation". **6M**

Ans. P.4-7, Q.8 & 9.

WINTER - 11 (CT)

Q.1. Construct a PDA accepting $\alpha = \{a^n b^m a^n \mid m, n \geq 1\}$ by null store. Construct corresponding CFG accepting the same set. **8M**

Ans. P.4-17, Q.27.

Q.2. Construct a PDA for the language

$\lambda = \{a^{2n} b^n \mid n \geq 1\}$ **5M**

Ans. P.4-17, Q.26.

SUMMER - 12 (CS)

Q.1. Design PDA for $\{WW^R / W \text{ in } (0+1)^*\}$ using empty stack acceptance. Also show all accessible ID's for the string "001100". **8M**

Ans. P.4-16, Q.23.

Q.2. Give grammar for language defined by following PDA.

$M = (\{q_0, q_1\}, \{0, 1\}, \{Z_0, X\}, \delta, q_0, Z_0, \phi)$ and δ is given by

$\delta(q_0, 1, Z_0) = \{(q_0, XZ_0)\}, \delta(q_0, \in, Z_0) = \{q_0, \in\}$

$\delta(q_0, 1, X) = \{(q_0, XX)\}, \delta(q_1, 1, X) = \{q_1, \in\}$

$\delta(q_0, 0, X) = \{(q_1, X)\}, \delta(q_1, 0, Z_0) = \{q_0, Z_0\}$ **6M**

Ans. P.4-21, Q.32.

SUMMER - 12 (CT)

Q.1. Design PDA for accepting the language

$L = \{WW^R \mid W \in \{a, b\}^*\}$ **7M**

Ans. P.4-15, Q.23.

WINTER - 12 (CS)

Q.1. Construct a deterministic PDA accepting

$L = \{W \in \{a, b\}^* \mid$ the number of a's in W equals the number of b's in W$\}$ by final state. **7M**

Ans. P.4-16, Q.24.

Q.2. Construct a context free grammar G that accepts the set accepted by PDA.

$M = (\{q_0, q_1, \{a, b\}, \{A, Z\}, \delta, q_0, Z, \{q_1\})$

$\delta(q_0, a, Z) = \{(q_0, AZ)\}$

$\delta(q_0, b, A) = \{(q_0, AA)\}$

$\delta(q_0, a, A) = \{(q_1, \in)\}$ **7M**

Ans. P.4-20, Q.31.

Q.3. Explain the closure properties of context free language. **6M**

Ans. P.4-26, Q.40.

WINTER - 12 (CT)

Q.1. Construct the Push-Down Automata for the given language

$L = \{a^n b^{n+m} c^m / n, m \geq 0\}$. **8M**

Ans. P.4-12, Q.16.

SUMMER - 13 (CS)

Q.1. Design push down automata for the language $L = \{WW^R / W \text{ in } (0+1)^*\}$ using empty stack acceptance. Convert this PDA into PDA using acceptance by final state. **8M**

Ans. P.4-16, Q.23.

Q.2. Convert the PDA.

$P = (\{p, q\}, \{0, 1\}, \{x, z\}, \delta, q, z)$

to a CFG if δ is given by :

1. $\delta\ (q, 1, z) = \{(q, xz)\}$
2. $\delta\ (q, 1, x) = \{(q, xx)\}$
3. $\delta\ (q, 0, x) = \{(p, x)\}$
4. $\delta\ (q, \in, x) = \{(q, \in)\}$
5. $\delta\ (p, 1, x) = \{(p, \in)\}$
6. $\delta\ (p, 0, z) = \{(q, z)\}$ **8M**

Ans. P.4-21, Q.32.

SUMMER - 13 (CT)

Q.1. Construct pushdown automata for the language $L = \{0^n 1^{2n} \mid n \geq 1\}$ **9M**

Ans. P.4-9, Q.12.

Q.2. Explain method to get PDA with final state from PDA accepting by empty stack. **5M**

Ans. P.4-18, Q.28.

SOLVED QUESTION BANK

[Sequence given as per syllabus]

INTRODUCTION

The pushdown automaton is essentially a finite automaton with control of both an input tape and a stack, or "first in last out" list. That is, symbols may be entered or removed only at the top of the list. When a symbol is entered at the top, the symbol previously at the top becomes second from the top, the symbol previously second from the top becomes third, and so on. Similarly, when a symbol is removed from the top of the list, the symbol previously second from the top becomes the top symbol, the symbol previously third from the top becomes second, and so on.

PUSH-DOWN AUTOMATA

Q.1. Give the analytical definition of pushdown automata.

OR What is Push Down Automata? ***CT : S-08(1M)***

OR State types of Push Down Automata.

Ans. Pushdown Automata :

- The pushdown automata is the machine counter-part of context free grammars.
- Pushdown automaton is a nondeterministic device and the deterministic version accepts only a subset of all CFL's.
- The pushdown automata is essentially a finite automaton with control of both an input tape and a stack, or "first in last out list". That is, symbols may be entered or removed only at the top of the list.
- The first element inserted is the last one to be removed.
- The PDA will have an input tape, a finite control and a stack.
- The stack is a string of symbols from some alphabet.

Formal Defination :

PDA consist of 7 tuple

$M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$

where,

$Q \rightarrow$ set of finite states

$\Sigma \rightarrow$ set of input symbols

$\Gamma \rightarrow$ finite set of stack symbols

$\delta \rightarrow$ Transition mapping function

$Z_0 \rightarrow$ Initial stack symbol

$q_0 \rightarrow$ Initial state

$F \rightarrow$ Set of final state

δ (S, input, stack symbol) $\rightarrow$ (S, Replaced symbol and top of stack)

Types of Push Down Automata :

(i) Non deterministic pushdown automata (NPDA)

(ii) Deterministic pushdown automata (DPA)

Q.2. Explain the model of push down automata.

CT : W-09(3M), S-08(2M)

Ans. Model of Pushdown Automata :

A push down automata is a finite state machine that is equipped with a memory device that functions as a push-down store.

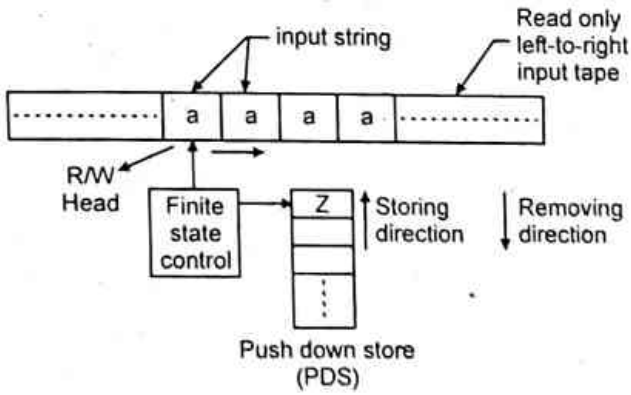


Fig. Model of PDA

PDA consist of four components :

(1) An input tape

(2) A read/write head

(3) A control unit

(4) A memory unit or stack

(1) An input tape :

The input tape consists of a linear configuration of cells each of which contains an inputs symbol from an alphabet. This tape can be moved one cell at a time to the right.

(2) A read/write head :

The head reads only one input symbol at a time and always moves in right direction. The head positioned over the current stack element can read and write special stack characters from that position.

(3) A control unit or finite state control :

A control unit contains both tape heads and finds itself at any moment in a particular state. The transitions executed by the control unit of a push down automaton involve operations to state symbols in, and retrieve symbols from its stack.

(4) A memory unit or stack :

The stack is a sequential structure that has a first element and grows in either direction from the other end. The current stack element is always the top element of the stack. This stack also operates as push down store.

Q.3. What do you mean by DCFL and DPDA? Explain.

Ans. Deterministic Push Down Automata (DPDA) :

- Let $M = (Q, \Sigma, \Gamma, q_0, Z_0, F, \delta)$ be a pushdown automata. M is deterministic if there is no configuration for which M has a choice of more than one move.
- In other words, M is deterministic if it satisfies both the following conditions.

1. For any $q \in Q, a \in \Sigma \cup \{\epsilon\}$ and $X \in \Gamma$, the set $\delta(q, a, X)$ has at most one element.
2. For any $q \in Q$, and $X \in \Gamma$, if $\delta(q, \in, X) \neq f$, then, $\delta(q, a, x) = f$ for every $a \in \Sigma$.

- The first of these conditions simply requires that for any given input symbol and any stack top at most one move can be made.
- The second condition is that when a ϵ-move is possible for some configuration, no input-consuming alternative is available.

Example : The language $L = \{a^n b^n \mid n \geq 0\}$ is a deterministic context-free language. The PDA $M = (\{q_0, q_1, q_2\}, \{a, b\}, \{0, 1\}, \delta, q_0, 0, \{q_0\})$ with

$\delta(q_0, a, 0) = \{(q_1, 10)\}$,

$\delta(q_1, a, 1) = \{(q_1, 11)\}$,

$\delta(q_1, b, 1) = \{(q_2, \Lambda)\}$

$\delta(q_2, b, 1) = \{(q_2, \Lambda)\}$

$\delta(q_2, \Lambda, 0) = \{(q_0, \Lambda)\}$ accepts that language. It satisfied the conditions and is therefore deterministic.

Deterministic context free language (DCFL) :

- A language L is a deterministic context free language (DCFL) if there is a deterministic PDA (DPDA) accepting L.

Example : $L = \{a^n b^n \mid n \geq 0\}$ is DCFL.

Q.4. Give application of PDA. ***CT : S-08,11(2M)***

Ans. Applications of PDA : PDA is used to construct two types of parser.

(1) **Top-Down Parser :** In top-down parser, virtual reconstruction of the derivation tree starts from the starting non-terminal of the respective context-free grammar.

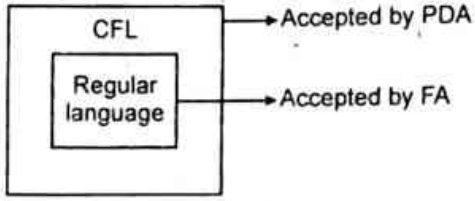
(2) **Bottom - Up Parser :** In bottom-up parser reconstruction of the derivation tree starts from the symbols of the input sentence up to the starting non-terminal of the respective grammar.

Q.5. Justify the statement 'PDA is more powerful than FA'. ***CT : W-09(3M)***

Ans.

- The language accepted by FA is called as regular language.
- But FA can not accept all context free grammar because FA have limited memory where as the recognition of CFL requires storing an unlimited amount of information.

  Example : The language $L = \{a^n b^n : n > 0\}$ is not accepted by FA because in this language we not only check that all a's precede the first b but we must count the equal number of a and b. Since n is unlimited so this counting can not be done with FA which have finite memory.
- But above language L is accepted by PDA can also accept CFL and regular language.
- Also PDA has potentially unlimited amount of memory in the form of stack.
- In this stack we add the elements in a linear way while removing the elements we follow the LIFO basis i.e. most recently added element is removed first.
- Hence we can say that FA accept only regular language but PDA can accept regular language as well as CFL with unlimited amount of memory. Hence PDA is more powerful than FA.



NON-DETERMINISM

Q.6. Explain Nondeterministic Pushdown Automata in detail.

Ans. Non-Deterministic PDA (NPDA) :

- In NPDA, for a particular input we have two or more transitions to go to next state.
- It means NPDA is one for which at certain time we may have to select a particular path among possible paths.
- The Non-deterministic Pushdown Automata is a new class of automata, more "powerful" than the finite state machines, that can be associated with the context-free languages.
- This automaton is like a finite automaton but is augmented by an auxiliary memory that is structured as a stack.
- The stack can be used as a storage mechanism that can contain unbounded number of items.
- Items can be pushed onto and poped from the stack according to Last In First Out (LIFO) policy. A schematic representation of NPDA is given in Fig. below.

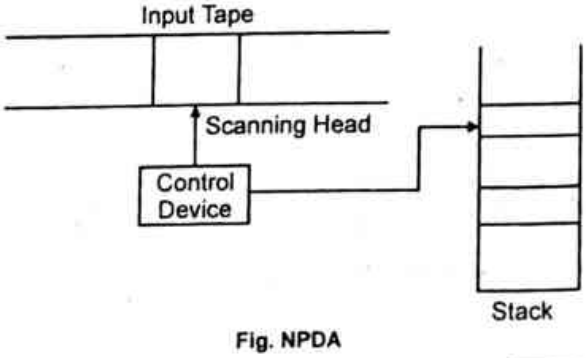


Fig. NPDA

Working of NPDA :

- As in finite-state machines, during the evolution of the NPDA the scanning head reads the input tape from left to right, the control unit is in one of the states and the stack contains stack symbols.
- Each move of NPDA is a function of scanned symbol 'a', the present-state, q and the symbol on the top of the stack, 'A', that is, of (q, a, A), which is called a instantaneous description (ID) of a pushdown automaton.
- It consists of three steps :

(i) Scanning head moves to right.
(ii) Control unit commuting to a new state, q and
(iii) Change in the top of the stack with a (possibly empty) string of stack symbols.

- In NPDA, we may also perform a move without scanning any symbol, that is, in transition of (q, a, A) we may have $a = \lambda$, empty string.
- An NPDA consists of 7 tuples.
  $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$
  where,
  $Q \rightarrow$ finite set of states
  $\Sigma \rightarrow$ finite set of input symbols
  $\Gamma \rightarrow$ finite set of stack symbols
  $q_0 \rightarrow$ Initial state
  $Z_0 \rightarrow$ Initial symbol in stack
  $F \rightarrow$ Set of final state, $F \subseteq Q$
  $\delta \rightarrow$ Transition function mapping from $Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow Q \times \Gamma^*$.
- NPDA are equivalent to CFG and more powerful than deterministic one (DPDA)

Q.7. Differentiate between Deterministic PDA and Non-deterministic PDA.

Ans.

Sr. No.	Deterministic PDA	Non-Deterministic PDA
1.	A deterministic PDA is one for which every input string has a unique path through the machine.	A non-deterministic PDA is one in which for particular input we have two or more transitions to go to next state.
2.	DPDA is less powerful than NPDA.	NPDA is more powerful than DPDA.
3.	DPDA can not be converted to NPDA.	There is no way to translate NPDA into DPDA.
4.	A push-down automaton M= $(Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ is deterministic if 1. δ (q, a, z) is either empty or single move.	A push-down automaton. M = $(Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ is non-deterministic if 1. δ (q, a, z) has more than one moves.

Q.8. Comment "NPDA is more powerful than DPDA". **CS : W-11(3M) CT : S-11(3M)**

Ans. NPDA is more powerful than DPDA because of following reasons :

- Some of the CFL's which cannot be recognized by DPDA, can be accepted by the NPDA.
- The class of languages accepted by nondeterministic PDA is larger than that of the class of languages accepted by DPDA.

Q.9. Comment 'The Class of languages accepted by pushdown automata are closed under complementation'. **CS : W-11(3M)**

Ans.

- Consider any deterministic pushdown automaton M.
- By lemma it can be assumed that M has only halting computations, and with no loss of generality it can be assumed that |g| < = 1 in each transition rule (q, a, b, p, g).

From M to Meof :

- Let Meof be a deterministic pushdown automaton that accepts L(M), and that in each of its computations halts after consuming all the input.
- Meof can be constructed from M in the following manner. Let Meof be M initially with an added trap state qtrap, and added transition rule of the form (qtrap, a, e, qtrap, e) for each input symbol a. Then repeatedly add to Meof a new transition rule of the form (q, a, b, qtrap, b), as long as Meof

does not have a next move from state q on input a and topmost pushdown content b.

Elimination of Mixed States :

- Call a state q of a pushdown automaton a reading state, if a is an input symbol in each transition rule (q, a, b, p, g) that originates at state q. Call the state q and e state, if a = e in each transition rule (q, a, b, p, g) that originates at state q. If the state q is neither a reading state nor an e state then call it a mixed state.
- If q is a mixed state of Meof, then each of the transition rules (q, a, b, p, g) of M that satisfies |a| = 1, can be replaced by a pair transition rules (q, e, b, qb, e) and (qb, a, e, p, g), where qb is a new intermediate, nonaccepting state. Using such transformations Meof can be modified to include no mixed states.

ACCEPTANCE BY TWO METHODS AND THEIR EQUIVALENCE

Q.10.Construct a PDA for following language.

$L = \{a^n b^n \mid n > 0\}$

OR Construct a PDA accepting a set of all string over {a, b} with equal number of a's and b's.

CS : S-10(6M)

Ans. Step 1 :

(1) Let n = 1

$L_1 = \{ab\}$

(2) Let n = 2

$L_2 = \{aa\ bb\}$

Let n = 3

(3) $L_3 = \{aaa\ bbb\}$

Valid : {ab, aabb, aaabbb, ...}

Invalid : {a, b, aa, ba, ...}

Analysis :

1. String has fixed length.
2. String start with 'a' and end with 'b'.

Step 2 :

Algorithm :

1. Push '1' for input symbol 'a' and POP '1' for input symbol 'b'.
2. Replace z_0 by $\in$

Transition :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, 1\,Z_0)$

loop 2. $\delta(q_0, a, 1) \rightarrow (q_0, 11)$

3. $\delta(q_0, b, 1) \rightarrow (q_1, \in)$

loop 4. $\delta(q_1, b, 1) \rightarrow (q_1, \in)$

5. $\delta(q_1, \in, Z_0) \rightarrow (q_2, \in)$

Transition Diagram :

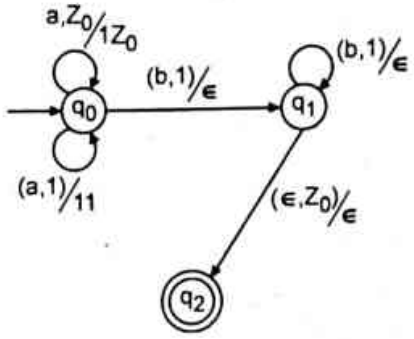


Fig. Transition diagram

Q.11. Construct PDA for the language.

$L = \{a^p b^q c^r \mid p, q, r \geq 1 \text{ \& } q = p + r\}$

Ans. Step 1 :

1. Let p, r = 1 $\quad \therefore q = 1 + 1 = 2$

   L_1 {abbc}

2. Let p, r = 2 $\quad \therefore q = 2 + 2 = 4$

   L_2 = {aa bbbb cc}

3. Let p, r = 3 $\quad \therefore q = 3 + 3 = 6$

   L_3 = {aaa bbbbbb ccc}

4. Let p = 2, r = 3 $\quad \therefore q = 2 + 3 = 5$

   L_4 = {aa bbbbb ccc}

Valid : {abbc, aabbbbcc, aaabbbbbbccc.....}

Invalid : {a, b, c, ab .....}

Analysis :

1. Length of string is fixed.
2. String start with a and end with c.
3. The total number of b are addition of total number of a's and c's.

Step 2 :

Algorithm :

1. Push '1' for input symbol 'a' and POP '1' for input symbol 'b'.
2. When stack comes on its initial state again and the current input symbol is 'b' then push '1' for

input symbol 'b' and POP '1' for input symbol 'c'.

3. Replace z_0 by $\in$.

Transition :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, 1Z_0)$

loop 2. $\delta(q_0, a, 1) \rightarrow (q_0, 11)$

3. $\delta(q_0, b, 1) \rightarrow (q_1, \in)$

loop 4. $\delta(q_1, b, 1) \rightarrow (q_1, \in)$

5. $\delta(q_1, b, Z_0) \rightarrow (q_2, 1Z_0)$

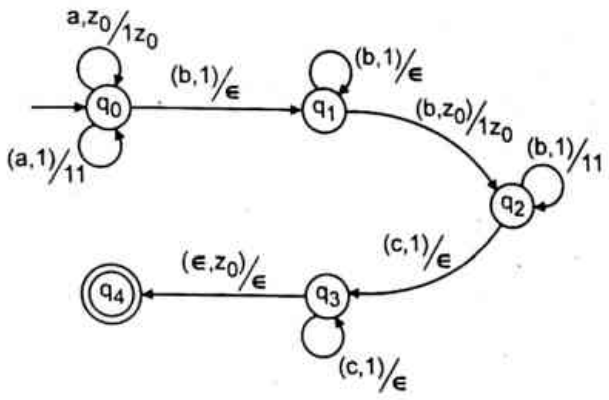
loop 6. $\delta(q_2, b, 1) \rightarrow (q_2, 11)$

7. $\delta(q_2, c, 1) \rightarrow (q_3, \in)$

loop 8. $\delta(q_3, c, 1) \rightarrow (q_3, \in)$

9. $\delta(q_3, \in, Z_0) \rightarrow (q_4, \in)$

Transition Diagram :



Q.12.Construct PDA for the language.

$L = \{a^n\ b^{2n} \mid n > 0\}$ **CT : W-09(7M)**

OR Desing PDA for a language given below. Give the algorithm and transition diagram for the same.

$L = \{a^n\ b^{2n} \mid n \geq 1\}$ **CT : S-09,10(7M)**

OR Construct PDA for accepting language :

$L = \{0^n\ 1^{2n} \mid n \geq 1\}$ **CT : S-13(9M)**

Ans. Step 1 :

1. Let n = 1

   $L_1 = \{abb\}$

2. Let n = 2

   $L_2 = \{aabbbb\}$

3. Let n = 3

   $L_3 = \{aaabbbbbb\}$

Valid : {abb, aabbbb, aaabbbbbb,.....}

Invalid : {a, b, ab, ba, aabb,.....}

Analysis :

1. Length of string is fixed.
2. String start with a and end with b.
3. The number of b's are twice than number a's.

Step 2 :

Algorithm :

1. Push two, 1's for input symbol a and POP '1' for input symbol 'b'.
2. Replace Z_0 by $\in$.

Transition :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, 1Z_0)$

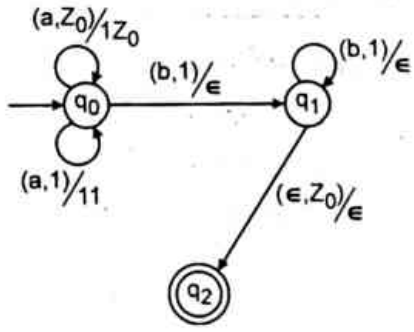
loop 2. $\delta(q_0, a, 1) \rightarrow (q_0, 11)$

3. $\delta(q_0, b, 1) \rightarrow (q_1, \in)$

loop 4. $\delta(q_1, b, 1) \rightarrow (q_1, \in)$

5. $\delta(q_1, \in, Z_0) \rightarrow (q_2, \in)$

Transition Diagram :



Q.13. Construct PDA for the language.

$L = \{a^n\ b^m c^{|n-m|} \mid n, m > 0\}$

CT : W-08(13M)

OR Construct PDA accepting the language :

$L = \{a^m\ c^{m-n} b^n \mid m, n \geq 1\}$ **by empty stack.**

CT : S-11(8M)

OR Design PDA for language :

$L = \{a^n\ b^m c^{|m-n|} \mid m, n > 1\}$ **Give solution for various possibilities of 'L'.**

CS : S-09(7M),W-09(8M)

Ans. Step 1 :

Case 1 :

n > m

1. Let n = 2 , m = 1

L_1 = {aabc}

2. Let n = 3, m = 2

L_2 = {aaabbc}

3. Let n = 4, m = 3

L_3 = {aaaabbbc}

L = {aabc, aaabbc, aaaabbbc, ...}

Case 2 :

m > n

1. Let n = 1, m = 2

L_1 = {a bb c}

2. Let n = 2, m = 3

L_2 = {aa bbb c}

∴ L = {abbc, aabbbc, ...}

Case 3 :

n = m

L = {ab, aabb, aaabbb .....}

Case 4 :

n = 0, m = ?

L = {bc, bbcc, bbbccc, ...}

Case 5 :

m = 0, n = ?

Let {ac, aacc, aaaccc, ...}

Case 6 :

n = 0 = m

L = {∈}

Step 2 :

Algorithm :

Case 1 :

1. Push '1' for 'a' and POP '1' for 'b' and 'c'
2. Replace Z_0 by ∈.

Case 2 :

1. Push '1' for input symbol 'a' and POP for input symbol 'b'.
2. When stack comes on its initial position or state again and the current input symbol 'b' than push '1' for input symbol 'b' and POP '1' for input symbol, 'c'
3. Replace caps by ∈.

Case 3 :

1. Push '1' for input symbol 'a' and POP '1' for input symbol 'b'.
2. Replace z_0 by ∈.

Case 4 :

1. Push '1' for input symbol 'b' and POP '1' for input symbol 'c'.
2. Replace Z_0 by ∈.

Case 5 :

1. Push '1' for input symbol 'a' and POP '1' for input symbol 'c'.
2. Replace Z_0 by ∈.

Case 6 :

Replace Z_0 by ∈.

Transition :

For case 1 :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, 1\,Z_0)$

loop 2. $\delta(q_0, a, 1) \rightarrow (q_0, 11)$

3. $\delta(q_0, b, 1) \rightarrow (q_1, \in)$

loop 4. $\delta(q_1, b, 1) \rightarrow (q_1, \in)$

5. $\delta(q_1, c, 1) \rightarrow (q_2, \in)$

loop 6. $\delta(q_2, c, 1) \rightarrow (q_2 \in)$

7. $\delta(q_2, \in, Z_0) \rightarrow (q_f, \in)$

Transition :

For case 2 :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, 1\,Z_0)$

loop 2. $\delta(q_0, a, 1) \rightarrow (q_0, 11)$

3. $\delta(q_0, b, 1) \rightarrow (q_1, \in)$

loop 4. $\delta(q_1, b, 1) \rightarrow (q_1, \in)$

5. $\delta(q_1, c, 1) \rightarrow (q_2, \in)$

loop 6. $\delta(q_2, c, 1) \rightarrow (q_2 \in)$

7. $\delta(q_2, \in, Z_0) \rightarrow (q_f, \in)$

Transition :

For case 3 :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, 1\,Z_0)$

loop 2. $\delta(q_0, a, 1) \rightarrow (q_0, 11)$

3. $\delta(q_0, b, 1) \rightarrow (q_1, \in)$

loop 4. $\delta(q_1, b, 1) \rightarrow (q_1, \in)$

5. $\delta(q_1, \in, Z_0) \rightarrow (q_2, \in)$

Transition :

For case 4 :

1. $\delta(q_0, b, Z_0) \rightarrow (q_0, 1\,Z_0)$

loop 2. $\delta(q_0, b, 1) \rightarrow (q_0, 11)$

3. $\delta(q_0, c, 1) \rightarrow (q_1, \in)$

loop 4. $\delta(q_1, c, 1) \rightarrow (q_1, \in)$

5. $\delta(q_1, \in, Z_0) \rightarrow (q_2, \in)$

Transition :

For case 5 :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, 1\,Z_0)$

loop 2. $\delta(q_0, a, 1) \rightarrow (q_0, 11)$

3. $\delta(q_0, c, 1) \rightarrow (q_1, \in)$

loop 4. $\delta(q_1, c, 1) \rightarrow (q_1, \in)$

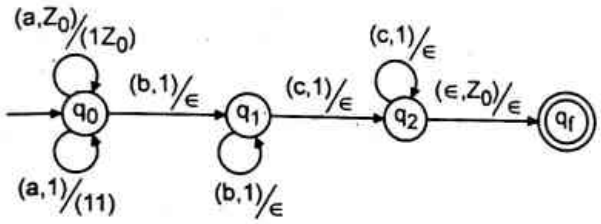
5. $\delta(q_1, \in, Z_0) \rightarrow (q_2, \in)$

For case 6 :

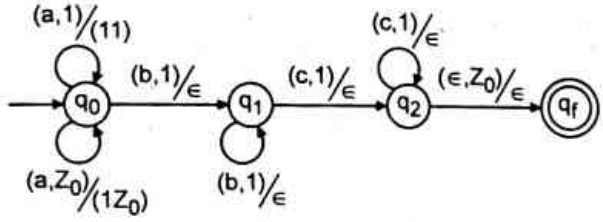
1. $\delta(q_0, \in, Z_0) \rightarrow (q_1, \in)$

Transition Diagram :

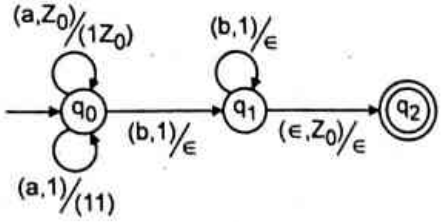
Case : 1



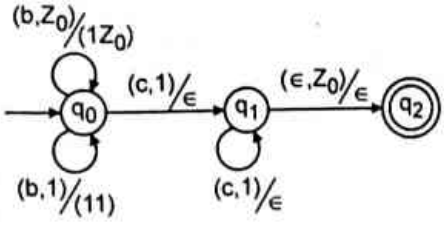
Case : 2



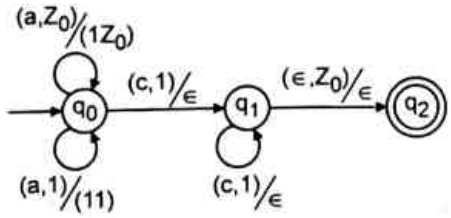
Case : 3



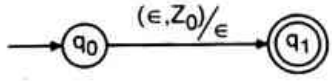
Case : 4



Case : 5



Case : 6



Q.14. Design a pushdown automata for accepting the following languages :

(i) $L = \{0^n\ 1^m\ 0^{m+n} \mid m, n \geq 1\}$ **CT : IV-06(4M)**

Ans. Step 1 :

Analysis :

(1) The language is fixed.

(2) All strings start with 0 and end with 0.

(3) In between, strings consist of 1.

(4) The number of 0's after 1 is equal to the starting 0's before one and number of 1's.

Valid strings : L = {0100, 001000, 00110000 ...}

Invalid strings : {010, 01000, 0010000, ...}

Step 2 :

Algorithm :

1. For 0's before 1, push 0 to stack and for 1's, push 1 to stack.
2. For 0's after 1, if input symbol is 0 stack top is 1 or 0, pop corresponding 1 or 0.

Transitions :

$\delta(q_0, 0, Z_0) \rightarrow (q_0, 0, Z_0)$

$\delta(q_0, 0, 0) \rightarrow (q_0, 00)$

$\delta(q_0, 1, 0) \rightarrow (q_1, 10)$

$\delta(q_1, 1, 1) \rightarrow (q_1, 11)$

$\delta(q_1, 0, 1) \rightarrow (q_2, \in)$

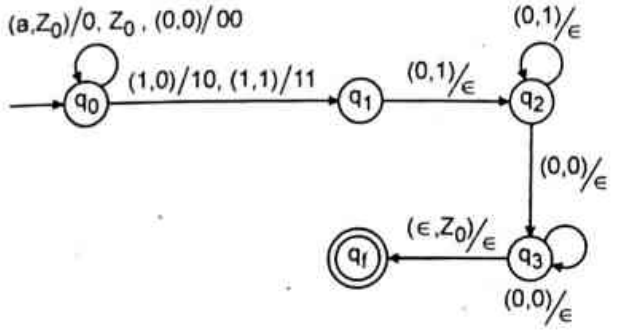
$\delta(q_2, 0, 1) \rightarrow (q_2, \in)$

$\delta(q_2, 0, 0) \rightarrow (q_3, \in)$

$\delta(q_3, 0, 0) \rightarrow (q_3, \in)$

$\delta(q_3, \in, Z_0) \rightarrow (q_f, \in)$

Transition Diagram :



Q.15. Construct a PDA that is accepted by empty stack for the language. **CT : W-09(7M)**

$L = \{a^m\ b^m\ c^n \mid m, n \geq 1\}$

Ans. Step 1 :

Analysis :

(1) The language is fixed.

(2) It consist of equal number of a and b and any number of c.

(3) The string starts with a and ends with c.

Valid strings :

L = {abc, aabbc, aabbcc, ...}

Invalid strings : {aabc, aabbbc, abbc .....}

Step 2 :

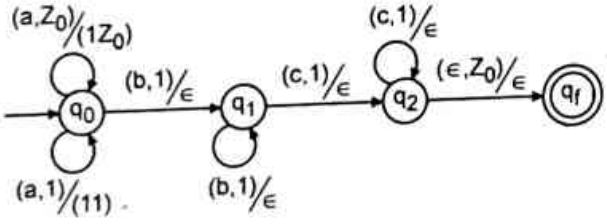
Algorithm :

1. For input symbol a, push 1 to stack and for input symbol b, pop 1 from stack.
2. Skip symbol c.

Transitions :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, 1\,Z_0)$
2. $\delta(q_0, a, 1) \rightarrow (q_0, 1\,1)$
3. $\delta(q_0, b, 1) \rightarrow (q_1, \in)$
4. $\delta(q_1, b, 1) \rightarrow (q_1, \in)$
5. $\delta(q_1, c, Z_0) \rightarrow (q_2, Z_0)$
6. $\delta(q_2, c, Z_0) \rightarrow (q_2, Z_0)$
7. $\delta(q_2, c, Z_0) \rightarrow (q_f, \in)$

Transition Diagram :



Hence CFG for L is,

G = ({S, A, B}, {a, b, ∈}, P, S)

where P consist of

$S \rightarrow AB$

$A \rightarrow aAb \mid ab \mid \in$

$B \rightarrow cB \mid \in$

Q.16. Construct Push Down Automata for the language.

$$L = \left\{ a^p\ b^q\ c^m \;\middle|\; \begin{matrix} p + m = q \\ p, m\ q \geq 1 \end{matrix} \right\}$$

OR Construct Push Down Automata for the given language $L = \{a^n\ b^{n+m}\ c^m / n, m \geq 0\}$.

CT : W-12(8M)

Ans. Step 1 :

Analysis :

(1) The language is fixed.

(2) String starts with a and ends with c.

(3) The strings consist of number of b equals the number of a and c.

Valid string :

L = {abbc, aabbbc, aabbbbcc, ...}

Invalid strings : {abc, abbcc, aabbcc, ... }

Step 2 :

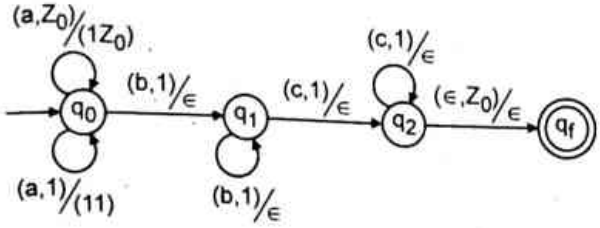
Algorithm :

1. For input symbol a, push 1 to stack and for input symbol b, pop 1 from stack till the stack becomes empty.
2. When stack contains Z_0 and input symbol is b, again add 1 to stack.
3. For input symbol c, pop 1 from stack.

Transitions :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, 1\,Z_0)$
2. $\delta(q_0, a, 1) \rightarrow (q_0, 1\,1)$
3. $\delta(q_0, b, 1) \rightarrow (q_1, \in)$
4. $\delta(q_1, b, 1) \rightarrow (q_1, \in)$
5. $\delta(q_1, b, Z_0) \rightarrow (q_2, 1\,Z_0)$
6. $\delta(q_2, b, 1) \rightarrow (q_2, 1\,1)$
7. $\delta(q_2, c, 1) \rightarrow (q_3, \in)$
8. $\delta(q_3, c, 1) \rightarrow (q_3, \in)$
9. $\delta(q_3, \in, Z_0) \rightarrow (q_f, \in)$

Transition Diagram :



Q.17. Design a PDA for language.

$L = \{W/W \in (a, b)^* / n_a(W) > n_b(W)\}$

CS : S-11(7M)

Ans. Step 1 :

Analysis :

(1) The language is variable.

(2) String starts with a or b.

(3) The number of a should be greater than that of b.

Valid strings : baaba λ , aabaa λ

Step 2 :

Algorithm :

1. Push 1 for a and 0 for b if Z_0 in the stack.
2. Push if top of the stack and incoming character is same.
3. POP if top of the stack and incoming characters is different.

Transitions :

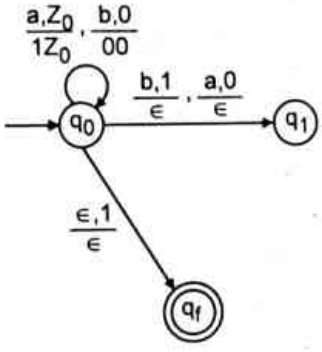
Starting with a :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, 1Z_0)$
2. $\delta(q_0, a, 1) \rightarrow (q_0, 11)$
3. $\delta(q_0, b, 1) \rightarrow (q_1, \in)$
4. $\delta(q_0, \in, 1) \rightarrow (q_f, \in)$

Starting with b :

1. $\delta(q_0, b, Z_0) \rightarrow (q_0, 0Z_0)$
2. $\delta(q_0, b, 0) \rightarrow (q_0, 0, 0)$
3. $\delta(q_0, a, 0) \rightarrow (q_1, \in)$
4. $\delta(q_0, \in, 1) \rightarrow (q_f, \in)$

Transition graph for PDA :



Q.18. Construct a push-down automaton (PDA) that accepts by empty stack, the language is as follows :

$L = \{uawb \mid u, w \in (a, b)^* \text{ and } |u| = |W|\}$

Ans. Step 1 :

Analysis :

(1) The language is variable.

(2) String may start with a or b but always ends with b with equal number of a and b.

Valid strings :

L = {abaabb, aabaabbb, abaabab, ...}

Invalid strings : L = {aba, abb, ababaa, ...}

Step 2 :

Algorithm :

1. Push a symbol of input string on stack and continue storing until the other symbol occurs.
2. If the top of stack is a and the current input is b, remove a from stack. Repeat the process for input b and stack top is a.

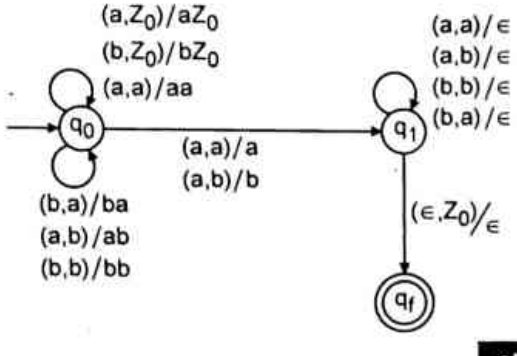
Transition :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, aZ_0)$
2. $\delta(q_0, b, Z_0) \rightarrow (q_0, bZ_0)$
3. $\delta(q_0, a, a) \rightarrow (q_0, aa)$

   $\delta(q_0, a, a) \rightarrow (q_1, a)$
4. $\delta(q_0, b, a) \rightarrow (q_0, ba)$
5. $\delta(q_0, a, b) \rightarrow (q_0, ab)$

   $\delta(q_0, a, b) \rightarrow (q_1, b)$
6. $\delta(q_0, b, b) \rightarrow (q_0, bb)$
7. $\delta(q_1, a, a) \rightarrow (q_1, \in)$
8. $\delta(q_1, a, b) \rightarrow (q_1, \in)$
9. $\delta(q_1, b, b) \rightarrow (q_1, \in)$
10. $\delta(q_1, b, a) \rightarrow (q_1, \in)$
11. $\delta(q_1, \in, Z_0) \rightarrow (q_f, \in)$

Transition graph :



Q.19. Construct Push down automata for the language. **CT : S-06(6M)**

$L = \{0^{m+n}\ 1^m 0^n \mid m, n \geq 0\}$

Ans. Step 1 :

Analysis :

(1) The language is fixed.

(2) String starts with 0 and ends with 0.

(3) The number of 0's before 1 is equal to number of 1's and 0's.

Valid strings :

L = {0010, 000110, 000100, 00001100}

Invalid strings : L = {010, 00100, 001100, ...}

Step 2 :

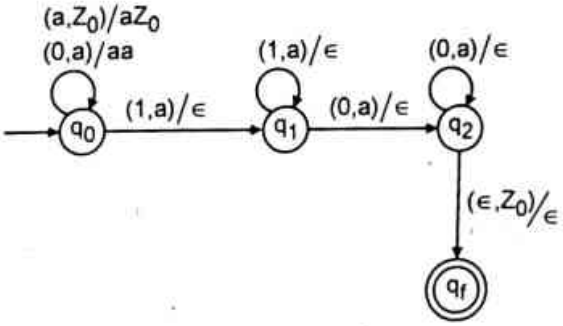
Algorithm :

1. For all input symbol 0 before 1, push a to stack.
2. For all 1's and 0's after 1, pop a from stack.

Transition :

1. $\delta(q_0, 0, Z_0) \rightarrow (q_0, aZ_0)$
2. $\delta(q_0, 0, a) \rightarrow (q_0, aa)$
3. $\delta(q_0, 1, a) \rightarrow (q_1, \epsilon)$
4. $\delta(q_1, 1, a) \rightarrow (q_1, \epsilon)$
5. $\delta(q_1, 0, a) \rightarrow (q_2, \epsilon)$
6. $\delta(q_2, 0, a) \rightarrow (q_2, \epsilon)$
7. $\delta(q_2, \epsilon, Z_0) \rightarrow (q_f, \epsilon)$

Transition graph :



Q.20. Construct Push down Automata for accepting the language. **CT : W-06(4M)**

$L = \{b^n \subset a^n \mid n \geq 1\}$

Ans. Step 1 :

Analysis :

(1) The language is fixed.

(2) String starts with b and ends with a.

(3) Starting consist of equal number of b and a.

Valid strings :

L = {bca, bbcaa, bbbcaaa, ...}

Invalid strings : L = {bbca, bcaa, bbcaaa, ...}

Step 2 :

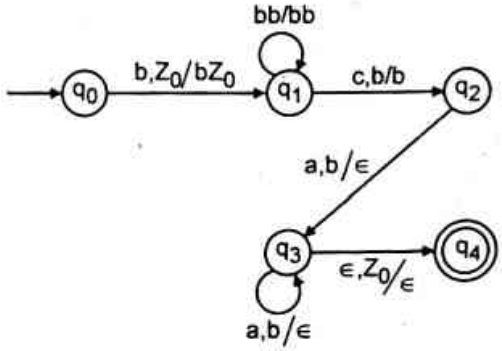
Algorithm :

1. For all input symbol b, push 1 to stack.
2. For input c, skip c.
3. For all input symbol a, pop 1 from stack.

Transition :

1. $\delta(q_0, b, Z_0) \rightarrow (q_0, b\ Z_0)$
2. $\delta(q_1, b, b) \rightarrow (q_1, bb)$
3. $\delta(q_1, c, b) \rightarrow (q_2, b)$
4. $\delta(q_2, a, b) \rightarrow (q_3, \epsilon)$
5. $\delta(q_3, a, b) \rightarrow (q_3, \epsilon)$
6. $\delta(q_3, \epsilon, Z_0) \rightarrow (q_4, \epsilon)$

Transition graph :



Q.21. Construct Push down automata for the language.

$L = \{0^n\ 1^m \mid n \geq m\}$

Ans. Step 1 :

Analysis :

(1) The language is fixed.

(2) The string start with 0 and ends with 1.

(3) The number of 0's are equal or greater than 1's.

Valid strings :

L = {01, 001, 0011, 00011, 0000111, ...}

Invalid strings : L = {011, 010, 0111, 00111, ..}

Step 2 :

Algorithm :

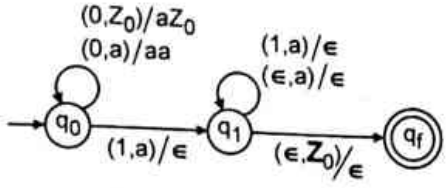
1. For all input symbol 0, push a to stack.
2. For all input symbol 1, pop a from stack.

Transition :

1. $\delta(q_0, 0, Z_0) \rightarrow (q_0, a\ Z_0)$

2. $\delta(q_0, 0, a) \rightarrow (q_0, aa)$
3. $\delta(q_0, 1, a) \rightarrow (q_1, \in)$
4. $\delta(q_1, 1, a) \rightarrow (q_1, \in)$
5. $\delta(q_1, \in, a) \rightarrow (q_1, \in)$
6. $\delta(q_1, \in, Z_0) \rightarrow (q_f, \in)$

Transition graph :



Q.22. Design NPDA for accepting following language : $L = \{WCW^R \mid W \in \{a, b\}^* W^R$ is reverse of W} ***CT : W-07, S-08(8M)***

OR Design pushdown automata for accepting the following language :

$L = \{XCX^R \mid X \in (a, b)^*\}$ ***CS : S-09(6M)***

Ans. Step 1 :

Analysis :

1. Length of string is variable.
2. String start or end either with 'a' or 'b'.
3. String is palindrome which separates 'c'.

Valid : L = {abcba, ababcbaba, bbbabcbabbb, bacab, babbaaacaaabbab, ...}

Step 2 :

Algorithm :

(A) String starts with 'a' :

1. Push '1' for input symbol 'a'.
2. When top of the stack is '1' and current input symbol is 'b' then push '0' for input symbol 'b'.
3. When top of the stack is '1' for '0' and current input symbol is 'c' then perform no operation for input symbol 'c'.
4. After 'c' when top of the stack is '0' or '1' then POP '1' for 'a' and '0' for input symbol 'b'.
5. Replace z_0 by $\in$.

(B) String start with 'b' :

1. Push '0' for input symbol 'b'.
2. When top of the stack is '0' and current input symbol is 'a' then push '1' for input symbol 'a'.
3. When top of the stack is '0' or '1' and the current input symbol is 'c' then perform no operation for input symbol 'c'.
4. After 'c' when top of the stack is '0' or '1' then POP '1' for 'a' and '0' for 'b'.
5. Replace z_0 by $\in$.

Transition :

For 'a' :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, 1Z_0)$
2. $\delta(q_0, a, 1) \rightarrow (q_0, 11)$
3. $\delta(q_0, b, 1) \rightarrow (q_1, 01)$
4. $\delta(q_1, b, 0) \rightarrow (q_1, 00)$

For 'b' :

1. $\delta(q_0, b, Z_0) \rightarrow (q_0, 0Z_0)$
2. $\delta(q_0, b, 0) \rightarrow (q_0, 00)$
3. $\delta(q_0, a, 0) \rightarrow (q_1, 10)$
4. $\delta(q_1, a, 1) \rightarrow (q_1, 11)$

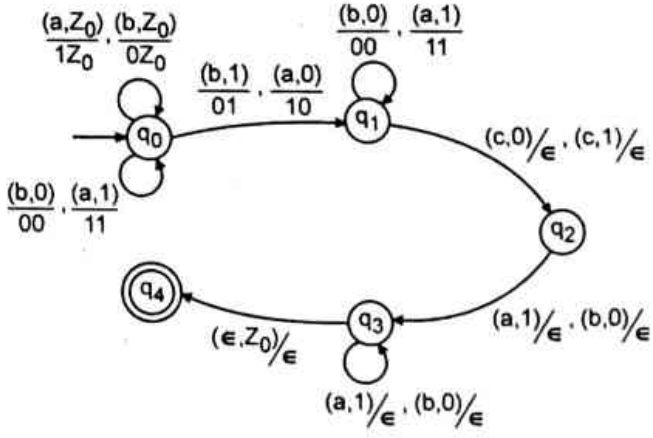
For 'c' :

1. $\delta(q_1, c, 0) \rightarrow (q_2, \in)$
2. $\delta(q_1, c, 1) \rightarrow (q_2, \in)$

Poping :

1. $\delta(q_2, a, 1) \rightarrow (q_3, \in)$
2. $\delta(q_3, a, 1) \rightarrow (q_3, \in)$
3. $\delta(q_2, b, 0) \rightarrow (q_3, \in)$
4. $\delta(q_3, b, 0) \rightarrow (q_3, \in)$
5. $\delta(q_3, \in, Z_0) \rightarrow (q_4, \in)$

Transition Diagram :



Q.23. Design PDA for accepting the language.

$L = \{WW^R \mid W \in (a, b)^*\}$ is reverse of W.

CT : S-12, W-10(7M)

OR Design PDA for $\{WW^R \mid W \text{ in } (0+1)^*\}$ using empty stack acceptance. Also show all accessible ID's for the string 001100. **CS : S-12(8M)**

OR Construct PDA for accepting language

$L = \{WW^R \mid W \in (011)^*\}$ **CT : S-06(5M)**

OR Design PDA for language

$L = \{WW^R \mid W \text{ in } (0+1)^*\}$ using empty stack acceptance. Convert this PDA into PDA using acceptance by final state. **CS : S-13(8M)**

Ans. Step 1 :

Analysis :

1. Length of the string is variable.
2. String starts or end either with 'a' or 'b'.
3. String is palindrome.

Valid string : L = {abba, baab, aabbbbaa, baaaab, ..}

Step 2 :

Algorithm :

(A) String starts with 'a' :

1. Push '1' for input symbol 'a' and POP '1' for 'b'.
2. When top of the stack is '1' then current input symbol is 'b' .
3. When the stack comes on its initial state again and current input symbol is 'b' then push '1' for input symbol 'b' and POP '1' for input symbol 'a'.
4. Replace 'z_0' by '$\in$'.

(B) String starts with 'b' :

1. Push '1' for input symbol 'b' and POP '1' for input symbol 'a'.
2. When the stack comes on its initial state again and current input symbol is 'a' then push '1' for input symbol 'a' and POP '1' for input symbol 'b'.
3. Replace 'z_0' by '$\in$'.

Transition :

$\delta(q_0, a, Z_0) \rightarrow (q_0, 1\,Z_0)$

$\delta(q_0, a, 1) \rightarrow (q_0, a1)$

$\delta(q_0, a, 1) \rightarrow (q_1, \in)$

$\delta(q_0, a, 0) \rightarrow (q_0, a0)$

$\delta(q_0, b, Z_0) \rightarrow (q_0, 0, Z_0)$

$\delta(q_0, b, 0) \rightarrow (q_0, b0)$

$\delta(q_0, b, 0) \rightarrow (q_1, \in)$

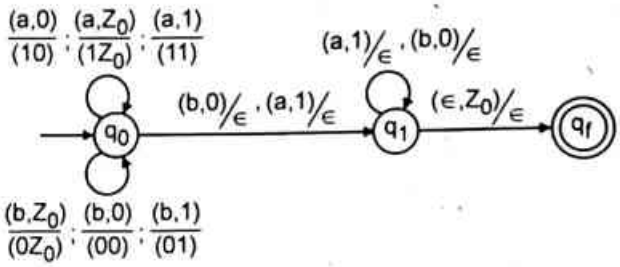
$\delta(q_0, b, 1) \rightarrow (q_0, 01)$

$\delta(q_1, a, 1) \rightarrow (q_1, \in)$

$\delta(q_1, b, 0) \rightarrow (q_1, \in)$

$\delta(q_1, \in, Z_0) \rightarrow (q_f, \in)$

Transition Diagram :



Q.24. Construct a non-deterministic push down automata for the following language :

$L = \{W \in \{a, b\}^* \mid n_a(W) = n_b(W)\}$

CS : W-12(7M)

Ans. Step 1 :

Analysis :

(1) The language is variable.

(2) The string may start with a or b and ends with a or b.

(3) String contains equal number of a and b.

Valid string :

L = {ab, ba, abab, baab, babaab, ...}

Invalid strings : L = {abb, bab, baabb, ...}

Step 2 :

Algorithm :

1. Push a symbol of input string on stack and continue storing until the other symbol occurs.
2. If the top of stack is a and the current input symbol is b, remove a from stack. Repeat the process for input b and stack top a.

Transitions :

$\delta(q_0, a, Z_0) \rightarrow (q_0, a\,Z_0)$

$\delta(q_0, b, Z_0) \rightarrow (q_0, b\,Z_0)$

$\delta(q_0, a, a) \rightarrow (q_0, aa)$

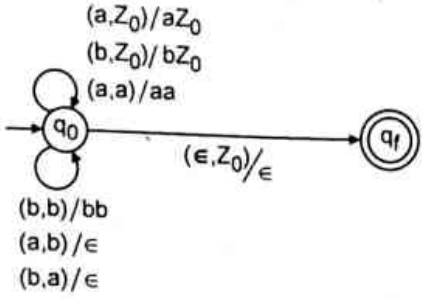
$\delta(q_0, b, b) \rightarrow (q_0, bb)$

$\delta(q_0, a, b) \rightarrow (q_0, \in)$

$\delta(q_0, b, a) \rightarrow (q_0, \in)$

$\delta(q_0, \in, Z_0) \rightarrow (q_f, \in)$

Transition graph :



Q.25. Construct PDA for the language $L = \{a^n b^m c^m d^n \mid m, n \geq 1\}$ **CS : W-11(7M)**

Ans. Step 1 :

1. Let n = 1 and m = 1
   $L_1 = \{abcd\}$
2. Let n = 1 and m = 2
   $L_2 = \{abbccd\}$
3. Let n = 2 , m = 1
   $L_3 = \{aabcdd\}$

Valid String : {abcd, aabcdd, abbccd, aabbccdd, ...}

Invalid string : {abd, aabcd, abbcd, ...}

Analysis :

1. String has fixed length.
2. String starts with a and ends on d with either equal number of a's and d's or equal number of b's and c's.

Step 2 :

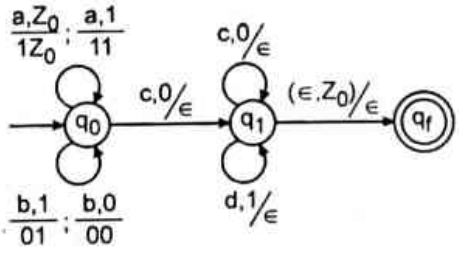
Algorithm :

1. Push '1' for input symbol 'a' and push '0' for input symbol 'b'.
2. Pop '0' for input symbol 'c' and Pop '1' for input symbol 'd'.
3. Replace Z_0 by $\in$.

Transitions :

1. $\delta(q_0, a, Z_0) \rightarrow (q_0, 1\,Z_0)$
2. $\delta(q_0, a, 1) \rightarrow (q_0, 11)$
3. $\delta(q_0, b, 1) \rightarrow (q_0, 01)$
4. $\delta(q_0, b, 0) \rightarrow (q_0, 00)$
5. $\delta(q_0, c, 0) \rightarrow (q_1, \in)$
6. $\delta(q_1, c, 0) \rightarrow (q_1, \in)$
7. $\delta(q_1, d, 1) \rightarrow (q_1, \in)$
8. $\delta(q_1, d, 1) \rightarrow (q_1, \in)$
9. $\delta(q_1, \in, Z_0) \rightarrow (q_f, \in)$

Transition graph :

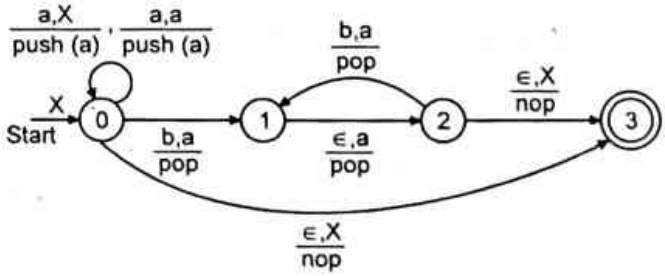


Q.26. Construct a PDA to accept the language $\{a^{2n} b^n \mid n >= 1\}$. **CT : W-11(5M)**

Ans. ID for input aaaabb :

(0, aaaabb, X)
(0, aaabb, aX)
(0, aabb, aaX)
(0, abb, aaaX)
(0, bb, aaaaX)
(1, b, aaaX)
(2, b, aaX)
(1, ∈, aX)
(2, ∈, X)
(3, ∈, X)

Transition diagram :



Q.27. Construct PDA accepting = $\{a^n b^m a^n / m, n\, b >= 1\}$ by null store. Construct corresponding CFG accepting the same set. **CT : W-11(8M)**

Ans. If A = $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ is a PDA accepting L by null store, we can find a PDA B = $(Q', \Sigma, \Gamma', \delta_B, q_0', Z_0', F')$ which accept L by final state, i.e., L = N (A) = T(B).

Using the above theorem, we have

B = $(\{q_0, q_1, q_0', q_f\}, \{a, b\}, \{a, Z_0, Z_0'\}, \delta, q_0', Z_0', \{q_f\})$

where δ is given by

$\delta(q_0', \lambda, Z_0') = \{(q_0, Z_0, Z_0')\}$

$\delta(q_0, \lambda, Z_0') = \{(q_f, \lambda)\} = \delta(q_0, \lambda, Z_0')$

$\delta(q_1, \lambda, Z_0') = \{(q_f, \lambda)\} = \delta(q_1, \lambda, Z_0')$

$\delta(q_0, a, Z_0) = \{(q_0, a\ Z_0)\}$

$\delta(q_0, a, a) = \{(q_0, aa)\}$

$\delta(q_0, b, a) = \{(q_1, a)\}$

$\delta(q_1, b, a) = \{(q_1, a)\}$

$\delta(q_1, a, a) = \{(q_f, \lambda)\}$

$\delta(q_1, \lambda, Z_0) = \{(q_1, \lambda)\}$

Q.28. Explain acceptance of string by PDA in terms of final state and in terms of empty push down stack. ***CS : W-10(6M)***

OR Explain method to get PDA with final state from PDA accepting by empty stack. ***CT : S-13(5M)***

Ans.

- For every PDA accepting by empty stack, there exists equivalent PDA accepting by final stack.
- The equivalence is defined in terms of language accepted, that means for every PDA accepting by empty stack, there exists a PDA accepting the same language, but accepting its input by final state.

Example :

Let $M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, \phi\}$ be a PDA, accepting by empty stack.

Let us construct PDA M, using M as follows.

$M_1 = \{Q \cup \{q_0, q_f\}, \Sigma, \Gamma, \cup \{X_0\}, \delta\ q_0, X_0, q_f\}$

Where,

$\delta(q_0, \in, X_0) = \delta(q_0, z_0, X_0)$

For all q in Q and α in $\Sigma \cup \{\in\}$ and Z in Γ

$\delta(q, a, z) = \delta(q, a, z)$

For all q in Q, $\delta(q, \in, X_0)$ contains $(q_f, \in)$.

- Certainly M_1 accepts λ [M], because M_1 first enters from its initial configuration into the initial configuration of M_1, without consuming any input symbol.

Then it carries out the simulation of M, and when M empties its stack, then M_1, enters into its final state.

- Thereby providing that for every PDA accepting by empty stack, there exists an equivalent PDA accepting by final state.

CONVERSION OF PDA TO CFG

Q.29. Give the rules for conversion of PDA to CFG.

Ans. PDA is given by $M = \{Q, \Sigma, \delta, q_0, \Gamma, Z_0, F\}$

where,

Q - set of states.

Σ - set of input symbols

δ - Transform set

q_0 - Initial state

Γ - stack symbol

Z_0 - Initial stack symbol

F - Final state.

and CFG is given by

$G = \{V, T, P, S\}$

V - Non-terminal

T - Terminal

P - Production

S - Start symbol

Rules for conversion of PDA to CFG :

1. Find the terminal.
2. **Start symbol :** The production rules for the start symbol is denoted as follows .

   $S \rightarrow (q_0\ Z_0\ q_0)$

   where $Q = \{q_0\ q_1\}$

   $S \rightarrow (q_0\ Z_0\ q_1)$
3. **Non-terminals :**

   $S \in V$

   $V \in S \cup \{q\ Z\ q'\}$

   where $q, q' \in Q$

   $Z \in \Gamma$

   This rule will cover all the states in q and q'Z represent all the symbols which are permitted on the stack including Z_0.

   $Q = \{q_0, q_1\}$ $(q_0\ Z_0\ q_0) = A$

   $Z = \{Z_0, a\}$ $(q_0\ Z_0\ q_1) = B$

   $q = \{q_0, q_1\}$ $(q_1\ Z_0\ q_0) = C$

$q' = \{q_0, q_1\}$ $(q_1\ Z_0\ q_1) = D$

$(q_0\ a\ q_0) = E$

$(q_0\ a\ q_1) = F$

$(q_1\ a\ q_0) = G$

$(q_1\ a\ q_1) = H$

4. Production Rules :

In the PDA, the push operation and the POP operation can be differentiated by the states of the stack.

(i) Deletion / POP operation :

For each transition of the form

$\delta(q, a, Z_0) \rightarrow (q', \lambda)$

then the rules is denoted by

$(q\ z_o\ q') \rightarrow a$

Example :

1. $\delta(q_0, a, 1) \rightarrow (q_1, \lambda)$

$(q_0\ 1\ q_1) \rightarrow a$

2. $\delta(q_0, b, a) \rightarrow (q_1, \lambda)$

$(q_0\ a\ q_1) \rightarrow b$

Since, $(q_0\ a\ q_1) = F$

$\therefore F \rightarrow b$

(ii) Push operation :

Push operation in PDA is denoted by

$\delta(q, a, Z) \rightarrow (q', Z_1\ Z_2 \ldots\ldots\ldots Z_n)$

The production rule for push operation will be represented as follows.

$[q\ Z\ q'] \rightarrow a\,[q_1\ Z_1\ q_2][q_2\ Z_2\ q_3][q_m\ Z_n\ q']$

(iii) Skip operation :

The skip operation in the PDA is denoted by

$\delta(q_0, a, Z) \rightarrow (q_1, Z)$

The production rule for skip operation is denoted as follows.

$(q_0\ Z\ q') \rightarrow a\,(q_1\ Z\ q')$

Q.30. For the PDA $M = \{(q_0, q_1), (0, 1, Z_0),$ $\delta(q_0, \in, Z_0) = \{(q_0, \in)\}$ where δ is given below :

$\delta(q_0, \in, Z_0)$ $\{(q_1, \in)\}$

$\delta(q_0, 0, Z_0)$ $\{(q_0, 0\ Z_0)\}$

$\delta(q_0, 0, 0)$ $\{(q_0, 00)\}$

$\delta(q_0, 1, 0)$ $\{(q_0, 10)\}$

$\delta(q_0, 1, 1)$ $\{(q_0, 11)\}$

$\delta(q_0, 0, 1)$ $\{(q_1, \in)\}$

$\delta(q_1, 0, 1)$ $\{(q_1, \in)\}$

$\delta(q_1, 0, 0)$ $\{(q1, \in)\}$

$\delta(q_1, \in, Z_0)$ $\{(q_1, \in)\}$

Obtain the CFG generation the language accept by M (This PDA accepts $L = L = 1^n\ 0^{m+n}$).

Ans. Step 1 : $\Sigma = (0, 1, Z_0)$

Step 2 : $S \rightarrow [q_0, Z_0, q_0]$

$S \rightarrow [q_0, Z_0, q_1]$

Step 3 :

$[q_0, Z_0, q_0] = A$

$[q_0, Z_0, q_1] = B$

$[q_1, Z_0, q_0] = C$

$[q_1, Z_0, q_1] = D$

$[q_0, 0, q_0] = E$

$[q_0, 0, q_1] = F$

$[q_1, 0, q_0] = G$

$[q_1, 0, q_1] = H$

$[q_0, 1, q_0] = I$

$[q_0, 1, q_1] = J$

$[q_1, 1, q_0] = K$

$[q_1, 1, q_1] = L$

Step 4 :

Production Rules

1. POP operation :

(i) $\delta(q_0, \in, Z_0) = \{(q_1, \in)\}$

$\therefore [q_0\ Z_0\ q_1] \rightarrow \in$

$B \rightarrow \in$

(ii) $\delta(q_1, \in, Z_0) = \{(q_1, \in)\}$

$\therefore [q_1\ Z_0\ q_1] \rightarrow \in$

$D \rightarrow \in$

2. Push operation :

(i) $\delta(q_0, 0, Z_0) = (q_0\ 0Z_0)$

$[q_0, Z_0, q_0] \rightarrow 0\,[q_0, 0, q_0][q_0, Z_0, q_0]$

$[q_0, Z_0, q_0] \rightarrow 0\,[q_0, 0, q_1][q_1, Z_0, q_0]$

$[q_0, Z_0, q_1] \rightarrow 0\,[q_0, Z_0, q_0][q_0, Z_0, q_1]$

$[q_0, Z_0, q_1] \rightarrow 0\,[q_0, 0, q_1][q_1, Z_0, q_1]$

$A \rightarrow 0\,EA$

$A \rightarrow 0\,FC$

$B \rightarrow 0\,EB$

$B \rightarrow 0\,FD$

(ii) $\delta(q_0, 0, 0) = \{(q_0, 00)\}$

$[q_0, 0, q_0] \rightarrow 0\,[q_0, 0, q_0][q_0, 0, q_0]$

$[q_0, 0, q_0] \rightarrow 0\,[q_0, 0, q_1][q_1, 0, q_0]$

$[q_0, 0, q_1] \rightarrow 0\,[q_0, 0, q_0][q_0, 0, q_1]$

$[q_0, 0, q_1] \rightarrow 0\,[q_0, 0, q_1][q_1, 0, q_1]$

$E \rightarrow 0\,EE$

$E \rightarrow 0\,FG$

$F \rightarrow 0\,EF$

$F \rightarrow 0\,FH$

(iii) $\delta(q_0, 1, 0) = \{(q_0, 10)\}$

$[q_0, 0, q_0] \rightarrow 1\,[q_0, 1, q_0][q_0, 0, q_0]$

$[q_0, 0, q_0] \rightarrow 1\,[q_0, 1, q_1][q_1, 0, q_0]$

$[q_0, 0, q_1] \rightarrow 1\,[q_0, 1, q_0][q_0, 0, q_1]$

$[q_0, 0, q_1] \rightarrow 1\,[q_0, 1, q_1][q_1, 0, q_1]$

$E \rightarrow 1\,IE$

$E \rightarrow 1\,JG$

$F \rightarrow 1\,IF$

$F \rightarrow 1\,JH$

(iv) $\delta(q_0, 1, 1) = \{(q_0, 11)\}$

$[q_0, 1, q_0] \rightarrow 1\,[q_0, 1, q_0][q_0, 1, q_0]$

$[q_0, 1, q_0] \rightarrow 1\,[q_0, 1, q_1][q_1, 1, q_0]$

$[q_0, 1, q_1] \rightarrow 1\,[q_0, 1, q_0][q_0, 1, q_1]$

$[q_0, 1, q_1] \rightarrow 1\,[q_0, 1, q_1][q_1, 1, q_1]$

$I \rightarrow 1\,I\,I$

$I \rightarrow 1\,J\,K$

$J \rightarrow 1\,I\,J$

$J \rightarrow 1\,J\,L$

3. Skip operation :

(i) $\delta(q_0, 0, 1) = \{(q_1, \in)\}$

$\therefore [q_0, 1, q_1] \rightarrow 0$

$\therefore J \rightarrow 0$

(ii) $\delta(q_1, 0, 1) = \{(q_1, \in)\}$

$\therefore [q_1, 1, q_1] \rightarrow 0$

$\therefore L \rightarrow 0$

(iii) $\delta(q_1, 0, 0) = \{(q_1, \in)\}$

$\therefore [q_1, 0, q_1] \rightarrow 0$

$\therefore H \rightarrow 0$

Therefore, the production becomes :

$S \rightarrow A \mid B$

$A \rightarrow 0\,E\,A \mid 0\,F\,C$

$B \rightarrow 0\,E\,B \mid 0\,F\,D \mid \in$

$E \rightarrow 0\,E\,E \mid 0\,F\,G \mid 1\,I\,E \mid 1\,J\,G$

$F \rightarrow 0\,E\,F \mid 0\,F\,H \mid 1\,I\,F \mid 1\,J\,H$

$I \rightarrow 1\,I\,I \mid 1\,J\,K$

$J \rightarrow 1\,I\,J \mid 1\,J\,L \mid 0$

$L \rightarrow 0$

$H \rightarrow 0$

$D \rightarrow \in$

Q.31. Design a CFG for following PDA.

CS : W-10(8M)

$\{(q_0, q_1)\{a, b\}, \{Z_0, Z_0\}\, \delta, q_0, Z_0\}$

$\delta(q_0, b, Z_0) \rightarrow (q_0, Z_0, Z_0)$

$\delta(q_0, \in, Z_0) \rightarrow (q_0, \in)$

$\delta(q_0, b, Z) \rightarrow (q_0, Z, Z_0)$

$\delta(q_0, a, Z) \rightarrow (q_1, Z)$

$\delta(q_1, b, Z) \rightarrow (q_1, \in)$

$\delta(q_1, a, Z_0) \rightarrow (q_1, Z_0)$

OR Construct CFG that accepts the set accepted by $M = (\{q_0, q_1\}\ \{a, b\}, \{A, Z\}, \delta, q_0, Z, \{q_1\})$

$\delta(q_0, a, Z) = \{(q_0, A\,Z)\}$; $\delta(q_0, b, Z) = \{(q_0, A\,Z)\}$; $\delta(q_0, a, A) = \{(q_1, \in)\}$

CS : W-12(7M)

Ans. Step 1 : $\Sigma = (a, b)$

Step 2 : $S \rightarrow [q_0, Z_0, q_0]$

$S \rightarrow [q_0, Z_0, q_1]$

Step 3 :

$[q_0, Z_0, q_0] = A$

$[q_0, Z_0, q_1] = B$

$[q_1, Z_0, q_0] = C$

$[q_1, Z_0, q_1] = D$

$[q_0, Z_0, q_0] = E$

$[q_0, Z_0, q_1] = F$

$[q_1, Z_0, q_0] = G$

$[q_1, Z_0, q_1] = H$

Step 4 :

Production rules :

1. POP operation :

$\delta(q_0, \in, Z_0) \rightarrow (q_0, \in)$

$\therefore [q_0, Z_0, q_0] \rightarrow \in$

$A \rightarrow \in$

$\delta(q_1, b, Z) \rightarrow (q_1, \in)$

$\therefore [q, Z, q_1] \rightarrow b$

$H \rightarrow b$

2. Push operation :

The input symbol is b in first operation and stack symbol can be Z and Z_0.

The resultant states can be q_0 and q_1 The initial state will be q_0.

1. $\delta(q_0, b, Z_0) \rightarrow (q_0\ Z, Z_0)$

$[q_0, Z_0, q_0] \rightarrow b\,[q_0, Z, q_0][q_0, Z_0, q_0]$

$[q_0, Z_0, q_0] \rightarrow b\,[q_0, Z, q_1][q_1, Z_0, q_0]$

$[q_0, Z_0, q_1] \rightarrow b\,[q_0, Z, q_0][q_0, Z_0, q_1]$

$[q_0, Z_0, q_1] \rightarrow b\,[q_0, Z, q_1][q_1, Z_0, q_1]$

$A \rightarrow b\,EA$

$A \rightarrow b\,FC$

$B \rightarrow b\,EB$

$B \rightarrow b\,FD$

2. $\delta(q_0, b, Z_0) \rightarrow (q_0\ Z, Z_0)$

$[q_0, Z, q_0] \rightarrow b\,[q_0, Z, q_0][q_0, Z_0, q_0]$

$[q_0, Z, q_0] \rightarrow b\,[q_0, Z, q_1][q_1, Z_0, q_0]$

$[q_0, Z, q_1] \rightarrow b\,[q_0, Z, q_0][q_0, Z_0, q_1]$

$[q_0, Z, q_1] \rightarrow b\,[q_0, Z, q_1][q_1, Z_0, q_1]$

$E \rightarrow b\,EA$

$E \rightarrow b\,FC$

$F \rightarrow b\,EB$

$F \rightarrow b\,FD$

3. Skip operation :

(i) $\delta(q_0, a, Z) \rightarrow (q_1, Z)$

$[q_0, Z, q_0] \rightarrow a\,[q_1, Z, q_0]$

$[q_0, Z, q_1] \rightarrow a\,[q_1, Z, q_1]$

$E \rightarrow a\,G$

$F \rightarrow a\,H$

(ii) $\delta(q_1, a, Z_0) \rightarrow (q_1, Z_0)$

$[q_1, Z_0, q_0] \rightarrow a\,[q_1, Z_0, q_0]$

$C \rightarrow a\,C$

$D \rightarrow a\,D$

$S \rightarrow A \mid B$

$A \rightarrow b\,EA \mid b\,FC \mid \in$

$B \rightarrow b\,EB \mid b\,FD$

$C \rightarrow a\,C$

$D \rightarrow a\,D$

$E \rightarrow b\,EA \mid b\,FC \mid a\,G$

$F \rightarrow b\,EB \mid b\,FD \mid a\,H$

$G \rightarrow \in$

$H \rightarrow b$

Q.32. Give the CFG generating the language accepted by the following PDA :

$M = (\{q_0, q_1\}, \{0, 1\}, \{Z_0, X\}\ \delta, q_0, Z_0, \phi)$.

Where δ is given below : ***CS : S-12(6M)***

(i) $\delta(q_0, 1, Z_0) = \{(q_0, XZ_0)\}$

(ii) $\delta(q_0, 1, X) = \{(q_0, XX)\}$

(iii) $\delta(q_0, 0, X) = \{(q_1, X)\}$

(iv) $\delta(q_0, \in, Z_0) = \{(q_0, \in)\}$

(v) $\delta(q_1, 1, X) = \{(q_1, \in)\}$

(vi) $\delta(q_1, 0, Z_0) = \{(q_0, Z_0)\}$

OR Convert PDA to CFG

$P = (\{p, q\}, \{0, 1\}\ \{x, Z\},\ \delta, p, Z)$

$\delta(q, 1, Z) = \{(q, x\ Z)\}$

$\delta(q, 1, x) = \{(q, x\ x)\}$

$\delta(q, 0, x) = \{(p, x)\}$

$\delta(q, \in, Z) = \{(q, \in)\}$

$\delta(p, 1, x) = \{(p, \in)\}$

$\delta(p, 0, Z) = \{(q, Z)\}$ ***CS : S-13(8M)***

Ans. The production of an equivalent CFG are obtained as follows :

$S \rightarrow [q_0, Z_0, q_0]$

$S \rightarrow [q_0, Z_0, q_1]$

Because there are two states q_0 and q_1 therefore, the grammar should generate all those strings that allows PDA M to erase Z_0 from the stack by starting in q_0 and either ending up in q_0 or q_1 using sequence of moves.

(i) **For a move $\delta(q_0, 1, Z_0) = \{(q_0, XZ_0)\}$:** We get the following productions :

$[q_0, Z_0, q_0] \rightarrow 1\,[q_0, X, q_0][q_0, Z_0, q_0]$

$[q_0, Z_0, q_0] \rightarrow 1\,[q_0, X, q_1][q_1, Z_0, q_0]$

$[q_0, Z_0, q_1] \rightarrow 1\,[q_0, X, q_0][q_0, Z_0, q_1]$

$[q_0, Z_0, q_1] \rightarrow 1\,[q_0, X, q_1][q_1, Z_0, q_1]$

(ii) **For a move $\delta(q_0, 1, X) = \{(q_0, XX)\}$:** We get the following productions :

$[q_0, X, q_0] \rightarrow 1\,[q_0, X, q_0][q_0, X, q_0]$

$[q_0, X, q_0] \rightarrow 1\,[q_0, X, q_1][q_1, X, q_0]$

$[q_0, X, q_1] \rightarrow 1\,[q_0, X, q_0][q_0, X, q_1]$

$[q_0, X, q_1] \rightarrow 1\,[q_0, X, q_1][q_1, X, q_1]$

(iii) **For a move $\delta(q_0, 0, X) = \{(q_1, X)\}$:** We get the following productions :

$[q_0, X, q_0] \rightarrow 0\,[q_1, X, q_0]$

$[q_0, X, q_1] \rightarrow 0\,[q_1, X, q_1]$

(iv) **For a move $\delta(q_0, \in, Z_0) = \{(q_0, \in)\}$:** We get the following productions :

$[q_0, Z_0, q_0] \rightarrow \in$

(v) **For a move $\delta(q_1, 1, X) = \{(q_1, \in)\}$:** We get the following productions :

$[q_1, X, q_1] \rightarrow 1$

(vi) **For a move $\delta(q_1, 0, Z_0) = \{(q_0, Z_0)\}$:** We get the following productions :

$[q_1, Z_0, q_0] \rightarrow 0\,[q_0, Z_0, q_0]$

$[q_1, Z_0, q_1] \rightarrow 0\,[q_0, Z_0, q_1]$

If we rename the objects as shown below :

Object	Name
$[q_0, Z_0, q_0]$	A
$[q_0, Z_0, q1]$	B
$[q_1, Z_0, q_0]$	C
$[q_1, Z_0, q_1]$	D
$[q_0, X, q_0]$	E
$[q_0, X, q_1]$	F
$[q_1, X, q_0]$	G
$[q_1, X, q_1]$	H

Then the production can be written as :

$S \rightarrow A$

$S \rightarrow B$

$A \rightarrow 1\,E\,A$

$A \rightarrow 1\,F\,C$

$B \rightarrow 1\,E\,B$

$B \rightarrow 1\,F\,D$

$E \rightarrow 1\,E\,E$

$E \rightarrow 1\,F\,G$

$F \rightarrow 1\,E\,F$

$F \rightarrow 1\,F\,H$

$E \rightarrow 0\,G$

$F \rightarrow 0\,H$

$A \rightarrow \in$

$H \rightarrow 1$

$C \rightarrow 0\,A$

$D \rightarrow 0\,B$

Therefore, the productions become :

$S \rightarrow A \mid B$

$A \rightarrow 1\,EA \mid 1\,FC \mid \in$

$B \rightarrow 1\,EC \mid 1\,FD$

$E \rightarrow 1\,EE \mid 1\,FG \mid 0G$

$F \rightarrow 1\,EF \mid 1\,FH \mid 0H$

$D \rightarrow 0B$

$H \rightarrow 1$

$C \rightarrow 0A$

Since G is useless, therefore we get :

$S \rightarrow A \mid B$

$A \rightarrow 1\,EA \mid 1\,FC \mid \in$

$B \rightarrow 1\,EC \mid 1\,FD$

$E \rightarrow 1\,EE$

$F \rightarrow 1\,EF \mid 1\,FH \mid 0H$

$D \rightarrow 0B$

$H \rightarrow 0$

$C \rightarrow 0A$

As a result of which E becomes useless, therefore, we get :

$S \rightarrow A \mid B$

$A \rightarrow 1\,FC \mid \in$

$B \rightarrow 1\,FD$

$F \rightarrow 1\,FH \mid 0H$

$D \rightarrow 0B$

$H \rightarrow 1$

$C \rightarrow 0A$

As a result D and B becomes useless, therefore, we get :

$S \rightarrow A$

$A \rightarrow 1FC \mid \in$

$F \rightarrow 1FH \mid 0H$

$H \rightarrow 1$

$C \rightarrow 0A$

which can be reduced to :

$S \rightarrow A$

$A \rightarrow 1F0A \mid \in$

$F \rightarrow 1F1 \mid 01$

Q.33. For the PDA $M = (\{q_0, q_1\}, \{0,1\}, \{0,1,Z_0\}, \delta, q_0, Z_0, \phi)$, where δ is given below :

$\delta(q_0, \in, Z_0) = \{(q_1, \in)\}$

$\delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\}$

$\delta(q_0, 0, 0) = \{(q_0, 00)\}$

$\delta(q_0, 1, 0) = \{(q_0, 10)\}$

$\delta(q_0, 1, 1) = \{(q_0, 11)\}$

$\delta(q_0, 0, 1) = \{(q_1, \in)\}$

$\delta(q_1, 0, 1) = \{(q_1, \in)\}$

$\delta(q_1, 0, 0) = \{(q_1, \in)\}$

$\delta(q_1, \in, Z_0) = \{(q_1, \in)\}$

Obtain the CFG generating the language accepted by M (This PDA accepts $L = 1^n 0^{m+n}$})

Ans. The productions of an equivalent CFG are obtained as follows :

$S \rightarrow [q_0, Z_0, q_0]$

$S \rightarrow [q_0, Z_0, q_1]$

Because there are two states q_0 and q_1, therefore, the grammar should generate all those strings that allows PDA M to erase Z_0 from the stack by starting in q_0 and either ending up in q_0 or q_1', using sequence of moves.

(i) For a move $\delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\}$: We get the following productions :

$[q_0, Z_0, q_0] \rightarrow 0[q_0, 0, q_0][q_0, Z_0, q_0]$

$[q_0, Z_0, q_0] \rightarrow 0[q_0, 0, q_1][q_1, Z_0, q_0]$

$[q_0, Z_0, q_1] \rightarrow 0[q_0, 0, q_0][q_0, Z_0, q_1]$

$[q_0, Z_0, q_1] \rightarrow 0[q_0, 0, q_1][q_1, Z_0, q_1]$

(ii) For a move $\delta(q_0, 0, 0) = \{(q_0, 00)\}$: We get the following productions :

$[q_0, 0, q_0] \rightarrow 0[q_0, 0, q_0][q_0, 0, q_0]$

$[q_0, 0, q_0] \rightarrow 0[q_0, 0, q_1][q_1, 0, q_0]$

$[q_0, 0, q_1] \rightarrow 0[q_0, 0, q_0][q_0, 0, q_1]$

$[q_0, 0, q_1] \rightarrow 0[q_0, 0, q_1][q_1, 0, q_1]$

(iii) For a move $\delta(q_0, 1, 0) = \{(q_0, 10)\}$: We get the following productions :

$[q_0, 0, q_0] \rightarrow 1[q_0, 1, q_0][q_0, 0, q_0]$

$[q_0, 0, q_0] \rightarrow 1[q_0, 1, q_1][q_1, 0, q_0]$

$[q_0, 0, q_1] \rightarrow 1[q_0, 1, q_0][q_0, 0, q_1]$

$[q_0, 0, q_1] \rightarrow 1[q_0, 1, q_1][q_1, 0, q_1]$

(iv) For a move $\delta(q_0, 1, 1) = \{(q_0, 11)\}$: We get the following productions :

$[q_0, 1, q_0] \rightarrow 1[q_0, 1, q_0][q_0, 1, q_0]$

$[q_0, 1, q_0] \rightarrow 1[q_0, 1, q_1][q_1, 1, q_0]$

$[q_0, 1, q_1] \rightarrow 1[q_0, 1, q_0][q_0, 1, q_1]$

$[q_0, 1, q_1] \rightarrow 1[q_0, 1, q_1][q_1, 1, q_1]$

(v) For a move $\delta(q_0, 0, 1) = \{(q_1, \in)\}$: We get the following productions :

$[q_0, 1, q_1] \rightarrow 0\,s$

(vi) For a move $\delta(q_1, 0, 1) = \{(q_1, \in)\}$: We get the following productions :

$[q_1, 1, q_1] \rightarrow 0$

(vii) For a move $\delta(q_1, 0, 0) = \{(q_1, \in)\}$: We get the following productions :

$[q_1, 0, q_1] \rightarrow 0$

(viii) For a move $\delta(q_1, \in, Z_0) = \{(q_1, \in)\}$: We get the following productions :

$[q_1, Z_0, q_1] \rightarrow \in$

(ix) For a move $\delta(q_0, \in, Z_0) = \{(q_1, \in)\}$: We get the following productions :

$[q_0, Z_0, q_1] \rightarrow 0$

If we rename the objects as follows :

Object	Name
$[q_0, Z_0, q_0]$	A
$[q_0, Z_0, q_1]$	B
$[q_1, Z_0, q_0]$	C

$[q_1, Z_0, q_1]$ D
$[q_0, 0, q_0]$ E
$[q_0, 0, q_1]$ F
$[q_1, 0, q_0]$ G
$[q_1, 0, q_1]$ H
$[q_0, 1, q_0]$ I
$[q_0, 1, q_1]$ J
$[q_1, 1, q_0]$ K
$[q_1, 1, q_1]$ L

Then the production can be written as :

$S \rightarrow A$
$S \rightarrow B$
$A \rightarrow 0\ EA$
$A \rightarrow 0\ FC$
$B \rightarrow 0\ EB$
$B \rightarrow 0\ FD$
$E \rightarrow 0\ EE$
$E \rightarrow 0\ FG$
$F \rightarrow 0\ EF$
$F \rightarrow 0\ FH$
$E \rightarrow 1\ IE$
$E \rightarrow 1\ JG$
$F \rightarrow 1\ IF$
$F \rightarrow 1\ JH$
$I \rightarrow 1\ II$
$I \rightarrow 1\ JK$
$J \rightarrow 1\ IJ$
$J \rightarrow 1\ JL$
$J \rightarrow 0$
$L \rightarrow 0$
$H \rightarrow 0$
$D \rightarrow \epsilon$
$B \rightarrow \epsilon$

Therefore, the productions become :

$S \rightarrow A \mid B$
$A \rightarrow 0\ EA \mid 0\ FC$
$B \rightarrow 0EB \mid 0FD \mid \epsilon$
$E \rightarrow 0EE \mid 0FG \mid 1IE \mid 1JG$
$F \rightarrow 0EF \mid 0FH \mid 1IF \mid 1JH$
$I \rightarrow 1II \mid 1JK$
$J \rightarrow 1IJ \mid 1JL \mid 0$
$L \rightarrow 0$
$H \rightarrow 0$
$D \rightarrow \epsilon$

Since C, G and K are useless, we get :

$S \rightarrow A \mid B$
$A \rightarrow 0EA$
$B \rightarrow 0EB \mid 0FD \mid \epsilon$
$E \rightarrow 0EE \mid 1IE$
$F \rightarrow 0EF \mid 0FH \mid 1IF \mid 1J\ H$
$I \rightarrow 1II$
$J \rightarrow 1I\ J \mid 1J\ L \mid 0$
$L \rightarrow 0$
$H \rightarrow 0$
$D \rightarrow \epsilon$

As a result of which A, and E becomes useless, therefore, we get :

$S \rightarrow B$
$B \rightarrow 0FD \mid \epsilon$
$F \rightarrow 0FH \mid 1IF \mid 1JH$
$I \rightarrow 1\ I\ I$
$J \rightarrow 1I\ J \mid 1JL \mid 0$
$L \rightarrow 0$
$H \rightarrow 0$
$D \rightarrow \epsilon$

As a result of which I becomes useless, therefore, we get the following productions:

$S \rightarrow B$
$B \rightarrow 0FD \mid \epsilon$
$F \rightarrow 0FH \mid JH$
$J \rightarrow 1JL \mid 0$
$L \rightarrow 0$
$H \rightarrow 0$
$D \rightarrow \epsilon$

Which can be reduced to :

$S \rightarrow B$
$B \rightarrow 0F \mid \epsilon$
$F \rightarrow 0\ F\ 0 \mid J\ 0$
$J \rightarrow 1J\ 0 \mid 0$

CONVERSION OF CFG TO PDA

Q.34. Give steps for conversion of CFG to PDA.

Ans. Steps for conversion of CFG to PDA :

1. If it is required to design PDA from given grammar, the given grammar should be CFG present in GNF form.
2. The grammar will not consists of rule $S \rightarrow \epsilon$ then only it can be converted into PDA.

   For rule :

   $S \rightarrow aA_1 A_2 A_3 ....A_n$

   the generated PDA will be

   $\delta (q_0, a, z_0) \rightarrow (q_1, A_1 A_2 A_3 .....A_n)$

   Example :

   $S \rightarrow a A$

   $\delta (\delta_0, a, z_0) \rightarrow (q_1, A)$

   $S \rightarrow x AB$

   $\delta (q_0, x, z_0) \rightarrow (q_1, AB)$

   $S \rightarrow y ABC$

   $\delta (q_0, y, z_0) \rightarrow (q_1, ABC)$
3. Remaining rules of the context free grammar will use the same method for getting converted into PDA transitions but they will use appropriate stack top symbol and input symbol. For all other rules other than 5 the state will be q_1.
4. If the production rule contains only terminal symbol on the RHS then delete/POP rule of PDA will be followed. This rule will delete the symbol present on stack top.

Q.35. Construct a PDA for a CFG and show the acceptance /rejection of string ω = aabaa .

S → a Sa | b S b | a | b

A → A 0 | 1 0. ***CT : S-10(7M)***

Ans. M = {Q, Σ, Γ , δ, q_0, Z_0, F}

Formal definition :

M = {Q, Σ, Γ , δ, q_0, Z_0, F}

M = ({a}, {a, b}, {S, a, b}, δ, q, S)

1. $S \rightarrow a Sa$

   $\delta (q, a, S) \rightarrow (q, Sa)$
2. $S \rightarrow b S b$

   $\delta (q, b, S) \rightarrow (q, Sb)$
3. $S \rightarrow a$

   $\delta (q, a, S) \rightarrow (q, \epsilon)$
4. $S \rightarrow b$

   $\delta (q, b, S) \rightarrow (q, \epsilon)$

$\delta (q, a a b a a S)$

$\vdash (q, a a b a a , a Sa)$

$\vdash (q, a b a a, Sa)$

$\vdash (q, a b a a, a S aa)$

$\vdash (q, b a a, S aa)$

$\vdash (q, b a a, b aa)$

$\vdash (q, a a , a a)$

$\vdash (q, a a)$

$\vdash (q, \epsilon)$

Hence, the given string W is accepted.

Q.36. Design PDA to accept following CFG.

S → XY

X → AX | BX | a

Y → YA | YB | a

A → a

B → a ***CS : W-09(6M)***

Ans. Formal Definition :

M = {Q, Σ, Γ, δ, q_0, Z_0, F}

M = {{q}, {a}, {Z_0, S, X, Y, A, B}, δ, q, Z_0, q }

1. $S \rightarrow XY$

   $S \rightarrow a Y$

   $\delta (q, a, S) \rightarrow (q, Y)$
2. $X \rightarrow AX$

   $X \rightarrow a X$

   $\delta (q, a, X) \rightarrow (q, X)$
3. $X \rightarrow BX$

   $X \rightarrow a X$

   $\delta (q, a, X) \rightarrow (q, X)$
4. $X \rightarrow a$

   $\delta (q, a, X) \rightarrow (q, \epsilon)$
5. $Y \rightarrow YA$

   $Y \rightarrow a A$

   $\delta (q, a, Y) \rightarrow (q, A)$
6. $Y \rightarrow YB$

   $Y \rightarrow a B$

   $\delta (q, a, Y) \rightarrow (q, B)$
7. $Y \rightarrow a$

   $\delta (q, a, Y) \rightarrow (q, \epsilon)$
8. $A \rightarrow a$

   $\delta (q, a, A) \rightarrow (q, \epsilon)$
9. $B \rightarrow a$

   $\delta (q, a, B) \rightarrow (q, \epsilon)$

Q.37. Construct a PDA equivalent to following. grammar.

S → 0BB

B → 0S | 1S|0.

Also, check whether the given string is accepted by PDA or not?

'010000' OR '010^4'.

CS : S-10(7M), W-10(5M), CT : S-07(5M)

Ans. Formal Definition :

$M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$

$M = \{\{q\}, \{0, 1\} \{Z_0, S, B\} \delta, q, Z_0, q\}$

1. $S \rightarrow 0BB$

   $\delta (q, 0, S) \rightarrow (q, BB)$

2. $B \rightarrow 0S$

   $\delta (q, 0, B) \rightarrow (q, S)$

3. $B \rightarrow 1S$

   $\delta (q, 1, B) \rightarrow (q, S)$

4. $B \rightarrow 0$

   $\delta (q, 0, B) \rightarrow (q, \in)$

5. $\delta (q, 010000, S)$

   $\vdash (q, 0\ 1\ 0\ 0\ 0\ 0, 0\ B\ B)$

   $\vdash (q, 1\ 0\ 0\ 0\ 0, B\ B)$

   $\vdash (q, 1\ 0\ 0\ 0\ 0, 1\ S\ B)$

   $\vdash (q, 0\ 0\ 0\ 0, S\ B)$

   $\vdash (q, 0\ 0\ 0\ 0, 0\ B\ B\ B)$

   $\vdash (q, 0\ 0\ 0, B\ B\ B)$

   $\vdash (q, 0\ 0, 0\ B\ B)$

   $\vdash (q, 0\ 0, B\ B)$

   $\vdash (q, 0\ 0, 0\ B)$

   $\vdash (q, 0, B)$

   $\vdash (q, 0, 0)$

   $\vdash (q, \in)$

6. $\delta (q, 010000, S)$

   $\vdash (q, 0\ 1\ 0\ 0\ 0\ 0, 0\ B\ B)$

   $\vdash (q, 1\ 0\ 0\ 0\ 0, B\ B)$

   $\vdash (q, 1\ 0\ 0\ 0\ 0, 1\ S\ B)$

   $\vdash (q, 0\ 0\ 0\ 0, S\ B)$

   $\vdash (q, 0\ 0\ 0\ 0, 0\ B\ B\ B)$

   $\vdash (q, 0\ 0\ 0, B\ B\ B)$

   $\vdash (q, 0\ 0, 0\ B\ B)$

   $\vdash (q, 0\ 0, B\ B)$

   $\vdash (q, 0\ 0, 0\ B)$

   $\vdash (q, 0, B)$

   $\vdash (q, 0, 0)$

   $\vdash (q, \in)$

Q.38. Construct CFG for the language :

$L = \{0^n\ 1^{2n} \mid n \geq 1\}$

Ans. $L = \{011, 001111, 000111111, \ldots\ldots\}$

Hence CFG is,

$G = (\{S, A, B\}, \{0, 1\}, P, S)$

where P consist of

$S \rightarrow ABB$

$A \rightarrow AABB \mid 0$

$B \rightarrow ABBB \mid 1$

Q.39. Construct CFG for the language

$L = \{a^{3n} \mid n \geq 1\}$ and convert the CFG into CNF.

Ans. $L = \{aaa, aaaaaaaaa, \ldots\ldots\}$

Hence CFG is

$G = (\{A\}, \{a\}, P, S)$

where P consist of

$S \rightarrow Aaaa$

$A \rightarrow AA \mid aaa \mid \in$.

CLOSURE AND DECISION PROPERTIES OF CFLS AND PUMPING LEMMA

Q.40. Explain closure properties of context free languages.

CS : W-12(6M), CT : W-06(6M), S-07(5M)

Ans. Closure properties : Context - free languages are closed under several common operations :

(1) Union :

- Suppose we have grammars for two languages, with start symbols S and T.
- Rename variables as needed to ensure that the two grammars don't share any variables.
- Then construct a grammar for the union of the languages, with start symbol Z, by taking all the rules from both grammars as adding a new rule Z - > S | T.

(2) Concatenation :

- Suppose we have grammars for two languages, with start symbols S and T.
- Rename variables as needed to ensure that the two grammars don't share any variables.
- Then construct a grammar for the union of the languages, with start symbol Z, by taking all the rules from both grammars as adding a new rule Z -> ST.

(3) Star :

- Suppose we have grammars for two language L with start symbols S.
- The grammar for L*, with start symbol T, contains all the rules from the original grammar plus the rule T -> TS | ε.

(4) String reversal :

- Reverse the character string on the righthand side of every rule in the grammar.

(5) Homomorphism :

- Suppose that we have a grammar G for language L and a homomorphism h.
- To construct a grammar for h(L), modify the righthand side of every rule in G to replace each terminal symbol t with its image h(t) under the homomorphism.

(6) Intersection with a regular language :

- The intersection of a context-free language and a regular language is always context-free.
- To show this, assume we have a PDA M accepting the context-free language and a DFA N accepting the regular language.
- Use the product construction to create a PDA which simulates both machines in parallel.
- This works because only M needs to manipulate the stack ; N never touches the stack.

Non-closure facts :

Context-free languages are not closed under set intersection or set complement.

(1) Intersection :

- Consider the languages L_1 and L_2 defined by $L_1 = \{a^n b^n c^j : n, j \geq 0\}$ and $L_2 = \{a^j b^n c^n : n, j \geq 0\}$. They are both context-free.
- However, their intersection is the language $L = \{a^n b^n c^n : n \geq 0\}$. We used the pumping lemma to show that L is not context-free.

(2) Set complement :

- There are two approaches for set complement.
- First, we can use de Morgan's laws to show that closure under set complement plus closure under union would imply closure under intersection.
- Alternatively, there are concrete examples of context-free languages whose closure isn't context free.

Example 1 :

- Consider $L_3 = \{a^i b^j c^k : i != j \text{ or } j != k\}$. It's fairly easy to build a context-free grammar for L_3.
- Suppose its complement L'_3 were context-free. Then $L'_3 \cap a^* b^* c^*$ would be context-free.
- But this language is just $\{a^n b^n c^n : n \geq 0\}$, which we know not to be regular.

Example 2 :

- $L = \{w \neq w : w \text{ in } \{a, b\}^*\}$. We can show that L is not context-free using the pumping lemma. However, it's not too hard to build a context-free grammar for the complement of L.
- Like regular languages, context-free languages are not closed under the subset/superset relationship.

Example 1 : $a^*b^*c^*$ is context-free (in fact regular), but contains the non-context-free subset $a^n b^n c^n$. And $a^n b^n c^n$ contains the context-free subset {ab, aabb, aaabbb}.

Example 2 : $L = \{w \# w : \text{in } \{a,b\}^*\}$. We can show that L is not context-free using the pumping lemma. However, it's not too hard to build a context-free grammar for the complement of L.

Q.41. Give the decision properties of CFL.

Ans. The decision properties of CFL are :

(1) Emptiness.

(2) Finiteness.

(3) Membership.

(1) Emptiness :

Theorem : For a given CFG, there is an algorithm to determine whether a CFL L is empty.

Proof : Given a grammar G for the language L, we decide whether the start symbol S of G is generating. i.e. whether S derives at least one string. L is empty if and only if S is not generating.

1) For each non-terminal N that has some productions of the form

$V_n \rightarrow a$

where a is terminal or string of terminals, we choose one of them and put a in all other productions having V_n at right side, thus eliminating non-terminal V_n altogether.

We have changed the grammar so that it no longer accepts the same language. It may no longer be in CNF. Every word that can be generated from the new grammar could have been generated by the old CFG. If old CFG generated any words, then the new one does also.

(2) Repeat step-1 until it eliminates S or it eliminates no new non-terminals. If S has been eliminated, then the CFG produces some words, if not then not.

Example :

Consider the CFG.

$S \rightarrow AB$

$A \rightarrow CA|CC$

$B \rightarrow DB|DD$

$C \rightarrow a$

$D \rightarrow b$

Solution : First replace all C's by a and D's by b, we get

$S \rightarrow AB$

$A \rightarrow aA|aa$

$B \rightarrow bB|bb$

Next we replace A's by aa and all B's by bb, we get

$S \rightarrow aabb$

Now replace S's by aabb. It is discovered that S has been eliminated so CFG produces at least one word.

(2) Finiteness :

Theorem : For a given CFG, there is an algorithm to determine whether a CFL L generates an infinite language or a finite language.

Proof : Assume that grammar contains no $\in$-productions (if there is any useless or $\in$-production in the grammar then we have to remove those productions) or no unit-production, and no useless symbols. Also assume that grammar has a production like.

$X \Rightarrow xXy$

1) As the grammar is assumed to have no $\in$-production and no unit production, x and y cannot both be simultaneously empty. Since X is neither nullable nor a useless symbol, we have :

$S \Rightarrow uXv \Rightarrow \omega$

and $X \Rightarrow z$

where u, v and z are in V_t. But then

$S \Rightarrow uXv \Rightarrow ux^nXy^nv \Rightarrow ux^nzy^nv$

is possible for all n, so that L(G) is infinite.

If no variable can repeat, then the length of any derivation is bounded by $|V_n|$. In that case, L(G) is finite.

2) We need to determine whether the grammar has some repeating variables, to get an algorithm for determining whether L(G) is finite. This can be done by drawing a dependency graph for the variables in such a way that there is an edge (X, Y) whenever there is a corresponding production

$X \rightarrow xYy$

Then any variable which is at the base of a cycle is a repeating one. Thus, the grammar has a repeating variable if and only if the dependency graph has a cycle. As we have an algorithm for deciding whether a grammar has a repeating variable, we have an algorithm for determining whether or not L(G) is infinite.

3) **Membership :**

Theorem : Membership of a string w in a CFL L can also be decided. Given a CFG and a string w, we can decide whether or not w can be generated by the CFG.

Proof : First convert the grammar from CFG to CNF form and make a list of all the non terminals in the grammar $S, X_1 X_2, X_3 \ldots\ldots$ Let the string we are examining for membership in the language be denoted by

$w = a_1, a_2, a_3 \ldots, a_n$

Now we find the position of every possible substring of w and also decide which substring of w are producible from which non-terminal S. For example, the substring $a_4 \ldots a_6$ can be derived from the non-terminal X_8, the substring $a_7, \ldots a_{10}$ can be derived from the non-terminal X_2, and we have the CNF production

$X_4 \rightarrow X_8 X_2$

then we can conclude that the total substring $a_4 \ldots a_{10}$ can be derived from the non-terminal X_4.

(1) First change the CFG into CNF if it is not already in CNF form.

(2) Considered all the substring of length 1 of w and for such derivations from the CNF production non terminal → terminal.

Substring	All producing non-terminals
a_1	X...
a_2	.
.	.
a_n	X....

(3) Now consider all substrings of length 2, such as $a_1 a_2$. This can be produced by a non-terminal X_p if the first half can be produced by some non-terminal X_q and the second half by some non-terminal X_r, and there is a rule of production in the grammar that says $X_p \rightarrow X_q X_r$. We can check all the rules of production to determine whether the length-2 substring can be produced.

Substring	All producing non-terminals
$a_1 a_2$	X...
$a_2 a_3$	.
..	.
$a_{n-1} a_n$	X....

(4) Next consider all the substrings of length three for example $a_1 a_2 a_3$ as

Substring	All producing non-terminals
$a_1 a_2 a_3$	X...
$a_2 a_3 a_4$	.
..	.
$a_{n-2} a_{n-1} a_n$	X....

(5) Examine all substrings of length 4 and break them into parts of three different strings as :

(a) The first three letters and the last letter.

(b) The first two letters and the last two letters.

(c) The first letter and the last three letters.

For all these possibilities, check the list to whether nonterminals produce these substrings and whether the non-terminals can be merged into one by a rule of production.

$X \rightarrow XX$

Substring	All producing non-terminals
$a_1 a_2 a_3 a_4$	X...
$a_2 a_3 a_4 a_5$	.
..	.
$a_{n-3} a_{n-2} a_{n-1} a_n$	X....

We continue the same process with substring of length 5, length 6 and so on. The whole process terminates when we have all of w as the length of the substring as :

Substring	All producing non-terminals
$a_1 a_2 \ldots\ldots a_n$	N

Example :

Consider the following CFG

$S \rightarrow AA$

$A \rightarrow AA$

$A \rightarrow a$

Solution :

The above grammar is in CNF. Now consider the substring of length 1 as :

Substring	All producing non-terminals
$a_1 = a$	A
$a_2 = a$	A
$a_3 = a$	A

The substrings of length 2 are the same, aa, and are factorable into exactly AA. This is the right side of two productions whose left sides are S and A.

Substring	All producing non-terminals
$a_1\ a_2 = aa$	S, A
$a_2\ a_3 = aa$	S, A

There is only one substring of length-3, and it can be factored into a_1 times a_2a_3 or a_1a_2 times a_3. The first case gives the non-terminal possibilities. AS or AA, and the second gives SA or AA. But only AA is the right side of a production. The left sides are S and A. Thus, the list concludes with the following :

Substring	All producing non-terminals
$w = a_1\ a_2\ a_3$	S, A

From the list, we see that the word w can be derived from the start symbol S and so it is in the language.

Q.42. Prove that pumping lemma, the language L is not context free language. $L_1 = \{a^i\ b^i\ c^i \mid i \geq 1\}$

CT : S-06(7M)

Ans.

- Consider the language $L_1 = \{a^i\ b^i\ c^i \mid i \geq 1\}$. Suppose L were context free and n be the constant.
- Consider $Z = a^n b^n c^n$ .Write z = uvwxy so as to satisfy the conditions of the pumping lemma. Since $|v\ w\ x| \leq n$, it is not possible for vx to contain instances of a's and c's, because the rightmost a is n +1 positions away from the leftmost c.
- If v and x consist of a's only, then uwy (the string uv^iwx^iy with i = 0) has n b's and n c's but fewer than n a's, since $|vx| \geq 1$.
- Thus, uwy is not of the form $a^jb^jc^j$. But the pumping lemma vwy is in L_1, a contradiction.
- The cases where v and x consist only of b's or only of c's are disposed of similarly.
- If vx has a's and b's, then uwy has more c's than a's or b's and again it is not in L_1. If vx contains b's and c's, a similar contradiction results.
- We conclude that L_1 is not a context free language.

POINTS TO REMEMBER :

(1) A pushdown automata is an extension of finite automata, with control of both an input tape and a stack.

(2) A PDA may accept either by empty stack or by final state.

(3) Acceptance by empty stack and acceptance by final state are equivalent, because for every PDA accepting by empty stack, there exists an equivalent PDA accepting by final state and vice-versa.

(4) For every context-free language there exists an NDPDA, but for every context-free language there does not exist a DPDA. That means there are some context-free languages for which there exists no DPDA, they can be recognized only by NDPDA.

(5) For every context-free language there exists an equivalent single state NDPDA accepting by empty stack, therefore, for every NDPDA accepting by empty stack there exists an equivalent single state NDPDA accepting by empty stack.

UNIT - V
TURING MACHINES

SYLLABUS

Turing machines, TM as acceptor, TM as transducers, Variations of TM, Linear bounded automata, TM as computer of function.

UNIVERSITY PAPER SOLUTIONS
SINCE SUMMER - 2006

Note to the students :
- The questions given below are from previous Nagpur University examination papers.
- Though these questions are from old university papers, they have been included in the new syllabus.
- Questions which are out of new syllabus are not included here.

SUMMER - 06 (CT)

Q.1. Explain with the help of mathematical representation, what is Turing Machine ? **5M**

Ans. P.5-7, Q.1.

Q.2. Design Turing machine that copies strings of 1's without loosing original string. **8M**

Ans. P.5-37, Q.46.

Q.3. Is following grammar LR (0) **8M**

$S \rightarrow Aa \mid bAc \mid Bc \mid bBa$

$A \rightarrow d$

$B \rightarrow d$

Ans. P.5-51, Q.58.

Q.4. Write short notes on Universal Turing Machine. **4M**

Ans. P.5-27, Q.26(A).

Q.5. Given an arbitrary Turing machine M. Prove that the problem "whether language accepted by M is a regular set" is undecidable.. **7M**

Ans. P.5-24, Q.22.

Q.6. Write short note on Church's Hypothesis. **5M**

Ans. P.5-30, Q.31.

WINTER - 06 (CT)

Q.1. What is a Turing machine ? Explain the basic model of Turing machine in detail. **5M**

Ans. P.5-7, Q.1.

Q.2. Design a Turing machine to accept the language.

$L = \{ 0^n 1^n \ / n \geq 1 \}$

Clearly state any assumptions you make. Explain briefly the operation of your machine. **9M**

Ans. P.5-14, Q.9.

Q.3. Construct the nonempty sets of items for the following LR(0) grammar :

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$

$E \rightarrow (E)$ **9M**

Ans. P.5-55, Q.64.

Q.4. Explain the model of linear bounded automata. **4M**

Ans. P.5-34, Q.40.

SUMMER - 07 (CT)

Q.1. Design Turing machine to recognize the language.

$L = \{ 0^n 1^n 0^n / n \geq 1 \}$ **8M**

Ans. P.5-15, Q.10.

Q.2. What are the various types of Turing Machine? Explain each type in brief. **8M**

Ans. P.5-21, Q.19.

Q.3. Construct Turing machine for computing
(i) Successor Function. (ii) The zero function Z. **7M**

Ans. P.5-11, Q.5.

Q.4. What do you mean by computable function? "Turing Machine computes function of one, function of two and function of multiple arguments" Comment on this statement. **5M**

Ans. P.5-36, Q.43(B).

Q.5. Design T.M. to find 2's complement of a given binary number. **6M**

Ans. P.5-37, Q.45.

Q.6. Explain typical encoding scheme used to code symbols on universal Turing Machine. **5M**

Ans. P.5-27, Q.26(B).

WINTER - 07 (CT)

Q.1. Explain the model of Turing machine in brief. Give its formal definition. **5M**

Ans. P.5-7, Q.1.

Q.2. Design a Turing machine that can accept the string over {0, 1} to recognize all strings containning even number of 1's. **8M**

Ans. P.5-36, Q.44.

Q.3. In how many ways can we represent Turing machine model? Explain each alternative with suitable example. **8M**

Ans. P.5-8, Q.3.

Q.4. Write short note on Universal Turing machine. **4M**

Ans. P.5-27, Q.26(A).

Q.5. Obtain LR (0) parsing table for the grammar :

$E \rightarrow E + T / T$

$T \rightarrow T * F/F$

$F \rightarrow F * /a / b$ **10M**

Ans. P.5-53, Q.62.

Q.6. Explain the model of linear bounded automata. **5M**

Ans. P.5-34, Q.40.

SUMMER - 08 (CT)

Q.1. Collect LR (0) items for the following grammar.

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$ **8M**

Ans. P.5-48, Q.57.

Q.2. In how many ways can we represent Turing machine Model? Explain each alternative with suitable example. **8M**

Ans. P.5-8, Q.3.

Q.3. Design a Turing Machine that can accept the string over {0, 1} to recognize all string containning even no of 1's. **5M**

Ans. P.5-36, Q.44.

Q.4. Given an arbitrary Turing machine M. Prove that the problem "whether language accepted by M is a regular" is undecidable. **7M**

Ans. P.5-24, Q.22.

WINTER - 08 (CT)

Q.1. Explain concept of Universal Turing Machine. **5M**

Ans. P.5-27, Q.26(A).

Q.2. Design TM for language given below. Explain briefly the operation of TM.

$L = \{ 0^n 1^n 2^n / n \geq 1 \}$. **9M**

Ans. P.5-15, Q.10.

Q.3. Design TM for language to find 2's complement of a binary number. **7M**

Ans. P.5-37, Q.45.

Q.4. Explain Model of linear bounded Automata. **3M**

Ans. P.5-34, Q.40.

Q.5. Construct LR (0) parsing table for following grammar. **5M**

$S \rightarrow CC$

$C \rightarrow aC$

$C \rightarrow d$

Ans. P.5-51, Q.59.

Q.6. Write short note on Church's Hypothesis. **3M**

Ans. P.5-30, Q.31.

SUMMER - 09 (CS)

Q.1. Construct LR (0) parser for following grammar :

$E \rightarrow E + T / T$

$T \rightarrow T * F /F$

$F \rightarrow (E) / id$

Also show the parsing of string id + id * id

Also define

(1) Top down parsing

(2) Bottom up parsing **12M**

Ans. P.5-33 & 49, Q.57 & 38.

Q.2. Design a TM (Turing Machine) to perform 2's complement of binary input. **8M**

Ans. P.5-37, Q.45.

Q.3. Design a Turing Machine, to operate on UNARY number and perform addition of UNARY number. The TM will perform addition by input restore method. **8M**

Ans. P.5-38, Q.47.

Q.4. Write short note on : **8M**

(i) Multitrack Turing Machine.

(ii) Multi tape Turing Machine.

Ans. P.5-21 & 23, Q.19 & 20.

SUMMER - 09(CT)

Q.1. Construct the nonempty sets of items for the following LR (0) grammar.

$E \rightarrow E + T/T$

$T \rightarrow T * F/F$

$F \rightarrow id$ **9M**

Ans. P.5-52, Q.60.

Q.2. Design a Turing machine over (1, b) which can compute concatenation function over $\Sigma = \{1\}$ i.e. if a pair of words $\{w_1, w_2\}$ is input, the output has to be $w_1 w_2$. Assume b as separator of w_1 and w_2. **8M**

Ans. P.5-16, Q.12.

Q.3. Design TM to check well formedness of the parenthesis. **8M**

Ans. P.5-16, Q.11.

Q.4. Write short note on Universal Turining Machine. **5M**

Ans. P.5-27, Q.26(A).

Q.5. Write short note on Church's Hypothesis. **5M**

Ans. P.5-30, Q.31.

WINTER - 09 (CS)

Q.1. For following grammar, generate various state and design DFA using LR(0) method.

$E \rightarrow E + T / T$

$T \rightarrow T * F / F$

$F \rightarrow (E) / id.$ **10M**

Ans. P.5-49, Q.57.

Q.2. Explain left factoring with example. **3M**

Ans. P.5-33, Q.39.

Q.3. Design TM for $L = \{ W W^R / W \in (a, b)^* \}$.

W^R - reverse of W. Draw transition table. **7M**

Ans. P.5-18, Q.15.

Q.4. Design a TM to perform addition of two unary number. **6M**

Ans. P.5-38, Q.47.

Q.5. Using Multitrack tape TM, design an algorithm to check whether number present on TAPE is PRIME. Explain multitape turing machine. **7M**

Ans. P.5-21 & 23, Q.19 & 21.

Q.6. Explain blank tape halting problem. **5M**

Ans. P.5-26, Q.24.

WINTER - 09 (CT)

Q.1. Explain the model of Turing m/c in brief. Give its formal definition. **5M**

Ans. P.5-7, Q.1.

Q.2. Design a T.M. to check well formedness of parenthesis. **8M**

Ans. P.5-16, Q.11.

Q.3. Construct a T.M. that accept the following language :

$L = \{ a^n b^n c^n / n \geq 1 \}$ **8M**

Ans. P.5-15, Q.10.

Q.4. Write a short note on Universal T.M. **3M**

Ans. P.5-27, Q.26(A).

Q.5. Explain the model of linear bounded automata. **5M**

Ans. P.5-34, Q.40.

SUMMER - 10 (CS)

Q.1. Prove that empty tape problem is unsolvable **6M**

Ans. P.5-26, Q.24.

Q.2. Design TM to find 2's complement of binary number. **7M**

Ans. P.5-37, Q.45.

Q.3. Construct a TM for language

$L = \{ w C w / w \in (a + b)^* \}$ **7M**

Ans. P.5-44, Q.51.

Q.4. Write short note on :

(1) Multitape Turing Machine.

(2) Multidimensional Turing Machine. **6M**

Ans. P.5-21, Q.19.

Q.5. Design TM to add two given integers. Also show the transition graph and transition table. Illustrate your design with suitable sequence of i/ps with help of ID's. **9M**

Ans. P.5-38, Q.47.

SUMMER - 10 (CT)

Q.1. Design a TM that can compute proper subtraction i.e. m = n, where m and n are + ve integer.
m – n is defined as m – n if m > n
and 0 if m < = n. **8M**

Ans. P.5-40, Q.48.

Q.2. Explain Linear Bounded Automata. **5M**

Ans. P.5-34, Q.40.

Q.3. Draw LR (0) parsing table for the grammar
$S \rightarrow Aa\ Ab$
$S \rightarrow Bb\ Ba$
$A \rightarrow \in$
$B \rightarrow \in$ **13M**

Ans. P.5-48, Q.56.

Q.4. Write short note on Universal T.M. **3M**

Ans. P.5-27, Q.26(A).

WINTER - 10 (CS)

No question asked in University Exam.

WINTER - 10 (CT)

Q.1. Describe the power of Turing Machine over finite state machine. **4M**

Ans. P.5-29, Q.30(A).

Q.2. Design a TM that replaces all occurence of '111' by '101' from a sequence of 0's and 1's. **9M**

Ans. P.5-17, Q.13.

Q.3. Construct a Turing Machine to accept the following language.
$L = \{ W / W \in (a, b)^*, n_a(\omega) = n_b(\omega) \}$
where,
$n_a(\omega)$ = Number of a's in W.
$n_b(\omega)$ = Number of b's in W. **8M**

Ans. P.5-17, Q.14.

Q.4. Construct the nonempty sets of items for the following LR(0) grammar.
$E \rightarrow E + T / T$
$T \rightarrow T * F / F$
$F \rightarrow id$ **8M**

Ans. P.5-52, Q.60.

Q.5. Write short notes on Multitape Turing Machine. **4M**

Ans. P.5-21, Q.19.

SUMMER - 11 (CS)

Q.1. Define TM. Construct a TM to recognize palindrome string over {a, b} with C as separator. **7M**

Ans. P.5-7 & 44, Q.1 & 51.

Q.2. Design a TM to perform addition of two unary numbers. **8M**

Ans. P.5-38, Q.47.

Q.3. Construct LR (0) items for the grammar given. Find its equivalent DFA. Check the parsing by taking a suitable derived string. **10M**
$S' \rightarrow S$
$S \rightarrow AS / \in$
$A \rightarrow aA/b$

Ans. P.5-57, Q.65(A).

Q.4. Explain difference between top down and bottom up parsing approach. **3M**

Ans. P.5-33, Q.38.

Q.5. Show that halting problem is unsolvable. **2M**

Ans. P.5-28, Q.27.

Q.6. Describe the linear bounded automata and the language it accepts. **5M**

Ans. P.5-34, Q.40.

SUMMER - 11 (CT)

Q.1. Design a TM to accept the following language
$L = \{ a^n\ b^{2n}\ c^n / n \geq 1 \}$ **13M**

Ans. P.5-19, Q.16.

Q.2. Show that the grammar $G : S \rightarrow a\ A$
$A \rightarrow Abb / b$ is an LR (0) grammar. **4M**

Ans. P.5-58, Q.66.

Q.3. Write a short notes on **9M**
(1) Universal TM.
(2) Linear Bounded Automata.

Ans. P.5-27, Q.26(A) & 46.

WINTER - 11 (CS)

Q.1. Construct LR (0) parser for the following grammar :

$S \rightarrow CC$

$C \rightarrow cC$

$C \rightarrow d$ **9M**

Ans. P.5-51, Q.59.

Q.2. Construct a Turing machine accepting the language

$L = \{ a^n b^{2n} c^n / n \geq 1 \}$ **13M**

Ans. P.5-19, Q.16.

Q.3. Show that halting problem of Turing Machine is undecidable. **7M**

Ans. P.5-28, Q.27.

Q.4. Write short note on : **14M**

(i) Universal Turing Machine.

Ans. P.5-27, Q.26(A).

(ii) Linear Bounded automata

Ans. P.5-34, Q.40.

(iii) Church Turing Thesis.

Ans. P.5-30, Q.31.

WINTER - 11 (CT)

Q.1. Design Turing machine to perform multiplication of two numbers. **11M**

Ans. P.5-41, Q.49.

Q.2. Explain the model of Turing machine. **3M**

Ans. P.5-7, Q.1.

Q.3. Write short note on Church's Hypothesis. **4M**

Ans. P.5-30, Q.31.

Q.4. Construct LR (0) parsing table for following grammar.

$S \rightarrow |S| / 0S0 / 10$ **9M**

Ans. P.5-47, Q.55.

Q.5. Show that halting problem of TM is unsolvable. **6M**

Ans. P.5-28, Q.27.

SUMMER - 12 (CS)

Q.1. Design turing machine to accept the language, $L = \{w : |w| \text{ is multiple of } 3\}$. Also draw transition diagram.

Ans. P.5-43, Q.50.

Q.2. Design turing m/c to calculate multiplication of two integer numbers. **9M**

Ans. P.5-41, Q.49.

Q.3. What do you understand by multitape turing machine? **4M**

Ans. P.5-21, Q.19.

Q.4. Show that there is no algorithm for deciding if any two turing machine M_1 and M_2 accept the same language. **7M**

Ans. P.5-26, Q.25.

SUMMER - 12 (CT)

Q.1. Construct the non empty sets of items for the following LR(0) grammar :

$E \rightarrow E + T / T$

$T \rightarrow T * F/F$

$F \rightarrow id$ **8M**

Ans. P.5-52, Q.60.

Q.2. Describe the power of TM over push down automata. **2M**

Ans. P.5-30, Q.30(B).

Q.3. Discuss various types of TM models. **7M**

Ans. P.5-21, Q.19.

Q.4. Design a TM to compute 2's complement of binary number. **6M**

Ans. P.5-37, Q.45.

Q.5. Write short note on : **9M**

(1) Universal TM

(2) Church's Hypothesis.

(3) Linear Bounded automata

Ans. P.5-27, 30 & 34, Q.26(A), 31 & 40.

Q.6. Design TM to recognize words in

$L = \{ a^n b^n a^n / n \geq 1 \}$ **7M**

Ans. P.5-15, Q.10.

WINTER - 12 (CS)

Q.1. Design a Turing machine that recognizes the presence of substring '101' and replaces it with '110'. **7M**

Ans. P.5-17, Q.13.

Q.2. Design TM to compute.

$$f(m, n) = \begin{cases} m - n & \text{if } m > n \\ 0 & \text{if } m \leq n \end{cases}$$ **7M**

Ans. P.5-40, Q.48.

Q.3. Prove that the halting problem of TM is undecidable. **6M**

Ans. P.5-28, Q.27.

Q.4. Explain various modification of TM **5M**

Ans. P.5-21, Q.19.

WINTER - 12 (CT)

Q.1. Design TM that finds integer division i.e. $\left(\text{int}\left(\frac{m}{n}\right)\right)$ of two number m and n, where $(m \geq n)$ **9M**

Ans. P.5-43, Q.50.

Q.2. Obtain LR (0) parsing table for the following grammar

$S \rightarrow a\ BDh$

$B \rightarrow cA$

$A \rightarrow b\ A/\epsilon$

$D \rightarrow EF$

$E \rightarrow g/\epsilon$

$F \rightarrow f/\epsilon$. **10M**

Ans. P.5-58, Q.67.

Q.3. Write short note on linear Bounded Automata. **3M**

Ans. P.5-34, Q.40.

Q.4. Write short note on Universal Turing Machine. **4M**

Ans. P.5-27, Q.26(A).

Q.5. Give mathematical definition of Turing machine. Discuss various types of TM. **5M**

Ans. P.5-7 & 21, Q.1 & 19.

Q.6. Design a TM for the given language.

$L = \{ w\ C\ \bar{w} / w \in (a + b)^* \}$ **8M**

Ans. P.5-44, Q.51.

SUMMER - 13 (CS)

Q.1. Design turing machine to calculate proper subtraction of two integer number. **8M**

Ans. P.5-39, Q.48.

Q.2. What do you mean by non-deterministic turing machine? **5M**

Ans. P.5-21, Q.19.

Q.3. Design turing machine to accept all strings of language $L = \{0^n\ 1^m\ 0^n / m, n \geq 1\}$

Also draw transition diagram. **7M**

Ans. P.5-20, Q.17.

Q.4. What do you understand by multitape turing machine? **4M**

Ans. P.5-21, Q.19.

SUMMER - 13 (CT)

Q.1. Explain working of multitrack model of TM. **6M**

Ans. P.5-23, Q.20.

Q.2. Design a TM for proper subtraction of two integer number. **8M**

Ans. P.5-39, Q.48.

Q.3. Prove that digital computer is equivalent to turing machine. **5M**

Ans. P.5-25, Q.23.

Q.4. Write a short note on Church's Hypothesis. **4M**

Ans. P.5-30, Q.31.

SOLVED QUESTION BANK

[Sequence given as per syllabus]

INTRODUCTION

The limitation of FSM motivates us to look for a machine more powerful than FSM. As we have seen, FSM cannot remember an webifrurily long sequence of symbol, because the head can move only in one direction i.e. to right always. It cannot come back to retrieve the information stored on the tape, because of which, it cannot multiply two number and it cannot check for well-formedness of parentheses as well as for palindrome sequences. Above all characteristics or requirements are satisfied by a machine called as Turing machine which was proposed by Turing, a decade prior to designing of first shared-program computer. This concept of Turing machine had given rise to concept of algorithm and finite procedures.

TURING MACHINES

Q.1. Explain Model of Turing Machine.

CT : W-11(3M)

OR Define TM. **CS : S-11(2M)**

OR What is Turing Machine? Explain the basic model of Turing Machine in details.

CT : W-06(5M)

OR Explain the model of Turing Machine in brief. Give its formal definition. **CT : W-07,09(5M)**

OR Explain with the help of mathematical representatnion, what is Turing Machne?

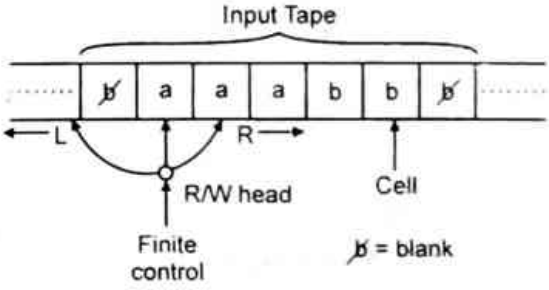
CT : S-06(5M)

OR Give mathematical foundation of turing machine. **CT : W-12(1M)**

Ans. Turing Machine :

- Turing machine is capable of performing all the computation which computer can perform.
- Turing machine has a finite control on input tape, that is divided into cells and a tape head that scans one cell at a time and can be move to left as well as right.
- The input of length 'n' whereas the remaining infinity of cells hold a special symbol called as blank symbol denoted by 'B' or ' $\flat$ '.
- The move of a turing machine depends on the present state of the finite control and the symbol scanned by tape head.

Block diagram :



Formal definition of Turing machine :

Turing machine consist of following tuples :

$TM = \{Q, \Sigma, \Gamma, \delta, q_0, F, \flat\}$

Where,

$Q \rightarrow$ Finite set of states

$\Sigma \rightarrow$ Input symbols

$\Gamma \rightarrow$ Symbols permitted on tape

$\delta \rightarrow$ Transition.

$q_0 \rightarrow$ Initial symbol.

$F \rightarrow$ Final state

$\flat \rightarrow$ Last symbol on stack.

Q.2. Explain Transition format for Turing Machine.

Ans. Turing Machine should provide following information :

Change of state based on state changed rules :

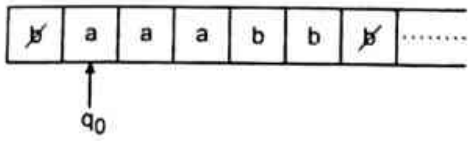
(a) Change of symbol on the tape.

(b) Change in the direction of R/W head.

- **Input Transitions :**
  It will provide information about current state and current input symbol.
- **Output Transitions :**
  It will provide information about next state, change in the symbol, direction of the R/W head.
- **Transition :**
  δ (Current state, i/p symbol) $\rightarrow$ (next state, O/P symbol, Direction (R or L))

Operation :

(1) Insert Operation :

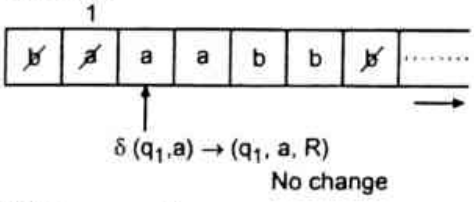


$\therefore \delta(q_0, a) \rightarrow (q_1, 1, R)$

$\delta(q_1, b) \rightarrow (q_2, 0, L)$

(2) Skip Operation :

This operation will permit to skip the symbol present on tape. There will be no change to the symbol and R/W head moves in the forward direction.



(3) Delete operation :

It will change the symbol on tape and replace it with blank.

$\delta(q_0, a) \rightarrow (q_1, b, R)$

Q.3. In how many ways can we represent Turing Machine Model? Explain each alternative with suitable example. ***CT : W-07, S-08(8M)***

Ans. Representation of Turing machine :

A Turing machine can be represented or describe by following three ways :

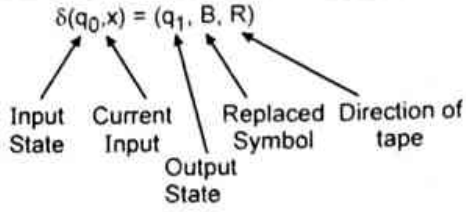
(1) Instantaneous descriptions using move-relation.

(2) Transition diagram.

(3) Transition table.

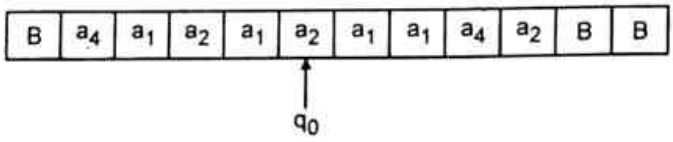
(1) Instantaneous Description Using Move :

- The arrangement of a Turing machine depends on the present state of the finite control and the next input symbol, which depends on the string written on an input tape at that instance of time.
- If the current state is q_0, and the input or the next symbol appearing on the tape is x, and if the Turing machine changes its state to state q_1 in the right direction and replaces the input symbol x by B (blank symbol) then it can be represented by following notation :

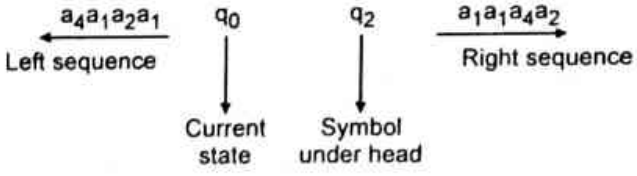


- An instantaneous descriptions of a Turing machine M is string $aq\gamma$, where q is the present state of M, the entire input string is split as $\alpha\gamma$, the first symbol of γ is the current symbol a under the R/W head and γ has all the subsequent symbols of the input string, and the string α is the substring of the input string formed by all the symbols to the left of a.

Example :



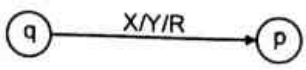
- The present symbol under the R/W head is a_2. The present state is q_0. Thus a_2 is written to the right of q_0.
- The nonblank symbols to the left of a_2 form the string $a_4\ a_1\ a_2\ a_1$ is written to the left of q_0.
- The sequence of nonblank symbols to the right of a_2 is $a_1\ a_1\ a_4\ a_2$.
- The instantaneous descriptions is given as :



(2) Transition Diagram :

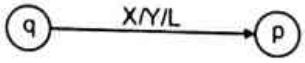
- A Transition Diagram of the next-move functions δ of a TM is graphical representation consisting of a finite number of nodes and directed-labelled arcs between the nodes.
- Each node represents a state of the TM and a label on an arc from one state to a state represents the information about the required input symbol, the transition and the action on the part of the control of TM.

- We now represent a move $\delta(q, X) = (p, Y, R)$ graphically as follows :



where X/Y specify that X is to be replaced by Y in the type cell scanned, and R specify that the tape head is to be moved one cell right, and an arrow from q to p, specify that state is to be changed from p to q.

- And a move $\delta(q, X) = (p, Y, L)$ is represented graphically as follows :



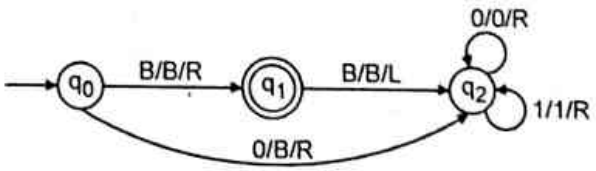
Where X/Y specify that X is to be replaced by Y in the tape cell scanned, and L specify that the tape head is to be moved one cell left, and an arrow from q to p, again specify that state is to be changed from p to q.

(3) Transition Table :

- A transition table is tabular representation of a function like δ that takes two arguments and returns a state, replace symbol and tape direction.
- The rows of the table correspond to the states, and the columns correspond to the input symbols along with B (blank symbol).

Example :

For the transition system shown in figure can be represented in tabular form as shown below.



Present State	0	1	B
$\rightarrow q_0$	(q_1, B, R)	-	(q_2, B, R)
q_1	$(q_0, 0, R)$	$(q_1, 1, R)$	(q_2, B, L)
(q_2)	-	-	-

The initial start are mark with the ($\rightarrow$) arrow symbol, which indicate the initial state and the final state is represented by drawing the circle on the final state.

Example :

Construct a Turing machine accepting

$L = \{0^n 1^n \mid n \geq 0\}$ or $L = \{a^n b^n \mid n \geq 0\}$

Solution :

- The given language L will accept the set of string which consists of 0's followed by 1's.
- The number of 0's and 1's should be same and the minimum number of 0's and 1's are zero or more.
- Some strings accepted by the above machine are as follows :

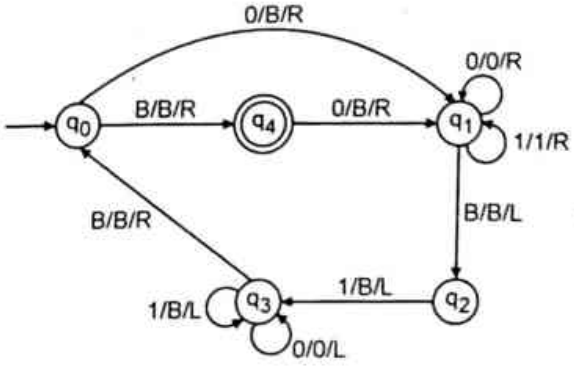
  $L = \{ \varepsilon, 01, 0011, 000111, 00001111 \ldots\ldots\}$
- When the Turing machine will get the first input as 0, it remember leftmost 0, by replacing with B.
- Then it moves the tape head in right direction keeping the symbols (0 or 1) it scans as it is, until it gets rightmost 1, it remembers rightmost 1, by replacing it with B, and moves the tape head in left direction keeping the symbols it scans as it is till reaches the B, on getting B, it moves the tape head one position right and repeats the above cycle if it gets 0.
- If it gets B instead of 0, then the string is accepted, thus the Turing machine enters into the final state.
- The transition table shows the moves of the Turing machine :

Present State	0	1	B
$\rightarrow q_0$	(q_1, B, R)	-	(q_4, B, R)
q_1	$(q_0, 0, R)$	$(q_1, 1, R)$	(q_2, B, L)
q_2	-	(q_3, B, L)	-
q_3	$(q_3, 0, L)$	(q_3, B, L)	(q_0, B, R)
(q_4)	-	-	-

Thus, the Turing machine M is

$M = (\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \{a, b, B\}, \delta, q_0, B, \{q_4\})$

- The transition diagram corresponding to the above table is shown in following figure.



Q.4. Explain different techniques used in the design of Turing machine.

Ans. Following important techniques are used in the design of Turing machine :

(1) Storage in the finite control.
(2) Multiple tracks.
(3) Checking off symbols.
(4) Shifting over.
(5) Subroutines.

(1) Storage in the finite control :

- Finite control is used to hold a finite amount of information. For this, the state is written as a pair of elements, one exercising control and the other storing a symbol.

  Example : Consider a Turing machine M that looks at the first input symbol, records it in its finite control, and checks that the symbol does not appear elsewhere on its input. Here, M accepts the regular set.

  $M = (Q, \{0, 1\}, \{0, 1, B\}, \delta, \{q_0, B\}, B, F)$,

  where Q is $\{q_0, q_1\} \times \{0, 1, B\}$.

  That is Q consists of pairs $[q_0, 0], [q_0, 1], [q_0, B], [q_1, 0], [q_1, 1]$, and $[q_1, B]$. The set F is $\{(q_1, B)\}$.

- The intention is that the first component of the state controls the action, while the second component "remembers" a symbol.

  So, define δ as follows :

(i) **(a)** $\delta([q_0, B], 0) = ([q_1, 0], 0, R)$

(b) $\delta([q_0, B], 1) = ([q_1, 1], 1, R)$

- Initially, q_0 is the control component of the state, and M moves right.
- The first component of M's state becomes q_1 and the first symbol seen is stored in the second component.

(ii) **(a)** $\delta([q_1, 0], 1) = ([q_1, 0], 1, R)$,

(b) $\delta([q_0, 1], 0) = ([q_1, 1], 0, R)$

If M has a zero stored and sees a 1 or vice versa, then M continues to move to the right.

(iii) **(a)** $\delta([q_1, 0], B) = ([q_1, B], 0, L)$

(b) $\delta([q_1, 1], B) = ([q_1, B], 0, L)$

- M enters the final state $[q_1, B]$, if it reaches a blank symbol without having first encountered a second copy of the leftmost symbol.
- If M reaches a blank in state $[q_1, 0]$ or $[q_1, 1]$, it accepts. For state $[q_1, 0]$ and symbol 0 or for state $[q_1, 1]$ and symbol 1, δ is not defined.
- Thus, if M encounters the tape symbol stored in its state, M halts without accepting.
- In general, we can allow the finite control to have k components, all but one of which store information.

(ii) Multiple tracks :

- It is assumed that the tape of the Turing machine is divided into k tracks, for any finite k.

  This arrangement is shown in figure. for k = 2.
- The symbols on the tape are considered k tuples, one component for each track.

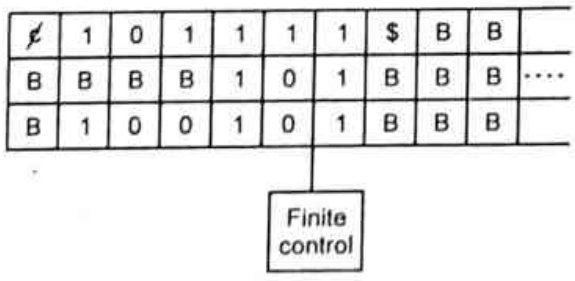


Fig. A three track turing machine

- The tape in figure shows a Turing machine that takes a binary input greater than 2, written on the first track and determines whether it is a prime.
- The input is surrounded by ¢ and $ on the first track. Thus, the allowable input symbols are $[¢, B, B], [0, B, b], [1, B, B]$ and $[\$, B, B]$.

- These symbols can be identified with ¢, 0, 1, and respectively, when viewed as input symbols. The blank symbols can be identified with [B, B, B].

(iii) Checking off symbols :

- Checking off symbols is a useful trick for visualizing how a TM recognizes languages defined by repeated strings, such as $\{ww \mid w \text{ in } \Sigma^*\}$, $\{wcy \mid w \text{ and } y \text{ in } \Sigma^* \; w \neq y\}$ or $\{ww^R \mid w \text{ in } \Sigma^*\}$
- It is also useful when lengths of substrings must be compared, such as in the languages $\{a^i b^j \mid i \geq 1\}$ or $\{a^i b^i c^k \mid i \neq j \text{ or } j \neq k\}$
- We introduce an extra track on the tape that holds a blank or w.
- The ✓ appears when the symbol below it has been considered by the TM in one of its comparisons.

(iv) Shifting over :

- It is possible for a Turing machine to make space on its tape by shifting all non blank symbols a finite number of cells to the right.
- For doing this, the tape head makes an excursion to the right, repeatedly storing the symbols read in its finite control and replacing them with symbols read from cells to the left.
- The TM can then return to vacated cells and print symbols of its choosing.
- If space is available, it can push blocks of symbols left in similar manner.

(v) Subroutines :

- A Turing machine can simulate any type of subroutine found in programming languages, including procedures and any of the known parameter passing mechanisms.
- The general idea is to write part of a TM program to serve as a subroutine, it will have a initial state and a designated return state which termporarily has no move and which will be affect a return to the calling routine.
- To design a TM that calls the subroutine, a new set of states the subroutine is made and a move from the return state is specified.
- The call is affected by entering the initial state for the subroutine and the return is affected by the move from the return state.

Q.5. Construct the Turing machine for computing :
(i) Successor function. **CT : S-07(7M)**
(ii) The zero function Z.

Ans.

(1) Turing machine for computing successor function :

- The successor function S is defined by $S(a_1) = a_1 + 1$, for all $a_1 \geq 0$.
- So the initial tape expression can be taken as $X = 1^{a1} X_1 \, ß \, y$.
- At the end of computation, we require 1^{a1+1} to appear to the left of y. Hence we define a TM by taking

  $Q = \{q_{01} \ldots\ldots q_9\}$, $\Gamma = \{ß, 1, X_1, Y\}$

  $X = 1^{a1} X_1 \, ß \, Y$

  where P consist of the production. Their corresponding operations are explained as follows.

(i) $q_0 \, ß \, R \, q_0$, $q_0 \, 1 \, ß \, q_1$, $q_0 \, X_1 \, R \, q_6$:

- If M starts from initial ID, the head replaces the first 1 it encounters by ß.
- Afterwords the head moves to the right until it encounters Y.

(ii) $q_1 \, ß \, R \, q_1$, $q_1 \, 1R \, q_1$, $q_1 \, X_1 \, Rq_1$, $q_1 \, Y_1 \, q_2$:

- Y is replaced by 1 and M enters q_2.
- Once the end of the input tape is reached, y is added to next cell. M enters q_3.

(iii) $q_2 \, 1R \, q_2$, $q_2 \, ß \, Y \, q_3$:

The head moves to the left and the state is not changed until X_1 is encountered.

(iv) $q_3 \, ß \, L \, q_3$, $q_3 \, 1 \, L \, q_3$, $q_3 \, YL \, q_3$, $q_3, X, L \, q_4$:

- On encountering X_1, the head moves to the left and M enters q_4.
- Once again the head moves to the left till the left end of the input string is reached.

(v) $q_4\ 1L\ q_4, q_4\ b\ 1\ q_5$:

- The left most blank (of (i)) is replaced by 1 and M enters q_5.
- Thus at the end of operations (i) - (v), the input part remains unaffected but the first 1 is added to the left of y.

(vi) $q_5\ 1R\ q_0$:

The head scans the second 1 of the input string and moves right and M enters q_0.

(vii) $q_6\ b\ R\ q_6, q_6\ 1R\ q_6, q_6\ X_1\ R\ q_6, q_6\ YL\ q_7$:

- M in state q_0 scans X_1, moves right and enters q_6.
- It continues to move to the right until it encounters y.

(viii) $q_7\ 1L\ q_7, q_7\ b_1\ q_8$:

- On encountering y, the head moves to the left, and M enters q_7, after which the head moves to the left until it encounters b appearing to the left of 1^{a1} of the output part.
- This b is changed to 1, and M enters q_8.

(ix) $q_8\ b\ L\ q_8, q_8\ 1L\ q_8, q_8\ YL\ q_8, q_8\ X_1\ X_1\ q_9$:

- Once M is in q_8, the head continues to move to the left and on scanning X_1, M enters q_0.
- As there is no quadruple starting with q_9, M halts.
- The machine halts and the terminal ID is $1^{a1}\ q_9\ X_1\ 1^{a1+1}\ y$.

Thus M halts, TM computed the successor function.

(2) Turing Machine for computing the zero function :

- The zero function Z is defined as $Z\ (a_1) = 0$ for all $a_1 \geq 0$.
- So the initial tape can be taken as $X = 1^{a1}\ X_1\ b\ Y$.
- Since we required value of $Z\ (a_1)$, to appear to the left of y, we require the machine to halt without changing the input.
- Thus we define a TM by taking $Q = \{q_0, q_1\}$, $\Gamma = \{b, 1, X_1, Y\}$, $X = 1^{a1}\ X_1\ b\ Y$ and P consist of $q_0\ b\ R\ q_0$, $q_0\ 1R\ q_0, q_0\ X_1\ X_1\ q_1, 1_0\ bR\ q_0$ and $q_0\ 1R\ q_0$ are used to move to the right until X_1 is encountered.
- $q_0\ X_1\ X_1\ q_1$ enables the TM to enter the state q_1.
- M enters q_1 without altering the tape symbol.
- In terms of change of ID's, we have $q_0\ 1^{a1}\ X_1\ b\ y \vdash 1^{a1}\ q_0\ X_1\ b\ y \vdash 1^{a1}\ q_1\ X_1\ b\ y$.
- As there is no quadruple starting with q_1, M halts and Res (X) = $1a^1\ q_1\ X_1\ b\ y$. By deleting q_1 in Res (X), we get $1^{a1}\ X_1$ by yielding 0.

Q.6. Consider the turing machine M described by the transition table given below. Describe the processing of :
(i) 0011 (ii) 001 (iii) 011 (iv) 000111 using ID's
Which of the above strings are accepted by M?

Present State	Tape Symbol				
	0	1	x	y	b
$\rightarrow q_1$	x R q_2				b R q_5
q_2	0 R q_2	y L q_3		y R q_2	
q_3	0 L q_4		x R q_5	y L q_3	
q_4	0 L q_4		x R q_1		
q_5				y R q_5	b R q_5
(q6)					

Ans. Here q_6 is a final state.

(i) 0011

$q_1\ 0\ 0\ 1\ 1 \vdash X\ q_2\ 0\ 1\ 1$
$\vdash X\ 0\ q_2\ 1\ 1$
$\vdash X\ q_3\ 0\ Y\ 1$
$\vdash q_4\ X\ 0\ Y\ 1$
$\vdash X\ q_1\ 0\ Y\ 1$
$\vdash X\ X\ q_2\ Y\ 1$
$\vdash X\ X\ Y\ q_2\ 1$

$\vdash$ X X q_3 Y Y

$\vdash$ X q_3 X Y Y

$\vdash$ X X q_5 Y Y

$\vdash$ X X Y q_5 Y

$\vdash$ X X Y Y q_5 ß

$\vdash$ X X Y Y ß q_6 → final state

Since q_6 is a final state, String 0011 is accepted

(ii) 011

q_1 0 1 1 $\vdash$ X q_2 1 1

$\vdash$ q_3 X Y 1

$\vdash$ X q_5 Y 1

$\vdash$ X Y q_5 1

Since $\delta(q_5, 1)$ is not defined

Hence 011 is rejected.

(iii) 001

q_1 0 0 1 $\vdash$ X q_2 0 1

$\vdash$ X 0 q_2 1

$\vdash$ X q_3 0 Y

$\vdash$ q_4 X 0 Y

$\vdash$ X q_1 0 Y

$\vdash$ X X q_2 Y

$\vdash$ X X Y q_2 ß

$\vdash$ 0 0 1 is rejected

$\delta(q_2,$ ß) is not defined.

Hence 001 is rejected

(iv) 000111

q_1 0 0 0 1 1 1 $\vdash$ X q_2 0 0 1 1 1

$\vdash$ X 0 q_2 0 1 1 1

$\vdash$ X 0 0 q_2 1 1 1

$\vdash$ X 0 q_3 0 Y 1 1

$\vdash$ X q_4 0 0 Y 1 1

$\vdash$ q_4 X 0 0 Y 1 1

$\vdash$ X q_1 0 0 Y 1 1

$\vdash$ X X q_2 0 Y 1 1

$\vdash$ X X 0 q_2 Y 1 1

$\vdash$ X X 0 Y q_2 1 1

$\vdash$ X X 0 q_3 Y Y 1

$\vdash$ X X q_3 0 Y Y 1

$\vdash$ X q_4 X 0 Y Y 1

$\vdash$ X X q_1 0 Y Y 1

$\vdash$ X X X q_2 Y Y 1

$\vdash$ X X X Y q_2 Y 1

$\vdash$ X X X Y q_2 Y 1

$\vdash$ X X X Y Y q_2 Y 1

$\vdash$ X X X Y q_3 Y Y

$\vdash$ X X X q_3 Y Y Y

$\vdash$ X X q_3 X Y Y Y

$\vdash$ X X X q_5 Y Y Y

$\vdash$ X X X Y q_5 Y Y

$\vdash$ X X X Y Y q_5 Y

$\vdash$ X X X Y Y Y q_5 ß

$\vdash$ X X X Y Y Y ß q_6

∴ 000111 string is accepted.

Q.7. Draw the computation sequence for the input string 00 using the following Turing machine description.

Present State	Tape Symbol		
	ß	0	1
→ q_1	1L q_2	0R q_1	-
q_2	ß R q_3	0L q_2	1L q_2
q_3	-	ß R q_4	ß R q_5
q_4	0R q_5	0R q_4	1R q_4
* q_5	0L q_2	-	-

Ans. String 00

q_1 0 0 $\vdash$ 0 q_1 0

$\vdash$ 0 0 q_1 ß

$\vdash$ 0 q_2 0 1

$\vdash$ q_2 0 0 1

$\vdash$ ß q_2 0 0 1

$\vdash$ q_2 ß 0 0 1

$\vdash$ ß q_3 0 0 1

$\vdash$ ß ß q_4 0 1

$\vdash$ ß ß 0 q_4 1

$\vdash$ ß ß 0 1 q_4 ß

$\vdash$ ß ß 0 1 0 q_5 ß

$\vdash$ ß ß 0 1 q_2 0 0

$\vdash$ ß ß 0 q_2 1 0 0

$\vdash$ ß ß q_2 0 1 0 0

$\vdash$ ß q_2 ß 0 1 0 0

$\vdash$ ß ß q_3 0 1 0 0

$\vdash$ ß ß ß q_4 1 0 0

$\vdash$ ß ß ß 1 q_4 0 0

$\vdash$ ß ß ß 1 0 q_4 0

$\vdash$ ß ß ß 1 0 0 q_4 ß

$\vdash$ ß ß ß 1 0 0 0 q_5 ß

$\vdash$ ß ß ß 1 0 0 q_2 0 0

$\vdash$ ß ß ß 1 0 q_2 0 0 0

$\vdash$ ß ß ß 1 q_2 0 0 0 0

$\vdash$ ß ß ß q_2 1 0 0 0 0

$\vdash$ ß ß q_2 ß 1 0 0 0 0

$\vdash$ ß ß ß q_3 1 0 0 0 0

$\vdash$ ß ß ß ß q_5 0 0 0 0

String 00 is accepted.

TURNING MACHINE AS ACCEPTOR

Q.8. Explain how a Turing machine can be viewed as a language acceptors.

Ans. Turing machine as acceptors :

Turing machne can be viewed as a language accepters in the following sense.

- A string w is written on the tape, with blanks filling out the unused portion.
- The machine is started in the initial state q_0 with the R/W head positioned on the leftmost symbol of w.
- If, after a sequence of moves, the Turing machine enters a final state and halts, then w is considered to be accepted.
- Formally, we can say that if

  $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ is a Turing machine.

  Then the language accepted by M is

  $L(M) = \{ w \mid w \in \Sigma^* \text{ and } q_0 w \vdash^* \alpha_1 q_f \alpha_2 \text{ for some } q_f \in F \text{ and } \alpha_1, \alpha_2 \in \Gamma^* \}$

- Given a TM recognizing a language L, we assume without loss of generality that the TM halts, i.e. it has no next move, whenever the input is accepted.
- However, for word not accepted, one of the two things can happen, i.e. the machine can halt in a nonfinal state or it can enter an infinite loop and never halt.
- Any string for which M does not halt is by definition not in L (M).

Q.9. Construct a TM over $\Sigma(a, b)$ in which string consist of following Language.

$L = \{ a^n b^n / n > 0 \}$

OR Design a Turing machine to accept the language. $L = \{ 0^n 1^n / n \geq 1 \}$. Clearly state any assumptions you make. Explain briefly the operation of your machine. ***CT : W-06(9M)***

Ans. L = { ab, aabb, aaabbb, ...}

Simulation :

$\delta(q_0, aabb) \vdash$ (a a b b)

↑

q_0

$\vdash$ (* a b b)

↑

q_1

$\vdash$ (* a b b)

↑

q_1

$\vdash$ (* a * b)

↑

q_0

$\vdash$ (* * * b)

↑

q_1

$\vdash$ (* * * b)

↑

q_1

$\vdash$ (* * * *)

↑ ↑ ↑

q_0

$\vdash$ (ß * * * *)

↑

q_0

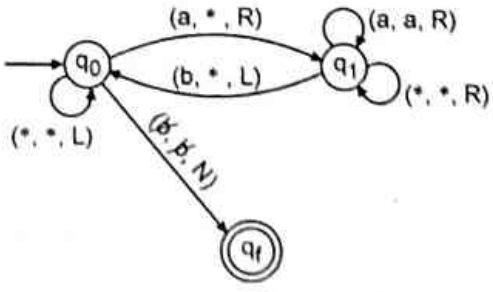
$\vdash$ (q)

Transition Table :

Present state	Tape Symbol			
	a	b	*	ß
→ q_0	$(q_1, *, R)$	-	$(q_0, *, L)$	$(q_f, ß, N)$
q_1	(q_1, a, R)	$(q_0, *, L)$	$(q_1, *, R)$	-
* q_f	-	-	-	-

(ß a a a b b b ß)
↑
q_0
(ß * a a b b b ß)
↑ ↑
q_1 q_1
ß * a a * b b ß
↑
q_0
ß * a * * b b ß
↑ ↑
q_1 q_1
ß * a * * * b ß
↑ ↑ ↑
q_0 q_0 q_0
ß * * * * * b ß
↑
q_1
ß * * * * * * ß
↑
q_0

Transition Diagram :



Q.10. Design TM for language given below. Explain briefly the operation of your machine.

$L = \{a^n b^n c^n \mid n > 0\}$

OR Design TM to recognize words in

$L = \{a^n b^n a^n \mid n \geq 1\}$. **CT : S-12(7M)**

OR Construct a T.M. that accept the following language.

$L = \{a^n b^n c^n \mid n \geq 1\}$ **CT : W-09(8M)**

OR Design Turing Machine to recognize the language.

$L = \{0^n 1^n 0^n \mid n \geq 1\}$ **CT : S-07(8M)**

OR Design TM for language given below. Explain briefly operation of TM. $L = \{0^n 1^n 2^n / n \geq 1\}$. **CT : W-08(9M)**

Ans. L = {012, 001122, 000111222.....}

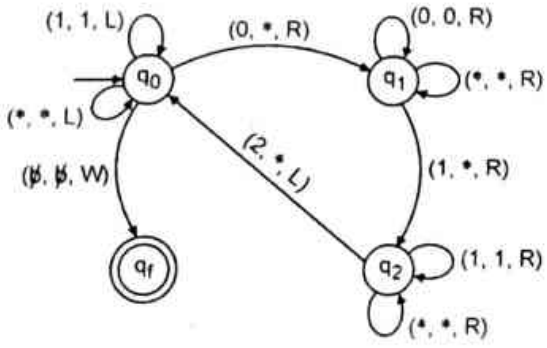
Simulation :

(ß 0 0 1 1 2 2 ß)
↑
q_0
(ß * 0 1 1 2 2 ß)
↑ ↑
q_1 q_1
(ß * 0 * 1 2 2 ß)
↑ ↑
q_2 q_2
(ß * 0 * 1 * 2)
↑ ↑ ↑
q_0 q_0 q_0
(ß * * * 1 * 2)
↑ ↑
q_1 q_1
(ß * * * * * 2)
↑ ↑
q_2 q_2
(ß * * * * * * ß)
↑
q_0

Transition Table :

Present State	Tape Symbol				
	0	1	2	*	ß
→ q_0	$(q_1, *, R)$	$(q_0, 1, L)$	-	$(q_0, *, L)$	$(q_f, ß, N)$
q_1	$(q_1, 0, R)$	$(q_1, *, R)$	-	$(q_1, *, R)$	-
q_2	-	$(q_2, 1, R)$	$(q_0, *, L)$	$(q_2, *, R)$	-
* q_f	-	-	-	-	-

Transition Diagram :



Q.11. Design a TM which can scan a well define parenthesis.

OR Design TM to check well formedness of the parenthesis. ***CT : S-09, W-09(8M)***

Ans. L = {( ), (( )), (( ) ( )), (( ) ( ) ( ).....}

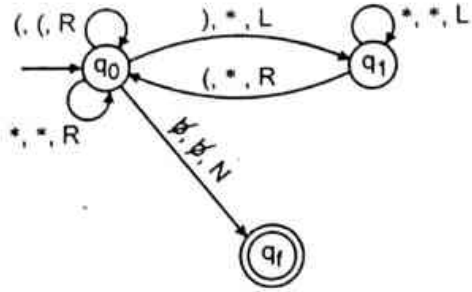
Simulation :

ß (( ) ( )) ß
↑ ↑
q_0 q_0
ß ((* ( )) ß
↑
q_1
ß (* * ( )) ß
↑ ↑
q_0 q_0
ß (* * (*) ß
↑
q_1
ß (* * * *) ß
↑ ↑
q_0 q_0
ß (* * * * * ß
↑ ↑ ↑ ↑ ↑ ↑
q_1 q_1
ß * * * * * * ß
↑ ↑ ↑ ↑ ↑ ↑
q_0

Transition Table :

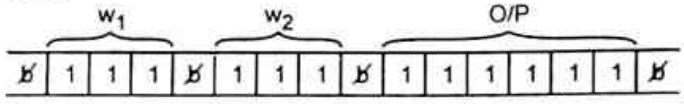
Present State	Tape Symbol			
	(	)	*	ß
→ q_0	q_1, (, R	q_1, *, L	q_0, *, R	(q_f, ß, N)
q_1	q_1, *, R	-	q_1, *, L	-
* q_f	-	-	-	-

Transition Diagram :



Q.12. Design a TM over summation (1, ß) which can compute a concatenation function over Σ = {1} i.e, if a pair of words (w_1, w_2) is I/P and O/P has to be $w_1 . w_2$. ***CT : S-09(8M)***

Ans.



L = 1 1 1 ß 1 1 1

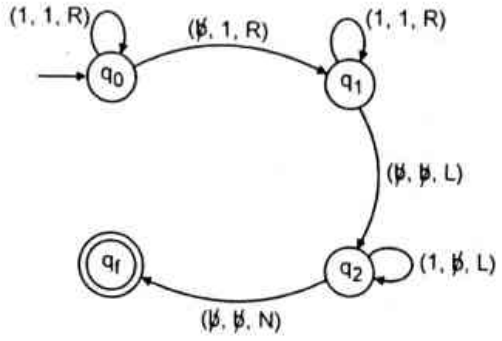
Simulation :

(ß 1 1 1 ß 1 1 1 ß)
(ß 1 1 1 ß 1 1 1 ß)
↑ ↑ ↑ ↑
q_0 q_0
(ß 1 1 1 1 1 1 1 ß)
↑ ↑ ↑ ↑
q_1 q_1
(ß 1 1 1 1 1 1 1 ß)
↑
q_2
(ß 1 1 1 1 1 1 ß ß)
↑ ↑ ↑ ↑ ↑ ↑ ↑
q_2 q_2
(ß 1 1 1 1 1 1 ß ß)

Transition Table :

Present State	Tape Symbol	
	1	ß
→ q_0	(q_1, 1, R)	(q_1, 1, R)
q_1	(q_1, 1, R)	(q_2, ß, L)
q_2	(q_2, ß, N)	(q_f, ß, N)
* q_f	-	-

Transition Diagram :



Q.13. Design a TM which scan every sequence of '111' by '101', when string consist of 1 and 0.

OR Design a TM that replaces all occurrence of '111' by '101' from a sequence of 0's and 1's.

CT : W-10(9M)

OR Design Turing Machine that recognizes the presence of substring 101 and replaces it with '110'. **CS : W-12(7M)**

Ans. L = {1 1 1 1 1 1 1 0 1, 0 1 1 1 0 1 1 1 0,.....}

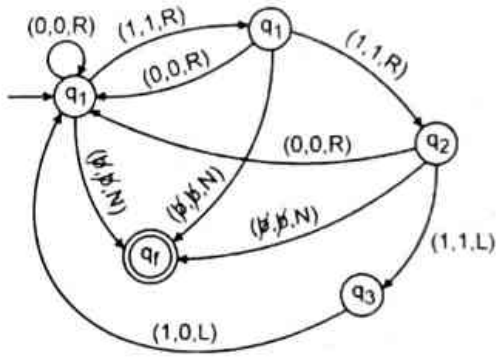
Simulation :

```
0  1  1  1  0  1  1  1  0
↑  ↑  ↑  ↑
q0 q0 q1 q2
0  1  1  1  0  1  1  1  0
      ↑
      q3
0  1  0  1  0  1  1  1  0
   ↑  ↑  ↑  ↑  ↑  ↑  ↑
   q0 q1 q0 q1 q0 q1 q2
0  1  0  1  0  1  1  1  0
                  ↑
                  q3
0  1  0  1  0  1  0  1  0  ß
                  ↑  ↑  ↑  ↑  ↑
                  q0 q1 q0 q1 q0
```

Transition Table :

Present State	Tape Symbol		
	1	0	ß
→ q_0	q_1, 1, R	q_0, 0, R	q_f, ß, N
q_1	q_2, 1, R	q_0, 0, R	q_f, ß, N
q_2	q_3, 1, L	q_0, 0, R	q_f, ß, N
q_3	q_0, 0, L	-	-
* q_f	-	-	-

Transition Diagram :



Q.14. Design a Turing Machine (TM) which accepts the language L = {W ∈ (a + b) * |W contains equal no. of a's and b's}

Use only single track TM. After the machine halts in the final state, the head of the mc should be on the first blank of the tape.

OR Construct Turing Machine to accept the following language.

$L = \{W/\ W \in (a, b)^*,\ n_a(w) = n_b(w)\}$

where,

$n_a(w)$ = Number of a's in W.

$n_b(w)$ = Number of b's in W. **CT : W-10(8M)**

Ans. Analysis :

(1) The language is variable.

(2) The strings can start with a or b and end with a or b.

(3) The strings contain equal number of a and b.

Valid strings : ab, abab, bababa, bbaa, ...

Invalid strings : aba, aab, baa, bab, ...

Algorithm :

(1) Replace first symbol a with * and move in forward direction till symbol b is not found.

(2) Once symbol b is found replace it with * and move in the left direction till symbol ß is not found i.e. start of string.

(3) If the first symbol in the string is b, then replace it with *. Move in the right direction till symbol a is not found.

(4) Once symbol a is found replace it by * and move in the left direction till symbol ß is not found. After completing step (2) or (4) machine will go to the initial state i.e. q_0.

(5) If machine terminates with the combination (q_0, b) the string is valid.

But if the machine terminates with combination (q_1, b) or (q_3, b) the string is invalid.

Transition of TM :

$\delta(q_0, a) \rightarrow (q_1, *, R)$

$\delta(q_1, a) \rightarrow (q_1, a, R)$

$\delta(q_1, b) \rightarrow (q_2, *, L)$

$\delta(q_2, *) \rightarrow (q_2, *, L)$

$\delta(q_2, a) \rightarrow (q_2, a, L)$

$\delta(q_2, b) \rightarrow (q_2, b, L)$

$\delta(q_2, \text{b}) \rightarrow (q_0, \text{b}, R)$

$\delta(q_0, *) \rightarrow (q_0, *, R)$

$\delta(q_0, b) \rightarrow (q_3, *, R)$

$\delta(q_3, b) \rightarrow (q_3, b, R)$

$\delta(q_3, a) \rightarrow (q_2, *, L)$

$\delta(q_0, \text{b}) \rightarrow (q_1, \text{b}, N)$

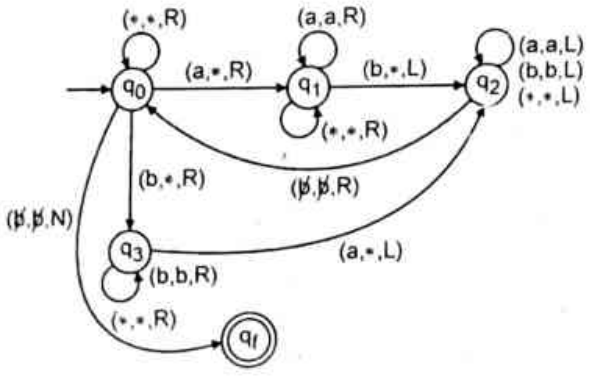
$\delta(q_3, *) \rightarrow (q_3, *, R)$

$\delta(q_1, *) \rightarrow (q_1, *, R)$

Transition Table of TM :

Present State	Tape Symbol			
	a	b	*	b
$\rightarrow q_0$	$q_1, *, R$	$q_3, *, R$	$q_0, *, R$	q_f, b, N
q_1	q_1, a, R	$q_2, *, L$	$q_1, *, R$	-
q_2	q_2, a, L	q_2, b, L	$q_2, *, L$	q_0, b, R
q_3	$q_2, *, L$	q_3, b, R	$q_3, *, R$	
$* q_f$	-	-	-	-

Transition graph of TM :



Q.15. Design a Turing machine that recognizes the language : $L = \{WW^R \mid W \in (0+1)^*\}$. $W^R \rightarrow$ Reverse of W.

OR Design TM for $L = \{WW^R \mid W \in (a, b)^*\}$. W^R - Reverse of W. Draw transition table.

CS : W-09(7M)

Ans. Analysis :

(1) Language is variable.

(2) The string may either start with 0 or 1, may either end with 0 or 1.

(3) The strings are palindromes of even length.

Valid Strings : 0110, 1001, 101101, 100001, ...

Invalid strings : 101, 1101 .....

Algorithm and Transitions of TM :

(1) Replace first 0 and 1 by X and Y respectively.

$\delta(q_0, 0) \rightarrow (q_1, X, R)$

$\delta(q_0, 1) \rightarrow (q_2, Y, R)$

(2) Skip all 0 and 1 till blank symbol b is not found.

$\delta(q_1, 0) \rightarrow (q_1, 0, R)$

$\delta(q_1, 1) \rightarrow (q_1, 1, R)$

(3) Now TM moves to left direction

$\delta(q_1, \text{b}) \rightarrow (q_3, \text{b}, L)$

(4) Skip all '0' and '1' till blank symbol b and move to left.

$\delta(q_2, 0) \rightarrow (q_2, 0, R)$

$\delta(q_2, 1) \rightarrow (q_2, 1, R)$

$\delta(q_2, \text{b}) \rightarrow (q_4, \text{b}, L)$

(5) Replace rightmost 0 by X and 1 by y respectively and move to left.

$\delta(q_3, 0) \rightarrow (q_5, X, L)$

$\delta(q_4, 1) \rightarrow (q_6, Y, L)$

$\delta(q_5, 0) \rightarrow (q_5, 0, L)$

$\delta(q_5, 1) \rightarrow (q_5, 1, L)$

$\delta(q_6, 0) \rightarrow (q_6, 0, L)$

$\delta(q_6, 1) \rightarrow (q_6, 1, L)$

$\delta(q_5, X) \rightarrow (q_0, X, R)$
$\delta(q_6, Y) \rightarrow (q_0, Y, R)$
} Change state to q_0 and move right side

$\delta(q_0, X) \rightarrow (q_f, X, R)$

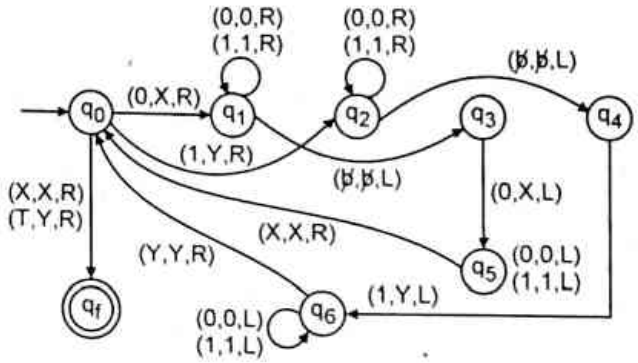
$\delta(q_0, Y) \rightarrow (q_f, Y, R)$

VBD

Transition Table :

Present/State	Tape Symbol				
	0	1	X	Y	♭
$\rightarrow q_0$	(q_1, X, R)	(q_2, Y, R)	(q_f, X, R)	(q_f, Y, R)	-
q_1	$(q_1, 0, R)$	$(q_1, 1, R)$	-	-	$(q_3, ♭, L)$
q_2	$(q_2, 0, R)$	$(q_2, 1, R)$	-	-	$(q_4, ♭, L)$
q_3	(q_5, X, L)	-	-	-	-
q_4	-	(q_6, Y, L)	-	-	-
q_5	$(q_5, 0, L)$	$(q_5, 1, L)$	(q_0, X, R)	-	-
q_6	$(q_6, 0, L)$	$(q_6, 1, L)$	-	(q_0, Y, R)	-
$*q_f$	-	-	-	-	-

Transition graph of TM :



Q.16. Design turing machine for following :

$L = \{a^n b^{2n} c^n / n \geq 1\}$.

CT : S-11, CS : W-11(13M)

Ans. The construction of the required TM is given by the following steps :

Step 1 : q_0 is the initial state on scanning the leftmost a, the R/W head replaces a by B, and moves to right by changing state q_0 to q_1.

Step 2 : On reading the leftmost b, the R/W head replaces two b's by B's and moves to the right by changing states q_1 to q_2 and q_3.

Step 3 : On scanning the leftmost c, the R/W head replaces c by B, and moves to the right by changing state q_3 to q_4.

Step 4 : After reading the next c, the R/W head move to the left until it finds the rightmost a and M changes its state q_4 to q_5. After reading the rightmost a, the R/W head moves to the left without replacing a when it leads B on q_5 the R/W head moves to the right and M enters q_6. As a result, the leftmost a, b and c are replaced by B.

Step 5 : Steps 1-4 are repeated until all a's, b's and c's are replaced by blanks.

Let us see the processing of input string abbc is given as follows :

q_0 abbc $\vdash$ B$q_1$bbc

$\vdash$ BB$q_2$bc

$\vdash$ BBBq_3c

$\vdash$ BBBBq_4B

$\vdash$ BBBq_7BB

Thus, q_0 abbc $\vdash^*$ BBBBq_7

- As q_7 is the final state, the input string abbc is accepted by TM.

- Now we can construct the transition table for TM.

Transition Table :

Present state	Input Tape Symbol			
	a	b	c	B
$\rightarrow q_0$	BRq_1			BRq_7
q_1	aRq_1	BRq_2		BRq_1
q_2		BRq_3		
q_3		bRq_3	BRq_4	BRq_3
q_4			cLq_5	BLq_7
q_5	aLq_6	bLq_5		BLq_5
q_6	aLq_6			BRq_0
q_7	-	-	-	-

- It can be seen from the table that the string of the form of $a^n b^{2n} c^n$ are accepted by TM.
- If n = 0, the TM should accept the blank symbol. From the first row of the table we can see that the TM also accepts the blank symbol.
- Hence the TM recognizes the languages.

$L = \{a^n b^{2n} c^n \mid n \geq 0\}$

Q.17. Design TM for the following :

$L = \{0^n 1^m 0^n / m, n \geq 1\}$. Also draw transition diagram. ***CS : S-13(7M)***

Ans. Analysis :

(1) Language is fixed

(2) Starts with 0 followed by 1 and ends with 0.

Valid strings :

ß 0 0 0 1 1 1 0 0 0 ß

Algorithm :

(1) Assume q_0 as the initial state.

(2) Replace the first symbol 0 with a, change the state to q_1 and skip remaining symbols 0 till symbol 1 arrives.

ß	0	0	0	1	1	1	0	0	0	ß

$\delta(q_0, 0) \rightarrow (q_1, a^*, R)$

$\delta(q_1, 0) \rightarrow (q_1, 0, R)$

$\delta(q_1, 1) \rightarrow (q_2, 1, R)$

$\delta(q_2, 1) \rightarrow (q_2, 1, R)$

$\delta(q_2, 0) \rightarrow (q_3, a^*, L)$

$\delta(q_3, 1) \rightarrow (q_4, 1, L)$

$\delta(q_4, 1) \rightarrow (q_4, 1, L)$

$\delta(q_4, 0) \rightarrow (q_5, 0, L)$

$\delta(q_5, 0) \rightarrow (q_5, 0, L)$

$\delta(q_5, *) \rightarrow (q_0, *, R)$

Additional Transition :

$\delta(q_2, *) \rightarrow (q_2, *, R)$

$\delta(q_3, *) \rightarrow (q_3, *, L)$

Termination Condition :

$\delta(q_0, 1) \rightarrow (q_5, 1, R)$

$\delta(q_5, 1) \rightarrow (q_5, 1, R)$

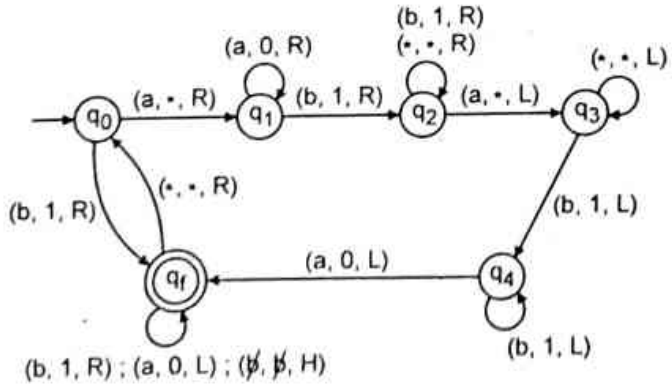
$\delta(q_5, *) \rightarrow (q_5, *, R)$

$\delta(q_5, ß) \rightarrow (q_F, ß, H)$

Transition Table :

State	a	b	*	ß
q_0	$q_1, *, R$	$q_5, 1, R$	-	-
q_1	$q_1, 0, R$	$q_2, 1, R$	-	-
q_2	$q_3, *, L$	$q_2, 1, R$	$q_2, *, R$	-
q_3	-	$q_4, 1, L$	$q_3, *, L$	-
q_4	$q_5, 0, L$	$q_4, 1, L$	-	-
$*q_f$	$q_5, 0, L$	$q_5, 1, R$	$q_0, *, R$	$q_f, ß, H$

Transition Diagram :



TM AS TRANSDUCERS

Q.18. Write a short note on "Turing machines as Transducers".

Ans. Turing Machines as Transducers :

- In language theory, accepters are quite adequate.

- Turing machines are not only interesting as language accepters, they provide us with a simple abstract model for digital computers, in general.
- Since the primary function of a computer is to transform input into output, it acts as a transducer.
- If we want to model computers using Turing machines, we have to emphasize at this aspect.
- The input for a computation will be all the nonblank symbols on the tape at the initial time.
- At the conclusion of the computation, the output will be whatever is then on the tape.
- Thus, we can view a Turing machine transducer M as an implementation of a function f defined by

  $$w' = f(w),$$

  Provided that

  $$q_0 w \vdash^* M\, q_f\, w',$$

  For some final state q_f.
- A function f having domain D is said to be Turing-computable or just computable if there exist some Turing machine

  $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, such that

  $q_0 w \vdash^* M\, q_f\, f(w), q_f \in F,$

  $q_f \in F$, for all $w \in D$.

VARIATION IN TM

Q.19. What are the various types of turing machine? Explain each type in brief.

CT: S-07(8M)

OR Discuss various types of TM models.

CT : W-12(4M), S-12(7M)

OR Explain various modification of TM.

CS : W-12(5M)

OR What do you mean by non-deterministic turing machine. **CS : S-13(5M)**

OR Write short note on Multidimensional Turing machine. **CS : S-10(3M)**

OR Explain multitape turing machine.

CS : W-09(3M)

OR Write short note on multitape turing machine.

CT : W-10(4M), CS : S-09,12,13(4M), S-10(3M)

Ans. There are following 6 types of turing machine :

(1) Turing Machine with Two-way infinite Tape :

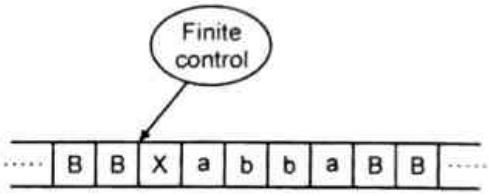
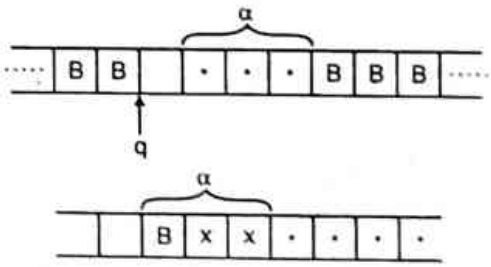


Fig. TM with two way infinite tape

- A Turing machine with a two-way infinite tape is denoted by $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, as in the original model.
- As its name implies, the tape is infinite to the left as well as to the right. We denote an ID of such a device as for the one-way infinite TM.
- The ID of two way infinite Turing machine is same as that of one way infinite Turing machine.
- The difference arises on relation t_M for two transitions.

(i) If $(q, x) = (P, Y, L)$ then $Q \times \alpha __M$ PBY α this is because the tape is infinite towards left.



(ii) If $\delta(q, x) = (P, B, R)$ Then QX $\alpha\, T_M\, P\alpha$ In P α we have not inserted B because it is infinite towards left so no need to explicitly show it.

- The initial ID is q_0W. While there was a left end to the tape in the original model, there is no end of the tape for the Turing machine to "fall off", so it can proceed left as far as it wishes. Relation M relates two ID's if the one on the right can be obtained from the one on the left some number of moves.

(2) Multi-tape Turing Machine :

- A multi-tape Turing machine consists of a finite control with k tape heads and k tapes; each tape is infinite in both directions.
- On a single move depending on the state of the finite control and symbol scanned by each of the tape heads, the machine can

(i) Change state.

(ii) Print a new symbol on each of the cells scanned by its tape heads.

(iii) Move each of its tape heads, independently, one cell to the left or right, or keep it stationary.

Initially, the input appears on the first tape, and the other tapes are blank.

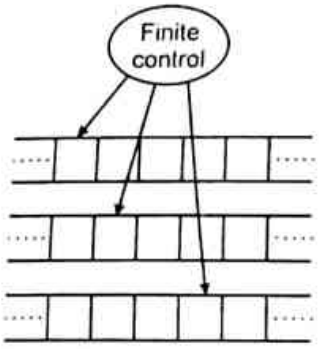


Fig. Multitape turing machine

(3) Non-deterministic Turing Machines.

- A non-deterministic Turing machine is a device with finite control and a single one way infinite tape.
- For a given state and tape symbol scanned by the tape head, the machine has a finite number of choices for the next move.
- Each choice consists of a new state, a tape symbol to print, and a direction of head motion.
- The non-deterministic TM is not permitted to make a move in which the next state is selected from one choice and the symbol printed and/or direction of head motion are selected from other choices.
- The non deterministic TM accepts its input if any sequence of choices of moves leads to an accepting state.
- As with the finite automaton, the addition of non-determinism to the Turing machine does not allow the device to accept new languages.

Disadvantages :

(i) The addition of non determinism to the turing machine does not allow the device to accept new language.

(ii) The combination of non-determinism with any of the extensions presented or to be presented such as two-way infinite or multiple TM's does not add additional power.

(4) Multidimensional Turing Machines :

- The multidimensional Turing machine has the usual finite control, but the tape consists of a k-dimensional array of cells infinite in all 2k directions, for some fixed k.
- Depending on the state and symbol scanned, the device changes state, prints a new symbol, and moves its tape head in one of 2k directions, either positively or negatively, along one of the k axes
- Initially, the input is along one axis, and the head is at the left end of the input.
- At any time, only a finite number of rows in any dimension contain nonblank symbols, and these rows each have only a finite number of nonblank symbols.

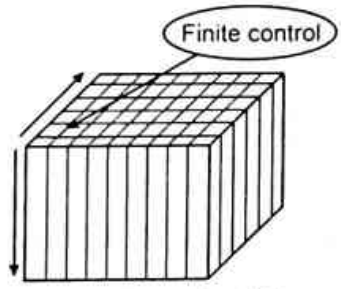


Fig. 3-dimensional TM

(5) Multi-head Turing Machines :

- A k-head Turing machine has some fixed number, k, of heads.
- The heads are numbered 1 through k, and a move of the TM depends on the state and on the symbol scanned by each head.
- In one move, the heads may each move independently left, right, or remain stationary.

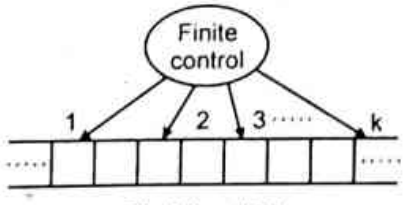


Fig. k-head TM

(6) Off-line Turing machines :

- An off-line Turing machine is a multi-tape TM whose input tape is read only.
- Usually we surround the input by end-markers, ¢ on the left and $ on the right.

- The Turing machine is not allowed to move the input tape head off the region between ¢ and $.
- Offline TM is just a special case of the multi-tape TM, and is no more powerful than any of the models we have considered.
- Conversely, an off-line TM can simulate any TM M by using one more tape than M. The first thing the off-line TM does is copy its own input onto the extra tape, and it then simulates M as if the extra tape were M's input.

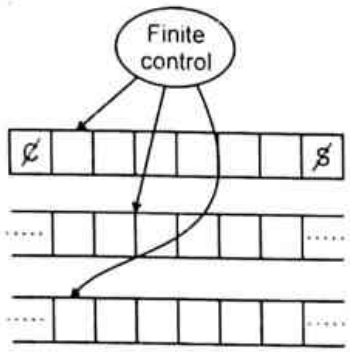


Fig. Offline TM

Q.20. Explain working of multitrack Turing Machine. ***CT : S-13(6M)***

OR Write short note on multitrack Turing machine. ***CS : S-09(4M)***

Ans. Multitrack Turing Machine :

- A Multitrack Turing machine is a specific type of Multi-tape Turing machine.
- In a standard n-tape Turing machine, n heads move independently along n tracks.
- In a n-track Turing machine, one head reads and writes on all tracks simultaneously.
- A tape position in a n-track turing Machine contains n symbols from the tape alphabet.
- It is equivalent to the standard Turing machine and therefore accepts precisely the recursively enumerable languages.

Formal definition :

A multitape Turing machine can be formally defined as a 6-tuple, $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$, where,

Q is a finite set of states

Σ is a finite set of symbols called the tape alphabet

$\Gamma \in Q$

$q_0 \in Q$ is the initial state

$F \subseteq Q$ is the set of final or accepting states.

$\delta \subseteq (Q \setminus A \times \Sigma) \times (Q \times \Sigma \times d)$ is a relation on states and symbols called the transition relation.

$\delta(Q_i, [x_1, x_2 \ldots\ldots x_n]) = (Q_j, [y_1, y_2 \ldots\ldots y_n], d)$

where $d \in , r$

Proof of equivalency to standard Turing machine :

- This will prove that a two-track turing machine is equivalent to a standard turing machine.
- This can be generalized to a n-track turing machine. Let L be a recursively enumerable language.
- Let $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ be standard turing machine that accepts L. Let M' is a two-track Turing machine.
- To prove M = M' it must be shown that $M \subseteq M'$ and $M' \subseteq M$
- If all but the first track is ignored then M and M' are clearly equivalent $M' \subseteq M$.
- The tape alphabet of a one-track Turing machine equivalent to a two-track Turing machine consists of an ordered pair.
- The input symbol a of a Turing machine M' can be identified as an ordered pair [x, y] of Turing machine M.
- The one-track Turing machine is :

  $M = (Q, \Sigma \times B, \Gamma \times \Gamma, \delta', q_0, F)$ with the transition function $\delta(q_i, [x_1, x_2]) = \delta'(q_i [x_1, x_2])$

  This machine also accepts L.

Q.21. Using multitrack tape turing machine, design an algorithm to check whether number present on Tape is PRIME. ***CS : W-09(4M)***

Ans.

- The aim of solving this problem is to detect whether the number which is encoded on the data tape is prime number or not.
- Primality detection is realized by similar principles which are used in the divisibility problem.

- The input data type can appear as follows :

...	#	#	#	1	1	1	1	1	1	1	#	#	#	.....

Fig. Example of the data type of primality detection of number 7

- If number which is encoded in unary number system on the data type is prime number, the modified data type should look like :

.....	#	#	X	1	1	1	1	1	1	1	#	#	#	.....

Fig. Response of the Turing machine (primality was detected)

- Solving of this problem is highly complex and requires a multitude of inner states.
- Composition of the table which contains the rules for the turing machine control represents extremely complicated task.
- These rules must provide information on algorithm of prime numbers classification for the Turing machine.

Evolutionary-estimated generation of program for the Turing machine :

- Above-mentioned findings result that it is not simple to arrange the table which contains the rules assigning arguments of the transfer function to the response from the Turing machine thus a program for the Turing machine.
- Complications which occur in the development process of the program for the Turing machine represent the problem which can be tried to solve by recent computational methods.
- These methods we use belong to methods which utilize components of artificial intelligence, especially evolutionary methods.
- It can be said we let machine to program another machine.
- The idea of employing evolutionary methods for composition of the rules for the Turing machine consists of following statements.

(i) There are set of parameters of the Turing machine $(Q, \Sigma, \Gamma, q_0, B, F)$.

(ii) These parameters can be set in accordance with deliberation, previous observation or randomness.

(iii) It is necessary to encode the problem which can be solved to the form of the data tape.

(iv) It is necessary to encode the result of the problem which can be solved to the form of the data tape.

(v) There must be set the initial position of the Turing machine's head.

(vi) Artificial intelligence itself can compile the rules for the Turing machine control.

- At first sight, these statements represent generally universal instructions how to obtain the rules for solving whatever problem by the Turing machine.
- Nevertheless, if there is taken a think of above-mentioned instruction there can be found out some difficulties and problems :

(i) Whole process of the rules synthesis entirely depends on setting parameters of the Turing machine as are initial state, final state and number of inner states especially. If there is set e.g. insufficient amount of inner states, it may happen that the rules won't be designed properly or it will be impossible to design the rules.

(ii) As shown formerly, there is necessary to regard solving problems not like manipulation with data tape symbols but understand their fundamentals. This can be a huge problem for proper designing of the rules by artificial intelligence.

Q.22. Given an arbitrary Turing Machine 'M'. Prove that the problem "whether language accepted by 'M' is a regular set" is undecidable. ***CT : S-06,08(7M)***

Ans.

- Consider any instance (M, x) of the membership problem for Turing machines.
- From (M, x) construct a Turing machine M_x that accepts $\{a^i b^i \mid i \geq 0\}$ if M accepts x, and that accepts the empty set if M does not accept x.
- Specifically, M_x on any given input w starts the computation by checking whether $w = a^i b^i$ for some $i \geq 0$.
- If the equality $w = a^i b^i$ holds for no $i \geq 0$., then M_x rejects w. Otherwise, M_x simulates the computation of M on input x.

- In the latter case, M_x accepts w if it determines that M accepts x, and M_x rejects w if it determines that M rejects x.
- Consequently, M_x accepts a regular language (which is the empty set) if and only if M does not accept x.
- The result follows from the undecidability of the membership problem for Turing machines

Example : Let M be the Turing machine given in figure.

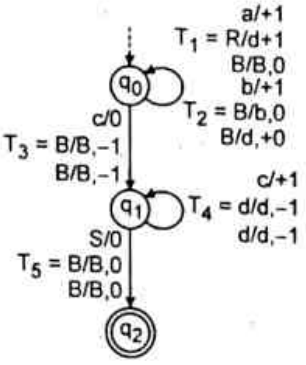


Fig. (a) A Turing machine M

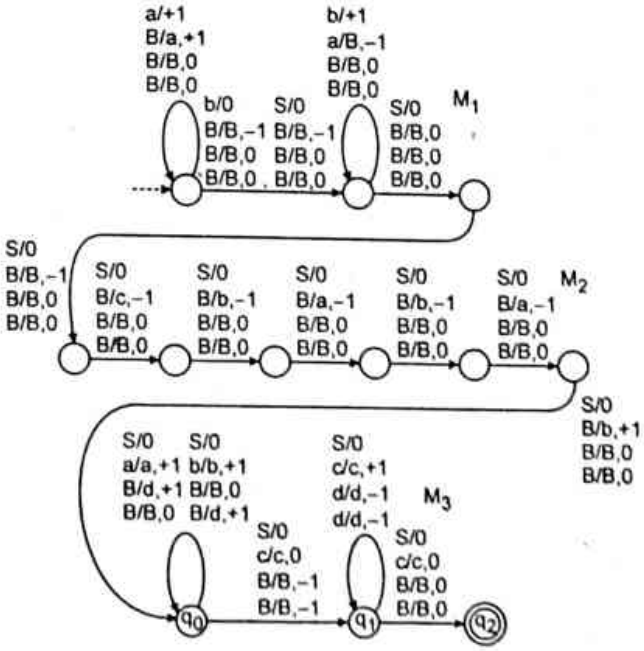


Fig. (b) A corresponding Turing machine M_{ababc} that accepts $(a^ib^i \mid i \geq 0)$ if and only if M accepts ababc.

- Let x = ababc. Then M_x in the proof of theorem can be as in figure (b). M_x has one more auxiliary work tape than M and consists of three subcomponents M_1, M_2 and M_3.
- M_1 checks that the given input is of the form $a^i b^i$ for some $i \geq 0$. M_2 stores the string $\not\in] \times \$]$ in the first auxiliary work tape. M_3 is just M modified to read its input from the first auxiliary work tape. $\not\in]$ and $\$]$ are the symbols used in the first auxiliary work tape representing the endmarkers $\not\in$ and \$, respectively.
- The universe of the undecidable problems includes numerous examples.
- For many of these problems the proof of undecidability is quite involved.
- The selection that has been made here should be appreciated at least for the simplification that it allows in introducing the concepts under consideration.

Q.23. Prove that digital computer is equivalent to Turing Machine. **CT : S-13(5M)**

Ans.

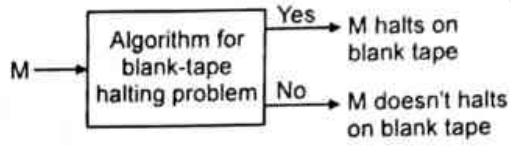
- A recent discussion on Robtiron has cast doubt on whether Alan Turing can be considered the "father of Computer Science" (to the extent any single person can be thus credited), given that computing machines such as the Antikythera mechanism have existed for many centuries.
- However, this point overlooks the fact that there is a difference between a "computing machine" and a "computer".
- In its broadest sense, anything that provides a convenient abstraction for making inferences about the world is a "computing machine".
- Therefore , in that sense, we have had computing machines for over 35,000 years-that's the age of the earlist counting implements known, the Lebombo and Ishango Bones.
- However, what differentiates a "computer" (in the sense we have used the word since Turing) from a computing machine in its generality; while a computing machine is designed to work on a specific problem class (no matter how large that class is), a computer can perform any conceivable computation, including the simulation of other computers and computing machines, both less and more sophisticated than the computer itself.

- This is no meaningless distinction-it is in fact the root of the computing revolution, as relatively simple silicon processors can run software programs much more complex than themselves.
- The Antikythera is certainly a remarkable device, but its by no means a computer; its merely a computing machine.
- We cannot, for example, use the Antikythera to solve differential equations, and we certainly cannot use it to simulate another device, say an arithmetic calculator.
- Likewise, the most sophisticated calculators still fail the Turing threshold, as they cannot be used to simulate the operation of other calculators.
- For a very enlightening account of Turings's breakthrough paper "On computable Numbers, with an Application to the Entscheidungsproblem", which introduces Turing machines and many of the theoretical foundations of modern computing, it recommend reading the excellent Annotated Turing.
- It provides a wealth of explanations without which it wouldn't ever hope to understand Turing's paper, besides providing important historical context to its development and relevance.

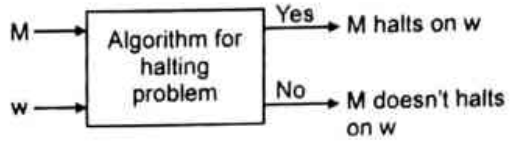
Q.24. Prove that empty tape problem is Unsolvable. **CS : S-10(6M)**

OR Explain blank tape halting problem, **CS : W-09(5M)**

Ans. Blank-tape halting algorithm :

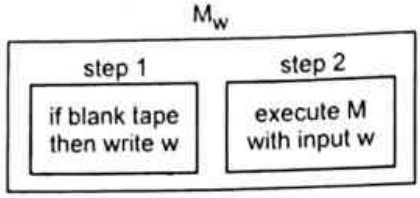


We want to design the halting alogorithm :



Construct a new machine M_w :

- On blank tape writes w.
- Then continues execution like M.



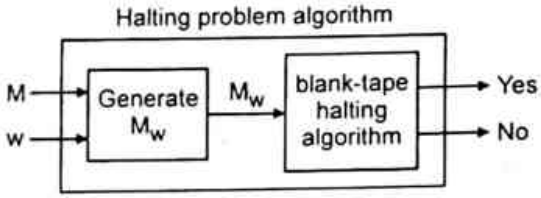
M halts on input string w

M_w halts when started with blank tape.

Algorithm for halting problem :

Inputs : machine M and string w

1. Construct M_w
2. Run algorithm for blank-tape halting problem with input M_w



Since the halting problem is undecidable, the blank-tape halting problem is also undecidable.

Q.25. Show that there is no algorithm for deciding if any two turing machine M1 and M2 accept the same language. **CS : S-12(7M)**

Ans. Proof :

- Let T be a Turing machine implementing the algorithm that translates strings over the alphabet of L1 to strings over the alphabet of L2.
- If L2 is decidable, there is a halting Turing machine M2 that accepts it. A multi-tape Turing machine M1 that decides L1 can be constructed as follows : On input string w, M1 invokes T to generate the strings z, which it then passes to M2.
- If M2 accepts z, M1 accepts w. If M2 rejects it, so does M1.

  Thus, M1 decides L1.
- Suppose now that L1 is unsolvable. Assuming that L2 is decidable, from the above construction, L1 is decidable, contradicting this assumption.

- Thus, L2 cannot be decidable. The power of this lemma will be apparent in the next section.

Q.26.(A) Write short note on Universal Turing Machine. **CT : S-06, W-07,12(4M), S-09(5M), S-10,11,12, W-09(3M), CS : W-11(4M)**

OR Explain concept of Universal Turing Machine. **CT : W-08(5M)**

Ans. Universal Turing Machine :

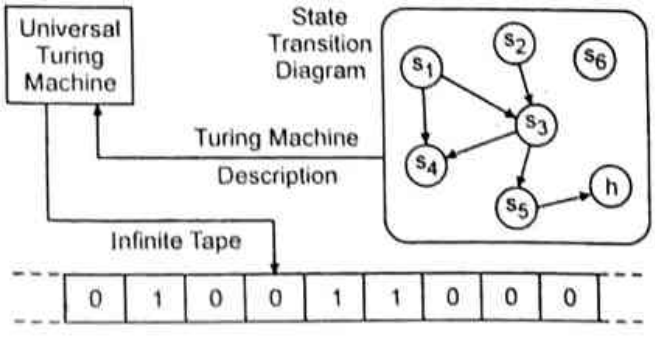
- A " Universal Turing Machine", or 'UTM', can emulate any other specific Turing machine, by defining states and symbols. The UTM is defined with certain capabilities.

(i) The UTM can define the symbols that the specific Turing machine will use.

(ii) It can define the symbols that encode the states and transition rules for the specific Turing machine.

(iii) It can encode the rules for that specific Turing machine onto the input tape.

- A single-tape UTM needs to define a marker to mark the end of the "specific" program and the start of the specific TM's program and its data.
- As noted, it is simpler to describe a UTM with multiple tapes.
- The Universal Turing Machine is remarkably similar to the Von Neumann model of a computer, where both programs and data can be stored on the same medium.
- Any modern computer capable of copying a program file from one medium to another, and later running that program, follows this architecture.



The Universal Turing Machine Emulates Other Turing Machines :

- As noted, it is easier to describe any UTM as having three tapes, although it does not require them.
- The first tape encodes the set of states for the specific Turing machine to be emulated.
- The second tape is an input for that specific TM.
- The third tape is a working memory for the current state of the emulated machine.
- **The UTM's states follow the following processes :**

(1) Read the current cell in the "data tape".

(2) Read the "program tape" to find the instruction for the current status and the current data cell, and note this on the "status tape". This instruction includes the new state.

(3) If the new state is "halt", then set the UTM itself into the "halt" state; otherwise proceed to step 4.

(4) Apply the instruction from the state to the "data tape". This might rewrite a cell, and move the "data tape" to the right or left.

(5) Update the "status tape".

(6) Continue at step # 1 above.

A Universal Turing Mahcine Can Emulate Itself :

- Since any specific Turing machine can be emulated by a Universal Turing machine, it follows that a UTM could emulate itself.
- Again, it would be simplest to add another program tape for the emulated UTM, and an extra data tape as well.

Q.26.(B) Explain typical encoding scheme used to code symbols on Universal Turing machine. **CT : S-07(5M)**

Ans.

- A TM M that takes as input, an encoded version of another TM M and an input w to M', M will simulate the processing w on M'.
- If M halts on input w, M will halt if M rejects on input w, M will reject.
- M is a general purpose TM M is a program run on the general purpose TM.

Encoding scheme : We need a way to encode a TM so that it can be provided as input to T.

Define sets : Q $\{q_1 q_2, \ldots\}$ = set of all possible states that may appear in any TM = set of all possible tapes symbols that can be written on any TM. So for any TM.

Assumptions : q_1 will always be the start state. There will only be 1 final state. It will be q_2. The machine always halts in q_2.

Q.27. Show that Halting Problem of Turing machine is undecidable. **CT : W-11(6M)**
CS : S-11(2M), W-11(7M), W-12(6M)

Ans. Theorem 1 : The halting problem is undecidable.

Proof : This is going to be proven by "proof by contradiction".

- Suppose that the halting problem is decidable. Then there is a Turing machine T that solves the halting problem.
- That is, given a description of a Turing machine M (over the alphabet Σ) and a string w, T writes "yes" if M halts on w and "no" if M does not halt on w, and then T halts.

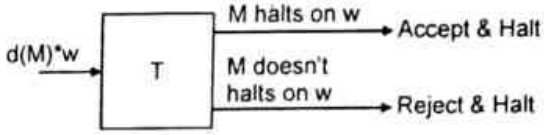


Fig. Turing Machine T

- We are now going to construct the following new Turing machine T_c.
- First we construct a Turing machine T_m by modifying T so that if T accepts a string and halts, then T_m goes into an infinite loop (T_m halts if the original T rejects a string and halts).

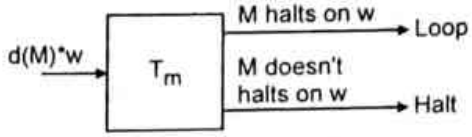


Fig. Turing Machine T_m

- Next using T_m we are going to construct another Turing machine T_c as follows :

  T_c takes as input a description of a Turing machine M, denoted by d (M), copies it to obtain the string d (M)* d (M), where* is a symbol that separates the two copies of d (M) and then supplies d (M)* d (M) to the Turing machine T_m.

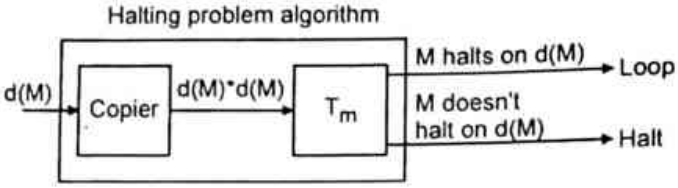


Fig. Turing Machine T_c

- Let us now see what T_c does when a string describing T_c itself is given to it.
- When T_c gets the input d (T_c), it makes a copy, constructs the string d (T_c) * d (T_c) and gives it to the modified T.
- Thus the modified T is given a description of Turing machine T_c and the string d (T_c).

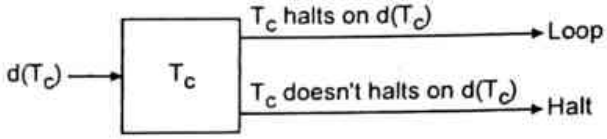


Fig. Turing Machine T_c on input d (T_c)

- The way T was modified, the modified T is going to go into an infinite loop if T_c halts on d (T_c) and halts if T_c does not halt on d (T_c).
- Thus T_c goes into an infinite loop if T_c halts on d (T_c) and it halts if T_c does not halt on d (T_c). This is a contradiction.
- This contradiction has been deduced from our assumption that there is a Turing machine that solves the halting problem.
- Hence that assumption must be wrong. Hence there is no Turing machine that solves the halting problem.

Q.28. Explain restricted Turing Machine.

Ans. Multi-stack Machines :

- A deterministic two stack machine is a deterministic Turing machine with a read only input, two different storage tapes which are acting as a stack.

- If a head moves on either tape a blank is printed on that, tape.

Counter Machine :

- Counter Machines are nothing but off-line Turing Machine whose storage tapes are semi-infinite and contains only two symbols Z and B.
- Z serves as a bottom of stack marker which is scanned by the tape at initial itself and it does not appear on any other set.
- An integer I can be stored on the tape by moving tape head of the Turing machine I cells right to the Z, similarly a stored number can be incremented or decremented by moving the tape head to the right or to the left respectively.

Restriction on the number of states and symbols :

- Another kind of restriction to the Turing machine is the limit on the tape alphabet Γ and/or on the number of states used.
- This type of Turing machine is less powerful than the other type of Turing machines because of limit on the number of symbols and states.

Q.29. Explain composite TM and iterated TM

Ans. Composite TM and Iterated TM :

- In order to able to describe composite TMs without having to describe every primitive operation as a separate TM, it is sometimes helpful to use mixed notation in which some but not all of the states of a TM are shown.
- For example in figure, which is an abbreviated version of the one in figure has a fairly obvious meaning.
- If the current type symbol is execute the TM T : if it is b. halt; and if it is anything else, crash.
- In the first case, assuming T halts; and if it is anything else, crash.
- In the first case, assuming T halts, repeat the execution of T until T halts scanning some symbol other than a; if at that point the tape symbol is b, halt, otherwise crash.

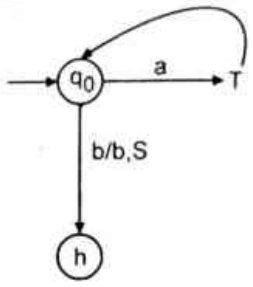


Fig. (a)

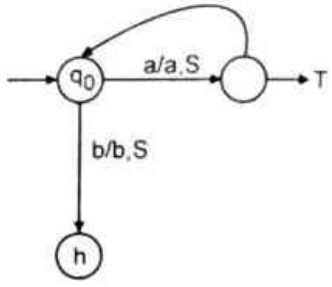
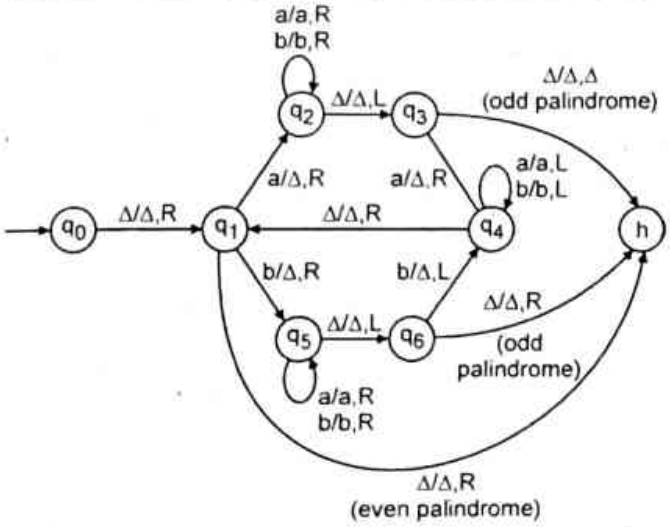


Fig. (b)

Q.30.(A) Describe the power of Turing machine over finite state machine. **CT : W-10(4M)**

Ans.

- Let us consider the non-regular language pal of palindromes over {a, b} and construct non-determinism, we can get a hint as how to proceed by imagining how we might check a long string by hand to see if it is a palindrome.
- One way would be to position our two forefingers at the two ends. As our eyes jump repeatedly back and forth compairing the two ends symbols, our fingers, which are the markers that tell our eyes how far to go, gradually move towards the centre.
- The generality of TM model allows us to translate this almost directly into a TM algorithm.
- The markers at each end are blank squares.
- Moving the markers towards the centre corresponds to erasing (i.e, changing to blanks) the symbols that have just been tested.
- The tape head moves repeatedly back and forth, compairing the symbol at one end of the remaining nonblank string to the symbol at the other end.
- The transition diagram is shown in figure below.
- Again the tape alphabet is {a, b} the same as the input alphabet.
- We trace the moves made by the machine for three different input strings a non-palindrome, an even length palindrome, and an odd length palindrome.



Q.30.(B) Describe power of TM over push down autometa. **CT : S-12(2M)**

Ans.

- A Turing machine (TM) is the most powerful kind of automaton.
- In fact, Turing machines are capable of solving any problem that can be solved by computation.
- Turning machines are similar to PDAs : They consist of states and transitions, and use an infinite tape for storage.
- Unlike PDAs, however, the use of the tape by a Turing machine is not limited to pushing and popping symbols : it can move the tape either left or right after writing a symbol on the tape.
- Unlike a PDA, the tape is used as both the input string and the temporary storage.
- In addition, when the Turing machine terminates we can consider the contents of the tape to be output. In this way, a Turing machine is more powerful than a recognizer for strings in a language.

Q.31. Write a short note on Church's hypothesis.

CT : S-06,09(5M), S-12, W-08(3M), S-13, W-11(4M)

OR Write short note on Church-Turing thesis.

CS : W-11(5M)

Ans. Church's Hypothesis :

- The assumption that the intuitive notion of "computable function" can be identified with the class of partial recursive functions is known as Church's hypothesis or the Church -Turing thesis.
- As long as our intuitive notion of "computable" places no bound on the number of steps or the amount of storage, it would seem that the partial recursive functions are intuitively computable, although some would argue that a function is not "computable" unless we can bound the computation in advance or at least establish whether or not the computation eventually terminates.
- In this hypothesis, it is less clear whether the class of partial recursive functions includes all computable functions.
- Logicians have presented many other formalisms such as the λ -calculus, post systems and general recursive functions.
- All have been shown to define the same class of functions. i.e. the partial recursive functions.
- In addition, abstract, computer models, such as the random access machine [RAM], also give rise to partial recursive functions.
- The RAM consists of an infinite number of memory words, numbered 0, 1 .....each of which can hold any integer and a finite number of arithmetic registers capable of holding any integer.
- Integers may be decoded into the usual sorts of computer instructions.

Q.32. Write a short note on counter machines.

Ans. Counter Machines :

- Counter machines are off line turing machines whose storage tapes are semi-infinite and whose tape alphabets contain only two symbols, Z and B (blank).
- Symbol Z, which serves as a bottom of stack marker, appears initially on the cell scanned by the tape head and may never appear on any other cell.
- An integer i can be stored by moving the tape head i cells to the right of Z.
- A stored number can be incremented or decremented by moving the tape head right or left.
- We can test whether a number is zero by checking whether Z is scanned by the head, but

whether two numbers are equal cannot be directly tested.

Figure shows a counter machine.

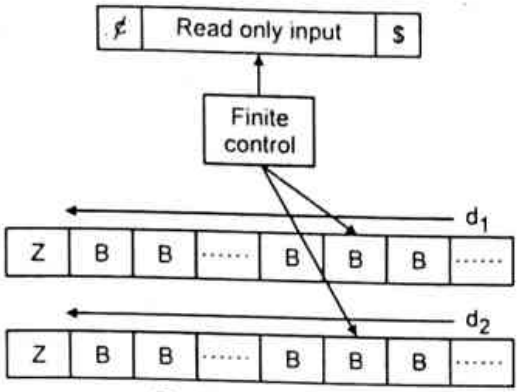


Fig. Counter Machine

- ¢ and $ are used as end markers on the input. Here Z is the non blank symbol on each type.
- Counter machine can be described by the state, the input tape conents, the position of the input head and the distance of the storage heads from the symbol Z (d_1 and d_2 as shown in fig above) are called as counts on the tapes.
- The counter machine can really store a count on each tape and tell if that count is zero.

Q.33. Prove Simulation of random access machines by Turing machines.

Ans. Theorem : A Turing machine can simulate a RAM, provided that the elementary RAM instructions can themselves be simulated by a TM.

Proof :

- We use a multi-tape TM M to perform the simulations.
- One tape of M holds the words of the RAM that have been given values.
- The tape looks like

  $\#0^* v_0 \#1^* v_1 \#10^* v_2 \#.....\#i^* v_i \#.....,$

  Where v_i is the contents, in binary, of the i^{th} word.
- At all times, there will be some finite number of words of the RAM that have been used, and M needs only to keep a record of values up to the larger numbered word that has been used so far.
- The RAM has some finite number of arithmetic registers.
- M uses one tape to hold each register contents, one tape to hold the location counter, which contains the number of the word from which the next instruction is to be taken, and one tape as a memory address register from which the number of a memory word may be placed.
- Suppose that the first 10 bits of an instruction denote one of the standard computer operations, such as LOAD, STORE, ADD, and so on, and that the remaining bits denote the address of an operand.
- While we shall not discuss the details of implementation for all standard computer instructions, an example should make the techniques clear.
- Suppose the location counter tape of M holds number i in binary. M searches its first tape from the left, looking for # i*.
- If a blank is encountered before finding idea there is no instruction in word i, so the RAM and M halt.
- If #i* is found, the bits following * up to next # are examined.
- Suppose the first 10 bits are the code for "ADD" to register "2" and the remaining bits are some number j in binary.
- M adds 1 to i on the location counter tape and copies j onto the memory address tape.
- Then M searches for #j* on the first tape, again starting from the left (note that # 0* marks the left end).
- If #j* is found, we assume word j holds 0 and go on to the next instruction of the RAM. If # j* v_1 is added to the contents of register 2, which is stored on its own tape. We then repeat the cycle with the next instruction.

Q.34. Prove that a four-counter machine can simulate an arbitrary Turing machine.

Ans.

- Let a stack have k – 1 tape symbols, Z_1, Z_2, ... Z_{k-1}. Then we can represent the stack

$Z_{i_1}, Z_{i_2}, \ldots\ldots Z_{i_m}$ uniquely by the count in base k. $j = i_m + k\, i_{m-1} + k^2 i_{m-2} + \ldots\ldots + k^{m-1} i_1$,

- Those integers whose base-k representation contains the digit 0 do not represents stack.
- Let us assume that the symbol Z_r is pushed onto the top of the stack $Z_{i_1}, Z_{i_2}, \ldots\ldots Z_{i_m}$. The count associated with $Z_{i_1}, Z_{i_2}, \ldots\ldots Z_{i_m}$ Z_r is jk + r.
- For getting this new count, the counter machine repeatedly moves the head of the first counter one cell to the left and the head of second k cells to the right.
- When the head of the first counter reaches the nonblank symbol, the second counter will hold the count jk. Then r is added to the count.
- If Z_{i_m} i.e. the top symbol of the stack is popped, j should be replaced by [j/k], the integer part of j/k.
- We repeatedly keep on decrementing the count on the first counter by k and adding one to the second count. When the first count is zero, the second count will be [j/k].
- If the count j is stored on one counter, the four counter machine can copy j to another counter, computing j mod k in its finite control.

Q.35. Prove that a two-counter machine can simulate an arbitrary Turing machine.

Ans.

- Let four counters have counts *i, j, k,* and *l*.
- One counter can represent these four by the number $n = 2^i 3^j 5^k 7^l$. Since 2, 3, 5 and 7 are primes, *i, j, k* and *l* can be uniquely recovered from n.
- To increment *i, j, k* or *l* by 1, we multiply n by 2, 3, 5 or 7, respectively.
- For doing this, if another counter is set to zero, we can move the head of this counter 2, 3, 5 or 7 cells to the right, each time we move the head of the first counter one cell to the left.
- When the first counter holds zero, the second will hold 2n, 3n, 5n or 7n respectively.
- In the same way n is divided by 2, 3, 5, or 7 respectively for decrementing *i, j, k,* or *l* by 1.
- The two counter machine can be used to determine the next move of the four counter machine.
- The two-counter machine always scans the same cell of the input tape as the four-counter machine does.
- The state of the four counter 'machine' is stored in the finite control of the two-counter machine, the two counter machine has only to determine which, if any *i, j, k* and *l* are 0.
- By passing n from one counter to the other, the finite control of the two-counter machine can determine if n is divisible by 2, 3, 5, 7 or any product of these.

Q.36. Construct CSG for the language :

(i) $\mathbf{L = \{a^n b^n c^m d^m \mid n \geq 1, m \geq 1\}}$

(ii) $\mathbf{L = \{a^n b^m c^m d^n \mid n \geq 1, m \geq 1\}}$

(iii) $\mathbf{L = \{a^i b^n c^n \mid i \geq 0, n \geq 1\}}$

Ans. $S \rightarrow AABBCDS_1 \mid AABBCD$

$S_1 \rightarrow AABBCDS_1 \mid AABBCD$

$BB\ AA \rightarrow AABBCAA \rightarrow AACCBB \rightarrow BBC$

$aA \rightarrow aa \quad aAbB \rightarrow aa\ bb$

$bB \rightarrow bb \quad bC \rightarrow bc \quad cD \rightarrow cd$

(ii) $S \rightarrow AABCDDS_1 \mid AABCDD$

$S_1 \rightarrow AABCDDS_1 \mid AABCD$

$BAA \rightarrow AAB \quad CAA \rightarrow AAC \quad CB \rightarrow BC$

$DDC \rightarrow CDD$

$aA \rightarrow aa \quad aaB \rightarrow aab$

$bC \rightarrow bc \quad dD \rightarrow dd$

(iii) $S \rightarrow ABBCC\ S_1\ ABBCC$

$S_1 \rightarrow ABBCC\ S_1 \mid ABBCC$

$BBA \rightarrow ABB\ \ CCA \rightarrow ACC\ \ CCBB \rightarrow BBCC$

$A \rightarrow a\ \ aA \rightarrow aa\ \ abB \rightarrow abb \rightarrow bBcC \rightarrow bbcc.$

Q.37. Give a grammar which is CFG but not CSG.

Ans. The grammar which is CSG but not CFG is given by a language $L = \{a^n b^n c^n \mid n \geq 1\}$

The grammar is

$S \rightarrow abc \mid aAbc,$

$Ab \rightarrow bA$

$Ac \rightarrow Bbcc,$

$bB \rightarrow Bb,$

$aB \rightarrow aa \mid aaA.$.

We can see how this works by looking at a derivation of $a^3b^3c^3$.

$S \Rightarrow aAbc \Rightarrow abAc \Rightarrow abBbcc$

$\Rightarrow aBbbcc \Rightarrow aaAbbcc \Rightarrow aabAbcc$

$\Rightarrow aabbAcc \Rightarrow aabbBbccc$

$\Rightarrow aabBbbccc \Rightarrow aaBbbbccc$

$\Rightarrow aaabbbccc.$

The grammar which is CFG but not CSG is as follows.

$S \rightarrow BC$

$B \rightarrow AB \mid \in$

$A \rightarrow 011 \mid 1$

$C \rightarrow DC \mid \in$

$D \rightarrow 01$

Q.38. Explain the difference between Top down and bottom up parsing. **CS : S-11(3M)**

OR Define Top down parsing and Bottom up parsing. **CS : S-09(4M)**

Ans. Top-down parsers :

- It starts constructing the parse tree at the top (root) of the parse tree and move down towards the leaves.
- It is easy to implement by hand, but work with restricted grammars.

  Example : Predictive parsers (e.g., LL(k))
- **Bottom-up parsers :** It build the nodes on the bottom of the parse tree first. It is suitable for automatic parser generation, handle a larger class of grammars.

  Example : shift-reduce parser (or LR (k) parsers)
- Both are general techniques that can be made to work for all languages (but not all grammars!).
- Given language can be described by several grammars.
- Both of these grammars describe the same language.

  E -> E + Num

  E -> Num

  E -> Num + E

  E -> Num
- The first one, with it's left recursion, causes problems for top down parsers.
- For a given parsing technique, we may have to transform the grammar to work with it.

Q.39. Explain Left Factoring with example. **CS : W-09(3M)**

Ans. Left Factoring :

- If a grammar contains pair of productions of the form $A \rightarrow \alpha B \mid \alpha \gamma$ where both the productions begins with non empty string derived from α, then the grammar contains left factoring.

  Removing left factoring in most introductory compiler texts done as follows :
- Given

  $A \rightarrow xy \mid xz$

  Left factoring yields :

  $As \rightarrow y \mid z$

  $A \rightarrow xAs$

  Now that's perform the transformations in both ordering.
- A predictive parser (a top-down parser without backtracking) insists that the grammar must be left-factored.

  grammar→ a new equivalent grammar suitable for predictive parsing

  stmt→ if expr then stmt else stmt |

  if expr then stmt
- When we see if, we cannot know which production rule to choose to re-write stmt in the derivation.
- In general,

  $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$

  where α is non-empty and the first symbols of β_1 and β_2 (if they have one) are different.
- When processing α, we cannot know whether expand A to $\alpha\beta_1$ or A to $\alpha\beta_2$

- But, if we re-write the grammar as follows
  $A \rightarrow \alpha A'$
  $A' \rightarrow \beta_1 | \beta_2$ so, we can immediately expand A to $\alpha A'$
- For each non-terminal A with two or more alternatives (production rules) with a common non-empty prefix, let say
  $A \rightarrow \alpha\beta_1 | .....| \alpha\beta_n | \gamma_1 | .....| \gamma_m$
  Convert it into
  $A \rightarrow \alpha A' | \gamma_1 | .....| \gamma_m$
  $A' \rightarrow \beta_1 | .....| \beta_n$

LINEAR BOUNDED AUTOMATA

Q.40. Explain model of Linear Bounded Automata.

CT : W-06(4M), W-07,09, S-10(5M), W-08(3M)

OR Describe the linear bounded automata and language it accepts. ***CS : S-11(5M)***

OR Write short note on linear bounded automata. ***CT : S-11,12, W-12(3M), CS : W-11(5M)***

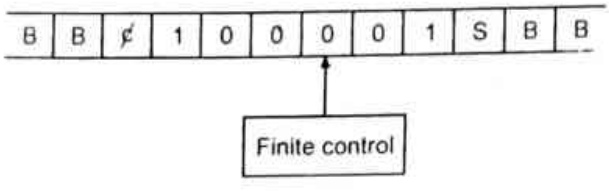
Ans. Linear Bounded Automata :

- Linear Bounded Automata is nothing but the non-deterministic Turing machine such that :

(1) Its alphabet consists of two special symbols which are used as end-markers. Those symbols are ¢ and $.

(2) The movement of tape head of the Linear Bounded Automata is restricted between ¢ and $, up to $, on the right hand side where as ¢ on the left hand side.

- From the above restrictions we can conclude that the tape of the linear bounded automata is having finite number of symbols over it and the movement of tape head is being restricted to that portion of tape only.
- A simple model of the Linear Bounded Automata is as given below.



Q.41. Discuss the equivalence of linear bounded automata and context sensitive grammar.

Ans.

- We now show that except for the fact that the LBA can accept $\in$ while a CSG cannot generate $\in$.
- The linear bounded automata's accepts exactly the context sensitive languages.

Theorem : If L is CSL, then L is accepted by some linear bounded automata.

Proof :

- Presented with cw$ on its tape, the LBA starts by writing the symbol S on a second track below the leftmost symbol of w.
- If $w = \in$, the LBA instead halts without accepting.
- Next the LBA repeatedly guesses a production and a position in the sentential form written on the second track.
- It applies the production, shifting the portion of the sentential form to the right whenever the sentential form expands.
- If, however, the new sentential form is longer than w, the LBA halts without acceptance.
- Thus the LBA will accept w if there is a derivation $S \Rightarrow w$ such that no intermediate sentential form is longer than w.
- But since the right side of any production in a CSG is as long or longer than the left side, there could not be a derivation $S \Rightarrow \alpha \Rightarrow w$, where α is longer than w.
- Thus the LBA accepts all and only, the words generated by the CSG.

Q.42. Prove the following.

(i) Every context sensitive language L is recursive.

(ii) There exists recursive language that is not contexts sensitive.

Ans.

(i) Every context sensitive language L is recursive.

- Suppose L is generated by the CSG G. Then L can be accepted by a LBA M which is essentially a special type of NTM.

- Now we show how to modify M so that it still accepts L but so that any sequence of moves it to halt or crash.
- The modified machine M' still uses the markers <and>, and they remain in place ; however, M' is no longer prohibited from moving its tape head to the right of >.
- The LBA M simulates a derivation in G, using the second "track" of the tape between the two markers.
- At the end of of each iteration of this simulation, M' performs two additional steps.
- First, M' copies the current string of the derivation onto the empty portion of the tape to the right.
- In effect, this results in M' keeping a history of the derivation, the string on the tape to the right of the > marker looks like this :

  $S \Delta \alpha_1 \Delta \alpha_2 \Delta \ldots\ldots \Delta a_n$

  where α_i is the current string after the ith iterations of the simulation.
- Second M' determines whether α_n, the most recent string, is a duplicate of any of the previous α_i s and crashes if it is.
- With this modification, it is no longer possible for M' to loop forever.
- Eventually , one of the three things happens.
- Either the current string in the simulated derivation matches the original input x, in which case M halts; or an iteration of the loop results in a string longer than x, in which case M' crashes; or one of strings obtained shows up for the second time (because all these strings are length |x| or shorter), in which case M' crashes.
- The strings x for which some sequence of moves of M's result in the first outcome precisely those that can be generated by G. We conclude that L is accepted by a NTM for which every sequence of moves ends in a halt or a crash.

(ii) There exists recursive language that is not context sensitive.

Theorem : There is a recursive language L over {a, b} such that $L - \{\in\}$ is not context - sensitive.

Proof :

- The proof uses a diagonal argument let us fix an infinite set $A = \{A_1, A_2, \ldots.\}$ so that any CSL over {a, b} is generated by a CSG in which the start symbol is A_1 and all variables of elements of A.
- The remainder of the proof depends on finding a way to enumerate all such CSGs explicitly, in the form $G_1, G_2, \ldots\ldots$
- Here is one way to do it. First, list all CSGs for whcih the only variable is A_1 and every productions $\alpha \rightarrow \beta$ has $|\alpha|, |\beta| \leq 1$; there are only finitely many.
- Next, list those not already listed for which the only variable are A_1 and A_2 and every production $\alpha \rightarrow \beta$ has $|\alpha|, |\beta| \leq 2$.
- If we continue in this way, it is clear that every CSG subject to previous condition is listed eventually.
- Now let $x_1, x_2, \ldots\ldots$be the non null elements of {a, b}* listed in canonical order and let L be the language of all strings x_i for which $x_i \notin L(Gi)$.
- We claim that L is recursive and context sensitive.
- Each L(Gi) is recursive, according to theorem that every context sensitive language is recursive.
- Therefore, given $x \in \{a, b\}^*$ we can determine whether $x \in L$ by finding i for which $x = x_i$ and then testing x for membership in L(Gi). Therefore L is recursive.
- If L is context-sensitive, L is the same as L (Gi) for some i.
- In this case, consider whether the corresponding x_i can be an element of L. If it is, then by definition of L, $x \notin L(Gi) = L$.
- If it is not, however, then $x_i \notin L(Gi)$, and therefore $x \in L$ by definition of L.
- This contradiction proves that L cannot be context - sensitive.

TM AS COMPUTER OF FUNCTION

Q.43.(A) Explain turing machine as computer of function.

Ans. Turing Machine as a Computer Functions :

- In addition to using Turing machine as a language recognizer or acceptor, Turing machine can also be used as a computer functions, means a function whose arguments are integers and result is an integer.
- For example $f\ (i_1, i_2, ...i_n) = m$, where $i_1, i_2, ...i_n$ are arguments all integers, and m is a result.
- The string ω is presented into the form of $\# \omega \#$, that is string, ω is surrounded by blanks symbols from both sides and is placed on the left most square of tape, the head of Turing machine is positioned at the right most blank symbol which immediately follows the string ω.

Q.43.(B) What do you mean by computable function? "Turing machine computes functions of one; function of two and functions of multiple arguments". Comment on this statement. **CT : S-07(5M)**

Ans. Computable function :

- A function f on a certain domain is said to be computable if there exists a TM that computes the value of f for all arguments in its domain. A function is uncomputable if no such TM exists.

"Turing machine computes functions of one; functions of multiple arguments".

Proof :

- Assume we want to construct a TM which can compute $f(X_1, ...X_m)$ over N for given arguments $a_1.....a_m$.
- Initially, the input $a_1, a_2.....a_m$ appears on the input tape separated by markers $x_1 ..... x_m$.
- The computed value $f(a_1.....a_m)$ say, c appears on the input tape, once the computation is over.
- To locate C we need another marker say y. The value c appears to right of X_m and to the left of y
- To make the construction simpler, we use the tally representation where 0 is represented by string of b's.
- A positive integer n is represented by a string consisting of n 1's. So the initial tape expression takes the form

  $1^{a1} x_1 1^{a2} x_2.....1^{am} x_m$ by.

  As a result of computation, the initial ID $q_0 1^{a1} x_1 1^{a2} x_2.....1^{am} x_m$ by is changed to a terminal ID of the form

  $1^{a1} x_1 1^{a2} x_2.....1^{am} x_m 1^c q'y$ for some $q' \in Q$.
- In fact, the position of q' in a terminal ID is immaterial and it can appear any where in Res (x).
- The computed value is found between x_m and y. Sometimes we may have to omit the leading b's.
- We say that a function $f(x_1.....x_m)$ is Turing-computable for arguments $a_1.....a_m$. If there exists a turing maching for which

  $q_0 a^{a1} x_1 1^{a2} x_2.....1^{am} x_m$ by

  $\vdash ID_n$ where ID_n is a terminal ID containing $f(a_1.....a_m)$ to the left of y.

Q.44. Design a TM over $\Sigma(0, 1)$ such that TM accept all string having even no. of 1.

OR Design a Turing machine that can accept the string over {0, 1} to recognize all strings containinig even number of 1's.

CT : W-07(8M), S-08(5M)

Ans. L = {011, 11011011011, 11010101010, ...}

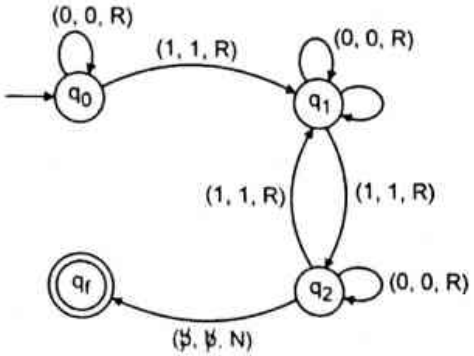
Simulation :

b	0	1	0	1	0	1	1	b	
	↑	↑	↑	↑	↑	↑	↑	↑	even
	q_0	q_0	q_1	q_1	q_2	q_2	q_1	q_2	
b	1	1	0	1	1	0	1	b	
	↑	↑	↑	↑	↑	↑	↑	↑	odd
	q_0	q_1	q_2	q_2	q_1	q_2	q_2	q_1	

Transition Table :

Present State	Tape Symbol		
	0	1	ß
→ q_0	q_0, 0, R	q_1, 1, R	-
q_1	q_1, 0, R	q_2, 1, R	-
q_2	q_2, 0, R	q_1, 1, R	q_f, ß, N
* q_f	-	-	-

Transition Diagram :



Q.45. Construct a TM over Σ (0 1), TM accepts 2's complement of string.

OR Design T. M. to find 2's complement of a given binary number. ***CT : S-07,12(6M), W-08(7M), CS : S-09(8M), S-10(7M)***

Ans. L = 0 0 1 1 0 1

Simulation :

(ß 0 0 1 1 0 1 ß)
↑ ↑ ↑ ↑ ↑ ↑ ↑
q_0 q_1

(ß 0 0 1 1 0 1 ß)
↑
q_2

(ß 0 0 1 1 1 1 ß)
↑
q_2

(ß 0 0 1 0 1 1 ß)
↑
q_2

(ß 0 0 0 0 1 1 ß)
↑
q_2

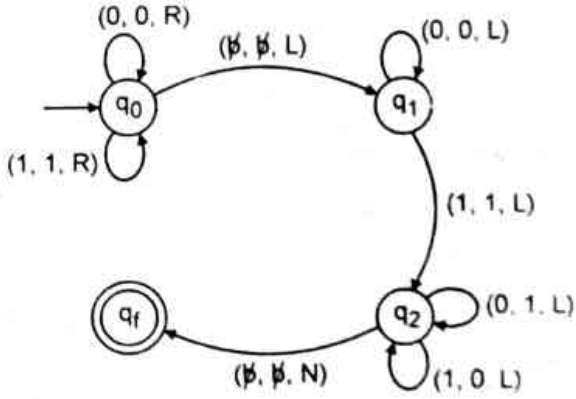
(ß 0 1 0 0 1 1 ß)
↑
q_2

(ß 1 0 0 0 1 1 ß)

Transition Table :

Present State	Tape Symbol		
	0	1	ß
→ q_0	(q_1, 0, R)	(q_0, 1, R)	(q_1, ß, L)
q_1	(q_1, 0, L)	(q_2, 1, L)	-
q_2	(q_2, 1, L)	(q_2, 0, L)	(q_f, ß, N)
* q_f	-	-	-

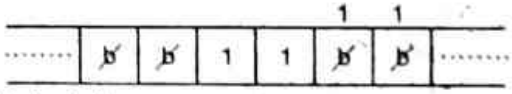
Transition Diagram :



Q.46. Design a TM that copies the string of 1's without loosing the original string.

CT : S-06(8M)

Ans.

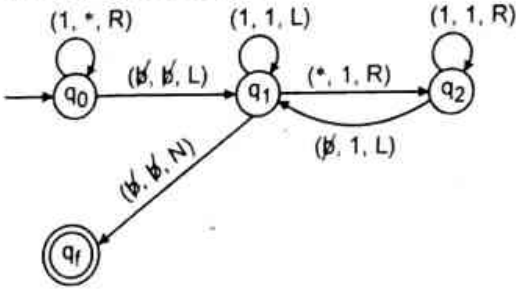


ß * 1 1 ß
↑
q_1
ß 1 1 1 ß
↑
q_2
ß 1 1 1 ß
↑
q_2
ß 1 1 1 1
↑ ↑
q_1 q_1

Transition Table :

Present State	Tape Symbol		
	1	*	ß
$\rightarrow q_0$	q_0, *, R	-	q_1, ß, L
q_1	q_1, 1, L	q_2, 1, R	(q_f, ß, N)
q_2	q_2, 1, R	-	q_1, 1, L
$^*q_f$	-	-	-

Transition Diagram :



Q.47. Design a TM which perform addition of two positive integer i.e, here m and n are the I/P and O/P should be m + n.

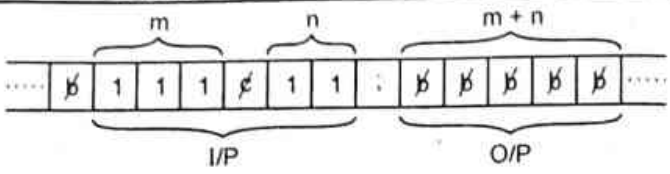
OR Design a TM to add two given integers. Also show the transition graph and transition table. Illustrate your design with suitable sequence of inputs with the help of ID's . ***CS : S-10(9M)***

OR Design TM to add two given integers.

OR Design TM to perform addition of two unary number. ***CS : S-11(8M), W-09(6M)***

OR Design a Turing Machine, to operate on UNARY number and perform addition of UNARY number. The TM will perform addition by input restore method.

CS : S-09(8M)



Let m = 3, n = 2

∴ m + n = 3 + 2 = 5

Transition Table :

Present State	Tape Symbol				
	1	*	ß	¢	;
$\rightarrow q_0$	q_1, *, R	-	-	q_0, ¢, R	q_f, ;, N
q_1	q_1, 1, R	-	-	q_2, ¢, R	q_3, ;, R
q_2	q_2, 1, R	-	-	-	q_3, ;, R
q_3	q_3, 1, R	-	q_4, 1, L	-	-
q_4	q_4, 1, L	q_0, *, R	-	q_4, ¢, L	q_4, ;, L
$^*q_f$	-	-	-	-	-

Simulation :

ß 1 1 ¢ 1 1 ; ß ß ß ß
↑
q_0
ß * 1 ¢ 1 1 ; ß ß ß ß
↑ ↑
q_1 q_1
ß * 1 ¢ 1 1 ; ß ß ß ß
↑ ↑ ↑
q_2 q_2
ß * 1 ¢ 1 1 ; ß ß ß ß
↑
q_3
ß * 1 ¢ 1 1 ; ß ß ß

Simulation :

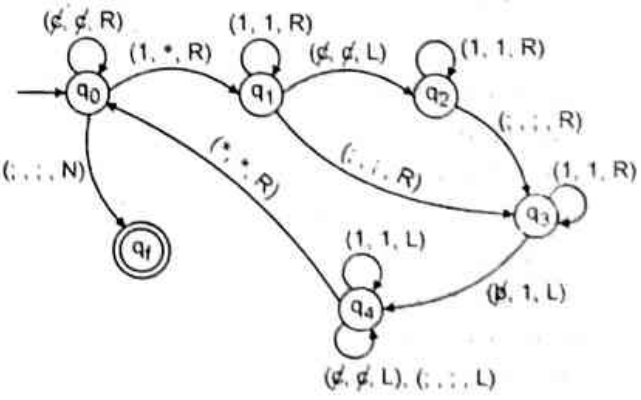
For m = 5 and n = 3,

m + n = 5 + 3 = 8

```
Ƀ 1 1 1 1 1 ¢ 1 1 1 ; Ƀ
  ↑
  q0
Ƀ * 1 1 1 1 ¢ 1 1 1 ; Ƀ
    ↑ ↑ ↑ ↑ ↑
   q1    q1
Ƀ * 1 1 1 1 ¢ 1 1 1 ; Ƀ
        ↑ ↑ ↑ ↑
       q2  q2
Ƀ * 1 1 1 1 ¢ 1 1 1 ; Ƀ
                  ↑
                 q3
Ƀ * 1 1 1 1 ¢ 1 1 1 ; 1 Ƀ
        ↑ ↑ ↑ ↑ ↑
                q4
Ƀ * 1 1 1 1 ¢ 1 1 1 ; 1 Ƀ
  ↑ ↑ ↑ ↑ ↑ ↑
  q0      q4
Ƀ * 1 1 1 1 ¢ 1 1 1 ; 1 Ƀ
  ↑ ↑
 q4 q0
Ƀ * * 1 1 1 ¢ 1 1 1 ; 1 Ƀ
      ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
     q0   q1 q2   q2 q3
Ƀ * * 1 1 1 ¢ 1 1 1 ; 1 1 Ƀ
    ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
   q0                q4
Ƀ * * * 1 1 ¢ 1 1 1 ; 1 1 Ƀ
        ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
       q1   q2     q3
Ƀ * * * 1 1 ¢ 1 1 1 ; 1 1 Ƀ
                    ↑
                   q3
Ƀ * * * 1 1 ¢ 1 1 1 ; 1 1 1 Ƀ
      ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
                   q4
Ƀ * * * * 1 ¢ 1 1 1 ; 1 1 1 Ƀ
          ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
        q1 q1 q2    q3
Ƀ * * * * 1 ¢ 1 1 1 ; 1 1 1 1 Ƀ
                    ↑
                   q4
Ƀ * * * * 1 ¢ 1 1 1 ; 1 1 1 1 Ƀ
        ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
                   q4
```

```
Ƀ * * * * * ¢ 1 1 1 ; 1 1 1 1 Ƀ
            ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
           q2   q3       q3
Ƀ * * * * * ¢ 1 1 1 ; 1 1 1 1 1 Ƀ
          ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
                      q4
Ƀ * * * * * ¢ 1 1 1 ; 1 1 1 1 1 Ƀ
          ↑ ↑
         q0 q0
Ƀ * * * * * ¢ * 1 1 ; 1 1 1 1 1 Ƀ
            ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
           q1  q2 q3
Ƀ * * * * * ¢ * 1 1 ; 1 1 1 1 1 1 Ƀ
          ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
                      q4
Ƀ * * * * * ¢ * * 1 ; 1 1 1 1 1 1 1 Ƀ
            ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
                      q4
Ƀ * * * * * ¢ * * * ; 1 1 1 1 1 1 1 Ƀ
              ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
             q1 q2       q2
Ƀ * * * * * ¢ * * * ; 1 1 1 1 1 1 1 1 Ƀ
            ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
           q4              q4
Ƀ * * * * * ¢ * * * ; 1 1 1 1 1 1 1 1 Ƀ
              ↑
             q0
```

Transition Diagram :



Q.48. Design a TM which perform subtraction of two positive integers i.e, m and n are I/P and O/P should be m – n such that m ≥ n .

OR Design a TM to calculate proper subtraction of two integer number.

CS : S-13(8M), CT : S-13(8M)

VBD

OR Design TM to compute

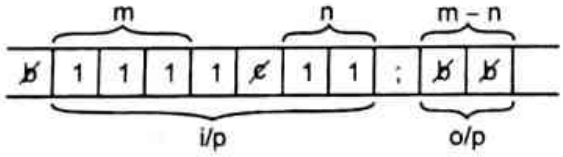
$$f(m, n) = \begin{cases} m - n & ; \quad m > n \\ 0 & ; \quad m \leq n \end{cases}$$

CS : W-12(7M)

OR Design a TM that can compute proper subtraction i.e. m = n, where m and n are positive integer. m – n is defined as m – n if m > n and 0 if m < = n.

CT : S-10(8M)

Ans.



Let m = 4, n = 2, m – n = 4 – 2 = 2

Simulation :

```
 1 1 1 1 ¢ 1 1 ;
 ↑
 q0
x 1 1 1 ¢ 1 1 ;
   ↑ ↑ ↑ ↑ ↑
  q1 q1 q1 q1 q2
x 1 1 1 ¢ y 1 ;
↑ ↑ ↑ ↑
q3    q3
x 1 1 1 ¢ y 1 ;
   ↑
  q0
x x 1 1 ¢ y 1 ;
     ↑ ↑ ↑ ↑ ↑
    q1 q1 q1 q2 q2
x x 1 1 ¢ y y
     ↑ ↑ ↑
    q3 q3 q3
x x 1 1 ¢ y y ;
     ↑
    q0
x x x 1 ¢ y y ;
       ↑ ↑ ↑ ↑ ↑
      q1 q1 q2 q2 q2
x x x 1 ¢ y y ; b
                 ↑
                q4
x x x 1 ¢ y y ; 1
     ↑ ↑ ↑ ↑ ↑
    q3       q3
x x x 1 ¢ y y ; 1
       ↑
      q0
```

x x x x ¢ y y ; 1 ♭
↑ ↑ ↑ ↑ ↑ ↑
q_1 q_2 q_2 q_2 q_4 q_4

x x x x ¢ y y ; 1 1
↑ ↑ ↑ ↑
q_3 q_3

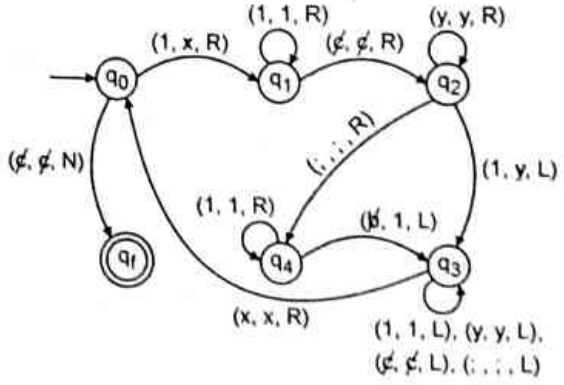
x x x x ¢ y y ; 1 1
↑
q_0

x x x x ¢ y y ; 1 1
↑
q_f

Transition Table :

Present state	Tape Symbol					
	1	x	y	¢	;	♭
$\rightarrow q_0$	q_1, x, R	-	-	q_f, ¢, N	-	-
q_1	q_1, 1, R	-	-	q_2, ¢, R	-	-
q_2	q_3, y, L	-	q_2, y, R	-	q_4, ;, R	-
q_3	q_3, 1, L	q_0, x, R	q_3, y, L	q_3, ¢, L	q_3, ;, L	-
q_4	q_4, 1, R	-	-	-	-	q_3, 1, L
$*q_f$	-	-	-	-	-	-

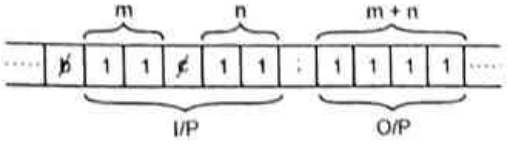
Transition Diagram :



Q.49. Design a TM which perform multiplication of two positive integer i.e, m and n are I/P and m × n is O/P. ***CT : W-11(11M)***

OR Design turing m/c to calculate multiplication of two integer number. ***CS : S-12(9M)***

Ans.

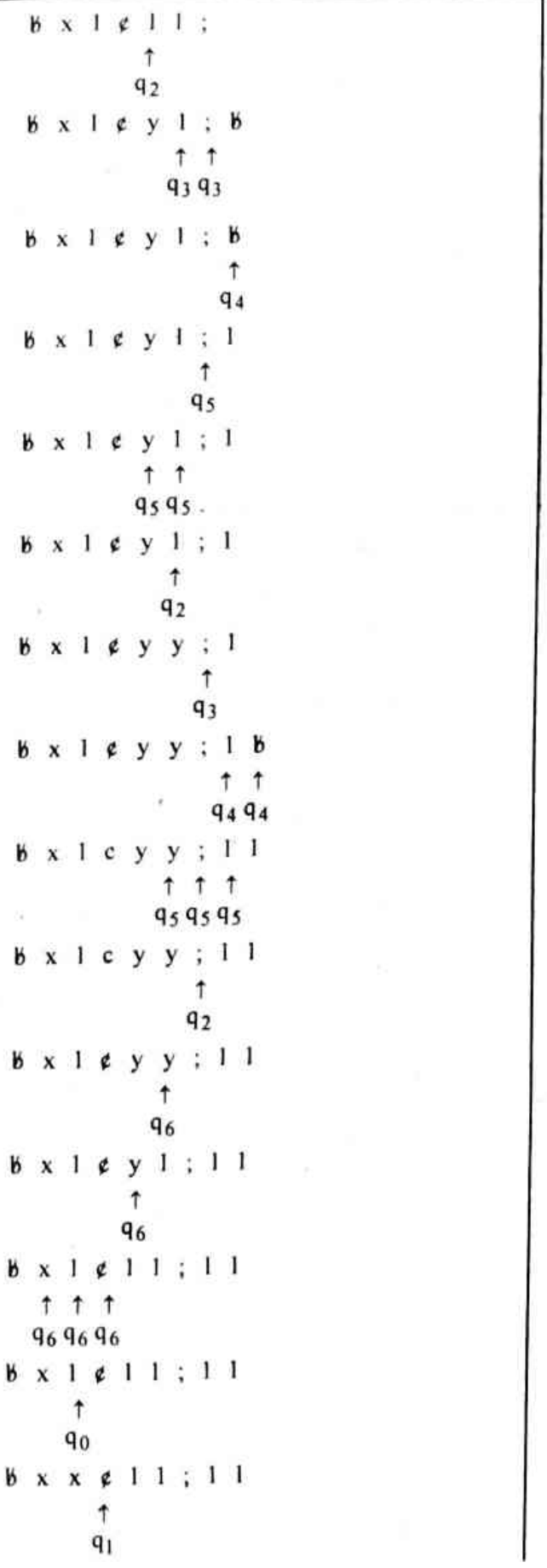


Let m = 2 and n = 2, m × n = 2 × 2 = 4.

Simulation :

♭ 1 1 ¢ 1 1 ;
↑
q_0

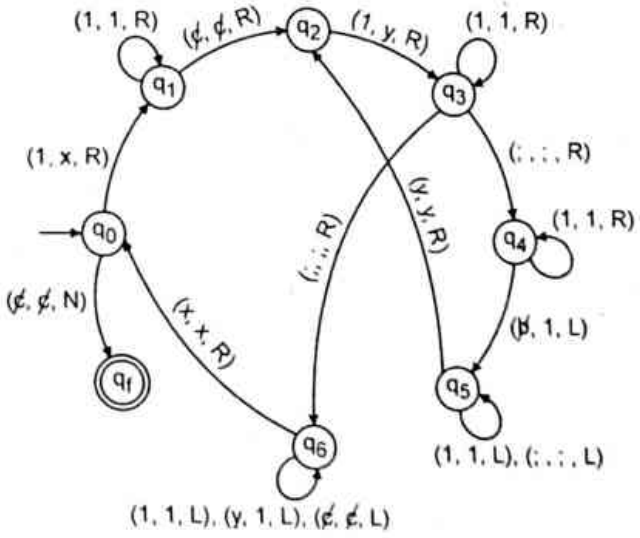
♭ x 1 ¢ 1 1 ;
↑ ↑
q_1 q_1


B x 1 ¢ 1 1 ;
q2
B x 1 ¢ y 1 ; B
q3 q3
B x 1 ¢ y 1 ; B
q4
B x 1 ¢ y 1 ; 1
q5
B x 1 ¢ y 1 ; 1
q5 q5
B x 1 ¢ y 1 ; 1
q2
B x 1 ¢ y y ; 1
q3
B x 1 ¢ y y ; 1 B
q4 q4
B x 1 c y y ; 1 1
q5 q5 q5
B x 1 c y y ; 1 1
q2
B x 1 ¢ y y ; 1 1
q6
B x 1 ¢ y 1 ; 1 1
q6
B x 1 ¢ 1 1 ; 1 1
q6 q6 q6
B x 1 ¢ 1 1 ; 1 1
q0
B x x ¢ 1 1 ; 1 1
q1
B x x ¢ 1 1 ; 1 1
q2
B x x ¢ y 1 ; 1 1
q3 q3
B x x ¢ y 1 ; 1 1 1
q4 q4 q4
B x x ¢ y 1 ; 1 1 1
q5 q5
B x x ¢ y 1 ; 1 1 1
q2
B x x ¢ y y ; 1 1 1
q3
B x x ¢ y y ; 1 1 1
q4
B x x ¢ y y ; 1 1 1 B
q4
B x x ¢ y y ; 1 1 1 1
q5 q5
B x x ¢ y y ; 1 1 1 1
q2
B x x ¢ y y ; 1 1 1 1
q6
B x x ¢ y 1 ; 1 1 1 1
q6
B x x ¢ 1 1 ; 1 1 1 1
q6 q6
B x x ¢ 1 1 ; 1 1 1 1
q0

Transition Table :

Present state	Tape Symbol					
	1	x	y	¢	♭	;
$\rightarrow q_0$	q_1, x, R	-	-	$q_f, ¢, N$	-	-
q_1	$q_1, 1, R$	-	-	$q_2, ¢, R$	-	-
q_2	q_3, y, R	-	-	-	-	$q_6, ;, L$
q_3	$q_3, 1, R$	-	-	-	-	$q_4, ;, R$
q_4	$q_4, 1, R$	-	-	-	$q_5, 1, L$	-
q_5	$q_5, 1, L$	-	q_2, y, R	-	-	$q_5, ;, L$
q_6	$q_6, 1, L$	q_0, x, R	$q_6, 1, L$	$q_6, ¢, L$	-	-
$*q_f$	-	-	-	-	-	-

Transition Diagram :



Q.50. Construct a TM which perform a division operation and leaves some remainder.

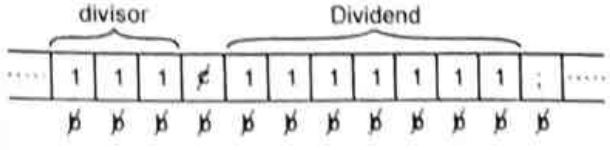
OR Design TM that finds integer division i.e. $\left(\text{int}\left(\frac{m}{n}\right)\right)$ **of two number m and n where** $(m \geq n)$.

CT : W-12(9M)

OR Design turing machine to accept the language L = { w : |w| is multiple of 3}. Also draw Transition diagram.

CS : S-12(7M)

Ans.



Simulation :

♭ 1 1 1 ¢ 1 1 1 1 1 1 1 ;
↑
q_0

♭ x 1 1 ¢ 1 1 1 1 1 1 1 ;
↑ ↑ ↑
q_1

♭ x 1 1 ¢ 1 1 1 1 1 1 ;
↑
q_2

♭ x 1 1 ¢ y 1 1 1 1 1 1 ;
↑ ↑ ↑ ↑
q_3 q_3

♭ x 1 1 ¢ y 1 1 1 1 1 1 ;
↑
q_6

♭ x x 1 ¢ y 1 1 1 1 1 1 ;
↑ ↑ ↑ ↑
q_1 q_2

♭ x x 1 ¢ y y 1 1 1 1 1 ;
↑ ↑
q_3 q_3

♭ x x 1 ¢ y y 1 1 1 1 1 ;
↑
q_0

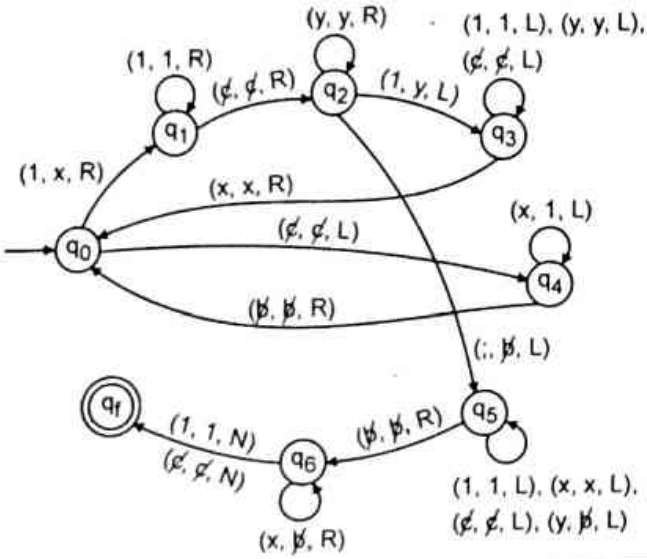
♭ x x x ¢ y y 1 1 1 1 1 ;
↑ ↑ ↑ ↑
q_1 q_2 q_2 q_2

♭ x x x ¢ y y y 1 1 1 1 ;
↑ ↑ ↑
q_3 q_3 q_3

Transition Table :

Present State	Tape Symbol					
	1	x	y	;	¢	♭
$\rightarrow q_0$	q_1, x, R	-	-	-	q_4, ¢, L	-
q_1	q_1, 1, R	-	-	-	q_2, ¢, R	-
q_2	q_3, y, L	-	q_2, y, R	q_5, ♭, L	-	-
q_3	q_3, 1, L	q_0, x, R	q_3, y, L	-	q_3, ¢, L	-
q_4	-	q_4, 1, L	-	-	-	q_0, ♭, R
q_5	q_5, 1, L	q_5, x, L	q_5, ♭, L	-	q_5, ¢, L	q_6, ♭, R
q_6	q_f, 1, N	q_6, ♭, R	-	-	q_f, ¢, N	-
$*q_f$	-	-	-	-	-	-

Transition Diagram :



Q.51. Construct a Turing Machine to recognize palindrome string over {a, b} with 'C' as separator. **CS : S-11(7M)**

OR Construct TM For language
L = {WCW | w ∈ (a, b)*} **CS : S-10(7M)**

OR Design a TM for given language.
L = {WCW | w ∈ (a + b)*} **CT : W-12(8M)**

Ans. The language can be represented as
L = {WCW | w ∈ (a, b)}

Analysis :

(1) The language is variable.

(2) String may either start with a or b and end with a or b.

(3) The string are palindromes of odd length.

Valid string : abcba, aabcbaa, abbcbba,

Invalid string : abcab, bacba, ...

Algorithm :

(1) In this, the string formed contains C as the middle element. Hence TM traverse upto the symbol C and then change the state to q_1.

(2) Then check the symbol on right side of C, if it is a, on left we should also get a.

(3) If so, again go right to check next symbol and repeat the procedure till machine gets blank. If blank comes then move to left and again it should get blank. If so then accept the string otherwise reject.

Transitions of TM :

$\delta(q_0, a) \rightarrow (q_0, a, R)$

$\delta(q_0, b) \rightarrow (q_0, b, R)$

$\delta(q_0, C) \rightarrow (q_1, C, R)$

$\delta(q_1, a) \rightarrow (q_2, X, L)$

$\delta(q_1, b) \rightarrow (q_4, X, L)$

$\delta(q_2, C) \rightarrow (q_2, C, L)$

$\delta(q_4, C) \rightarrow (q_4, C, L)$

$\delta(q_2, a) \rightarrow (q_3, X, R)$

$\delta(q_4, b) \rightarrow (q_3, X, R)$

$\delta(q_3, C) \rightarrow (q_3, C, R)$

$\delta(q_3, X) \rightarrow (q_3, X, R)$

$\delta(q_3, a) \rightarrow (q_2, X, L)$

$\delta(q_2, X) \rightarrow (q_2, X, L)$

$\delta(q_3, b) \rightarrow (q_4, X, L)$

$\delta(q_3, \flat) \rightarrow (q_5, \flat, L)$

$\delta(q_4, X) \rightarrow (q_4, X, L)$

$\delta(q_5, C) \rightarrow (q_5, C, L)$

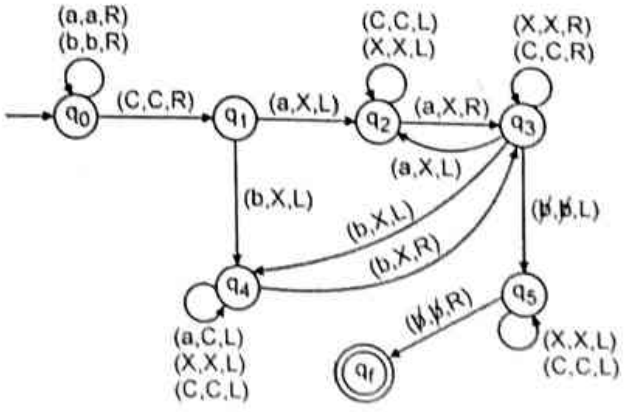
$\delta(q_5, X) \rightarrow (q_5, X, L)$

$\delta(q_5, \flat) \rightarrow (q_f, \flat, R)$

Transition Table :

Present/State	Tape Symbol				
	0	b	X	Y	♭
$\rightarrow q_0$	q_0, a, R	q_0, b, R	-	q_1, C, R	-
q_1	q_2, X, L	q_4, X, L	-	-	-
q_2	q_3, X, R	-	q_2, X, L	q_2, C, L	-
q_3	q_2, X, L	q_4, X, L	q_3, X, R	q_3, C, R	$q_5, \flat, L$
q_4	q_4, C, L	q_3, X, R	q_4, X, L	q_4, C, L	-
q_5	-	-	q_5, X, L	q_5, C, L	$q_f, \flat, R$
$*q_f$	-	-	-	-	-

Transition Graph of TM :



LR (0) GRAMMARS

Q.52. Define :

(1) Right sentential form

(2) Handle

(3) Viable prefix

(4) Augmented grammar.

Ans.

(1) Right sentential form :

- Given a context free grammar G. If $S \to \alpha$, where α may contain nonterminals as well as terminals ; then α is said to be sentential form of G.
- A right sentential form is a sentential form G, that can be derived by rightmost derivation.

(2) Handle :

- A handle of a right sentential form γ is a production $A \to \beta$ and position of β in γ, where the string β will be found and replaced by A to produce the previous right sentential form in the right most derivation of γ

  i.e.if $S \to \alpha\ A\beta \to \alpha\gamma\beta$,

  then $A \to \gamma$, is a handle of $\alpha\gamma\beta$ in the position following α.

  Consider the grammar

  $E \to E + E \mid E * E \mid$ id and the rightmost derivation

  $E \to E + E \to E + E * E \to E + E * id$

  $\to E + id * id \to id + id * id$

- Given below are the handles of the sentential forms occuring in the above derivation.

Sentential form	Handle
id + id * id	$E \to$ id, at the position preceding +
E + id* id	$E \to$ id, at the position following +
E + E * id	$E \to$ id, at the position following *
E+ E * E	$E \to E*E$, at the position following +
E + E	$E \to E +E$, at the position preceding the endmarker

(3) Viable prefix :

- Viable prefix of a right sentential form is that prefix of sentential form which may contain handle but no symbol to the right of the handle
- Therefore if a finite state machine recognizing viable prefixes of the right sentential forms is constructed it can be used to guide the selection of handle in the shift reduce parser.

(4) Augmented grammar :

- To construct a DFA recognizing the viable prefixes we make use of augmented grammar, which is defined as follows.
- If G = (V, T, P, S) is a given grammar then the augmented grammar will be

  $G_1\ (V \cup \{S_1\}, T, P \cup \{S_1 \to S\})$ i.e. we add a unit production $S_1 \to S$ to the grammar G and make S_1 to be the new start symbol, the resulting grammar will be an augmented grammar.
- The purpose of augmenting the grammar is to make it explicitly clear when to accept the string and stop parsing it will be when the parser is on the verge of carrying out reduction using $S_1 \to S$.
- Consider the transition $\delta(A \to \alpha, B\beta, \in) = B \to \gamma$.
- This transition is required because, if the current state is $A \to \alpha, B\beta$ that means we have not yet seen a string derivable from the non terminal β and $\beta \to \gamma$ being a production of the grammar, unless we see γ we will not get B, therefore we have walk on the path recognizing γ, which requires entering into the state identified $\beta \to \gamma$ without consuming any input symbol.

Q.53. Define LR (0) grammar.

Ans. G is an LR (0) grammar if

(i) its start symbol does not appear on the right side of any production, and

(ii) for every viable prefix γ of G, whenever $A \to \alpha$, is a complete item valid for γ, then no other complete item nor any item with a terminal to the right of the dot is valid for γ.

There is no prohibition against several incomplete iterms being valid for γ as long as no complete item is valid.

Q.54. Explain how to compute the FIRST and FOLLOW.

Ans.

- Consider $A \rightarrow \alpha_1 | \alpha_2 | \alpha_3 | .....$ are the A-productions in the grammar and a parser is required to predict, which α_i is the right choice for the expansion of non terminal A, then to compute this information two functions are used called FIRST and FOLLOW.
- FIRST (α) = Set of those terminals with which strings derivable from α starts.
- If α = XYZ, then FIRST(α) is computed as follows :

  FIRST (α) = FIRST (XYZ) = {X}, if X is terminal

  Otherwise, if X is non-terminal contain ϵ-production then

  FIRST (α) = FIRST (XYZ) = FIRST (X) , if X does not derive to an empty string, i.e. if FIRST (X) does not contain ϵ if FIRST (X) contain ϵ then

  FIRST (α) = FIRST (XYZ) = FIRST (X) = $\{\epsilon\} \cup$ FIRST (YZ)
- FIRST (YZ) is computed in a identical manner as :

  FIRST (YZ) = {Y} if Y is terminal

  Otherwise,

  FIRST (YZ) = FIRST (Y), if Y does not derive to an empty string i.e. if FIRST (Y) does not contain ϵ

  if FIRST (Y) contain ϵ then

  FIRST (YZ) = FIRST (Y) – $\{\epsilon\} \cup$ FIRST (Z)

Example : Consider a grammar :

$S \rightarrow ACB | CbB | Ba$

$A \rightarrow da | BC$

$B \rightarrow g | \epsilon$

$C \rightarrow h | \epsilon$

FIRST (S) = FIRST (ACB) $\cup$ FIRST (CbB) $\cup$ FIRST (Ba) .....(I)

FIRST (A) = FIRST (da) $\cup$ FIRST (BC)

= {d} $\cup$ FIRST (BC) .....(II)

FIRST (B) = FIRST (g) $\cup$ FIRST (ϵ)

= {g, ϵ}

FIRST (C) = FIRST (h) $\cup$ FIRST (ϵ)

= {h, ϵ}

Therefore,

FIRST (BC) = FIRST (B) – {ϵ} $\cup$ FIRST (C)

= {g, ϵ} – {ϵ} $\cup$ {h, ϵ}

= {g, h, ϵ}

Substituting in (II) we get :

FIRST (A) = {d} $\cup$ {g, h, ϵ}

= {d, g, h, ϵ}

FIRST (ACB) = FIRST (A) – {ϵ} $\cup$ FIRST (CB)

= {d, g, h} $\cup$ FIRST (CB) .....(III)

FIRST (CB) = FIRST (C) – {ϵ} $\cup$ FIRST (B)

= {h, ϵ} – {ϵ} $\cup$ {g, ϵ}

= {g, h, ϵ}

Therefore substituting in (III) we get :

FIRST (ACB) = {d, g, h} $\cup$ {g, h, ϵ}

= {d, g, h, ϵ}

Similarly,

FIRST (CbB) = FIRST (C) – {ϵ} $\cup$ FIRST (bB)

= {h, ϵ} – {ϵ} $\cup$ {b}

= {b, h}

Q.55. Construct LR (0) for S → 1S1 / 0S0/ 10.

CT : W-11(9M)

Ans. The Augmented grammar is,

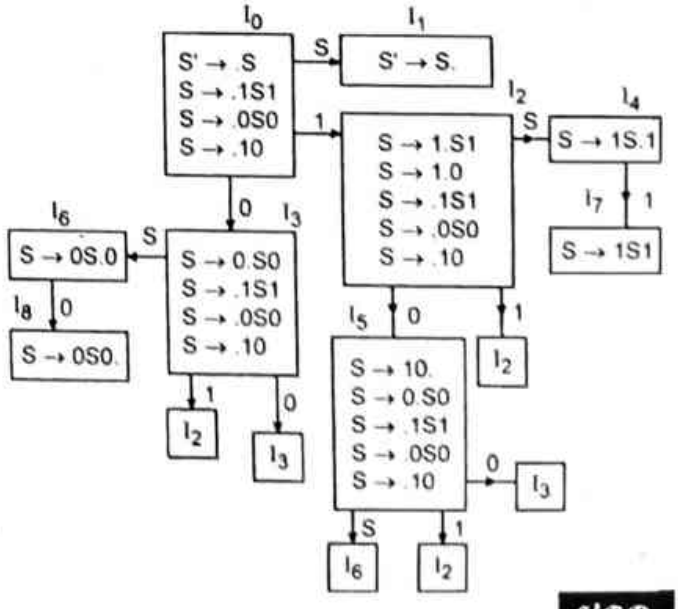
$S' \rightarrow S$

$S \rightarrow 1S1$ .....(1)

$S \rightarrow 0S0$ .....(2)

$S \rightarrow 10$ .....(3)

The transition diagram for the DFA which recognizes all the viable prefix of the given grammar is as follows :



FOLLOW (S) = {$, 1, 0}

Parsing table :

	ACTION			GOTO
	1	0	$	S
I_0	S_2	S_3		1
I_1			Accept	
I_2	S_2	S_5		4
I_3	S_2	S_3		6
I_4	S_7			
I_5	R_3, S_2	R_3, S_3	R_3	6
I_6		S_8		
I_7	R_1	R_1	R_1	
I_8	R_2	R_2	R_2	

∴ Table contains multiple entries. Thus grammar is not LR (0)

Q.56. Draw LR (0) Parsing table for the grammar

S → AaAb /BbBa

A → ∈

B → ∈

Design SLR (1) Parser. ***CT : S-10(13M)***

Ans. The Augmented grammar is,

S' → S

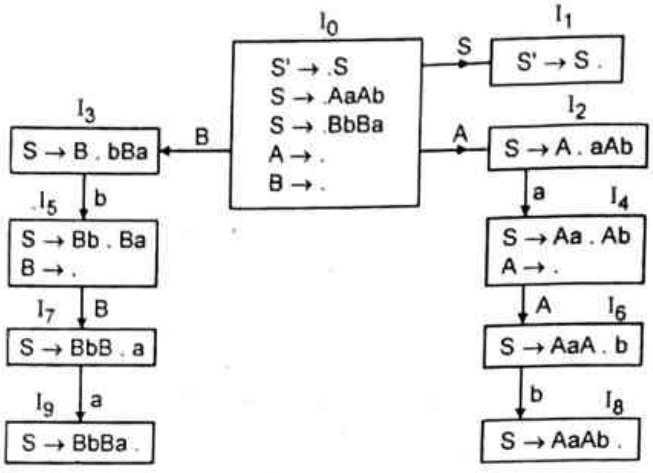
S → AaAb .....(1)

S → BbBa .....(2)

A → ∈ .....(3)

B → ∈ .....(4)

The transition diagram for the DFA which recognizes all the viable prefix of the given grammar is as follows :



Parsing table :

	ACTION			GOTO		
	a	b	$	S	A	B
I_0	R_3, R_4	R_3, R_4		1	2	3
I_1			Accept			
I_2	S_4					
I_3		S_5				
I_4	R_3	R_3			6	
I_5	R_4	R_4				7
I_6		S_8				
I_7	S_9					
I_8			R_1			
I_9			R_2			

∵ Table contains multiple entries. Thus grammar is not SLR(1)

Q.57. Construct the non-empty set of items for the following LR(0) grammar. Also construct the SLR parsing table for the same.

E → E + T / T

T → T * F/ F

F → (E) / id

OR Collect LR(0) items for the following grammar.

E → E + T

E → T

T → T* F

T → F

F → (E)

F → id

CT : S-08(8M)

VBD

OR Construct LR(0) parser for following grammer.

E → E + T/T

T→ T * F/F

F → (E)/id

Also show the parsing of string id + id * id. **CS : S-09(8M)**

OR For following grammar, generate various state and design DFA using LR(0) method.

E → E + T/T

T → T * F/F

F → (E)/id **CS : W-09(10M)**

Ans. The augmented grammar is,

E' → E

E → E + T .....(1)

E → T .....(2)

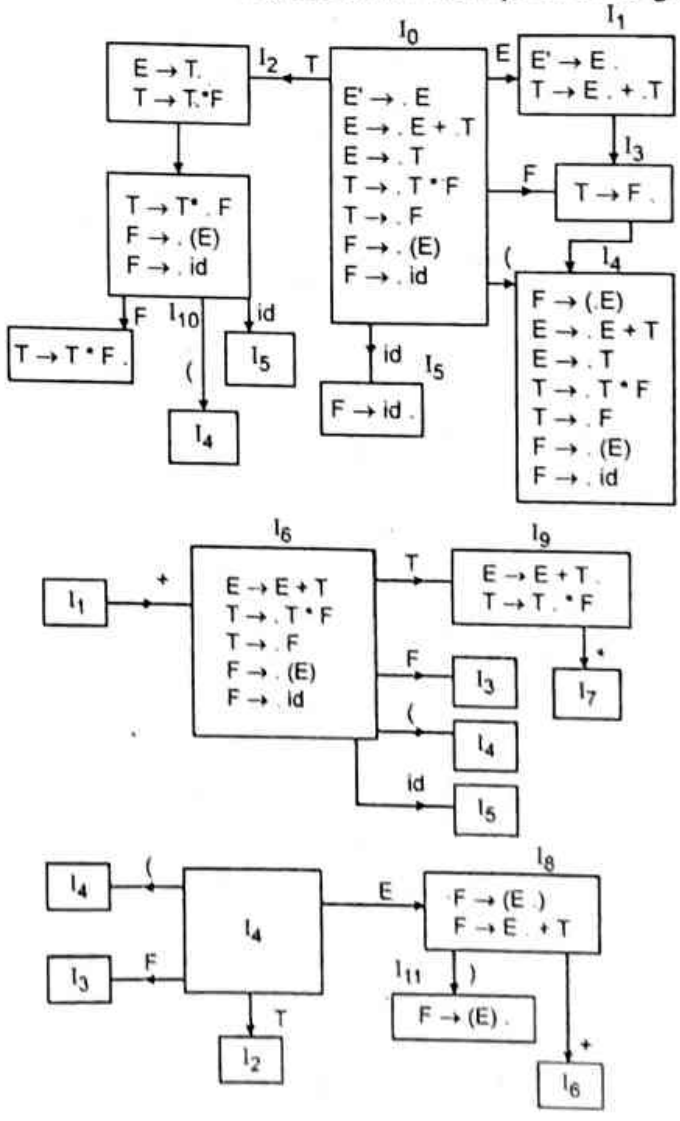
T → T * F .....(3)

T → F .....(4)

F → (E) .....(5)

F → id .....(6)

The transition diagram for the DFA which recognizes all the viable prefix of the given grammar is as follows :



FOLLOW (E) = {$, +,)}

FOLLOW (T) = {$, +,), *}

FOLLOW (F) = {$, +,), *}

Parsing table :

	ACTION						GOTO		
	+	*	id	(	)	$	E	T	F
I_0			S_5	S_4			1	2	3
I_1	S_6					Accept			
I_2	R_2	S_7			R_2	R_2			
I_3	R_4	R_4			R_4	R_4			
I_4				S_4			8	2	3
I_5	R_6	R_6			R_6	R_6			
I_6			S_5	S_4				9	3
I_7			S_5	S_4					10
I_8	S_6				S_{11}				
I_9	R_1	S_7			R_1	R_1			
I_{10}	R_3	R_3			R_3	R_3			
I_{11}	R_5	R_5			R_5	R_5			

∵ Table does not contains multiple entries. Thus grammar is LR (0).

The various steps in the parsing of the input string "id + id *id " are shown below in terms of the contents of stack and unspent input.

Stack Contents	Input	Moves
S	id + id * id $	shift id
S id	+id * id $	reduce by E → id
SE	+ id * id $	shift +
SE +	id * id $	shift id
SE + id	* id $	reduce by E → id
SE + E	* id $	shift *
SE + E *	id $	shift id
SE + E * id	$	reduce by E → id
SE + E * E	$	reduce by E → E * E
SE + E	$	reduce by E → E + E
S E	$	accept.

Q.58. Is the following grammar LR(0).

S → Aa / bAc/Bc / bBa

A → d

B → d **CT : S-06(8M)**

Ans. The augmented grammar is,

S' → S

S → Aa ....(1)

S → bAc ....(2)

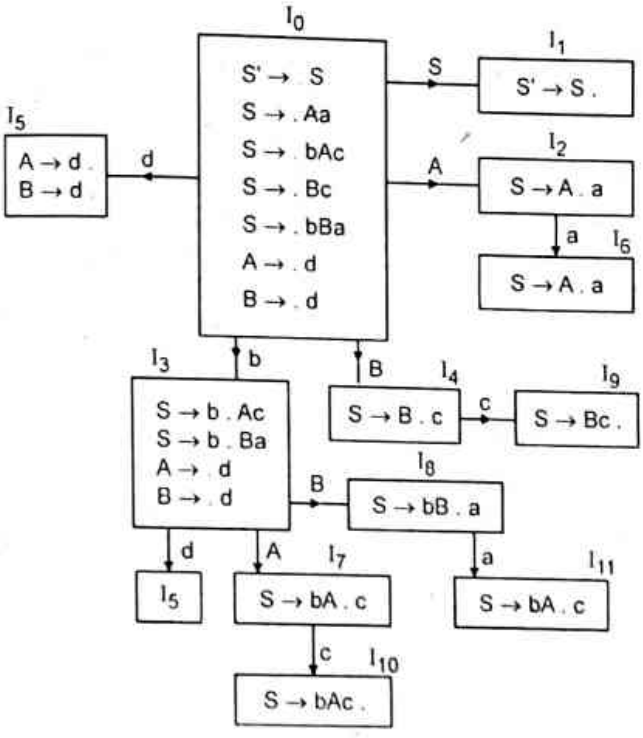
S → Bc .....(3)

S → bBa .....(4)

A → d .....(5)

B → d .....(6)

The transition diagram of DFA which recognizes all the viable prefixes of grammar are as follows :



FOLLOW (S) = {S},

FOLLOW (A) = {a, c},

FOLLOW (B) = {a, c}

The parsing table :

	ACTION					GOTO		
	a	b	c	d	$	S	A	B
I_0		S_3		S_5		1	2	4
I_1					Accept			
I_2	S_6							
I_3				S_5			7	8
I_4				S_9				
I_5	R_5, R_6			R_5, R_6				
I_6					R_1			
I_7				S_{10}				
I_8	S_{11}							
I_9					R_3			
I_{10}					R_2			
I_{11}					R_4			

∵ Table contains multiple entries. Thus grammar is not LR (0).

Q.59. Construct LR(0) parser for following grammar.

S → CC

C → aC / d **CS : W-11(9M)**

OR Construct LR(0) parsing table for following grammar.

S → CC

C → aC

C → d **CT : W-08(5M)**

Ans. The augmented grammar is,

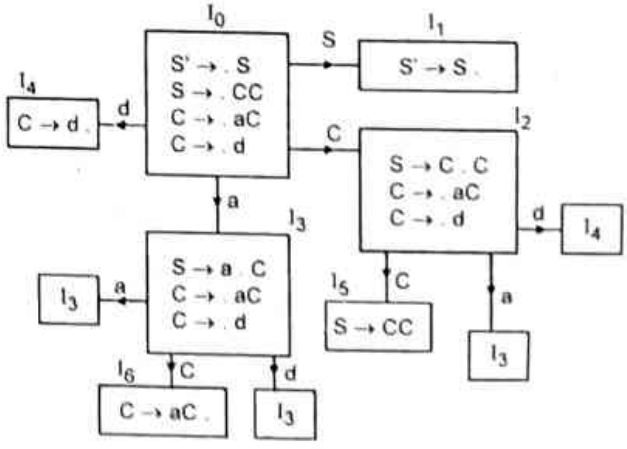
S' → S

S → CC .....(1)

C → aC .....(2)

C → d .....(3)

The transition diagram of the DFA which recognizes all the viable prefixes of given grammar are :



FOLLOW (S) = {$}

FOLLOW (C) = {a, d}

Parsing table :

	ACTION			GOTO	
	a	d	S	S	C
I_0	S_3	S_4		1	2
I_1			Accept		
I_2	S_3	S_4			5
I_3	S_3	S_4			6
I_4	R_3	R_3			
I_5			R_1		
I_6	R_2	R_2			

∵ Table does not contains multiple entries. Thus grammar is LR (0)

Q.60. Construct non empty sets of items for following LR(0) grammar.

E → E + T/ T

T → T * F /F

F → id ***CT : S-12, W-10(8M)***

OR Construct the nonempty set of items for following LR(0) grammar.

E → E + T /T

T → T * F/F

F → id ***CT : S-09(9M)***

Ans. The augmented grammar will be,

E' → E

E → E + T .....(1)

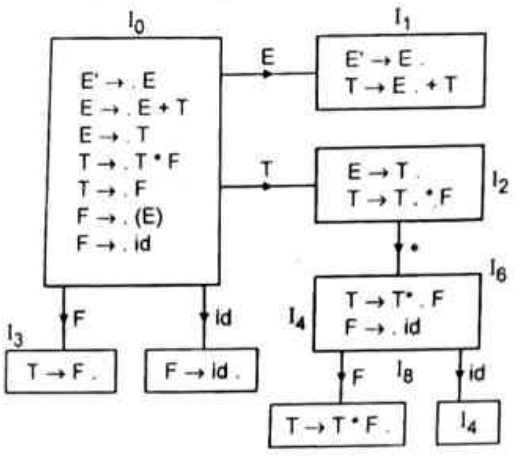
E → T .....(2)

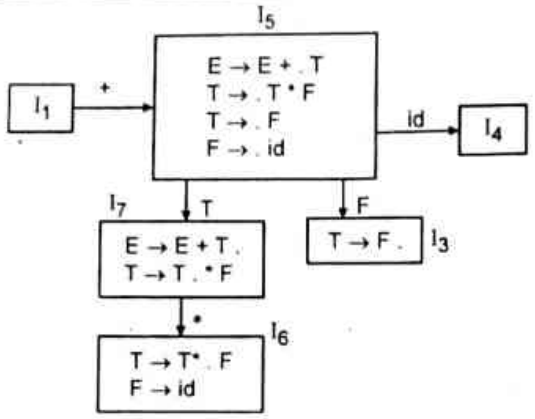
T → T * F .....(3)

T → F .....(4)

F → id .....(5)

The transition diagram of the DFA which recognizes all the viable prefixes of given grammar are :





FOLLOW (E) = {$, +}

FOLLOW (T) = {$, +, *}

FOLLOW (F) = {$, +, *}

	ACTION				GOTO		
	+	*	id	S	E	T	F
I_0			S_4		1	2	3
I_1	S_5			Accept			
I_2	R_2	S_6		R_2			
I_3	R_4	R_4		R_4			
I_4	R_5	R_5		R_5			
I_5			S_4			7	3
I_6			S_4				8
I_7	R_1	S_6		R_1			
I_8	R_3	R_3		R_3			

∵ Table does not contains multiple entries. Thus grammar is LR (0).

Q.61. Consider the grammar :

S → M / U

M → iMe M / a

U → i S / i MeU

Get SLR (1) parsing table for the grammar.

Ans. The augmented grammar will be,

S' → S

S → M .....(1)

S → U .....(2)

M → iMeM .....(3)

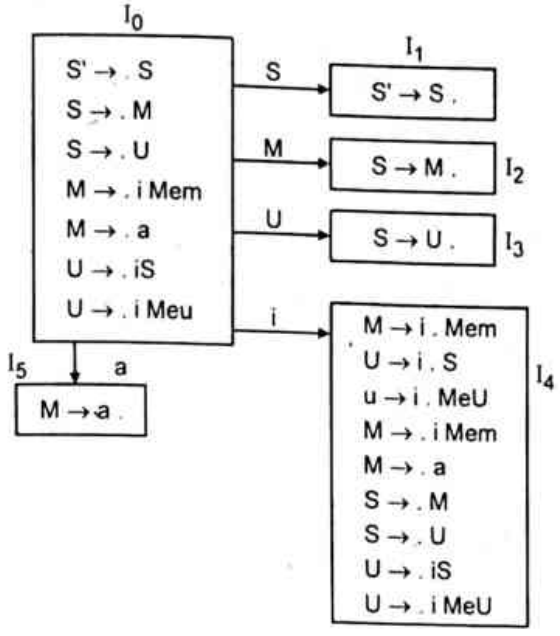
M → a .....(4)

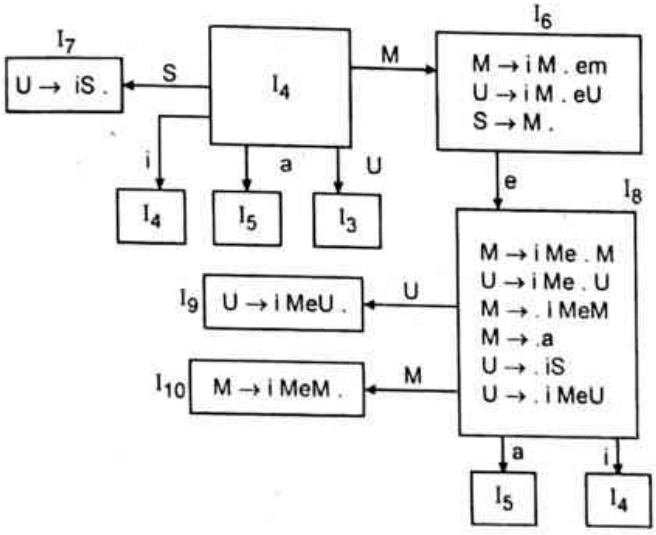
U → i S .....(5)

U → iMeU .....(6)

VBD

The transition diagram of DFA which recognizes all the viable prefixes of given grammar are as follows :





FOLLOW (S) = {$}

FOLLOW (M) = {$, e}

FOLLOW (U) = {$}

Parsing table :

	ACTION				GOTO		
	i	e	a	$	S	M	U
I_0	S_4		S_5		1	2	3
I_1				Accept			
I_2				R_1			
I_3				R_2			
I_4	S_4		S_5		7	6	3
I_5			R_4	R_4			
I_6		S_8		R_1			
I_7				R_5			
I_8	S_4		S_5			10	9
I_9				R_6			
I_{10}			R_3	R_3			

∵ Table does not contains multiple entries. Thus grammar is SLR (1).

Q.62. Get LR (0) parsing table for the grammar

E → E + T / T

T → T * F /F

F → F * / a / b **CT : W-07(10M)**

Ans. The augmented grammar is,

E' → E

E → E + T ....(1)

E → T .....(2)

T → T * F ....(3)

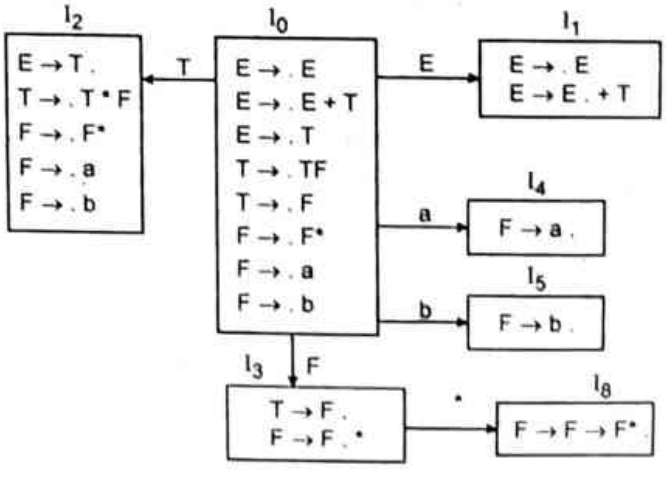
T → F .....(4)

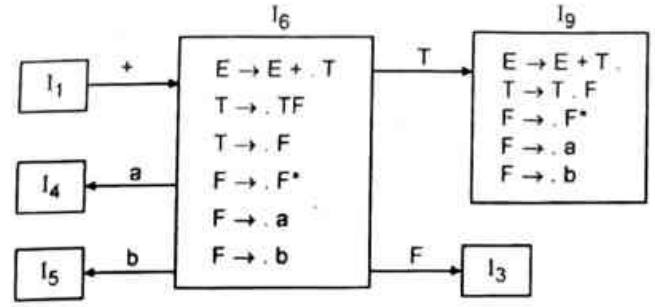
F → F * .....(5)

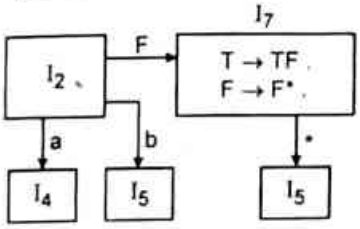
F → a .....(6)

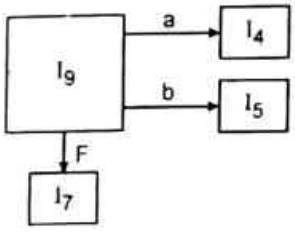
F → b .....(7)

The transition diagram of DFA which recognizes all the viable prefixes of given are as under :









FOLLOW (E) = {$, +}

FOLLOW (T) = {$, +, a, b}

FOLLOW (F) = {$, +, a, b, *}

Parsing table :

	ACTION					GOTO		
	+	*	a	b	$	E	T	F
I_0			S_4	S_5		1	2	3
I_1	S_6				Accept			
I_2	R_2		S_4	S_5	R_2			7
I_3	R_4	S_8	R_4	R_4	R_4			
I_4	R_6	R_6	R_6	R_6	R_6			
I_5	R_7	R_7	R_7	R_7	R_7			
I_6			S_4	S_5			9	3
I_7	R_3	S_3	R_3	R_3	R_3			
I_8	R_5	R_5	R_5	R_5	R_5			
I_9	R_1		S_4	S_5	R_1			7

∵ Table does not contains multiple entries. Thus grammar is LR (0).

Q.63. Get LR (0) parsing table for the grammar,

A → X

X → Start T_0 finish

T → Td ; / ∈

O → O ; E /E

E → X / P

Ans. The augmented grammar is,

A' → A

A → X .....(1)

X → Start T_0 finish .....(2)

T → Td ; ....(3)

T → ∈ ...(4)

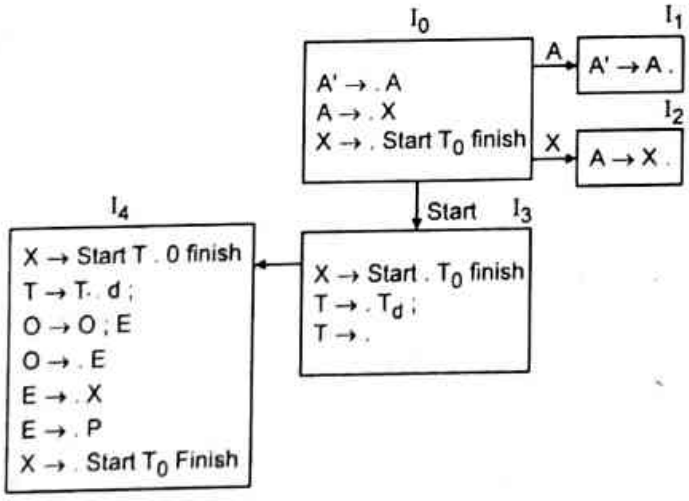
O → O ; E .....(5)

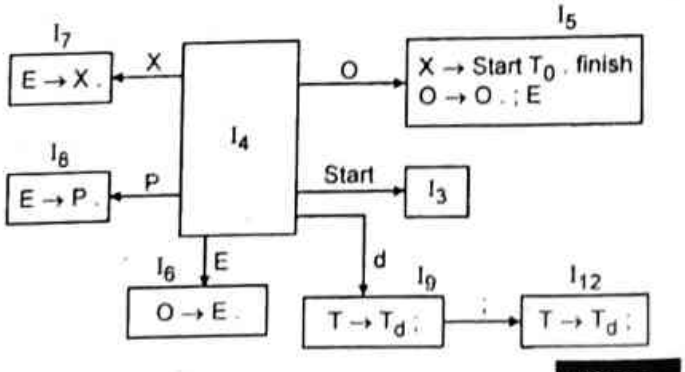
O → E ....(6)

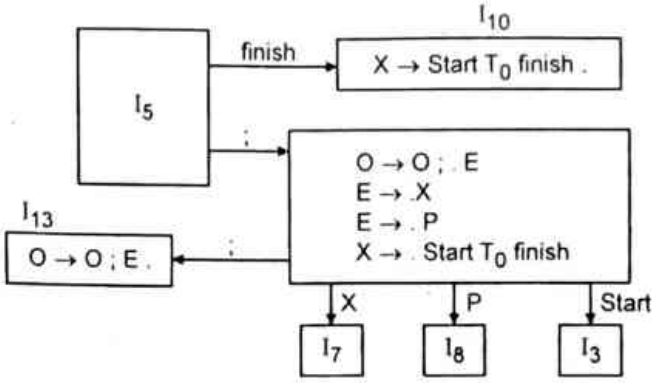
E → X ....(7)

E → P .....(8)

The transition diagram of DFA which recognize all the viable prefixes of given grammar are :







FOLLOW (A) = {$}
FOLLOW (X) = {$, finish, ; }
FOLLOW (T) = { start, d, p}
FOLLOW (O) = {finish, ;}
FOLLOW (E) = {finish, ;}

The parsing table :

	ACTION						GOTO				
	Start	**finish**	**d**	**;**	**p**	**S**	**A**	**X**	**T**	**O**	**E**
I_0	S_3						1	2			
I_1						Accept					
I_2						R_1					
I_3	R_4			R_4		R_4			4		
I_4	S_3		S_9		S_8			7		5	6
I_5		S_{10}			S_{11}						
I_6		R_6			R_6						
I_7		R_7			R_7						
I_8		R_8			R_8						
I_9					S_{12}						
I_{10}		R_2			R_2	R_2					
I_{11}	S_3				S_{13}	S_8		7			
I_{12}	R_3			R_3		R_3					
I_{13}		R_5			R_5						

∵ Table does not contains multiple entries. Thus grammar is LR (0)

Q.64.Construct the non empty sets of item for following LR(0) grammar : **CT : W-06(9M)**

E → E + E / E * E /(E) / id

Ans. The augmented grammar will be,

$E' \rightarrow E$

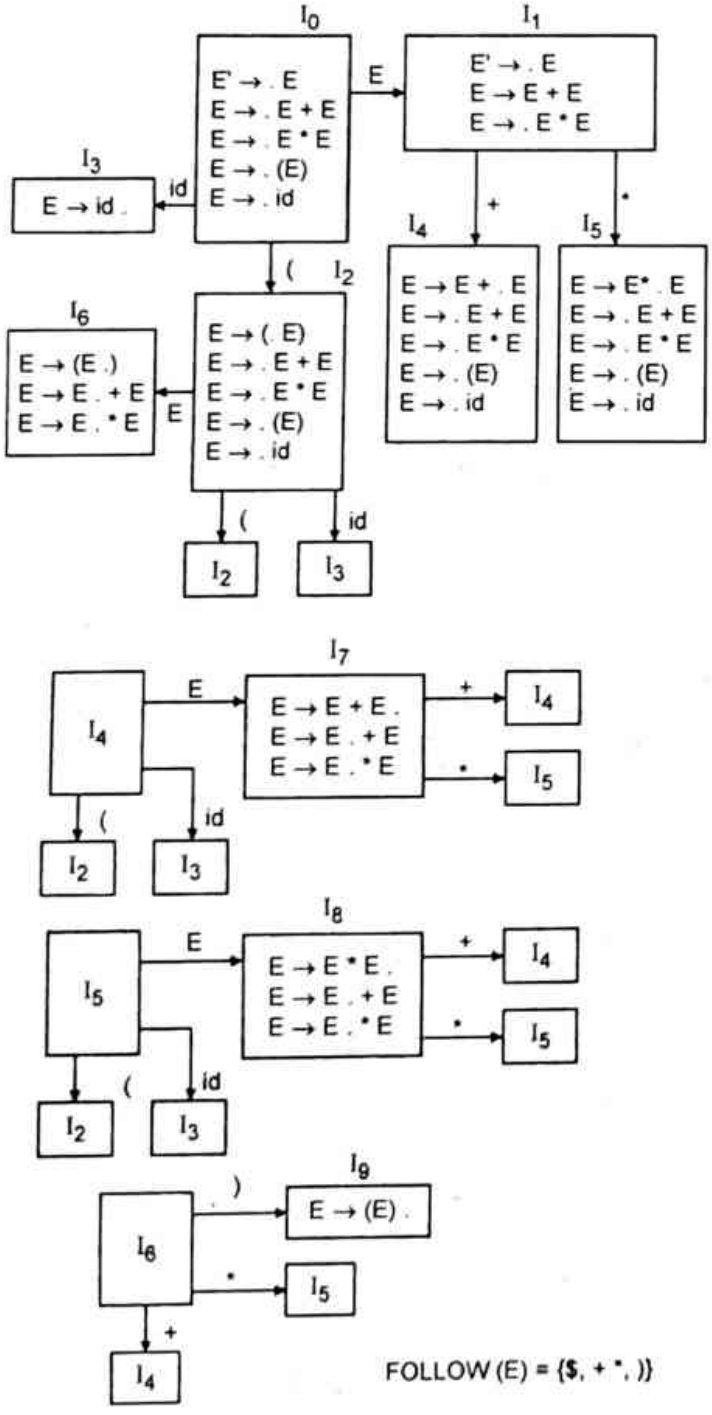
$E \rightarrow E + E$ .....(1)

$E \rightarrow E * E$ .....(2)

$E \rightarrow (E)$ .....(3)

$E \rightarrow ID$ ....(4)

The transition diagram of DFA which recognizes all the viable prefixes of given grammar are as follows :



FOLLOW (E) = {$, + *,)}

The parsing table :

	ACTION						GOTO
	+	*	id	(	)	$	E
I_0			S_3	S_2			1
I_1	S_4	S_5				Accept	
I_2			S_3	S_2			6
I_3	R_4	R_4			R_4	R_4	
I_4			S_3	S_2			7
I_5			S_3	S_2			8
I_6	S_4	S_5			S_9		
I_7	R_1, S_4	R_1, S_5			R_1	R_1	
I_8	R_2, S_4	R_2, S_5			R_2	R_2	
I_9	R_3	R_3			R_3	R_3	

∵ Table contains multiple entries. Thus grammar is not LR (0)

Q.65. Given the grammar,

$S \rightarrow i\ SeS / iS / a$

Design the SLR (1).

Ans. The augmented grammar is,

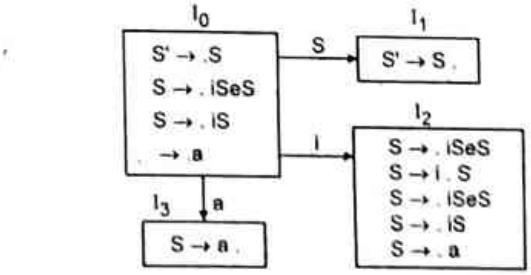
$S' \rightarrow S$

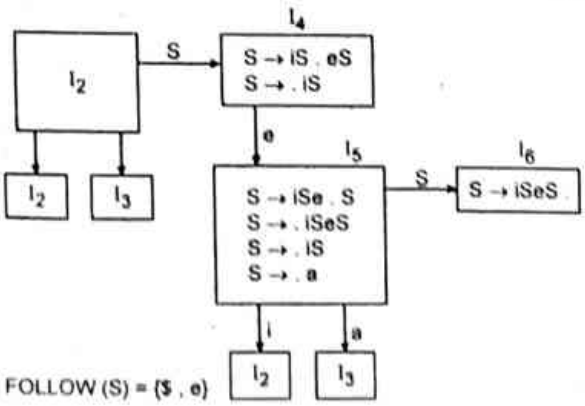
$S \rightarrow i\ SeS$ ....(1)

$S \rightarrow i\ S$ ....(2)

$S \rightarrow a$ .....(3)

The transition diagram of DFA which recognizes all the viable prefixes for given grammar are as follows :





The parsing table :

	ACTION				GOTO
	i	e	a	$	S
I_0	S_2		S_3		1
I_1				Accept	
I_2	S_2		S_3		4
I_3		R_3		R_3	
I_4		R_2, S_5		R_2	
I_5	S_2		S_3		6
I_6		R_1		R_1	

∵ Table contains multiple entries. Thus grammar is not SLR (1)

Q.65.(A) Construct LR(0) items for grammar given. Find its equivalent DFA. Check the parsing by taking a suitable derived string :

$S' \rightarrow S$

$S \rightarrow AS / \in$

$A \rightarrow aA/b$ **CS : S-11(10M)**

Ans. Augmented Grammar

$S' \rightarrow S$

$S \rightarrow AS$ .... (1)

$S \rightarrow .$ .... (2)

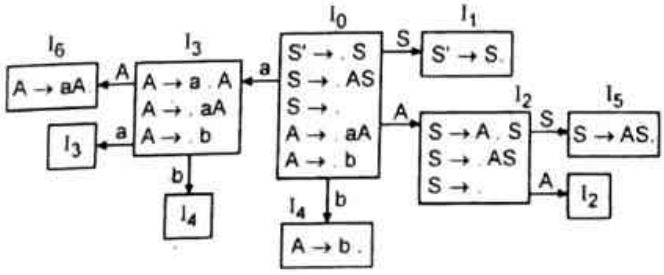
$A \rightarrow aA$ ..... (3)

$A \rightarrow b$ .... (4)

Follow (S) {$}.

Follow (A) = {$}

The transition diagram of DFA which recognizes all prefix of given grammar are as follows :



Parsing Table :

P/S	Action Table			GOTO	
	a	b	$	S	A
I_0	S_3	S_4	R_2	1	2
I_1			accept		
I_2			R_2	5	2
I_3	S_3	S_4			6
I_4			R_4		
I_5			R_1		
I_6			R_3		

Given Grammar is LR(0).

Q.66. Show that :

G : S → aA

A → Abb/b is LR(0) grammer

CT : S-11(4M)

Ans. Augmented Grammar is

S' → S

A → aA .... (1)

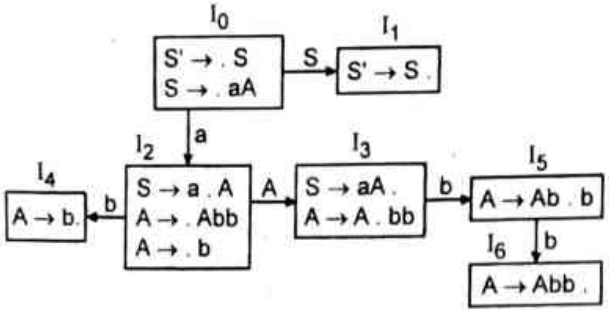
A → Abb .... (2)

A → b ..... (3)

Follow (S) = {$}.

Follow (A) = {b, $}

The transition diagram of DFA which recognizes all prefix of given grammar are as follows :



Parsing Table :

P/S	Action Table			GOTO	
	a	b	$	S	A
I_0	S_2			1	
I_1			accept		
I_2		S_4			3
I_3		S_5	R_1		
I_4		R_3	R_3		
I_5		S_6			
I_6		R_2	R_2		

Hence, given grammar is LR(0).

Q.67. Obtain LR(0) parsing table for

S → aBDh

B → CA

A → bA/∈

D → EF

E → g/∈

F → f/∈

CT : W-12(10M)

Ans. Augmented Grammar

S' → S

S → aBDh ..... (1)

B → cA ..... (2)

A → bA ..... (3)

A → ∈ ..... (4)

D → EF .... (5)

E → g ..... (6)

E → ∈ ..... (7)

F → f ..... (8)

F → ∈ ..... (9)

Follow (S) = {$}

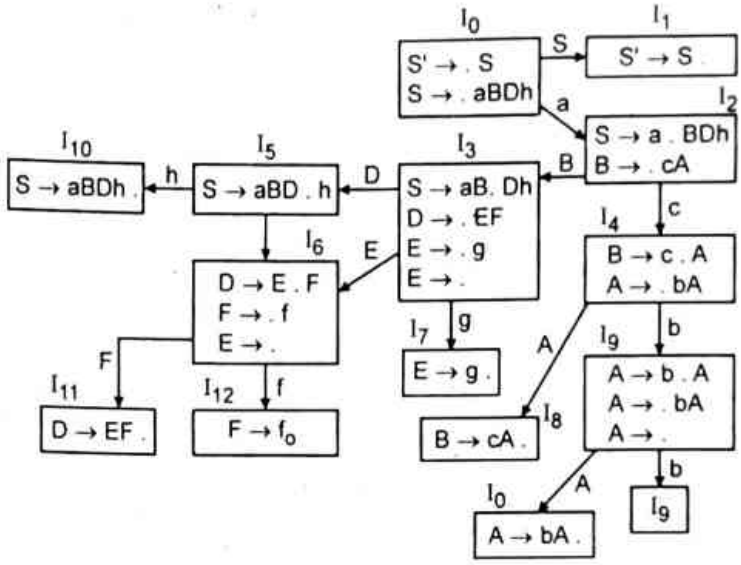
Follow (B) = {g, f, h}

Follow (A) = {g, f, h}

Follow (D) = {h}

Follow (E) = {f, h}

Follow (F) = {h}



Parsing table :

	a	c	h	b	g	f	S	S	B	A	D	E	F
I_0	S_2							1					
I_1							accept						
I_2		S_4							3				
I_3			R_7		S_7	R_7					5	6	
I_4			R_4	S_9	R_4	R_4				8			
I_5			S_{10}										11
I_6			R_9			S_{12}							
I_7			R_6			R_6							
I_8			R_2		R_2	R_2							
I_9			R_4	S_9	R_4	R_4				13			
I_{10}							R_1						
I_{11}			R_5										
I_{12}			R_8										
I_{13}			R_3		R_3	R_3							

Hence, given grammar is LR(0).

POINTS TO REMEMBER :

(1) Turing machine model reflects the computing capability of general purpose computer.

(2) A class of languages recognized by Turing machines is called as recursively enumerable languages or Turing recognizable languages.

(3) There exist a subclass of recursively enumerable languages, called as recursive languages, containing those languages, that have at least one Turing machine which halts on all input.

(4) There exist several variants of Turing machine, all having same power.

(5) For every multitape Turing machine there exists an equivalent single tape Turing machine.

(6) For every non-deterministic Turing machine there exist an equivalent deterministic Turing machine.

(7) Turing machine can be used as a generator, generating the r. e. languages.

(8) If L is a r. e. or Turing recognizable language then, there exist a Turing machine generating the members of L in some order.

(9) If L is a recursive or Turing decidable language then, there exist a Turing machine generating the members of L in canonical order.

(10) A canonical order is the one in which the words are generated in the increasing order of their sizes and for the same size, in the increasing order of their numeric value. For example if the words belonging to Σ^* are to be generated in the canonical order, where $\sum = \{0, 1\}$, then we get the following order :

$\in$, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, ...

(11) If L can be generated in canonical order then L is recursive language or Turing decidable language.

(12) If L = L (G), for some type (0) or unrestricted grammar G = (V, T, P, S), then L is a r. e. language, i.e., there exist some Turing machine M accepting L.

(13) Linear bounded automata (LBA) is a Turing machine in which the tape on which it computes is restricted to the portion containing input w, plus the left and right end makers ¢, and $ respectively.

(14) The class of languages accepted by LBA is called as context-sensitive languages.

UNIT - VI
RECURSIVE LANGUAGES

SYLLABUS

Recursively enumerable (r.e.) set, recursive sets, Decidability and solvability, Post correspondence Problem (PCP), Introduction to recursive function theory, primitive recursive functions, Ackerman function.

UNIVERSITY PAPER SOLUTIONS
SINCE SUMMER - 2006

Note to the students :
- The questions given below are from previous Nagpur University examination papers.
- Though these questions are from old university papers, they have been included in the new syllabus.
- Questions which are out of new syllabus are not included here.

SUMMER - 06 (CT)

Q.1. Show that recursive langauges are closed under complementation. **4M**

Ans. P.6-8, Q.4.

Q.2. What do you mean by "Reduction of problem"? Explain with the help of suitable example. **6M**

Ans. P.6-15, Q.17.

Q.3. What is meant by "Undecidability of problem " and discuss the various applications of post correspondence problem. **6M**

Ans. P.6-14, 18, Q.16(a), 22.

Q.4. Show that the function f of two variables defined by $f(x, y) = y^x$ is primitive recursive. **7M**

Ans. P.6-29, Q.47.

Q.5. Define bounded minimalization. **6M**

Ans. P.6-25, Q.35.

WINTER - 06 (CT)

Q.1. Explain the following with respect to recursive and the recursively enumerable languages :

(i) The complement of a recursive language is recursive.

(ii) The union of two recursive and two recursively enumerable languages is recursive and recursively enumerable respectively.

(iii) If a language L and its complement $\overline{L}$ are both recursively enumerable then L and hence $\overline{L}$ is recursive. **11M**

Ans. P.6-8, 9, Q.4, 5, 6.

Q.2. Given that L is recursively enumerable but not recursive. Comment on the truth/ falsity of each of the following :

(i) L is CFL,

(ii) $\overline{L}$ recursive.

(iii) L is necessarily infinite. **6M**

Ans. P.6-10, Q.8.

Q.3. Explain how Post correspondence problem is useful in proving that particular problems are undecidable. Also explain modified version of PCP in detail. **7M**

Ans. P.6-18, Q.22, 23.

Q.4. What do you mean by primitive recursive function ? What are the basic functions available? What are the various operations which we can carry out on this? **6M**

Ans. P.6-28, Q.42, 43, 44.

Q.5. Write short note on :

(i) Bounded Minimalization

(ii) μ-recursive function

(iii) Unbounded Minimalization

(iv) Mod and DIV functions. **13M**

Ans. P.6-24, 25, 26, Q.37, 34, 35, 36.

SUMMER - 07 (CT)

Q.1. Explain the properties of recursive and recursively enumerable languages. **8M**

Ans. P.6-7, 8, 9, Q.2, 4, 5, 6.

Q.2. What are the applications of post correspondence problem? Write in brief about post correspondence problem. **8M**

Ans. P.6-15, 18, Q.18, 22.

Q.3. Show that f (x, y) = x^y is a primitive recursive function. **5M**

Ans. P.6-29, Q.47.

Q.4. Ackermann's function is defined by ,

A (0, y) = y + 1

A (x + 1, 0) = A (x, 1)

A (x + 1, y + 1) = A (x, A (x + 1, y))

Compute

A (1, 1), A (2, 1), A (1, 2), A (2, 2) **6M**

Ans. P.6-38, Q.68.

WINTER - 07 (CT)

Q.1. What do you mean by primitive recursive function over n and over a, b? **5M**

Ans. P.6-28, 35, Q.42, 61.

Q.2. Does a PCP with two lists x = (b, bab^3, ba) and y = (b^3, ba, a) have a solution? Explain. **8M**

Ans. P.6-16, Q.19.

Q.3. Write short notes on :

(a) Post Correspondence Problem

(b) Properties of Recursive and Recursively enumerable language. **10M**

Ans. P.6-7, 8, 9, 15, Q.2, 4, 5, 6, 18.

Q.4. Define μ - recursive function. **3M**

Ans. P.6-26, Q.37.

SUMMER - 08 (CT)

Q.1. Define Ackerman's function. Compute A (1, 1), A (2, 1) and A (2, 2). **6M**

Ans. P.6-38, Q.68.

Q.2. What is the post correspondence problem? Prove that PCP with two lists X = (01, 1, 1), Y = (01^2 , 10, 1^2) has no solution. **7M**

Ans. P.6-15, 17, Q.18, 21(a).

Q.3. Show that the functions $f_1(x, y) = x + y$ is primitive recursive. **6M**

Ans. P.6-29, Q.45.

Q.4. What is meant by "reduction of problem A to problem B"? Explain clearly with a suitable example. **4M**

Ans. P.6-15, Q.17.

Q.5. Give the recursive definition for n!. **4M**

Ans. P.6-23, Q.33.

Q.6. What is meant by "Undecidability of problem"? Discuss the various applications of post correspondence problem. **6M**

Ans. P.6-14, 18, Q.16(a), 22.

Q.7. Given an arbitrary Turing machine M. Prove that the problem "whether language accepted by M is a regular" is undecidable. **7M**

Ans. P.6-20, Q.27.

WINTER - 08 (CT)

Q.1. Write short notes on :

(i) Mod and Div Functions.

(ii) Post Correspondence Problem. **9M**

Ans. P.6-15, 24, Q.18, 34.

Q.2. Let Σ = {0, 1} and let the A and B lists defined in table. Suggest whether this PCP has a solution or not. Explain your suggestion : **4M**

i	List A	List B
1	1	111
2	10111	10
3	10	0

Ans. P.6-16, Q.19.

Q.3. What is Primitive Recursive function? Explain the category of basic function. **6M**

Ans. P.6-28, Q.42, 44.

Q.4. Explain the properties of recursive and recursively enumerable language. **4M**

Ans. P.6-7, 8, 9, Q.2, 4, 5, 6.

Q.5. Explain about the undecidability of a problem. **3M**

Ans. P.6-14, Q.16(a).

Q.6. Write short notes on :

(i) Unbounded Minimization

(ii) Bounded Minimization. **4M**

Ans. P.6-25, Q.35, 36.

SUMMER - 09 (CS)

Q.1. Explain the various properties of recursive enumerable language. **5M**

Ans. P.6-7, 8, 9, Q.2, 4, 5, 6.

Q.2. Using Ackermann's Principle calculate A (1, 1) A (1, 2) A (2, 1). **5M**

Ans. P.6-38, Q.68.

Q.3. Show that following functions are PRF :

(i) $f(x, y) = x^y$

(ii) $f(x) = x^2$ **7M**

Ans. P.6-29, Q.47.

Q.4. What is PCP ? Give one solution for PCP for following sequence : **6M**

i	A	B
1	0	000
2	010001	01
3	01	1

Ans. P.6-15, 16, Q.18, 19.

Q.5. Write short note on :

(i) Halting problem. **4M**

Ans. P.6-19, Q.25.

SUMMER - 09 (CT)

Q.1. Explain Post Correspondene Problem in detail. **6M**

Ans. P.6-15, Q.18.

Q.2. Write short notes on :

(i) Mod and Div Function. **5M**

Ans. P.6-24, Q.34.

Q.3. What is Primitive Recursive Function ? Explain the category of basic function. **6M**

Ans. P.6-28, Q.42, 44.

Q.4. Explain about the undecidability of a Problem. **3M**

Ans. P.6-14, Q.16(a).

Q.5. Explain the properties of recursive and recursively enumerable language. **4M**

Ans. P.6-7, 8, 9, Q.2, 4, 5, 6.

Q.6. Write short notes on (any THREE) :

(i) Bounded Minimalization

(ii) Unbounded Minimalization

(iii) Halting problem. **13M**

Ans. P.6-19, 25, Q.35, 36, 25.

WINTER - 09 (CS)

Q.1. What is Post correspondence problem ? What is MPCP ? For the given set of list derive solution for PCP : **5M**

List	X	Y
1	b	b^3
2	bab^3	ba
3	ba	a

Ans. P.6-15, 16, 18, Q.18, 19, 23.

Q.2. Prove that following functions are primitive recursive function :

$f(x) = x^2$

$f(x, y) = x * y$ **5M**

Ans. P.6-29, Q.46, 47.

Q.3. Using Ackermann's Principle solve A (2, 2). **3M**

Ans. P.6-38, Q.68.

Q.4. Explain any three properties of Recursive Enumerable Language (REL). **5M**

Ans. P.6-7, 8, 9, Q.2, 4, 5, 6.

Q.5. With example explain Reducing problem. **3M**

Ans. P.6-15, Q.17.

WINTER - 09 (CT)

Q.1. Explain the properties of Recursive and Recursively enumerable language. **6M**

Ans. P.6-7, 8, 9, Q.2, 4, 5, 6.

Q.2. What are the applications of PCP? Does the PCP with $X = (1^3, 01^2)$ and $Y = (1^3, 101^3)$ have a solution ? **7M**

Ans. P.6-16, 18, Q.22, 19.

Q.3. Show that f (x, y) = x + y is primitive recursive function. **6M**

Ans. P.6-29, Q.45.

Q.4. Write short note on :

(i) μ -recursive function

(ii) Bounded minimalization. **7M**

Ans. P.6-25, 26, Q.35, 37.

SUMMER - 10 (CS)

Q.1. Show that Ackermann's function is not primitive recursive. **6M**

Ans. P.6-35, Q.60.

Q.2. Prove that mod (x, y) and Div (x, y) are primitive recursive. **7M**

Ans. P.6-24, Q.34.

Q.3. Does PCP with two lists x = {b, bab^3, ba} and y = (b^3, ba, a) have a solution ? **5M**

Ans. P.6-16, Q.19.

SUMMER - 10 (CT)

Q.1. Explain PCP. Also state which of the following PCP's have a solution ?
(i) {(01, 011), (1, 10) (1, 11)}
(ii) {(b^3, ab^2), (b^3, bab^3)} **7M**

Ans. P.6-15, 16, 17, Q.18, 21(a), 20(b).

Q.2. Explain the properties of recursive and recursively enumerable language. **4M**

Ans. P.6-7, 8, 9, Q. 2, 4, 5, 6.

Q.3. Explain about the undecidability of problem. **3M**

Ans. P.6-14, Q.16(a).

Q.4. Explain Halting problem. **6M**

Ans. P.6-19, Q.25.

WINTER - 10 (CS)

No question asked in University exam.

WINTER - 10 (CT)

Q.1. Write short notes on :
Primitive Recursive functions. **5M**

Ans. P.6-28, Q.42.

Q.2. What are closure properties of recursively enumerable language? **5M**

Ans. P.6-7, 8, 9, Q.2, 4, 5, 6.

Q.3. Define Ackermann's function. Compute A(1, 1), A(2, 1) and A(2, 2) **8M**

Ans. P.6-38, Q.68.

Q.4. What is the post correspondence problem? Prove that PCP with two list x = (01, 1, 1),
y = (10^2, 10, 1^2) has no solution. **5M**

Ans. P.6-15, 17, Q.18, 21(a).

Q.5. Write notes for : **10M**
(i) μ - recursive function.
(ii) Mod and DIV function.
(iii) Bounded and unbounded minimalization.

Ans. P.6-24, 25, 26, Q.37, 34, 35, 36.

SUMMER - 11 (CS)

Q.1. Explain post correspondence problem.
Let Σ = {0, 1} and A and B be list of three strings each as given below. Has post correspondence problem a solution? If so, give the solution. **7M**

	List A	List B
i	Wi	Xi
1	1	11
2	10111	10
3	10	0

Ans. P.6-15, 16, Q.18, 19.

Q.2. What do you mean by term reduction? **3M**

Ans. P.6-15, Q.17.

Q.3. What is Ackermann function? Explain with suitable example. **4M**

Ans. P.6-38, Q.68.

Q.4. What do you mean by primitive recursive function? Show that the addition of two integers X and Y is primitive recursive? **7M**

Ans. P.6-28, 29, Q.42, 45.

Q.5. Show that halting problem is unsolvable. **2M**

Ans. P.6-20, Q.26.

SUMMER - 11 (CT)

Q.1. What is reduction technique? Use this technique to prove the undecidability of halting problem of turing machine. **8M**

Ans. P.6-15, 20, Q.17, 28.

Q.2. What do you mean by recursive language? Show that if L and $\overline{L}$ are both recursively enumerable, then L is recursive. **5M**

Ans. P.6-7, 9, Q.1, 6.

Q.3. Write short notes on : **4M**
(i) Post Correspondence Problem

Ans. P.6-15, Q.18.

Q.4. Define primitive recursive function. Show that the following functions are primitive recursive :
(i) q (x, y) = the quotient obtained when x is divided by y.
(ii) r (x, y) = the remainder obtained when x is divided by y. **7M**

Ans. P.6-24, 28, Q.34, 42.

Q.5 What do you mean by partial recursive function? **2M**

Ans. P.6-23, Q.32.

WINTER - 11 (CS)

Q.1. What is post correspondence problem ? Does the following instance of PCP have the solution? If yes, then find the solution. If not then state the reason. **6M**

i	List A	List B
1	1	111
2	10111	10
3	10	0

Ans. P.6-15, 16, Q.18, 19.

Q.2. Show that union of two recursive language is recursive. **3M**

Ans. P.6-8, Q.5.

Q.3. Define primitive recursive function. Show that the following functions are primitive recursive.

(i) $f(x, y) = x^y$

(ii) $\max(x, y) = x$ if $x \geq y$

$= y$ otherwise. **7M**

Ans. P.6-28, 29. 38, Q.42, 47, 67.

Q.4. Show that complement of recursive language is always recursive. **3M**

Ans. P.6-8, Q.4.

Q.5. Show that halting problem of Turing machine is undecidable. **3M**

Ans. P.6-20, Q.27.

WINTER - 11 (CT)

Q.1. Explain the properties of recursive and recursively enumerable languages. **6M**

Ans. P.6-7, 8, 9, Q.2, 4, 5, 6.

Q.2. Explain post correspondence problem. Does following (PCP)

$S = \{(0, 10), \{1^20, 0^3), (0^21, 10)\}$ have solution? Justify. **7M**

Ans. P.6-15, 16, Q.18, 19.

Q.3. What do you mean by primitive recursive function? Give its three basic functions. **7M**

Ans. P.6-28, Q.42, 44.

Q.4. Show that following functions are PRF:

(i) $f(x, y) = x * y$

(ii) $f(x, y) = x^y$. **6M**

Ans. P.6-29, Q.46, 47.

Q.5. Write short notes on : **7M**

(i) Bounded and unbounded minimalization.

(ii) μ recursive function.

Ans. P.6-25, 26, Q.35, 36, 37.

Q.6. Explain halting problem of Turing Machine. Also show that halting problem of TM is unsolvable. **6M**

Ans. P.6-19, 20, Q.25, 26.

SUMMER - 12 (CS)

Q.1. Show that the function

$G(x, y) = x^y$ is primitive recursive. **3M**

Ans. P.6-29, Q.47.

Q.2. Prove the following for Ackermann function :

$A(2, y) = 2y + 3$ **4M**

Ans. P.6-39, Q.69.

Q.3. Let G1 be any unrestricted grammar and G2 any regular grammar. Show that the problem

$L(G1) \cap L(G2) = \phi$ is undecidable. **6M**

Ans. P.6-14, Q.16(b).

SUMMER - 12 (CT)

Q.1. What do you mean by recursion? Explain the properties of recursive and recursively enumerable languages. **5M**

Ans. P.6-7, 8, Q.2, 4, 5, 6.

Q.2. Define Ackermann's Function and compute $A(1, 1)$, $A(2, 1)$ and $A(2, 2)$. **6M**

Ans. P.6-38, Q.68.

Q.3. What is the Post Correspondence Problem? Prove that PCP with two list : **5M**

$X = (01, 1, 1)$

$Y = (01^2, 10, 1^2)$ has no solution.

Ans. P.6-15, 17, Q.18, 21(a).

Q.4. Write short notes on : **7M**

(i) μ -recursive function.

(ii) Halting problem.

Ans. P.6-19, 26, Q.25, 37.

WINTER - 12 (CS)

Q.1. Explain post correspondence problem. Consider the post correspondence system described by the following lists.

α = { 10, 01, 0, 100, 1}

β = {101, 100, 10, 0, 010}

Does this PCP have a solution ? **4M**

Ans. P.6-15, 17, Q.18, Q.21(b).

Q.2. What do you mean by Primitive Recursive function ? Show that the max function defined as

$$\text{Max}(x, y) = \begin{cases} 0 & \text{if } x > y \\ 0 & \text{if } x \le y \end{cases}$$ is primitive recursive. **4M**

Ans. P.6-28, 38, Q.42, 67.

Q.3. Prove that if a language L and its complement L' are both recursively enumerable then both are recursive. **3M**

Ans. P.6-9, Q.6.

Q.4. Explain the halting problem of Turing machine. Prove that the halting problem of TM is undecidable. **6M**

Ans. P.6-19, 20, Q.25, 27.

WINTER - 12 (CT)

Q.1. State and explain Ackermann's Function. **5M**

Ans. P.6-38, Q.68.

Q.2. Write short notes on recursive enumerable language and recursive language. **4M**

Ans. P.6-7, 8, 9, Q.2, 4, 5, 6.

Q.3. What is meant by Undecidability? **4M**

Ans. P.6-14, Q.16(a).

Q.4. Define primitive recursive function. Show that the function f (x, y) = x * y is primitive recursive. **6M**

Ans. P.6-28, 29, Q.42, 46.

Q.5. "Every function in primitive recursive class is Turing computable". Comment. **3M**

Ans. P.6-31, Q.52.

SUMMER - 13 (CS)

Q.1. Show that the following question is decidable : "The set of codes for turing machine that halt within 10 moves on every input". **5M**

Ans. P.6-13, Q.15(b).

Q.2. Show that following function are primitive recursive :

(i) equal (m, n) = 1 if m = n

= 0 otherwise,

(ii) abs (x, y) = |x − y| **3M**

Ans. P.6-38, Q.66.

SUMMER - 13 (CT)

Q.1. Prove that following function are primitive recursive

$f(x) = x^3$, $f(x, y) = x^4$ **6M**

Ans. P.6-30, Q.48(b).

Q.2. What is Ackermann's function? Calculate A (1 , 1), A (1, 2), A (2, 1). **7M**

Ans. P.6-38, Q.68.

Q.3. Does the PCP with $x = (b^3, ab^2)$ and $y = (b^3, bab^3)$ have solution? **5M**

Ans. P.6-16, Q.20(b).

Q.4. Write short note on :

Halting problem in TM. **4M**

Ans. P.6-19, Q.25.

SOLVED QUESTION BANK

[Sequence given as per syllabus]

INTRODUCTION

A problem whose language is recursive is said to be decidable. Otherwise, a problem is undecidable. That is, a problem is undecidable if there is no algorithm that takes as input an instance of the problem and determines whether the answer to that instance is "yes" or "no".

RECURSIVELY ENUMERABLE (R.E.) SET, RECURSIVE SETS

Q.1. Explain Recursively Enumerable and Recursive sets languages.

OR What do you mean by recursive language?

CT : S-11(1M)

Ans. Recursively Enumerable and Recursive Languages :

- A language L is said to be recursively enumerable, if there exists a turing machine that accepts L.
- This definition implies only that there exists a Turing machine M, such that, for every $u \in L$.

  $q_0\, u \mapsto M x_1\, q_f\, x_2$

  with q_f as final state.
- The definition does not say anything about what happens for u not in L; it may be that the machine halts in a nonfinal state or that it never halts and goes into an infinite loop.
- A language L on L is said to be recursive, if there exist a turing machine M that accepts L and that halts on every $u \in \Sigma^+$.
- In other words, we can say that a language is recursive iff there exists a membership, algorithm for it or if there exists a TM that recognizes L.
- If a language is recursive, then there exists an easily constructed enumeration procedure.
- Suppose that M is a turing machine that determines membership in a recursive language L.
- First, we construct another turing machine, say M', that generates all strings in Σ^+, in proper order, say $u_1, u_2, \ldots$ As these strings are generated, they become the input to M, which is modified so that it writes strings in its tape only if they are in L.
- There is also an enumeration procedure for every recursively enumerable language, but it is not easy to see.
- It is easy to see that every language for which an enumeration procedure exists is recursively enumerable.
- We simply compare the given input string against successive strings generated by the enumeration procedure.
- If $u \in L$, then we will eventually find a match, and the process can be terminated.

Q.2. What do you mean by recursion? Give the properties of recursive and recursively enumerable languages.

CS : W-09(1M) S-09(1M), CT : S-07,12, W-07,09,11(2M), S-09,10, W-08,10,12(1M)

Ans. Recursion : Recursion is the process of defining something in term of itself.

Properties of Recursive and Recursively enumerable language :

(1) The complement of recursive language is recursive.

(2) The union of two recursive language is recursive. The union of two recursively enumerable language is recursively enumerable.

(3) If a language L and its complement $\overline{L}$ are both recursively enumerable. Then L (and hence $\overline{L}$) is recursive.

(4) If L is a recursive language then $\Sigma^* - L$ is recursive.

Q.3. Explain the trivial and Non-trivial properties of r.e. languages :

Ans. Trivial and Non-trivial properties of r.e. languages :

- A property of r.e. languages is trivial if either no

r.e. languages has that property, or all r.e. languages have that property.

- A property of r.e. languages is said to be non-trivial if some r.e. languages have that property, but not all r.e. languages have that property.
- Emptiness, regularity, context freedom are some of the non-trivial properties of the r.e. languages.
- A non-trivial property of r.e. languages is undecidable.
- Because if a non-trivial property of r.e. languages would have been decidable, then it would have been possible to construct an algorithm for Lu as shown in figure.

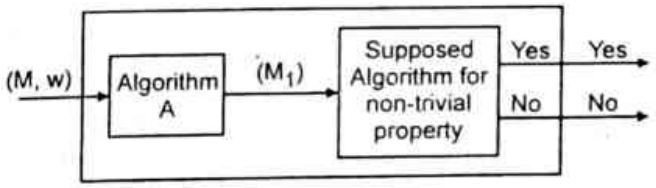


Fig. Algorithm for Lu.

- Algorithm A takes α a pair of lists (M, w), as input and generates a Turing machine M_1, which accepts a language L, having non-trivial property under consideration if and only if M accepts w and accepts nothing if M does not accepts w.
- M_1 generated by algorithm A works as follows : Since, the property under consideration is non-trivial, there exist some r.e. language satisfying that property.
- Let L be such a language and L being r.e. language, there exist some Turing machine say M_1, accepting L.
- M_1 first simulates M on w and if M accepts w, it simulates M_L. When M does not accept w, then M_1 accepts nothing.
- Therefore L (M_1) = L (M_1) = L, otherwise L (M_1) = ϕ.

Q.4. Justify : The complement of a recursive language is recursive. ***CS:W-09(1M),W-11(3M),S-09(1M) CT : W-06(4M), S-07(2M), S-09,10,12, W-07,08,09,10,11,12(1M)***

OR Show that recursive languages are closed under complementation. ***CT : S-06(4M)***

Ans. Justification :

- Let L be a recursive language and M a Turing machine that halts on all inputs and accepts L.

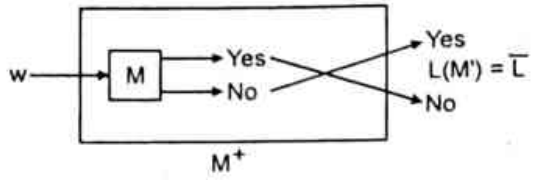


Fig. Construction showing that recursive languages are closed under complementation M

- Construct M' from M so that if M enters a final state on input w, then M' halts without accepting.
- If M halts without accepting, M' enters a final state.
- Since one of these two events occurs. M' is an algorithm.
- Clearly L (M') is the complement of L and thus the complement of L is a recursive language.

Q.5. The union of two recursive languages is recursive. The union of two recursively enumerable language is recursively enumerabley. ***CS : S-09, W-09(1M), W-11(3M), CT : W-06(4M), S-07(2M), W-07,08,09,10,11,12,S-09,10(1M)***

Ans. Case 1 : Recursive

- If L_1 and L_2 are recursive language then there exists Turing machine M_1 and M_2 accepting L_1 and L_2 resepctively and their halting is guaranteed as shown in fig. (a).

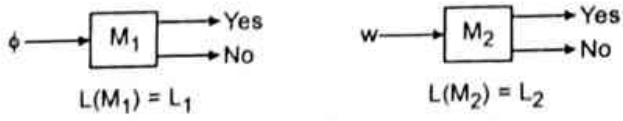


Fig. (a)

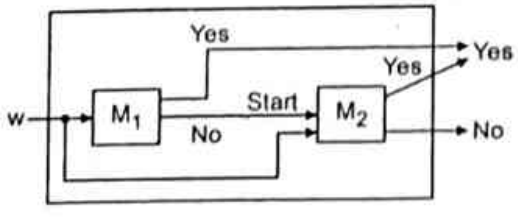


Fig.(b) L (M) = L (M_1) ∪ L (M_2) L = $L_1 \cup L_2$

- Construct a Turing machine M which takes a w string as input and simulates M_1 on w.
- If M_1 accepts w then M accepts w and halt.
- But if M_1 rejects w then M carries out simulation M_2 on w.
- If M_2 accepts w then M also accepts w and if M_2 rejects w M also rejects w and halt as shown in fig. (b).
- Therefore it is clear that Turing machine M accepts $L(M) = L(M_1) \cup L(M_2)$ i.e. $L_1 \cup L_2$.
- Hence we conclue that if L_1 and L_2 are recursive then L_1 and L_2 is always recursive.

Case 2 : Recursively Enumerable

- If L_1 and L_2 are recursively enumerable language then there exists Turing machine M_1 and M_2 accepting L_1 and L_2 respectively.

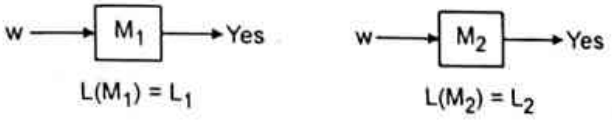
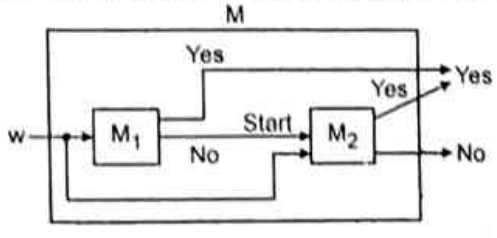


Fig. (c)

- Construct a Turing machine M which takes w as input.
- M_1 and M_2 both will simulate on w. If any of them or both (M_1 and M_2) accepts w, then M accepts w and halt.
- But if w is not accepted by M_1 and M_2 both, then there is no guarantee whether M_1 and M_2 will halt or not.
- Certainly L (M) is the union of L (M_1) and L (M_2). M accepts union of L_1 and L_2.
- Therefore $L_1 \cup L_2$ is recursively enumerable because halting of M can not be guaranteed.



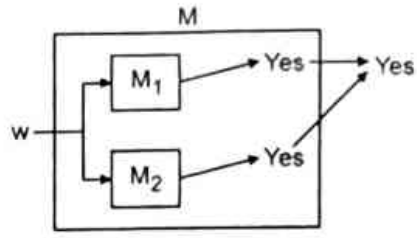


Fig. (d) Construction for union

Q.6. Justify : If a language L and its complement $\overline{L}$ are both recursively enumerable, then L (and hence $\overline{L}$) is recursive.

CS : S-09, W-09(1M), W-12(3M), CT : W-06(3M), S-09,10(2M), W-07,08,09,10,11,12(1M), S-07,11(4M)

Ans. Justification :

- If L and $\overline{L}$ both are recursively enumerable, then there exists Turing machines M_1 and M_2 accepting L and $\overline{L}$ respectivley as shown in fig. (a).

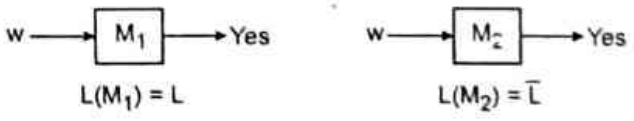


Fig. (a)

- We can construct a Turing machine M which takes w as input.
- M simulates M_1 and M_2 on w and halt as shown in fig. (b).
- If M_1 accepts w then M accepts w. If M_2 rejects w then M accepts w.
- Thus M will always say either "yes" or "no" but will never say both.
- Since L and $\overline{L}$ are complementary languages w will be either in L or in $\overline{L}$ and hence it will be either accepted to L or $\overline{L}$. Thus halting of M is guaranteed and it is clear that M accepts L.
- We conclude that it is always possible to construct Turing machine accepting L, it follows that L is recursive.
- Since L is recursive, $\overline{L}$ is also recursive.
- Let L and $\overline{L}$ be a pair of complementary languages. Then either :

(a) Both L and $\overline{L}$ are recursive,

(b) Neither L nor $\overline{L}$ is recursively enumerable, or

(c) One of L and $\overline{L}$ is recursively enumerable, but not recursive, the other is not recursively enumerable.

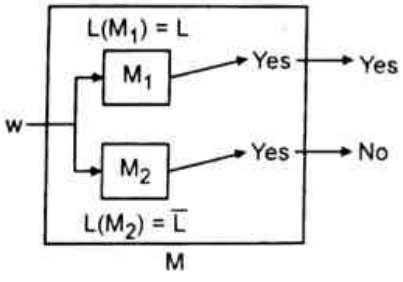


Fig. (b)

Q.7. Explain the relationship between recursive, recursively enumerable and non recursively enumerable languages.

Ans.

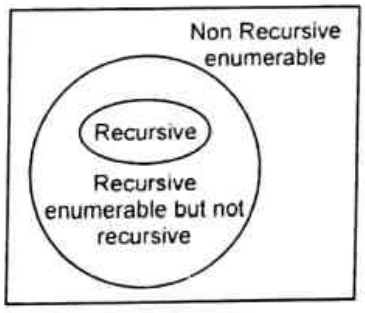


Fig. Relationship between recursive, R E and Non-RE language.

Every language that is recursive is recursively enumerable :

- If M is Turing machine recognizing L, then a modified machine that crashes instead of leaving output 0 accepts L.
- If M is Turing machine accepting L, there may be strings not in L for which M crashes and therefore never produces an answer.
- If language L and its complement $\overline{L}$ are both recursively enumerable, then L and $\overline{L}$ are recursive.
- Also every recursive enumerable languages are not recursive.

Q.8. A language L is recursively enumerable but not recursive. Comment on each of the following statement providing a brief justification for your answer.

(i) L is CFL.

(ii) $\overline{L}$ is recursive but not recursively enumerable.

(iii) The membership problem of L is undecidable.

(iv) $\overline{L}$ is finite. **CT : W-06(6M)**

Ans.

(i) **L is CFL :** A language that is accepted by Turing machine is said to be recursively enumerable. Recursive sets are those language accepted by at least one Turing machine that halts on all inputs (note that halting may or may not be preceded by acceptance).

The recursive sets are a proper subclass of recursive enumerable sets. By CYK algorithm every CFL is a recursive sets. Hence L is a CFL.

(ii) **$\overline{L}$ is recursive but not recursively enumerable :** L is recursively enumerable but not recursive. By properties of recursively enumerable language, $\overline{L}$ is recursive. Since L and $\overline{L}$ is a pair of complementary language and L is recursively enumerable but not recursive. Hence $\overline{L}$ is not recursively enumerable.

(iii) **The membership problem of L is undecidable :** If we cannot solve the problem of whether a given TM M accept the particular string w, we obviously can not expect to solve the more general membership problem of recursively enumerable language. This is most well known undecidable decision problem. Because of the way TM accepts a string, the problem is known as the halting problem. Since halting problem of turing machine is undecidable. Hence the membership problem L is undecidable.

(iv) **$\overline{L}$ is finite :** Let y be a set of recursive enumerable language, each a subset of $(0+1)^*$. According to Rice's theorem for recursively enumerable index sets.

L_y is recursively enumerable if and only if

(1) L is in y and $L \subseteq L'$ for some recursively enumerable L', then L' is in y.

(2) L is a infinite set in y, then there is some finite subset L' of L that is in y.

(3) The set of finite languages in y is enumerable.

Since L is a recursively enumerable language and L is an infinite set in y. Hence there is some finite subset L' of L that is in y. So $\overline{L}$ is finite.

Q.9. Comment on truth / falsity of the following statement.

Not every context free language is recursive.

Ans. True :

- We know that the problem whose language is recursive is said to be decidable. Otherwise the problem is undecidable.
- There exists some context free grammar for which it is undecidable whether the intersection of two CFG's is empty.
- Also there exists some CFG whose grammar's are ambiguous and inherent ambiguity of CFL's is undecidable.
- It means that there exists some CFG language which is not recursive. Hence not every context free language is recursive.

Q.10. Show that L_d is non recursively enumerable.

Ans. Theorem : L_d is non recursively enumerable.

Proof :

- Suppose we have a list of $(0+1)^*$ in canonical order, where w_i is the i^{th} word and M_j is the TM whose code is integer j written in binary.
- Imagine an infinite table for all i and j where w_i is in $L(M_j)$. Following fig. suggests such a table; 0 means w_i is not in $L(M_j)$ and 1 means it is.

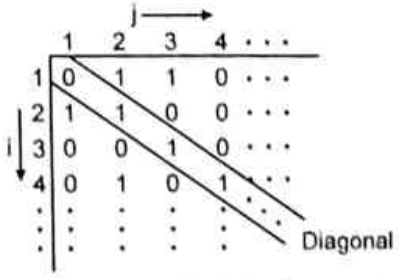


Fig. Hypothetical table indicating acceptance of words by TM's

- We construct a language L_d by using the diagonal entries of the table to determine membership in L_d
- To guarantee that no TM accepts L_d, we insist that w_i is in L_d if and only if the (i, i) entry is 0, that is, if M_i does not accept w_i.
- Suppose that some TM M_j accepted L_d. Then we are faced with the following contradiction.
- If w_j is in L_d, then the (j, j) entry is 0, implying that w_j is not in $L(M_j)$ and contradicting $L_d = L(M_j)$.
- On the other hand, if w_j is not in L_d, then the (j, j) entry is 1, implying that w_j is in $L(M_j)$, which again contradicts $L_d = L(M_j)$.
- As w_j is either in or not in L_d, we conclude that our assumption $L_d = L(M_j)$, is false. Thus, no TM in the list accepts L_d, and no TM whatsoever accepts L_d.

  We have thus proved L_d is not recursively enummerable.

Q.11. Define universal language.

Ans. Universal language : The universal language L_u is called as "universal" since the question of whether any particular string w in $(0 + 1)^*$ is accepted by any particular Turing machine M is equivalent to the question of whether (M', W) is in L_u where M' is the TM with tape alphabet {0, 1, B} equivalent to M.

Q.12. Prove that L_u is recursively enumerable.

Ans. Theorem : L_u is recursively enumerable.

Proof :

- We shall exhibit a three - tape TM M_1 accepting L_u.
- The first tape of M_1 is the input tape and the input head on that tape is used to look up moves of the TM M when given code <M, w> as input.

- Note that the moves of the M are found between the first two blocks of three 1's.
- The second tape of M_1 will simulate the tape of M.
- The alphabet of M is {0, 1, B } so each symbol of M's tape can be held in one tape cell of M_1's second tape.
- Observe that, if we did not restrict the alphabet of M, we would have to use many cells of M_1's tape to simulate one of M's cells but the simulation could be carried out with a little more work.
- The third tape holds the state of M, with q_i represented by 0^i.
- The behavior of M_1 is as follows :

(1) Check the format of tape 1 to see that it has a prefix of the form 111 $code_1$ 11 $code_2$ 11.....11 $code_r$ 111, and that there are no two codes that begin with 0^i10^j1 for the same i and j. Also check that if $0^i10^j10^k\ 10^l10^m$ is a code, then $1 \le j \le 3$, $1 \le l \le 3$ and $1 \le m \le 2$. Tape 3 can be used as a scratch tape to facilitate the comparison of codes.

(2) Initialize tape 2 to contain w, the portion of the input beyond the second block of three 1's. Initialize tape 3 to hold a single 0, representing q_1. All three tape heads are positioned on the leftmost symbols. These symbols may be marked, so the heads can find their way back.

(3) If tape 3 holds 00, the code of the final state, halt and accept.

(4) Let X_j be the symbol currently scanned by tape head 2 and let 0^i be the current contents of tape 3. Scan tape 1 from the left end to the second 111, looking for a substring beginning 110^i10^j1. If no such string is found, halt and reject ; M has no next move and has not accepted. If such a code is found, let it be $0^i10^j10^k\ 10^l10^m$. Then put 0^k on tape 3, print X_l on the tape cell scanned by head 2 and move that head in direction D_m. Note that we have checked in (1) that $1 \le l \le 3$ and $1 \le m \le 2$. Go to step (3).

- It is straight forward to check that M_1 accepts <M, w> if and only if M accepts w.
- It is also true that if M runs forever on w, M_1 will run forever on <M, w>, and if M halts on w without accepting, M_1 does the same on <M, w>.
- The existence of M_1 is sufficient to prove that L_u is recursively enumerable. However, by theorems.

(i) If a language L is accepted by a multitape Turing machine, it is accepted by a single-tape Turing Machine.

(ii) If $L \le ( ) + 1)^*$ and L is r.e., then L is accepted by a one-tape TM with tape alphabet {0, 1, B}, we can find a TM with semiinfinite tape and alphabet {0, 1, B} accepting L_u. This particular TM M_u, is called as universal Turing machine, since it does the work of any TM with input alphabet {0, 1}.

Q.13. Show that L_u is not recursive.

Ans. Theorem : L_u is not recursive.

Proof :

- Suppose A were an algorithm recognizing L_u. Then we could recognize as follows.
- Given string w in $(0 + 1)^*$, determine by an easy calculation the value of i such that $w = w_i$.
- Integer i in binary is the code for some TM M_i.
- Feed (M_i, w_i) to algorithm A and accept w, if and only if M_i accepts w_i.
- The construction is shown in fig.

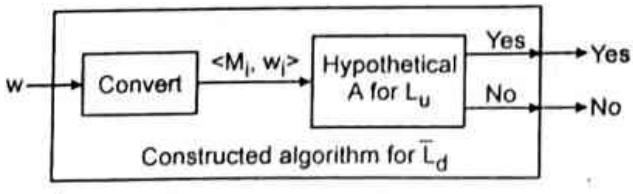


Fig. Reduction of $\overline{L}_d$ to L_u

- It is easy to check that the constructed algorithm

accepts w, if and only if $w = w_i$ and w_i is in L (M_i). Thus we have an algorithm for $\overline{L}_d$.

- Since no such algorithm exists, we know our assumption, that algorithm A for L_u exists, is false.
- Hence L_u is recursively enumerable but not recursive.

Q.14. Prove the following : A language L is recursively enumerable if there is a TM that accepts L.

Ans. Theorem : A language $L \subseteq \Sigma^*$ is recursively enumerable (i.e can be accepted by some TM) if and only if L can be enumerated by some TM.

Proof :

- Suppose T is a TM enumerating L.
- A machine T_1 that accepts L is constructed with one more tape than T, tape 1 being the extra tape that serves as the input tape.
- The TM T_1 simulates T, except that every time # is written on tape 2, the simulation of T pauses while T_1 compares its input string with the string listed just before the #.
- If the two strings match, T_1 halts.
- It is clear that T_1 accepts precisely the strings that are generated on tape 2, which by assumption are the elements of L.
- More precisely, T_1 can incorporate the transitions of T intact, treating the tapes other than tape 1 as the tapes of T and ignoring tape 1, with this change : Every transition that involves writing # on tape 2 is replaced by a sequence of transitions in which the tapes other than 1 and 2 are ignored, the input string is compared with the string just before the # on tape 2, and, if they do not match, T_1 is returned to the configuration that would have resulted from the original transition.
- In the converse direction, suppose T accepts L.
- We construct a three tape TM T_1 enumerating L.
- Tape 1 is the output tape, tape 2 is used by T_1 for generating strings in Σ^*, and tape 3 is the tape T_1 used for simulating the action of T on each string generated.
- In order to avoid the difficulty discussed previously T_1, simulates longer and longer finite sequences of moves of T, rather than trying to carry the processing of T on a single string to completion.
- For this reason, tape 2 saves not only the strings that have been generated so far, but also the number of moves of T that have been carried out on each one.

DECIDABILITY AND SOLVABILITY

Q.15.(a) Write detailed note on decidability of problems.

Ans. Decidable languages :

- A problem with only two answers Yes/No can be considered as a language L.
- An instance of the problem with the answer 'Yes' can be considered as an element of the corresponding language L_i an instance with answer 'No' is considered as an element not is L.
- A problem with two answers (Yes/No) is decidable if the corresponding language is recursive. In this case, the language 'L' is called decidable.

Examples :

(1) A DFA is decidable.

(2) Context free grammer G that accepts the input string w is decidable.

(3) A context sensitive grammar G that accepts the input string w is decidable.

Q.15.(b) Show that the following question is decidable : "The set of codes for turing machine that halt within 10 moves on every input". **CS : S-13(5M)**

Ans.

- Let's consider the more general problem of machines which stop after at most N steps, for some N – 1. (The following is a substantial simplification of a previous version of this answer, but is effectively equivalent.)

- If the machine stops after at most N steps, then only the cells 0, 1, ..., N − 1 on the tape are significant. Then it suffices to simulate the machine M on all input strings of the form $x \in \sum N$, of which there are a finite number.
- If any of these simulation fail to enter a halting state by the N^{th} transition, this indicates that any input string starting with x is one for which the machine does not stop within the first N steps.
- If all of these simulations halt by the N^{th} transition, then M halts within N steps on all inputs of any length (of which the substring of length N is all that it ever acts on).
- If M stops in not more than 10 steps, then the position M can reach on the normally infinite tape are limited. Thus the infinite tape can be simulated by a finite one. This means that the tape can be simulated by a finite automaton. It follows that a turing machine M that stops in not more than 10 steps is similar to some finite automaton M′.
- Let Q be the set of states of M, $F \subset Q$ the set of accepting states and Γ be the alphabet. Then we build the set of states Q′ of M′ as follows :
  $Q = \{\langle n, q, s, p, a\rangle \mid n \in \{0, ..., 10\}\, q \in Q, s \in \Gamma,$
  $p \in \{-10, ..., 10\}, a \equiv q \in F\}$ where p is the position of the read/write head above the tape. We can restrict the position to {− 10, ... , 10} because the number of allowed computing steps limits the number of reachable positions.
- Having a state $\langle n, q, s, p, a\rangle$ of the finite automaton M′ then means that we are at state q of the original automaton, with s on the tape at position p where also the read/write head is positioned, after the n^{th} computing step. The state is an accepting one if a ≡ true.
- Transforming the transition relation of a concrete turing machine is a little more work but not necessary for the original question, because it is enough to show that the state space is finite (and thus we can just test each input with a length of at most 10 symbols on each such automaton). The idea is to build a new transition relation that goes from a state $\langle n, q, s, p, a\rangle$ to a state $\langle n + 1, q', s', p', a'\rangle$ in the n^{th} computing step iff the transition $\langle q, s, p\rangle \rightarrow \langle q', s', p'\rangle$ was in the original transition relation,

Q.16.(a) Write a short note on undecidability of a problem.

CT : S-06,08,09,10, W-08(3M), W-12(4M)

Ans. Undecidability of a problem :

- The problem is said to be undecidable if there is no algorithm that takes as input an instance of the problem and determines whether the answer to that instance is "yes" or "no".
- The theory of undecidability is concerned with the existence or non existence of algorithms for solving problems with an infinity of instances.

Example :

- If we cannot solve the problem of whether a given TM M accepts the particular strings w, we obviously cannot expect to solve the more general membership problem for recursively enumerable languages. This is most-known undecidable decision problem.
- Undecidability of a problem means that there is no single algorithm guranteed to produce the correct answer for every instance.
- An unintuitive consequence of the definition of "undecidable" is that problems with only a single instance are trivially decidable.
- The problems whose language is not recursive are said to be undecidable problems.

Q.16.(b) Let G1 be any unrestricted grammar and G2 any regular grammar. Show that the problem L (G1) ∩ L (G2) = ϕ is undecidable.

CS : S-12(6M)

Ans. Proof :

- Construct the context-free grammars G_1, G_2 that correspond to an (I, A)-instance (g, h) of the PCP and these grammars are unambiguous.
- Let G_0 be the context-free grammar $G_0 = ([S_0, S', S],\ I \cup A, S_0, P_0)$, where P_0

contains the productions of both G_1 and G_2 and also, the productions $S_0 \to S$ and $S_0 \to S'$. Thus, $L(G_0) = L(G_1) \cup L(G_2)$.

- We claim that G_0 is ambiguous if and only if $L(G_1) \cap L(G_2) \neq \phi$. Indeed, if $t \in L(G_1) \cap L(G_2)$, then we have the leftmost derivations $S \underset{G_1, \text{left}}{\overset{+}{\Rightarrow}} t$ and $S' \underset{G_2, \text{left}}{\overset{+}{\Rightarrow}} t$ which entail the existence of the leftmost derivations.

$$S_0 \underset{G_0, \text{left}}{\Rightarrow} S \underset{G_0, \text{left}}{\overset{+}{\Rightarrow}} t$$

and

$$S_0 \underset{G_0, \text{left}}{\Rightarrow} S' \underset{G_0, \text{left}}{\overset{+}{\Rightarrow}} t$$

Thus, G_0 is ambiguous.

- Equally, if G_1 is ambiguous, then there are two distinct leftmost derivations d, d' of t in G_0. The first production to be applied in a derivation $S \underset{G_0, \text{left}}{\overset{*}{\Rightarrow}} t$ is either $S_0 \to S$ or $S \to S'$. If d, d' were to begin with the application of the same production, say $S_0 \to S$, then both derivations could take place in G_1, which contradicts the non-ambiguity of G_1. Thus, d and d' must begin with different productions, which means that they have the form :

$$d : S_0 \underset{G_0, \text{left}}{\Rightarrow} S \underset{G_0, \text{left}}{\overset{*}{\Rightarrow}} t$$

and

$$d' : S_0 \underset{G_0, \text{left}}{\Rightarrow} S' \underset{G_0, \text{left}}{\overset{*}{\Rightarrow}} t$$

Therefore, we have both $S \underset{G_0, \text{left}}{\overset{*}{\Rightarrow}} t$ and $S' \underset{G', \text{left}}{\overset{*}{\Rightarrow}} t$, so $t \in L(G_1) \cap L(G_2)$.

- If the ambiguity of context-free grammars were decidable, we could decide whether $L(G_0)$ is ambiguous and therefore, we could decide whether $L(G_1) \cap L(G_2) \neq \phi$. Thus, the ambiguity of context-free grammars is undecidable.

Q.17. What is meant by reduction of one problem to another?

OR Explain clearly with a suitable example the concept of "reduction of one problem to another". ***CT : S-06(6M)***

OR What is meant by "Reduction of problem A to problem B"? ***CT : S-08(4M)***

OR With example explain Reducing problem. ***CS : W-09, S-11(3M), CT : S-11(4M)***

Ans. Reduction of one problem to another :

- In general, if we have an algorithm to convert instances of one problem P_1 to instance of a problem P_2 that have the same answer, then we can say that P_1 reduces P_2.
- When we say problem P_1 can reduced to problem P_2, usually we mean that solving P_2 would allow us to solve P_1, or that, in some sense, P_1 is no harder than P_2. Thus if P_1 is not recursive, then P_2 cannot be recursive.
- If P_2 is non recursively enumerable then P_1 cannot be recursively enumerable.
- If there is reduction from P_1 to P_2 then :

(a) If P_1 is undecidable then so is P_2.

(b) If P_1 is non - recursively enumerable then so is P_2.

POST CORRESPONDENCE PROBLEM

Q.18. Write short note on :

Post correspondence machine.

CS : S-09,11, W-11(3M), W-09(2M), W-12(4M), CT : S-08,09(6M), W-07,08(5M), S-07,11, W-11(3M), S-10,12,W-10(1M)

OR Post correspondence problem.

OR What do you mean by "Post correspondence problem"? Explain.

Ans. Post correspondence problem (PCP) :

- An instance of Post Correspondence Problem (PCP) consists of two lists : $A = w_1 \ldots\ldots w_k$ and

$B = x_1 ..... x_k$ of strings over some alphabet Σ.

- This instance of PCP has a solution if there is any sequence of integers $i_1, i_2, ... i_m$, with $m \geq 1$ such that

  $w_{i_1}, w_{i_2}, ..., w_{i_m} = x_{i_1}, x_{i_2}, ...x_{i_m}$

- The sequence $i_1, ..., i_m$ is a solution to this instance of PCP.

Example : Let Σ (0, 1). Let A and B be lists of three strings each, as defined in fig. below.

	List A	List B
i	w_i	x_i
1	1	111
2	10111	10
3	10	0

Fig. An instance of PCP

- In this case PCP has a solution. Let $m = 4$, $i_1 = 2$, $i_2 = 1$, $i_3 = 1$, and $i_4 = 3$.

  Then $w_2 w_1 w_1 w_3 = x_2 x_1 x_1 x_3 = 101111110$

- Post correspondence problem can be used to show that a wide variety of problems are undecidable.

Q.19. Let $\Sigma = \{0, 1\}$ and let A and B lists are defined in fig. suggest whether the PCP has a solution or not. Explain your suggestion.

CS: S-11(4M), W-11(3M), CT : W-08,09,11(4M)

	List A	List B
I	w_i	x_i
1	1	111
2	10111	10
3	10	0

OR Does PCP with two list X {b, bab^3, ba} and Y {b^3, ba, a} have a solution ?

CS : S-09(3M), W-09(2M), S-10(5M), CT : W-07(8M)

Ans. The PCP has a solution for P = 4

$i_1 = 2$, $i_2 = 1$, $i_3 = 1$ and $i_4 = 3$ then

$w_2 w_1 w_1 w_3 = x_2 x_1 x_1 x_3$

$= 101111110$

which is the solution of instance of PCP .

Q.20.(a) Let $\Sigma = \{0, 1\}$ and let A and B lists as defined in following fig. Suggest whether PCP has a solution or not.

	List A	List B
i	w_i	x_i
1	10	101
2	011	11
3	101	011

Fig. PCP Instance

Ans.

- Suppose that this instance of PCP has a solution $i_1, i_2, ... i_m$.
- Clearly, $i_1 = 1$, since no string beginning with $w_2 = 011$ can equal a string beginning with $x_2 = 11$; no string beginning with $w_3 = 101$ can equal a string beginning with $x_3 = 011$.
- We write the string from list A above the corresponding string from B. So far we have

  10

  101
- The next selection from A must begin with a 1. Thus $i_2 = 1$ or $i_2 = 3$. But $i_2 = 1$ will not do, since no string beginning with $w_1 w_1 = 1010$ can equal a string beginning with $x_1 x_1 = 101101$.
- With $i_2 = 3$, we have

  10101

  101011
- Since the string from list B again exceeds the string from list A by the single symbol 1, a similar argument shows that $i_3 = i_4 = ... = 3$.
- Thus there is only one sequence of choices that generates compatible strings, and for this sequence string B is always one character longer.
- Thus this instance of PCP has no solution.

Q.20.(b) Does the PCP with x = (b^3, ab^2) and y = (b^3, bab^3) have solution?

CT : S-10(2M), 13(5M)

Ans. In this problem we have to consider that list A has $w_1 = b^3$, $w_2 = ab^2$ and list B has $x_1 = b^3$

$x_2 = bab^3$ as an instance of PCP.

	List A	List B
i	w_i	x_i
1	b^3	b^3
2	ab^2	bab^3

Table = instance of PCP

As for i = 1

$w_1 = x_1 = b^3$

Hence this PCP have a solution.

Q.21.(a) Prove that PCP with two list x = (01, 1, 1) $y = (01^2, 10, 1^2)$ has no solution.

CT : S-08,10,12(4M), W-10(4M)

	List x	List y
i	w_i	z_i
1	01	01^2
2	1	10
3	1	1^2

Ans.

- Suppose that this instance has a solution $i_1, i_2, \ldots i_m$

  Let $i_1 = 1$
- We write the string from list x above the corresponding string from y. So far we have

  0 1

  $0\ 1^2$
- The next selection from list x begin with 2 or 3. Thus $i_2 = 2$ or $i_2 = 3$.
- But $i_2 = 1$ will not do, since no string beginning with $w_1w_1 = 0101$ can equal a string beginning with $z_1z_1 = 01^201^2$.

  Case 1 : $i_2 = 2$

  with $i_2 = 2$ we have

  011

  $01^2\ 10$

  Now with $i_2 = 3$ we have

  0111

  $01^2\ 1001^2$
- Since the string from list y exceeds the string from list x. A similar argument shows that $i_4 = 3$.
- Thus according to above sequence of choices that generate compatible strings, the string y is always longer.
- Thus this instance of PCP has no solution.

  Case 2 : $i_2 = 3$

  with $i_2 = 3$ we have

  011

  $01^2\ 1^2$
- Since the string from list y again exceeds the string from list x, a similar argument shows that $i_3 = i_4 = \text{----} = 3$.
- Thus according to above sequence of choices that generates compatible strings for which string y is always longer by some character. Thus this instance has no solution.
- Hence the given PCP has no solution.

Q.21.(b). Consider the PCP system described by the following lists.

α = { 10, 01, 0, 100, 1}

β = {101, 100, 10, 0, 010}

Does this PCP have a solution ?

CS : W-12(2M)

Ans. Let α, β be the lists of five string each as shown in figure below.

List i	α_i	β_i
1	10	101
2	01	100
3	0	10
4	100	0
5	1	010

Fig. PCP Instance

In this case,

List α = α_1 α_4 α_5 α_2

10 100 1 01

List β = 101 0 010 100

β_1 β_4 β_5 β_2

This PCP does not have solution.

Q.22. Give an application of PCP.

CT.: S-06,07(4M), W-06,09, S-08(3M)

Ans. An application of PCP : Post's Correspondence Problem can be used to show that a wide variety of problems are undecidable. We give only one application here : the undecidability of ambiguity for context - free grammars.

Theorem : It is undecidable whether an arbitrary CFG is ambiguous.

Proof :

- Let $A = w_1, w_2 \ldots\ldots, w_n$ and $B = x_1, x_2, \ldots\ldots, x_n$ be two lists of words over a finite alphabet Σ.
- Let $a_1, a_2, \ldots\ldots a_n$ be new symbols.
- Let $L_A = \{w_i, w_{i_2} \ldots\ldots w_{i_m} a_{i_m} a_{i_{m-1}} \ldots\ldots a_{i_1} \mid m \geq 1\}$ and $L_B = \{x_{i_1}, x_{i_2} \ldots\ldots x_{i_m} a_{i_m} a_{i_{m-1}} \ldots\ldots a_{i_1} \mid m \geq 1\}$ Let G be the CFG $(\{S, S_1, S_2\}, \Sigma \cup \{a_1 \ldots\ldots a_n\}, P, S)$, where P contains the productions $S \rightarrow S_A$, $S \rightarrow S_B$ and for $1 \leq i \leq n$, $S_A \rightarrow w_i S_A a_i$, $S_A \rightarrow w_i a_i$, $S_B, \rightarrow x_i S_B a_i$ and $S_B \rightarrow x_i a_i$.
- The grammar G generates the language L_A L_B.
- If the instance (A, B) of PCP has a solution, say $i_1, i_2, \ldots\ldots, i_m$, then there is a word in L_A that equals the word in L_B. This word has a leftmost derivation beginning $S \rightarrow \overset{*}{S_A}$ and another beginning $S \rightarrow S_B$. Hence in this case G is ambiguous.
- Conversely, suppose G is ambiguous. Since the a's dictate the productions used, it is easy to show that any word derived from S_A has only one leftmost derivation from S_A.
- Similarly, no word derived from S_B has more than one leftmost derivation from S_B. Thus it must be that some word has leftmost derivations from both S_A and S_B.
- If this word is $ya_{im}, a_{i_{m-1}} \ldots\ldots a_{i_1}$, where y is in E^* then $i_1, i_2, \ldots\ldots i_m$ is a solution to PCP.
- Thus G is ambiguous if and only if the instance (A, B) of PCP has a solution.
- We have thus reduced PCP to the ambiguity problem for CFG's.
- That is, if there were an algorithm for the latter problem, we could construct an algorithm for PCP, which does not exist.
- Thus the ambiguity problem for CFG's is undecidable.

Q.23. Explain the modified version of PCP.

CS : W-09(1M), CT : W-06(4M)

Ans. The Modified Post's Correspondence Problem (MPCP) is the following :

Given lists A and B, of k strings each from Σ^*, say

$A = w_1, w_2, \ldots\ldots, w_k$ and $B = x_1, x_2, \ldots\ldots, x_k$; does there exist a sequence of integers, $i_1, i_2, \ldots\ldots i_r$, such that

$w_1 w_{i_1} w_{i_2}, \ldots\ldots\ldots, w_{i_r} = x_1 x_{i_1} x_{i_2}, \ldots x_{i_r}$

The difference between the MPCP and PCP is that in the MPCP, a solution is required to start with the first string on each list.

Q.24. Prove that if PCP were decidable, then MPCP would be decidable.

Ans. Theorem : If PCP were decidable, then MPCP would be decidable. That is, MPCP reduces to PCP.

Proof :

- Let $A = w_1, w_2, \ldots\ldots w_k$ and $B = x_1, x_2 \ldots\ldots, x_k$ be an instance of the MPCP.
- We convert this instance of MPCP to an instance of PCP that has a solution if and only if our MPCP instance has a solution.
- If PCP were decidable, we would then be able to solve the MPCP, proving the theorem.
- Let Σ be the smallest alphabet containing all the symbols in lists A and B, and let ¢ and $ not be in Σ.
- Let y_i be obtained from w_i by inserting the symbol ¢ after each character of w_i and let z_i be obtained from x_i by inserting the symbol ¢ ahead of each character of x_i.

- Create new words

  $y_0 = ¢\ y_1, \quad z_0 = z_1 z_0 = z_1$

  $y_{k+1} = \$, \quad z_{k+1} = ¢\$$

- Let $C = y_0, y_1, \ldots y_{k+1}$ and $D = z_0, z_1, \ldots z_{k+1}$.

For example : The lists C and D constructed from the lists A and B are shown in fig. below.

		List A	List B
M	i	w_i	x_i
P	1	1	111
C	2	10111	10
P	3	10	0

	List C	List D
i	y_i	z_i
0	c 1 c	c 1 c 1 c 1
1	1 c	c 1 c 1 c 1
2	1 c 0 c 1 c 1 c 1 c	c 1 c 0
3	1 c 0 c	c 0
4	$	c $

Fig. Corresponding instances of MPCP and PCP

- In general, the lists C and D represent an instance of PCP.
- We claim that this instance of PCP has a solution, if and only if the instance of MPCP represented by lists A and B has a solution.
- To see this, note that $1, i_2, i_2 \ldots\ldots i_r$ is a solution to MPCP with lists A and B, then $0, i_1, i_2, \ldots\ldots, i_r, k+1$ is a solution to PCP with lists C and D.
- Likewise, if $i_1, i_2, \ldots\ldots i_r$ is a solution to PCP with lists C and D, then $i_1 = 0$ and $i_r = k+1$, since y_0 and z_0 are the only words with the same index that begin with the same symbol and y_{k+1} and z_{k+1} are the only words with the same index that end with the same symbol.
- Let j be the smallest integer such that $i_j = k+1$. Then $i_1, i_2, \ldots\ldots i_j$ is also a solution, since the symbol \$ occurs only as the last symbol of y_{k+1} and z_{k+1} and for no l, where $1 \ l \ j$ is $i_l = k+1$.
- Clearly $1, i_2, i_3, \ldots\ldots, i_{j-1}$ is a solution to MPCP for lists A and B.
- If there is an algorithm to decide PCP, we can construct an algorithm to decide MPCP by converting any instance of MPCP to PCP as above.

HALTING PROBLEM

Q.25. Explain in detail Halting problem of Turing machine. **CS : W-12(3M)**

OR Write a short note on :

Halting problem **CS : S-09(4M),**

CT : S-09(5M), S-10(6M), W-11(3M), S-12,13(4M)

Ans. Halting Problem :

- If we cannot solve the problem of whether a given TM T accepts the particular string e (T), we obviously cannot expect to solve the more general membership problem for recursively enumerable languages. This is the most well-known unsolvable decision problem.
- Because of the way a TM accepts a string, the problem is known as the halting problem.
- The halting problem is stated as follows :

  Does there exists a TM which for given a description of an arbitrary TM, T_A and the representation of its input string x, can decide whether or not machine T_A halts when given x as its input string.
- This can be expressed in the form of computability as follows :

  Does there exists a total function

  $f : N \times N \rightarrow \{0, 1\}$ defined as

  $f(x, y) = 1$, if T_y comes to a halt state on reaching x

  $= 0$ if T_y does not halt on reaching x

  In the above statement, T_y denotes the y^{th} TM according to the enumeration of the set of TMs.

Example :

- An instance of the halting problem consists of a TM and a string.

- With our encoding function e, we can represent an instance (T, w) by the string e(T) e (w), and therefore we consider the language
  H = {e (T) e (w) w ∈ L (T) }
- To say that the halting problem is unsolvable is to say that H is not recursive.
- However, we prove it more systematically by introducing a general technique that will be useful from now on, the idea of reducing one problem to another.
- When we say problem P_1 can be reduced to problem P_2, usually we mean that solving P_2 would allow us to solve P_1, or that, in some sense, P_1 is no harder than P_2.
- Note that the second version (or the contrapositive of the first) can also be interpreted to mean that if P_1 is hard, then so P_2 is hard.

Q.26. Show that the halting problem is unsolvable. ***CS : S-11(2M), CT : W-11(3M)***

Ans.

- Consider the halting problem is solvable. Therefore there exists an algorithm P to which when a pair (M, w) (i.e. encoded representation of Turing machine and w is given as input) then it reaches to accept halt if M halts on w and it reaches reject halt if M does not halt as shown in fig.

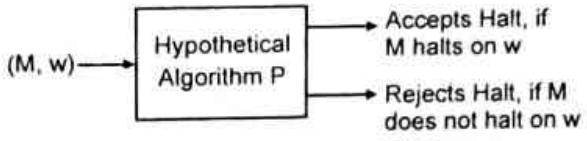


Fig.

- Let us construct an algorithm Q using P as follows :
  Q takes < M >
  That is encoding of an arbitary Turing machine as input and copies it to obtain <M, M> and then applies P to this input. Q loops forever if P reaches to accept halt and Q halts if P reaches to reject halt.
- Since Q is assumed to be an algorithm there exists a Turing machine M' which execute algorithm Q.
- Since P works for any arbitrary Turing machine and Q is using P, Q must also work for arbitrary Turing machine. Hence Q must work also for M'.
- Therefore if M = M', i.e for any string w if M works then M' also works for the same input but we get the following contradiction :

(1) M' halt if M' loops forever and

(2) M' loops forever if M' halts and hence P cannot be algorithm.

- Hence the halting problem is unsolvable.

Q.27. Show that it is undecidable whether a TM halts on all inputs. ***CS : W-11,12(3M) CT : S-08(7M)***

Ans.

- Let M = (Q, Σ, Γ, δ, q_0, B, F) be a TM.
- We construct another TM M', such that M' halts on x if and only if M accepts x.
- M is designed to behave as follows :
- First, it shifts its input one position right, placing a left end marker $ on the leftmost cell.
- M' then simulates M.
- If δ (q, X) is undefined, and either (i) q is nonaccepting and X is any symbol in Γ {$} [note that δ (q, $) is surely undefined], or (ii) q is accepting and X is $, then M'scanning X in state q moves right and enters state p_1
- In state p_1, scanning any symbol, M' moves left and enters state p_2.
- In that state, M' moves right and enters p_1 again.
- Thus M' loops forever if M either halts in a nonaccepting state or falls off the left end of the tape in any state.
- If M enters an accepting state, but scanning $, then halts. Thus halts if and only if M accepts its input, as desired.

Note : The same proof can be used to show that

(a) "Does a TM halt on a given input" is undecidable.

(b) "Does a TM halt on some input " is undecidable.

Q.28. Show using the undecidability of halting problem that there exists no algorithm to determine if any arbitrary Turing machine halts on empty strings. ***CT : S-11(4M)***

Ans.

- Let M be an arbitrary Turing machine over = {0, 1}.

- Construct a new Turing machine M' over $\Sigma = \{0, 1\}$ which generate any word w in Σ^* onto its tape and then applies Turing machine M to that w.
- Our objective is to start with a pair (M, w) and find a TM so that (T, w) is a yes - instance - of the halting problem if and only if M' is a yes instance of accepts $\in$.
- In other words, we want M' to behave so that $\in$ belongs to L (M') if and only if w belongs to L (T).
- At first, this seems as though it should be difficult. Halting on input $\in$ affects with halting on input w.
- The point is that we are allowed to use w in the construction of M'; in particular, M' can start with an empty tape and manufacture a tape with w on it by simply writing w.
- Therefore, we choose to be the composite of TM.
  M' = Write (w) M
  where write (w) is the TM that halts with tape contents Bw when started with a blank tape.
- It is clear that M' halts on input $\in$ if and only if M halts on input w.
- Therefore, if M halts then M' also halts.
- Since halting problem is undecidable, M' halts on empty string is undecidable therefore M accepts $\in$ or empty string is undecidable.

INTRODUCTION TO RECURSIVE FUNCTION THEORY

Q.29. Write detailed notes on Recursive functions.

Ans. Recursive Function :

- A function which calls itself directly or indirectly and terminates after finite number of steps is called as recursive function.
- In recursive function, terminating point is known as base point.
- A recursive function will be good if it has terminating point and its call to itself should be nearer to its terminating point.
- Leg g be a function of n + 1 arguments, g' be a function of n arguments, and b be a function of m arguments where $m \leq n + 2$. We define the function g recursively using the g' and b function as follows :

  If $(x_1, x_2, \ldots x_n, 0) = g'(y_1, y_2, \ldots y_n)$ then

  $g(x_1, x_2, \ldots, x_n, x_{n+1} + 1)$

  $= b(x_1, x_2, \ldots, x_n, x_{n+1},$

  $g(x_1 x_2, \ldots, x_n, x_{n+1} + 1)$

  where g' and b are defined functions.
- A recursive function is a function defined by a finite set of rules that for various arguments specify the function in terms of variables, non negative integer constants, the successor (add one) function, the function itself or an expression built from these by composition of functions.

  For example : Ackermann's function is defined by the rules, for m, n = 0, 1, 2,

(1) A (m, n) = n + 1, m = 0
i..e A (0, n) = n + 1

(2) A(m, n) = A (m – 1, 1) ,
$m \neq 0, n = 0$
i.e. A (m, 0) = A (m –1 , 1)

(3) A(m, n) = A (m –1 , A (m, n – 1))
if $f(i_1, \ldots, i_k)$
is defined for all $i_1, \ldots, i_k$, then we say that f is total recursive function.

- A function $f(i_1, \ldots, i_k)$ computed by a Turing machine is called a partial recursive function.
- In a sense, the partial recursive functions are analogous to the recursive enumerable languages, since they are computed by Turing machines that may or may not halt on a given input.
- The total recursive functions corresponds to the recursive lanaguages, since they are computed by TM's that always halt.
- All common arithmetic functions on integers, such as multiplication, n!, $[\log_{2^n}]$ and 2^n are total recursive functions.
- There are primitive recursive functions and μ-recursive functions also.

- A function is said to be recursive if it can be obtained from the initial or basic function by a finite number of application of the operations of composition, recursion and minimization over regular function.

Q.30. Give and prove recursion theorem.

Ans. The recursion theorem :

- The recursion theorem states that every total recursive function σ mapping indices (integers denoting Turing machines) of partial recursive functions into indices of partial recursive functions has a fixed point x_0 such that $f_{x0}(y) = f_{\sigma(x0)}(y)$ for all y..
- In other words, if we modify all Turing machines in some manner, there is always some Turing machine M_{x0} for which the modified Turing machine $M_{s(x0)}$ computes the same function as the unmodified Turing machine.
- At first this sounds impossible, since we can modify each Turing machine to add 1 to the originally computed function.
- One is tempted to say that $f(y) + 1 \neq f(y)$, but note that if $f(y)$ is everywhere undefined, then $f(y) + 1$ does equal $f(y)$ for all y.

Theorem : For any total recursive function σ, there exists an x_0 such that $f_{x_0}(x) = f_{\sigma(x0)}(x)$ for all x.

Proof : For each integer i, construct a TM that on input x computes $f_i(i)$ and then simulates by means of an universal TM, then $f_i(i)$ TM on x. Let g (i) be index of the TM so constructed. Thus for all i and x,

$$f_{g(i)}(x) = f_{f_i(i)}(x) \qquad \text{----- (1)}$$

Observe that g (i) is a total function even if $f_i(i)$ is not defined. Let j be an index of the function σg. That is, j is an integer code for a TM that, given input i, computes g (i) and then applies σ to g (i). Then for $x_0 = g(j)$ we have

$$f_{x0}(x) = f_{g(j)}(x)$$
$$= f_{f_j(j)}(x)$$
$$= f_{\sigma(g(j))}(x) \text{ since } f_j \text{ is the function } \sigma g$$
$$= f_{\sigma(x0)}(x)$$

Thus x_0 is a fixed point of the mapping σ. That is $TM x_0$ and $TM \sigma(x_0)$ compute the same function.

Q.31. Give an application of recursion and S_{mn} theorem.

Ans. Definition of Recursion : The following operation which defines a function $f(x_1, x_2, \ldots\ldots x_n, y)$ of n + 1 variables by using two other functions g $(x_1, x_2, \ldots\ldots x_n)$ and h $(x_1, x_2, \ldots, x_n, y, z)$ of n-variables, respectively, is called recursion.

$$f(x_1, x_2, \ldots\ldots x_n, 0) = g(x_1, x_2, \ldots\ldots, x_n)$$
$$f(x_1, x_2, \ldots\ldots x_n, y+1) = h(x_1, x_2, \ldots\ldots, x_n, y,$$
$$f(x_1, x_2, \ldots\ldots, x_n, y))$$

- Let $M_1, M_2, \ldots\ldots$ be any enumeration of all Turing machines.
- We do not require that this enumeration be the "standard" one introduced but only that whatever representation is used for a TM, we can by an algorithm convert from that representation to the 7-tuple notation introduced and vice versa.
- Then we can use the recursion theorem to show that for some i, M_i and M_{i+1} both compute the same function.
- Let σ (i) be the total recursive function defined as follows.
- Enumerate TM's $M_1, M_2, \ldots\ldots$until one with integer code i is found.
- Note that the states of the TM must be considered in all possible orders to see if i is a code for this TM.
- Having found that M_j has code i, enumerate one more TM, M_{j+1}, and let σ (i) be the code for M_{j+1}.
- Then the recursion theorem applied to this σ says there is some x_0 for which Mx_0 and define the same function of one variable.

Q.32. Explain the categories of basic functions. Discuss various approaches used to build a complicated function.

OR What do you mean by partial recursive function? **CT : S-11(2M)**

Ans. Consider only functions of one or two variables whose domain is either N, the set of all negative integers, or N × N, and whose range is in N.

The categories of basic function are as follows :

(i) Composition of function : A composition function can be defined by the combination of two or more functions. Let $f : A \rightarrow B$ and $g : B \rightarrow C$ be two functions. The composition of f and g written as g o f is a function from A to C defined as :

g o f = { (z, c) | for b, (a, b) in f, some (b, c) is in g}.

(ii) Primitive recursive function : A function is called primitive recursive if and only is it can be constructed from the basic function like zero, successor, projector, by successive composition and primitive recursion.

F (x, 0) = g, (x), f (x, y + 1) = h (g_2 (x, y), f (x, y)), for defined functions g_1, g_2 and h.

Primitive recursive function includes the following functions :

(A) Zero function :

Zero function is denoted by Z and defined as follows :

$Z(n) = 0$ for $\forall n \in N$

The zero function returns zero regardless of its argument.

(b) Successor Function :

Successor function is denoted by S and defined as follows :

$S(n) = n + 1$ for $\forall n \in N$

The successor function takes two argument and returns the succeeding number.

Example :

S (2) = 2 + 1 = 3, where 2, 3 ∈ N

Z (0) = 0 where 0 ∈ N

Z (2) = 0 where 2 ∈ N

(c) Projection function :

Projection function is denoted by P and defined as follows :

$P_{in}(a_1, a_2, ...a_n) = a_i$,

where $a_i \in N$ for i = 1, 2,.....n and $i \leq n$.

The projection function takes n arguments and returns their i_{th} argument. When it takes one argument it returns its argument as its value.

(iii) Total recursive function :

A total function is a subclass of partial function. A function is called total function if it is defined for all its arguments. Let $f(a_1, a_2.....a_n)$ be a function and define on function $g(b_1, b_2.....b_m)$, then f is total function if every elements of f is assigned to some unique element of function g.

Example :

Addition of two positive integers is a total recursive function.

Solution :

Let f (m, n) = m + n is the addition function from N × N to N of two positive integers m and n. This function can be defined recursively as :

f (m, 0) = 1

f (m, n + 1) = f (m, n) + 1

Since f is defined for all values of m, n ∈ N. So f is a total recursive function.

(iv) Partial Recursive function :

A function is called partial recursive if it is defined for some of its arguments. Let $f(a_1, a_2.....a_n)$ be a function and defined on function $g(b_1, b_2.....b_m)$, then f is partial function if some elements of f is assigned to almost one element of function g. A partial function is recursive if

- It is an initial function over N, or
- It is obtained by applying recursion or composition or minimization on initial function over N.

Q.33. Give a recursive definition for n!

CT : S-08(4M)

Ans. Let f(n) = n !

The factorial of zero is 1. Therefore, 0! = 1.

$f(0) = 0! = 1$ and

$\therefore f(1) = 1! = 1$

For every $n \geq 1$, $f(n) = n\, f(n-1)$

To evaluate f (4), for example, we can use the definition

$f(4) = 4 \times f(3)$

$f(4) = 4 \times 3 \times f(2)$

$f(4) = 4 \times 3 \times 2 \times f(1)$

$f(4) = 4 \times 3 \times 2 \times 1$ *[$\because f(1)=1$]*

$f(4) = 24$

Hence, recursive function can be best used for finding the factorial of a number.

Q.34. Write a short note on Mod and Div functions.

CS : S-10(7M), CT : W-06,10(3M), W-08(4M), S-09(5M)

OR Show that the following functions are primitive recursive :

(i) q (x, y) = the quotient obtained when x is divided by y.

(ii) r (x, y) = the remainder obtained when x is divided by y. ***CT : S-11(5M)***

Ans. Mod and Div function :

- For natural numbers x and y with y > 0, we denote by Div (x, y) and Mod (x, y) the integer quotient and remainder, respectively, when x is divided by y.
- **For example :** Div (8, 5) = 1, Mod (8, 5) = 3, and Mod (12, 4) = 0. As it stands, these are not total functions on $\mathcal{N} \times \mathcal{N}$, because we do not allow division by 0 ; however, it will be useful to extend the definition and to show that the results are actually primitive recursive.
- Let us say that for any x, Div (x, 0) = 0 and Mod (x, 0) = x.

  Then the usual formula

  $x = y * \text{Div}(x, y) + \text{Mod}(x, y)$

  still holds for every x and y, and

  $0 \ \text{Mod}(x, y) < y$

  is true as long as y > 0.
- We begin by showing that Mod is primitive recursive.
- The derivation involves recursion in the first variable, and for this reason we let

  $R(x, y) = \text{Mod}(y, x)$
- The primitive recursiveness of Mod follows from that of P.
- The following formulas can be verified easily.

  $R(x, 0) = \text{Mod}(0, x) = 0$

  $R(x, k+1) = \text{Mod}(k+1, x)$

$$= \begin{cases} R(x,k)+1 & \text{if } x \neq 0 \text{ and } R(x,k)+1 < x \\ 0 & \text{if } x \neq 0 \text{ and } R(x,k)+1 = x \\ k+1 & \text{if } x = 0 \end{cases}$$

For example :

$R(5, 6+1) = \text{Mod}(7, 5) = \text{Mod}(6, 5) + 1$

since $5 \neq 0$ and $\text{Mod}(6, 5) + 1 = 1 + 1 < 5$, and

$R(5, 9, +1) = \text{Mod}(10, 5) = 0$

since $\neq 0$ and $\text{Mod}(9, 5) + 1 = 4 + 1 = 5$. The function h defined by

$$h(x_1, x_2, x_3) = \begin{cases} x_3 + 1 & \text{if } x_1 \neq 0 \text{ and } x_3 + 1 < x_1 \\ 0 & \text{if } x_1 \neq 0 \text{ and } x_3 + 1 = x_1 \\ x_2 + 1 & \text{if } x_1 = 0 \end{cases}$$

is not a total function, since it is undefined if $x_1 \neq 0$ and $x_3 + 1 > x_1$. However, the modification

$$h(x_1, x_2, x_3) = \begin{cases} x_3 + 1 & \text{if } x_1 \neq 0 \text{ and } x_3 + 1 < x_1 \\ 0 & \text{if } x_1 \neq 0 \text{ and } x_3 + 1 \geq x_1 \\ x_2 + 1 & \text{if } x_1 = 0 \end{cases}$$

can also be used.

- The function R is obtained by primitive recursion from C_0^1 and this modified h, theorem implies that h is primitive recursive. Therefore, so are R and Mod.
- The function Div can now be handled in a similar way. If we define Q (x, y) to be Div (y, x), then it is not hard to check that Q is obtained by primitive recursion from C_0^1 and the primitive recursive function h_1 defined by

$$h(x_1, x_2, x_3) = \begin{cases} x_3 & \text{if } x_1 \neq 0 \text{ and } \text{Mod}(x_2, x_1) + 1 < x_1 \\ x_3 + 1 & \text{if } x_1 \neq 0 \text{ and } \text{Mod}(x_2, x_1) + 1 = x_1 \\ 0 & \text{if } x_1 = 0 \end{cases}$$

(Note that for any choice of (x_1, x_2, x_3), precisely one of the predicates appearing in this definition is true.)

Q.35. Define bounded minimalization.

CT : S-06(6M),W-06(3M), W-08(2M) S-09(4M), W-09(3M), W-10,11(2M)

OR Write short note on :

Bounded minimalization.

Ans. Bounded minimalization :

- For an (n + 1) – place predicate P, the bounded minimalization of P is the function $m_p : N^{n+1} \rightarrow N$ defined by

  $m_P(X,k)$

  $$= \begin{cases} \min \{y \mid 0 \le y \le k \text{ and } P(X,y)\} & \text{if this set is not empty} \\ & \text{otherwise} \end{cases}$$

- The symbol μ is often for the minimalization operator, and we sometimes write

  $m_P(X,k) = k_{\mu y[P(X,y)]}$

- An important special case is that in which P (X, y) is (f (X, y) = 0), for some $f : N^{n+1} \rightarrow N$. In this case m_P is written m_f and referred to as the bounded minimalization of f.

  Proposition 1 :

  If f:Nm+1→N is primitive recursive, so is fbmin.

  Proof :

  Define g:=sgn ∘ f. Then

  g(x,y):={01}if f(x,y)=0, otherwise.

  As f is primitive recursive, so is g, since the sign function sgn is primitive recursive.

  Next, the function gp obtained from g by bounded product has the following properties :

  (i) if gp(x,y)=1, then gp(x,z)=1 for all z<y,

  (ii) if gp(x,y)=0, then gp(x,z)=0 for all z≥y.

- Finally, the function (gp)s obtained from gp by bounded sum has the property that, when applied to (x,y), counts the number of z≤y such that gp(x,z)=1.

- This means that (gp)s=fbmin for all (x,y)∈Nm+1. Since gp is primitive recursive, so is (gp)s, or that fbmin is primitive recursive.

- In fact, if f is a (total) recursive function, so is fbmin, because all of the derived functions in the proof above preserve primitive recursiveness as well as totalness.

Q.36. Write short note on :

Unbounded minimization or minimalization.

CT : S-09(4M), W-06,08,10,11(2M)

Ans. Unbounded minimalization :

- If P is an (n + 1) – place predicate, the unbounded minimalization of P is the partial function $M_p \; N^n \rightarrow N$ defined by

  $M_P(X) = \min \{y \mid P(x, y) \text{ is true}\}$

  $M_P(X)$ is undefined at any $x \in N^n$ for which there is no y satisfying P (X, y).

- The notation μy [P (X, y)] is also used for $M_P(X)$.

- In the special case in which P (X, y) = (f (X, y) = 0). We write $M_P = M_f$ and refer to this function as the unbounded minimalization of f.

- The fact that we want M_P to be a computable partial function for any computable P also has another consequence.

- Suppose, again, that the algorithm we are relying on for computing $M_P(X)$ is the simple - minded one of evaluating P (X, y) for increasing values of y.

- Suppose also that for a particular y_0, P (X, y_0) is undefined. There might be a value $y_1 > y_0$ for which P (X, y_1) is true.

- However, if we get stuck in an infinite loop while trying to evaluate P (X, y_0), we will never get around to considering P (X, y_1).

- This problem can be avoided by stipulating that unbounded minimalization be applied only to total predicates or total function.

- Unbounded minimalization is the last of the operations needed in order to characterize the computable functions.

Q.37. Define μ-recursive functions.

CT : S-12(3M), W-06,09(4M), W-07,10,11(3M)

Ans. μ-recursive functions :

- The set of M of μ- recursive, or simply recursive, partial functions is defined as follows :

(1) All initial functions are elements of M.

(2) Any function obtained from elements of M by composition or primitive recursion is an element of M.

(3) For any $n \geq 0$ and any total function $f : \mathcal{N}^{n+1} \to \mathcal{N}$ in M, the function

$M_f : \mathcal{N}^n \to \mathcal{N}$ defined by

$M_f(X) = \mu y [f(X, y)]$

$= 0$ is an element of M.

(4) No other functions are in the set M.

- A function is said to be μ- recursive, if it can be constructed from the basic functions by a sequence of applications of the μ-operator and the operations of composition and primitive recursion.

Q.38. Explain the concept of minimalization or minimization or μ-operation.

Ans. Minimalization :

- A function $f(x_1, x_2, ..., x_n)$ is said to be defined from a total function g $f(x_1, x_2, ..., x_n, y)$ by minimization (minimalization) or μ operation if

$$f(x_1, x_2, ..., x_n) = \begin{cases} \mu_y(g\langle x_1, x_2, ..., x_n, y\rangle = 0) & \text{if there is such a y} \\ \text{undefined} & \text{otherwise} \end{cases}$$

where μ_y means the least y greater than or equal to zero :

i.e. $\mu_y (g(x, y))$ = smallest y such that $g(x, y) = 0$.

- From the definition it follows that $f(x_1, x_2, ..., x_n)$ is :vell - defined and total if g is regular.
- If g is not regular, then the operation of minimization may produce a partial function.

Let, $g(x, y) = x + y - 3$

which is a total function. If $x \leq 3$, then

$y = 3 - x$

is the result of the minimalization, but if $x > 3$, then there is no y such that $x + y - 3 = 0$, Therefore,

$\mu_y (g(x, y)) = 3 - x$, for $x \leq 3$

= undefined, for $x > 3$.

- We see from this that even though g (x, y) is a total function, $\mu_y (g(x, y))$ may only be partial.
- As the above example shows, the minimalization operation opens the possibility of defining partial functions recursively. But it turns out that it also extends the power to define total functions so as to include all computable functions.

Q.39. Explain : All μ-recursive partial functions are computable.

Ans.

- The proof is by structural induction.
- We know that partial functions obtained from computable functions by either composition or primitive recursion are computable.
- Therefore, it is enough to show that if $f : \mathcal{N}^{n+1} \to \mathcal{N}$ is a computable total function, then its unbounded minimalization M_f is a computable partial function.

Q.40. Prove that every computable function $f : N^n \to N$ is μ - recursive

Ans. Theorem : Every computable function

$f : N^n \to N$ is μ - recursive

Proof :

- If f is computed by the TM T, then we establish the formula

$f = \text{Result}_T\ 0\ f_T\ 0\ \text{InitConfig}^{(n)}$

- For any n-tuple X, $\text{InitConfig}^{(n)}(X)$ is the number of the initial TM configuration corresponding to input X.
- It does not matter which TM we use here because we have agreed to label the initial state of any TM as q_1.

- The numeric function f_T corresponds to the processing done by T: that is f_T takes a number that represents an initial configuration of T and returns the number representing the halting configuration ultimately reached by T.
- The function Result $_T$ takes the number of this halting configuration, in which the string representing output f(X) is on the tape, and produces the numeric value f(X).
- The function Result $_T$ is one of several whose value at a number m depends on whether m is the number of a configuration of T.
- The first step is therefore to examine the 1-place predicate IsConfig$_T$ defined by

  IsConfig$_T$ (n) = (n is a configuration number for T)
- Let T be a TM computing f. Then for any m and k, Halting$_T$

  (Trace $_T$ (m, k) is 0 if and only if m is a configuration number for T and, beginning in configuration m, T halts within k moves. Let MovesToHalt : N $\rightarrow$ N be defined by

  MovesToHalt (m)

  $= \mu k\ [\text{Halting}_T\ \text{Trace}_T\ (m, k)) = 0]$

  and define $f_T : \mathcal{N} \rightarrow \mathcal{N}$ by

  $f_T : (m) = \text{Trace}_T$ (m, MovesToHalt(m))
- We may describe the functions MovesToHalt and f_T as follows :
- If m is a configuration number for T and if T eventually halts when starting from configuration m, then MovesToHalt(m) is the number of moves from that point before T halts, and f_T (m) is the number of the halting configuration that is eventually reached.
- For any other m, both functions are undefined.
- Because MovesToHalt is obtained from a primitive recursive (total) function by unbounded minimalization, MovesToHalt is μ-recursive, and because f_T is obtained by composition from μ-recursive functions, f_T is also μ-recursive.
- We claim that $f = \text{Result}_T\ (f_T\ (\text{InitConfig}^{(n)}))$ which means that for any X, f(X) is defined if and only if Result$_T$ (f_T (InitConfig$^{(n)}$(X)) is defined, and that if these numbers are defined, then they are equal.
- To see this, on the one hand, suppose that f (X) is defined.
- In this case, if T begins in the configuration (InitConfig$^{(n)}$ (X)) , it eventually halts.
- Therefore, f_T (InitConfig$^{(n)}$(X)) is the configuretion number of the halting configuration $(h, \underline{\Delta} 1^{f(x)})$, and when Result$_T$ is applied to this configuration number it produces f(X).
- On the other hand, suppose that f(X) is undefined. Then T fails to halt on input X, and this means that f_T (InitConfig$^{(n)}$(X)) is undefined. The proof is complete.

Q.41. Prove that :

Let $f : \sum_1^* \rightarrow \sum_2^*$. Then f is computable if and only if f is μ - recursive.

Ans. Proof :

- We denote by g_1 and g_2 the Godel numbering functions for the two alphabets and by g_1' and g_2' their respective left inverses.
- Suppose, on the one hand, that f is computable, and let T_f be a TM (whose tape alphabet contains the symbols of Σ_1 and Σ_2) computing f.
- We want to show that f : N is μ-recursive.
- Although we will not show all the details, it is easy to see from the definition f that a composite TM can be constructed from three simpler ones : one that takes the initial tape $\Delta 1^n$ and halts with tape $\Delta g_1'(n)$; the TM T_f which then halts with tape $\Delta f\ (g_1'(n))$; and a third that calcualtes the Godel number of this string to yield the result, $g_2\ (f\ (n))) = {}_f\ (n)$.

- On the other hand, suppose that f is μ-recursive. Then, by definition, ρ_f is μ-recursive.
- Therefore, ρ_f is computable.
- Let T be a TM computing it. From the formula for ρ_f we have

$$g_2'(\rho_f(n)) = g_2'(g_2(fg_1'(n))))$$
$$= f(g_1'(n)) \text{ for any } n.$$

- Applying this formula when $n = g_1(x)$, we obtain

$$f(x) = f(g_1'(g_1(x))) = g_2'(\rho_f(g_1(x)))$$

- Just as before, we can now construct a composite TM T_f to perform this computation, and it follows that f is computable.

PRIMITIVE RECURSIVE FUNCTIONS

Q.42. Explain clearly what is meant by primitive recursive function. **CT : W-06,08, S-09(3M)**

OR What do you mean by primitive recursive function over n and over a, b? **CT : W-07(2M)**

OR Define primitive recursive function.

CS : S-11, W-11(3M), W-12(2M) CT : W-10(5M), S-11(2M), W-11(4M), W-12(3M)

Ans. Primitive recursive function : The set PR of primitive recursive functions is defined as follows :

(i) All initial functions are elements of PR.

(ii) For any $k \geq 0$ and $m \geq 0$; if f : $\mathcal{N}^k \to \mathcal{N}$ and $g_1, g_2, \ldots, g_k : \mathcal{N}^m \to \mathcal{N}$ are elements of PR, then the function $f(g_1, g_2, \ldots, g_k)$ obtained from f and $g_1, g_2, g_3, \ldots, g_k$ by composition is an element of PR.

(iii) For any $n \geq 0$, any function $g : \mathcal{N}^n \to \mathcal{N}$ in PR, and any function $h : \mathcal{N}^{n+2} \to \mathcal{N}$ in PR, the function $f : \mathcal{N}^{n+1} \to \mathcal{N}$ obtained from g and h by primitive recursion is in PR.

(iv) No other functions are in the set PR.

- Primitive recursive functions are those that have primitive recursive derivations.
- A function f has a primitive recursive derivation if there is a finite sequence of functions $f_0, f_1, \ldots, f_j$ so that $f_j = f$ and each function f_i in the sequence is either an initial function or obtainable from earlier functions in the sequence by composition or primitive recursion.
- The set PR is the smallest set of functions that contains the initial function and is closed under the operation of composition and primitive recursion.

Q.43. Define primitive recursive operation. **CT : W-06(2M)**

Ans. Primitive recursive operation : Suppose that $n \geq 0$, and g and h are functions of n and n + 2 variables, respectively. The function f ; $N^{n+1} \to N$ defined by the formulas,

$$f(X, 0) = g(X)$$
$$f(X, k+1) = h(X, k, f(X, k))$$

for every $X \in N^n$ and every $k \leq 0$, is said to be obtained from g and h by the operation of primitive recursion.

Q.44. Give the list of some primitive functions.

OR Explain category of basic primitive function. **CT : W-06(2M), W-08, S-09(3M), W-11(4M)**

Ans.

- The following are some of the primitive functions which are used frequently.
- In some cases we give only an informal definition, although a formal definition using the initial functions and other primitive recursive functions can be given easily.

(1) Sign function or nonzero test function, sg :

$$Sg(0) = 0 \quad Sg(y+1) = 1$$
$$\text{or } Sg(0) = Z(0) \quad Sg(y+1)$$
$$= S(Z(U_2^2 < y, sg(y)))$$

(2) Zero text function $\overline{Sg}$:

$$\overline{Sg}(0) = 1 \quad \overline{sg}(y+1) = 0$$

(3) Predecessor function, P :

$$P(0) = 0 \; P(y+1) = y = U_1^2(y, P(y))$$

Note that

$$P(0) = 0 \; P(1) = 0 P(2) = 1 \quad P(3) = 2 \ldots\ldots$$

(4) Odd and even parity function, Pr :

$Pr(0) = 0 \quad Pr(y+1) = \overline{S_g}(U_2^2(y, Pr(y)))$

$Pr(0) = 0 \quad Pr(1) = 1 \quad Pr(2) = 0$

$Pr(3) = 1 \ldots$

(5) Proper subtraction function, $\dot{-}$:

$z \dot{-} 0 = x \quad x \dot{-} (y+1) = P(x \dot{-} y)$

Note that $x \dot{-} y = 0$ for

$x < y$ and $x \dot{-} y = x - y$ for $x \geq y$.

(6) Absolute value function, | | :

$|x - y| = (x \dot{-} y) + (y \dot{-} x)$

(7) Min and max function :

min (x, y) = minimum of x and y

$\min(x, y) = x \dot{-} (x \dot{-} y)$

Similarly, max (x, y)

= maximum of x and y

$\max(x, y) = y + (x \dot{-} y)$

(8) The square function :

$f(y) = y^2 = U_1^1(y) * U_1^1(y)$

Q.45. Show that the function $f_1(x, y) = x + y$ is primitive recursive. ***CT : S-08, W-09(6M)***

OR Show that the addition of two integers x and y is primitive recursive. ***CS : S-11(4M)***

Ans.

- As add is not an initial function and there is no obvious way to obtain by composition, we try to obtain it by primitive recursion.

$\therefore f(x, y) = add(x, y)$

$add(x, y) = x + y \quad \ldots(1)$

$\therefore add(x, 0) = x + 0 = x$

$\therefore add(x, y+1) = x + y + 1$

$= x + y + 1$

$\therefore add(x, y+1) = add(x, y) + 1 \quad \ldots(2)$

- This equation show that add can be derived from primitive recursive function using primitive recursion. Hence addition of two integers is primitive recursive.
- To add 2 and 3, we apply above rules successively.

$add(3, 2) = add(3, 1) + 1$

$= (add(3, 0) + 1) + 1$

$= (3 + 1) + 1$

$= 4 + 1$

$= 5$

Q.46. Show that the function multiply from N × N to N, defined by the formula Mult (x, y) = x * y is primitive recursive.

CT : W-11(3M), W-12(3M), CS : W-09(2M)

Ans.

- To obtain Mult, primitive recursion is used.

$Mult(x, 0) = 0$

$Mult(x, k+1) = x * (k+1)$

$= Add(x * k, x)$

$= Add(x, Mult(x, k))$

- In the form h (x, k, Mult (x, k)) , x and Mult (x, k) are the first and third coordinates of the 3-tuple (x, k, Mult (x, k)), we use the function f = Add (P_1^3, P_3^3), obtained from Add, P_1^3 and P_3^3 and by composition.
- The function Mult is obtained from 0 (i.e. the initial function C_0^1) and f using the operation of primitive recursion. Therefore, Mult is also primitive recursive.

Q.47. Show that the function g (x, y) = x^y is primitive recursive.

CS : S-09(7M), W-11(2M), W-09, S-12(3M), CT : S-06(7M), S-07(5M), W-11(3M)

Ans.

- We have a function of two variable.

$g(x, y) = x^y$ --- given

$g(x, 0) = x^0 = 1 \quad \ldots(1)$

$g(x, y+1) = Mult(x, g(x, y))$

- Since g (x, y + 1) is defined using multiplication function which is primitive recursive function already shown in previous example. Hence g (x, y) is also primitive recursive.

To find g (2, 2) , we apply the rules sucessively.

$g(2, 2) = Mult(2, g, (2, 1))$

$= Mult(2, Mult(2, g(2, 0)))$

$= Mult(2, Mult(2, 1))$

$= Mult(2, add(2, Mult(2, 0)))$

= Mult (2, add (2, 0))
= Mult (2, 2)
= add (2, Mult (2, 1))
= add (2, add (2, Mult (2, 0)))
= add (2, add (2, 0))
= add (2, 2)
= (add (2, 1) + 1)
= ((add (2, 0) + 1) + 1
= (2 + 1) + 1
= 4.

Q.48.(a) Show that the function [x/2] which is equal to the greatest integer which is ≤ x / 2 is primitive recursive.

Ans.

- Now [0/2] = 0, [1/ 2] = 0, [2/2] = 1, [3/2] = 1 etc, so that [x/2] = x/2 when x is even and [x/2] = (x−1) /2 when x is odd.
- In order to distinguish between even and odd functions, we have already defined the parity function which is primitive recursive.
  [0/2] = 0 [(y + 1) / 2] = [y/2] + Pr (y)
  where Pr (y) denotes the parity function which is 1 when y is odd and which is 0 when y is even.
- Hence the given function is primitive recursive.

Q.48.(b) Prove that following functions are primitive recursive $f(x) = x^3$, $f(x, y) = x^4$.

CT : S-13(6M)

Ans. Note that $x^0 = 1$ for $x \neq 0$, and we put $x^0 = 0$ for $x = 0$. Also $x^{y+1} = x^y * x$;

Hence $F(x, y) = x^y$ is defined as

$F(x, 0) = sg(x)$

Now for the recursion equations

$f(x) = x^3$ & $f(x) = x^4$

We know that $f(x, y+1) = x * f(x, y)$

$= U_1^3(x, y, f(x,y)) * U_3^3(x, y, f(x, y))$

Since f(x, y + 1) is defined using multiplication function which is primitive recursive function. Hence, f(x, y) is also primitive recursive.

Also a function f has a primitive recursive derivative if there is a finite sequence of functions $f_0, f_1 \ldots f_j$ so that f_j = f and each function f_i in the sequence is either an initial function or obtainable from earlier functions in the sequence by composition or primitive recursion. This condition is satisfied by $f(x) = x^3$ and $f(x, y) = x^4$. Hence they are primitive recursive functions.

Q.49. Prove the following theorem :

Let f be a primitive recursive function of n variables. Then

(1) For any k ≥ 1, any function g of n + k variables obtained from f by introduction "dummy" variables (e.g. if n = 2 and k =1, g $(x_1 x_2 x_3) = f(x_1, (x_3))$ is also a primitive recursive.

(2) Any function g of n variables obtained from f by permuting variables (e.g. if n = 2, of $(x_1, x_2) = f(x_2, x_1)$) is primitive recursive.

(3) For any k with 1 ≤ k ≤ n, any function g of n − k variables obtained from f by substituting constants for k of the variables of f (e.g. if n = 2 and k = 1, g(x) = f (5, x)) is primitive recursive.

(4) For any k with 1 ≤ k ≤ n, any function g of n − k variables obtained from δ by repeating a variable k times (e.g. if n = 3 and k = 1, g $(x_1, x_2) = f(x_1, x_2, x_1)$) is primitive recursive.

Ans. Proof : These are all variants of the same general situation.

$g(x_1, x_2, \ldots\ldots x_r) = f(z_1, z_2, \ldots\ldots, z_n)$

where for each i, z_i is either $x_j = P_j^r$ $(x_1, \ldots\ldots x_r)$ for some j or $a = C_a^r(x_1, \ldots\ldots x_r)$ for some a.

For example : The function g shown in part 1 could be written as :

$g(x_1, x_2, x_3) = f(P_1^3(x_1, x_2, x_3),$

$P_3^3(x_1, x_2, x_3))$

In all four cases, the result is true because all the functions P_j^r and C_a^r are primitive recursive and g is obtained from f and some of these functions by composition.

Q.50. Show that the function f of two variables defined by $f(x,y) = y^x + x^x + x^7$ is primitive recursive.

Ans.

- For simplicity, consider a function f_1 defined by $f_1(x, y) = x^y$, we can write
  $f(x, 0) = 1$
  $f_1(x, k+1) = \text{Mult}(x, x^k)$
  $= \text{Mult}(x, f_1(x, k))$
- By the formulae h (x, y, z) = Mult (x, z) and by the theorem :
  For any $k \geq 1$, any function g of n + k variables obtained from f by introducing "dummy" variables (e.g. if n = 2 and k = 1, $g(x_1, x_2, x_3) = f(x_1, x_3)$) is also primitive recursive.
  f_1 is primitive recursive.
- Since, $y^x = f_1(y, x)$ it follow that the first term in the formula for f is primitive recursive, (∵ any function g of n variables obtained from f by permuting variables e.g. if n = 2, $g(x_1, x_2) = f(x_2, x_1)$) is primitive recursive.
- The second and third terms are primitive recursive functions of x because of

(i) For any k with $1 \leq k \leq n$, any function g of n – k variables obtained from f by substituting constant for k of the variables of f (e.g, if n = 2 and k = 1, g (x) = f (5, x)) is primitive recursive.

(ii) For any k with $1 \leq k \leq n$, any function g of n – k variables obtained from f by repeating a variable k times (e.g. if n = 3 and k = 1 $g(x_1, x_2) = f(x_1, x_2, x_1)$) is primitive recursive.

- Finally, since f (x, y) = Add (Add y^x, x^x), x^7), we can use the fact that composition preserves primitive recursiveness to conclude that f is primitive recursive.

Q.51. Show that "The subtraction function, with the modification that is necessary to guarantee that its values are always non-negative, is primitive recursive".

OR Show that the function given below is primitive recursive.

$$\text{Sub}(x,y) = \begin{cases} x - y & \text{if } x \geq y \\ 0 & \text{otherwise} \end{cases}$$

Ans.

- To show this, we begin with the function Pred (short for predecessor), defined by

$$\text{Pred}(x) = \begin{cases} 0 & \text{if } x = 0 \\ x - 1 & \text{if } x \geq 1 \end{cases}$$

The formulas
Pred (0) = 0
Pred (k +1) = k

- Together with theorem "for any k ≥ 1, any function g of n + k variable obtained from f by introducing "dummy variable" (e.g. if n = 2 and k = 1, $g(x_1, x_2, x_3) = f(x_1, x_3)$ is also primitive recursive if f be a primitive recursive function of n variables ".
- We show that Pred can be derived from primitive recursive functions using primitive recursion.
- If we define Sub by

$$\text{Sub}(x,y) = \begin{cases} x - y & \text{if } x \geq y \\ 0 & \text{otherwise} \end{cases}$$

then we can easily check the equations
Sub (x, 0) = x
Sub (x, k + 1) = Pred (Sub (x, k))
from which it follows that Sub is primitive recursive. This operation is often written as "– –" and is referred to as proper subtraction, or the minus operation.

Q.52. With suitable examples , explain "Every function in primitive recursive class is Turing computable". **CT : W-12(3M)**

Ans.

- We show the following three statements : Every initial function is a total computable function ; any function obtained from total computable functions by composition is also a total computable function; and any function obtained from total computable functions by primitive recursion is also a total computable function.
- For the sake of of simplicity, we show that if h : $\mathcal{N}^m \to \mathcal{N}$ is obtained from f : $\mathcal{N}^n \to \mathcal{N}$ and

g_1 and g_2, both functions from $\mathcal{N}^n$ to $\mathcal{N}$, by composition, and if these three functions are computable, then h is.

- The argument is valid even if all the functions are partial functions, and it extends in an obvious way to the more general case in which f is a function of k variables.
- Let T_f, T_1 and T_2 be TMs computing f, g_1, and g_2, respectively.
- We will construct a TM T_h to compute h. To simplify notation, we denote by X the m-tuple $(x_1, x_2, ...x_m)$ and by 1^X the string $1^{x_1} \Delta 1^{x_2} \Delta ..... \Delta 1^{x_m}$
- The TM T_h begins with tape contents $\underline{\Delta} 1^X$, and it must use this input twice, once to compute each of the g_i's.
- It does this by copying the input to produce the tape.

  $\Delta 1^X \underline{\Delta} 1^X$

  executing T_1 to produce $\Delta 1^X \underline{\Delta} 1^{g_1(X)}$, and then making another copy of the input and executing T_2, to obtain

  $\Delta 1^X \Delta 1^{g_1(X)} \underline{\Delta} 1^{g_2(X)}$
- At this point it deletes the original input and executes T_f on the input $1^{g_1(X)} \Delta 1^{g_2(X)}$, which produces the desired output.
- For any choice of X, T_h fails to halt during the execution of T_i of $g_i(X)$ is undefined, and fails to halt during the execution of T_f if both $g_i(X)$'s are defined but $f(g_1(X), g_2(X))$ is undefined. Therefore, T_h computes h.
- For the final step of the proof, suppose that $g: N^n \to N$ and $h: N^{n+2} \to N$ are computable and that f is obtained from g and h by primitive recursion.
- We let T_g and T_h be TMs computing g and h, respectively, and we construct a TM T_f to compute f.
- The original tape of T_f looks like this :

  $\underline{\Delta} 1^{x_1} \underline{\Delta} 1^{x_2} \Delta ....\underline{\Delta} 1^{x_n} \Delta 1^{x_{n+1}}$

  which we abbreviate $\underline{\Delta} 1^{x_n} \Delta 1^{x_{n+1}}$.
- As before, X denotes $(x_1, ...x_n)$.
- In the case in which $f(X, x_{n+1})$ is defined, T_f works as follows. First it determines whether x_{n+1} is not 0, and if it is, T_f returns the tape head to square 0, executes T_g, and halts.
- If x_{n+1} is not 0, the tape contents are $\Delta 1^X \Delta 1^{k+1}$ for some k, and in this case T_f computes f (X, 0), uses it to compute f (X, 1) uses that to compute f (X, 2)..... and ultimately uses f (X, k) to compute f (X, k + 1).
- It prepare to do this by creating the tape

  $\# 1^X \Delta 1^k \Delta 1^X \Delta 1^{k-1} \Delta 1^x \Delta ....$

  $\Delta 1^2 \Delta 1^x \Delta 1^1 \Delta 1^X \Delta 1^0 \underline{\Delta} 1^X$
- It executes T_g and moves the tape head, obtaining

  $\# 1^X \Delta 1^k \Delta 1^X \Delta .....\Delta 1^1 \underline{\Delta} 1^X \Delta 1^0 \Delta 1^{f(x,0)}$

  Here, we use the fact that f (X, 0) = g (X).
- At this point T_f begins a loop, during each iteration of which it executes T_h and moves the tape head.
- After the first iteration, since f (X, 1) = H (X, 0, f (X, 0)), the tape looks like this :

  $\# 1^X \Delta 1^k \Delta 1^X \Delta .....\underline{\Delta} 1^X \Delta 1^1 \Delta 1^{f(X,1)}$

  and T_f is ready to execute T_h again to compute f (X, 2). Continuous iterations finally produce

  $\# 1^X \Delta 1^k \Delta 1^{f(X,k)}$

  at which point changing # to Δ and executing T_h once more produces the desired result.

Q.53. Justify giving reason.

"There is a computable total function from N to N that is not primitive recursive".

Ans.

- We describe a function that cannot be primitive recursive and present an argument based on the church-turing thesis that it is computable.

- Let Σ be alphabet containing all the symbols we might need to describe numeric functions.
- Any primitive recursive function is specified by primitive recursive derivation that, in turn, can be specified by a string over Σ.
- The remainder of the proof depends on the following statement : There is an algorithm to determine, given some string in Σ^*, whether it represents a primitive recursive derivation of a function of one variable.
- The proof uses a diagonal argument. First, place an ordering on Σ, so that we may consider the canonical order on the set Σ^*.
- Consider the function $f : \mathcal{N} \to \mathcal{N}$. For each i, examine the string of Σ^* in order, discarding those that do not represent primitive recursive derivations of functions from N to N, until the i^{th} string is found that does represent such a derivation, let the function being derived be f_i , and let
  $f(i) = f_i(i) + 1$
- The function f is total. One the one hand, f is computable, because we have described an algorithm for finding the derivation of fi and the derivation constitutes an algorithm for computing f (i).
- On the other hand, f cannot be primitive recursive. Every primitive recursive function of one variable is fi for some i, however, no matter what i we choose, f is different from fi because f (i) $\neq$ f (i).

Q.54. Explain what is meant by primitive recursive predicates.

Ans. Primitive recursive predicates :

- A "condition" P depending on the variable $X \in N^v$, so that P(X) is either true or false, is called a predicate. More precisely, it is n-place predicate, which is simply a partial function from N^n to {true, false}.
- Closely associated with a predicate P is its characteristic function $X_P : N^n \to \{0, 1\}$, defined by $X_P(X) = \begin{cases} 1 & \text{if } P(X) \text{ is true} \\ 0 & \text{otherwise} \end{cases}$
- Since X_P is a numeric function, all the properties of functions are applicable to it and by association, to P.
- In particular, P is computable if X_P is and P is primitive recursive if X_P is.
- When we say that the characteristic function X_L of a language L is computable, we are saying that we can decide whether a given input is in L.
- When we say that X_P is computable, we are saying something similar : There is an algorithm to determine whether given X satisfies P or makes P(X) true.

Q.55. Prove the theorem :

If P_1 and P_2 are primitive recursive n-place predicates, then so are the predicates $P_1 \wedge P_2$, $P_1 \vee P_2$, and $\neg P_1$.

Ans. Proof : The result follows from these three equations,

$$XP_1 \wedge P_2 = XP_1 * XP_2$$

$$XP_1 \vee P_2 = XP_1 + XP_2 - XP_1 \wedge P_2$$

$$X(\neg P_1) = 1 - XP_1$$

[$\because$ "–" is a proper subtraction].

Q.56. The relational predicates are LT, EQ, GT, LE, GE, and NE. The expression LT (x, y) is true if x < y and false otherwise, and the definitions of the other five similarly suggested by their names.

Show that all these predicates are primitive recursive.

Ans.

- Consider the function Sg of one variable defined by $Sg(0) = 0 S_g(k+1) = 1$
- This function takes the value 0 if x = 0 and the value 1 otherwise, and its definition makes it clear that it is primitive recursive.
- Now we may write
  $X_{LT}(x, y) = Sg(y - x)$

since $x < y$ if and only if $y - x > 0$.

- This equation shows that X_{LT} is obtained from primitive recursive functions by composition and is therefore primitive recursive.
- The result for the equality predicate follows from the formula.

  $X_{EQ}(x, y) = 1 - (Sg(x - y) + Sg(y - x))$
- If $x < y$ or $x > y$, then one of the terms $x - y$ and $y - x$ is nonzero, and therefore the expression in parentheses is nonzero, causing the final result to be 0.
- If $x = y$, then the parenthesized expression is 0 because both terms are 0, and the final result is 1.
- Although the other four relational predicates can be handled in the same way, it is easier to use the formulas

  $LE = LT \vee EQ$

  $GT = \neg LT$

  $NE = \neg EQ$

  which together with previous theorem imply that all these predicates are primitive recursive.

Q.57. Consider the function

equals (x, y) = 1 if x = y

= 0 if x y

Show that this function is primitive recursive.

Ans.

- In order to show equals (x, y) is primitive recursive, we first introduce the function Sg of one variable defined by

  $Sg(0) = 0$

  $Sg(k+1) = 1$
- This function takes value 0 if $x = 0$ and the value 1 otherwise. Its definition makes it clear that it is primitive recursive.
- Consider the relational predicate LT (Less than). The expression LT (x, y) is true if $x < y$ and false otherwise.
- Now we may write

  $LT(x, y) = Sg(y - x)$

  Since $x < y$ if and only if $y - x > 0$. This equation show that LT (x, y) is obtained from primitive recursive functions by composition and is therefore primitive recursive.
- Now we may write equals $(x, y) = 1 - (Sg(x - y) + Sg(y - x))$ ....(1)
- Note that if $x < y$ and $y > x$, then one of the terms $x - y$ and $y - x$ is nonzero, and therefore the expression in parenthesis is nonzero causing the final result to be 0.
- If $x = y$, then parenthesized expression is 0 because both terms are 0, and the final result is 1.
- From equation 1 it is clear that equals (x, y) is obtained from primitive recursive functions by composition and is therefore primitive recursive.

OR

- We introduce zero test function which is primitive recursive function.

  $\bar{Sg}(0) = 1$

  $\bar{Sg}(y + 1) = 0$
- Now we define the equals (x, y) in terms of zero test function.

  $equals(x, y) = \bar{Sg}\,|x - y|$
- Since $\bar{Sg}\,|x - y|$ is primitive recursive function.

  Also equals (x, y) = 1 for x = y and otherwise equals (x, y) = 0.
- Thus equals (x, y) = $\bar{Sg}\,|x - y|$ is the required characteristic function.
- We know that the function is said to be recursive if its characteristic function is primitive recursive. Hence equals (x, y) is primitive recursive.

Q.58. Show that the function given below is primitive recursive.

$$f(x) = \begin{cases} x^2 & \text{if } x \text{ is even} \\ x + 1 & x \text{ is odd} \end{cases}$$

Ans.

- The function f is said to be primitive recursive if its characteristic function is primitive recursive.
- The corresponding predicate is also called primitive recursive.
- The given function f (x) can be expressed as

$$f(x) = \begin{cases} P_1^1(x) * P_1^1(x) & \text{if } x \text{ is even} \\ S(x) & \text{if } x \text{ is odd} \end{cases}$$

  where S(x) is successor function $S(x) = x + 1$.
- Since the given function is constructed from the basic function as projection function and

successor function by successive composition. Hence it is primitive recursive.

To find f (2), f (3)

$\therefore$ f (2) = P_1^1 (2) × P_1^1(2)

= 2 * 2

= 4

$\therefore$ f (3) = S (3)

= 3 + 1

= 4

Q.59. Show that the function f (x_1, x_2, y) defined as

$$h(x_1, x_2, y) = \begin{cases} x_2 & x_1 > y \\ (x_1 * y) + x_2 & x_1 \le y \end{cases}$$

is primitive recursive.

Ans. The required function can be expressed as

$f(x_1, x_2, y) = x_1 + (x_1 * y) * \overline{Sg}(x_1 \div y)$

Since zero test function is already a primitive recursive function. The $f(x_1, x_2, y)$ is also primitive recursive.

Q.60. Show that Ackermann's function is not primitive recursive. ***CS : S-10(6M)***

Ans.

- Consider the function
  g(i) = A (i, i).
  If A were primitive recursive, then so would g.
- But then, according to theorem.
  "Let f be any primitive recursive function. Then there exists some integer n such that
  f (i) < A (n, i)
  for all i = n, n + 1 .....".
- Hence there exists n such that
  g (i) < A (n, i) for all i.
- If we pick i = n, we get the contradiction
  g (n) = A (n, n)
  < A (n, n),
  proving that A cannot be primitive recursive.

Q.61. Show that the following function are primitive recursive.

(i) Constant function a and b

(ii) Identity function

(iii) Concatenation

(iv) Transpose

(v) Head function

(vi) Tail function

(vii) The conditional function

"if $X_1 \neq \wedge$ then X_2 else X_3".

OR What do you mean by primitive recursive function over a and b? ***CT : W-07(3M)***

Ans.

(i) Constant function a and b :

First we introduce Nil function which is primitive recursive.

nil (x) = $\in$

For any constant function a we have

a (x) = a

Thus, a (x) = constant a (nil (x)).

Thus, the function a (x) is the composition of the initial function constant and the initial function nil and hence it is primitive recursive.

For any constant function b

b (x) = b

Thus, b (x) = constant b (nil (x))

Thus, the function b (x) is the composition of the initial function constant and function nil, hence it is primitive recursive.

(ii) Identity function :

Identity function is defined as follows :

identity ($\in$) = $\in$

identity (a, x) = constant a (x)

identity (b, x) = constant b (x)

Since function identity is defined by using the initial function constant, thus it is a primitive recursive.

(iii) Concatenation :

The concatenation function is defined as follows :

concat $(x_1, x_2) = x_1 x_2$

concat $(\in, x_2)$ = identity (x_2)

concat (ax_1, x_2) = constant a (concat (x_1, x_2))

concat (bx_1, x_2) = constant b (concat (x_1, x_2))

Since , concatenation function is defined by identity and constant function which are primitive recursive. Thus concatenation function is primitive recursive.

(iv) Transpose :

Any transpose function is defined as follows

transpose ($\in$) = $\in$

transpose (a x) = concat (transpose (x), a (x))

transpose (b x) = concat (trans (x), b (x))

Since, transpose function is defined using concatenation function which is primitive recursive. Thus transpose function is also primitive recursive.

(v) Head function :

The head function is defined as follows

head ($\in$) = $\in$

head (a x) = constant a (x)

head (b x) = constant b (x)

The head function is defined by using the constant function which is primitive recursive. Hence the head function is primitive recursive.

(vi) Tail function :

Tail function is defined as follows

tail ($\in$) = $\in$

tail (a x) = identity (x).

tail (b x) = identity (x).

Since, the above tail function is defined by using identity function which is primitive recursive. Hence tail function is primitive recursive.

(vii) The conditional function "if $X_1 \neq \wedge$ then X_2 else X_3" :

$X : X_1 = '\wedge'$ Zero function

$X : X_2 = X + 1$ Successor function

$X : X_3 = (X_1 + 1 - - - - - X_n)$ X Projection function.

The projection function is also called the generalized identity function.

A function X is called primitive recursive if it can be obtained from the initial functions by a finite number of operations of composition and recursion.

Q.62. Theorem - Prove that :

If P is primitive recursive (n + 1) place predicate, both the predicates E_P and A_P are also primitive recursive.

Ans. Proof :

- In order to simplify the proof, it is useful to introduce two other "bounded operations".
- The factorial function is defined by

$$x! = \prod_{i=1}^{x} i$$

- If this notation is unfamiliar, in general, $\prod_{i=j}^{k} p_i$ stands for the product $p_j * p_{j+1} \; p_k$ if $k \geq j$, and for 1 if $k < j$.
- We can generalize this definition by allowing the factors to be more general than I, in particular, to involve other variables, and by allowing sums as well as products.
- By definition of bounded universal quantification, A_P (X, k) is true if and only if P (X, i) is true for every i with $0 \leq i \leq k$.
- Therefore, X_{A_P} (X, k) = 1 if and only if all the terms X_P (X, i) are also 1.
- This equivalence implies that

$$X_{A_P}(X, k) = \prod_{i=0}^{k} X_P (X, i)$$

- Saying that there is an i so that $0 \leq i \leq k$ and P (X, i) is true, is the same as saying that P (X, i) is not always false for these i's.
- In other words,

$$E_P(X,k) = \neg A \neg_P (X,k)$$

It follows from this formula that E_P is also primitive recursive.

Q.63. Theorem - Prove that :

Let $n \geq 0$, and suppose that $g : N^{n+1} \rightarrow N$ is primitive recursive. Then the functions $f_1, f_2 : N^{n+1}$ N defined below are also primitive recursive :

$$f_1 (X,k) = \sum_{i=0}^{k} g (X, i)$$

$$f_2 (X,k) = \sum_{i=0}^{k} g (X, i)$$

for any $X \in N^n$ and $k \geq 0$. (The functions f_1 and f_2 are said to be obtained from g by

bounded sums and bounded products, respectively.)

Ans. Proof : We give the proof for f1, and the other is almost identical. We may write

$f_1(X, 0) = g(X, 0)$

$f_1(X, k+1) = f_1(X, k) + g(X, k+1)$

Therefore, f_1 is obtained by primitive recursion from the two primitive recursive functions g_1 and h.

where $g_1(X) = g(X, 0)$, and

$h(X, y, z) = z + g(X, y+1)$

Q.64. Let $\phi\lfloor\sqrt{x}\rfloor$ be the greatest integer $\leq\sqrt{x}$. Show that $\lfloor\sqrt{x}\rfloor$ is primitive recursive.

Ans.

- Observe that $(y+1)^2 x$ is zero for $(y+1)^2 x$ and non zero $(y+1)^2 > x$.
- Therefore, $\overline{Sg}\ (y+1)^2 \dot{-} x)$ is 1 if $(y+1)^2 \leq x$ and cannot be equal to 0.
- The smallest value of y for which $(y+1)^2 > x$ is the required number ; hence

  $\lfloor\sqrt{x}\rfloor = \mu_y(\overline{sg}\,(y+1)^2 \dot{-} x) = 0$

  Note that $\lfloor\sqrt{x}\rfloor$ is defined for all x and hence is a recursive function.

Q.65. Prove that the function $Move_T$: N is primitive recursive.

Ans. Proof :

- We may write

  $Move_T(m)$

$$= \begin{cases} g_n\,(\text{NewState}(m), \text{NewPosn}(m), \\ \quad \text{newTapeNum}(m) \\ \quad\quad \text{if m is a configuration number} \end{cases}$$

- All these functions NewState, NewPosn, and NewTapeNum have the value 0 at any point m that is not a configuration number.
- For a configuration number m, NewState(m) is the resulting state if T can move from configuration m, and State(m) otherwise, the other two functions are defined similarly.
- Thus to show that $Move_T$ is primitive recursive, it is sufficient to show that these three New functions are primitive recursive.
- In the argument it helps to have one more function, New Symbol, defined analogously.
- So far, our description of New State(m) has involved three cases. One case corresponds to the primitive recursive predicate $\neg$ Is $Config_T$. The other two cases may be divided into subcases, corresponding to the various possible combinations of State(m) and Symbol(m).
- Because these two functions are primitive recursive, so are the predicates defining the subcases.
- In each subcase, the value of NewState(m) is either m or the value specified by the transition table for T.
- Therefore, since NewState is defined by cases that involve only primitive recursive functions and predicates, it must also be primitive recursive. The argument to show that NewSymbol is primitive recursive is exactly the same.
- The proof for New Posn is almost the same. This function may also be defined by cases, the same ones involved in the definition of New State.
- In each case, New Posn(m) is either 0, if m is not configuration number, Posn(m), if T cannot move from configuration m, or if the move does not change the position of the tape head ; Posn(m) + 1, if the move shifts the head to the right ; or Posn(m) −1, if the move shifts the head to the left. Therefore, NewPosn is primitive recursive.
- The definition of NewTapeNum can also be made using the same cases, with slightly more complicated formulas.
- Suppose that Posn(m) = i, Symbol(m) = j, and NewSymbol (m) = j'. The difference between TapeNum(m) and NewTapeNum(m) is that the first number involves the factor PrNo(i) and the

second has $PrNo(i)^j$ instead ; the exponents differ by $j - j'$ = NewSymbol (m) – Symbol (m).

- Thus in this subcase, NewTapeNum(m) can be expressed as

$$TapeNum(m) * PrNo(Posn(m))^{NewSymbol(m)-Symbol(m)}$$

if NewSymbol(m) ≥ Symbol(m), and Div(TapeNum(m), $PrNo(Posn(m))^{NewSymbol(m)-Symbol(m)}$) otherwise. Since both formulas define primitive recursive functions, the function NewTapeNum is primitive recursive.

Q.66. Show that following function are primitive recursive :

(i) equal (m, n) = 1 if m = n
= 0 otherwise.

(ii) abs (x, y) = | x – y| ***CS : S-13(3M)***

Ans.

(i) equal (m, n) = 1 ÷ sign (m ÷ n) + sign (n ÷ m)

Since ÷ and sign are primitive recursive functions equal (m, n) is also primitive recursive.

Sign function is defined as follows :

Sign (m) = 0, if m = 0
= 1, if m > 0

It can be proved that sign functions is primitive recursive as follows :

Sign (0) = 0 = z (n).

Sign (n + 1) = 1 = s (z (n)).

(ii) | x – y| = (x ÷ y) + (y ÷ x)

where ÷ represents the proper subtraction, which is primitive recursive function.

Q.67. Show that MAX (a, b) MIN (a,b) is PR.

CS : W-11,12(2M)

Ans. Min (a, b) can be expressed as,

min (a, b) = x – (x – y) .....(1)

The function (x – y) is called proper subtraction function defined by

x – y = x . y if x ≥ y

x – 0 = x and

x – (y + 1) =p (x – y) .....(2)

This function is defined by recursive using the PRF.

Hence from (1) and (2)

min (a, b) is PRF.

MAX (a, b) can be expressed as,

MAX (a, b) = a + (a – b)

The function (a, b) is called as proper subtraction again defined as above.

Hence MAX (a, b) is PRF.

ACKERMANN FUNCTION

Q.68. Define Ackermann's function. Compute A (1, 1), A (2, 1) and A (2, 2) .

CS : S-09(5M), W-09(3M), S-11(4M), CT : S-07,08,12(6M), S-13(7M), W-10(8M), W-12(5M)

Ans. For m, n = 0, 1, 2, ...Ackermann function is defined as

(1) A (m, n) = n + 1, m = 0,
i.e A (0, n) = n + 1

(2) A (m, n) = A (m – 1, 1) m = 0, n = 0
i.e. A (m , 0)
= A (m – 1, 1)

(3) A(m , n) = A ((m –1), A (m, n –1))

Base values are the pairs (0, 1), (0, 1) (0, 2) (0, 3) ..... (0, n) .....

To find A (2, 1) and A (2, 2) we have

A (2, 2) = A (1, A (2, 1)) from (3) .....(a)

A (2, 1) = A (1, A (2, 0)) .....(b)

A (2, 0) = A (1, 1) from (2) .....(c)

A (1, 1) = A (0, A (1, 0)) from (3) .....(d)

A (1, 0) = A (1, 0) from (2) .....(e)

A (0, 1) = 2 from (1) ....(f)

∴ A (1, 0) = 2, from (e) ....(g)

A (1, 1) = A(0, A (1, 0)) = A (0, 2) from (g) ....(h)

= 3 from (1)

∴ A (2, 0) = A (1, 1) from (c) = 3 from (h) ....(i)

∴ A (2, 1) = A (1, 3) from (b) and (i)
= A (0, A, (1, 2)) by (3)
= A (0, A (0, A (1, 1))) by (3)
= A (0,A (0, 3)) from (h)

= A (0, 4) from (1)

= 5 from (1) ....(j)

A (2, 2) = A (1, A (2, 1)) from (1)

= A (1, 5) from (j)

= A (0, A (1, 4)) from (3)

= (A (0, A (0, A (1, 3))

= A (0, A (0, A (0, A (1, 2))))

= A (0, A (A (0, A (0, A (1, 1)))))

= A (0, a (0, A (0, A (0, 3)))) from (h)

= A (0, A (0, A (0, 4))) by 1

= A (0, A (0, 5)) by 1

= A (0, 6) = 7 by 1.

Q.69. Prove the following for Ackermann function :

A (2, y) = 2y + 3. **CS : S-12(4M)**

Ans. We can prove this with the help of induction.

(a) Basis : When y = 0 then given function will be

A (2, y) = A (2, 0)

= 2 × 0 + 3 = 3

L.H.S. = A(2, 0)

= A(1, 1) *[∵ A (x, 0) = A (x – 1, 1)]*

= A (0, A(1, 0))

[∵ A(x, y + 1) = A (x – 1, A (x, y))]

= A (1, 0) + 1 *[∵ A (0, y) = y + 1]*

= 2 + 1 *[∵ A (1, 0) = 2 given in problem]*

= 3

= R.H.S.

Hence basis is true.

(b) Hypothesis : Let A (2, y) = 2y + 3 is true for y = 1, 2, 3.....n – 1.

i.e. we have A (2, y) = 2y + 3 for y < n

(c) Inductive Step : We want to prove now for y = n.

A (2, n) = A (1, A(2, n – 1))

[∵ A (x, y + 1) = A (x – 1, A (x, y))]

= A (1, 2n + 1)) *[By using hypothesis]*

[∵ A (2, n – 1) = 2 (n – 1) + 3 = 2n – 2 + 3 = 2n + 1]

= 2n + 1 + 3 *[By using first proof]*

= 2n + 3 *[By using first proof]*

= R.H.S.

Hence Proved.

POINTS TO REMEMBER :

(1) A language is said to be recursively enumerable if there exists a turing machine M that accepts L such that for every $u \in L$.

$q_0\, u \rightarrow Mx_1\, q_f\, x_2$

where q_f = final state

q_0 initial state.

(2) A language L is said to be recursive if there exist a turing machine M that accepts L and that halts on every $u \in \Sigma^+$.

(3) Properties of recursively enumerable (i.e., r.e.) languages can be classified as trivial or non-trivial properties.

(4) A property is trivial if either no r.e. languages has that property, or all r.e. languages have that property.

(5) A property is said to be non-trivial if some r.e. languages have that property, but not all r.e. languages have that property.

(6) Emptiness, regularity, context freedom are some of the non-trivial properties of the r.e. languages.

(7) If the language is recursive then the problem is decidable. Whereas if the language is not recursive, then the problem is undecidable.

(8) It is also possible to prove that a given problem is undecidable, by reducing an already known undecidable problem to a given problem.

(9) *Reduction of a problem A to problem B refers to showing that it is possible to realize an algorithm for A, if exist some algorithm for problem B.*

(10) *Undecidability refers to getting an answer to a question, whether there exist an algorithm to solve a given problem.*

(11) *Recursive function theory gives us another approach for formalizing the notion of computation.*

(12) *Zero function, Successor function and Projection function are considered to be the initial or basic functions.*

(13) *A class of functions called as μ-recursive functions constitutes a class of computable functions.*

(14) *A function f is said to be primitive recursive, if it can be obtained from the initial or basic functions by a finite number of operations of composition and recursion.*

(15) *A function is μ-recursive, if and only if it can be obtained from the initial or basic functions by applying finite number of operations of composition, primitive recursive and unbounded minimalization to regular functions.*

WINTER EXAMINATION - 2013
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	15 M
Unit - II	16 M
Unit - III	41 M
Unit - IV	13 M
Unit - V	28 M
Unit - VI	17 M
Questions covered in VBD	130 M (100%)

WINTER EXAM.-2013
QUESTION PAPER

SECTION - A

Q.1.(a) Prove the following by principle of induction :

(i) $1 + 4 + 7 + ... + (3n - 2) = \frac{n(3n-1)}{2}$.

(ii) $2^n > n$ for all $n > 1$. **5M**

(b) Discuss the Chomsky Hierachy of language. Identify the type of following grammar :

$AB \rightarrow CDB$

$AB \rightarrow CdEB$

$ABcd \rightarrow abCD / Bcd$

$B \rightarrow b$. **6M**

(c) Construct DFA over alphabet – {a, b} recognizing the set all strings that began with 'a' but do not contain 'aab' as a substring. **3M**

Q.2.(a) Construct Minimum state automata for the finite automata given below :

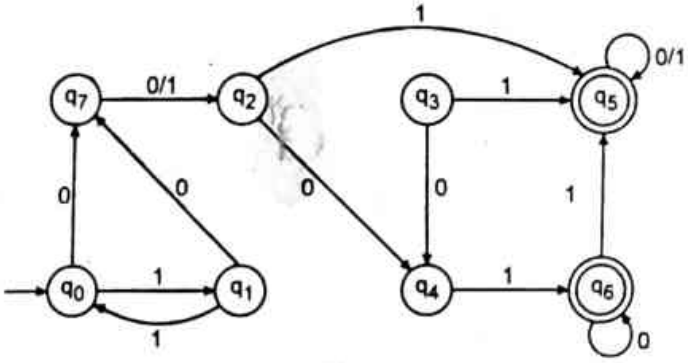


Fig.

7M

(b) Design a Moore machine to find out how many times symbol 'b' occurs in a long i/p string of a's and b's . (No of b^s is equal to number of 1^s in output string). **6M**

Q.3.(a) Find Regular expression for the following automata :

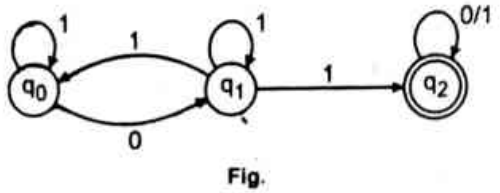


Fig.

6M

(b) Construct a DFA for the regular expression. $ab\,(a + ab)^* (a + aa)$. **7M**

Q.4.(a) Find Regular expression corresponding to each of following subsets of E = {0, 1}.

(i) The language of all strings containing no more than one occurrence of string 00. **2M**

(ii) The language of all strings in which every 0 is followed by 11. **2M**

(b) Prove that the language :

$L = \{(ab)^n a^k \mid n > k; k \geq 0\}$ is not regular. **5M**

(c) Consider the CFG

$S \rightarrow XY$

$X \rightarrow aX \mid bX \mid \in$

$Y \rightarrow bbb$

Prove that this generates the language of all strings $(a + b)^*$ bbb. **6M**

Q.5.(a) Find the equivalent Chomky normal forms for the following grammar :

$S \rightarrow A \mid B \mid C$

$A \rightarrow aAa \mid B$

$C \rightarrow aCaa \mid D$

$D \rightarrow baD \mid abD \mid aa$. **6M**

(b) Write CFG grammar for

(i) Set of all strings over an alphabet {a, b}, with exactly twice as many a's as b's .

(ii) $L(G) = \{a^i b^j c^j \mid i, j \geq 0\}$. **7M**

SECTION - B

Q.6.(a) Construct a PDA for the language

$L = \{x \in (a, b)^* \mid n_a(x) = n_b(x)\}$. **7M**

(b) Explain the closure properties of context free language. **6M**

Q.7.(a) Design a Turing machine which subtracts two unary numbers i.e. m – n where $m \geq n$. Draw the transition graph for it. **8M**

(b) Explain the various types of Turing machine with suitable diagram. **5M**

Q.8.(a) What is the difference between Recursive language and a recursively enumerable language? **3M**

(b) Construct a TM that accepts

$L = \{0^{2n} \mid n \geq 0\}$. **5M**

(c) If L is a recursive language over Σ show that τ (τ is defined as $\Sigma \forall _ L$) is also recursive. **5M**

Q.9.(a) Explain the post correspondence problem. Consider the following two lists.

$L_1 : a^2, b^2, ab^2$

$L_2 : a^2b, ba, b$

Does this distance of PCP have the solution ? If yes, then find the solution. **6M**

(b) Show that $f(x, y) = x^2y^4 + 7xy^3 + 4y^5$ is primitive recursive. **4M**

(c) Explain the halting problem of Turing machine. **4M**

Q.10.(a) Write short notes on :

(i) Church Turing Thesis.

(ii) Linear Bounded automata.

(iii) Decidability and Decidable language.

(iv) Countability and Diagonalization. **13M**

SUMMER EXAM.-2014

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 15)

Q.1.(a) Prove the following by principle of induction :

(i) $1 + 4 + 7 + ... + (3n - 2) = \frac{n(3n-1)}{2}$.

(ii) $2^n > n$ **for all** $n > 1$. **5M**

Ans.

(i) $1 + 4 + 7 + ... + (3n - 2) = \frac{n(3n-1)}{2}$

(1) Basic step :

For n = 1, we have

L.H.S. = (3n – 2)

= 1

$\text{R.H.S.} = \frac{n(3n-1)}{2} = \frac{2}{2}$

= 1

$\therefore$ L.H.S. = R.H.S.

(2) Induction hypothesis :

Assume that for some $n \geq 1$

$1 + 4 + 7 + ... + (3n - 2) = \frac{n(3n-1)}{2}$

For every natural no. upto and including n.

(3) Induction step :

For n = n + 1

L.H.S = $1 + 4 + 7 + ... + (3n - 2) + (3n + 1 - 2)$

$= \frac{n(3n-1)}{2} + (3n - 1)$

$= \frac{n(3n-1) + 2(3n-1)}{2}$

$= \frac{(n+2)(3n-1)}{2}$

Now,

$\text{R.H.S} = \frac{(n+1)(3n+1-1)}{2}$

$= \frac{(n+1)(3n)}{2}$

$= \frac{(n+2)(3n-1)}{2}$

$\therefore$ L.H.S = R.H.S.

(ii) $2^n > n$ for all $n > 1$

(1) Induction hypothesis :

Assume for some $n > 1$

$\therefore \quad n = z$

$\therefore \quad 2^2 > 2$

$\therefore \quad 4 > 2$

(2) Induction step :

For n + 1 given equation is

$2^{n+1} > n + 1$

For $n > 1$ and onwards.

By principal of mathematical induction statement $2^n > n$ for all $n > 1$ is true for all natural number is n.

Q.1.(b) Discuss the Chomsky Hierachy of language. Identify the type of following grammar :

AB → CDB

AB → CdEB

ABcd → abCD/Bcd

B → b. **6M**

Ans. See P.1-11, Q.22.

(i) AB → CDB :

Type 1

(ii) AB → CdEB :

Type 1

(iii) ABcd → abCD / Bcd :

Type 0

(iv) B → b.

Type-3

Q.10.(a)(iv) Write a short note on Countability and Diagonalization. **4M**

Ans. See P.1-5, Q.5.

UNIT - II

(Total Marks : 16)

Q.1.(c) Construct DFA over alphabet – {1, b} recognizing the set all strings that began with 'a' but do not contain 'aab' as a substring. **3M**

Ans. DFA

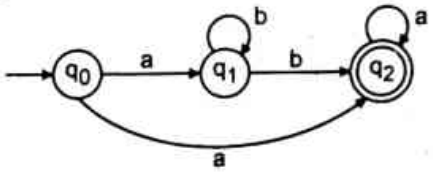


Fig.

Q.2.(a) Construct Minimum state automata for the finite automata given below :

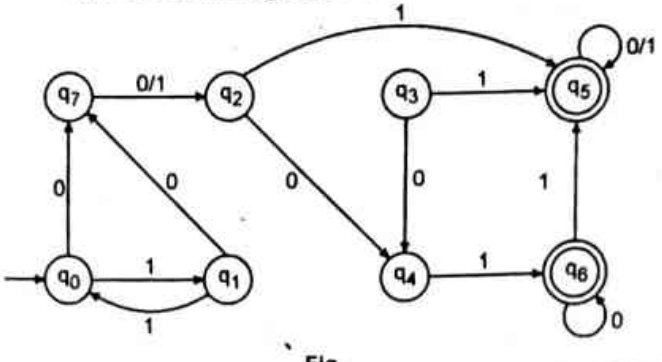


Fig.

7M

Ans.

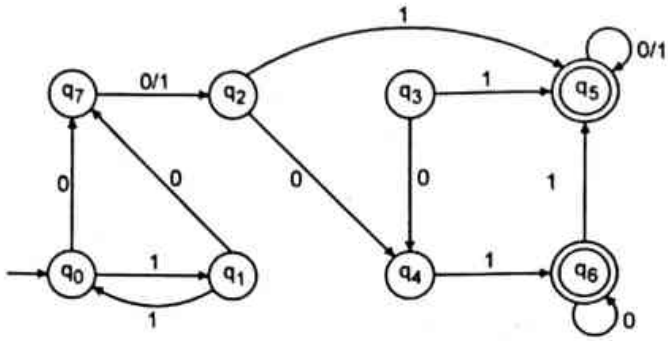


Fig.

Transition table for DFA :

State \ Σ	0	1
q_0	q_7	q_1
q_1	q_7	q_0
q_2	q_4	q_5
q_3	q_4	q_5
q_4	-	q_6
* q_5	q_5	q_5
* q_6	q_6	q_5
q_7	q_2	q_2

Group 1 = $\{q_0, q_1, q_2, q_3, q_4, q_7\}$

Group 2 = $\{q_5, q_6\}$

We consider group 1

∴ Pair from group 1 are

$\{q_0, q_1\}, \{q_0, q_2\}, \{q_0, q_3\}, \{q_0, q_4\}$

$\{q_0, q_7\}$

∴ **Step 1 :**

(i) $\{q_0, q_1\}$

	0	1
q_0	q_7	q_1
q_1	q_7	q_0

∴ $q_0 = q_1$

Same group

(ii) $\{q_0, q_2\}$

	0	1
q_0	q_7	q_1
q_2	q_4	q_5

∴ $q_0 \neq q_2$

Diff. group

VBD

(iii) $\{q_0, q_3\}$

	0	1
q_0	q_7	q_1
q_3	q_4	q_5

Diff. group

$\therefore\ q_0 \neq q_3$

(iv) $\{q_0, q_4\}$

	0	1
q_0	q_7	q_1
q_4		q_6

Diff. group

$\therefore\ q_0 \neq q_4$

(v) $\{q_0, q_7\}$

	0	1
q_0	q_7	q_1
q_7	q_2	q_2

Same group

$\therefore\ q_0 = q_7$

$\therefore$ So, new groups are

$\{q_0, q_1, q_7\}\ \{q_2, q_3, q_4\}\ \{q_5, q_6\}$

Step 2 :

Consider $\{q_0, q_1, q_7\}$

Pairs are $\{q_0, q_1\}\ \{q_1, q_7\}$

(i) $\{q_0, q_1\}$

	0	1
q_0	q_7	q_1
q_1	q_7	q_0

same group

$\therefore\ q_0 = q_1$

(ii) $\{q_0, q_7\}$

	0	1
q_0	q_7	q_1
q_7	q_2	q_2

Diff. group

$\therefore\ q_0 \neq q_7$

$\therefore$ So, new groups are

$\{q_0, q_1\}\ \{q_7\}\ \{q_2, q_3, q_4\}\ \{q_5, q_6\}$

Step 3 :

Consider $\{q_2, q_3, q_4\}$

Pairs are

$\{q_2, q_3\}\ \{q_2, q_4\}$

(i) $\{q_2, q_3\}$

	0	1
q_2	q_4	q_5
q_3	q_4	q_5

Same group

$\therefore\ q_2 = q_3$

(ii) $\{q_2, q_4\}$

	0	1
q_2	q_4	q_5
q_4	-	q_6

Same group

$\therefore\ q_2 = q_4$

Final combination of groups are :

$\{q_7\}\ \{q_0, q_1\}\ \{q_2, q_3, q_4\}\ \{q_5, q_6\}$

Transition table :

State \ Σ	0	1
$\{q_0\ q_1\}$	q_7	$\{q_0\ q_1\}$
$\{q_2\ q_3\ q_4\}$	$\{q_2\ q_3\ q_4\}$	$\{q_5\ q_6\}$
* $\{q_5\ q_6\}$	$\{q_5\ q_6\}$	$\{q_5\ q_6\}$
$\{q_7\}$	$\{q_2\ q_3\ q_4\}$	$\{q_2\ q_3\ q_4\}$

Transition Graph :

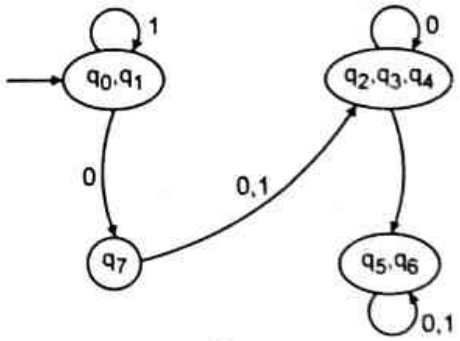


Fig.

Q.2.(b) Design a Moore machine to find out how many times symbol 'b' occurs in a long i/p string of a's and b's . (No of b^s is equal to number of 1^s in output string). **6M**

Ans. Let assume the output alphabets

$\Delta = \{y, n\}$

- y indicates that b is occurred in the string and n indicates that be is not occurred if a is occurred.
- Then calculate number of y occurred will give the count of no. of b's in output.

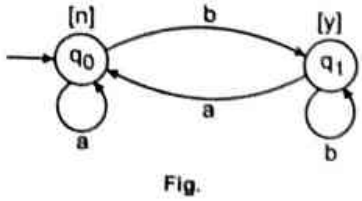


Fig.

Transition table for, moore machine

Present state	m = a	m = b	Output
→ q_0	q_0	q_1	n
q_1	q_0	q_1	y

Output table :

State	Output (Δ)
q_0	n
q_1	y

UNIT - III

(Total Marks : 41)

Q.3.(a) Find Regular expression for the following automata :

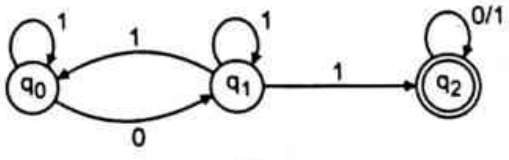


Fig.

6M

Ans.

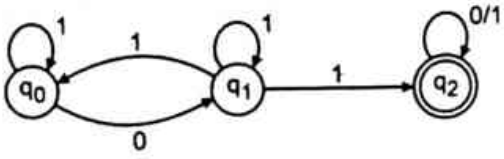


Fig.

Equation for all states :

$q_0 = q_1 1 + \epsilon$ ... (1)

$q_1 = q_0 0 + q_1 1$ ... (2)

$q_2 = q_2 0 + q_2 1 + q_1 1$ ... (3)

$q_2 = q_2 (0 + 1) + q_1 1$

By Arden's Theorem,

$q_2 + q_1 1 (0 + 1)^*$ ... (4)

From (2)

$q_1 = (q_1 1 + \epsilon) 0 + q_1 1$

$= q_1 10 + 0 + q_1 1$

$q_1 = q_1 (10 + 1) + 0$

By Arden's Theorem,

$q_1 = 0 (10 + 1)^*$ ... (5)

∴ From (4)

$q_2 = 0 (10 + 1)^* (1 (0 + 1)^*)$

∴ Regular expression = $0 (10 + 1)^* (1 (0 + 1)^*)$

Q.3.(b) Construct a DFA for the regular expression, ab (a + ab) * (a + aa) . **7M**

Ans. Transition graph for NFA :

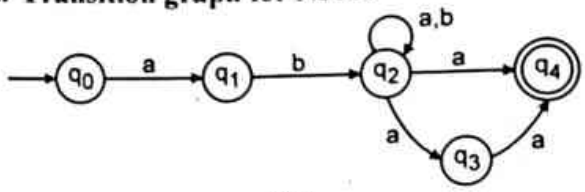


Fig.

Transition table :

Σ / State	a	b
q_0	q_1	-
q_1	-	q_2
q_2	$q_2\ q_4\ q_3$	q_2
q_3	q_4	-
* q_4	-	-

(1) $\left.\begin{matrix}\delta'(q_0, a) = q_1 \\ \delta'(q_0, b) = \phi\end{matrix}\right\}$ A

(2) $\left.\begin{matrix}\delta'(q_1, a) = \phi \\ \delta'(q_1, b) = q_2\end{matrix}\right\}$ B

(3) $\left.\begin{matrix}\delta'(q_2, a) = q_2\ q_3\ q_4 \\ \delta'(q_2, b) = q_2\end{matrix}\right\}$ C

(4) $\left.\begin{matrix}\delta'(q_2\ q_3\ q_4, a) = q_2\ q_3\ q_4 \\ \delta'(q_2\ q_3\ q_4, b) = q_2\end{matrix}\right\}$ *D

Transition table :

Σ / State	a	b
→ A	B	ϕ
B	ϕ	C
C	D*	B
*D	D*	B

Transition graph for DFA :

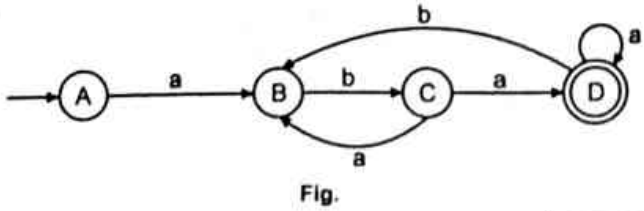


Fig.

VBD

Q.4.(a) Find Regular expression corresponding to each of following subsets of E = {0, 1}.

(i) The language of all strings containing no more than one occurrence of string 00. **2M**

(ii) The language of all strings in which every 0 is followed by 11. **2M**

Ans. (i) See P.3-20, Q.24(2).

(ii) See P.3-12, Q.5.

Q.4.(b) Prove that the language :

$L = \{(ab)^n a^k \mid n > k; k \geq 0\}$ is not regular. **5M**

Ans. See P.3-58, Q.94.

Q.4.(c) Consider the CFG

$S \rightarrow XY$

$X \rightarrow aX \mid bX \mid \in$

$Y \rightarrow bbb$

Prove that this generates the language of all strings (a + b) * bbb. **6M**

Ans. Valid strings generated by R.E. (a + b) * bbb arcs. {abbb, bbbb, aabbb, bbbbb, bbb}

$\therefore$ (1) $S \rightarrow XY$

$\rightarrow aXY$

$\rightarrow a \in bbb$

$\rightarrow abbb$

(2) $S \rightarrow XY$

$\rightarrow bXY$

$\rightarrow bbXY$

$\rightarrow bb \leftarrow bbb$

$\rightarrow bbbbb$

(3) $S \rightarrow XY$

$\rightarrow aXY$

$\rightarrow aaXY$

$\rightarrow aa \in bbb$

$\rightarrow aabbb$

CFG generate the language of all strings (a + b) * bbb.

Q.5.(a) Find the equivalent Chomky normal forms for the following grammar :

$S \rightarrow A \mid B \mid C$

$A \rightarrow aAa \mid B$

$C \rightarrow aCaa \mid D$

$D \rightarrow baD \mid abD \mid aa$. **6M**

Ans. Remove unit production and useless symbol from grammar B is useless symbol.

$\therefore S \rightarrow aAa \mid aCaa \mid baD \mid abD \mid aa$

$A \rightarrow aAa$

$C \rightarrow aCaa \mid baD \mid abD \mid aa$

$D \rightarrow baD \mid abD \mid aa$

Add new production

$E \rightarrow a$

$F \rightarrow b$

$S \rightarrow aAE \mid aCD \mid bED \mid aFD \mid aE$

$A \rightarrow aAE$

$C \rightarrow aCD \mid bED \mid aFD \mid aE$

$D \rightarrow bED \mid aFD \mid aE$

$E \rightarrow a$

$F \rightarrow b$

Q.5.(b) Write CFG grammar for

(i) Set of all strings over an alphabet {a, b}, with exactly twice as many a's as b's.

(ii) $L(G) = \{a^i b^j c^j \mid i, j \geq 0\}$. **7M**

Ans. (i) Valid stings :

{abb, aabbbb, bba, bbaaaa, ...}

$\therefore$ CFG : $S \rightarrow AB$

$A \rightarrow aAb \mid aaA \mid a \mid \in$

$B \rightarrow bAa \mid bbB \mid b \mid \in$.

The grammar with the following production will generate context grammar G

$S \rightarrow$ (AAB, ABA, BAA)

$A \rightarrow$ (a, BAAA, ABAA, AABA, AAAB)

$B \rightarrow$ (b, BBAA, BABA, aBAAB, ABAB, AABB)

(ii) See P.3-63, Q.110(iii).

UNIT - IV

(Total Marks : 13)

Q.6.(a) Construct a PDA for the language

$L = \{x \in (a, b)^* \mid n_a(x) = n_b(x)\}$. **7M**

Ans. See P.4-16, Q.24.

Q.6.(b) Explain the closure properties of context free language. **6M**

Ans. See P.4-26, Q.40.

UNIT - V
(Total Marks : 28)

Q.7.(a) Design a Turing machine which subtracts two unary numbers i.e. m – n where m ≥ n. Draw the transition graph for it. **8M**

Ans. See P.5-39, Q.48.

Q.7.(b) Explain the various types of Turing machine with suitable diagram. **5M**

Ans. See P.5-21, Q.19.

Q.8.(b) Construct a TM that accepts

$L = \{0^{2n} \mid n \geq 0\}$. **5M**

Ans. See P.5-36, Q.44.

Q.9.(c) Explain the halting problem of Turing machine. **4M**

Ans. See P.5-28, Q.27.

Q.10.(a)(i) Write a short note on Church Turing Thesis.

Ans. See P.5-30, Q.31. **3M**

Q.10.(a)(ii) Write a short note on Linear Bounded automata. **3M**

Ans. See P.5-34, Q.40.

UNIT - VI
(Total Marks : 17)

Q.8.(a) What is the difference between Recursive language and a recursively enumerable language? **3M**

Ans. See P.6-7, Q.1.

Q.8.(c) If L is a recursive language over Σ show that τ (τ is defined as Σ ∀ _ L) is also recursive. **5M**

Ans. See P.6-9, Q.6.

Q.9.(a) Explain the post correspondence problem. consider the following two lists.

$L_1 : a^2, b^2, ab^2$

$L_2 : a^2b, ba, b$

Does this distance of PCP have the solution ? If yes, then find the solution. **6M**

Ans. See P.6-17, Q.21(a).

Q.10.(a)(iii) Write a short note on Decidability and Decidable language. **3M**

Ans. See P.1-5, Q.5(3) and See P.6-13, Q.15(a).

SUMMER EXAMINATION - 2014
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	26 M
Unit - III	26 M
Unit - IV	28 M
Unit - V	23 M
Unit - VI	29 M
Questions covered in VBD	160 M (100 %)

SUMMER EXAM.-2014
QUESTION PAPER

Q.1.(a) Define the following terms with the help of examples :
(i) Transitive Closure
(ii) Reflexive Transitive Closure
(iii) Equivalence Relation. **6M**

(b) Prove the following by Mathematical Induction :

$$1 + 2 + 3 + ...n - 1 = \frac{n(n-1)}{2}.$$ **4M**

(c) Write a short note on Pigeonhole principle. **4M**

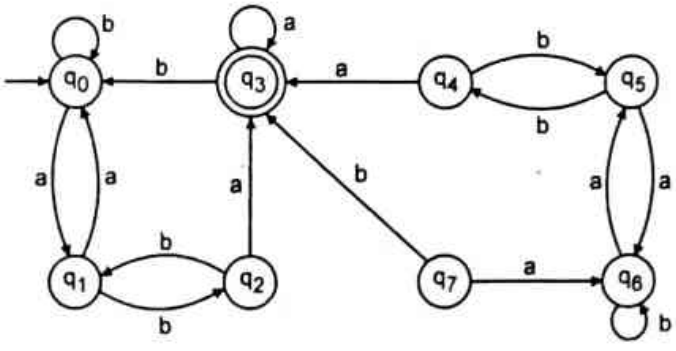
OR

Q.2.(a) Explain the following with suitable examples :
(i) Alphabet set
(ii) Proper prefix
(iii) Proper suffix
(iv) Power set. **8M**

(b) Write short notes on :
(i) Countability
(ii) Diagonalization. **6M**

Q.3.(a) Design a finite state machine which determines the residue mod 3 for any binary string. Design Moore and Mealy machine for the same. **6M**

(b) Minimize the DFA : **4M**



(c) Differentiate between NFA and DFA. **3M**

OR

Q.4.(a) Design a DFA accepting following language :

$$L = \left\{ w \;\middle|\; \begin{array}{l} w \in (a \mid b)^* \\ n(a)\ w \text{ MOD } 3 > nb(w) \text{ MOD } 3 \end{array} \right.$$

where na(w) → Number of a's in w
nb(w) → Number of b's in w. **8M**

(b) Design a Moore and Melay Machine that generates output EVEN if the number of a's are Even and generates output ODD if the number of a's are odd for $\Sigma = \{a, b\}$. **5M**

Q.5.(a) Convert the following Grammar from Right Linear to Left Linear form :
$S \rightarrow 01A \mid 10$
$A \rightarrow 10A \mid 10.$ **5M**

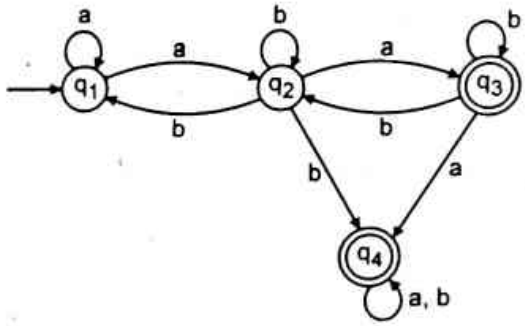
(b) Reduce the following Grammar :
$S \rightarrow AB \mid CA$
$B \rightarrow BC \mid AB$
$A \rightarrow a$
$C \rightarrow aB \mid b.$ **4M**

(c) Write a short note on Ambiguity of Grammar. Prove that the following Grammar is ambiguious :
$S \rightarrow aSbS \mid bSaS \mid \in.$ **4M**

OR

Q.6.(a) Design a NFA with $\in$-moves for the following regular expression then find equivalent DFA :
$(a + b)^*\ aba'\ (a + ba)^*\ b.$ **8M**

(b) Find Regular Expression for following finite Automata : **5M**



Q.7.(a) Design PDA to check for a well formed parenthesis, write an algorithm for the same. Explain it with transition diagram. **6M**

(b) Convert the PDA accepting following language $L = \{a^n b^{2n} \mid n \geq 1\}$ into equivalent CFG. **8M**

OR

Q.8.(a) Construct a PDA for the CFG and show the acceptance/ rejection of string
w = abababb
$S \rightarrow aSa \mid bSb \mid a \mid b.$ **7M**

(b) Construct PDA for the language :
$L = \{a^n b^n c^m d^m / m, n \geq 1\}.$ **7M**

Q.9.(a) Explain various types of Turing Machine. **6M**

VBD

(b) Design a Turing Machine to perform 2's complement of binary number. [Explain with Transition diagram and algorithm]. **5M**

(c) Explain Unrestricted Grammar. **2M**

OR

Q.10.(a) Design a Turing machine for :

$L = \{W \subset W^R \mid W = (a, b)^*\}$. **7M**

(b) What are the closure properties of Recursive Enumerable Language? **6M**

Q.11.(a) Show that Halting problem is unsolvable. **4M**

(b) Explain PCP problem and give solution for following :

$X = \{babbb, ba, b\}$, $Y = \{ba, a, bbb\}$. **5M**

(c) Define μ-recursive function. **4M**

OR

Q.12.(a) Find Ackermann's function A (1, 1) = ? and A (2, 3) = ? **4M**

(b) Explain PRF and prove that the function is PRF :

(i) $f(x, y) = x + y$

(ii) $f(x, y) = x^y$. **6M**

(c) Explain Church's Hypothesis. **3M**

SUMMER EXAM.-2014

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 28)

Q.1.(a) Define the following terms with the help of examples :

(i) Transitive Closure

(ii) Reflexive Transitive Closure

(iii) Equivalence Relation. **6M**

Ans. See P.1-5, Q.4.

Q.1.(b) Prove the following by Mathematical Induction :

$$1 + 2 + 3 + ...n - 1 = \frac{n(n-1)}{2}.$$ **4M**

Ans.

(i) Basic steps :

For n = 1

L. H. S = 0 .....(1)

and R. H. S. $= \frac{n(n-1)}{2}$

$= \frac{1(1-1)}{2}$

$= \frac{0}{2} = 0$ .....(2)

$\therefore$ Hence from (1) and (2)

L.H.S = R.H.S.

(ii) Induction hypothesis :

Assume that for some $n \geq 1$

$$1 + 2 + 3 + ...n - 1 = \frac{n(n-1)}{2}$$

(iii) Induction step :

For n = n + 1

L.H.S. : $1 + 2 + 3 + ...+ (n + 1 - 1)$

$= 1 + 2 + 3 + ...+ n$

$= 1 + 2 + 3 + ...+ (n - 1) + n$

$= \frac{n(n-1)}{2} + n$

$= \frac{n(n-1) + 2n}{2}$

$= \frac{n^2 - n + 2n}{2}$

$= \frac{n^2 + n}{2} = \frac{n(n+1)}{2}$

R.H.S. : for n = n + 1

$= \frac{(n+1)(n+1-1)}{2}$

$= \frac{(n+1)(n)}{2} = \frac{n^2 + n}{2}$

$\Rightarrow \frac{n(n+1)}{2}$

$\therefore$ L.H.S = R.H.S.

Q.1.(c) Write a short note on Pigeonhole principle. **4M**

Ans. See P.1-10, Q.16.

Q.2.(a) Explain the following with suitable examples :

(i) Alphabet set

(ii) Proper prefix

(iii) Proper suffix

(iv) Power set. **8M**

Ans. See P.1-4, 11, Q.2 & 20.

Q.2.(b) Write short notes on : **6M**

(i) Countability.

Ans. See P.1-6, Q.7.

(ii) Diagonalization.

Ans. See P.1-5, Q.5.

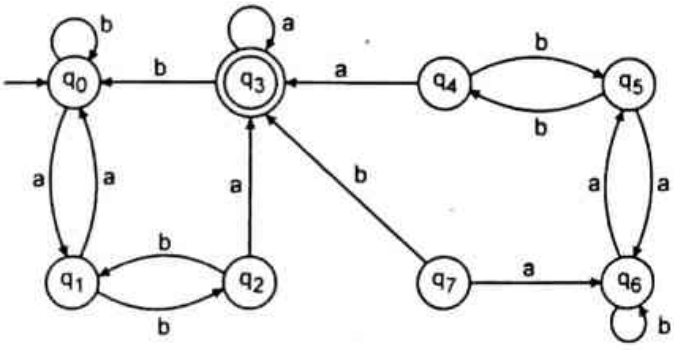
UNIT - II

(Total Marks : 26)

Q.3.(a) Design a finite state machine which determines the residue mod 3 for any binary string. Design Moore and Mealy machine for the same. **6M**

Ans. See P.2-32, 2-54, Q.44 & 54.

Q.3.(b) Minimize the DFA : **4M**



Ans. See P.2-41, Q.58.

Q.3.(c) Differentiate between NFA and DFA. **3M**

Ans. See P.2-13, Q.12.

Q.4.(a) Design a DFA accepting following language :

$$L = \left\{ w \;\middle|\; \begin{array}{l} w \in (a|b)^* \\ n(a)\, w \text{ MOD } 3 > nb(w) \text{MOD } 3 \end{array} \right.$$

where na(w) → Number of a's in w

nb(w) → Number of b's in w. **8M**

Ans. See P.2-33, Q.48.

Q.4.(b) Design a Moore and Melay Machine that generates output EVEN if the number of a's are Even and generates output ODD if the number of a's are odd for Σ = {a, b}. **5M**

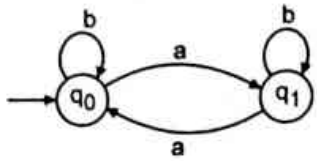
Ans. We require two state q_0 and q_1.

q_0 denote even number of a and q_1 denotes odd number of a.

First we construct FSM for even number of a odd number of a.

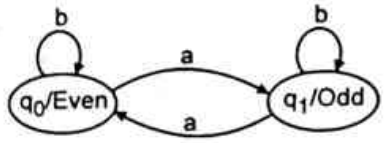
Σ / Q	a	b
→ q_0	q_0	q_1
q_1	q_1	q_0

Transition diagram :



Now we will add outputs.

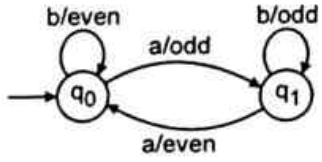
Moore Machine :



Transition table for Moore Machine :

Present State	Next state		Outputs
	a	b	
→ q_0	q_1	q_0	even
q_1	q_0	q_1	odd

Mealy Machine :



Transition table for mealy machine :

Present State	Next state			
	a	Output	b	Output
→ q_0	q_1	odd	q_0	even
q_1	q_0	Even	q_1	odd

UNIT - III

(Total Marks : 26)

Q.5.(a) Convert the following Grammar from Right Linear to Left Linear form

S → 01A | 10

A → 10A |10. **5M**

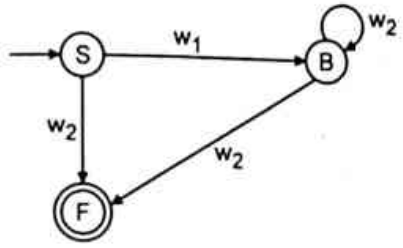
Ans. Let $w_1 = 01$

and $w_2 = 10$

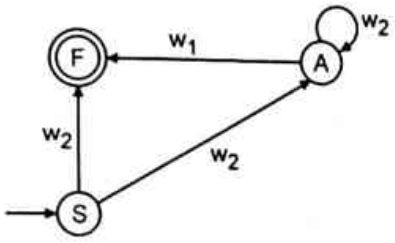
$\therefore\ S \rightarrow w_1 A / w_2$

$A \rightarrow w_2 A / w_2$

Transition graph for right linear grammar :



Transition graph for left linear grammar :



Left linear grammar :

$S \rightarrow A\, w_2 / w_2$

$A \rightarrow A\, w_2 / w_1$

$\therefore\ S \rightarrow A\,10/10$

$A \rightarrow A\,10/01.$

Q.5.(b) Reduce the following Grammar :

$S \rightarrow AB \mid CA$

$B \rightarrow BC \mid AB$

$A \rightarrow a$

$C \rightarrow aB \mid b.$ **4M**

Ans. See P.3-73, Q.139.

Q.5.(c) Write a short note on Ambiguity of Grammar. Prove that the following Grammar is ambiguious :

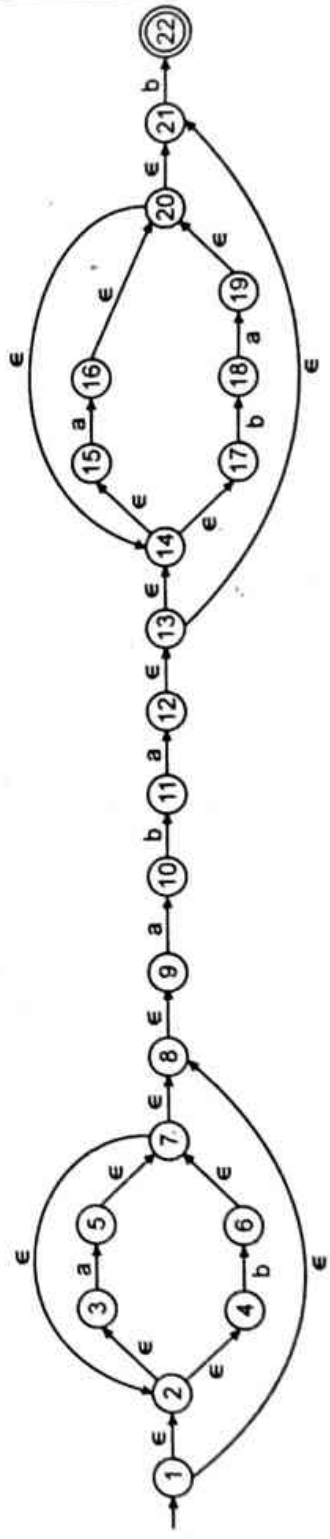
$S \rightarrow aSbS \mid bSaS \mid \in.$ **4M**

Ans. See P.3-66, 68, Q.120 & 127.

Q.6.(a) Design a NFA with ϵ-moves for the following regular expression then find equivalent DFA :

$(a + b)^* aba' (a + ba)^* b.$ **8M**

Ans. Transition graph for given regular grammar :

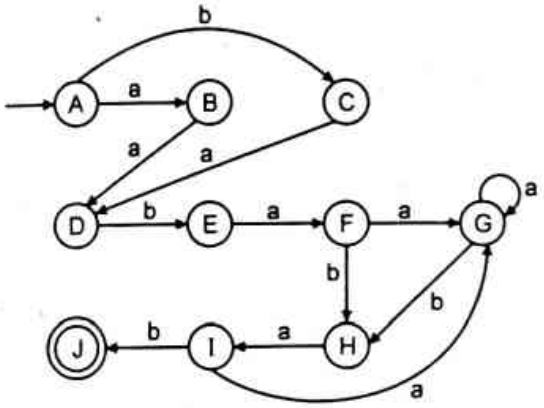


Transition table :

State	a	b
A = {1, 2, 3, 4, 8, 9}	B = {2, 3, 4, 5, 7, 8, 9}	C = {6, 7, 8, 2, 3, 4, 9}
B = {2, 3, 4, 5, 7, 8, 9}	D = {10}	ϕ

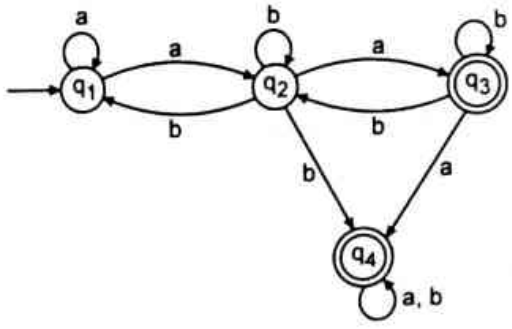
C = {2, 3, 4, 6, 7, 8, 9}	D = {10}	ϕ
D = {10}	-	E = {11}
E = {11}	F = {12, 13, 14, 15, 17, 21}	ϕ
F = {12, 13, 14, 15, 17, 21}	G = {16, 20, 14, 15, 17}	H = {18}
G = {14, 15, 16, 17, 20}	G = {15, 20, 14, 15, 17}	H = {18}
H = {18}	I {14, 15, 17, 19, 20, 21}	ϕ
I = {14, 15, 17, 19, 20, 21}	G	* J = {22}

Transition diagram :



DFA

Q.6.(b) Find Regular Expression for following finite Automata : **5M**



Ans. Equation for all states :

$q_1 = q_1a + q_2b + \epsilon$ .....(1)

$q_2 = q_1a + q_2b + q_3b$ .....(2)

$q_3 = q_2a + q_3b$ .....(3)

$q_4 = q_2b + q_3a + q_4a + q_4b$ .....(4)

$\therefore \underbrace{q_3}_{R} = \underbrace{q_2}_{Q}\underbrace{a}_{R} + \underbrace{q_3}_{B}\underbrace{b}$

By Arden Theorem

$q_3 = q_2a\,b^*$ .....(5)

Put value of q_3 equation (2), we get

$q_2 = q_1a + q_2b + (q_2a\,b^*)\,b$

$q_2 = q_1a + q_2\,(b + a\,b^*)\,b$

By Arden Theorem

$q_2 = q_1a\,((b + a\,b^*)\,b)^*$ .....(6)

$\therefore$ Put value of q_2 in equation (1) we get

$q_1 = q_1a + q_1a\,((b + a\,b^*)\,b)^*\,b + \epsilon$

$= q_1\,[a + a\,((b + a\,b^*)\,b)^*\,b + \epsilon$

$\therefore$ By Arden Theorem, we get

$q_1 = [a + a\,((b + a\,b^*)\,b)^*\,b]^*$

Put value of q_1 in equation (6) we get

$q_2 = [a + a\,((b + a\,b^*)\,b)^*\,b]^*\,a\,((b + a\,b^*)\,b)^*$

Put value of q_2 in equation (5) we get

$q_3 = [a + a\,((b + a\,b^*)\,b)^*\,b]^*\,a\,((b + a\,b^*)\,b)^*\,a\,b^*$

$\therefore\ q_4 = a_2b + q_3a + q_4\,(a + b)$

$\therefore\ q_4 = [[a + a\,((b + a\,b^*)\,b)^*\,b]^*\,a\,((b + a\,b^*)\,b)^*]\,b + [[a + a\,((b + a\,b^*)\,b)^*\,b]^*\,a\,((b + a\,b^*)\,b)^*\,a\,b^*]\,a + q_4\,(a + b)$

$\therefore$ By Arden Theorem, we get

$q_4 = [[a + a\,((b + a\,b^*)\,b)^*\,b]^*\,a\,((b + a\,b^*)\,b)^*]\,b + [[a + a\,((b + a\,b^*)\,b)^*\,b]^*\,a\,((b + a\,b^*)\,b)^*\,a\,b^*]\,a\,(a + b)^*$

Regular expression is $q_3 + q_4$:

$= [a + a\,((b + a\,b^*)\,b)\,b]^*\,a\,((b + a\,b^*)\,b)^*\,a\,b^* + [[a + a\,((b + a\,b^*)\,b)^*\,b)^*\,b]^*\,a\,((b + a\,b^*)\,b)^*\,]\,b + [[a + a\,((b + a\,b^*)\,b)^*\,b]^*\,a\,((b + a\,b^*)\,b)^*\,a\,b^*]\,a\,(a + b)^*.$

UNIT - IV

(Total Marks : 28)

Q.7.(a) Design PDA to check for a well formed parenthesis, write an algorithm for the same. Explain it with transition diagram. **6M**

Ans. The well-formedness of parenthesis mean that the input should start with 'C' opening parenthesis and it should be followed by ')' closing parenthesis and number of '(' and ')' should be equal. The input should not start with ')'.

Algorithm :

Step 1 : For all input symbol '(' push a to the stack.

Step 2 : For all input symbol ')' pop a from the stack.

Transition :

(1) $\delta(q_0, (, Z_0) \rightarrow (q_0, a\ Z_0)$

(2) $\delta(q_0, (, a) \rightarrow (q_0, a\ a)$

(3) $\delta(q_0,), a) \rightarrow (q_1, \in)$

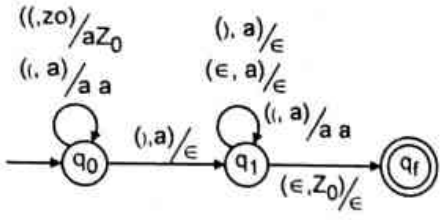
(4) $\delta(q_1,), a) \rightarrow (q_1, \in)$

(5) $\delta(q_1, (, a) \rightarrow (q_1, a\ a)$

(6) $\delta(q_1, \in, a) \rightarrow (q_1, \in)$

(7) $\delta(q_1, \in, Z_0) \rightarrow (q_f, \in)$

Transition graph :



Q.7.(b) Convert the PDA accepting following language $L = \{a^n b^{2n} \mid n \geq 1\}$ into equivalent CFG. **8M**

Ans. See P.4-9, Q.12.

Q.8.(a) Construct a PDA for the CFG and show the acceptance/ rejection of string

w = ababab

S → aSa | bSb | a | b. **7M**

Ans. $M = \{Q, \Sigma, T, \delta\ q_0, Z_0, f\}$

Formal definition :

$M = \{Q, \Sigma, T, \delta\ q_0, Z_0, f\}$

$m = (\{a\}, \{a, b\}, \{S, a, b\}, \delta, q, S\}$

Transitions :

(1) $S \rightarrow a\ S\ a$

$\delta(q, a, S) \rightarrow (q, Sa)$

(2) $S \rightarrow b\ S\ b$

$\delta(q, b, S) \rightarrow (q, Sb)$

(3) $S \rightarrow a$

$\delta(q, a, S) \rightarrow (q, \in)$

(4) $S \rightarrow b$

$\delta(q, b, S) \rightarrow (q, \in)$

Given :

ω = "a b a b a b"

$\vdash$ (q, a b a b a b S)
$\vdash$ (q, a b a b a b, aSa)
$\vdash$ (q, b a b a b, Sa)
$\vdash$ (q, b a b a b, bSba)
$\vdash$ (q, a b a b, Sba)
$\vdash$ (q, a b a b, aSa ba)
$\vdash$ (q, b a b, S a b a)
$\vdash$ (q, b a b, b a b a)
$\vdash$ (q, a b, aba)
$\vdash$ (q, b, ba)
$\vdash$ (q, a)

Stack is not empty.

Hence the given string is rejected.

Q.8.(b) Construct PDA for the language :

$L = \{a^n b^n c^m d^m / m, n \geq 1\}$. **7M**

Ans. Let n = 1 and m = 1

$L_1 = \{a\ b\ c\ d\}$

Let n = 1 and m = 2

$L_2 = \{a\ b\ c\ c\ d\ d\}$

Let n = 2, m = 1

$L_3 = \{a\ a\ b\ b\ c\ d\}$

$\therefore$ Valid string = {abcd, adccdd, aabbcd}

Analysis :

(1) String are fixed length.

(2) String starts with a and each with of with either equal number of a's b's or equal number of c's and d's.

Algorithm :

Step 1 : Push a symbol of input string on stack and continue storing until the other symbol occur.

Step 2 : If the top of stack symbol in a and the current input symbol in b remove a stack top a.

Step 3 : If the top of stack is Z_0 and the current input symbol is c, then push c an to the stack and continue storing until other symbol occur.

Step 4 : If the top of stack is c and the current input symbol in d, the pop c from the stack Repeat this process for input d and stack top in c.

Step 5 : Replace Z_0 by ϵ.

Transition :

(1) $\delta(q_0, a, Z_0) \rightarrow (q_0, a\ Z_0)$

(2) $\delta(q_0, a, a) \rightarrow (q_0, a\ a)$

(3) $\delta(q_0, b, a) \rightarrow (q_1, \epsilon)$

(4) $\delta(q_1, b, a) \rightarrow (q_1, \epsilon)$

(5) $\delta(q_1, c, Z_0) \rightarrow (q_1, c\ Z_0)$

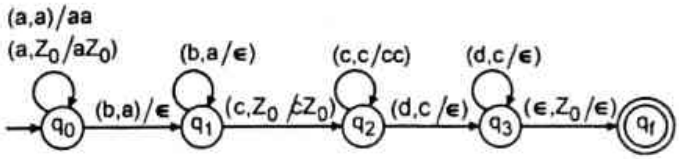
(6) $\delta(q_2, c, c) \rightarrow (q_2, c\ c)$

(7) $\delta(q_2, d, c) \rightarrow (q_3, \epsilon)$

(8) $\delta(q_3, d, c) \rightarrow (q_3, \epsilon)$

(9) $\delta(q_3, \epsilon, Z_0) \rightarrow (q_F, \epsilon)$

Transition diagram :



UNIT - V

(Total Marks : 23)

Q.9.(a) Explain various types of Turing Machine. **6M**

Ans. See P.5-21, Q.19.

Q.9.(b) Design a Turing Machine to perform 2's complement of binary number. [Explain with Transition diagram and algorithm]. **5M**

Ans. See P.5-37, Q.45.

Q.9.(c) Explain Unrestricted Grammar. **2M**

Ans. The unrestricted grammar is defined as :

$G = (V_N, V_T, P, S)$

Where, V_N = a finite set of non-terminal

V_T = a finite set of terminal

S =Starting non-terminal, $S \in V_N$ and

P in set of production.

In grammar, every productor rule is of the form : $\alpha \rightarrow \beta$ where α and β are arbitrary string of grammar symbol with $\alpha \neq \epsilon$ i.e. they can be any string of terminals and non terminal.

Q.10.(a) Design a Turing machine for :

$L = \{W \subset W^R \mid W = (a, b)^*\}$. **7M**

Ans. $L = \{W \subset W^R \mid W = (a, b)^*\}$

This string is formed as abcda i.e. c is the middle element.

In this machine traverse upto the symbol c and then change the stage i.e. q_1.

i.e.

$\delta(q_0, a) = q_0, a, R$

$\delta(q_0, b) = q_0, b, R$

$\delta(q_0, c) = q_1, c, R$

The check the symbol on right side of 'c' of it is zero left side also we should get zero. If so again forward to check next symbol and repeat the procedure hill machine gets blank. If blank then come back and again it should get blank if so then accept, otherwise reject.

$\delta(q_1, a) \rightarrow (q_2, X, L)$

$\delta(q_1, b) \rightarrow (q_4, X, L)$

$\delta(q_2, c) \rightarrow (q_2, C, L)$

$\delta(q_4, c) \rightarrow (q_4, C, L)$

$\delta(q_2, a) \rightarrow (q_3, X, R)$

$\delta(q_4, b) \rightarrow (q_3, X, R)$

$\delta(q_3, c) \rightarrow (q_3, C, R)$

$\delta(q_3, X) \rightarrow (q_3, X, R)$

$\delta(q_3, a) \rightarrow (q_2, X, L)$

$\delta(q_4, X) \rightarrow (q_4, X, L)$

$\delta(q_2, X) \rightarrow (q_2, X, L)$

$\delta(q_3, 1) \rightarrow (q_4, X, L)$

$\delta(q_3, B) \rightarrow (q_5, B, L)$

$\delta(q_5, C) \rightarrow (q_5, C, L)$

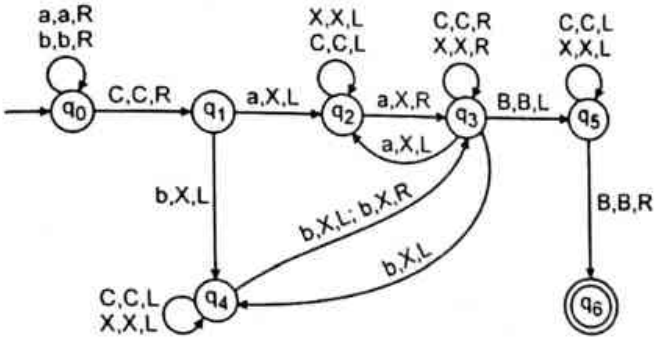
$\delta(q_5, X) \rightarrow (q_5, X, L)$

$\delta(q_5, B) \rightarrow (q_6, B, R)$

Transition table :

Present State	Tape symbol				
	a	b	c	X	B
$\rightarrow q_0$	q_0, a, R	q_0, b, R	q_1, c, R		
q_1	q_2, X, L	q_4, X, L			
q_2	q_3, X, R		q_2, C, L	q_2, X, L	
q_3	q_2, X, L	q_4, X, L	q_3, C, R	q_3, X, R	q_5, B, L
q_4		q_3, X, R	q_4, C, L	q_4, X, L	
q_5			q_5, C, L	q_5, X, L	q_6, B, R
q_6	–	–	–	–	–

Transition diagram :



Q.12.(c) Explain Church's Hypothesis. **3M**

Ans. See P.5-30, Q.31.

UNIT - VI

(Total Marks : 29)

Q.10.(b) What are the closure properties of Recursive Enumerable Language? **6M**

Ans. See P.6-7, 8, 9, Q.2, 4, 5, & 6.

Q.11.(a) Show that Halting problem is unsolvable. **4M**

Ans. See P.6-19, 20, Q.25 & 26.

Q.11.(b) Explain PCP problem and give solution for following :

X = {babbb, ba, b}, Y = {ba, a, bbb}. **5M**

Ans. See P.6-16, Q.19.

Q.11.(c) Define μ-recursive function. **4M**

Ans. See P.6-26, Q.37.

Q.12.(a) Find Ackermann's function A (1, 1) = ? and A (2, 3) = ? **4M**

Ans. Ackermann's function is defined by :

$A(0, y) = y + 1$ .....(1)

$A(x + 1, 0) = A(x, 1)$ .....(2)

$A(x + 1, y + 1) = A(x, A(x + 1, y))$ ....(3)

$A(1, 1) = A(0 + 1, 0 + 1)$

$= A(0, A(1, 0)$

$= A(0, A(0, 1)$

$= A(0, 2)$

$= 2 + 1$

$A(1, 1) = 3$

$A(2, 3) = A(1 + 1, 2 + 1)$

$= A(1, A(1 + 1, 2))$

$= A(1, A(2, 2))$

$A(2, 2) = A(1 + 1, 1 + 1)$

$= A(1, A(2, 1))$

$A(2, 1) = A(1 + 1, 0 + 1)$

$= A(1, A(2, 0))$

$= A(1, A(1 + 1, 0))$

$= A(1, A(1 + 1, 0))$

$= A(1, A(1, 1))$

$= A(1, 3)$

As, $A(1, 1) = 3$

$= A(1, 3)$

$= A(0 + 1, 2 + 1)$

$= A(0, A(1, 2))$

$= A(0, A(0 + 1, 1 + 1))$

$= A(0, A(0, A(1, 1))$

$= A(0, A(0, 3))$

$= A(0, 4)$

$= 5$

$\therefore A(2, 2) = A(1, A(2, 1))$

$= A(1, 5)$

$\therefore A(1, 5) = A(0 + 1, 4 + 1)$

$= A(0, A(1, 4))$

$= A(1 + A(1, 9)$

$= 1 + A(0 + 1, 3 + 1)$

$= 1 + A(0, A(1, 3))$

$= 1 + A(0, A(1, 3))$

$= 1 + 1 + A(1, 3)$

$= 1 + 1 + A(0 + 1, 2 + 1)$

$= 1 + 1 + A(0, A(1, 2))$

$= 1 + 1 + 1 + A(1, 2)$

$= 1 + 1 + 1 + A(0 + 1, 1 + 1)$

$= 1 + 1 + 1 + A(0, A(1, 1))$

$= 1 + 1 + 1 + A(0, 3)$

$= 1 + 1 + 1 + 4$

$= 7$

$\therefore A(2, 2) = 7$

$\therefore A(2, 3) = A(1, 7)$

$A(1, 7) = A(0 + 1, 6 + 1)$

$= A(0, A(1, 6))$

$= 1 + A(1, 6)$

$= 1 + A(0 + 1, 5 + 1)$

$= 1 + A(0, A(1, 5))$

$= 1 + 1 + A(1, 5)$

$= 1 + 1 + 7$

$\therefore$ As $A(1, 5) = 7$

$= 9$

$\therefore A(2, 3) = 9.$

Q.12.(b) Explain PRF and prove that the function is PRF :

(i) $f(x, y) = x + y$

(ii) $f(x, y) = x^y$. **6M**

Ans. See P.6-29, Q.45 & 47.

WINTER EXAMINATION - 2013
B. E. V SEM. (COMPUTER TECHNOLOGY)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	00 M
Unit - II	27 M
Unit - III	32 M
Unit - IV	07 M
Unit - V	26 M
Unit - VI	40 M
Questions covered in VBD	132 M (100 %)

WINTER EXAM.-2013
QUESTION PAPER

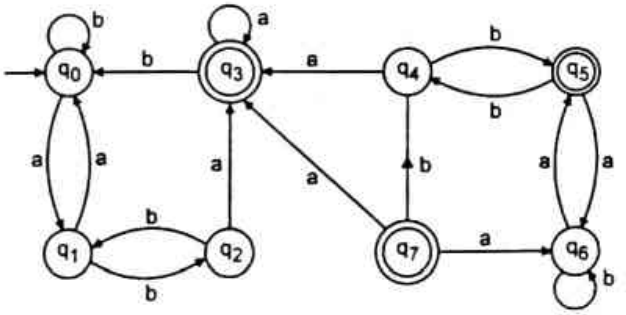
SECTION - A

Q.1.(a) Differentiate between deterministic and Non-Deterministic Finite Automata with an example. **3M**

(b) Construct a DFA to accept the language :
$L = \{w \in (a, b)^* / w$ does not have abb as substring$\}$. **8M**

(c) Construct a F.A. to accept the combination of a's and b's containing starting and ending with v same symbol. **3M**

Q.2.(a) Construct the minimum state automation equivalent to the transition diagram given in Fig :



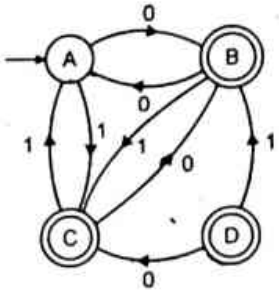
8M

(b) Construct Moore machines for the input from Σ^*, where $\Sigma = \{0, 1\}$. If inputs end with 10011, output should be 'X' ; if inputs end with 01100, output should be 'Y' ; otherwise output should be 'Z'. **5M**

Q.3.(a) Construct a DFA for the given R.E :
$R = 1.0 + (0 + 11)\, 0^*1$ **7M**

(b) Show that the given language is not regular using pumping lemma.
$L = \{a^n b^m c^l / n, m \geq 1 \,\&\, l \geq n + m\}$ **6M**

Q.4.(a) Construct the regular expression for the following state diagram. **7M**



(b) Show that grammar is ambiguous :
$S \rightarrow 0Sl / 0Al / 0$
$A \rightarrow 0AA / lS$ **6M**

Q.5.(a) Construct a PDA for the Language :
$L = \{a^{2n}\, b^2 / n \geq 0\}$. **7M**

(b) Explain Chomsky Normal Form and Greibach Normal Form. **3M**

(c) Obtain the GNF for the following grammar :
$S \rightarrow AA / a$
$A \rightarrow SS / b$ **3M**

SECTION - B

Q.6.(a) Explain Linear Bounded Automata. **5M**

(b) Design a Turing Machine for calculation of proper subtraction of Two Integer Number.
$a - b = a - b$ if $a > b$
$= 0$ otherwise. **9M**

Q.7.(a) Construct Turing Machine for the following Language :
$L = \{a^n\, b^n\, c^{2n} / n \geq 1\}$. **8M**

(b) Explain properties of Recursive and Recursive Enumerable Languages. **5M**

Q.8.(a) Explain about the undecidability of problem. **5M**

(b) Define primitive recursive function. Show that the function $f(x, y) = x * y$ is primitive recursive. **8M**

Q.9.(a) Define Ackermann's function and compute :
(i) A (1, 1)
(ii) A (2, 1)
(iii) A (2, 2) **9M**

(b) Write short notes on μ - Recursive Function. **4M**

Q.10. Write short notes on : **13M**

(i) Post Correspondence Problem.

(ii) Universal T.M.

(iii) Bounded Minimization.

WINTER EXAM.-2013
SOLUTION
(Unit-Wise)

UNIT - I

No questions asked in University Exam.

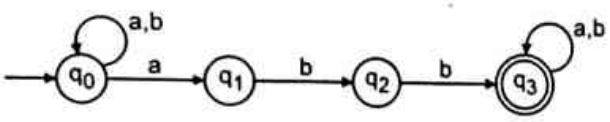
UNIT - II
(Total Marks : 27)

Q.1.(a) Differentiate between deterministic and Non-Deterministic Finite Automata with an example. **3M**

Ans. See P.2-13, Q.12.

Q.1.(b) Construct a DFA to accept the language : $L = \{w \in (a, b)^* / w$ does not have abb as substring$\}$. **8M**

Ans. The R.E. for containing substring aab is
$(a + b)^*$ abb $(a + b)^*$
Finite automata for the regular expression is :



Transition table for N.F.A :

δ	a	b
q_0	q_0 q_1	q_0
q_1	-	q_2
q_2	-	q_3
$*q_3$	q_3	q_3

Transition table for D.F.A :

δ	a	b
q_0	q_0 q_1	q_0
$[q_0, q_1]$	$[q_0, q_1]$	$[q_0, q_2]$
$[q_0, q_2]$	$[q_0, q_1]$	$[q_0, q_3]$
$[q_0, q_3]$	$[q_0, q_1, q_3]$	$[q_0, q_3]$
$[q_0, q_1, q_3]$	$[q_0, q_1, q_3]$	$[q_0, q_2, q_3]$
$[q_0, q_2, q_3]$	$[q_0, q_1, q_3]$	$[q_0, q_3]$

Renaming the state

$A = q_0$

$B = [q_0, q_1]$

$C = [q_0, q_2]$

$D = [q_0, q_3]$

$E = [q_0, q_1, q_3]$

$F = [q_0, q_2, q_3]$

Transition table for D.F.A :

δ	a	b
A	B	A
B	B	C
C	B	D
* D	E	D
* E	E	F
* F	E	D

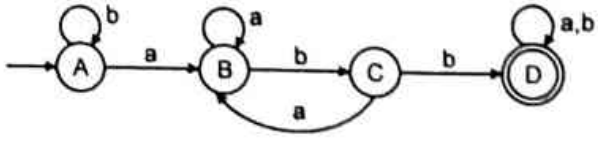
Here, D = F so after optimization transition table is :

δ	a	b
A	B	A
B	B	C
C	B	D
* D	E	D
* E	E	D

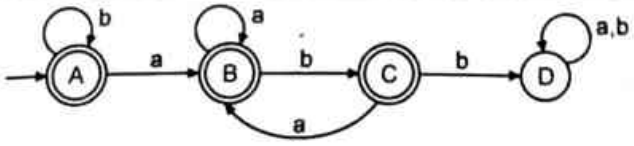
Here D = E so replace E by D, we get :

δ	a	b
A	B	A
B	B	C
C	B	D
* D	D	D

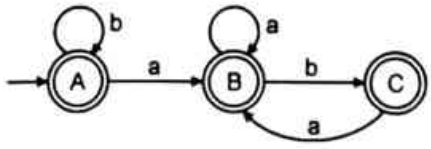
- Transition diagram for D.F.A. containing abb as substring.



- So, DFA for all possible set of a and b not containing abb as substring so we complement the state i.e. we convert all final state to non final state and vice vera.

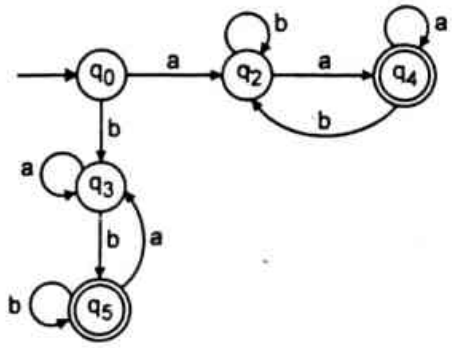


Remove dead state D we get,

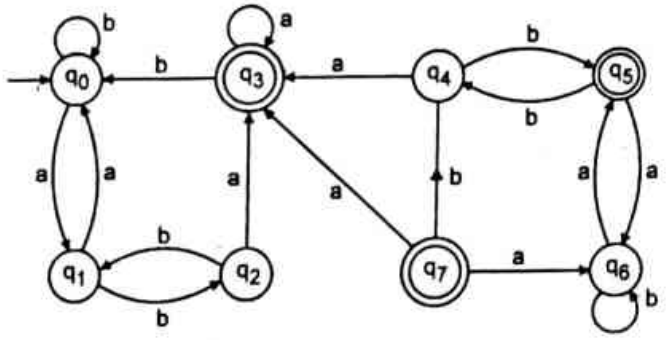


Q.1.(c) Construct a F.A. to accept the combination of a's and b's containing starting and ending with v same symbol. **3M**

Ans. Transition diagram :



Q.2.(a) Construct the minimum state automation equivalent to the transition diagram given in Fig :



8M

Ans. Transition table :

Σ / State	a	b
→ q_0	q_1	q_0
q_1	q_0	q_2
q_2	q_3	q_1
* q_3	q_3	q_0
q_4	q_3	q_5
* q_5	q_6	q_4
q_6	q_5	q_6
* q_7	q_6	q_3

From given transition table

Group I : $\{q_3, q_5, q_7\}$ set by final set

Group II : $\{q_0, q_1, q_2, q_4, q_6\}$ set of non-final set.

The pair of states are equivalent if their transition on all input symbols are same column wise. There we all consider the pair of states from group 2 as :

(i) $\{q_0, q_1\}$

(ii) $\{q_0, q_2\}$

(iii) $\{q_0, q_4\}$

(iv) $\{q_0, q_6\}$

Step 1 :

(i) $\{q_0, q_1\}$:

	a	b
q_0	q_1	q_0
q_1	q_0	q_2

↓ ↓

From same group

$\therefore q_0 = q_1$

(ii) $\{q_0, q_2\}$:

	a	b
q_0	q_1	q_0
q_2	q_2	q_1

↓

From different group

$\therefore q_0 \neq q_2$

(iii) $\{q_0, q_4\}$:

	a	b
q_0	q_1	q_0
q_4	q_3	q_5

↓ ↓

From different group

$\therefore q_0 \neq q_4$

(iv) $\{q_0, q_6\}$:

	a	b
q_0	q_1	q_0
q_6	q_5	q_6

↓

From different group

$\therefore q_0 \neq q_6$

$\therefore$ So new group are :

$\{q_3, q_5, q_7\}\{q_0, q_1\}, \{q_2, q_4, q_6\}$

Step 2 :

(i) $\{q_2, q_4\}$

(ii) $\{q_2, q_6\}$

(i) $\{q_2, q_4\}$:

	a	b
q_2	q_3	q_1
q_4	q_3	q_5

↓

From different group

$\therefore q_2 \neq q_4$

(ii) $\{q_2, q_6\}$:

	a	b
q_2	q_3	q_1
q_6	q_5	q_6

↓

From different group

$\therefore q_2 \neq q_6$

$\therefore$ So new group are :

$\{q_3, q_5, q_7\}, \{q_0, q_1\}, \{q_2\}, \{q_4, q_6\}$

Step 3 : $\{q_4, q_6\}$:

	a	b
q_4	q_3	q_5
q_6	q_5	q_6

↓

From different group

$\therefore q_4 \neq q_6$

$\therefore$ So new group are :

$\{q_3, q_5, q_7\}\{q_0, q_1\}, \{q_2\}\{q_4\}\{q_6\}$

Step 4 : $\{q_0, q_1\}$:

	a	b
q_0	q_1	q_0
q_1	q_0	q_2

↓

From different group

$\therefore q_0 \neq q_1$

$\therefore$ So new group are :

$\{q_3, q_5, q_7\}\{q_0\}\{q_1\}\{q_2\}\{q_4\}\{q_6\}$

Step 5 :

(i) $\{q_3, q_5\}$

(ii) $\{q_3, q_7\}$

(i) $\{q_3, q_5\}$:

	a	b
q_3	q_3	q_0
q_5	q_6	q_4

↓ ↓

From different group

$\therefore q_3 \neq q_5$

(ii) $\{q_3, q_7\}$:

	a	b
q_3	q_3	q_0
q_7	q_6	q_3

↓ ↓

From different group

$\therefore q_3 \neq q_7$

$\therefore$ So new group are

$\{q_3\}\{q_5, q_7\}\{q_0\}\{q_1\}\{q_2\}\{q_4\}\{q_6\}$

Step 6 : $\{q_4, q_7\}$:

	a	b
q_5	q_6	q_4
q_7	q_6	q_3

↓

From different group

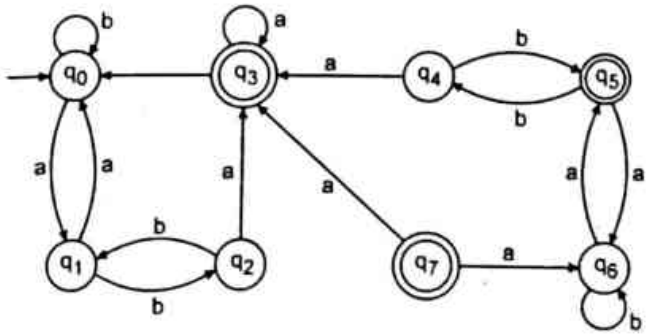
$\therefore q_5 \neq q_7$

$\therefore$ So final new group are :

$\{q_3\}\{q_5\}\{q_7\}\{q_0\}\{q_1\}\{q_2\}\{q_4\}\{q_6\}$

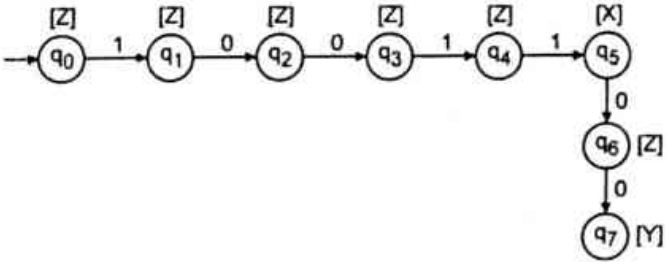
∴ So the given F.A. is already minimized.

∴ Transition graph and D.F.A. :

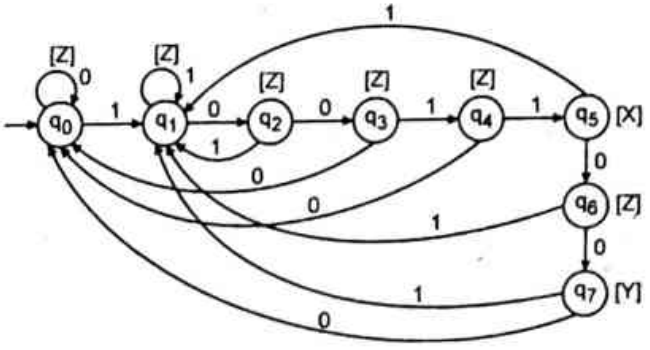


Q.2.(b) Construct Moore machines for the input from Σ^*, where $\Sigma = \{0, 1\}$. If inputs end with 10011, output should be 'X' ; if inputs end with 01100, output should be 'Y' ; otherwise output should be 'Z'. **5M**

Ans. Consider a Moore machine in which if inputs ends in '10011', the output should be X, if the inputs ends with '01100' the output should be 'Y' and for other string output should be Z.



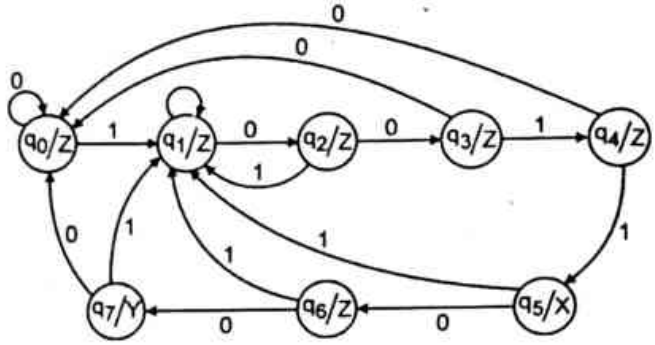
Now, insert the possibility of 1's and 0's for each state then the above graph becomes.



Transition table for Moore machine :

Present state	Next state		Output
	a = 0	a = 1	
→ q_0	q_0	q_1	Z
q_1	q_2	q_1	Z
q_2	q_3	q_1	Z
q_3	q_0	q_4	Z
q_4	q_0	q_5	Z
q_5	q_6	q_1	X
q_6	q_7	q_1	Z
q_7	q_0	q_1	Y

Transition diagram for more machine :



UNIT - III

(Total Marks : 32)

Q.3.(a) Construct a DFA for the given R.E :

$R = 1.0 + (0 + 11)\,0^*1$ **7M**

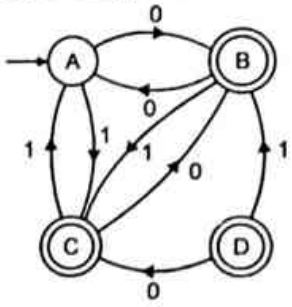
Ans. See P.3-18, Q.22.

Q.3.(b) Show that the given language is not regular using pumping lemma.

$L = \{a^n b^m c^l / n, m \geq 1 \,\&\, l \geq n + m\}$ **6M**

Ans. See P.3-56, Q.88.

Q.4.(a) Construct the regular expression for the following state diagram.



7M

Ans. Let Rename states are :

$A = q_0$

$B = q_1$

$C = q_2$

$D = q_3$

∴ The equation for state becomes :

$q_0 = q_2 1 + q_1 0 + \in$ ....(1)

$q_1 = q_0 0 + q_2 0 + 1q_3$ ....(2)

$q_2 = q_0 1 + q_1 1 + 0q_3$ ....(3)

$q_3 = \in$ ....(4)

Put value of q_3 in equation (2) and (3), we get,

$q_1 = q_0 0 + q_2 0 + 1 \in$

$q_2 = q_0 1 + q_1 1 + 0 \in$

∴ By indentity rule we get,

$q_1 = q_0 0 + q_2 0$

$q_2 = q_0 1 + q_1 1$

∴ The regular expression can be obtained from :

$\therefore q_1 + q_2 = q_0 0 + q_2 0 + q_0 1 + q_1 1$

$= q_0 (0 + 1) + q_2 0 + q_1 1$ ....(A)

Now ,

$q_1 = q_0 0 + q_0 10 + q_1 10$

$= q_0 (0 + 10) + q_1 10$

$= q_0 (0 + 10) (10)^*$ ....(B)

$\therefore q_2 = q_0 1 + q_0 (0 + 10) (10)^* 1$

$= q_0 (1 + (0 + 10) (10)^* .1)$ .....(C)

∴ From A, B, C

$\therefore q_1 + q_2 = q_0 (0 + 1) + (1 + (0 + 10) (10)^* .1) 0$

$+ q_0 (0 + 10) (10)^* 1$

$= q_0 [(0 + 1) + (1 + (0 + 10) (10)^* . (1) 0$

$+ (01 + 10) (10)^* .1]$ ....(D)

From A, B & C

$q_0 = ((1 + (0 + 10) (10)^* .1) 1 + (0 + 10) (10)^* 0)^*$

....(E

∴ From D and E

$\therefore$ R.E. $= ((1 + (0 + 10) (10)^* .1) 1 + (0 + 10) (10^* 0)^*$

$[(0 + 1) + (1 + (0 + 10) (10)^* .1) 0 + (0 + 10) (10^* .1]$

Q.4.(b) Show that grammar is ambiguous :

$S \rightarrow 0S1/0A1/0$

$A \rightarrow 0AA/1S$ **6M**

Ans. Consider the input string 00010111

Left derivation :

$S \rightarrow 0S1$

$\rightarrow 00S11$

$\rightarrow 000A111$

$\rightarrow 000\ 1S111$

$\rightarrow 0001\ 0111$

OR

$S \rightarrow 0A1$

$\rightarrow 00S11$

$\rightarrow 000A111$

$\rightarrow 000\ 1S111$

$\rightarrow 0001\ 0111$

Since there are two left must derivation for the string 00010111.

The given grammar is ambiguous.

Q.5.(b) Explain Chomsky Normal Form and Greibach Normal Form. **3M**

Ans. See P.3-80, 85, Q.159 & 173.

Q.5.(c) Obtain the GNF for the following grammar :

$S \rightarrow AA/a$

$A \rightarrow SS/b$ **3M**

Ans. See P.3-89, Q.178.

UNIT - IV

(Total Marks : 07)

Q.5.(a) Construct a PDA for the Language :

$L = \{a^{2n}\ b^2 / n \geq 0\}$. **7M**

Ans. See P.4-17, Q.26.

UNIT - V

(Total Marks : 26)

Q.6.(a) Explain Linear Bounded Automata. **5M**

Ans. See P.5-34, Q.40.

Q.6.(b) Design a Turing Machine for calculation of proper subtraction of Two Integer Number.

$a - b = a - b$ if $a > b$

$= 0$ otherwise. **9M**

Ans. See P.5-40, Q.48 [Note : Replace m by a and n by b]

Q.7.(a) Construct Turing Machine for the following Language :

$L = \{a^n b^n c^{2n} / n \geq 1\}$. **8M**

Ans. The language is fixed.

Start with a followed by b and ends with C.

Valid string :

a b c c, aa bb cccc,

Algorithm :

(1) Assume q_0 in the initial state.

(2) Replace the first symbol by a X. Change the state to q_1 and skip remaining a till symbol b arrives.

(3) On reading the symbol b, replace it with Y and change the state to q_2 and skip remaing b till symbol C arrives.

(4) On reading the left most C, the R/W head replace two 'C' by Z and moves towards left by changing state q_2 to q_3 and q_4.

(5) Step 1 to 4 are replaced while all a's, b's and c are replaced.

q_0 aa bb cccc |- X q_1 abbccc

|- X a q_1 bbcccc

|- X a Y q_2 b cccc

|- X a Y b q_2 cccc

|- X a Y b z q_3 ccc

|- X a Y b zz q_4 cc

|- X a Y b q_4 zzcc

|- X a Y q_4 b zzcc

|- X q_4 a Y b zzcc

|- X q_0 a Y b zzcc

|- XX q_1 Y b zzcc

|- XXY Y q_2 zzcc

|- XXYYZZ q_2 CC

|- XXYYZZ q_3 C

|- XXYYZZZ q_4 Z

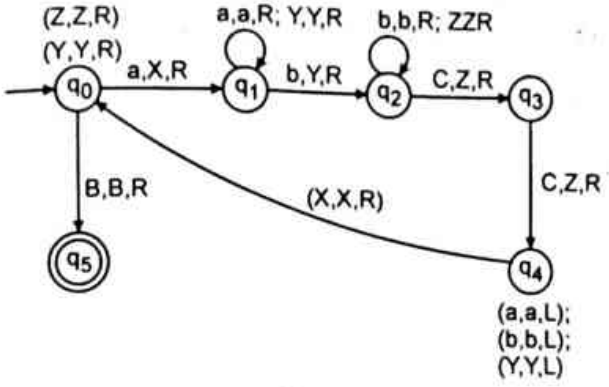
|- XXYYZZZZ q_5

As q_5 is the final state the input string aabbcccc accepted by Turing machine.

Transition table

Present state	Input tape symbol						
	a	b	c	X	Y	Z	B
$\rightarrow q_0$	q_1, X, R				q_0, Y, R	q_0, Z, R	q_5, B, R
q_1	q_1, a, R	q_2, Y, R			q_1, Y, R		
q_2		q_2, b, R	q_3, Z, R			q_2, Z, R	
q_3			q_4, Z, R				
q_4	q_4, a L	q_4, b L		q_0, X, R	q_4, Y, L		
q_5							

Transition diagram :



Q.10(ii) Write short note on Universal T.M. **4M**

Ans. See P.5-27, Q.26.

UNIT - VI

(Total Marks : 40)

Q.7.(b) Explain properties of Recursive and Recursive Enumerable Languages. **5M**

Ans. See P.6-7, Q.2.

Q.8.(a) Explain about the undecidability of problem. **5M**

Ans. See P.6-14, Q.16(a).

Q.8.(b) Define primitive recursive function. Show that the function f (x, y) = x * y is primitive recursive. **8M**

Ans. See P.6-28, 29, Q.42 & 46.

Q.9.(a) Define Ackermann's function and compute :

(i) A (1, 1)

(ii) A (2, 1)

(iii) A (2, 2) **9M**

Ans. See P.6-38, Q.68.

Q.9.(b) Write short notes on μ - Recursive Function. **4M**

Ans. See P.6-26, Q.37.

Q.10. Write short notes on : **9M**

(i) Post Correspondence Problem.

(iii) Bounded Minimization.

Ans. See P.6-15, 25, Q.18 & 35.

NOTES

SUMMER EXAMINATION - 2014
B. E. IV SEM. (COMPUTER TECHNOLOGY) (CBS)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	28 M
Unit - III	41 M
Unit - IV	13 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

SUMMER EXAM.-2014
QUESTION PAPER

Q.1.(a) Explain Chomsky hierarchy of languages. **8M**

(b) If R = {(1, 1), (2, 2), (3, 1), (1, 4), (2, 4)} on a set A = {1, 2, 3, 4}. Obtain R*, the reflexive transitive closure of R. **5M**

OR

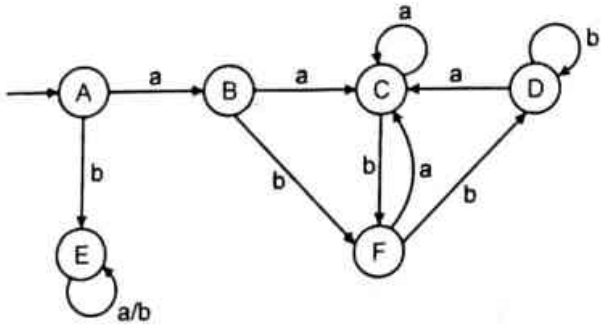
Q.2.(a) Prove the following by Principle of Induction :

(i) $1 + 2 + 3 + \ldots + n = n(n+1)/2$

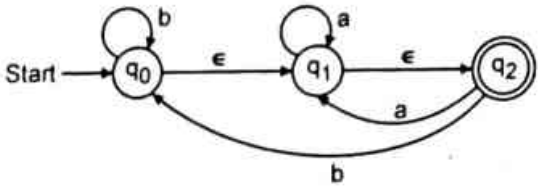
(ii) $\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$ **8M**

(b) Explain Pigeonhole principle with example. **5M**

Q.3.(a) Construct a minimum state automata equivalent to the finite automata given in Fig. below : **5M**

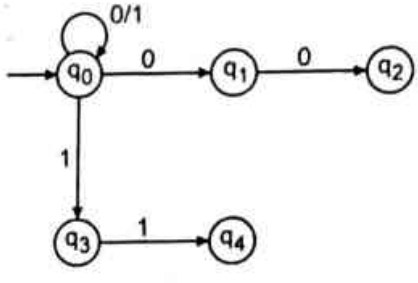


(b) Convert the given NFA with ∈-moves to NFA without ∈-moves and its equivalent DFA. **6M**



OR

Q.4.(a) Construct a deterministic finite automata equivalent to following NDFA. **6M**



(b) Construct a Moore machine equivalent to the Mealy machine represented by the transition table given below : **8M**

Present State \ Σ	Next State			
	a = 0		a = 1	
	State	Output	State	Output
q_1	q_1	1	q_2	0
q_2	q_4	1	q_4	0
q_3	q_2	0	q_3	1
q_4	q_3	0	q_1	0

Q.5.(a) Give regular expression and transition diagram for finite automata for the following regular sets of string over $\overline{\Sigma} = \{a, b\}$:

(i) All string beginning with a and ending with b.

(ii) All strings with at least two consecutive a.

(iii) All string containing the substring aba. **6M**

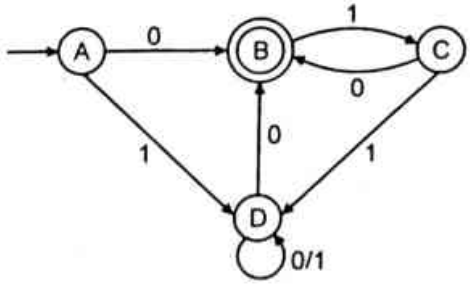
(b) Find whether the following grammar is ambiguous or not :

$S \rightarrow a \mid abSb \mid aAB$

$A \rightarrow bS \mid aAAB$

$B \rightarrow b \mid \in$. **4M**

(c) Obtain right linear grammar for the following DFA. **4M**



OR

Q.6.(a) Reduce the given grammar by eliminating useless symbols :

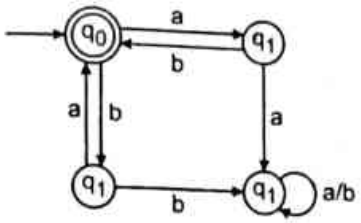
$S \rightarrow AB \mid CA$

$B \rightarrow BC \mid AB$

$A \rightarrow a$

$C \rightarrow aB \mid b$. **4M**

(b) Find the regular expression corresponding to the automata given in Fig. below : **6M**



(c) Find the grammar in CNF equivalent to :

$S \rightarrow aAbB$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$ **4M**

Q.7.(a) Give formal definition of push down automata and construct a PDA that is accepted by empty stack for the language :

$L = \{W \subset W^R \mid w \in (a, b)^*\}$ **8M**

(b) Construct PDA equivalent to the following grammar :

$S \rightarrow aAA$

$A \rightarrow aS \mid bS \mid a.$ **5M**

OR

Q.8.(a) Explain closure properties of CFLs. **5M**

(b) Prove that $L = \{a^i b^i c^i \mid i \geq 1\}$ is not a CFL. **8M**

Q.9.(a) Design Turing machine which accepts the language $L = \{a^n b^n c^n \mid n \geq 1\}$. **8M**

(b) What do you mean by Turing machine? Explain it with mathematical representation. **5M**

OR

Q.10.(a) Explain various types of Turing machine. **4M**

(b) Design a Turing machine that replaces all occurrences of 111 by 101 from a sequence of 0's and 1's. **9M**

Q.11(a) Define Ackerman Function. Compute :

A (1, 1), A (2, 1) and A (2, 2). **7M**

(b) What is post correspondence problems? Prove that PCP with two lists :

$X = (01, 1, 1)$

$Y = (01^2, 10, 1^2)$ has a solution or not. **6M**

OR

Q.12.(a) Explain what is meant by the term "Primitive recursive function". **5M**

(b) What are the closure properties of recursive and recursively enumerable language? **8M**

SUMMER EXAM.-2014

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Explain Chomsky hierarchy of languages. **8M**

Ans. See P.1-11, Q.22.

Q.1.(b) If R = {(1, 1), (2, 2), (3, 1), (1, 4), (2, 4)} on a set A = {1, 2, 3, 4}. Obtain R*, the reflexive transitive closure of R. **5M**

Ans. $R = \{(1, 1), (2, 2), (3, 1), (1, 4), (2, 4)\}$

Set $A = \{1, 2, 3, 4\}$

$\therefore R^2 = R.R$

$= \{(1, 1), (2, 2), (3, 1), (1, 4), (2, 4)\}.\{(1, 1), (2, 2), (3, 1), (1, 4), (2, 4)\}$

$= \{(3, 4)\}$

$\therefore R^3 = R^2.R$

$= \{(3, 4)\}.\{(1, 1), (2, 2), (3, 1), (1, 4), (2, 4)\}$

$= \phi$

$\therefore R^4 = R^5 = \phi$

$\therefore R^+ = R \cup R^2$

$= \{(1, 1), (2, 2), (3, 1), (1, 4), (2, 4), (3, 4)\}$

$\therefore R^* = R^+ \cup \{(1, 1), (2, 2), (3, 3), (4, 4)\}$

$= \{(1, 1), (2, 2), (3, 1), (3, 3), (1, 4), (2, 4), (3, 4) (4, 4)\}$

Q.2.(a) Prove the following by Principle of Induction :

(i) $1 + 2 + 3 + \ldots + n = n(n + 1)/2$

(ii) $\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$

Ans. (i) $1 + 2 + 3 + \ldots + n = n(n + 1)/2$:

(1) Basic step :

For n = 1

L. H. S. = 1 .....(1)

And R.H.S. = n (n + 1)/2

= 1 (1+1)/2

= 2/2 = 1 .....(2)

∴ Hence from (1) and (2)

L.H.S. = R. H. S.

(2) Induction Hypothesis :

Assume that for some $n \geq 1$

$$1 + 2 + 3 + ... + n = n\frac{(n+1)}{2}$$

(3) Induction set step :

For n = n + 1

∴ L.H.S.

$= 1 + 2 + 3 + ... + (n + 1)$

$= (1 + 2 + 3 + ... + n) + (n + 1)$

$$= \frac{n^2 + n}{2} + (n + 1)$$

$$= \frac{n^2 + n + 2n + 2}{2}$$

$$= \frac{n^2 + 3n + 2}{2}$$

$$= \frac{(n^2 + 2n + 1) + (n + 1)}{2}$$

$$= \frac{(n+1)^2 + (n+1)}{2}$$

R. H. S. for n = n + 1

$$= \frac{(n+1)(n+1+1)}{2}$$

$$= \frac{(n+1)(n+2)}{2}$$

$$= \frac{(n+1)^2 + (n+1)}{2} \quad ...$$

∴ L.H.S. = R. H. S.

(ii) $\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$ **8M**

Ans. See P.1-8, Q.11.

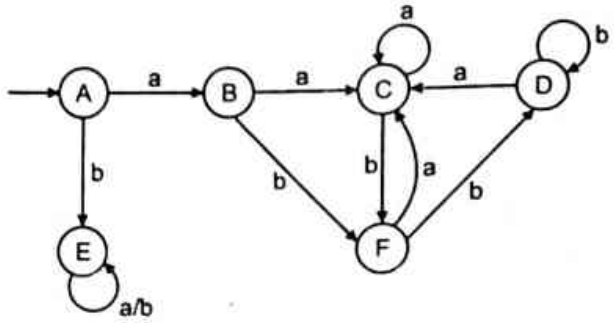
Q.2.(b) Explain Pigeonhole principle with example. **5M**

Ans. See P.1-10, Q.16.

UNIT - II

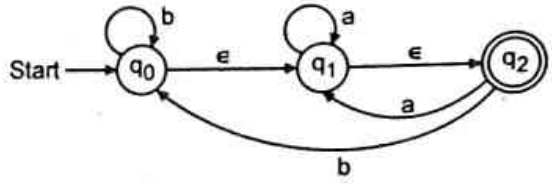
(Total Marks : 28)

Q.3.(a) Construct a minimum state automata equivalent to the finite automata given in Fig. below : **5M**



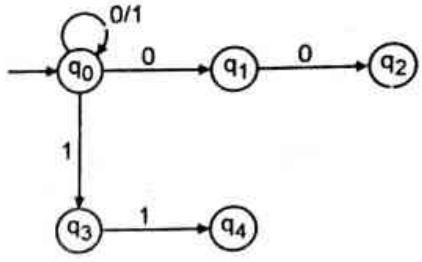
Ans. See P.2-35, Q.52 (example).

Q.3.(b) Convert the given NFA with ∈-moves to NFA without ∈-moves and its equivalent DFA. **6M**

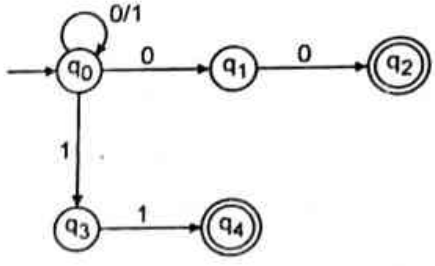


Ans. See P.2-26, Q.32.

Q.4.(a) Construct a deterministic finite automata equivalent to following NDFA. **6M**



Ans. Note : Final state is not given, so we assume q_2 and q_4 as final state.



Transition table of NFA :

Q \ Σ	0	1
→ q_0	q_0, q_1	q_0
q_1	q_2	-
* q_2	-	-
q_3	-	q_4
* q_4	-	-

Transition table of DFA :

Q \ Σ	0	1
q_0	q_0, q_1	q_0
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	q_0
* $\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	q_0

Rename states :

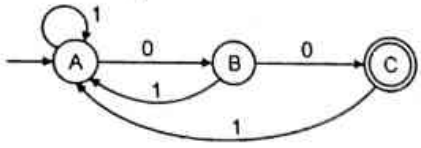
(1) $\{q_0\} = A$

$\{q_0, q_1\} = B$

$\{q_0, q_1, q_2\} = C$

Q \ Σ	0	1
A	B	A
B	C	A
* C	C	A

Transition diagram :



Q.4.(b) Construct a Moore machine equivalent to the Mealy machine represented by the transition table given below : **8M**

Present State \ Σ	Next State			
	a = 0		a = 1	
	State	Output	State	Output
q_1	q_1	1	q_2	0
q_2	q_4	1	q_4	0
q_3	q_2	0	q_3	1
q_4	q_3	0	q_1	0

Ans. See P.2-53, Q.71

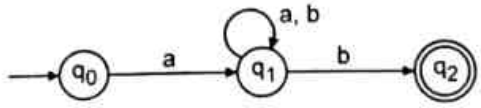
UNIT - III

(Total Marks : 41)

Q.5.(a) Give regular expression and transition diagram for finite automata for the following regular sets of string over $\overline{Z} = \{a, b\}$:

(i) All string beginning with a and ending with b.

Ans. Regular expression : a (a + b)* b



(ii) All strings with at least two consecutive a.

(iii) All string containing the substring aba. **6M**

Ans. See P.3-19, Q.23.

Q.5.(b) Find whether the following grammar is ambiguous or not :

S → a | abSb | aAB

A → bS | aAAB

B → b | ∈. **4M**

Ans. Consider the input string "abababb".

Left most Derivation :

S → a b S b

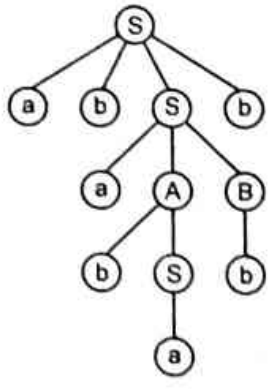
→ a b a A B b

→ a b a b S B b

→ a b a b a B b

→ a b a b a b b

Parse tree :



Also By left most derivation,

We get :

S → a A B

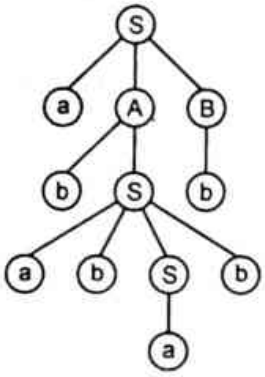
→ a b S B

$\rightarrow$ a b a b S b B

$\rightarrow$ a b a b a b B

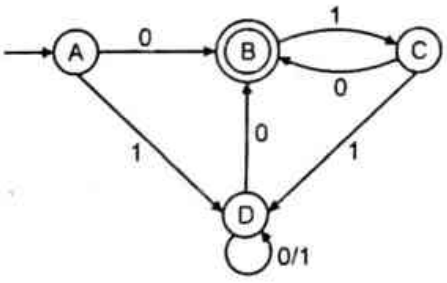
$\rightarrow$ a b a b a b b

Parse tree :



Hence, the string "abababb" has more than one parse tree. Hence, grammar is ambiguous.

Q.5.(c) Obtain right linear grammar for the following DFA. **4M**



Ans. Starting state of design becomes,

δ (A, 0) = B $\Rightarrow$ A $\rightarrow$ 0B

δ (A, 1) = D $\Rightarrow$ A $\rightarrow$ 1D

δ (B, 0) = D $\Rightarrow$ B $\rightarrow$ 0D

δ (B, 1) = C $\Rightarrow$ B $\rightarrow$ 1C

δ (C, 0) = B $\Rightarrow$ C $\rightarrow$ 0B

δ (C, 1) = D $\Rightarrow$ C $\rightarrow$ 1D

δ (D, 0) = D $\Rightarrow$ D $\rightarrow$ 1D

δ (A, 0) = B i.e. final state so A $\rightarrow$ 0 is also added.

δ (C, 0) = B i.e. final state, so C $\rightarrow$ 0 is also added.

$\therefore$ A $\rightarrow$ 0B | 1 D | 0

B $\rightarrow$ 0D | 1C

C $\rightarrow$ 0B | 1D | 0

$\therefore$ D $\rightarrow$ 0D | 1D

$\therefore$ D is useless symbol.

$\therefore$ After reduction techniques the grammar can be reduced as :

A $\rightarrow$ 0B | 0

B $\rightarrow$ 1C

C $\rightarrow$ 0B | 0

Q.6.(a) Reduce the given grammar by eliminating useless symbols :

S $\rightarrow$ AB | CA

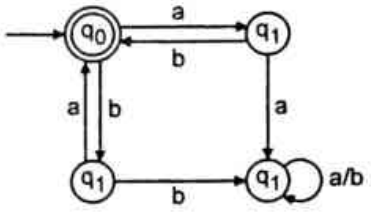
B $\rightarrow$ BC | AB

A $\rightarrow$ a

C $\rightarrow$ aB | b. **4M**

Ans. See P.3-73, Q.139.

Q.6.(b) Find the regular expression corresponding to the automata given in Fig. below : **6M**



Ans. See P.3-47, Q.68.

Q.6.(c) Find the grammar in CNF equivalent to :

S $\rightarrow$ aAbB

A $\rightarrow$ aA | a

B $\rightarrow$ bB | b

Ans. See P.3-82, Q.164.

Q.8.(a) Explain closure properties of CFLs. **5M**

Ans. See P.3-59, Q.98.

Q.8.(b) Prove that L = $\{a^i b^i c^i \mid i \geq 1\}$ is not a CFL. **8M**

Ans. See P.3-58, Q.97.

UNIT - IV

(Total Marks : 13)

Q.7.(a) Give formal definition of push down automata and construct a PDA that is accepted by empty stack for the language :

L = $\{W \subset W^R \mid w \in (a, b)^*\}$. **8M**

Ans. See P.4-15, Q.22.

Q.7.(b) Construct PDA equivalent to the following grammar :

$S \rightarrow aAA$

$A \rightarrow aS \mid bS \mid a.$ **5M**

Ans. Let the PDA equivalent to the given grammar is :

$M = (\{q\}, \{a, b\}, \{S, A\}, \delta, q, S, \phi)$

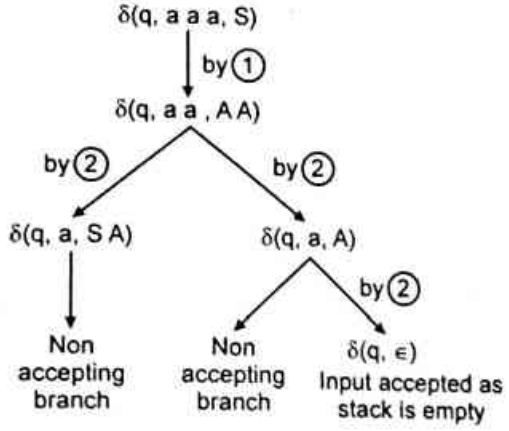
Where, f is defined by following rules.

$\delta(q, a, S) = \{(q, AA)\}$ because of $S \rightarrow a\,AA$ .....(1)

$\delta(q, a, A) = \{(q, S), (q, \in)\}$ because of $A \rightarrow aS$ and $A \rightarrow a$. .....(2)

$\delta(q, b, S) = \{(q, S)\}$ because of $A \rightarrow bS$. .....(3)

Let us check for string "aaa".



UNIT - V

(Total Marks : 26)

Q.9.(a) Design Turing machine which accepts the language $L = \{a^n b^n c^n \mid n \geq 1\}$. **8M**

Ans. See P.5-15, Q.10.

Q.9.(b) What do you mean by Turing machine? Explain it with mathematical representation. **5M**

Ans. See P.5-7, Q.1.

Q.10.(a) Explain various types of Turing machine. **4M**

Ans. See P.5-21, Q.19.

Q.10.(b) Design a Turing machine that replaces all occurrences of 111 by 101 from a sequence of 0's and 1's. **9M**

Ans. See P.5-17, Q.13.

UNIT - VI

(Total Marks : 26)

Q.11.(a) Define Ackerman Function. Compute : A (1, 1), A (2, 1) and A (2, 2). **7M**

Ans. See P.6-38, Q.68.

Q.11.(b) What is post correspondence problems? Prove that PCP with two lists :

$X = (01, 1, 1)$

$Y = (01^2, 10, 1^2)$ has a solution or not. **6M**

Ans. See P.6-15, 17, Q.18 & 21(a).

Q.12.(a) Explain what is meant by the term "Primitive recursive function". **5M**

Ans. See P.6-28, Q.42 & 44.

Q.12.(b) What are the closure properties of recursive and recursively enumerable language? **8M**

Ans. See P.6-7, 8, 9, Q.2, 4, 5, 6.

NOTES

WINTER EXAMINATION - 2014
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	26 M
Unit - V	30 M
Unit - VI	22 M
Questions covered in VBD	160 M (100 %)

WINTER EXAM. - 2014
QUESTION PAPER

Q.1.(a) With the help of Mathematical Induction prove that : **4M**

$$1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n \times (n+1)(2n+1)}{6}$$

(b) Explain the following terms : **6M**

(i) Null set (ii) Singleton set

(iii) Power set (iv) Subset

(v) Universal set (vi) Family of sets

(c) Describe the concept of pigeon hole principle with suitable example. **4M**

OR

Q.2.(a) What do you mean by the following terms:

(i) Prefix of a string.

(ii) Star closure of a language.

(iii) Positive closure of a language.

(iv) Proper prefixes and suffixes of a string. **4M**

(b) Explain chomsky hierarchy of formal languages. **6M**

(c) Explain the following terms with suitable examples : **4M**

(i) Closure of relation.

(ii) Reflexive closure.

(iii) Symmetric closure.

(iv) Transitive closure.

Q.3.(a) Construct a DFA which is equivalent to NFA

$m = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_3\})$

and transition δ is given in table. **6M**

Σ / Q	0	1
q_0	q_0q_1	q_0
q_1	q_2	q_1
q_2	q_3	q_3
q_3	– –	q_2

(b) Design a DFA for accepting

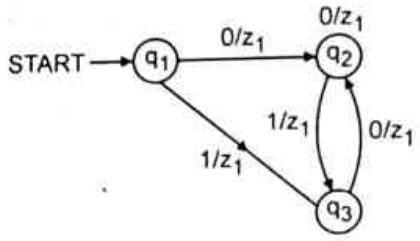
$$L = \{W \mid_{n(a)W \bmod 3 > n(b) \bmod 3}^{W \in (a/b)}$$

where, n(a) → No. of a's

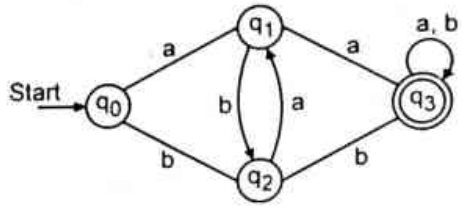
and n(b) → No. of b's **7M**

Q.4.(a) Design a moore and melay machine that will generate OUTPUT EVEN if the no. of a's are even and generate OUTPUT ODD if the no. of a's are ODD over $\Sigma = \{a, b\}$. **6M**

(b) Convert the given melay of equivalent moore machine : **7M**



Q.5.(a) Find the regular expression for the given transition diagram using Arden's theorem. **5M**



(b) Explain pumping lemma for regular sets. Show that

$L = \{0^i 1^i \mid i \geq 1\}$ is not Regular. **5M**

(c) Convert the grammar with productions : **4M**

$S \rightarrow ABa$

$A \rightarrow aab$

$B \rightarrow Ac$

to Chomsky normal form.

OR

Q.6.(a) Convert the following Grammar into GNF: **5M**

$S \rightarrow aAs$

$S \rightarrow a$

$A \rightarrow SbA$

$A \rightarrow SS$

$A \rightarrow ba$

(b) Obtain Left Linear Grammar equivalent to the following Right Linear Grammar : **5M**

$S \rightarrow 01A \mid 10$

$A \rightarrow 10A \mid 10$

(c) What is an ambiguous grammar? Check the ambiguity of the following grammar : **4M**

$S \rightarrow aSSb \mid bSSa \mid \epsilon$

Q.7.(a) Design PDA for the given language. Design the algorithm and transition diagram for the same : **8M**

$L = \{a^n b^{2n} \mid n \geq 1\}$

(b) Design a PDA for the following LPG : **5M**

$S \rightarrow XY$

$X \rightarrow AX \mid BX \mid a$

$Y \rightarrow YA \mid YB \mid ap$

$A \rightarrow a$

$B \rightarrow b$

OR

Q.8.(a) Construct a PDA accepting the

$\alpha = \{a^n b^m a^n \mid m, n \geq 1\}$

by null store. Construct a corresponding context free grammar accepting the same set. **8M**

(b) Construct a CFG which generate the following language : **5M**

$L = \{a^n b^m c^m d^n \mid m, n \geq 1\}$

and also design a PDA for above CFG

Q.9.(a) Describe the concept of Chruch's Hypothesis. **3M**

(b) Design a turing machine for (n + 1)/2 where n is a uninary number. **6M**

(c) Explain the concept of Linear Bounded Automata. **4M**

Q.10.(a) Design a TM that has

INPUT : # W # & generate O/P as

OUTPUT : # W # W **8M**

(b) Explain the following terms : **5M**

(i) Multitape Turing Machine

(ii) Universal Turing Machine.

Q.11.(a) Define Ackermann's Function. Compute A(1, 1), A (2, 1) and A(2, 2). **5M**

(b) Explain post correspondence problem. Prove that PCP with two list :

$X = (01, 1, 1)$

$Y = (01^2, 10, 1^2)$

Has no solution. **5M**

(c) What is meant by undecidability? **3M**

OR

Q.12.(a) "Every function in Primitive Recursive class is Turing Computable." Comment on the statement. **3M**

(b) Explain the properties of recursive and recursively enumberable language. **6M**

(c) Describe the concept of halting problem **4M**

WINTER EXAM.-2014 SOLUTION (Unit-Wise)

UNIT - I

(Total Marks : 28)

Q.1.(a) With the help of Mathematical Induction prove that : **4M**

$$1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n \times (n+1)(2n+1)}{6}$$

Ans. See. P.1-8, Q.11.

Q.1.(b) Explain the following terms : **6M**

(i) Null set :

Ans. The set that contain no element is called null set and is denoted by ϕ.

For example : $P = \{x : x^2 = 4, x \text{ is odd}\}$

(ii) Singleton set :

Ans. The set that contain only one element is called singleton set.

For example : A = {2} is a singleton set.

(iii) Power set :

Ans. See. P.1-4, Q.2(viii).

(iv) Subset :

Ans. If every element of set A is also the element of set B then A is called the subset of B and it is written as $A \subseteq B$. Here B is called superset of A.

(v) Universal set :

Ans. A universal set is a collection of all objects in a particular context or theory. All other sets is that framework constitute subset of the universal set, it is denote by letter U.

(vi) Family of sets :

Ans. If a set A contains elements which are itself sets, then it is called family of sets or a set of sets.

For example : P = {{1, b} {r}}

Here, P is a set of sets.

Q.1.(c) Describe the concept of pigeon hole principle with suitable example. **4M**

Ans. See. P.1-10, Q.16.

Q.2.(a) What do you mean by the following terms

(i) Prefix of a string

Ans. See. P.1-4, Q.2(iv).

(ii) Star closure of a language

Ans. See. P.1-11, Q.20.

(iii) Positive closure of a language

Ans. See. P.1-11, Q.20.

(iv) Proper prefixes and suffixes of a string. **4M**

Ans. See. P.1-4, Q.2(v), (vii).

Q.2.(b) Explain chomsky hierarchy of formal languages. **6M**

Ans. See. P.1-11, Q.22.

Q.2.(c) Explain the following terms with suitable examples :

(i) Closure of relation.

(ii) Reflexive closure.

(iii) Symmetric closure.

(iv) Transitive closure. **4M**

Ans. See. P.1-5, Q.4.

UNIT - II
(Total Marks : 26)

Q.3.(a) Construct a DFA which is equivalent to NFA

$m = (\{q_0, q_1, q_2, q_3\}, \{0,1\}, \delta, q_0, \{q_3\})$ and transition δ is given in table **6M**

Q \ Σ	0	1
q_0	q_0q_1	q_0
q_1	q_2	q_1
q_2	q_3	q_3
q_3	- -	q_2

Ans. See. P.2-19, Q.22.

Q.3.(b) Design a DFA for accepting

$$L = \{W \mid {}^{W\in(a,b)}_{n(a)W\,MOD\,3 > n(b)MOD\,3}\}$$

where, n(a) → No. of a's

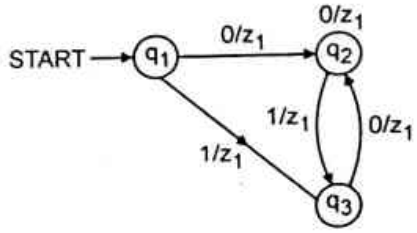
and n(b) → No. of b's **7M**

Ans. See. P.2-33, Q.48.

Q.4.(a) Design a moore and melay machine that will generate OUTPUT EVEN if the no. of a's are even and generate OUTPUT ODD if the no. of a's are ODD over Σ = {a, b}. **6M**

Ans. See. P.PS-12, Q.4.(b).

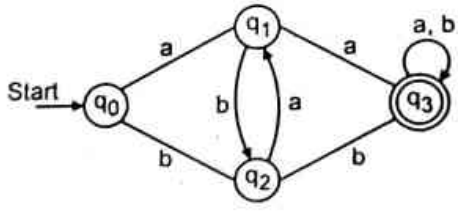
Q.4.(b) Convert the given melay of equivalent moore machine : **7M**



Ans. See. P.2-56, Q.78.

UNIT - III
(Total Marks : 28)

Q.5.(a) Find the regular expression for the given transition diagram using Arden's theorem. **5M**



Ans. See. P.3-50, Q.76.

Q.5.(b) Explain pumping lemma for regular sets. Show that

$L = \{0^i 1^i \mid i \geq 1\}$ is not Regular. **5M**

Ans. See. P.3-54, Q.84.

Q.5.(c) Convert the grammar with productions : **4M**

S → ABa

A → aab

B → Ac

to Chomsky normal form.

Ans. See. P.3-81, Q.161.

Q.6.(a) Convert the following Grammar into GNF :

S → aAs

S → a

A → SbA

A → SS

A → ba **5M**

Ans. S → aAs/a

A → SbA/SS/ba

Here, S → a already in GNF

∴ S → aAS/a

A → abA/aS/bS

Add B → b

∴ S → aAs/a

A → aBA/aS/bS

B → b.

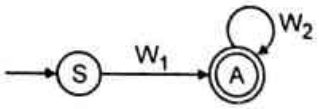
Q.6.(b) Obtain Left Linear Grammar Equivalent to the following Right Linear Grammar : **5M**

S → 01A | 10

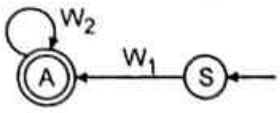
A → 10A | 10

Ans. Put $w_1 = 01$

& $w_2 = 10$



Now exchange the position of initial and final states and reverse the direction of all transitions.



$S \rightarrow A\, w_1 / w_1$

$A \rightarrow 1A\, w_2 / w_2$

∴ S → A01 10

A → A10/10

This is left linear Grammer.

Q.6.(c) What is an ambiguous grammar? Check the ambiguity of the following grammar : **4M**

S → aSSb | bSSa | ∈

Ans. **See. P.3-66, 68, Q.120, 127.**

UNIT - IV
(Total Marks : 26)

Q.7.(a) Design PDA for the given language. Design the algorithm and transition diagram for the same :

$L = \{a^n b^{2n} \mid n \geq 1\}$ **8M**

Ans. **See. P.4-9, Q.12.**

Q.7.(b) Design a PDA for the following LPG : **5M**

S → XY

X → AX | BX | a

Y → YA | YB | ap

A → a

B → b

Ans. **See. P.4-25, Q.36.**

Q.8.(a) Construct a PDA accepting the

$\alpha = \{a^n b^m a^n \mid m, n \geq 1\}$

by null store. Construct a corresponding context free grammar accepting the same set. **8M**

Ans. **See. P.4-17, Q.27.**

Q.8.(b) Construct a CFG which generate the following language

$L = \{a^n b^m c^m d^n \mid m, n \geq 1\}$

and also design a PDA for above CFG. **5M**

Ans. **See. P.4-17, Q.25.**

UNIT - V
(Total Marks : 30)

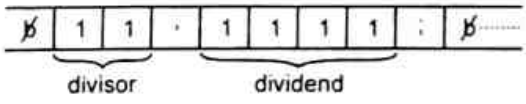
Q.9.(a) Describe the concept of Chruch's hypothesis. **3M**

Ans. **See. P.5-30, Q.31.**

Q.9.(b) Design a turing machine for (n + 1)/2 where n is a uninary number. **6M**

Ans. T.M. (n + 1)/2 n is uninary number :

- Let us represent the number in many format with the help of letter 1 e.g. "3" can be represented as "111" in unary format.
- Let us assumed that two numbers are separated by ',' and divisior is to be written first on the tape followed by dividend.
- As we know that, division can be represented as the repetitive subtraction of divisior from dividend.
- While subtracting replace each '1' of divisor by a and that of dividend by 'b'.
- After subtracting once again replace a's of division by all '1' and repeat the process till you cannot subtract again. Then halt and number of 1's remaining on tape will give you the exact value of the remainder.



S	1	,	;	Ƀ	a	b
q_0	q_1, a, R	q_4L				
q_1	R	q_2R				
q_2	q_3b, L	-	q_5b, L	-	-	R
q_3	L	L	-	-	q_0R	L
q_4	-	-	-	q_0R	1L	-
q_5	L	L	-	q_0R	L	Ƀ L
q_6	q_7N	q_7N	-	-	Ƀ R	-
q_7	-	-	-	-	-	-

Q.9.(c) Explain the concept of Linear Bounded Automata. **4M**

Ans. See. P.5-34, Q.40.

Q.10.(a) Design a TM that has

INPUT : # W # & generate O/P as

OUTPUT : # W # W **8M**

Ans. $\delta(q_0, \# W \# W) \vdash (\# W \# W$

$\uparrow$

q_0

$\vdash (* W \# W)$

$\uparrow$

q_1

$\vdash (** \# W)$

$\uparrow$

q_0

$\vdash (*** W)$

$\uparrow$

q_1

$\vdash (**** Ƀ)$

$\uparrow$

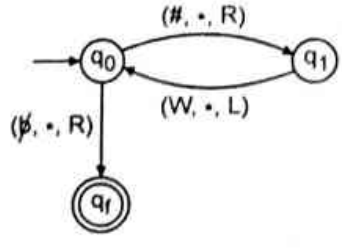
q_0

$\vdash (q)$

Transition Table :

Present state	Tape symbol
	# W * Ƀ
$\rightarrow q_0$	$(q_1, *, R)$ $(q_0, *, R)$ $(q_1, Ƀ, R)$
q_1	$(q_0, *, L)$
q_f	- - -

Transition Diagram :



Q.10.(b) Explain the following terms **5M**

(i) Multitape Turing Machine.

Ans. See. P.5-21, Q.19(2).

(ii) Universal Turing Machine

Ans. See. P.5-27, Q.26(A).

Q.12.(c) Describe the concept of halting problem **4M**

Ans. See. P.5-26, 28, Q.24, 27.

UNIT - VI

(Total Marks : 22)

Q.11.(a) Define Ackermann's Function. Compute A(1, 1), A (2, 1) and A(2, 2) **5M**

Ans. See. P.6-38, Q.68.

Q.11.(b) Explain post correspondence problem. Prove that PCP with two list

$X = (01, 1, 1)$

$Y = (01^2, 10, 1^2)$

Has no solution **5M**

Ans. See. P.6-17, Q.21.

Q.11.(c) What is meant by undecidability? **3M**

Ans. See. P.6-14, Q.16(a).

Q.12.(a) "Every function in Primitive Recursive class is Turing Computable." Comment on the statement. **3M**

Ans. See. P.6-31, Q.52.

Q.12.(b) Explain the properties of recursive and recursively enumberable language. **6M**

Ans. See. P.6-7, 8, Q.2, 3.

Notes :-

SUMMER EXAMINATION - 2015
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	28 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

SUMMER EXAM. - 2015
QUESTION PAPER

Q.1.(a) Explain Chomsky hierarchy of language in detail. **7M**

(b) Explain pigeon-hole principle. **6M**

OR

Q.2.(a) Explain the following terms : **6M**

(i) String

(ii) Prefix of string

(iii) Suffix of string

(iv) Star closure of string

(v) Plus (+) closure of string

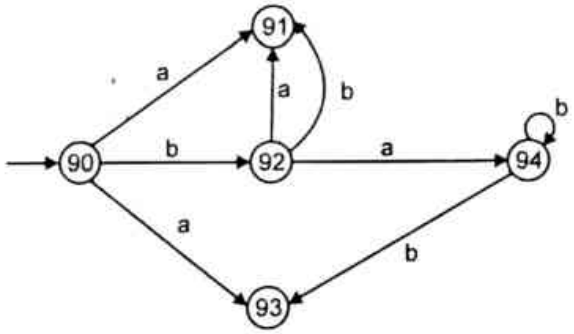
(vi) Power set

(b) Prove the given relation using principle of induction. $1 . 2 + 2 . 3 + 3 . 4 + \ldots\ldots + n(n+1)(n+2) = \frac{n(n+1)(n+2)(n+3)}{4}$ **5M**

(c) Define countability and diagonalization. **2M**

Q.3.(a) What is finite automata? Also, explain its applications. **6M**

(b) Convert the following NFA into equivalent DFA.



7M

OR

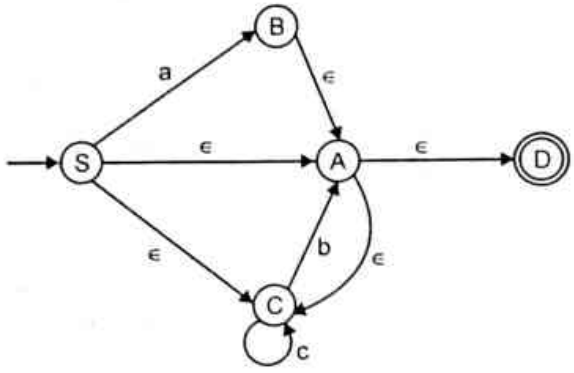
Q.4.(a) Differentiate between Mealy and Moore machine. **4M**

(b) Minimize the following DFA :

Σ / Q	a	b
q_0	q_1	q_0
q_1	q_0	q_2
q_2	q_3	q_1
(q3)	q_3	q_0
q_4	q_3	q_5
q_5	q_6	q_4
q_6	q_5	q_6
q_7	q_6	q_3

6M

(c) Calculate ϵ-closure of all state of following NFA. **3M**



Q.5. Write a regular expression for following language on {0 , 1} :

(i) All strings not ending with "01"

(ii) All strings which should not contain "101" as substring **14M**

OR

Q.6.(a) Convert the following left linear grammar into right linear grammar :

$S \rightarrow A0 \mid A1$

$A \rightarrow B00$

$B \rightarrow B0 \mid B1 \mid 0 \mid 1$. **5M**

(b) Reduce the following grammar :

$S \rightarrow aA \mid aBB$

$A \rightarrow aaA \mid \epsilon$

$B \rightarrow bB \mid bbC$

$C \rightarrow B$. **5M**

(c) Convert the grammar into CNF :

$S \rightarrow ABa$

$A \rightarrow aab$

$B \rightarrow AC$. **4M**

Q.7. Design a PDA for following. Also, construct CFG from designed PDA :

$L\{a^n b^m c^n d^m \mid m, n \geq 1\}$ **14M**

VBD

OR

Q.8.(a) Explain closure property of CFL. **5M**

(b) Construct a NPDA that accepts the language generated by the grammar

S → aSbb | aab

(c) Explain the model of PDA. **4M**

Q.9.(a) Explain linear bounded automata. **6M**

(b) Design a turing machine for following language

I. $\{a^n b^n c^n \; n \geq 1\}$ **7M**

OR

Q.10.(a) Explain types of turing machine. **6M**

(b) Design a Turing Machine to multiply two unary numbers. **7M**

Q.11.(a) Explain the properties of recursively enumerable set. **7M**

(b) Compute A(1, 1) A(1, 2) A(2, 1) using ackerman function. **6M**

OR

Q.12.(a) What is post correspondence problem and solve the following :

Sr. No.	List A	List B
(1)	01	011
(2)	1	10
(3)	1	11

6M

(b) Show that the following functions are primitive recursive :

$$f(x) = \begin{cases} x/2 & \text{when x is given} \\ (x-1)/2 & \text{when x is odd} \end{cases}$$ **7M**

SUMMER EXAM. - 2015

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Explain Chomsky hierarchy of language in detail. **7M**

Ans. See P.1-11, Q.22

Q.1.(b) Explain pigeon-hole principle. **6M**

Ans. See P.1-10, Q.16

Q.2.(a) Explain the following terms : **6M**

(i) String

(ii) Prefix of string

(iii) Suffix of string

(iv) Star closure of string

(v) Plus (+) closure of string

(vi) Power set

Ans. See P.1-4, Q.2(iii), (iv), (vi), (viii)

(iv) **Star closure of string :**

The set of all string over an alphabet Σ is called star closure of string and it is denoted by Σ^*

e.g. $\Sigma = \{0, 1\}$

$\Sigma^* = \{\epsilon, 0, 1, 00, 10, 11, 000 \ldots\ldots\}$

(v) **Plus (+) closure of string :**

The set of all non empty string over an alphabet Σ it is denoted by

Σ^+

$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \ldots\ldots$

$\Sigma^+ = \Sigma^* - \{\epsilon\}$

Q.2.(b) Prove the given relation using principle of induction $1 \cdot 2 \cdot 3 + 2 \cdot 3 \cdot 4 + \ldots\ldots + n(n+1)(n+2) = \frac{n(n+1)(n+2)(n+3)}{4}$ **5M**

Ans. See P.1-9, Q.15

Q.2.(c) Define countability and diagonalization. **2M**

Ans. See P.1-5, Q.5(1), (2)

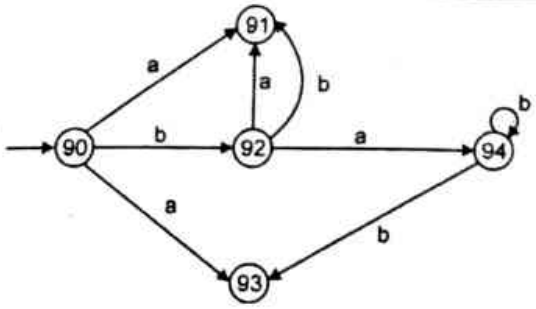
UNIT - II

(Total Marks : 26)

Q.3.(a) What is finite automata? Also, explain its applications. **6M**

Ans. See P.2-8 & 10, Q.1 & 4

Q.3.(b) Convert the following NFA into equivalent DFA. **7M**



Ans. Step 1 : NFA = ({90, 91, 92, 93}, {a, b}, δ , {90}, {94})

Transition table for NFA :

Σ / Q	a	b
→ 90	91, 93	92
91		-
92	91, 94	91
93	-	-
*94	-	93, 94

Step 2 : DFA (Σ) = NFA (Σ)

DFA (90) = NFA (90)

Step 3 : $Q' = 2^Q$, Q = 5

$\therefore 2^5 = 32$

$\therefore Q' = 32$

Step 4 : Transition table for DFA :

Σ / Q	a	b
{90}	{91, 93}	{92}
{91, 93}	–	–
{92}	{91, 94}	{91}
{91}	–	–
*{91, 94}	–	{93, 94}
*{93, 94}	–	{93, 94}

Step 5 :

Rename

90 → A {91, 94} → D

{91, 93} → B {91} → E

{92} → C {93, 94} → F

Transition table :

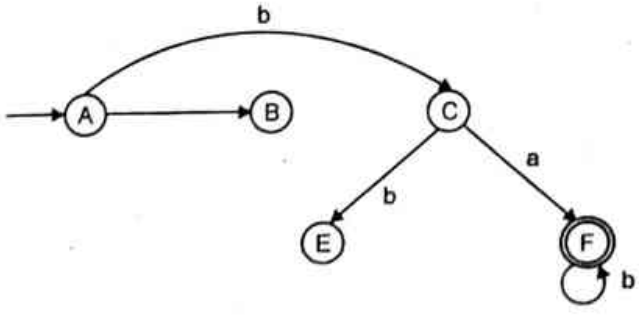
Σ / Q	a	b
A	B	C
B	-	-
C	D	E
*D	-	F
E	-	-
*F	-	F

Here D = F so, after optimization the transition table is

Transition table :

Σ / Q	a	b
A	B	C
B	-	-
C	F	E
*F	-	F
E	-	-

Step 6 : Transition diagram :



Q.4.(a) Differentiate between Mealy and Moore machine.

Ans. See P.2-49, Q.64

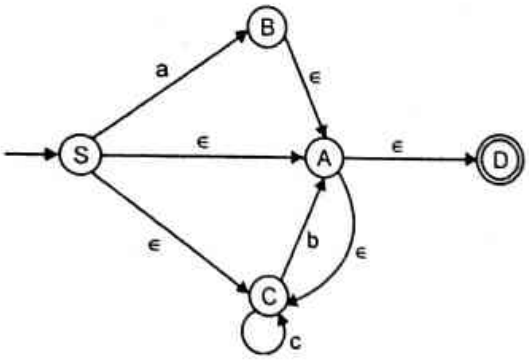
Q.4.(b) Minimize the following DFA

Q \ Σ	a	b
q_0	q_1	q_0
q_1	q_0	q_2
q_2	q_3	q_1
(q_3)	q_3	q_0
q_4	q_3	q_5
q_5	q_6	q_4
q_6	q_5	q_6
q_7	q_6	q_3

6M

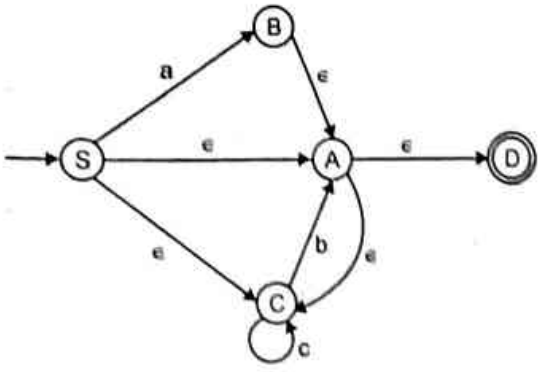
Ans. See P.2-41, Q.58

Q.4.(c) Calculate ∈-closure of all state of following NFA.



3M

Ans.



Transition table :

Q \ Σ	a	b	c	∈
→ S	B	-	C	A
B	-	A	-	-
C	-	A	C	-
A	-	-	-	D,C
*D	-	-	-	-

∈ - closure (S) = {S, A, C, D}

∈ - closure (B) = {B}

∈ - closure (C) = {C}

∈ - closure (A) = {A, C, D}

∈ - closure (D) = {D}

(1) $\delta'(S, a) = \in$ - closure $(\delta(\hat{\delta}(S, \in), a))$

$= \in$ - closure $(\delta(S, A, C, D), a))$

$= \in$ - closure $(\delta(S, a) \cup \delta(A, a) \cup \delta(C, a) \cup \delta(D, a))$

$= \in$ - closure (B)

= B

$\delta'(S, b) = \in$ - closure $(\delta(\hat{\delta}(S, \in), b))$

$= \in$ - closure $(\delta(S, A, C, D), b))$

$= \in$ - closure (A)

= (A, C, D)

$\delta'(S, c) = \in$ - closure $(\delta(\hat{\delta}(S, \in), c))$

$= \in$ - closure $(\delta(S, A, C, D), c))$

$= \in$ - closure (C)

= {C}

(2) $\delta'(B, a) = \in$ - closure $(\delta(\hat{\delta}(B, \in), a))$

$= \in$ - closure $(\delta(B, a))$

$= \in$ - closure (ϕ)

$= \phi$

$\delta'(B, b) = \in$ - closure $(\delta(\hat{\delta}(B, \in), b))$

$= \in$ - closure $(\delta(B, b))$

$= \in$ - closure (A)

= {A, C, D}

$\delta'(B, c) = \epsilon$ - closure $(\delta(\hat{\delta}(B, \epsilon), c))$

$=$ ϵ - closure $(\delta(B, c))$

$= \phi$

(3) $\delta'(C, a) = \epsilon$ - closure $(\delta(\hat{\delta}(C, \epsilon), a))$

$=$ ϵ - closure $(\delta(C, a))$

$=$ ϵ - closure (ϕ)

$= \phi$

$\delta'(C, b) = \epsilon$ - closure $(\delta(\hat{\delta}(C, \epsilon), b))$

$=$ ϵ - closure $(\delta(C, b))$

$=$ ϵ - closure (A)

$= \{A, C, D\}$

$\delta'(C, c) = \epsilon$ - closure $(\delta(\hat{\delta}(C, \epsilon), c))$

$=$ ϵ - closure $(\delta(C, c))$

$=$ ϵ - closure (C)

$= \{C\}$

(4) $\delta'(A, a) = \epsilon$ - closure $(\delta(\hat{\delta}(A, \epsilon), a))$

$=$ ϵ - closure $(\delta(A, C, D), a))$

$=$ ϵ - closure (ϕ)

$= \phi$

$\delta'(A, b) = \epsilon$ - closure $(\delta(\hat{\delta}(A, \epsilon), b))$

$=$ ϵ - closure $(\delta(A, C, D), b))$

$=$ ϵ - closure (A)

$= \{A, C, D\}$

$\delta'(A, c) = \epsilon$ - closure $(\delta(\hat{\delta}(A, \epsilon), c))$

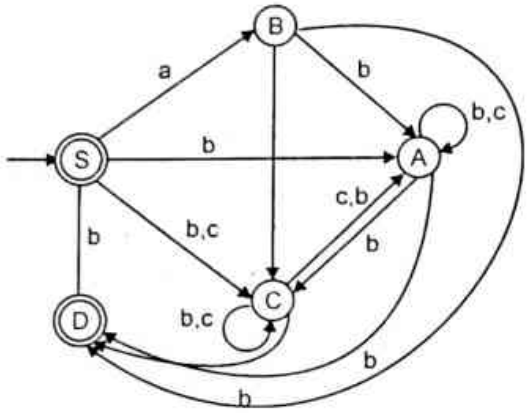
$=$ ϵ - closure $(\delta(A, C, D), c))$

$=$ ϵ - closure (C)

$= \{C\}$

Transition table :

Q \ Σ	a	b	c
*S	{B}	{A, C, D}	{C}
*A	ϕ	{A, C, D}	{C}
B	ϕ	{A, C, D}	ϕ
*C	ϕ	{A, C, D}	{C}
D	ϕ	ϕ	ϕ



UNIT - III

(Total Marks : 28)

Q.5. Write a regular expression for following language on {0 , 1} :

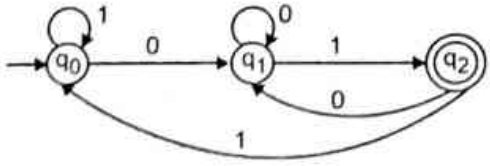
(i) All strings not ending with "01".

(ii) All strings which should not contain "101" as substring.

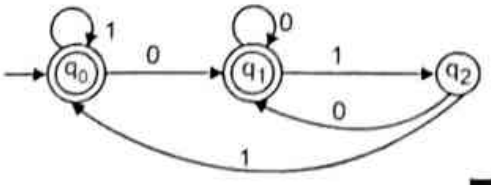
14M

Ans.

(i) String ending with 0 1 :



Interchange the position state final to non final and non final to final.



Regular expression

$q_0 = 1 q_0 + 1 q_2 + \epsilon$ ...(1)

$q_1 = 0 q_0 + 0 q_1 + 0 q_2$ ...(2)

$q_2 = 1 q_1$ ...(3)

Put equation (3) in equation (2)

$q_1 = 0 q_0 + 0 q_1 + 0 1 q_1$

$= 0 q_0 + (0 + 01) q_1$

by Arden's theorem

$q_1 = 0 q_1 (0 + 01)^*$ ...(4)

Put in equation (3), we get,

$q_2 = 1 0 (0 + 01)^* q_0$

Put in equation (1)

$q_0 = 1 q_0 + 10 (0 + 01)^* q_0 + \epsilon$

$= q_0 (1 + 10 (0 + 01)^* + \epsilon$

By Arden's theorem

$q_0 = [(1 + 10 (0 + 01)^*]^*$ ...(5)

Put equation (5) in equation (4)

$q_1 = 0 (0 + 01)^* [1 + 10 (0 + 10)^*]^*$

∴ Regular expression

$q_0 + q_1$

$= [(1 + 10 (0 + 01)^*] + 0 (0 + 01)^* [1 + 10 (0 + 10)^*]^*$

$= 0 (0 + 10)^* [(1 + 10 (0 + 01)^*]^*$

(2) All strings which should not contain "101" as substring

See P. 3-23, Q.30.

Q.6.(a) Convert the following left linear grammar into right linear grammar :

S → A0 | A1

A → B00

B → B0 | B1 | 0 | 1. **5M**

Ans. Let W = 0 0

Transition graph for left linear grammar :

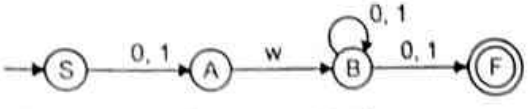
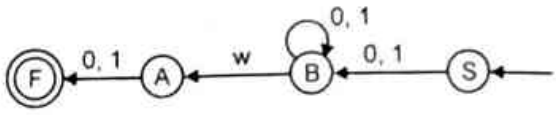


Fig. Transition graph for right linear grammar

Transition graph for right linear grammar :



Right linear grammar :

S → 0B | 1B

A → 0 | 1

B → 0B | 1B | wA

∴ S → 0B | 1B

A → 0 | 1

B → 0B | 1B | 0 0 B

Q.6.(b) Reduce the following grammar

S → aA | aBB

A → aaA | ∈

B → bB | bbC

C → B. **5M**

Ans. **See P.3-80, Q.156**

Q.6.(c) Convert the grammar into CNF :

S → ABa

A → aab

B → AC. **4M**

Ans. Consider production :

(1) S → ABa

Replace $R_1 \rightarrow a$

∴ $S \rightarrow AB R_1$

∴ Replace $R_2 \rightarrow AB$

∴ $S \rightarrow R_2 R_1$

(2) A → aab

$A \rightarrow R_1 R_1 b$

Replace $R_3 \rightarrow b$

∴ $A \rightarrow R_1 R_1 R_3$

∴ Replace $R_4 \rightarrow R_1 R_1$

∴ $A \rightarrow R_4 R_3$

(3) B → AC are already in CNF form

Grammar in CNF form is

$S \rightarrow R_2 R_1$

$A \rightarrow R_4 R_3$

$B \rightarrow AC$

$R_1 \rightarrow a$

$R_2 \rightarrow AB$

$R_3 \rightarrow b$

$R_4 \rightarrow R_1 R_1$

UNIT - IV

(Total Marks : 28)

Q.7. Design a PDA for following. Also, construct CFG from designed PDA :

I. $\{a^n b^m c^e d^m m, n \geq 1\}$. **[4M]**

Ans. Let n = 1, m = 1

$L_1 = \{a\ b\ c\ d\}$

Let n = 1, m = 2

$L_2 = \{a\ bb\ c\ dd\}$

Let n = 2, m = 1

$L_3 = \{aa\ b\ cc\ d\}$

Valid string : {a b c d, a b b c d d, a a b c c d ....}

String has fixed length.

String starts with a and ends on d. Equal no of a and c to equal no of b and d.

Algorithm :

Push all 'a' on to the stack. When read 'b' and stack top symbol is 'a' only change the state. When read 'c' pop all 'a' from the stack and when read 'd' only change the state.

Transition :

(1) $\delta(q_0, a, Z_0) \rightarrow (q_0, a\ Z_0)$

(2) $\delta(q_0, a, a) \rightarrow (q_0, a\ a)$

(3) $\delta(q_0, b, a) \rightarrow (q_1, a)$

(4) $\delta(q_1, c, a) \rightarrow (q_1, \epsilon)$

(5) $\delta(q_1, d, Z_0) \rightarrow (q_2, Z_0)$

(6) $\delta(q_2, \epsilon, Z_0) \rightarrow (q_1, \epsilon)$

CFG for PDA :

The production of an equivalent CFG are obtained as follows :

$S \rightarrow [q_0, Z_0, q_0]$

$S \rightarrow [q_0, Z_0, q_1]$

$S \rightarrow [q_0, Z_0, q_2]$

For move $\delta(q_0, a, Z_0) \rightarrow (q_0, a, Z_0)$

$[q_0, Z_0, q_0] \rightarrow a\,[q_0, a, q_0][q_0, Z_0, q_0]$

$[q_0, Z_0, q_0] \rightarrow a\,[q_0, a, q_1][q_1, Z_0, q_0]$

$[q_0, Z_0, q_1] \rightarrow a\,[q_0, a, q_0][q_0, Z_0, q_1]$

$[q_0, Z_0, q_1] \rightarrow a\,[q_0, a, q_1][q_1, Z_0, q_1]$

$[q_0, Z_0, q_2] \rightarrow a\,[q_0, a, q_0][q_0, Z_0, q_2]$

$[q_0, Z_0, q_2] \rightarrow a\,[q_0, a, q_2][q_2, Z_0, q_2]$

For move $\delta(q_0, a, a) = (q_0, a, a)$

$[q_0, a, q_0] \rightarrow a\,[q_0, a, q_0][q_0, a, q_0]$

$[q_0, a, q_0] \rightarrow a\,[q_0, a, q_1][q_1, a, q_0]$

$[q_0, a, q_1] \rightarrow a\,[q_0, a, q_0][q_0, a, q_1]$

$[q_0, a, q_1] \rightarrow a\,[q_0, a, q_1][q_1, a, q_1]$

$[q_0, a, q_2] \rightarrow a\,[q_0, a, q_0][q_0, a, q_2]$

$[q_0, a, q_2] \rightarrow a\,[q_0, a, q_2][q_2, a, q_2]$

For move $\delta(q_0, b, a) \rightarrow (q_1, a)$

$[q_0, a, q_0] \rightarrow a\,[q_1, a, q_0]$

$[q_0, a, q_1] \rightarrow a\,[q_1, a, q_1]$

For move $\delta(q_1, c, a) \rightarrow (q_1, \epsilon)$

$[q_1, a, q_1] \rightarrow C$

For move $\delta(q_1, d, Z_0) \rightarrow (q_2, Z_0)$

$[q_1, Z_0, q_1] \rightarrow d\,[q_2, Z_0, q_1]$

$[q_1, Z_0, q_2] \rightarrow d\,[q_2, Z_0, q_1]$

For move $\delta(q_2, \epsilon, Z_0) = (q_2, \epsilon)$

$[q_2, Z_0, q_2] \rightarrow \epsilon$

If we rename the object as

Object	Name
$[q_0, Z_0, q_0]$	A
$[q_0, Z_0, q_1]$	B
$[q_0, Z_0, q_2]$	C
$[q_1, Z_0, q_0]$	D
$[q_1, Z_0, q_1]$	E
$[q_2, Z_0, q_2]$	F
$[q_0, a, q_0]$	G
$[q_0, a, q_1]$	H
$[q_1, a, q_1]$	I
$[q_1, a, q_0]$	J
$[q_0, a, q_2]$	K
$[q_2, a, q_2]$	L
$[q_2, Z_0, q_1]$	M
$[q_1, Z_0, q_2]$	N

Then the production can be written as

S → A | B | C

A → aGA | aHD

B → aGB | aHE

C → aGC | aKF

G → bJ | bI

I → C

E → d M

N → dM

F → e

M and J is useless symbol.

S → A | B | C

A → aGA | aHD

B → aGB | aHE

C → aGC | aKF

G → bI

I → C

F → ∈

Q.8.(a) Explain closure property of CFL. **5M**

Ans. See P.4-26, Q.40

Q.8.(b) Construct a NPDA that accepts the languages generated by the grammar

S → aSbb | aab. **5M**

Ans. S → aSbb | aab

Valid string : {a a b, aaabbb, aaaabbbb ....}

$L = \{a^n b^m \mid n \geq 2, m \geq 1\}$

The NPDA :

Algorithm :

Push all 'a' onto the stack. When read 'b' pop 'a' onto the stack or either only change the state. When read 'b' and stack top symbol is Z_0 reaches final state.

Transition :

$\delta(q_0, a, Z_0) = (q_0, a\, Z_0)$

$\delta(q_0, a, a\, Z_0) = (q_0, aa)$

$\delta(q_0, b, a) = (q_1, a) / (q_0, \epsilon)$

$\delta(q_0, \epsilon, Z_0) = (q_f, Z_0)$

$\delta(q_1, b, a) = (q_1, \epsilon)$

$\delta(q_1, b, Z_0) = (q_f, \epsilon)$

Q.8.(c) Explain the model of PDA. **4M**

Ans. See P.4-5, Q.2

UNIT - V

(Total Marks : 26)

Q.9.(a) Explain linear bounded automata. **6M**

Ans. See P.5-34, Q.40

Q.9.(b) Design a turing machine for following language.

$L = \{a^n b^n c^n \mid n \geq 1\}$ **7M**

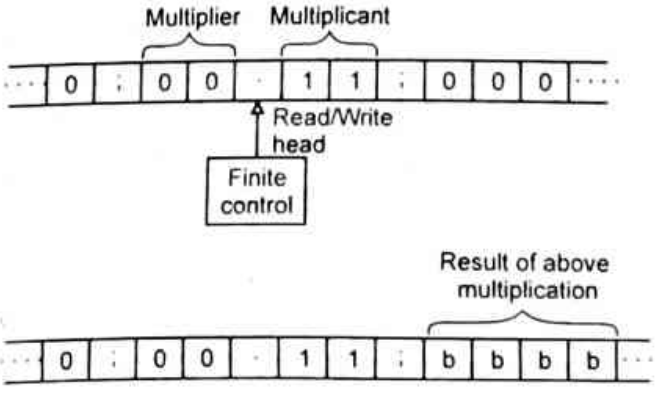
Ans. See P.5-15, Q.10

Q.10.(a) Explain types of turing machine. **6M**

Ans. **See P.5-21, Q.19**

Q.10.(b) Design a Turing Machine to multiply two unary numbers. **7M**

Ans. TM for multiplication of two unary numbers.



Consider the initial configuration of TM as shown in figure. Multiplier and multiplicant both are unary representation of the decimal numbers and represented using letter '1' e.g. 2 in unary format is written as '11'. Remaining of the tape is filled with 0's instead of blanks. The answer of multiplications are going to write after the ending of semicolon.

Algorithm :

If m is multiplier and n is the multiplicant then n × m can be viewed as n × m = n + n + ... 'm' no. of times.

Replace one '1' of multiplier by some other symbol, say '0' and addd the multiplicant to the right end means append 'n' no. of b's to the right end.

Repeat the procedure till all 1's in m get replaced by all 0's.

Let I = {0, 1, a, b, , }

S = {α, β, γ, δ, ϵ, ϕ = halt}

D = { L, R, N}

Transition table :

S \ I	0	1	a	b	,	.
α	L	0βR	-	-	ϕN	L
β	R	aR	-	-	γL	R
γ	-	-	1δR	-	R	L
δ	b ∈ L	R	-	R	R	-
ϵ	L	L	1δR	L	L	α N
ϕ	-	-	-	-	-	-

UNIT - VI

(Total Marks : 26)

Q.11.(a) Explain the properties of recursively enumerable set **7M**

Ans. **See P.6-7, Q.2**

Q.11.(b) Compute A(1, 1) A(1, 2) A(2, 1) using ackerman function. **6M**

Ans.

$A(1, 1) = A(0 + 1, 0 + 1)$

$= A(0, A(1, 0))$

$= A(0, A, (0, 1))$

$= A(0, 2)$

$= 3$

$A(1, 2) = A(0 + 1, 1 + 1)$

$= A(0, A(1, 1))$

$= A(0, 3) = 4$

$A(2, 1) = A(1 + 1, 0 + 1)$

$= A(1, A(2, 0))$

$= A(1, A(1, 1))$

$= A(1, 3)$

$= A(0+1, 2+1)$

$= A(0, A(1, 2)$

$= A(0, 4) = 5$

Q.12.(a) What is post correspondence problem and solve the following :

Sr. No.	List A	List B
(1)	01	011
(2)	1	10
(3)	1	11

6M

Ans. **See P.6-15, Q.18**

Suppose that this instance has a solution $i_1, i_2, \ldots\ldots i_m$

Let $i_1 = 1$

We write the string from list x above the corresponding string from y. So far we have

0 1

01^2

The next selection from list x begin with 2 or 3. Thus $i_2 = 2$ or $i_2 = 3$. But $i_2 = 1$ will not do, since no string beginning with $w_1w_1 = 0101$ can equal a string beginning with $z_1z_1 = 01^201^2$

Case 1 $i_2 = 2$

with $i_2 = 2$ we have

011

01^210

Now with $i_2 = 3$ we have

0111

01^21001^2

Since the string from list y exceeds the string from list x. A similar argument shows that $i_4 = 3$. Thus according to above sequence of choices that generate compatible strings. The string y is always longer. Thus this instance of PCP has no solution.

Case 2 : $i_2 = 3$

with $i_2 = 3$ we have

011

01^21^2

Since the string from list y again exceeds the string from list x, a similar argument shows that $i_3 = i_4 = \ldots = 3$. Thus according to above sequence of choices that generates compatible strings for which string y is always longer by some character. Thus this instance has no solution.

Hence the given PCP has no solution.

Q.12.(b) Show that the following functions are primitive recursive : **7M**

$$f(x) = \begin{cases} x/2 & , \text{ when } x \text{ is even} \\ (x-1)/2 & , \text{ when } x \text{ is odd} \end{cases}$$

Ans. Now $\left[\frac{0}{2}\right] = 0, \left[\frac{1}{2}\right] = 0, \left[\frac{2}{2}\right] = 1, \left[\frac{3}{2}\right] = 1.$

So that $\left[\frac{x}{2}\right] = x/2$ when x is even.

In order to distinguish between even and odd function, we have already defined the parity function which is primitive recursive.

$\left[\frac{0}{2}\right] = 0,$

$\therefore \left[\frac{x-1}{2}\right] = \left[\frac{x}{2}\right] - \left[\frac{1}{2}\right]$

$= \left[\frac{x}{2}\right] - P_r(x)$

where $P_r(x)$ denotes the parity functions which is x is odd i.e.

$\left[\frac{x}{2}\right]$ is even parity function subtract the odd functions it gives odd function.

$\therefore$ Hence the given functions is primitive recursive.

$P_r(x)$ denotes the parity function which is 1 when x is odd.

Notes :-

WINTER EXAMINATION - 2014
B.E. IV SEM. (COMPUTER TECHNOLOGY) (CBS)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	28 M
Unit - III	26 M
Unit - IV	28 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

WINTER EXAM. - 2014
QUESTION PAPER

Q.1.(a) Explain Chomsky Hierarchy of language in detail **6M**

(b) Solve using Mathematical Induction

$$\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$ **7M**

OR

Q.2.(a) Explain in brief diagonalization principle **5M**

(b) Are the following sets are closed under the following operations, if not what are the respective closure?

(i) The odd integers under multiplication.

(ii) The positive integers under division

(iii) The negative integers under subtraction.

(iv) The negative integers under multiplication

(v) The odd integers under division

(vi) The odd integers under addition **8M**

Q.3.(a) Construct a Moore machine whose output is 2's complement of binary number. Assume input to Moore machine is binary digit from MSB to LSB. **7M**

(b) Convert the given NFA to equivalent DFA. Also describe the language accepted by it **7M**

Q↓ \ E→	0	1
p	{p, q}	{p}
q	{r, s}	{t}
r	{p, r}	{t}
s	ϕ	ϕ
t	ϕ	ϕ

OR

Q.4.(a) Construct a DFA accepting string over decimal digit that represents decimal number divisible by 6.

(Hint : Number divisible by 2 and 3 are divisible by 6). **10M**

(b) Design a Moore machine that generates the output EVEN if the numbers of a's are even and generates output ODD if the number of a's are odd over $\Sigma = \{a, b\}$. **4M**

Q.5.(a) Give regular expression and transition diagram for finite automata for the following regular sets of string over $\Sigma = \{a, b\}$

(i) All string beginning with 'a' and ending with 'b'.

(ii) All string with atleast two consecutive 'a'.

(iii) All string containing the substring 'aba'. **7M**

(b) Give the statement of pumping Lemma and prove that $L = \{a^P \mid P \text{ is prime}\}$ is not regular. **6M**

OR

Q.6.(a) Reduce the following grammar :

$S \rightarrow ABC / BaB$

$A \rightarrow aA/BaC/aaa$

$B \rightarrow bBb/a$

$C \rightarrow CA/AC$ **5M**

(b) Show that following grammar is ambiguous. **2M**

$S \rightarrow aSbS \mid bSaS \mid \epsilon$

(c) Convert the given grammar into GNF.

$S \rightarrow AA \mid 0$

$A \rightarrow SS \mid 1$ **6M**

Q.7.(a) Design a PDA for accepting

$$L = \left\{ WCW^R \;\middle|\; W \in (0/1)^* \text{ and } W^R \text{ is reverse of } W \right\}$$ **8M**

(b) Define a PDA and explain the difference between acceptance by final state and acceptance by empty stack. **6M**

OR

Q.8.(a) Explain closure properties of CFL. **6M**

(b) Design a CFG for given PDA

$\partial(q_0, a, z_0) \Rightarrow (q_0, xz_0)$

$\partial(q_0, a, x) \Rightarrow (q_0, xx)$

$\partial(q_0, b, x) \Rightarrow (q_1, \epsilon)$

$\partial(q_1, b, x) \Rightarrow (q_1, \epsilon)$

$\partial(q_1, \epsilon, z_0) \Rightarrow (q_1, \epsilon)$ **8M**

Q.9.(a) Design a Turing Machine of 2'S complement of binary number **7M**

(b) Describe Linear Bounded Automata and the language accepted by LBA **6M**

OR

Q.10.(a) Construct a Turing Machine for performing multiplication of two numbers **8M**

(b) Explain different types of Turing Machine **5M**

Q.11.(a) Define Ackermann's function Compute for A (1, 1) A (2, 1), and A (2, 2). **7M**

(b) What is the significance of PCP? Obtain the solution for

$X = (01, 1, 1)$

$Y = (01^2, 10, 1^2)$ **6M**

OR

Q.12.(a) What are Primitive Recursive functions? Show that SUM (x, y) and SUB (x, y) are PRF. **6M**

(b) What are the closure properties of recursive and recursively enumerable languages? **7M**

WINTER EXAM. - 2014

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Explain Chomsky Hierarchy of language in detail **6M**

Ans. **See P.1-11, Q.22.**

Q.1.(b) Solve using Mathematical Induction.

$$\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$ **7M**

Ans. **See P.1-8, Q.11.**

Q.2.(a) Explain in brief diagonalization principle. **5M**

Ans. **See P.1-5, Q.5(1).**

Q.2.(b) Are the following sets are closed under the following operations, if not what are the respective closure?

(i) The odd integers under multiplication.

(ii) The positive integers under division.

(iii) The negative integers under subtraction.

(iv) The negative integers under multiplication.

(v) The odd integers under division.

(vi) The odd integers under addition. **8M**

Ans.

(i) **Closed :**

The odd integer are closed under multiplication

For example, $5 \times 7 = 35$.

Multiplying odd numbers always produce odd number. So odd integer are closed under multiplication.

(ii) **Not closed :**

The positive integer are not closed under division. For instance if we divide the integer

1 by 2 we get $\frac{1}{2}$ which is not an integer.

(iii) **Not closed :**

Subtraction of negative integer

$[-1] - [-2] = +1$

$+1$ are positive integer. So, negative integer are not closed under subtraction.

(iv) **Not closed :**

For example, $(-1) * (-2) = 2$

If we multiply two numbers that were in the negative number set and the product was not in that set. It is a positive number. So, negative integer are not closed under multiplication.

(v) **Not closed :**

The odd integers under division :

For instance if we divide the odd integer 1 by 3 we get $\frac{1}{3}$ which is not an integer. So odd integers under division is not closed.

(vi) **Not closed :**

The odd integers under addition :

For instance, if we add odd integer

For example, $3 + 7 = 10$ or $7 + 9 = 16$ is even. So odd integers under addition is not closed.

UNIT - II

(Total Marks : 28)

Q.3.(a) Construct a Moore machine whose output is 2's complement of binary number. Assume input to Moore machine is binary digit from MSB to LSB. **7M**

Ans.

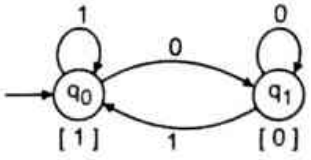


Fig.

State	O/P
q_0	1
q_1	0

Transition table for Moore machine :

Present state	I = 0	I = 1	Output
q_0	q_1	q_0	1
q_1	q_1	q_0	0

Q.3.(b) Convert the given NFA to equivalent DFA. Also describe the language accepted by it. **7M**

Q↓ \ E→	0	1
p	{p, q}	{p}
q	{r, s}	{t}
r	{p, r}	{t}
s	ϕ	ϕ
t	ϕ	ϕ

Ans.

Q↓ \ E→	0	1
→ p	{p, q}	{p}
q	{r, s}	{t}
r	{p, r}	{t}
s	ϕ	ϕ
* t	ϕ	ϕ

Note : p is the initial state

t is the final state

Transition table for DFA :

Q↓ \ E→	0	1
→ p	{p, q}	{p}
{p, q}	{p, q, r, s}	{q, t}
{p, q, r, s}	{p, q, r, s}	{p, t}
* {p, t}	{p, q}	{p}

Renaming state $P \rightarrow A$

$\{p, q\} \rightarrow B$

$\{p, q, r, s\} \rightarrow C$

$\{p, t\} \rightarrow D$

Q↓ \ E→	0	1
A	B	A
B	C	D
C	C	D
* D	B	A

Replacing C by B, we get

Q↓ \ E→	0	1
→ A	B	A
B	B	D
* D	B	A

DFA :

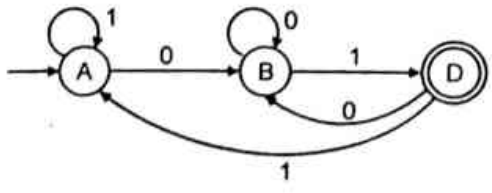


Fig.

The language accepted by this automata is the string that ends with 1 but it start with either zero or one

Q.4.(a) Construct a DFA accepting string over decimal digit that represents decimal number divisible by 6

(Hint : Number divisible by 2 and 3 are divisible by 6) **10M**

Ans. See P.2-33, Q.51.

Q.4.(b) Design a Moore machine that generates the output EVEN if the numbers of a's are even and generates output ODD if the number of a's are odd over $\Sigma = \{a, b\}$. **4M**

Ans. See P.PS-12, Q.4(b).

UNIT - III

(Total Marks : 26)

Q.5.(a) Give regular expression and transition diagram for finite automata for the following regular sets of string over $\Sigma = \{a, b\}$

(i) All string beginning with 'a' and ending with 'b'.

(ii) All string with atleast two consecutive 'a'.

(iii) All string containing the substring 'aba'. **7M**

Ans. See P.3-19, Q.23.

Q.5.(b) Give the statement of pumping lemma and prove that $L = \{a^P \mid P$ is prime$\}$ is not regular. **6M**

Ans. See P.3-53, Q.83, Q.81.

Q.6.(a) Reduce the following grammar :

S → ABC | BaB

A → aA | BaC | aaa

B → bBb | a

C → CA | AC **5M**

Ans. We will minimize the grammar by eliminating useless symbols.

The right side of both the C-productions contain C. Hence the non terminal C is not derivable to any terminal symbol. Therefore, C can be eliminated from the grammar.

∴ Reduced grammar is

S → BaB

A → aA/aaa

B → bBb/a

Since, A is not reachable state from any state, hence eliminating production A

So, minimized grammar is

S → BaB

B → bBb/a

Q.6.(b) Show that following grammar is ambiguous

S → aSbS | bSaS | ∈ . **2M**

Ans. See P.3-68, Q.127(ii).

Q.6.(c) Convert the given grammar into GNF

S → AA | 0

A → SS | 1 **6M**

Ans. See P.3-89, Q.178.

UNIT - IV

(Total Marks : 28)

Q.7.(a) Design a PDA for accepting

$$L = \left\{ WCW^R \;\middle|\; \begin{array}{l} W \in (0/1)^* \\ \text{and } W^R \text{ is reverse of } W \end{array} \right\}$$ **8M**

Ans. See P.4-15, Q.22.

Q.7.(b) Define a PDA and explain the difference between acceptance by final state and acceptance by empty stack. **6M**

Ans. See P.4-4, 18, Q.1, Q.28.

Q.8.(a) Explain closure properties of CFL. **6M**

Ans. See P.4-26, Q.40.

Q.8.(b) Design a CFG for given PDA

$\partial(q_0, a, z_0) \Rightarrow (q_0, xz_0)$

$\partial(q_0, a, x) \Rightarrow (q_0, xx)$

$\partial(q_0, b, x) \Rightarrow (q_1, \epsilon)$

$\partial(q_1, b, x) \Rightarrow (q_1, \epsilon)$

$\partial(q_1, e, z_0) \Rightarrow (q_1, \epsilon)$ **8M**

Ans. $\partial(q_0, a, z_0) = (q_0, xz_0)$

$\partial(q_0, a, x) = (q_0, xx)$

$\partial(q_0, b, x) = (q_1, e)$

$\partial (a_1, b, x) = (q_1, \epsilon)$

$\partial (q_1, \epsilon, z_0) = (q_1, \epsilon)$

The production of an equivalent CFG are obtained as follows :

$S \rightarrow [q_0, z_0, q_0]$

$S \rightarrow [q_0, z_0, q_1]$

q_0 and q_1 are two state, so the grammar should generate all those strings that allows PDA M to erase z_0 from the stack by stating in q_0 and either ending up in q_0 or q_1 using sequence of moves.

For a move : $\delta(q_0, a, z_0) = (q_0\ xz_0)$ we get following production :

$[q_0, z_0, q_0] \rightarrow a[q_0, x, q_0][q_0, z_0, q_0]$

$[q_0, z_0, q_0] \rightarrow a[q_0, x, q_1][q_1, z_0, q_0]$

$[q_0, z_0, q_1] \rightarrow a[q_0, x, q_0][q_0, z_0, q_1]$

$[q_0, z_0, q_1] \rightarrow a[q_0, x, q_1][q_1, z_0, q_1]$

For a move : $\delta[q_0, a, x] = [q_0, x, x]$:

$[q_0, x, q_0] \rightarrow a[q_0, x, q_0][q_0, x, q_0]$

$[q_0, x, q_0] \rightarrow a[q_0, x, q_1][q_1, x, q_0]$

$[q_0, x, q_1] \rightarrow a[q_0, x, q_0][q_0, x, q_1]$

$[q_0, x, q_1] \rightarrow a[q_0, x, q_1][q_1, x, q_1]$

For move : $\delta[q_0, b, x] = [q_1, \epsilon]$:

$[q_0, x, q_1] \rightarrow b$

For move : $\delta[q_1, b, x] = [q_1, \epsilon]$:

$[q_1, x, q_1] \rightarrow b$

For move : $\delta[q_1, \epsilon, z_0] = [q_1, \epsilon]$:

$[q_1, z_0, q_1] \rightarrow \epsilon$

Renaming the object :

Object	Name
$[q_0, z_0, q_0]$	A
$[q_0, z_0, q_1]$	B
$[q_1, z_0, q_0]$	C
$[q_1, z_0, q_1]$	D
$[q_0, x, q_0]$	E
$[q_0, x, q_1]$	F
$[q_1, x, q_0]$	G
$[q_1, x, q_1]$	H

Then the production can be written as

$S \rightarrow A$

$S \rightarrow B$

$A \rightarrow aEA$

$A \rightarrow aFC$

$B \rightarrow aEB$

$B \rightarrow aFD$

$E \rightarrow aEE$

$E \rightarrow aFG$

$F \rightarrow aEF$

$F \rightarrow aFH$

$F \rightarrow b$

$H \rightarrow b$

$D \rightarrow \epsilon$

$C \rightarrow \epsilon$

Production becomes

$S \rightarrow A \mid B$

$A \rightarrow aEA \mid aFC$

$B \rightarrow aEB \mid aFD$

$E \rightarrow aEE \mid aFG$

$F \rightarrow aEF \mid aFH \mid b$

$H \rightarrow b$

$D \rightarrow \epsilon$

Since G is useless we get

$S \rightarrow A \mid B$

$A \rightarrow aEA \mid aFC$

$B \rightarrow aEB \mid aFD$

$E \rightarrow aEE$

$F \rightarrow aEF \mid aFH \mid b$

$H \rightarrow b$

D → ∈

As a result E is useless symbol

S → A | B

A → aFC

B → aFD

F → aFH | b

H → b

D → ∈

As a result C is useless and A is also useless

S → B

B → aFD

F → aFH | b

H → b

D → G

UNIT - V

(Total Marks : 26)

Q.9.(a) Design a Turing Machine of 2'S complement of binary number. **7M**

Ans. **See P.5-37, Q.45.**

Q.9.(b) Describe Linear Bounded Automata and the language accepted by LBA **6M**

Ans. **See P.5-34, Q.40.**

Q.10.(a) Construct a Turing Machine for performing multiplication of two numbers **8M**

Ans. **See P.5-41, Q.49.**

Q.10.(b) Explain different types of Turing Machine. **5M**

Ans. **See P.5-21, Q.19.**

UNIT - VI

(Total Marks : 26)

Q.11.(a) Define Ackermann's function. Compute for A (1, 1) A (2, 1), and A (2, 2) **7M**

Ans. **See P.6-38, Q.68.**

Q.11.(b) What is the significance of PCP? Obtain the solution for

$X = (01, 1, 1)$

$Y = (01^2, 10, 1^2)$ **6M**

Ans. **See P.6-17, Q.21(a).**

Q.12.(a) What are Primitive Recursive functions? Show that SUM (x, y) and SUB (x, y) are PRF. **6M**

Ans. **See P.6-28, 29, 31, Q.42, Q.45, Q.51.**

Q.12.(b) What are the closure properties of recursive and recursively enumerable languages? **7M**

Ans. **See P.6-7, Q.1, Q.2.**

Notes :-

SUMMER EXAMINATION - 2015
B.E. IV SEM. (COMPUTER TECHNOLOGY) (CBS)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	26 M
Unit - V	26 M
Unit - VI	28 M
Questions covered in VBD	160 M (100 %)

SUMMER EXAM. - 2015

QUESTION PAPER

Q.1.(a) What do you mean by formal grammar? Explain formal grammar with suitable example. **6M**

(b) Prove the following theorem by principal of mathematical induction

$$\sum_{i=1}^{n} \frac{1}{i(i+1)} = \frac{n}{n+1}$$

Is true for all $n \geq 0$. **7M**

OR

Q.2.(a) What do you mean by countability and diagonalization? Explain the same in brief. **4M**

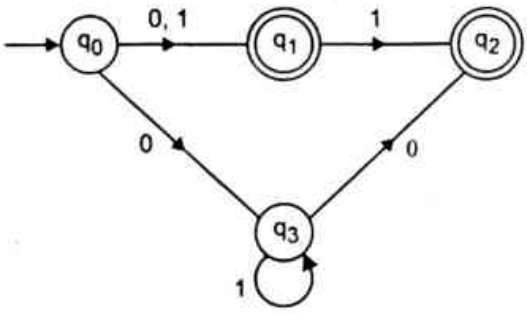
(b) If R = (1, 2), (2, 3), (1, 4), (4, 2), (3, 4). Find

(i) R^+

(ii) R^* **6M**

(c) Explain Pigeon- hole principle in brief. **3M**

Q.3.(a) Consider the NFA shown in the transition diagram. Obtain its equivalent DFA. **7M**



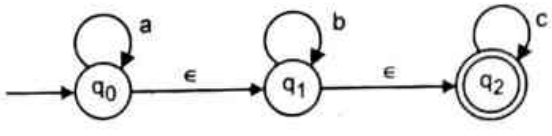
(b) Differentiate between :

(i) NFA and DFA

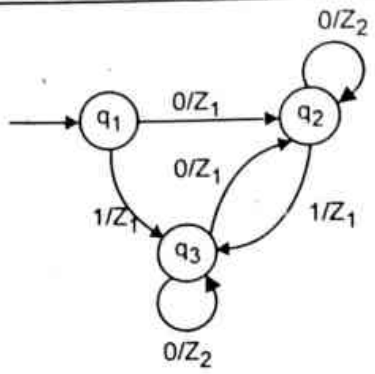
(ii) Moore and Mealy machine. **6M**

OR

Q.4.(a) Convert the given NFA with ϵ - moves into DFA. **7M**



(b) Obtain equivalent Moore machine for the given Mealy machine. **6M**



Q.5.(a) Consider the transition diagram shown in Fig. **6M**

(a) Construct its equivalent regular expression.

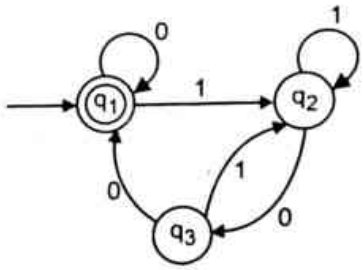


Fig.

(b) Consider the following grammar :

$S \rightarrow 0B \mid 1A$

$A \rightarrow 0 \mid 0S \mid 1AA$

$B \rightarrow 1 \mid 1S \mid 0BB$

For the string w = 00110101; find leftmost and rightmost derivation. Also draw its parse tree. **6M**

(c) Give and explain theorem of pumping lemma for regular sets. **2M**

OR

Q.6.(a) Convert the following grammar into CNF.

$S \rightarrow aAD$

$A \rightarrow aB \mid bAB$

$B \rightarrow b$

$D \rightarrow d$ **6M**

(b) Explain the steps used to obtain reduced CGF. Obtain reduced CFG for the grammar given below.

$S \rightarrow aA \mid aBB$

$A \rightarrow aaA \mid \epsilon$

$B \rightarrow bB \mid bbC$

$C \rightarrow B$ **8M**

Q.7.(a) Construct PDA that accepts the language

$L = \{0^n 1^m 0^{m+n} \mid n, m \geq 1\}$ **7M**

(b) State and explain closure properties of context free languages. **6M**

OR

Q.8.(a) Design a PDA accepting following language

$L = \{p^n q^{n+1} r^n \mid n \geq 1\}$ **7M**

(b) Is it true that non deterministic PDA is more powerful than that of deterministic PDA? Justify your answer. **3M**

(c) Consider the context free language

$L = \{a^{2n} b^n \mid n \geq 0\}$

Obtain its equivalent context free grammar **3M**

Q.9.(a) Design a turing machine that computes addition of two integers i.e.

$f(m, n) = m + n$ **7M**

(b) Write a short note on universal turing machine. **6M**

OR

Q.10.(a) Design a turing machine that can accept the string over {0, 1} to recognize all strings containing even number of 1'S. **7M**

(b) Design T. M. for language to find 2's complement of a binary number **6M**

Q.11.(a) What do you mean by recursive function? Design recursive function for performing addition of two positive integers m and n. **6M**

(b) Define Ackerman's function. **8M**

Compute

(i) A (1, 1)

(ii) A (2, 1)

(iii) A (2, 3)

OR

Q.12.(a) What do you mean by post correspondence problem? Decide whether the following (A, B) pair have a solution or not. If yes, give a solution, if no, why?

(i) $A(b, bab^3, ba), B = \{b^3, ba, a\}$

(ii) A (01, 1, 1), B = {011, 10, 11} **10M**

(b) Define decidable and undecidable language. **4M**

SUMMER EXAM. - 2015

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) What do you mean by formal grammar? Explain formal grammar with suitable example. **6M**

Ans. See P.1-11, Q.21.

Q.1.(b) **Prove the following theorem by principal of mathematical induction**

$$\sum_{i=1}^{n} \frac{1}{i(i+1)} = \frac{n}{n+1}$$

Is true for all $n \geq 0$. **7M**

Ans. **Given :** $\sum_{i=1}^{n} \frac{1}{i(i+1)} = \frac{n}{n+1}$

It can also be written as

$$\frac{1}{2} + \frac{1}{6} + \frac{1}{12} + \frac{1}{20} + \ldots \frac{1}{n(n+1)} = \frac{1}{n+1}$$

Step 1 : Proof for the basis for n = 1

$$L.H.S. = \frac{1}{2} \quad \ldots (1)$$

$$\text{And } R.H.S. = \frac{n}{n+1} = \frac{1}{1+1} = \frac{1}{2} \quad \ldots (2)$$

Hence from (1) and (2) result is true for n = 1.

Step 2 : By induction hypothesis, we have

$$\frac{1}{2} + \frac{1}{6} + \frac{1}{12} + \frac{1}{20} + \ldots + \frac{1}{i(i+1)} = \frac{n}{n+1}$$

As i = n in L.H.S.

$$\frac{1}{2} + \frac{1}{6} + \frac{1}{12} + \frac{1}{20} + \ldots + \frac{1}{n(n+1)} = \frac{n}{n+1}$$

Step 3 : We have to prove

$$\sum_{i=1}^{n+1} \frac{1}{i(i+1)} = \frac{n+1}{n+2}$$

$$L.H.S. = \sum_{i=1}^{n} \frac{1}{i(i+1)} + \frac{1}{(n+1)(n+2)}$$

$$= \frac{n}{n+1} = \frac{1}{(n+1)(n+2)}$$

$$= \frac{n(n+2)+1}{(n+1)(n+2)} = \frac{n^2+2n+1}{(n+1)(n+2)}$$

$$= \frac{(n+1)^2}{(n+1)(n+2)} = \frac{n+1}{n+2}$$

= R.H.S

Hence it is proved for $n = n + 1$

$\sum_{i=1}^{n} \frac{1}{i(i+1)} = \frac{n}{n+1}$ is true for all $n \geq 0$

Q.2.(a) What do you mean by countability and diagonalization? Explain the same in brief **4M**

Ans. See P.1-5, Q.5.

Q.2.(b) If R = (1, 2), (2, 3), (1, 4), (4, 2), (3, 4). Find

(i) R ' (ii) R* 6M

Ans. R = (1, 2), (2, 3), (1, 4), (4, 2), (3, 4)

RoR = {(1, 2), (2, 3), (1, 4), (4, 2), (3, 4)} o

{(1, 2), (2, 3), (1, 4), (4, 2), (3, 4)}

= {(1, 3), (2, 4), (1, 2), (4, 3), (3, 2)}

$R^3 = R^2oR$ = {(1, 3), (2, 4), (1, 2), (4, 3), (3, 2)} o {(1, 2), (2, 3), (1, 4), (4, 2), (3, 4)}

= {(1, 4), (2, 2), (1, 3), (4, 4), (3, 3)}

$R^4 = R^3oR$ = {(1, 4), (2, 2), (1, 3), (4, 4), (3, 3)} o {(1, 2), (2, 3), (1, 4), (4, 2), (3, 4)}

= {(1, 2), (2, 3), (1, 4), (4, 2), (3, 4)}

= R

$R^5 = R^4o\ R = R^2$, $R^6 = R^5o\ R = R^4o\ R = R^3$

So,

$R^* = R \cup R^2 \cup R^3$

R^* = {(1, 2), (2, 3), (1,4) (4, 2) (3, 4), (1, 3), (2, 4), (4, 3), (3, 2) (2, 2) (4, 4) (3, 3)}

R^* = $R^+ \cup$ {(1,1), (2, 2), (3, 3), (4, 4)}

R^* = {(1, 2) (2, 3) (1, 4) (4, 2) (3, 4) (1, 3) (2, 4) (4, 3) (3, 2) (2, 2) (4, 4) (3, 3), (1, 1)}

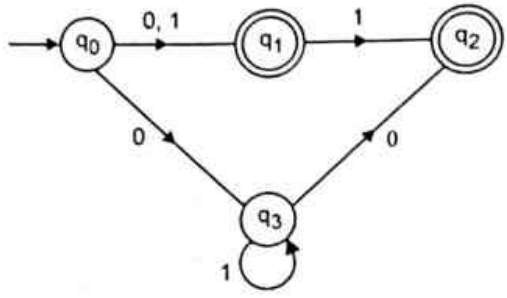
Q.2.(c) Explain Pigeon- hole principle in brief **3M**

Ans. See P.1-10, Q.16.

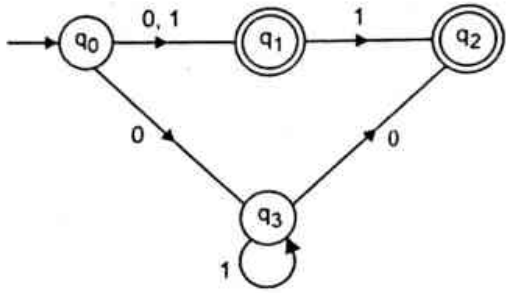
UNIT - II

(Total Marks : 26)

Q.3.(a) Consider the NFA shown in the transition diagram. Obtain its equivalent DFA. 7M



Ans.



Transition table for NFA :

Σ / Q	0	1
→ q_0	q_1 q_3	q_1
* q_1	-	q_2
* q_2	-	-
q_3	q_2	q_3

Transmition table for DFA :

Σ / Q	0	1
→ q_0	q_1 q_3	q_1
q_1	-	q_2
* q_1 q_3	q_2	q_2, q_3
* q_2	-	-
* q_2, q_3	q_2	q_3
q_3	q_2	q_3

Rename states :

$q_0 \rightarrow A$

$q_1q_1 \rightarrow C$

$q_2q_3 \rightarrow E$

$q_1 \rightarrow B$

$q_2 \rightarrow D$

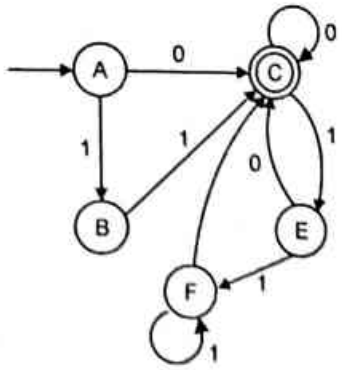
$q_3 \rightarrow F$

Q \ Σ	0	1
→ A	C	B
B	-	D
* C	D	E
* D	-	-
* E	D	F
F	D	F

Remove D and replace D by C.

Q \ Σ	0	1
→ A	C	B
B	-	C
* C	C	E
* E	C	F
F	C	F

Transition diagram :



Q.3.(b) Differentiate between : **6M**

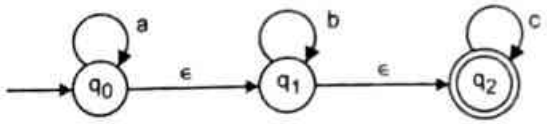
(i) NFA and DFA

Ans. See P.2-13, Q.12

(ii) Moore and Mealy machine

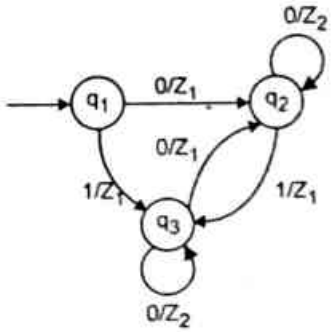
Ans. See P.2-49, Q.64

Q.4.(a) Convert the given NFA with ∈ - moves into DFA. **7M**



Ans. See P.2-21, Q.27.

Q.4.(b) Obtain equivalent Moore machine for the given Mealy machine **6M**



Ans. See P.2-56, Q.78.

UNIT - III

(Total Marks : 28)

Q.5.(a) Consider the transition diagram shown in Fig.

(a) Construct its equivalent regular expression. **6M**

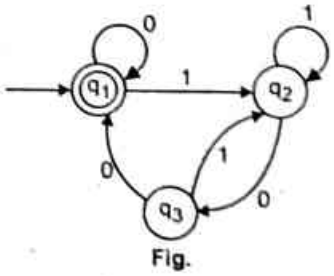


Fig.

Ans. See P.3-46, Q.66.

Q.5.(b) Consider the following grammar :

S → 0B | 1A

A → 0 | 0S | 1 AA

B → 1 | 1S | 0BB

For the string w = 00110101; find left most and rightmost derivation.

Also draw its parse tree. **6M**

Ans. W = 00110101

(I) Leftmost derivation :

S → 0B

→ 00 BB

→ 001SB

→ 0011ASB

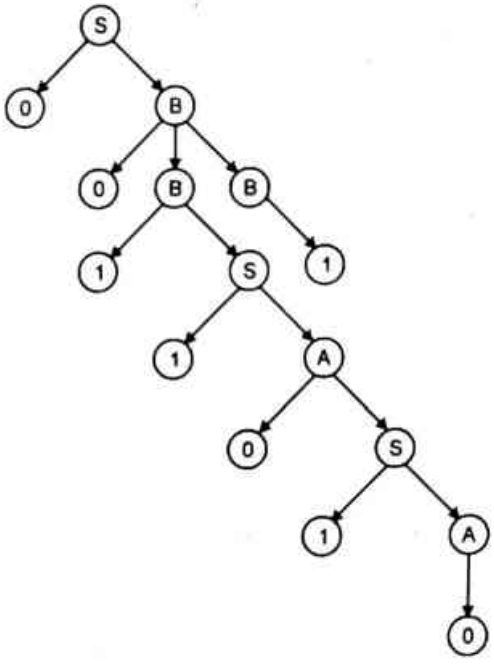
→ 00110SB

→ 001101AB

→ 0011010B

→ 00110101

Passe tree :



(2) Rightmost derivation

S → 0B

→ 00BB

→ 00B1

→ 00S1

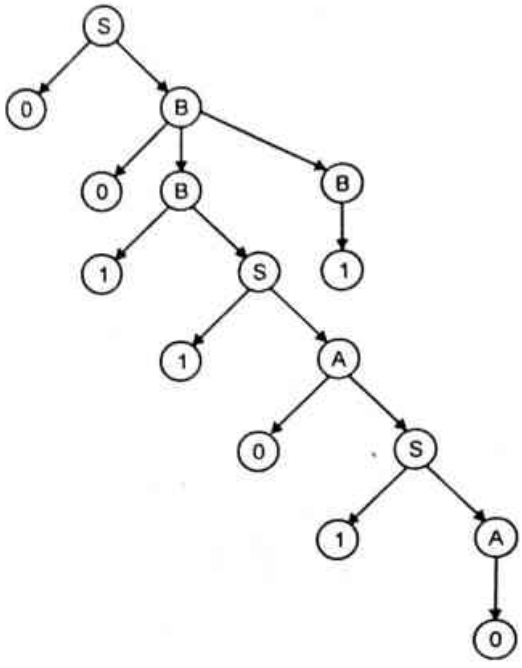
→ 0011A1

→ 00110S1

→ 001101A1

→ 00110101

Parse tree :



Q.5.(c) Give and explain theorem of pumping lemma for regular sets. **2M**

Ans. **See P.3-52, Q.80.**

Q.6.(a) Convert the following grammar into CNF.

S → aAD

A → aB | bAB

B → b

D → d **6M**

Ans. S → aAD

A → aB | bAB

B → b

D → d

S → aAD is replace by S → CAD and C → a

Similarly A → aB is replaced by A → CB

A → bAB is replaced by A → BAB and B → b

In the next stage, the production S → CAD is replaced by S → CE and E → AD and the production BAB is replaced by A → BF and F → AB.

The production for grammar in CNF :

S → CE

A → CB | BF

E → AD

F → AB

C → a

B → b

D → d

Q.6.(b) Explain the steps used to obtain reduced CGF. Obtain reduced CFG for the grammar given below

S → aA | aBB

A → aaA | ∈

B → bB | bbC

C → B **8M**

Ans. **See P.3-80, Q.156.**

UNIT - IV
(Total Marks : 26)

Q.7.(a) Construct DDA that accepts the language

$L = \{0^n 1^m 0^{m+n} \mid n, m \geq 1\}$ **7M**

Ans. **See P.4-11, Q.14.**

Q.7.(b) State and explain closure properties of context free languages. **6M**

Ans. **See P.4-26, Q.40.**

Q.8.(a) Design a PDA accepting following language

$L = \{p^n q^{n+1} r^n \mid n \geq 1\}$ **7M**

Ans. Let n = 1

$L_1 = \{ p q q r\}$

n = 2

$L_2 = \{ p p q q q r r\}$

String has fixed length.

String start at a and ends with r. equal no. P and r.

Algorithm :

Push P onto the stack. When read q only change the state and when read r pop all P from the stack.

Transition :

(1) $\delta(q_0, p, z_0) = (q_0, p\, z_0)$

$\delta(q_0, p, p) = (q_0, p\, p)$

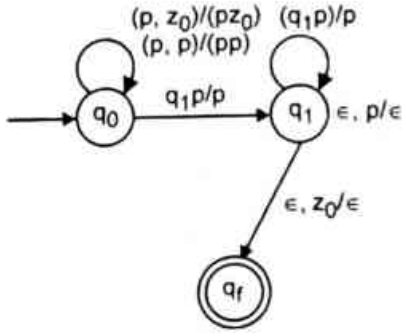
$\delta(q_0, q, p) = (q_1, p)$

$\delta(q_1, q, p) = (q_1, p)$

$\delta(q_1, r, p) = (q_1, \epsilon)$

$\delta(q_1, \epsilon, z_0) = (q_f, \epsilon)$

Transition graph :



Q.8.(b) Is it true that non deterministic PDA is more powerful than that of deterministic PDA? Justify your answer. **3M**

Ans. **See P.4-7, Q.8.**

Q.8.(c) Consider the context free language

$L = \{a^{2n} b^n \mid n \geq 0\}$

Obtain its equivalent context free grammar. **3M**

Ans. $L = \{a^{2n} b^n \mid n \geq 0\}$

L = { ∈ , aab, aaaabb, aaaaaabbb .....)

CFG :

S → AB / ∈

A → aaA / aa

B → bB / b

V = {S, A, B}, T = {a, b}} where P is given by S is start symbol.

UNIT - V
(Total Marks : 26)

Q.9.(a) Design a turing machine that computes addition of two integers i.e.

f (m, n) = m + n **7M**

Ans. **See P.5-38, Q.47.**

Q.9.(b) Write a short note on universal turing machine. **6M**

Ans. **See P.5-27, Q.26(A).**

Q.10.(a) Design a turing machine that can accept the string over {0, 1} to recognize all strings containing even number of 1'S. **7M**

Ans. **See P.5-36, Q.44.**

Q.10.(b) Design T. M. for language to find 2's complement of a binary number. **6M**

Ans. **See P.5-37, Q.45.**

UNIT - VI

(Total Marks : 28)

Q.11.(a) What do you mean by recursive function? Design recursive function for performing addition of two positive integers m and n. **6M**

Ans. **See P.6-21, 29, Q.29, 45.**

Q.11.(b) Define Ackerman's function.

Compute :

(i) A (1, 1)

(ii) A (2, 1)

(iii) A (2, 3) **8M**

Ans. **See P.6-38, Q.68.**

Q.12.(a) What do you mean by post correspondence problem? Decide whether the following (A, B) pair have a solution or note. If yes, give a solution, if no; why?

(i) A (b, bab^3, ba), B = (b^3, ba, a)

(ii) A (01, 1, 1), B = {011, 10, 11} **10M**

Ans. **See P.6-15, 16, 17, Q.18, 19, 21(a).**

Q.12.(b) Define decidable and undecidable language. **4M**

Ans. **See P.6-13, 14, Q.15(a), Q.16(a).**

Notes :-

WINTER EXAMINATION - 2015
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	28 M
Unit - V	28 M
Unit - VI	24 M
Questions covered in VBD	160 M (100 %)

WINTER EXAM.-2015
QUESTION PAPER

Q.1(a) Prove the following by principle of Induction :

(i) $1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n \times (n+1)(2n+1)}{6}$

(ii) $1 + 4 + 7 + \ldots + (3n-2) = \frac{n(3n-1)}{2}$ **7M**

(b) Explain the following terms :

(i) Null set. (ii) Sub set.

(iii) Proper prefix. (iv) Proper suffix. **6M**

OR

Q.2.(a) Describe the concept of Pigeon hole principle with suitable example. **5M**

(b) Discuss the Chomsky Hierarchy of language. Identify the type of following grammar :

$AB \rightarrow CDB$

$AB \rightarrow CdEB$

$ABcd \rightarrow abCDeBcd$

$B \rightarrow b$ **6M**

(c) Explain the concept of Diagonalization. **2M**

Q.3.(a) Differentiate between NFA and DFA. **5M**

(b) Design a DFA accepting following language : **8M**

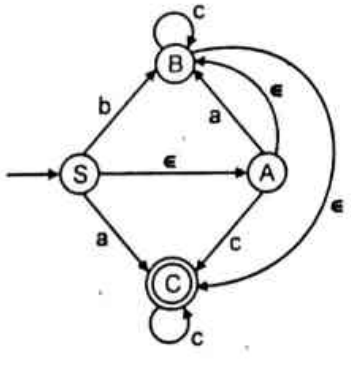
$$L = \left\{ \begin{matrix} w / w \in (a|b)^* \\ n_a(w) \bmod 4 \geq n_b(w) \bmod 3 \end{matrix} \right\}$$

Where $n_a(w) \rightarrow$ Number of a's in w.

$n_b(w) \rightarrow$ Number of b's in w.

OR

Q.4.(a) Convert the following NFA with ϵ transition into DFA. **7M**



(b) Design Moore and Melay machine that gives "1" as its output if the last three digits are 1's. Assume that $\epsilon = 0, 1$. **6M**

Q.5.(a) Obtain deterministic finite automata for the following regular expression :

$(0+1)^*10(0+1)^* + (0+1)^*11(0+1)^*$ **8M**

(b) Obtain right linear grammar equivalent to the following left linear grammar :

$S \rightarrow Sab / Aa$

$A \rightarrow Abb / bb$ **6M**

OR

Q.6.(a) Using pumping lema, prove that the language :

$L = \{a^{i3} / i \geq 1\}$

is not regular. **4M**

(b) Find grammar generating :

$L = (0^m 1^m 2^j / m \geq 1, j \geq 0)$ **6M**

(c) Find the equivalent CNF for the following grammar :

$S \rightarrow aB / ab$

$A \rightarrow aAB / a$

$B \rightarrow ABb / b$ **4M**

Q.7.(a) Design PDA for :

$L_s = \{ w c w^R / \omega \in (a,b)^* \}$

w^R = Reverse of w

Also explain the stack execution with valid string. **8M**

(b) Convert the grammar :

$S \rightarrow SOS / SOSOSIS / SISOSOS / \epsilon$

to a PDA that accepts the same language by empty stack. **6M**

OR

Q.8.(a) Design a PDA to accept all string.

over $L = (a^n b^m c^{|m-n|} \mid m, n >= 1)$

Explain all the cases. **7M**

(b) Given a grammar for the language N(M), where :

$M = (\{q_0, q_1\}, \{0, 1\}, \{z_0, x\}, \delta, q_0, z_0, \phi)$ and δ is given by

$\delta(q_0, 1, z_0) = \{(q_0, x\, z_0)\}\, \delta, (q_0, \epsilon, z_0) = \{q_0, \epsilon\}$.

$\delta(q_0, 1, x) = \{q_0, xx)\}, \delta(q_1, 1, x) = (q_1, \epsilon)$

$\delta(q_0, 0, x) = \{(q_1, x)\}, \delta(q_1, 0, z_0) = \{q_0, z_0\}$ **7M**

VBD

Q.9.(a) Explain the following terms :

(i) Multitape turing machine

(ii) Univeral turing machine **5M**

(b) Design a Turing machine to compute

$$f(m,n) = \begin{cases} m-n & \text{if } m>n \\ 0 & \text{if } m \le n \end{cases}$$ **8M**

OR

Q.10.(a) Construct a Turing machine accepting the language

$L = \{a^n b^{2n} c^n \mid n \ge 1\}$ **8M**

(b) Explain the concept of Linear Bounded Automata. **5M**

Q.11.(a) Define Ackermann's function. Compute A(1, 1), A(2, 1) and A(2, 2) **5M**

(b) Explain PCP problem and give solution for following : **6M**

List	x	y
1	b	b^3
2	bab^3	ba
3	ba	a

(c) Explain Unrestricted Grammar. **2M**

OR

Q.12.(a) Consider the function :

equals (x, y) = 1 if x = y

= 0 if $x \neq y$

Show that this function is primitive recursive. **5M**

(b) Write a short note on .

(i) Recursive function. (ii) Undecidability.

(iii) Blank tape halting problem. **8M**

WINTER EXAM.-2015

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Prove the following by principle of Induction: **7M**

(i) $1^2 + 2^2 + 3^2 + ... + n^2 = \frac{n \times (n+1)(2n+1)}{6}$

(ii) $1 + 4 + 7 + ... + (3n-2) = \frac{n(3n-1)}{2}$

Ans. (i) See. P.1-8, Q.11.

(ii) $1 + 4 + 7 + ... + (3n-2) = \frac{n(3n-1)}{2}$:

(1) **Basic step :**

For n = 1

L.H.S. = 3n - 2 = 3 × 1 - 2 = 1

R.H.S. $= \frac{n(3n-1)}{2} = \frac{1(3-1)}{2} = \frac{2}{2} = 1$

∴ L.H.S. = R.H.S.

(2) **Induction hypothesis :**

We have, 1 + 4 + 7 + ... + (3n - 2)

$= \frac{n(3n-1)}{2}$

(3) **Induction step :**

For n = n + 1, we have to prove that

1 + 4 + 7 + ... + (3n - 2) + (3(n + 1) - 2)

$= \frac{(n+1)(3(n+1)-1)}{2}$

$= \frac{(n+1)(3n+3-1)}{2}$

$= \frac{(n+1)(3n+2)}{2}$

∴ L.H.S.

= 1 + 4 + 7 + (.... + (3n - 2) + (3(n + 1) - 2)

$= \frac{n(3n-1)}{2} + (3(n+1)-2)$

$= \frac{n(3n-1) + 2[3(n+1)-2]}{2}$

$= \frac{3n^2 - n + 6(n+1) - 4}{2}$

$= \frac{3n^2 - n + 6n + 6 - 4}{2}$

$= \frac{3n^2 + 5n + 2}{2}$

$= \frac{(n+1)(3n+2)}{2}$

∴ L.H.S. = R.H.S.

Q.1.(b) Explain the following terms :

(i) Null set. (ii) Sub set.

(iii) Proper prefix. (iv) Proper suffix. **6M**

Ans. See. P.1-4, 11, Q.2.

(i) Null set :

- In mathematical sets, the null set also called the empty set, is the set that does not contain anything. It is symbolized ϕ or { }.
- There is only one null set. This is because there is logically only one way that a set can contain nothing.
- The null set makes it possible to explicitly define the results of operation on certain sets that would otherwise not be explicitly definable. The intersection of two disjoint set is the null set

e.g. $\{1, 3, 5, 7, 9 \ldots\} \cap \{2, 4, 6, 8, 10, \ldots\}$

$= \phi$

(ii) Sub set :

- A subnet is a portion of a set. B is a subset of A, if every member of B is a member of A. It is written as $B \subseteq A$.
- If B is a proper subset of A i.e. a subset other than the set itself, this is written BCA if B is not a subset of A, this is written $B \not\subseteq A$.

For example, the set {1} has two subsets ϕ and {1}.

Similarly, the set {1, 2} has subsets

ϕ, {1}, {2} and {1, 2}

Q.2.(a) Describe the concept of Pigeon hole principle with suitable example. **5M**

Ans. See. P.1-10, Q.16.

Q.2.(b) Discuss the Chomsky Hierarchy of language. Identify the type of following grammar :

$AB \rightarrow CDB$

$AB \rightarrow CdEB$

$ABcd \rightarrow abCDeBcd$

$B \rightarrow b$ **6M**

Ans. See. P.1-11, Q.22.

(i) $AB \rightarrow CDB$

This is type 0 grammar i.e. unrestrictal.

(ii) $AB \rightarrow CdEB$

This is type 0 grammar i.e. unrestrictal.

(iii) $ABcd \rightarrow abCDeBcd$

This is type 0 grammar i.e. unrestrictal.

(iv) $B \rightarrow b$

This is type 3 grammar i.e. regular.

Q.2.(c) Explain the concept of Diagonalization. **2M**

Ans. See. P.1-5, Q.5.

UNIT - II

(Total Marks : 26)

Q.3.(a) Differentiate between NFA and DFA. **5M**

Ans. See. P.2-13, Q.12.

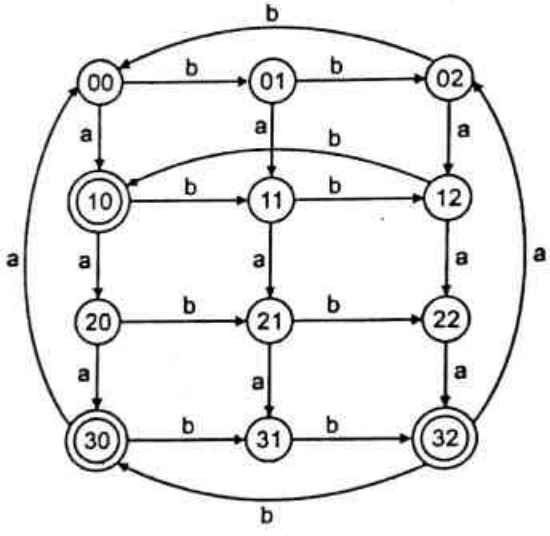
Q.3.(b) Design a DFA accepting following language :

$$L = \left\{ \begin{matrix} w/w \in (a|b)^* \\ n_a(w) \bmod 4 \geq n_b(w) \bmod 3 \end{matrix} \right\}$$

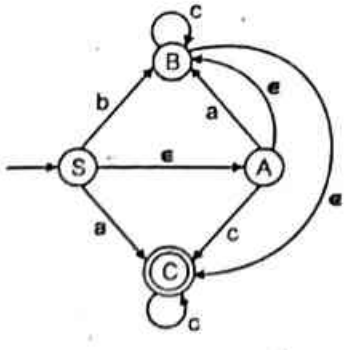
Where $n_a(w) \rightarrow$ Number of a's in w.

$n_b(w) \rightarrow$ Number of b's in w. **8M**

Ans.



Q.4.(a) Convert the following MFA with ϵ -transition into DFA. **7M**



Ans. See. P.2-25, Q.31.

Q.4.(b) Design Moore and Melay machine that gives "1" as its output if the last three digits are 1's. Assume that $\epsilon = 0, 1$. **6M**

Ans. See. P.2-58, Q.82.

UNIT - III

(Total Marks : 28)

Q.5.(a) Obtain deterministic finite automata for the following regular expression :

$(0 + 1)^*10(0 + 1)^* + (0 + 1)^*11(01)^*$ **8M**

Ans. See. P.3-22, Q.29.

Q.5.(b) Obtain right linear grammar equivalent to the following left linear grammar :

$S \rightarrow Sab / Aa$

$A \rightarrow Abb / bb$ **6M**

Ans. See. P.3-41, Q.52.

Q.6.(a) Using pumping lema, prove that the language:

$L = \{a^{i3} / i \geq 1\}$

is not regular. **4M**

Ans. $L = \{a^{i3} / i \geq 1\}$

Step 1 :

$i = 1$, length $= a^{1^3} = a = 1$

$i = 2$, length $= a^{2^3} =$ aaaaaaaa $= 8$

$i = 3$, length $= a^{3^3} = 27$

$i = 4$, length $= a^{4^3} = 64$

Hence, length of string in L is always perfect cube.

Step 2 :

Assume L is regular

Let n is called as constant of

Pumping lemma.

Let $W \in L$ and $W = xyz$

$\therefore W = a^{n^3}$

$\therefore |W| = n^3 = |xyz|$

Step 3 :

For $i = 1$

$xyz \in L$

For $i = 2$

$1 \leq |y| \leq n$

$1 \leq |xy^2z| \leq n$ .....(1)

But xy^2z is a combination of y and xyz.

Hence we will add the value of $|xyz| = n^3$ on both side of equation (1).

$n^3 + 1 \leq |xy^2z| \leq n + n^3$

Subtract 1 and add 1

$n^3 \leq xy^2z \leq n + n^3 + 1$

For $n = 1$

$1 < xy^2z < 1 + 1 + 1$

$1 < xy^2z < 3$

$\therefore$ length = 2 (not a perfect cube)

$\therefore$ Hence length string in L is not always a perfect cyber

Hence our assumption that L is regular is wrong

Hence by applying pumping lemma we prove that L is not regular.

Q.6.(b) Find grammar generating :

$L = (0^m 1^m 2^j / m \geq 1, j \geq 0)$ **6M**

Ans. See. P.3-64, Q.113.

Q.6.(c) Find the equivalent CNF for the following grammar :

$S \rightarrow aB / ab$

$A \rightarrow aAB / a$

$B \rightarrow ABb / b$ **4M**

Ans. See. P.3-85, Q.172.

UNIT - IV

(Total Marks : 28)

Q.7.(a) Design PDA for :

$L_a = \{w c w^R / w \in (a, b)^*\}$

w^R = Reverse of w

Also explain the stack execution with valid string. **8M**

Ans. See. P.4-15, Q.22.

Q.7.(b) Convert the grammar :

$S \rightarrow SOS / SOSOS1S / S1SOSOS / \epsilon$

to a PDA that accepts the same language by empty stack. **6M**

Ans. $S \rightarrow SOS / SOSOS1S / S1SOSOS / \epsilon$

The given grammer is not in GNF. So convert this grammer into GNF.

$S \rightarrow 0$ ($\because S \rightarrow \epsilon$)

$S \rightarrow 0\,S\,C_0\,S\,C_1\,S$

$S \rightarrow 1\,SC_0\,SC_0S$

$C_0 \rightarrow 0$

$C_1 \rightarrow 1$

$S \rightarrow \epsilon$

∴ Equivalent pushdown automata M is

$\delta(q, 0, s) = (q, s), (q, SC_0\,SC_1\,S), (q, \epsilon)$

$\delta(q, 1, s) = (q_1\,SC_0S\,C_0S), (q, \epsilon)$

$\delta(q, 0, C_0) = (q, \epsilon)$

$\delta(q, 1, C_1) = (q, \epsilon)$

$\delta(q, \epsilon, s) = (q, s), (q, SC_0SC_1S), (q, SC_0SC_0S)$

Considering a string 001

$$(q, 001, S) \Rightarrow (q, 01\,S)$$
$$\Downarrow$$
$$(q, 1, S)$$
$$\Downarrow$$
$$(q, \epsilon, \epsilon)$$

The string is accepted by empty stack.

Q.8.(a) design a PDA to accept all string

over $L = \{a^n\ b^m c^{|m-n|} \mid m, n >= 1\}$

Explain all the cases. **7M**

Ans. **See. P.4-9, Q.13.**

Q.8.(b) Given a grammar for the language N(M), where :

$M = (\{q_0, q_1\}, \{0, 1\}, \{z_0, x\}, \delta, q_0, z_0, \phi)$ and δ is given by

$\delta(q_0, 1, z_0) = \{(q_0, x, z_0)\}\delta, (q_0, \epsilon, z_0) = \{q_0, \epsilon\}$

$\delta(q_0, 1, x) = \{(q_0, xx)\}, \delta(q_1, 1, x) = (q_1, \epsilon)$

$\delta(q_0, 0, x) = (\{q_1, x\}), \delta(q_1, 0, z_0) = \{q_0, z_0\}$ **7M**

Ans. **See. P.4-21, Q.32.**

UNIT - V

(Total Marks : 28)

Q.9.(a) Explain the following terms. **5M**

(i) Multitape turing machine.

Ans. **See. P.5-21, Q.19.**

(ii) Univeral turing machine.

Ans. **See. P.5-27, Q.26(A).**

Q.9.(b) Design a turing machine to compute **8M**

$$f(m, n) = \begin{cases} m - n & \text{if } m > n \\ 0 & \text{if } m \le n \end{cases}$$

Ans. **See. P.5-40, Q.48.**

Q.10.(a) Construct a turing machine accepting the language **8M**

$L = \{a^n\ b^{2n}\ c^n \mid n \ge 1\}$

Ans. **See. P.5-19, Q.16.**

Q.10.(b) Explain the concept of Linear Bounded Automata. **5M**

Ans. **See. P.5-34, Q.40.**

Q.12.(b)(iii) Write a short note on Blank tape halting problem. **2M**

Ans. **See. P.5-26, Q.24.**

UNIT - VI

(Total Marks : 24)

Q.11.(a) Define Ackermann's function. Compute A(1, 1), A(2, 1) and A(2, 2). **5M**

Ans. **See. P.6-38, Q.68.**

Q.11.(b) Explain PCP problem and give solution for following : **6M**

List	x	y
1	b	b^3
2	bab^3	ba
3	ba	a

Ans. **See. P.6-15, 16, Q.18, 19.**

Q.11.(c) Explain Unrestricted Grammar. **2M**

Ans. **Unrestricted Grammar :**

- A type (0) grammar is the one, whose production rule is of the form

$\alpha \rightarrow \beta$ where, α & β are in $(V \cup T)^*$ i.e. they can be any string

of terminals and non-terminal and α & ϵ since there is no restriction on what can appear on the left hand side of production rule, as well as what can appear on right side of the production rule, it is capable of specifying the largest number of language.

- The language specifiable using type(0) grammer are called as recursively enumerable language.

Q.12.(a) Consider the function .

equals (x, y) =1 if x = y

= 0 if $x \neq y$

Show that this function is primitive recursive. **5M**

Ans. See. P.6-38, Q.66(i).

Q.12.(b) Write a short note on :

(i) Recursive function. **3M**

(ii) Undecidability. **3M**

Ans. **(i) Recursive function.**

- A function which calls itself directly or indirectly and terminals after finite number of steps is called as recursive function
- In recursive function, terminating point is known as a base point.
- A recursive function will be good if it has terminating point and it call to itself should be nearer to its terminating point.
- Let g be a function of n + 1 arguments, g' be a function of n arguments, h be a function of m arguments, where, $m \leq n + 2$. We define the function of recursively using g and h function as follows :

If $(x_1, x_2 \ldots\ldots x_n, 0) = g'(y_1, y_2 \ldots\ldots y_n)$ then

$g(x_1 x_2 \ldots x_n, x_{n+1} + 1)$

$= h(x_1, x_2 \ldots\ldots x_n, x_{n+1})\, g(x_1, x_2 \ldots x_n, x_{n+1} + 1)$

- Where g' and h are defined function.

(ii) Undecidability : See. P.6-14, Q.16(a).

Notes :-

SUMMER EXAMINATION - 2016
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	28 M
Unit - III	26 M
Unit - IV	28 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

SUMMER EXAM. - 2016
QUESTION PAPER

Q.1.(a) Answer following : **8M**

(i) Unrestricted grammar is also a context free grammar.

(a) True (b) False

(ii) Type 2 grammar is used in.

(a) Turing machine (b) Push down automata

(c) Linear bounded automata.

(iii) Which of the following is regular grammar and why?

(a) $S \rightarrow aaBa \mid \epsilon$ (b) $S \rightarrow aaB \mid ab \mid a$

(c) $S \rightarrow aSaba \mid ab$

(iv) Which of the following is type 0 grammar but not type 1 and why ?

(a) $S \rightarrow \epsilon$ (b) $aS \rightarrow abaS$

(c) $abSa \rightarrow abA$

(b) Prove that by principal of induction. **5M**

$$1.2.3 + 2.3.4 + \ldots + n(n+1)(n+2) = \frac{n(n+1)(n+2)(n+3)}{4}$$

OR

Q.2.(a) State and define Pigeon-hole principle with example. **5M**

(b) Explain following terms : **5M**

(i) Prefix of string.

(ii) Suffix of string.

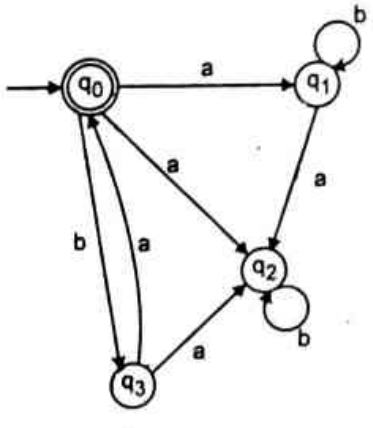
(iii) Proper prefix and proper suffix.

(iv) Alphabet.

(v) Length of string.

(c) Write a note on countability and diagonalization. **3M**

Q.3.(a) Convert the following NFA into equivalent DFA. **6M**



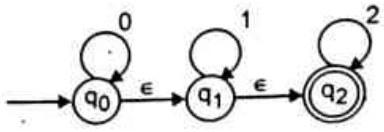
(b) Design a mealy machine to count no. of occurrence of ab and convert the resultant machine into moore M/C. **8M**

OR

Q.4.(a) Design a DFA for a string of decimal digits that are divisible by 3 **6M**

(b) State and explain application of F.A. **5M**

(c) Convert following NFA with ϵ -move into NFA without ϵ -moves. **3M**



Q.5.(a) Design the minimum state DFA from following regular expression.

$(ab)^* bab^* + ab^* (bb)^*$ **8M**

(b) Convert the following right linear grammar into left linear grammar :

$S \rightarrow abS \mid aA$

$A \rightarrow aaA \mid ba$ **5M**

OR

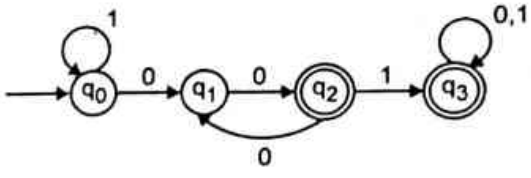
Q.6.(a) Convert the grammar into GNF.

$S \rightarrow aSa \mid bBb$

$B \rightarrow abB \mid aaAa$

$A \rightarrow Aa \mid a$ **6M**

(b) Construct a regular expression from following FA **7M**



Q.7. Construct CFG from the following $L = \{a^n b^n \mid n \geq 1\}$ i.e construct PDA from given language then convert the generated PDA into CFG. **14M**

OR

Q.8.(a) Explain following terms :

(i) NPDA and DPDA

(ii) Model of PDA

(iii) Acceptance by stack and acceptance by final state. **9M**

(b) Design a PDA from CFG

$S \rightarrow aSa \mid aSb \mid a$ **5M**

Q.9.(a) Explain the types of turing machine **6M**

(b) Design a the turing machine to multiply two unary numbers **7M**

OR

Q.10.(a) Design a turing machine to copy a string over $\Sigma = \{a, b\}^*$ **7M**

(b) Explain the model of Linear Bounded Automata **6M**

Q.11.(a) Explain the properties of recursively enumerable language **7M**

(b) Write short note on following

(i) Decidability and solvability

(ii) Primitive recursive function **6M**

OR

Q.12.(a) What is PCP? Explain modified PCP **6M**

(b) Solve using Ackerman function

A(1, 1), A(2, 1), A(2, 2), A(2, 3) **7M**

SUMMER EXAM. - 2016

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Answer following : **8M**

(i) Unrestricted grammar is also a context free grammar.

(a) True

(b) False

(ii) Type 2 grammar is used in.

(a) Turing machine

(b) Push down automata

(c) Linear bounded automata.

(iii) Which of the following is regular grammar and why?

(a) $S \rightarrow aaBa \mid \epsilon$

(b) $S \rightarrow aaB \mid ab \mid a$

(c) $S \rightarrow aSaba \mid ab$

(iv) Which of the following is type 0 grammar but not type 1 and why ?

(a) $S \rightarrow \epsilon$

(b) $aS \rightarrow abaS$

(c) $abSa \rightarrow abA$

Ans.

(i) (a) True

(ii) (b) Push down automata

(iii) (b) $S \rightarrow aaB \mid ab \mid a$ is a regular grammar, in regular grammar the left side of the production rule is a single non terminal or variable, and right side can be a terminal followed by a non-terminal or only a single terminal.

(iv) (c) $abSa \rightarrow abA$ is a type 0 grammar but not type 1

The grammar is of type $\alpha \rightarrow \beta$. In this production for type 1 grammar $\alpha \neq \epsilon$ and $|\beta| \geq |\alpha|$ i.e. they can be any string of terminal and non terminal and the length of the string on the right side must be greater than or equal to the length of the string on the left side.

i.e.

$|abSa| = 4$

$|abA| = 3$

In this production length of the string on the right in less than length of string on the left side so it is not type 1.

Q.1.(b) Prove that by principal of induction. **5M**

$1.2.3 + 2.3.4 + \ldots + n(n+1)(n+2)$

$$= \frac{n(n+1)(n+2)(n+3)}{4}$$

Ans. See P.1-9, Q.15.

Q.2.(a) State and define Pigeon-hole principle with example. **5M**

Ans. See P.1-10, Q.16.

Q.2.(b) Explain following terms : **5M**

(i) Prefix of string.

(ii) Suffix of string.

(iii) Proper prefix and proper suffix.

(iv) Alphabet.

(v) Length of string.

Ans. See P.1-4, Q.2.

(v) Length of string :

- The number of position for symbols in the string is known as length of string.

  Example : 01101 has length 5.

- There are only two symbols (0 and 1) in the string 01101 but 5 position for symbols.

  Notation of length $\omega : |\omega|$

  Example : $|011| = 3$ and $|\epsilon| = 0$.

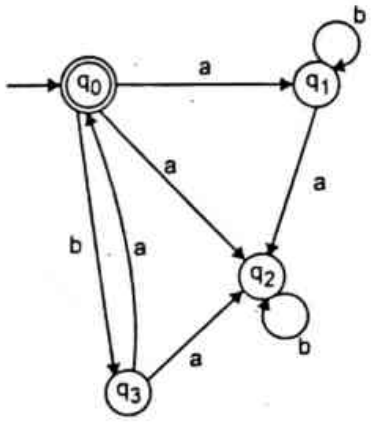
Q.2.(c) Write a note on countability and diagonalization. **3M**

Ans. See P.1-5, 6, Q.5, 7.

UNIT - II

(Total Marks : 28)

Q.3.(a) Convert the following NFA into equivalent DFA. **6M**



Ans. **Transition table for NFA :**

State	a	b
$\rightarrow \{q_0\}$	$\{q_1, q_2\}$	$\{q_3\}$
$\{q_1\}$	$\{q_2\}$	$\{q_1\}$
$\{q_2\}$	—	$\{q_2\}$
$\{q_3\}$	$\{q_0, q_2\}$	—

Transition table for DFA :

State	a	b
$A \rightarrow \{q_0\}$	$\{q_1, q_2\}$	$\{q_3\}$
B $\{q_1, q_2\}$	$\{q_2\}$	$\{q_1, q_2\}$
C $\{q_3\}$	$\{q_0, q_2\}$	—
D $\{q_2\}$	—	$\{q_2\}$
E * $\{q_0, q_2\}$	$\{q_1, q_2\}$	$\{q_3, q_2\}$
F $\{q_3, q_2\}$	$\{q_0, q_2\}$	$\{q_2\}$

Rename states : $\{q_0\} \rightarrow A$, $\{q_1, q_2\} \rightarrow B$, $\{q_3\} \rightarrow C$

$\{q_2\} \rightarrow D$, $\{q_0, q_2\} \rightarrow E$, $\{q_3, q_2\} \rightarrow F$

State	a	b
$* \rightarrow A$	B	C
B	D	B
C	E	—
D	—	D
* E	B	F
F	E	D

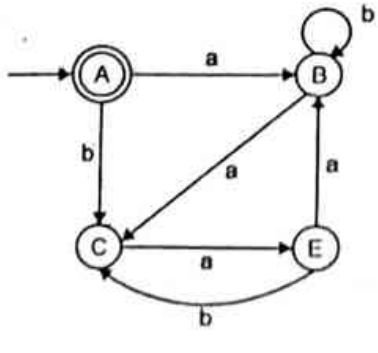
Remove F and replace F with D

State	a	b
* A	B	C
B	D	B
C	E	—
D	—	D
* E	B	D

Remove D and replace D by C.

State	a	b
$* \rightarrow A$	B	C
B	C	B
C	E	—
* E	B	C

Transition diagram :



Q.3.(b) Design a mealy machine to count no. of occurrence of ab and convert the resultant machine into moore M/C. **8M**

Ans.

- Let us assume that the output alphabet

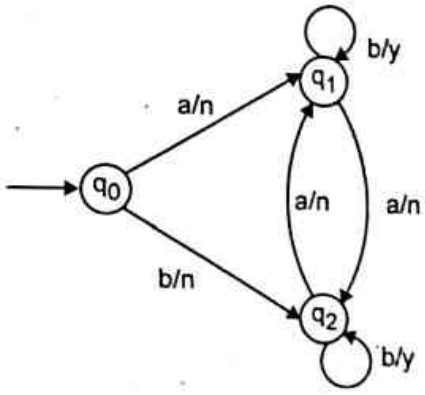
  $\Delta = \{y, n\}$

  'Y' is yes will be the output if string is accepted. and 'n' is 'no' will be the output if string is not accepted.

- The initial state is q_0. from q_0 there will be two transition. One on 'a' and other on 'b' as $\Sigma = \{a, b\}$.

- On 'a' we should make the transition to some other state say q_1 which will look for second 'b' to get 'ab' occurrence of symbol.

Transition diagram for Mealy machine :



Transition table for Mealy machine :

Present state	Next state			
	a		b	
	State	Output	State	Output
$\rightarrow q_0$	q_1	n	q_2	n
q_1	q_2	n	q_1	y
q_2	q_1	n	q_2	y

q_1 is associated with output n and y.

q_2 is associated with output n and y.

Split q_1 and q_2 into two state.

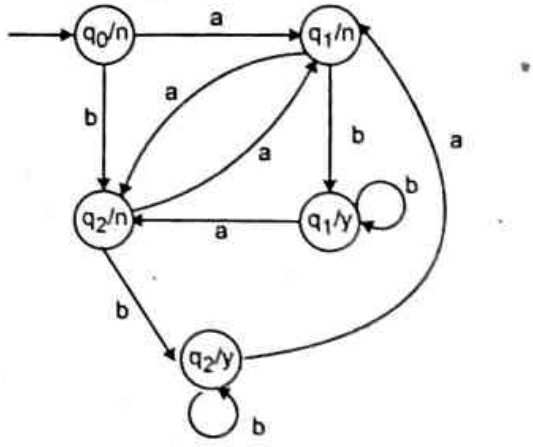
$q_1 \rightarrow q_1 n, q_1 y$

$q_2 \rightarrow q_2 n, q_2 y$

Transition table for Moore machine :

Present state	Next state		Output
	a	b	
$\rightarrow q_0$	q_1 n	q_2 n	n
q_1 n	q_2 n	q_1 y	n
q_1 y	q_2 n	q_1 y	y
q_2 n	q_1 n	q_2 y	n
q_2 y	q_1 n	q_2 y	y

Transition diagram :



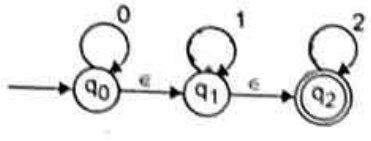
Q.4.(a) Design a DFA for a string of decimal digits that are divisible by 3. **6M**

Ans. See P.2-32, Q.47.

Q.4.(b) State and explain application of F.A. **5M**

Ans. See P.2-10, 8, Q.4, 1.

Q.4.(c) Convert following NFA with ∈ -move into NFA without ∈ -moves. **3M**



Ans. See P.2-21, Q.27.

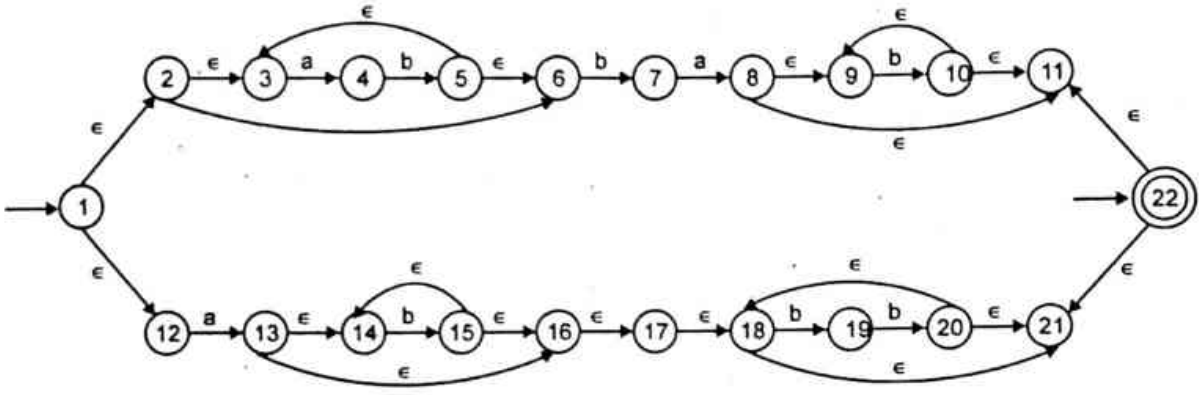
UNIT - III

(Total Marks : 26)

Q.5.(a) Design the minimum state DFA from following regular expression. 8M

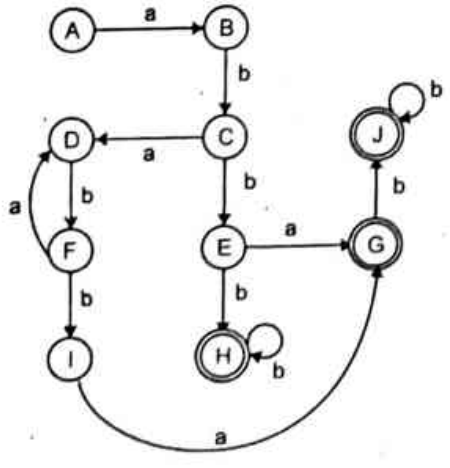
(ab)* bab* + ab* (bb)*

Ans. (ab)* bab* + ab* (bb)*



Transition table :

State	a	b
A = {1, 2, 3, 12}	B = {4, 13, 14}	ϕ
B = {4, 13, 14}	ϕ	C = {5, 3, 6, 15, 14, 16, 17, 18}
C = {5, 3, 6, 15, 14, 16, 17, 18}	D = {4}	E = {7, 15, 14, 16, 17,18, 19}
D = {4}	ϕ	F = {5, 3, 6}
E = {7, 15, 14, 16, 17, 18, 19}	G = {8, 9, 11, 22}	H = {15, 16, 14, 17, 18,20, 18, 19,20,21, 22}
F = {5, 3, 6}	D	I = {7}
*G = {8, 9, 11, 22}	ϕ	J = {10, 8, 11, 22}
*H = {15, 16, 14, 17, 18, 20, 19, 21, 22}	ϕ	H = {15, 14, 16, 17, 18, 21, 22, 19, 20, 21, 22}
I = {7}	G = {8, 9, 11, 22}	ϕ
*J = {10, 9, 11, 22}	ϕ	J = {10, 9, 11, 22}



Q.5.(b) Convert the following right linear grammar into left linear grammar : **5M**

$S \rightarrow abS \mid aA$

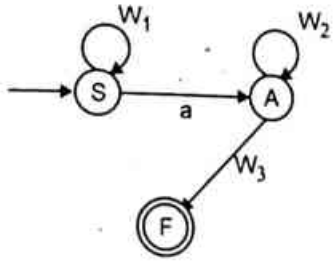
$A \rightarrow aaA \mid ba$

Ans. Let $w_1 = ab$

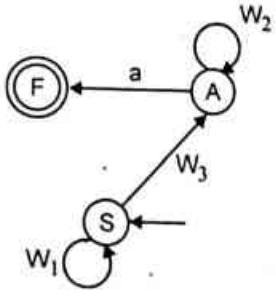
$w_2 = aa$

$w_3 = ba$

Transition graph for right linear grammar is



Transition graph for left linear grammar is



Equivalent left linear grammar is

$S \rightarrow Aw_3 \mid Sw_1$

$A \rightarrow Aw_2 \mid a$

$\therefore S \rightarrow Aba \mid Sab$

$A \rightarrow Aaa \mid a$

Q.6.(a) Convert the grammar into GNF :

$S \rightarrow aSa \mid bBb$

$B \rightarrow abB \mid aaAa$

$A \rightarrow Aa \mid a$ **6M**

Ans. $S \rightarrow aSa \mid bBb$

$B \rightarrow abB \mid aaAa$

$A \rightarrow Aa \mid a$

Now, here

$A \rightarrow a$ is in GNF from

(i) Consider $S \rightarrow aSa$

$S \rightarrow aSA$ $\because (A \rightarrow a)$

$S \rightarrow bBb$

Add $B \rightarrow b$

$\therefore S \rightarrow bBB$

$\therefore S \rightarrow aSA \mid bBB$

(ii) $B \rightarrow abB$

$B \rightarrow aBB$ $\because (B \rightarrow b)$

$B \rightarrow aaAa$

$\rightarrow aAAA$ $\because (A \rightarrow a)$

$\therefore B \rightarrow aBB \mid aAAA$

(iii) $A \rightarrow Aa \mid a$

$\therefore A \rightarrow AA$ $\because (A \rightarrow a)$

$\rightarrow aA$ $\because (A \rightarrow a)$

$\therefore A \rightarrow aA \mid a$

Required GNF from is

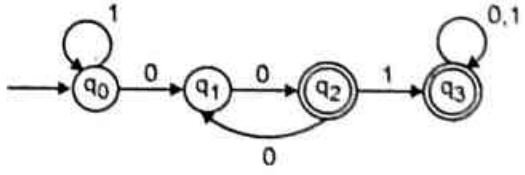
$S \rightarrow aSA \mid bBB$

$B \rightarrow aBB \mid aAAA$

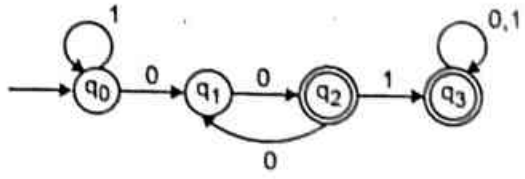
$A \rightarrow aA \mid a$

$B \rightarrow b$

Q.6.(b) Construct a regular expression from following FA. **7M**



Ans.



Equation for all state

$q_0 = q_0 1 + \epsilon$

$q_1 = q_0 0 + q_2 0$

$q_2 = q_1 0$

$q_3 = q_2 1 + 0q_3 + 1q_3$

$\underset{R}{q_0} = \underset{R}{q_0} \underset{P}{1} + \underset{Q}{\epsilon}$

By Arden theorem

$q_0 = \epsilon_1^*$

$\therefore \quad q_0 = 1^*$

$q_1 = 1^* 0 + q_2 0$

Put value of q_2

$q_1 = 1^* 0 + q_1 00$

By Arden theorem, we get

$q_1 = 1^* 0 (00)^*$

$\therefore$ Put value of q_1 in q_2

$q_2 = 1^* 0 (00)^* 0$ ... (1)

$\therefore \quad q_3 = 1^* 0 (00)^* 01 + (0 + 1) q_3$

By Arden theorem

$q_3 = 1^* 0 (00)^* 01 (0 + 1)^*$ ... (2)

$\therefore$ By adding equation 1 and 2

$q_2 + q_3 = 1^* 0 (00)^* 0 + 1^* 0 (00)^* 01 (0 + 1)^*$

$= 1^* 0 (00)^* 0 + 1^* 0 (00)^* 01 + (0 + 1)^*$

UNIT - IV

(Total Marks : 28)

Q.7. Construct CFG from the following $L = \{a^n b^n \mid n \geq 1\}$ i.e construct PDA from given language then convert the generated PDA into CFG **14M**

Ans. **See P.4-8, PS-61, Q.10, Q.8(b).**

Q.8.(a) Explain following terms :

(i) NPDA and DPDA

(ii) Model of PDA

(iii) Acceptance by stack and acceptance by final state. **9M**

Ans.

(i) **See P.4-5, 6, Q.3, 6.**

(ii) **See P.4-5, Q.2.**

(iii) **See P.4-18, Q.28.**

Q.8.(b) Design a PDA from CFG.

$S \rightarrow aSa \mid aSb \mid a$. **5M**

Ans. The grammar is not in GNF. So first obtain equivalent grammar in GNF

$S \rightarrow aSA \mid bSB \mid a$

$A \rightarrow a$

$B \rightarrow b$

Therefore the PDA equivalent to the given grammar is

$M = (\{q\}, \{a, b\}, \{S, A, B\}, \delta, q, S, \phi)$

Where δ is defined by following rules.

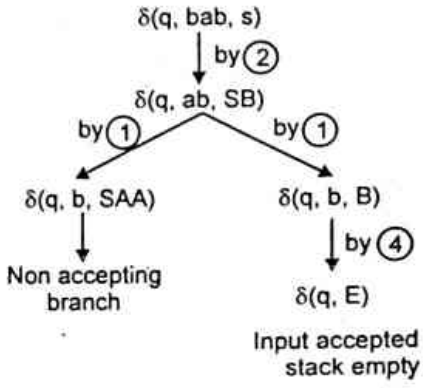
$\delta(q, a, S) = \{(q, SA), (q, \epsilon)\}$ because $S \rightarrow aSA / S \rightarrow a$ ... (1)

$\delta(q, b, S) = \{(q, SB)\}$ ... (2) $S \rightarrow bSB$

$\delta(q, a, A) = \{(q, \epsilon)\}$ ... (3) $A \rightarrow a$

$\delta(q, b, B) = \{(q, \epsilon)\}$ ... (4) $B \rightarrow b$

Let us check for string "bab"



UNIT - V

(Total Marks : 26)

Q.9.(a) Explain the types of turing machine. **6M**

Ans. **See P.5-21, Q.19.**

Q.9.(b) Design a turing machine to multiply two unary numbers. **7M**

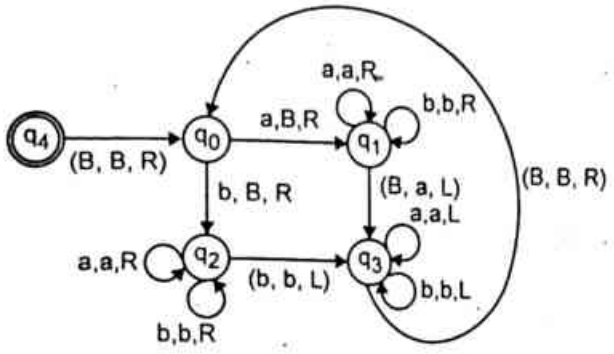
Ans. **See P.PS-54, Q.10(b).**

Q.10.(a) Design a turing M/c to copy a string over $\Sigma = \{a, b\}^*$. **7M**

Ans. Transition table for turing machine is as follows :

State	a	b	B
$\rightarrow q_0$	q_1, B, R	q_2, B, R	q_4, B, N
q_1	q_1, a, R	q_1, b, R	q_3, a, L
q_2	q_2, a, R	q_2, b, R	q_3, b, L
q_3	q_3, a, L	q_3, b, L	q_0, B, R
q_4	—	—	—

Transition diagram :



Q.10.(b) Explain the model of Linear Bounded Automata. **6M**

Ans. **See P.5-34, Q.40.**

UNIT - VI

(Total Marks : 26)

Q.11.(a) Explain the properties of recursively enumerable language. **7M**

Ans. **See P.6-7, Q.1, 2.**

Q.11.(b) Write short note on following :

(i) Decidability and solvability.

(ii) Primitive recursive function. **6M**

Ans.

(i) Decidability :

See P.6-13, Q.15(a).

Solvability :

- A problem is solvable if the language of all of its encoded "yes" instance is recursive. i.e. there is a TM that recognizes the language.
- One method of showing whether a given decision problem in unsolvable is to convert the encoding of the problem into another that we know to be either solvable or unsolvable. This is called reducing one language to another. We can take any encoded instance of one problem.
- Use a TM to compute a corresponding encoded instance of another problem.
- If this other problem has a TM that recognizes the set of "yes encoding" we can run that TM to solve the first problem.
- If $L_1 \leq L_2$ then

(a) If L_2 is recursive then L_1 is also recursive.

(b) If L_1 is not recursive then L_2 is not recursive.

- If P_1 and P_2 are decision problem with L_1 and L_2 the languages of "yes encoding" respectively and if $L_1 \leq L_2$ then

(a) If P_2 is solvable then P_1 is also solvable.

(b) If P_1 is unsolvable then P_2 is also unsolvable.

(ii) Primitive recursive function :

Ans. **See P.6-28, Q.42, 44.**

Q.12.(a) What is PCP? Explain modified PCP. **6M**

Ans. **See P.6-15, 18, Q.18, 23.**

Q.12.(b) Solve using Ackerman function

A(1, 1), A(2, 1), A(2, 2), A(2, 3). **7M**

Ans. **See P.PS-17, Q.12(a).**

Notes :-

WINTER EXAMINATION - 2015
B.E. IV SEM. (COMPUTER TECHNOLOGY) (CBS)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	26 M
Unit - III	26 M
Unit - IV	26 M
Unit - V	26 M
Unit - VI	28 M
Questions covered in VBD	160 M (100 %)

WINTER EXAM. - 2015
QUESTION PAPER

Q.1.(a) Explain the following terms :

(i) Closure of relation.

(ii) Countability.

(iii) Set operation.

(iv) String.

(v) Relation. **10M**

(b) Explain in brief diagonalization principle **2M**

OR

Q.2.(a) Explain generalized pigeonhole principle. **3M**

(b) Proye the following by principle of induction

$$\sum_{K=1}^{n} K^2 = \frac{n(n+1)(2n+1)}{6}$$ **5M**

(c) Prove that for any finite set $A, |2^A| = 2^{|A|}$. **8M**

Q.3.(a) Construct a Moore machine whose output is complement of a binary number. Assume input to Moore machine in binary digits from MSB and LSB **5M**

(b) Construct a finite automata accepting string over {0, 1} that do not contain '11' as substring b = {|p − r|} contains at least one '00' as substring. **8M**

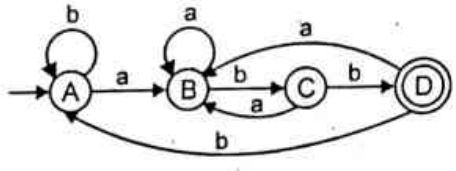
OR

Q.4.(a) Construct a DFA accepting strings over decimal digits that represents decimal number divisible by 6.

(Hint : Number divisible by 2 and 3 are divisible by 6). **10M**

(b) Differentiate between NFA, NFA ϵ -closure and DFA. **3M**

Q.5.(a) Show that the language accepted by following DFA can be represented by the regular expression (a/b)* abb. **7M**



(b) Give the statement of pumping lemma and prove that :

$L = \{a^P \mid P \text{ is prime}\}$, is not regular. **6M**

OR

Q.6.(a) Reduce the following grammar :

$S \rightarrow ABC \mid BaB$

$A \rightarrow aA \mid BaC \mid aaa$

$B \rightarrow bBb \mid a$

$C \rightarrow CA \mid AC$. **5M**

(b) Show that the grammar :

$S \rightarrow aB \mid ab$

$A \rightarrow aAB \mid a$

$B \rightarrow ABb \mid b$

is ambiguous. **5M**

(c) If w is in L (G), such that $|w| = l$. How long will be the derivation of w, if G is in :

(i) CNF (ii) GNF? **3M**

Q.7.(a) Define Push Down Automata and explain acceptance of string by PDA in final state and in NULL stack. **5M**

(b) Construct PDA for the language

$L = \{a^i b^j c^j \mid i, j \geq 1\}$. **8M**

OR

Q.8.(a) Explain closure properties of CFL. **5M**

(b) Construct PDA for the language :

$L \{a^p b^q c^r \mid p, q, r \geq 1 \text{ and } q = |p - r|\}$. **8M**

Q.9.(a) Construct a Turing Machine for the language :

$L = \{a^n b^n c^n \mid n \geq 1\}$. **7M**

(b) Describe linear bounded automata and the language accepted by LBA. **6M**

OR

Q.10. **(a)** Construct a Turing Machine for proper subtraction i.e. :

$$m - n = \begin{cases} m - n, & m \geq n \\ 0, & \text{otherwise} \end{cases}$$ **8M**

(b) Give the grammar for the language $L \{a^n b^n c^n \mid n \geq 1\}$ and derivation for the string $a^3 b^3 c^3$. **5M**

Q.11. **(a)** Show that the halting problem is unsolvable. **6M**

(b) Explain the closure properties of recursive and recursively enumerable language. **8M**

VBD

OR

Q.12.(a) Define Primitive Recursive function and show that :

$$DIFF(x, y) = \begin{cases} x - y & \text{if } x \geq y \\ 0 & \text{otherwise} \end{cases}$$

is primitive recursive. **8M**

(b) Given the two lists below. Is PCP solution exist for the list? If yes, show the solution : **6M**

i	List A	List B
1	1	11
2	10111	10
3	10	0

WINTER EXAM. - 2015
SOLUTION
(Unit-Wise)

UNIT - I
(Total Marks : 28)

Q.1.(a) Explain the following terms :

(i) Closure of relation.

(ii) Countability.

(iii) Set operation.

(iv) String.

(v) Relation. **10M**

Ans. See P.1-5, 3, 4, Q.4, 5(2), 1, 2(iii), 3.

Q.1.(b) Explain in brief diagonalization principle. **2M**

Ans. See P.1-5, Q.5.

Q.2.(a) Explain generalized pigeonhole principle. **3M**

Ans. See P.1-10, Q.17.

Q.2.(b) Prove the following by principle of induction

$$\sum_{K=1}^{n} K^2 = \frac{n(n+1)(2n+1)}{6}$$ **5M**

Ans. See P.1-8, Q.11.

Q.2.(c) Prove that for any finite set A,

$|2^A| = 2^{|A|}$. **8M**

Ans. Consider a set A = {1, 2, 3, 4}

$\eta(A) = 4$

$\therefore$ L.H.S $= |2^A| = |2^4| = |16| = 16$

R.H.S $= 2^{|A|} = 2^{|4|} = 2^4 = 16$

$\therefore$ L.H.S = R.H.S

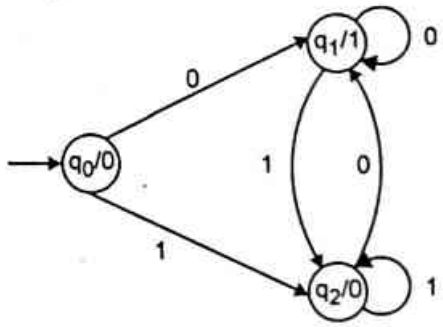
$\therefore$ $|2^A| = 2^{|A|}$

UNIT - II
(Total Marks : 26)

Q.3.(a) Construct a Moore machine whose output is complement of a binary number. Assume input to Moore machine in binary digits from MSB and LSB. **5M**

Ans. As we know, to generate complement of given binary number, the simple logic is that if input is 0 then the output will be 1 and if input is 1 then the output will be 0.

Transition diagram of moor machine is shown below :



Transition table can be defined as :

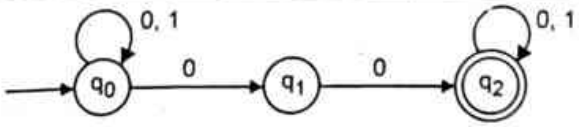
Present state	Next state		Output
	0	1	
→ q_0	q_1	q_2	0
q_1	q_1	q_2	1
q_2	q_1	q_2	0

Q.3.(b) Construct a finite automata accepting string over {0, 1} that do not contain '11' as substring b = {|p – r|} contains at least one '00' as substring. **8M**

Ans. **Finite condition :**

Containing 00 as substring

Regular expression $(0 + 1)^* 00 (0 + 1)^*$



Σ / Q	0	1
→ q_0	q_0, q_1	q_0
q_1	q_2	-
* q_2	q_2	q_2

Transition table of DFA :

Σ / Q	0	1
q_0	$\{q_0, q_1\}$	q_0
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	q_0
* $\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$
* $\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$

Renaming all state are,

$\{q_0\} = A$, $\{q_0, q_1\} = B$, $\{q_0, q_1, q_2\} = C$

$\{q_0, q_2\} = D$

Σ / Q	0	1
A	B	A
B	C	A
* C	C	D
* D	C	D

Remove D and replace D by C

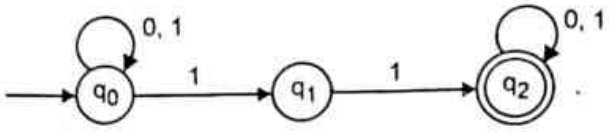
Σ / Q	0	1
A	B	A
B	C	A
* C	C	D

Infinite condition :

Do not contain 11 as substring.

Complement of condition : It contains 11 as substring

R.E $(0 + 1)^* 11(0 + 1)^*$



Transition table of NFA :

Σ / Q	0	1
q_0	q_0	$\{q_0, q_1\}$
q_1	-	q_2
* q_2	q_2	q_2

Transition table for DFA :

Σ / Q	0	1
q_0	q_0	$\{q_0, q_1\}$
$\{q_0, q_1\}$	q_0	$\{q_0, q_1, q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$

Renaming the states,

$\{q_0\} = P$, $\{q_0, q_1\} = Q$, $\{q_0, q_1, q_2\} = R$

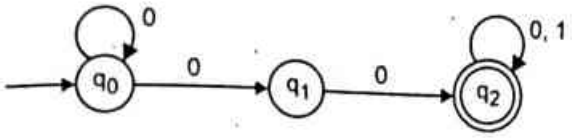
$\{q_0, q_2\} = S$

Σ / Q	0	1
P	P	Q
Q	P	R
* R	S	R
* S	S	R

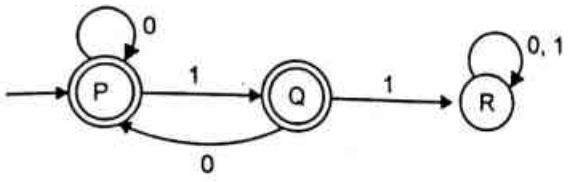
Remove S and replace S by R

Σ / Q	0	1
→ P	P	Q
Q	P	R
* R	R	R

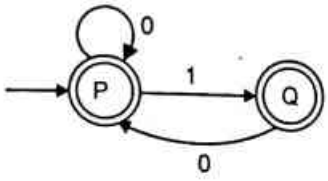
Transition diagram :



Now, take complement condition



R is dead state

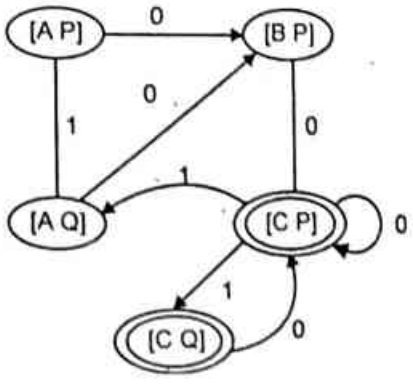


Q \ Σ	0	1
→* P	P	Q
* Q	P	-

Transition table for final DFA :

Q \ Σ	0	1
[AP]	[BP]	[AQ]
[BP]	[CP]	[AQ]
[AQ]	[BP]	[A]
*[CP]	[CP]	[CQ]
*[CQ]	[CP]	[C]

∴ Transition graph for final DFA



Q.4.(a) Construct a DFA accepting strings over decimal digits that represents decimal number divisible by 6.

(Hint : Number divisible by 2 and 3 are divisible by 6). **10M**

Ans. Finite automata which is divisible by 3.

Consider state as = $\{S_0, S_1, S_2\}$

In this case formula for solving given equation is 2

× odd remainder + next list

For initial state q_0 :

(i) $2 \times 0 + 0 = 0$ = remainder is zero so output is q_0.

(ii) $2 \times 0 + 1 = 1 = q_1$

(i) $2 \times 1 + 0 = 2 = q_2$

(ii) $2 \times 1 + 1 = 3$ = but 3 is divisible by 3 hence output will be q_0

(i) $2 \times 2 + 0 = 4$, Here remainder is 1 i.e. q_1.

(ii) $2 \times 2 + 1 = 5$, Here remainder is 2, i.e. q_2

Q \ Σ	0	1
→* q_0	q_0	q_1
q_1	q_2	q_0
q_2	q_1	q_2

Finite automata which is divisible by 2 :

Consider state as = $\{S_0, S_1\}$

For initial state q_0 :

(i) $2 \times 0 + 0 = 0 = q_0$

(ii) $2 \times 0 + 1 = 1 = q_1$

(i) $2 \times 1 + 0 = 2 = q_0$

(ii) $2 \times 1 + 1 = 3 = q_1$

Q \ Σ	0	1
→* q_0	q_0'	q_1'
q_1'	q_0'	q_1'

Final transition table which is divisible by 6 is :

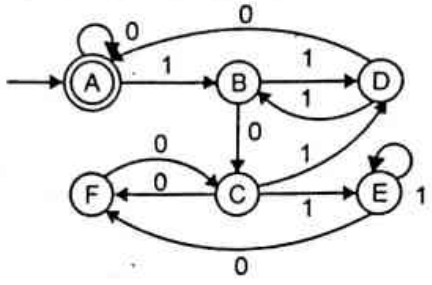
Q \ Σ	0	1
$q_0\ q_0'$	$q_0\ q_0'$	$q_1\ q_1'$
$q_1\ q_1'$	$q_2\ q_0'$	$q_0\ q_1'$
$q_2\ q_0'$	$q_1\ q_0'$	$q_2\ q_1'$
$q_0\ q_1'$	$q_0\ q_0'$	$q_1\ q_1'$
$q_2\ q_1'$	$q_1\ q_0'$	$q_2\ q_1'$
$q_1\ q_0'$	$q_2\ q_0'$	$q_0\ q_1'$

Renaming states,

$\{q_0\ q_1'\} = A$, $\{q_1\ q_1'\} = B$, $\{q_2\ q_0'\} = C$,

$\{q_0\ q_1'\} = D$, $\{q_2\ q_1'\} = E$, $\{q_1\ q_0'\} = F$.

Q \ Σ	0	1
*A	A	B
B	C	D
C	F	E
D	A	B
E	F	E
F	C	D

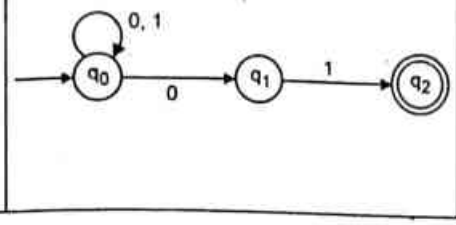


Q.4.(b) Differentiate between NFA, NFA ∈ -closure and DFA. 3M

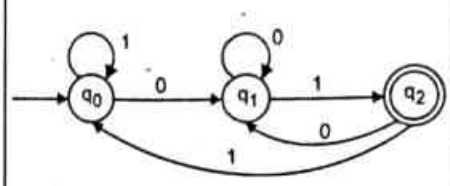
Ans.

Sr. no.	NFA	DFA	NFA ∈ closure
(1)	If the basic finite automata model is modified to allow zero, one or more transition from state to another state on the same input symbol The new finite automata is called NFA.	DFA consist of a finite set of state of transitions from state to state occur on an input symbols chosen from an alphabet Σ.	If a finite automata is modified to permit transitions without input symbol along with zero, one or more transition on the input symbols, then we get a NFA without ∈ -more.
(2)	NFA is denoted by 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where Q = finite set of states Σ = set of input symbols q_0 = start state F = final state δ = transition function from $Q \times \Sigma$ to 2^Q.	DFA m denoted by 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where Q = set of states Σ = set of input symbols q_0 = start state F = final state δ = transition function from $Q \times \Sigma$ to Q.	NFA with ∈ -moves is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ with all components as before but δ, the transition function maps to $Q \times (\Sigma \cup \{\epsilon\})$ to 2^Q.
(3)	In NFA there exists one or more transition from state to state on same input symbol.	In DFA there exists exactly one transition from state to state on same input symbol.	∈ - closure will never be an empty set, because q is always reachable from itself without consuming any input symbol.

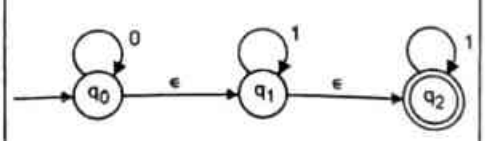
(4) Example :



Example :



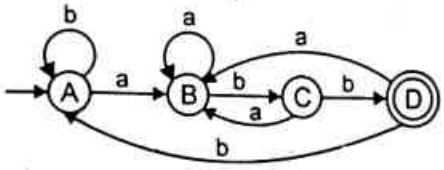
Example :



UNIT - III

(Total Marks : 26)

Q.5.(a) Show that the language accepted by following DFA can be represented by the regular expression (a/b)* abb. **7M**



Ans. Equations for all states

$A = Ab + Db + \epsilon$ ... (1)

$B = Ba + Aa + Ca + Da$ ... (2)

$C = Bb$ ... (3)

$D = Cb$ ... (4)

Put value of C and D in equation (2) we get

$B = Ba + Aa + Bba + Cba$

$B = Ba + Aa + Bba + Bbba$

$B = B(a + ba + bba) + Aa$

$B = Aa (a + ba + abb)^*$

$C \rightarrow Aa(a + ba + abb)^* b$

$D \rightarrow Aa (a + ba + abb)^* bb$

$A \rightarrow Ab + Aa (a + ba + abb)^* bbb$

$A \rightarrow A (a + b) (a + ba + abb)^* bbb + \epsilon$

$A \rightarrow [(a + b) (a + ba + abb)^* bbb]^*$

$\rightarrow (a + b)^* (a + ba + abb)^* (bbb)^*$

$B \rightarrow (a + b)^* (a + ba + abb)^* (bbb)^* a [a + abb + ba]^*$

$\rightarrow (a + b)^* a(bbb)^* (a + ba + abb)^*$

$C \rightarrow (a + b)^* a (bbb)^* (a + ba + abb)^* b$

$D \rightarrow (a + b)^* a (bbb)^* (a + ba + abb)^* bb$

$D \rightarrow (a + b)^* abb (a + ba + abb)^* (bbb)^*$

Q.5.(b) Give the statement of pumping lemma and prove that

$L\{a^P \mid P \text{ is prime}\}$, is not regular. **6M**

Ans. See P.3-53, Q.81, 83.

Q.6.(a) Reduce the following grammar :

$S \rightarrow ABC \mid BaB$

$A \rightarrow aA \mid BaC \mid aaa$

$B \rightarrow bBb \mid a$

$C \rightarrow CA \mid AC$. **5M**

Ans. A derives a terminal string

B derives a terminal string

But C does not derives a terminal string

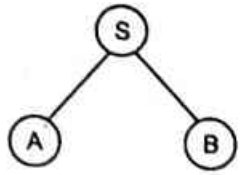
∴ Remove C, we get

$S \rightarrow AB \mid BaB$

$A \rightarrow aA \mid Ba \mid aaa$

$B \rightarrow bBb \mid a$

Draw dependency graph



Both A and B are reachable

∴ It is useful

∴ Reduce grammar is :

$S \rightarrow AB \mid BaB$

$A \rightarrow aA \mid Ba \mid aaa$

$B \rightarrow bBb \mid a$

Q.6.(b) Show that the grammar

$S \rightarrow aB \mid ab$

$A \rightarrow aAB \mid a$

$B \rightarrow ABb \mid b$

is ambiguous. **5M**

Ans. **See P.3-67, Q.123.**

Q.6.(c) If w is in L (G), such that $|w| = l$. How long will be the derivation of w, if G is in :

(i) CNF

(ii) GNF **3M**

Ans.

(i) In CNF of $|\omega| = l$ The length of derivation of ω will be $2l - 1$

(ii) In GNF the length of derivation of ω will be l i.e. length of string

UNIT - IV

(Total Marks : 26)

Q.7.(a) Define Push Down Automata and explain acceptance of string by PDA in final state and in NULL stack. **5M**

Ans. **See P.4-4, 4-18, Q.1, 28.**

Q.7.(b) Construct PDA for the language

$L = \{a^i b^i c^j \mid i, j \geq 1\}$. **8M**

Ans. **See P.4-12, Q.15.**

Q.8.(a) Explain closure properties of CFL. **5M**

Ans. **See P.4-26, Q.40.**

Q.8.(b) Construct PDA for the language :

$L\{a^p b^q c^r \mid p, q, r \geq 1 \text{ and } q = |p - r|\}$. **8M**

Ans. **See P.4-9, Q.13.**

UNIT - V

(Total Marks : 26)

Q.9.(a) Construct a Turing Machine for the language :

$L = \{a^n b^n c^n \mid n \geq 1\}$. **7M**

Ans. **See P.5-15, Q.10.**

Q.9.(b) Describe linear bounded automata and the language accepted by LBA. **6M**

Ans. See P.5-34, Q.40.

Q.10. (a) Construct a Turing Machine for proper subtraction i.e. :

$$m - n = \begin{cases} m - n, & m \geq n \\ 0, & \text{otherwise} \end{cases}$$ **8M**

Ans. See P.5-40, Q.48.

Q.10.(b) Give the grammar for the language $L = \{a^n b^n c^n \mid n \geq 1\}$ and derivation for the string $a^3 b^3 c^3$. **5M**

Ans. $L = \{a^n b^n c^n \mid n \geq 1\}$

The grammar in $\{\{A, B, C, S\}, \{a, b, c\} P, S\}$ where P in given by

$S \rightarrow aAbBcC$

$A \rightarrow aA \mid \epsilon$

$B \rightarrow bB \mid \epsilon$

$C \rightarrow cC \mid \epsilon$

Derivation of string $a^3b^3c^3$

$\Rightarrow S \rightarrow a\underline{A}bBcC$

$\rightarrow aa\underline{A}bBcC$

$\rightarrow aaa\underline{A}bBcC$

$\rightarrow aaab\underline{B}cC$

$\rightarrow aaabb\underline{B}cC$

$\rightarrow aaabbb\underline{B}cC$

$\rightarrow aaabbbc\underline{C}$

$\rightarrow aaabbbcc\underline{C}$

$\rightarrow aaabbbccc\underline{C}$

$\rightarrow aaabbbccc$

UNIT - VI

(Total Marks : 28)

Q.11.(a) Show that the halting problem is unsolvable. **6M**

Ans. See P.6-20, Q.26.

Q.11.(b) Explain the closure properties of recursive and recursively enumerable language. **8M**

Ans. See P.6-7, 8, 9, Q.2, 4, 5, 6.

Q.12.(a) Define Primitive Recursive function and show that :

$$DIFF(x, y) = \begin{cases} x - y & \text{if } x \geq y \\ 0 & \text{otherwise} \end{cases}$$

is primitive recursive. **8M**

Ans. See P.6-31, Q.51.

Q.12.(b) Given the two lists below. Is PCP solution exist for the list? If yes, show the solution : **6M**

i	List A	List B
1	1	11
2	10111	10
3	10	0

Ans. See P.6-15, 16, Q.18, 19.

Notes :-

SUMMER EXAMINATION - 2016
B.E. IV SEM. (COMPUTER TECHNOLOGY) (CBS)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	28 M
Unit - III	26 M
Unit - IV	26 M
Unit - V	33 M
Unit - VI	21 M
Questions covered in VBD	160 M (100 %)

SUMMER EXAM. - 2016
QUESTION PAPER

Q.1.(a) Prove the following relations using principle of Mathematical Induction : **8M**

(a) $1.2.3 + 2.3.4 + \ldots + n(n+1)(n+2) = \frac{n(n+1)(n+2)(n+3)}{4}$

(b) $\sum_{i=1}^{n} \frac{1}{i(i+1)} = \frac{n}{n+1}$ is true for all $n \geq 0$.

(b) Explain Pigeonhole principle with suitable example. **5M**

OR

Q.2.(a) Explain in detail about Chomsky hierarchy of grammar. **8M**

(b) Find R^* and R^+ if $R = \{(1, 1), (1, 2), (2, 3), (3, 1), (1, 4), (2, 4)\}$ on a set $A = \{1, 2, 3, 4\}$. **5M**

Q.3.(a) Design a DFA for string which consists of even number of 0's and odd number of 1's over $\Sigma = \{0, 1\}$. **6M**

(b) Design Moore machine for the input from $\Sigma = \{0, 1, 2\}$ which prints the residue mod 5 of the input treated as a ternary (base 3, with digits 0, 1, 2) number. **8M**

OR

Q.4.(a) Construct a minimum state DFA equivalent to the NFA given below : **8M**

NFA = $(\{A, B, C, D\}, \{a, b\}, \delta, A, \{B, D\})$

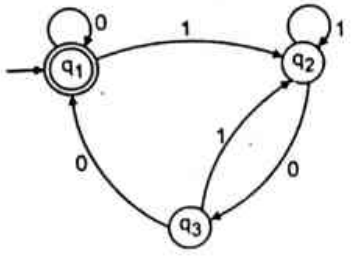
where δ is defined using Transition Table :

	a	b
A	{B, D}	{B}
B	{C}	{B, C}
C	{D}	{A}
D	ϕ	{A}

(b) Differentiate between : **6M**

(i) NFA and DFA (ii) Moore and Mealy machine.

Q.5.(a) Construct RE from Transition Diagram given below. **6M**



(b) Explain Pumping Lemma for Regular Languages. Also prove that $L = \{a^n b^{2n} \mid n \geq 1\}$ is non regular language. **7M**

OR

Q.6.(a) Describe the language defined by following regular expression.

(i) $a(a \mid b)^*b$

(ii) $c^* (a \mid (bc^*))^*$ **7M**

(b) Consider the grammar : **6M**

$S \rightarrow 0B/1A$

$A \rightarrow 0/0S/1AA$

$B \rightarrow 1/1S/0BB$

For the string "00110101", find

(i) LMD (Leftmost Derivation)

(ii) RMD (Rightmost Derivation)

Q.7.(a) State and explain closure properties of context free languages. **5M**

(b) Give formal definition of PDA with block diagram. Explain it. **4M**

(c) Consider the context free language $L = \{a^{2n} b^n / n \geq 0\}$, obtain its equivalent CFG. **4M**

OR

Q.8.(a) Construct PDA for the matching parenthesis "("(" ")")". **7M**

(b) Construct PDA for $L = \{w \subset \omega^R / C$ is some special symbol not in w$\}$ **6M**

Q.9.(a) Explain Linear Bounded Automata. **6M**

(b) Design a Turing Machine for following language. **8M**

$L = \{a^n b^n c^n / n \geq 1\}$

OR

Q.10.(a) Design a Turing Machine that can accept the string over (0, 1) to recognize all strings containing even number of 0's. **8M**

(b) Design T.M. for language to find 2's complement of a binary number. **6M**

Q.11.(a) Define Ackerman's function. Compute **8M**

(i) A (1, 1)

(ii) A (2, 1)

(iii) A (2, 3)

(b) Explain Church's Hypothesis in brief. **5M**

OR

Q.12.(a) What do you mean by PCP? Decide whether the following (A, B) pair have a solution or not. If Yes, give a solution. If No, then why?

(i) $A(b, bab^2, ba), B = \{b^2, ba, aa\}$

(ii) A(01, 1, 11), B = {011, 10, 11} **2M**

(b) Define decidable and undecidable language. **4M**

SUMMER EXAM. - 2016
SOLUTION
(Unit-Wise)

UNIT - I
(Total Marks : 26)

Q.1.(a)(a) Prove the following relations using principle of Mathematical Induction : **4M**

$$1.2.3 + 2.3.4 + \ldots + n(n+1)(n+2) = \frac{n(n+1)(n+2)(n+3)}{4}$$

Ans. **See P.1-9, Q.15.**

Q.1.(a)(b) Prove the following relations using principle of Mathematical Induction :

$\sum_{i=1}^{n} \frac{1}{i(i+1)} = \frac{n}{n+1}$ is true for all $n \geq 0$. **4M**

Ans. **See P.PS-67, Q.1(b).**

Q.1.(b) Explain Pigeonhole principle with suitable example. **5M**

Ans. **See P.1-10, Q.16.**

Q.2.(a) Explain in detail about Chomsky hierarchy of grammar. **8M**

Ans. **See P.1-11, Q.22.**

Q.2.(b) Find R^* and R^+ if R = {(1, 1), (1, 2), (2, 3), (3, 1), (1, 4), (2, 4)} on a set A = {1, 2, 3, 4}. **5M**

Ans. R = {(1, 1), (1, 2), (2, 3), (3, 1), (1, 4), (2, 4)}

Set A = {1, 2, 3, 4}

R^2 = {(1, 1), (1, 2), (2, 3), (3, 1), (1, 4) (2, 4)}

{(1, 1), (1, 2), (2, 3), (3, 1), (1, 4) (2, 4)}

= {(1, 2) (1, 4) (1, 3) (2, 1) (3, 2) (3, 4) (3, 1)}

$R^3 = R^2 . R$

= {(1, 2) (1, 4) (1, 3) (2, 1) (3, 2) (3, 4) (3, 1)}

{(1, 1), (1, 2), (2, 3), (3, 1), (1, 4) (2, 4)}

= {(1, 3) (1, 4) (1, 1) (2, 1) (2, 2) (2, 4) (3, 3)}

(3, 4) (3, 1) (3, 2) (3, 4)}

$R^4 = R^3 . R$

= {(1, 3) (1, 4) (1, 1) (2, 1) (2, 2) (2, 4) (3, 3)

(3, 4) (3, 1) (3, 2) (3, 4)}

{(1, 1), (1, 2), (2, 3), (3, 1), (1, 4) (2, 4)}

= {(1, 1) (1, 2) (1, 4) (2, 1) (2, 2) (2, 4) (2, 3)

(3, 1) (3, 2) (3, 4) (3, 3)}

$R^5 = R^4 . R$

= {(1, 1) (1, 2) (1, 4) (2, 1) (2, 2) (2, 4) (2, 3)

(3, 1) (3, 2) (3, 4) (3, 3)}

{(1, 1), (1, 2), (2, 3), (3, 1), (1, 4) (2, 4)}

= {(1, 2) (1, 4) (1, 3) (2, 1) (2, 2) (2, 4) (2, 3)

(2, 4) (3, 1) (3, 2) (3, 4) (3, 3)}

$R^6 = R^5 . R$

= {(1, 2) (1, 4) (1, 3) (2, 1) (2, 2) (2, 4) (2, 3)

(2, 4) (3, 1) (3, 2) (3, 4) (3, 3)}

{(1, 1), (1, 2), (2, 3), (3, 1), (1, 4) (2, 4)}

= {(1, 2) (1, 4) (1, 1) (2, 1) (2, 2) (2, 4) (2, 3)

(3, 1) (3, 2) (3, 4) (3, 3)}

$= R^4$

$\therefore R^* = R \cup R^2 \cup R^3 \cup R^4 \cup R^5$

= {(1, 1), (1, 2), (1, 4), (1, 3), (2, 1) (2, 2) (2, 3)

(2, 4) (3, 1) (3, 2) (3, 4) (3, 3)}

$\therefore R^+ = R^* \cup$ {(1, 1) (2, 2) (3, 3) (4, 4)}

= {(1, 1), (1, 2), (1, 4), (1, 3), (2, 1) (2, 2) (2, 3)

(2, 4) (3, 1) (3, 2) (3, 4) (3, 3) (4, 4)}

UNIT - II

(Total Marks : 28)

Q.3.(a) Design a DFA for string which consists of even number of 0's and odd number of 1's over $\Sigma = \{0, 1\}$. **6M**

Ans. See P.2-31, Q.43.

Q.3.(b) Design Moore machine for the input from $\Sigma = \{0, 1, 2\}$ which prints the residue mod 5 of the input treated as a ternary (base 3, with digits 0, 1, 2) number. **8M**

Ans. The ternary number is made up of 0, 1 and 2. We can interprete of ternany number n can be

(1) If we write '0' after 'n' then the no. becomes $3 \times n + 0$.

(2) If we can write '1' after 'n' then the no becomes $3n + 1$.

(3) If we write '2' after n then the no. becomes $3n + 2$

Now for residue module 5 we get remainders 0, 1, 2, 3, 4.

Then we assume various state for these remainder as

q_0 as remainder 0 state

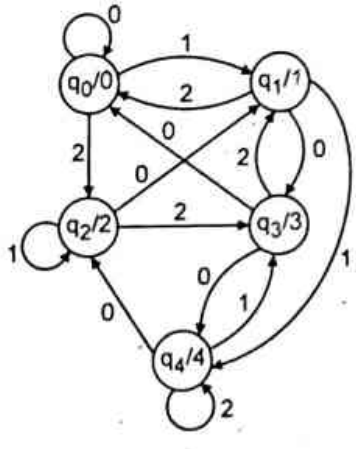
q_1 as remainder 1 state

q_2 as remainder 2 state

q_3 as remainder 3 state

q_4 as remainder 4 state

Now we design Moore machine



R	I/P		
0	0	=	0
0	1	=	1
0	2	=	2

R	I/P		
1	0	=	$3 \times n = 3 \times 1 = 3$
1	1	=	$3 \times 1 + 1 = 4$
1	2	=	$3 \times 1 + 2 = 5$

R	I/P		
2	0	=	$3n = 3 \times 2 = 6 = 1$
2	1	=	$3 \times 2 + 1 = 7 = 2$
2	2	=	$3 \times 2 + 2 = 8 = 3$

R	I/P		
3	0	=	$3 \times 3 = 9 = 4$
3	1	=	$3 \times 3 + 1 = 10 = 0$
3	2	=	$3 \times 3 + 2 = 11 = 1$

R	I/P		
4	0	=	$3 \times 4 = 12 = 2$
4	1	=	$3 \times 4 + 1 = 13 = 3$
4	2	=	$3 \times 4 + 2 = 14 = 4$

Q.4.(a) Construct a minimum state DFA equivalent to the NFA given below : **8M**

NFA = ({A, B, C, D}, {a, b}, δ, A, {B, D})

where δ is defined using Transition Table :

	a	b
A	{B, D}	{B}
B	{C}	{B, C}
C	{D}	{A}
D	ϕ	{A}

Ans. **Transition table for DFA :**

State	a	b
A	{B, D}	{B}
{B, D}	{C}	{A, B, C}
{B}	{C}	{B, C}
{C}	{D}	{A}
{A, B, C}	{B, C, D}	{A, B, C}
{B, C}	{C, D}	{A, B, C}
{D}	ϕ	{A}
{B, C, D}	{C, D}	{A, B, C}
{C, D}	{D}	{A}

Renaming the state : A → A, B → B, C → C, D → D

{B, D} → E, {A, B, C} → F, {B, C} → G, {B, C, D} → H

{C, D} → I

Transition table for DFA :

State	a	b
A	E	B
*E	C	F
*B	C	G
C	D	A
F	H	F
G	I	F
*D	ϕ	A
H	I	F
I	D	A

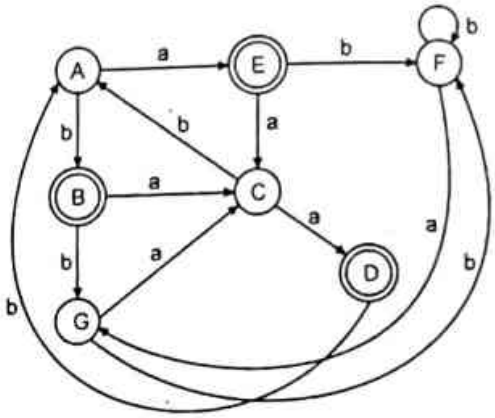
Remove I and replace I by C.

State	a	b
A	E	B
*E	C	F
*B	C	G
C	D	A
F	H	F
G	C	F
*D	ϕ	A
H	C	F

Remove H and replace H by G

State	a	b
A	E	B
*E	C	F
*B	C	G
C	D	A
F	G	F
G	C	F
*D	ϕ	A

Transition Diagram :



Q.4.(b) Differentiate between : **6M**

(i) NFA and DFA

Ans. See P.2-13, Q.12.

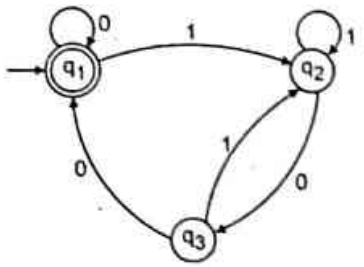
(ii) Moore and Mealy machine.

Ans. See P.2-49, Q.64.

UNIT - III

(Total Marks : 26)

Q.5.(a) Construct RE from Transition Diagram given below. **6M**



Ans. See P.3-46, Q.66.

Q.5.(b) Explain Pumping Lemma for Regular Languages. Also prove that

$L = \{a^n . b^{2n} \mid n \geq 1\}$ is non regular language. **7M**

Ans. See P.3-52, 56, Q.80, 89.

Q.6.(a) Describe the language defined by following regular expression.

(i) a(a | b)*b

(ii) c* (a | (bc*))* **7M**

Ans. **(i) a(a | b)*b :**

∴ (a/b)* = { ∈ , a, b, ab, ba, abb, aab ..... }

String start with a and ends with b.

So the language described by R. E. is

$S \rightarrow aABb$

$A \rightarrow aA \mid \epsilon$

$B \rightarrow bB \mid \epsilon$

$\therefore L = \{a^n b^m \mid n \geq 1, m \geq 1\}$

(ii) $c^*(a \mid (bc^*))^*$:

$c^* = \{\epsilon, c, cc, ccc, cccc \ldots\ldots\}$

$(b\,c)^* = \{\epsilon, bc, bcbc, bcbcbc \ldots\ldots\}$

$(a)(bc)^* = \{a, bc, bcbc \ldots\ldots\}$

$(a)(bc)^*)^* \{\epsilon, a, bc, abc, bcbc, abcbc \ldots\ldots\}$

$c^*(a \mid (bc^*))^* = \{\in, ca, cbc, ccbc \ldots\ldots\}$

$\therefore$ grammar described by R.E.

$S \rightarrow c\,AB \mid \epsilon$

$A \rightarrow aA \mid \epsilon$

$B \rightarrow bcB \mid \epsilon$

$\therefore$ The language described by R.E. is

$L = \{c^l a^m b^n c^n \mid l \geq 0, m \geq 1, n \geq 0\}$

Q.6.(b) Consider the Grammar : **6M**

$S \rightarrow 0B/1A$

$A \rightarrow 0/0S/1AA$

$B \rightarrow 1/1S/0BB$

For the string "00110101", find

(i) LMD (Leftmost Derivation)

(ii) RMD (Rightmost Derivation)

Ans. **See P.PS-69, Q.5(b).**

UNIT - IV

(Total Marks : 26)

Q.7.(a) State and explain closure properties of context free languages. **5M**

Ans. **See P.4-26, Q.40.**

Q.7.(b) Give formal definition of PDA with block diagram. Explain it. **4M**

Ans. **See P.4-4, 5, Q.1, 2.**

Q.7.(c) Consider the context free language $L = \{a^{2n} b^n / n \geq 0\}$, obtain its equivalent CFG **4M**

Ans. **See P.PS-71, Q.8(c).**

Q.8.(a) Construct PDA for the matching parenthesis "("(" ")")". **7M**

Ans. String has fixed length.

String start with (and end with).

Push '1' for input symbol '(' and pop 1 for input symbol' ')'.

Transitions :

$\delta(q_0, '(', z_0) \rightarrow (q_0, 1z_0)$

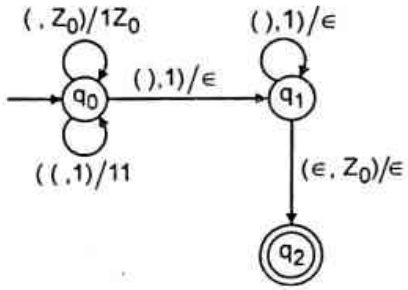
$\delta(q_0, '(', 1) \rightarrow (q_0, 11)$

$\delta(q_0, ')', 1) \rightarrow (q_1, \epsilon)$

$\delta(q_1, ')', 1) \rightarrow (q_1, \epsilon)$

$\delta(q_1, '\epsilon', z_0) \rightarrow (q_2, \epsilon)$

Transition diagram :



Q.8.(b) Construct PDA for $L = \{w \subset \omega^R / C$ is some special symbol not in w} **6M**

Ans. **See P.4-15, Q.22.**

UNIT - V

(Total Marks : 33)

Q.9.(a) Explain Linear Bounded Automata **6M**

Ans. **See P.5-34, Q.40.**

Q.9.(b) Design a Turing Machine for following language.

$L = \{a^n b^n c^n / n \geq 1\}$ **8M**

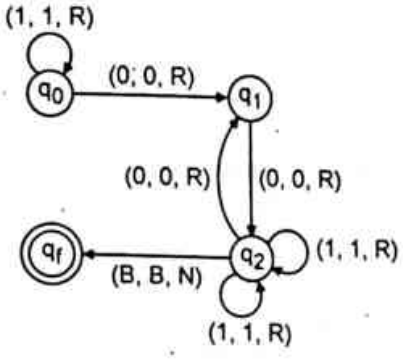
Ans. **See P.5-15, Q.10.**

Q.10.(a) Design a Turing Machine that can accept the string over (0, 1) to recognize all strings containing even number of 0's. **8M**

Ans. **Transition table :**

Present state	Tape Symbol		
	0	1	B
→ q_0	$q_1, 0, R$	$q_0, 1, R$	
q_1	$q_1, 0, R$	$q_1, 1, R$	
q_2	$q_1, 0, R$	$q_1, 1, R$	q_f, B, N
q_f	-	-	-

Transition Diagram :



Q.10.(b) Design T.M. for language to find 2's complement of a binary number. **6M**

Ans. **See P.5-37, Q.45.**

Q.11.(b) Explain Church's Hypothesis in brief. **5M**

Ans. **See P.5-30, Q.31.**

UNIT - VI

(Total Marks : 21)

Q.11.(a) Define Ackerman's function. Compute **8M**

(i) A (1, 1)

(ii) A (2, 1)

(iii) A (2, 3)

Ans. **See P.6-38, Q.68.**

P.PS-17, Q.12(a).

Q.12.(a) What do you mean by PCP? Decide whether the following (A, B) pair have a solution or not. If Yes, give a solution. If No, then why?

(i) $A(b, bab^2, ba), B = \{b^2, ba, aa\}$

(ii) A(01, 1, 11), B = {011, 10, 11} **9M**

Ans. **See P.6-15, 16, 17, Q.18, 19, 21(a).**

Q.12.(b) Define decidable and undecidable language. **4M**

Ans. **See P.6-13, 14, Q.15(a), 16(a).**

Notes :-

WINTER EXAMINATION - 2016
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	28 M
Unit - III	26 M
Unit - IV	26 M
Unit - V	28 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

WINTER EXAM. - 2016
QUESTION PAPER

Q.1.(a) Which type of grammar is more powerful and why? **5M**

(b) Describe the concept of pigeon-hole principal with example. **5M**

(c) Explain following terms :

(1) Star closure of language

(2) Positive closure of language

(3) String and length of string **3M**

OR

Q.2.(a) Explain following terms : **7M**

(1) Context sensitive grammar (2) Context free grammar

and also justify why context free grammar is context sensitive but vice versa is not possible?

(b) With the help of mathematical induction prove that : **6M**

(1) $1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n \times (n+1)(2n+1)}{6}$

(2) $1.2.3 + 2.3.4 + \ldots + n(n+1)(n+2)$

$= \frac{n(n+1)(n+2)(n+3)}{4}$

Q.3. Design a DFA for following :

(1) For a string over $\langle 0, 1 \rangle$ divisible by 3 binary. **3M**

(2) $L = \left\{ W \middle| \begin{array}{l} W = \{a\,b\}^* \\ n(a) \text{ of } W \pmod 4 = \\ n(b) \text{ of } W \pmod 4 \end{array} \right\}$

Where n (a) = no. of a's

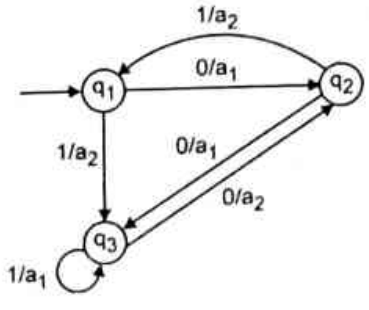
n (b) = no. of b's **5M**

(3) For a string which should contain 'ab' as substring $\Sigma = \{a, b\}^*$. **2M**

(4) For a string which should start with 1 and end with 01. Assume $\Sigma = \{0,1\}$. **4M**

OR

Q.4.(a) Convert the following Mealy machine into equivalent Moore machine : **7M**



(b) Minimize the following DFA : **7M**

	State	a	b
→	q_0	q_1	q_0
	q_1	q_6	q_2
	q_2	q_3	q_1
*	q_3	q_3	q_0
	q_4	q_3	q_5
	q_5	q_6	q_4
	q_6	q_5	q_6
	q_7	q_6	q_3

Q.5.(a) Obtain the regular expression from following grammar : **3M**

$S \rightarrow 1A \mid 0C \mid 0$

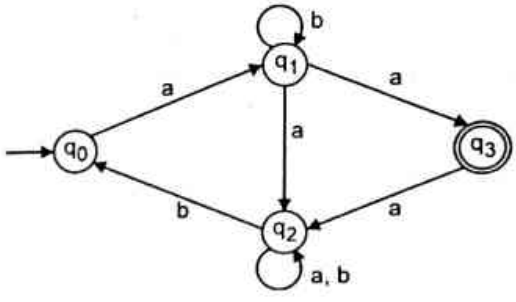
$A \rightarrow 1B$

$B \rightarrow 1B \mid 1D$

$C \rightarrow 0$

$D \rightarrow 0D \mid 01$

(b) Find the regular expression for the given transition diagram using Arden's Theorem : **7M**



(c) Match the following pairs : **3M**

Group A	Group B
Left linear grammar	$S \rightarrow aSSb \mid bSSa \mid \in$
Right linear grammar	$S \rightarrow Ba \mid aa \mid Saa$
Ambiguous grammar	$S \rightarrow abA \mid aA \mid b$

OR

Q.6.(a) Find the minimum state DFA for following regular expression

$ab^*(ab)^*(a + b)b^* + b$ **8M**

(b) Convert the grammar into Chomsky Normal Form. **5M**

$S \rightarrow ABa$

$A \rightarrow aab$

$B \rightarrow Ab$

Q.7.(a) Differentiate between NPDA and DPDA. **5M**

(b) Construct CFG from following PDA : **8M**

$\delta(q_0, 1, z_0) = (q_0, x\, z_0)$

$\delta(q_0, 1, x) = (q_0, xx)$

$\delta(q_0, 0, x) = (q_1, x)$

$\delta(q_0, \epsilon, z_0) = (q_0, \epsilon)$

$\delta(q_1, 1, x) = (q_1, \epsilon)$

$\delta(q_1, 0, z_0) = (q_0, z_0)$

OR

Q.8.(a) Explain the model of PDA. **2M**

(b) Construct PDA from the given grammar : **5M**

$E \rightarrow +EE \mid *EE \mid \$TF$

$T \rightarrow +T \mid +$

$F \rightarrow *F \mid +$

(c) Construct PDA for following : **6M**

$L = \{a^n b^m c^m d^n \mid n, m \geq 1\}$

Q.9.(a) Define the model of Turing machine and explain its tuples. **5M**

(b) Construct Turing machine for

Input = W

Output = WW

Where $W = \{a, b\}^*$. **9M**

OR

Q.10.(a) Explain the types of Turing machine. **6M**

(b) Design Turing machine to convert 111 into 101. **8M**

Q.11. Explain the following terms :

(1) Decidability and solvability. **3M**

(2) Recursive function. **2M**

(3) Post correspondence problem. **4M**

(4) Halting problem. **4M**

OR

Q.12.(a) Explain the properties of recursively enumerable language. **7M**

(b) Define Ackermann's function and compute

A(1, 1), A(2, 1), and A(2, 2). **6M**

WINTER EXAM. - 2016
SOLUTION
(Unit-Wise)

UNIT - I
(Total Marks : 26)

Q.1.(a) Which type of grammar is more powerful and why? **5M**

Ans. Type (0) grammar is more powerful.

A type (0) or unrestricated grammar is the one, whose production rule is of the form : $\alpha \rightarrow \beta$ where α and β are in $(VUT)^*$ i.e. they can be any string of terminals and non-terminals and $\alpha \neq \epsilon$.

Since there is no restriction on what can appear on the left hand side of production rule, as well as what can appear on the right of the production rule, it is capable of specifying the largest number of language. And the language specifiable using type (0) grammar are called recursively enumerable language. Therefore recursively enumerable language constitutes a largest class of specifiable language.

Q.1.(b) Describe the concept of pigeon-hole principal with example. **5M**

Ans. **See P.1-10, Q.16.**

Q.1.(c) Explain following terms :

(1) Star closure of language

(2) Positive closure of language

(3) String and length of string. **3M**

Ans. **See P.1-4, 11, Q.2, 20, P.PS-84, Q.2(b).**

Q.2.(a) Explain following terms : **7M**

(1) Context sensitive grammar

(2) Context free grammar

and also justify why context free grammar is context sensitive but vice versa is not possible?

Ans. **See P.1-13, Q.22(2), (3).**

Since there is a restriction in context free grammar that the left hand side of a production rule must be a single non-terminal. The number

of language that it can specify will be less than the number of language type (1) grammar. Except when $\alpha = \epsilon$, hence every language that can be specified using type (2) grammar not containing ϵ in always possible to specify using type (1) grammar, but vice-versa is not true.

Q.2.(b) With the help of mathematical induction prove that : **6M**

(1) $1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n \times (n+1)(2n+1)}{6}$

(2) $1.2.3 + 2.3.4 + \ldots + n(n+1)(n+2)$

$= \frac{n(n+1)(n+2)(n+3)}{4}$

Ans. See P.1-8, 9, Q.11, 15.

UNIT - II

(Total Marks : 28)

Q.3. Design a DFA for following :

(1) For a string over $(0, 1)$ divisible by 3 binary. **3M**

(2) $L = \left\{ W \middle| \begin{matrix} W = \{a,b\}^* \\ n(a) \text{ of } W \pmod 4 = \\ n(b) \text{ of } W \pmod 4 \end{matrix} \right\}$

Where n (a) = no. of a's

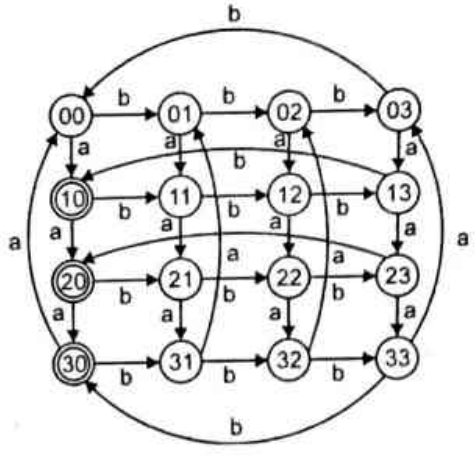
n (b) = no. of b's **5M**

(3) For a string which should contain 'ab' as substring $\Sigma = \{a, b\}^*$. **2M**

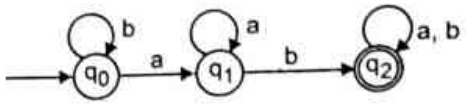
(4) For a string which should start with 1 and end with 01. Assume $\Sigma = \{0,1\}$. **4M**

Ans. See P.2-32, Q.47.

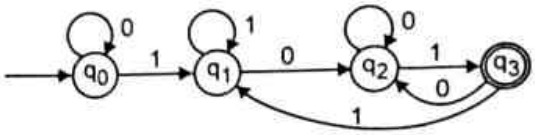
(2)



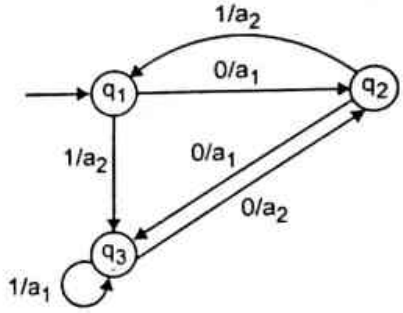
(3)



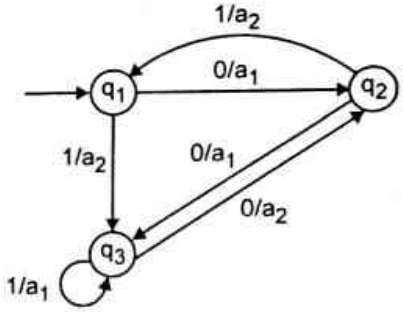
(4)



Q.4.(a) Convert the following Mealy machine into equivalent Moore machine : **7M**



Ans.



Transition table :

Present state	Next state			
	a = 0		a = 1	
	State	Output	State	Output
$\rightarrow q_1$	q_2	a_1	q_3	a_2
q_2	q_3	a_1	q_1	a_2
q_3	q_2	a_2	q_3	a_1

Output associated state :

State	Output
q_1	a_2
q_2	$\{a_1, a_2\}$
q_3	$\{a_1, a_2\}$

VBD

Split two state q_2 and q_3

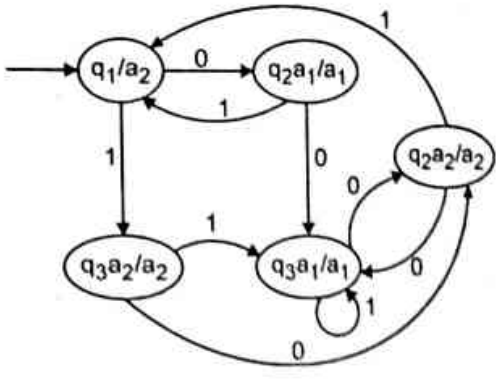
Transition table for Mealy machine :

Present state	Next state			
	a = 0		a = 1	
	State	Output	State	Output
$\rightarrow q_1$	q_2a_1	a_1	q_3a_2	a_2
q_2a_1	q_3a_1	a_1	q_1	a_2
q_2a_2	q_3a_1	a_1	q_1	a_2
q_3a_1	q_2a_2	a_2	q_3a_1	a_1
q_3a_2	q_2a_2	a_2	q_3a_1	a_1

Transition table for Moore machine :

Present state	Next state		Output
	a =0	a = 1	
$\rightarrow q_1$	q_2a_1	q_3a_2	a_2
q_2a_1	q_3a_1	q_1	a_1
q_2a_2	q_3a_1	q_1	a_2
q_3a_1	q_2a_2	q_3a_1	a_1
q_3a_2	q_2a_2	q_3a_1	a_2

Transition diagram :



Q.4.(b) Minimize the following DFA : **7M**

	State	a	b
$\rightarrow$	q_0	q_1	q_0
	q_1	q_6	q_2
	q_2	q_3	q_1
*	(q_3)	q_3	q_0
	q_4	q_3	q_5
	q_5	q_6	q_4
	q_6	q_5	q_6
	q_7	q_6	q_3

Ans. See P.2-41, Q.58.

UNIT - III

(Total Marks : 26)

Q.5.(a) Obtain the regular expression from following grammar : **3M**

$S \rightarrow 1A \mid oC \mid o$

$A \rightarrow 1B$

$B \rightarrow 1B \mid 1D$

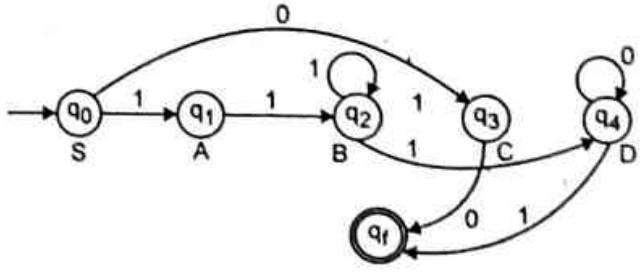
$C \rightarrow O$

$D \rightarrow oD \mid 01$

Ans. Let state q_0, q_1, q_2, q_3, q_4 corresponds to A, ϵ, S, A, B, C, D.

The finite automata represented by given grammar is

$\therefore$ 01 as 1



$q_0 = \epsilon$ ...(1)

$q_1 = q_0 1$ ...(2)

$q_2 = q_1 1 + q_2 1$ ...(3)

$q_3 = q_0 0$ ...(4)

$q_4 = q_2 1 + q_4 0$ ...(5)

$q_f = q_3 0 + q_4 1$ ...(6)

Put values of q_0 in equation (2)

$\therefore q_1 = \epsilon_1$ (by identity rule)

$\therefore q_1 = 1$ (7)

Put value of equation (7) in equation (3)

$q_2 = 11 + q_2 1$

Apply Arden theorem

$q_2 = 11(1)^*$ (8)

Put value of q_2 in equation (5)

$\therefore q_4 = 11(1)^* + q_4 0$

Apply Arden theorem

$q_4 = 11(1)^*(0)^*$ (9)

$q_3 = q_0 0$

$= \epsilon 0$

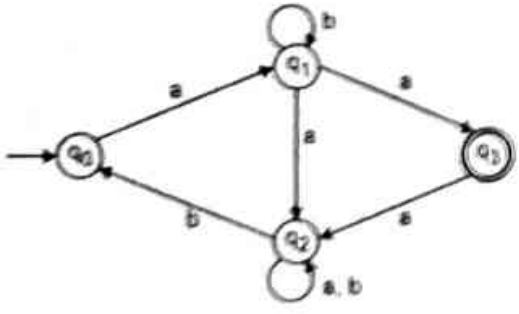
$\therefore q_3 = 0$ (10)

Put value of equation (9) and (10) to equation (6)

$\therefore q_F = q_3 0 + q_4(1)$

$q_F = 00 + 11(1)^*(0)^* 1$

Q.5.(b) Find the regular expression for the given transition diagram using Arden's Theorem : **7M**



Ans. Equation for all states

$q_0 = q_2 b + \epsilon$ (1)

$q_1 = q_0 a + q_1 b$ (2)

$q_2 = q_1 a + q_1 a + q_2 b + q_2 a$ (3)

$q_3 = q_1 a$ (4)

Put value of equation (4) in equation (3)

$q_2 = q_1 a + (a + b) q_2 + q_1 a a$

$q_2 = (a + aa) q_1 + (a + b) q_2$ (5)

Put value of equation (7) in equation (2)

$q_1 = q_1 b a + q_1 b + \epsilon$ (6)

Put value of equation (6) in equation (5)

$q_2 = (a + aa) q_2 ba + q_1 b + (a + b) q_2$

$q_2 = [(a + aa) ba + (a + b)] q_2 + q_1 b$ (7)

$q_1 = q_2 b a + q_1 b + \epsilon$

Apply Arden theorem, we get

$q_1 = (ba) q_2 (b)^* + \epsilon$

$q_1 = ba (b)^* q_2 + \epsilon$ (8)

Put these value in equation (7)

$q_2 = [(a + aa) ba + (a + b)] q_2 + ba (b)^* q_2 b + \epsilon$

$q_2 = [(a + aa) ba + (a + b) + (ba (b)^*) b] q_2 + \epsilon$

Apply Arden theorem, we get

$q_2 = \epsilon [(a + aa) ba + (a + b) + (ba(b)^* b]^*$

Put these value in state q_1, we get

$q_1 = ba (b)^* [(a + aa) ba + (a + b) + (ba (b)^* b)]^*$

$\therefore q_3 = q_1 a$

$= ba(b)^* [(a + aa) ba + (a + b) + (ba (b)^* b)]^* a$

Q.5.(c) Match the following pairs : **6M**

Group A	Group B
Left linear grammar	S → aSSb | bSba | ∈
Right linear grammar	S → Ba | aa | Saa
Ambiguous grammar	S → abA | aA | b

Ans. Left linear grammar → S → Ba | aa | Saa

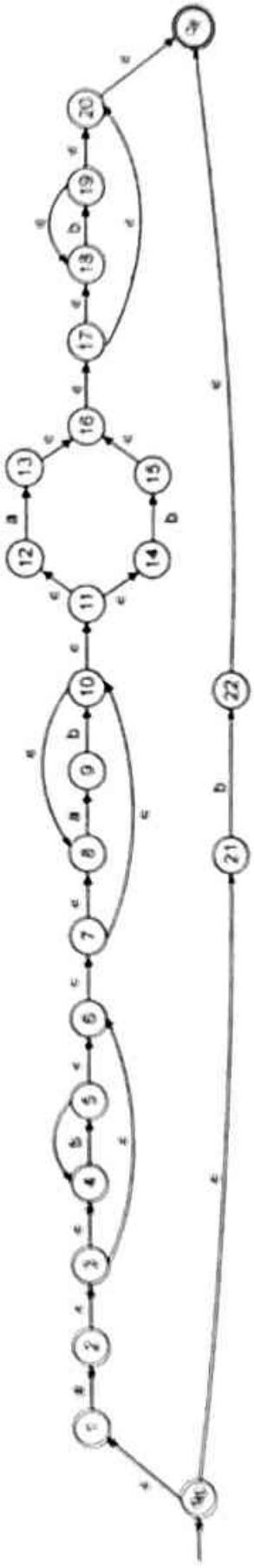
Right linear grammar → S → abA |aA | b

Ambiguous grammar → S → aSSb | bSba | ∈

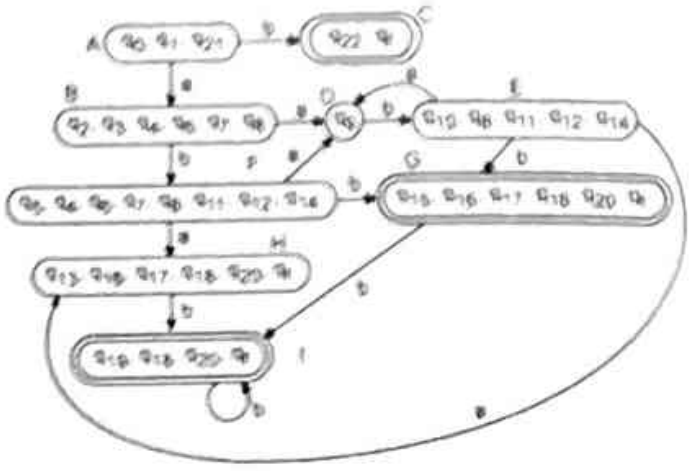
Q.6.(a) Find the minimum state DFA for following regular expression :

$ab^* (ab)^* (a + b) b^* + b$ **8M**

Ans. Transition graph for NFA :



Minimized DFA by direct method



Renaming the states

$q_0, q_1, q_{21} \rightarrow A$

$q_2, q_3, q_4, q_5, q_7, q_8 \rightarrow B$

$q_{22}, q_6 \rightarrow C$

$q_9 \rightarrow D$

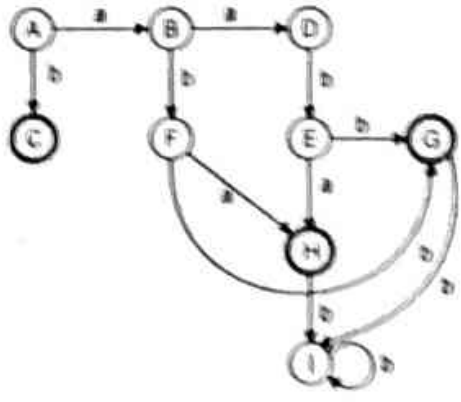
$q_{10}, q_{11}, q_{12}, q_{13}, q_{14} \rightarrow E$

$q_{15}, q_{16}, q_{17}, q_{18}, q_{19}, q_{20}, q_8 \rightarrow F$

$q_{23}, q_{24}, q_{25}, q_{26}, q_{27}, q_8 \rightarrow G$

$q_{28}, q_{29}, q_{30}, q_{31}, q_8 \rightarrow H$

$q_{32}, q_{33}, q_{34}, q_8 \rightarrow I$



	a	b
A	B	C
B	D	F
*C	-	-
D	-	E
E	H	G
F	H	G
G	H	G
H	H	G
I	H	G

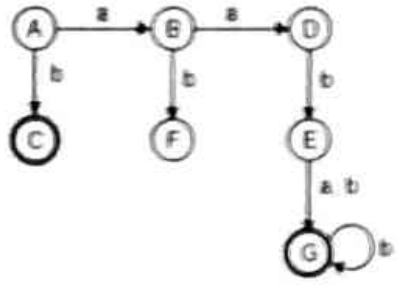
* G	-	1
* H	-	1
I	-	1

Remove H and I and replace G

	a	b
A	B	C
B	D	F
* C	-	-
D	-	E
E	G	G
F	C	G
* G	-	G

Remove F and replace by E

	a	b
A	B	C
B	D	E
* C	-	-
D	-	E
E	G	G
* G	-	G



Q.6.(b) Convert the grammar into Chomsky Normal Form. **5M**

S → ASa

A → aab

B → Ab

Ans. **See P.3-41, Q.16i.**

UNIT - IV

(Total Marks : 26)

Q.7.(a) Differentiate between NPDA and DPDA. **6M**

Ans. **See P.4-7, Q.7.**

Q.7.(b) Construct CFG from following PDA. **8M**

$\delta(q_0, 1, z_0) = (q_0, x z_0)$

$\delta(q_0, 1, x) = (q_0, xx)$

$\delta(q_0, 0, x) = (q_1, x)$

$\delta(q_0, \in, z_0) = (q_0, \in)$

$\delta(q_1, 1, x) = (q_1, \in)$

$\delta(q_0, 0, z_0) = (q_0, z_0)$

Ans. **See P.4-21, Q.32.**

Q.8.(a) Explain the model of PDA. **2M**

Ans. **See P.4-5, Q.2.**

Q.8.(b) Construct PDA from the given grammar. **8M**

E → + EE | *EE | $

T → + T | +

F → * F | +

Ans. Let the PDA equivalent to the given grammar is

M = ({q}, {+, *}, {E, T, F}, δ, q, I, ϕ)

where δ is defined by following rules

$\delta(q, +, E) = \{(q, EE)\}$ because E → +EE (1)

$\delta(q, *, E) = \{(q, EE)\}$ because E → * EE (2)

$\delta(q, \$, T) = \{(q, T E)\}$ because E → $TE (3)

$\delta(q, +, T) = \{(q, T)\}$ because T → +T (4)

$\delta(q, +, T) = \{(q, \in)\}$ because T → + (5)

$\delta(q, *, F) = \{(q, F)\}$ because F → * F (6)

$\delta(q, +, F) = \{(q, \in)\}$ because F → + (7)

Q.8.(c) Construct PDA for following. **6M**

$L = \{a^n b^m c^m d^n \mid n, m \geq 1\}$

Ans. **See P.4-47, Q.25.**

UNIT - V

(Total Marks : 28)

Q.9.(a) Define the model of Turing machine and explain its tuples. **8M**

Ans. **See P.5-7, Q.4.**

Q.9.(b) Construct Turing machine for

Input : W

Output : $W$$W$

Where $W = \{a, b\}^+$ **8M**

Ans. **See P.Pb-42, Q.10(e)**

VBD

Q.10.(a) Explain the types of Turing machine. **6M**

Ans. See P.S.21, Q.16

Q.10.(b) Design Turing machine to compute (11 into 0) **8M**

Ans. See P.S.17, Q.13

UNIT - VI

(Total Marks : 26)

Q.11 Explain the following terms

(1) Decidability and solvability **8M**

(2) Recursive function **2M**

(3) Post correspondence problem **4M**

(4) Halting problem **4M**

Ans. See P.S.-13, 21, 15, 19, Q.15(a), 29, 38, 25

See P.PS-89, Q.1(b)(iii)

Q.12.(a) Explain the properties of recursively enumerable languages **7M**

Ans. See P.S-7, 8, 9, Q.2, 6, 8, 9.

Q.12.(b) Define Ackermann's function and compute

A(1, 1), A(2, 1) and A(2, 2) **6M**

Ans. See P.S-38, Q.6B.

NOTES

SUMMER EXAMINATION - 2017
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	28 M
Unit - III	26 M
Unit - IV	28 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

SUMMER EXAM. - 2017

QUESTION PAPER

Q.1(a) With the help of mathematical induction prove that **8M**

(1) $1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n(n+1)(2n+1)}{6}$

(2) $1.1! + 2.2! + 3.3! + \ldots + n.n! = (n+1)! - 1$

(b) Explain pigeonhole principle with example. Also explain generalized pigeonhole principle **8M**

OR

Q.2.(a) Explain in detail Chomsky hierarchies of languages. **6M**

(b) What is countability and diagonalization? **4M**

(c) Let R = {(1, 2), (2, 3), (3, 1)} and A = {1, 2, 3}

Fine reflexive, symmetric and transitive closure of R. **3M**

Q.3.(a) Design DFA over $\Sigma = \{0, 1\}$ to accept strings containing even number of 0's and 1's. **6M**

(b) Consider ϵ - NFA **8M**

Q \ Σ	ε	a	b	c
→ p	–	p	q	r
q	p	q	r	q
*r	q	r	q	p

(i) Compute ϵ - closure of each state.

(ii) Compute all the strings of length 3 or less accepted by the automata.

(iii) Convert to DFA.

OR

Q.4.(a) Construct Mealy machine to compute 2's complement of binary number. Also convert this machine to Moore machine. **6M**

(b) Construct minimum state automata equivalent to given automata. **8M**

State	0	1
→ q_0	q_1	q_5
q_1	q_6	q_2
(*q_2)	q_6	q_2
q_3	q_1	q_6
q_4	q_6	q_3
q_5	q_6	q_4
q_6	q_6	q_6

Q.5. Construct T.A. to accept strings over $\Sigma = \{a, b\}$ containing atleast one aba and not containing bbb. **13M**

OR

Q.6.(a) Check whether given grammar is ambiguous or not **4M**

S → ε / Sa / bSS / SbS

(b) Reduce the grammar and find equivalent grammar. **5M**

S → aA / aBB

A → aaA / ε

B → bB / bbC

C → B

(c) Convert the following CFG into CNF. **4M**

S → bA / aB

A → bAA / aS / a

B → aBB / bS / b

Q.7.(a) Construct CFG from following PDA **10M**

$\delta(q_0, 1, Z_0) \rightarrow (q_0, XZ_0)$

$\delta(q_0, 1, X) \rightarrow (q_0, XX)$

$\delta(q_0, 0, X) \rightarrow (q_1, X)$

$\delta(q_0, \epsilon, Z_0) \rightarrow (q_0, \epsilon)$

$\delta(q_1, 1, X) \rightarrow (q_1, \epsilon)$

$\delta(q_1, 0, Z_0) \rightarrow (q_0, Z_0)$

(b) Explain the model of PDA. **4M**

OR

Q.8.(a) Convert the CFG into PDA **6M**

S → aAb / ε

A → bA / a

b → 1 and / 0

(b) Design PDA for 6M

$h = \{WW^R / W \in (a, b)^*\}$

Q.9.(a) Explain in detail, types of turing machines 6M

(b) Design turing machine for 7M

$h = \{a^n b^n c^n / n \geq 1\}$

OR

Q.10.(a) Design a turing machine for multiplication of unary numbers 9M

(b) Give the model of LBA and define it formally 4M

Q.11.(a) What is Post Correspondence Problem? Explain with example. Also explain modified PCP 6M

(b) Define Ackermann's function 7M

Compute A (1, 1), A(2, 1), A(2, 2)

OR

Q.12.(a) Explain properties of recursively enumerable languages 6M

(b) Write a note on recursive function 3M

(c) Explain halting problem 3M

(d) Define decidables and undecidables 4M

SUMMER EXAM. - 2017

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) With the help of mathematical induction, prove that 8M

(1) $1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n(n+1)(2n+1)}{6}$

(2) $1 \cdot 2 + 2 \cdot 3 + 3 \cdot 4 + \ldots + n(n+1) = \frac{1}{3}n(n+1)(n+2)$

Ans. See P.1.8, Q.11. 1)

Q.1.(b) Explain pigeonhole principle with example. Also explain generalized pigeonhole principle 8M

Ans. See P.1.10, Q.16, 1)

Q.2.(a) Explain in detail various formally languages 6M

Ans. See P.1.11, Q.5)

Q.2.(b) What is countability and diagonalization? 4M

Ans. See P.1.6, Q.5

Q.2.(c) Let R = {(1, 2), (2, 3), (3, 1)} and A = {1, 2, 3}

Find reflexive, symmetric and transitive closure of R 3M

Ans. R = {(1, 2), (2, 3), (3, 1)}, A = {1, 2, 3}

(1) Reflexive closure :

$R^1 = R \cup \{(1, 1), (2, 2), (3, 3)\}$

$= \{(1, 2), (2, 3), (3, 1), (1, 1), (2, 2), (3, 3)\}$

(2) Symmetric closure :

$S = R \cup \{(2, 1), (3, 2), (1, 3)\}$

$= \{(1, 2), (2, 1), (2, 3), (3, 2), (1, 3), (3, 1)\}$

(3) Transitive closure of R :

$R^+ = \{(1, 2), (2, 3), (3, 1)\}$

UNIT - II

(Total Marks : 28)

Q.3.(a) Design DFA over $\Sigma = \{0, 1\}$ to accept strings containing even number of 0's and 1's 6M

Ans. Select all combination of even and odd no. of 0's and 1's as states

i.e. q_0 : Even number of 0's and even number of 1's

q_1 : Even number of 0's and odd number of 1's

q_2 : Odd number of 0's and even number of 1's

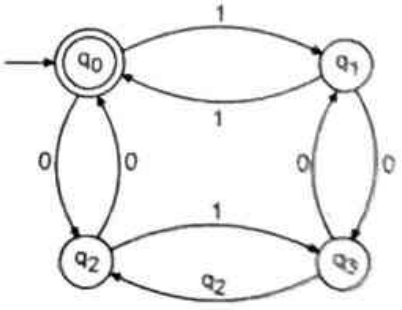
q_3 : Odd number of 0's and odd number of 1's

q_0 is even no. of 0's and even no. of 1's make it as final state

Transition table :

Q / Σ	0	1
→ (q0)	q2	q1
q1	q3	q0
q2	q0	q3
q3	q1	q2

Transition diagram :



Q.3.(b) Consider ϵ - NFA. **8M**

Q \ Σ	ϵ	a	b	c
→ p	–	p	q	r
q	p	q	r	ϕ
*r	q	r	ϕ	ϕ

(i) Compute ϵ -closure of each state.

(ii) Compute all the strings of length 3 or less accepted by the automata.

(iii) Convert to DFA.

Ans.

(i) ϵ - closure (p) = {p}

ϵ - closure (q) = {p, q}

ϵ - closure (r) = {q, r}

(ii) $\delta(p, a) = \epsilon$ - closure $(\delta(\hat{\delta}(p, \epsilon)), a)$

$= \epsilon$ - closure $(\delta(p, a))$

$= \epsilon$ - closure (p)

$= \{p\}$

$\delta(p, b) = \epsilon$ - closure $(\delta(\hat{\delta}(p, \epsilon)), b)$

$= \epsilon$ - closure $(\delta(p, b))$

$= \epsilon$ - closure (q)

$= \{p, q\}$

$\delta(p, c) = \epsilon$ - closure $(\delta(\hat{\delta}(p, \epsilon)), c)$

$= \epsilon$ - closure $(\delta(p, c))$

$= \epsilon$ - closure (r)

$= \{q, r\}$

$\delta(q, a) = \epsilon$ - closure $(\delta(\hat{\delta}(q, \epsilon)), a)$

$= \epsilon$ - closure $(\delta(p, q), a)$

$= \epsilon$ - closure (p, q)

$= \{p, q\}$

$\delta(q, b) = \epsilon$ - closure $(\delta(\hat{\delta}(q, \epsilon)), b)$

$= \epsilon$ - closure $(\delta(p, q), b)$

$= \epsilon$ - closure (q, r)

$= \{p, q, r\}$

$\delta(q, c) = \epsilon$ - closure $(\delta(\hat{\delta}(q, \epsilon)), c)$

$= \epsilon$ - closure $(\delta(p, q), c)$

$= \epsilon$ - closure (r)

$= \{q, r\}$

$\delta(r, a) = \epsilon$ - closure $(\delta(\hat{\delta}(r, \epsilon)), a)$

$= \epsilon$ - closure $(\delta(p, r), a)$

$= \epsilon$ - closure (p, r)

$= \{p, q, r\}$

$\delta(r, b) = \epsilon$ - closure $(\delta(\hat{\delta}(r, \epsilon)), b)$

$= \epsilon$ - closure $(\delta(q, r), b)$

$= \epsilon$ - closure (r)

$= \{q, r\}$

$\delta(r, c) = \epsilon$ - closure $(\delta(\hat{\delta}(r, \epsilon)), c)$

$= \epsilon$ - closure $(\delta(q, r), c)$

$= \epsilon$ - closure (p)

$= \{p\}$

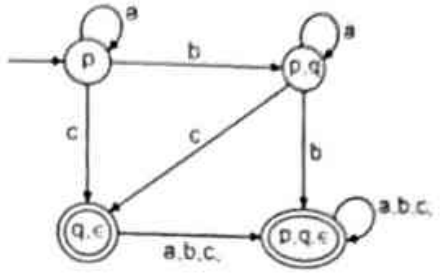
Transition table :

Q \ Σ	a	b	c
* → p	p	p, q	q, r
*q	p, q	p, q, r	q, r
*r	p, q, r	q, r	p

(iii) Transition table for DFA :

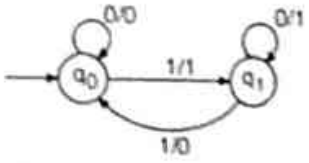
Σ / Q	a	b	c
→ p	p	{p, q}	{q, r}
{p, q}	{p, q}	{p, q, r}	{q, r}
* {q, r}	{p, q, r}	{p, q, r}	{p, q, r}
* {p, q, r}	{p, q, r}	{p, q, r}	{p, q, r}

Transition diagram for DFA :



Q.4.(a) Construct Mealy machine to compute 2's complement of binary number. Also convert this machine to Moore machine. **6M**

Ans. Transition diagram for Mealy machine :



Transition table for Mealy machine :

Present state	a = 0		a = 1	
	State	Output	State	Output
→ q_0	q_0	0	q_1	1
q_1	q_1	1	q_1	0

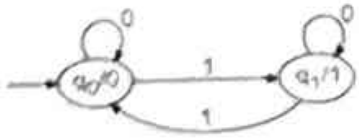
Output :

State	o/p
q_0	0
q_1	1

Transition table for Moore machine :

Present state	a = 0	a = 1	Output
→ q_0	q_0	q_1	0
q_1	q_1	q_0	1

Transition diagram for Moore machine :



Q.4.(b) Construct minimum state automata equivalent to given automata **8M**

State	0	1
→ q_0	q_1	q_5
q_1	q_6	q_2
q_2	q_0	q_2
q_3	q_2	q_6
q_4	q_7	q_5
q_5	q_2	q_6
q_6	q_6	q_4
q_7	q_6	q_2

Ans. See P.2-44, Q.6(f).

UNIT - III

(Total Marks : 26)

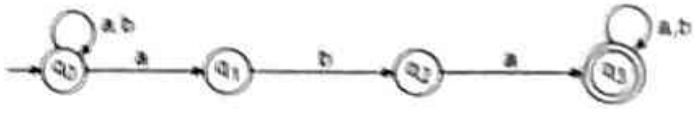
Q.5. Construct F.A. to accept strings over $\Sigma = \{a, b\}$ containing at least one aba and not containing bbb. **13M**

Ans.

Part I : Finite condition :

Containing aba as substring

Regular expression : $(a + b)^* aba (a + b)^*$



Transition table for NFA :

Σ / Q	a	b
→ q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1		$\{q_2\}$
q_2	$\{q_3\}$	
* q_3	$\{q_3\}$	$\{q_3\}$

Transition table for DFA :

Σ / Q	a	b
→ q_0	$\{q_0, q_1\}$	$\{q_2\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_2, q_3\}$
$\{q_0, q_2\}$	$\{q_0, q_1, q_3\}$	$\{q_2\}$
* $\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_2, q_3\}$
* $\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_2, q_3\}$
* $\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_2, q_3\}$

Renaming all states as,

$\{q_0\}$ = A, $\{q_0, q_1\}$ = B, $\{q_2, q_3\}$ = C, $\{q_0, q_1, q_3\}$ = D

$\{q_0, q_2, q_3\}$ = E, $\{q_0, q_3\}$ = F

Transition table for DFA :

Σ / Q	a	b
→ A	B	A
B	B	C
C	D	A
*D	D	E
*E	D	F
*F	D	F

State E and F both are final and equivalent. Hence replace E by F

Σ / Q	a	b
→ A	B	A
B	B	C
C	D	A
*D	D	F
*F	D	F

Also state D and F are equivalent, replace F by D

Transition table for DFA :

Σ / Q	a	b
→ A	B	A
B	B	C
C	D	A
*D	D	D

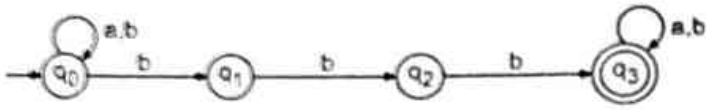
Part 2 : Infinite condition :

Do not contain bbb as substring

Complement of condition :

Containing bbb as substring

∴ Regular expression = $(a + b)^*$ bbb $(a + b)^*$



Transition table for NFA :

Σ / Q	a	b
→ q_0	q_0	q_0, q_1
q_1	-	q_2
q_2	-	q_3
* q_3	q_3	q_3

Transition table for DFA :

Σ / Q	a	b
q_0	q_0	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$
* $\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
* $\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2\}$
* $\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$

Renaming all states as

$[q_0] = A'$, $[q_0, q_1] = B'$, $[q_0, q_1, q_2] = C'$

$[q_0, q_1, q_2, q_3] = D'$,

$[q_0, q_3] = E'$, $[q_0, q_3, q_4] = F'$

Transition table :

Σ / Q	a	b
→ A'	A'	B'
B'	A'	C'
C'	A'	D'
* D'	E'	D'
* E'	E'	F'
* F'	E'	D'

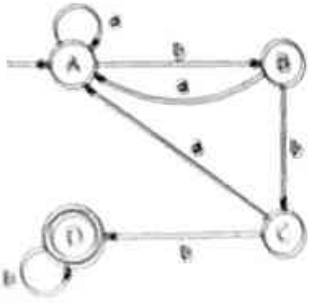
D and F are equivalent, so remove F and replace F by D

Σ / Q	a	b
→ A'	A'	B'
B'	A'	C'
C'	A'	D'
* D'	E'	D'
* E'	E'	D'

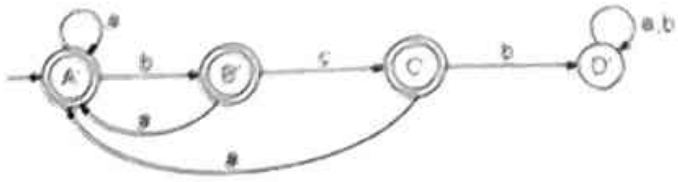
Again D and E are equivalent, remove E and replace E by D

Σ / Q	a	b
→ A	A	B
B	A	C
C	A	D
*D	D	D

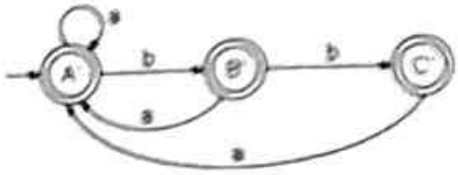
Transition graph for DFA :



Now, take complement condition



D is dead state



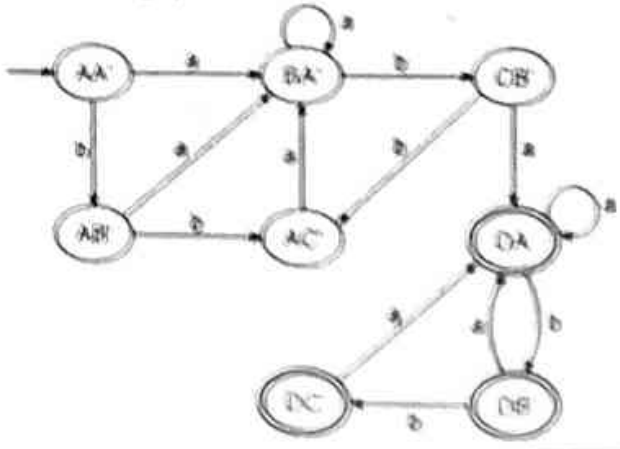
Transition table for DFA :

Σ / Q	a	b
→ A'	A'	B'
B'	A'	C'
* C'	A'	-

Transition table for final DFA :

Σ / Q	a	b
→ AA'	BA'	AB'
BA'	BA'	CB'
AB'	BA'	AC'
CB'	DA'	AC'
AC'	BA'	A
* DA'	DA'	DB'
* DB'	DA'	DC'
* DC'	DA'	D

Transition graph :

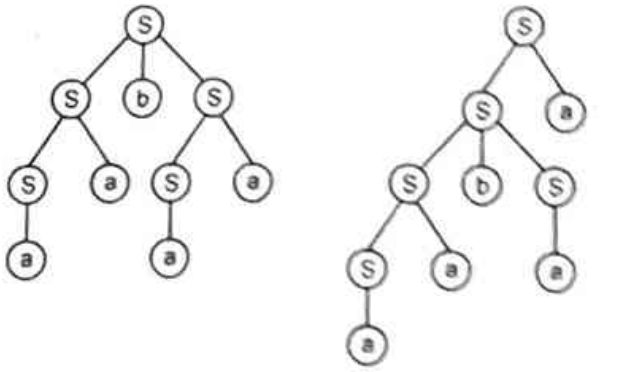


Q.6.(a) Check whether given grammar is ambiguous or not. **4M**

$S \rightarrow a / Sa / bSS / SbS$

Ans. Consider the input string aabaa

Parse tree :



Since we derive string aabaa by two ways. There exists two parse tree for string aabaa.

Therefore, given grammar is ambiguous.

Q.6.(b) Reduce the grammar and find equivalent grammar **5M**

$S \rightarrow aA / aBB$

$A \rightarrow aaA / \epsilon$

$B \rightarrow bB / bbC$

$C \rightarrow B$

Ans. **See P.3-80, Q.156.**

Q.6.(c) Convert the following CFG into CNF **4M**

$S \rightarrow bA / aB$

$A \rightarrow bAA / aS / a$

$B \rightarrow aBB / bS / b$

Ans. **See P.3-83, Q.166.**

UNIT - IV

(Total Marks : 28)

Q.7.(a) Construct CFG from following PDA **10M**

$\delta(q_0, 1, Z_0) \rightarrow (q_0, XZ_0)$

$\delta(q_0, 1, X) \rightarrow (q_0, XX)$

$\delta(q_0, 0, X) \rightarrow (q_1, X)$

$\delta(q_0, \epsilon, Z_0) \rightarrow (q_0, \epsilon)$

$\delta(q_1, 1, X) \rightarrow (q_1, \epsilon)$

$\delta(q_1, 0, Z_0) \rightarrow (q_0, Z_0)$

Ans. **See P.4-21, Q.32.**

Q.7.(b) Explain the model of PDA **4M**

Ans. **See P.4-5, Q.2.**

Q.8.(a) Convert the CFG into PDA. **6M**

$E \rightarrow aAB / d$

$A \rightarrow BA / a$

$B \rightarrow Ead / C$

Ans. **Formal definition :**

$M = (Q, \Sigma, T, \delta, q_0, Z_0, F)$

$M = (\{q\}, \{a, d, c\}, \{Z_0, E, A, B\}, \delta, q, Z_0, q)$

Transitions :

(1) $E \rightarrow aAB$

$\delta(q, a, E) \rightarrow (q, AB)$

(2) $E \rightarrow d$

$\delta(q, d, E) \rightarrow (q, E)$

Convert the grammar in GNF-first

$B \rightarrow Ead/C$

$\rightarrow (aAB/d)\,ad/C$

$B \rightarrow aABAE/aAE/C$ *[∵ A → a, E → d]*

$A \rightarrow BA/a$

$\rightarrow (aABAE/aAE)\,A/a$

$A \rightarrow aABAEA/aAEA/a$

(3) $A \rightarrow aABAEA$

$\delta(q, a, A) \rightarrow (q, ABAEA)$

(4) $A \rightarrow aAEA$

$\delta(q, a, A) \rightarrow (q, AEA)$

(5) $A \rightarrow a$

$\delta(q, a, A) \rightarrow (q, \epsilon)$

(6) $B \rightarrow aABAE$

$\delta(q, a, B) \rightarrow (q, ABAE)$

(7) $B \rightarrow aAI$

$\delta(q, a, B) \rightarrow (q, AI)$

$B \rightarrow C$

$\delta(q, C, B) \rightarrow (q, I)$

Q.8.(b) Design PDA for **8M**

$h = \{WW^R / W \in (a, b)^*\}$

Ans. **See P.4-15, Q.23.**

UNIT - V

(Total Marks : 26)

Q.9.(a) Explain in detail, types of turing machines **6M**

Ans. **See P.5-21, Q.19.**

Q.9.(b) Design turing machine for **7M**

$h = \{a^n b^n c^n / n \geq 1\}$

Ans. **See P.5-15, Q.10.**

Q.10.(a) Design a turing machine for multiplication of unary numbers **9M**

Ans. **See P.PS-54, Q.10(b).**

Q.10.(b) Give the model of LBA and define it formally **4M**

Ans. **See P.5-34, Q.40.**

UNIT - VI

(Total Marks : 26)

Q.11.(a) What is post correspondence problem? Explain with example. Also explain modified PCP **6M**

Ans. **See P.6-15, 18, Q.18, 23.**

Q.11.(b) Define Ackermann's function **7M**

Compute A (1, 1), A(2, 1), A(2, 2)

Ans. **See P.6-38, Q.68.**

Q.12.(a) Explain properties of recursively enumerable languages **5M**

Ans. **See P.6-7, Q.2.**

Q.12.(b) Write a note on recursive function **3M**

Ans. **See P.6-21, Q.29.**

Q.12.(c) Explain halting problem **3M**

Ans. **See P.6-19, Q.25.**

Q.12.(d) Define decidability and undecidability **2M**

Ans. **See P.6-13, 14, Q.15(a), 16(a).**

NOTES

WINTER EXAMINATION - 2016
B.E. IV SEM. (COMPUTER TECHNOLOGY) (CBS)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	28 M
Unit - III	28 M
Unit - IV	28 M
Unit - V	28 M
Unit - VI	28 M
Questions covered in VTU	168 M (100 %)

WINTER EXAM. - 2016
QUESTION PAPER

Q.1.(a) Define and explain the following terms with suitable examples **8M**

(i) Transitive closure.

(ii) Reflexive transitive closure

(iii) Equivalence relation.

(iv) Countability and diagonalization

(b) What is formal grammar? Explain different types of formal grammars by giving rules/format of production for each formal grammar. **5M**

OR

Q.2.(a) Prove the following by using principle of mathematical induction **6M**

(i) $1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n(n+1)(2n+1)}{6}$

(ii) $1 + 2 + \ldots n = \frac{n^2 + n}{2}$

(b) What is string, prefix, suffix, proper prefix and proper suffix with examples? **4M**

(c) Short note on pigeon hole principle. **3M**

Q.3.(a) Design DFA to check whether ternary number is divisible by three **5M**

(b) What are the applications of Finite Automata? Explain in detail **5M**

(c) Differentiate NFA and DFA. **3M**

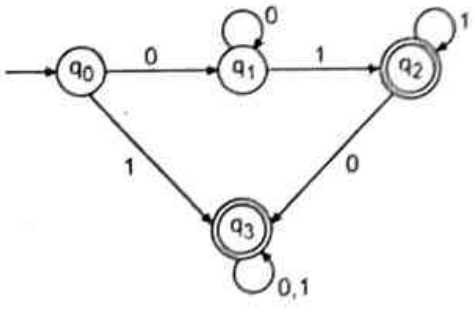
OR

Q.4.(a) Design DFA which accepts the set of strings that either start with 01 or end with 01 over $\Sigma\{0, 1\}$ **5M**

(b) Design Moore M/C for a binary I/P sequence such that if it has a substring 101 the M/C output A, if it has a substring 110, the machine O/P B, otherwise it O/P C. **6M**

(c) Give analytical definition of Moore and Mealy machine. **2M**

Q.5.(a) Find regular expression for the following transition diagram **8M**



(b) Obtain an equivalent left linear grammar for the following right linear grammar : **6M**

$S \rightarrow bA$

$A \rightarrow aA \mid bB \mid b$

$B \rightarrow bA$

Also mention the steps

OR

Q.6.(a) Find equivalent Greibach normal form representation for the following CFG. **8M**

$S \rightarrow AA \mid 0$

$A \rightarrow SS \mid 1$

(b) Construct NFA with and without ϵ -transition equivalent to the following regular expression : **6M**

$R_1 = 10 + (0 + 11)\,0^*1$

Q.7.(a) Construct PDA for the following language **7M**

$L = \{a^{2n}b^n \mid n > 0\}$

(b) Construct PDA equivalent to the following grammar : **7M**

$S \rightarrow aAA$

$A \rightarrow aS \mid bS \mid a$

and show acceptance rejection for the string 'aaabaaaaa'.

OR

Q.8.(a) Prove that $L = \{a^j b^j C^j \mid j \geq 1\}$ is not a CFL. **8M**

(b) Explain, with suitable example, closure and decision properties of CFL. **6M**

Q.9.(a) Design a T.M. to find 2's complement of a given binary number **7M**

(b) Explain briefly about universal turing machine. **6M**

OR

Q.10.(a) Design T.M. to recognize all palindrome strings over {a, b} **7M**

(b) What are different types of turing machine? Explain each type in brief **6M**

Q.11.(a) Define Ackerman's function. What is the significance of it? Compute A (1, 1), A (2, 1), A (1, 2) and (2, 2) **7M**

(b) Explain the properties of recursive enumerable language **6M**

OR

Q.12.(a) Show that the function ABS (x, y) = | x − y | is primitive recursive. **7M**

(b) What is PCP and modified PCP? Give one solution for the PCP for following sequences shown in table **6M**

i	A	B
1	0	000
2	010001	01
3	01	1

WINTER EXAM. - 2016
SOLUTION
(Unit-Wise)

UNIT - I
(Total Marks : 26)

Q.1.(a) Define and explain the following terms with suitable examples **8M**

(i) Transitive closure

(ii) Reflexive transitive closure

(iii) Equivalence relation

(iv) Countability and diagonalization

Ans. See P.1-5, Q.4, 5.

Q.1.(b) What is formal grammar? Explain different types of formal grammars by giving rules/format of production for each formal grammar **5M**

Ans. See P.1-11, Q.21, 22.

Q.2.(a) Prove the following by using principle of mathematical induction **8M**

(i) $1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n(n+1)(2n+1)}{6}$

(ii) $1 + 2 + \ldots n = \frac{n^2 + n}{2}$

Ans. See P.1-8, 7, Q.11, 10.

Q.2.(b) What is string, prefix, suffix, proper prefix and proper suffix with examples? **4M**

Ans. See P.1-4, Q.2.

Q.2.(c) Short note on pigeon hole principle **3M**

Ans. See P.1-10, Q.16.

UNIT - II
(Total Marks : 26)

Q.3.(a) Design DFA to check whether ternary number is divisible by three. **5M**

Ans. Consider states as $\{q_0, q_1, q_2\}$ and remainder are output for the machine

In this case formula for solving given equation is 3 × odd remainder + next bit

For initial state q_0 :

(i) 3 × 0 + 0 = 0, output is q_0

3 × 0 + 1 = 1, output is q_1

3 × 0 + 2 = 2, output is q_2

(ii) For state q_1 :

3 × 1 + 0 = 3, divisible by 3, output is q_0

3 × 1 + 1 = 4, remainder is 1, output is q_1

3 × 1 + 2 = 5, remainder is 2, output is q_2

(iii) For last state q_2 :

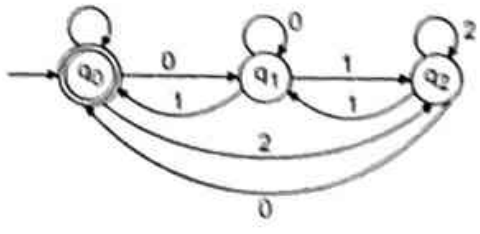
3 × 2 + 0 = 6, divisible by 3, output is q_0

3 × 2 + 1 = 7, remainder is 1, output is q_1

3 × 2 + 2 = 8, remainder is 2, output is q_2

Transition table :

	0	1	2
→ q_0	q_0	q_1	q_2
q_1	q_0	q_1	q_2
q_2	q_0	q_1	q_2



Q.3.(b) What are the applications of Finite Automata? Explain in detail **5M**

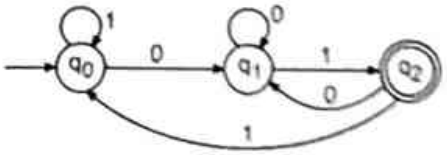
Ans. See P.2-10, Q.4.

Q.3.(c) Differentiate NFA and DFA. **3M**

Ans. See P.2-13, Q.12.

Q.4.(a) Design DFA which accepts the set of strings that either start with 01 or end with 01 over $\Sigma\{0, 1\}$. **5M**

Ans.



	0	1
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_0

Q.4.(b) Design Moore M/C for a binary I/P sequence such that if it has a substring 101 the M/C output A, if it has a substring 110, the machine O/P B, otherwise it O/P C. **6M**

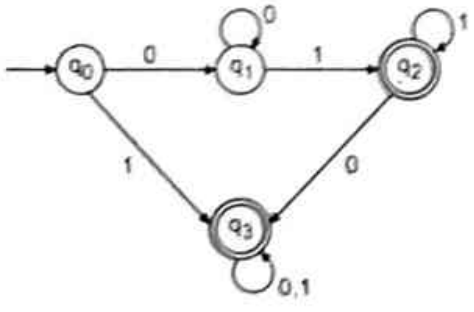
Ans. See P.2-55, Q.77.

Q.4.(c) Give analytical definition of Moore and Mealy machine. **2M**

Ans. See P.2-48, Q.63.

UNIT - III
(Total Marks : 28)

Q.5.(a) Find regular expression for the following transition diagram. **8M**



Ans. See P.3-51, Q.78.

Q.5.(b) Obtain an equivalent left linear grammar for the following right linear grammar. **6M**

$S \rightarrow bA$

$A \rightarrow aA \mid bB \mid b$

$B \rightarrow bA$

Also mention the steps.

Ans. See P.3-41, Q.53.

Q.6.(a) Find equivalent Greibach normal form representation for the following CFG. **8M**

$S \rightarrow AA \mid 0$

$A \rightarrow SS \mid 1$

Ans. See P.3-89, Q.178.

Q.6.(b) Construct NFA with and without ϵ -transition equivalent to the following regular expression. **6M**

$R_1 = 10 + (0 + 11)\,0^*1$

Ans. See P.3-18, Q.22.

UNIT - IV
(Total Marks : 28)

Q.7.(a) Construct PDA for the following language. **7M**

$L = \{a^{2n}b^n \mid n > 0\}$

Ans. See P.4-17, Q.26.

Q.7.(b) Construct PDA equivalent to the following grammar : **7M**

S → aAA

A → aS | bS | a

and show acceptance rejection for the string 'aaabaaaa'.

Ans. **Formal Definition :**

$M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$

$M = \{\{q\}, \{0, 1\}, \{Z_0, S, B\}, \delta, q, Z_0, q\}$

(1) $S \rightarrow aAA$

$\delta(q, a, S) \rightarrow (q, AA)$

(2) $A \rightarrow aS$

$\delta(q, a, A) \rightarrow (q, S)$

(3) $A \rightarrow bS$

$\delta(q, b, A) \rightarrow (q, S)$

(4) $A \rightarrow a$

$\delta(q, a, A) \rightarrow (q, \epsilon)$

(5) $\delta(q, aaabaaaa, S)$

$\vdash (q, aabaaaa, AA)$

$\vdash (q, abaaaa, SA)$

$\vdash (q, baaaa, AAA)$

$\vdash$ (q, aaaaa, SAA)

$\vdash$ (q, aaaa, AAAA)

$\vdash$ (q, aaa , AAA)

$\vdash$ (q, aa , AA)

$\vdash$ (q, a, A)

$\vdash$ (q, $\in$)

Q.8.(a) Prove that $L = \{a^i b^j C^{i+j} \mid j \geq 1\}$ is not a CFL. **8M**

Ans. See P.4-30, Q.42.

Q.8.(b) Explain, with suitable example, closure and decision properties of CFL. **6M**

Ans. See P.4-26, 27, Q.40, 41.

UNIT - V

(Total Marks : 26)

Q.9.(a) Design a T.M. to find 2's complement of a given binary number **7M**

Ans. See P.5-37, Q.45.

Q.9.(b) Explain briefly about universal turing machine. **6M**

Ans. See P.5-27, Q.26(A).

Q.10.(a) Design T.M. to recognize all palindrome strings over {a, b} **7M**

Ans. See P.5-18, Q.15.

Q.10.(b) What are different types of turing machine? Explain each type in brief. **6M**

Ans. See P.5-21, Q.19.

UNIT - VI

(Total Marks : 26)

Q.11.(a) Define Ackerman's function. What is the significance of it? **7M**

Compute A (1, 1), A (2, 1), A (1, 2) and (2, 2).

Ans. See P.6-38, Q.68, P.PS-54, Q.11(b).

Significance of Ackerman's function :

- The Ackerman's function was the first total computable function shown to be not primitive recursive. These are total computable functions that are not primitive recursive.
- It defines a complexity class that grows faster than those defined by primitive recursion such as exponentiation.
- Since the Ackerman's function grows so quickly, its sometimes used as a programming language implementation benchmark or stress test for recursion.

Q.11.(b) Explain the properties of recursive enumerable language **6M**

Ans. See P.6-7, Q.2.

Q.12.(a) Show that the function ABS $(x, y) = |x - y|$ is primitive recursive **7M**

Ans. See P.6-31, Q.51.

Q.12.(b) What is PCP and modified PCP? Give one solution for the PCP for following sequences shown in table : **6M**

i	A	B
1	0	000
2	010001	01
3	01	1

Ans. See P.6-15, 18, Q.18, 23.

Suppose that this instance of PCP has a solution $i_1, i_2, \ldots i_n$

Suppose i = 1 both string begin with 0

A 0

B 000

Since as string of A does not begin with 00

Consider i = 2

A 010001

B 01

The next selection from string B is i = 1

A 01000100

B 01000

The next selection from string B is i = 3

A 010001001

010001

But string B does not have string with 00 or 001

does not equal

Suppose string start with i = 3

But both string not begin with equal string

Thus, PCP has no solution.

NOTES

SUMMER EXAMINATION - 2017
B.E. IV SEM. (COMPUTER TECHNOLOGY) (CBS)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	28 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

SUMMER EXAM. - 2017
QUESTION PAPER

Q.1.(a) Define following properties on a relation **4M**

(1) Reflexive relation

(2) Symmetric relation

(3) Transitive relation

(4) Equivalence relation

(b) State and explain the pigeon hole principle **3M**

(c) Consider a relation R = {(a, b), (a, a), (b, b), (c, c), (d, d), (b, a)} on a set A = {a, b, c, d}

Is relation R a equivalence relation? If so, find the equivalence classes **6M**

OR

Q.2.(a) Write a short note on Chomsky hierarchy **6M**

(b) Prove the following using method of Induction

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$ **7M**

Q.3.(a) Convert the given non deterministic finite automata into a deterministic finite automata, state A is the initial state and states A, D are the final states **7M**

Present state	Next state	
	0	1
→ A*	{A, B, C, D}	-
B	{C, D}	-
C	-	{D}
D*	{D}	-

(b) Design a deterministic finite automata to accept the following strings over symbol 'a' and 'b' **6M**

(a) Strings that do not have 'bab' as a substring

(b) Strings that have 'aa' at most two times

OR

Q.4.(a) Minimize the given deterministic finite automata state A is the initial state and states D, F are the final states **7M**

Present state	Next state	
	0	1
→ A	B	D
B	A	D
C	B	E
D*	F	E
E	D	D
F*	E	E

(b) Differentiate between Moore machine and Mealy machine **3M**

(c) Design a Mealy machine to display the number of '0's seen in the input string in mod 4 system. Input string may contain symbol '0' and '1' **3M**

Q.5.(a) What is an ambiguous grammar? Show that the given grammar is an ambiguous grammar **3M**

S → aSbS | bSaS | ε

(b) Convert the given context free grammar into a Chomsky normal form. Give also the definition of Chomsky normal form. S → abAB, A → bAB | ε, B → BAa | A | ε **4M**

(c) Give the statement of pumping lemma and show that a language L = {ww/w is in (0/1)*} is a non regular language **7M**

OR

Q.6.(a) Design a regular grammar for the following language over symbol '0' and '1' **7M**

(1) String should not start and end with same symbol

(2) String should have even number of 1's

(b) Eliminate epsilon production from the given grammar

S → aSB | aA | bB, A → aA | ε, B → bB | ε **3M**

(c) Convert the given right linear grammar into it's equivalent left linear grammar

S → aaA | bC, A → bbB | b, B → aA | ε, C → ∅ **4M**

Q.7.(a) Construct a push down automata for the following languages **7M**

(1) L = {$a^m b^n c^p d^q$ | where m + n = p + q}

(2) L = {$a^n b^n$ | where n ≥ m + 3}

(b) Show that the context free languages are closed under the property of closure and union **7M**

VBD

OR

Q.8 (a) State whether the following statements are true justify your answer. **6M**

(1) For every nondeterministic pushdown automata there exists a equivalent deterministic push down automata

(2) Push down automata is more powerful than finite automata

(3) Context free languages are closed under the property of intersection

(b) Construct a push down automata for the following language **8M**

(1) $L = \{a^n b^m c^m \mid n \geq 1 \text{ and } m \geq 1\}$

(2) $L = \{a^n b^{2n} \mid n \geq 1\}$

Q.9 (a) Give the formal definition of a Turing machine and linear automata **8M**

(b) Write a short note on universal Turing machine **4M**

(c) Design a Turing machine to obtain 2's complement of binary number **8M**

OR

Q.10 (a) Design a Turing machine to accept a language

$L = \{a^n b^n c^n \mid \text{where } n \geq 1\}$ **8M**

(b) List and explain the variants of a Turing machine **4M**

Q.11 (a) Show that $f(x, y) = x + y$ is a primitive recursive function. **8M**

(b) Define the post correspondence problem and obtain solution for

(1) $w_1 = 11$ $w_2 = 100$ $w_3 = 111$

$x_1 = 111$, $x_2 = 001$, $x_3 = 11$

(2) $w_1 = 00$ $w_2 = 001$ $w_3 = 1000$

$x_1 = 0$, $x_2 = 11$, $x_3 = 011$ **4M**

(c) Give Ackermann's function and obtain solution for A(2, 2) and A(3, 3) **4M**

OR

Q.12 (a) Prove the following statements **8M**

(1) If L and $\bar{L}$ are both recursive enumerable language, then both L and $\bar{L}$ are recursive

(2) Union of two recursively enumerable languages is a recursive languages

(b) Write short note on undecidability **8M**

SUMMER EXAM. - 2017

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks 26)

Q.1 (a) Define following properties on a relation **4M**

(1) Reflexive relation

(2) Symmetric relation.

(3) Transitive relation

(4) Equivalence relation

Ans. See P.1-5, Q.4

(2) **Symmetric relation** : Suppose P is a set of properties of relation. The P closure of a relation R is the smallest relation E that includes all pair of R and possesses the properties in P.

It is symmetric because $a = b$ imply $b = a$

Example : Let $R = \{(1, 2), (2, 3), (3, 1)\}$ be a relation on set $\{1, 2, 3\}$

The symmetric closure of R is

$\{(1, 2), (2, 1), (3, 1), (2, 3), (3, 2), (1, 3)\}$

added by reflexivity

Q.1 (b) State and explain the pigeon hole principle **3M**

Ans. See P.1-10, Q.10.

Q.1 (c) Consider a relation $R = \{(a, b), (a, a), (b, b), (c, c), (d, d), (b, a)\}$ on a set $A = \{a, b, c, d\}$.

Is relation R a equivalence relation? If so, find the equivalence classes. **6M**

Ans. $R = \{(a, b), (a, a), (b, b), (a, a), (d, d), (b, a)\}$

A relation is said to be equivalence relation if it satisfies the following properties

(1) Reflexive

(2) Symmetric

(3) Transitive

The given relation is Reflexive as

$R = \{(a, a), (b, b), (c, c)$ and $(d, d)\}$

The given relation is Symmetric, because

(a, b) ∈ R and (b, a) ∈ R

The given relation is Transitive because

(a, b) ∈ R, (b, a) ∈ R and (a, a) ∈ R

So the given relation is equivalence

Q.2.(a) Write a short note on Chomsky hierarchy. **6M**

Ans. See P.1-11, Q.22.

Q.2.(b) Prove the following using method of induction.

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$ **8M**

Ans. See P.1-8, Q.11.

UNIT - II

(Total Marks : 26)

Q.3.(a) Convert the given non deterministic finite automata into a deterministic finite automata, state A is the initial state and states A, D are the final states. **7M**

Present state	Next state	
	0	1
→ A*	{A, B, C, D}	-
B	{C, D}	-
C	-	{D}
D*	{D}	-

Ans.

Present state	Next state	
	0	1
→ A*	{A, B, C, D}	-
B	{C, D}	-
C	-	{D}
D*	{D}	-

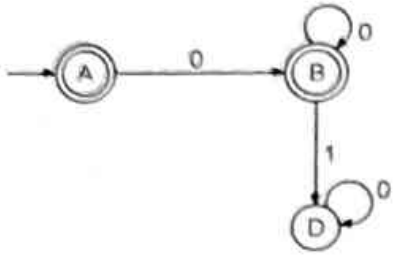
Transition table for DFA :

Present state	Next state	
	0	1
→ A*	{A, B, C, D}	-
{A, B, C, D}	{A, B, C, D}	{D}
{D}	{D}	-

Renaming state {A, B, C, D} → B

Present state	Next state	
	0	1
→ A*	B	-
B*	B	{D}
{D}	{D}	-

Transition diagram :



Q.3.(b) Design a deterministic finite automata to accept the following strings over symbol 'a' and 'b' : **6M**

(a) Strings that do not have 'bab' as a substring.

(b) Strings that have 'aa' at most two times.

Ans.

(a) First, we find NFA which for the string that have 'bab' as a substring

Transition table for NFA :

State	a	b
→ {q0}	{q1}	{q0, q1}
{q1}	{q2}	-
{q2}	-	{q3}
* {q3}	{q3}	{q3}

Transition table for DFA :

State	a	b
→ q0	q0	{q0, q1}
{q0, q1}	{q0, q2}	{q0, q1}

State	a	b
$\{q_0, q_2\}$	$\{q_0\}$	$\{q_0, q_1, q_3\}$
* $\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$
* $\{q_0, q_2, q_3\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_3\}$
* $\{q_0, q_3\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_3\}$

Renaming states

$\{q_0\}$ → A

$\{q_0, q_1\}$ → B

$\{q_0, q_2\}$ → C

$\{q_0, q_1, q_3\}$ → D

$\{q_0, q_2, q_3\}$ → E

$\{q_0, q_3\}$ → F

State	a	b
→ A	A	B
B	C	B
C	A	D
*D	E	D
*E	E	D
*F	E	D

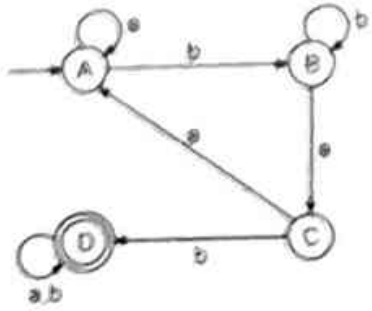
Remove F and replace F by E

State	a	b
→ A	A	B
B	C	B
C	A	D
*D	E	D
*E	E	D

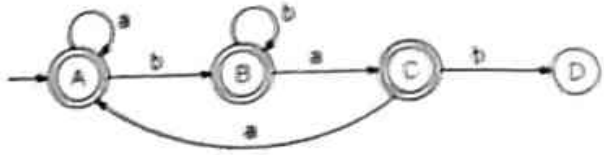
Remove E and replace E by D

State	a	b
→ A	A	B
B	C	B
C	A	D
*D	D	D

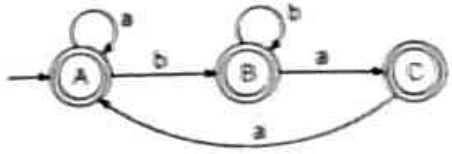
(B) A for string containing 'bab' as and substring



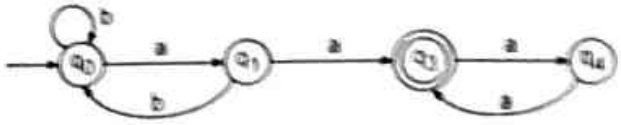
Construct DFA for string not containing 'bab' as substring, interchange the state i.e. final to non final and non final to final



D is a dead state, remove D



(B)



Q.4 (a) Minimize the given deterministic finite automata state A is the initial state and states D, F are the final states. **7M**

Present state	Next state	
	0	1
→ A	B	D
B	A	D
C	B	E
D*	E	F
E	D	D
F*	E	F

Ans.

Step 1 : Set of final state = {D, F} A

Set of non final state = {A, B, C, E} B

Step 2 : {A, B}

∴ A ≠ B

	0	1
A	B	D
B	A	D
	↓	↓
	B	A

{A, C}

∴ A ≠ C

	0	1
A	B	D
C	B	E
	↓	↓
	B	A and B

{A, E}

∴ A ≠ E

	0	1
A	B	D
E	D	D
	↓	↓
	A and B	A

{D, F}

	0	1
D	F	F
F	F	F
	↓	↓
	A	A

∴ D = F

{D, F} → A

{A, B} → B

{C, E} → C

Step 3 : {A, B}

∴ A ≠ B

	0	1
A	B	D
B	A	D
	↓	↓
	B	A

{C, E}

∴ C ≠ E

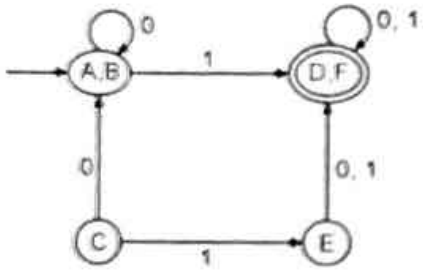
	0	1
C	B	E
E	D	D
	↓	↓
	A and B	A and C

New sets are

{A, B}, {D, F}, {C}, {E}

Transition table :

Σ / State	0	1
→ {A, B}	{A, B}	{D, F}
{C}	{A, B}	{F}
{E}	{D, F}	{D, F}
*{D, F}	{D, F}	{D, F}



Q.4.(b) Differentiate between Moore machine and Mealy machine. **5M**

Ans. See P.2-49, Q.6d

Q.4.(c) Design a Mealy machine to display the number of "0"s seen in the input string in mod 4 system. Input string may contain symbol "0" and "1". **5M**

Ans. $\delta(q_0, 0) = 2 \times 0 + 0 = 0 = q_0$

$\delta(q_0, 1) = 2 \times 0 + 1 = 1 = q_1$

$\delta(q_1, 0) = 2 \times 1 + 0 = 2 = q_2$

$\delta(q_1, 1) = 2 \times 1 + 1 = 3 = q_3$

$\delta(q_2, 0) = 2 \times 2 + 0 = 4/4 = 0 = q_0$

$\delta(q_2, 1) = 2 \times 2 + 1 = 5/4 = 1 = q_1$

$\delta(q_3, 0) = 2 \times 3 + 0 = 6/4 = 2 = q_2$

$\delta(q_3, 1) = 2 \times 3 + 1 = 7/4 = 3 = q_3$

Transition table for Moore machine :

Present state	a = 0	a = 1	Output
→ q_0	q_0	q_1	0
q_1	q_2	q_1	1
q_2	q_0	q_2	2
q_3	q_2	q_1	3

Transition table for Mealy machine :

Present state	a = 0		a = 1	
	State	Output	State	Output
→ q_0	q_0	0	q_1	1
q_1	q_2	2	q_1	1
q_2	q_0	0	q_2	1
q_3	q_2	2	q_1	3

UNIT - III

(Total Marks : 28)

Q.5.(a) What is an ambiguous grammar? Show that the given grammar is an ambiguous grammar [6M]

S → aSbS | bSaS | ε

Ans. See P.3-44, 45, Q.1.20, 12 Marks

Q.5.(b) Convert the given context free grammar into a Chomsky normal form. Give also the definition of Chomsky normal form [6M]

S → abAB, A → bAB/ε, B → BAa / A/a

Ans. S → abAB

A → bAB/ε

B → BAa / A/a

Eliminate ε - production in the given form

S → abAB/abB/abA/ab

A → bAB/bA/bB/b

B → BAa/Aa/ba/a

S → aSbS replaced as S → bSbS and S → ε and S → b

S → abSb is replaced by S → bAB

S → abA is replaced by S → bA

S → ab is replaced by S → b

S → CDAB is replaced by S → DAB and E → CD

S → CDB is replaced by DB

S → CDA is replaced by EA

S → EAB is replaced as FB

and F → EA

Ans. S → FB | DB | EA | CD

F → EA

E → CD

C → a

D → b

(i) A → bAB is replaced by DAB

D → b

A → DAB is replaced by CB

C → DA

A → bA is replaced by DA

A → bB is replaced by DB

S → CB | DA | DB | b

C → DA

D → b

(ii) B → BAa is replaced by BAC

B → BAC is replaced by BC and B → BA

A → Aa is replaced by B → bC

B → Ba is replaced by B → BC

B → a

B → BC | bC | BC | a

C → BA

The production in the CNF form is

S → FB | DB | EA | CD

A → CB | DA | DB | b

B → BC | bC | BC | a

$F \rightarrow EA$

$E \rightarrow CD$

$G \rightarrow DA$

$H \rightarrow BA$

$C \rightarrow a$

$D \rightarrow b$

Q.5.(c) Give the statement of pumping lemma and show that a language L = {ww/w is in $(0/1)^*$} is a non regular language. **5M**

See P.3-53, 55, Q.81, 87

Q.6.(a) Design a regular grammar for the following language over symbol '0' and '1' : **5M**

(1) String should not start and end with same symbol.

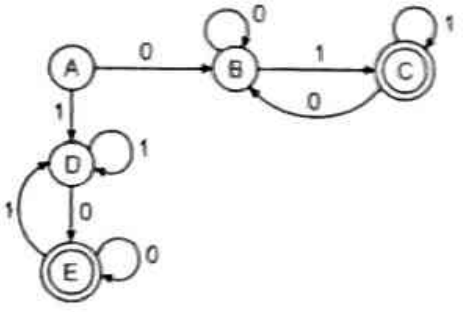
(2) String should have even number of 1's.

Ans.

(1) String should not start and end with same symbol

If the string starts with 0 then it ends with 1

The DFA is,



Let, G = {(A, B, C, D, E), (0, 1), P, A}

where P is given by

$A \rightarrow 0B$

$A \rightarrow 1D$

$B \rightarrow 0B$

$B \rightarrow 1C$

$C \rightarrow 1$

$D \rightarrow 1D$

$D \rightarrow 0E$

$D \rightarrow 0$

$E \rightarrow 0$

$E \rightarrow 1D$

The regular grammar is

$A \rightarrow 0B/1D$

$B \rightarrow 0B/1C/1$

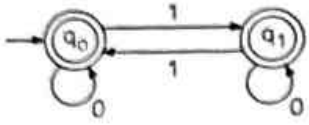
$C \rightarrow 1$

$D \rightarrow 0E/0$

$E \rightarrow 1D/0$

(2) String should have even number of 1's

DFA is



Reaming

$q_0 \rightarrow A_0$

$q_1 \rightarrow A_1$

Let $G = \{(A_0, A_1), (0,1), P, A_0)\}$

where P is given by

$A_0 \rightarrow 0A_0$

$A_0 \rightarrow 0$

$A_0 \rightarrow 1A_1$

$A_1 \rightarrow 0A_1$

$A_1 \rightarrow 1A_0$

$A_1 \rightarrow 1$

∴ The regular grammar is

$A_0 \rightarrow 0A_0 / 1A_1 / 0$

$A_1 \rightarrow 0A_1 / 1A_0 / 1$

Q.6.(b) Eliminate epsilon production from the given grammar

$S \rightarrow aSB / aA / bB$, $A \rightarrow aA / \varepsilon$, $B \rightarrow bB / \varepsilon$ **5M**

Ans. **See P.3-77, Q.149.**

Q.6.(c) Convert the given right linear grammar into its equivalent left linear grammar.

$S \rightarrow aaA / bC$, $A \rightarrow bbB / b$, $B \rightarrow cA / c$, $c \rightarrow d$ **4M**

Ans. $S \rightarrow aaA / bC$,

$A \rightarrow bbB / b$,

$B \rightarrow cA / c$,

$c \rightarrow d$

Let $w_1 = aa$

$w_2 = bc$

$w_3 = bb$

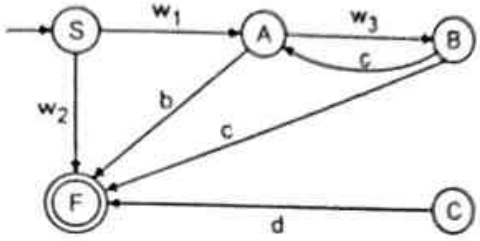
$\therefore S \to w_1A \mid w_2$

$A \to w_1B \mid b$

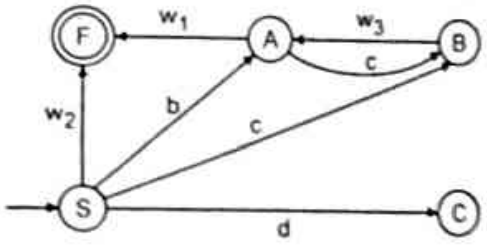
$B \to cA \mid c$

$c \to d$

Transition graph for right linear grammar :



Transition graph for left linear grammar :



Left linear grammar :

$S \to Ab \mid Bc \mid Cd \mid w_2$

$A \to Bc \mid w_1$

$B \to Aw_3$

$S \to Ab \mid Bc \mid Cd \mid bb$

$A \to Bc \mid aa$

$B \to Abb$

UNIT - IV

(Total Marks : 28)

Q.7 (a) Construct a push down automata for the following languages.

(1) $L = \{a^m b^n c^p d^q \mid \text{where } m + n = p + q\}$

(2) $L = \{a^n b^{2n} \mid \text{where } n \geq m \geq 3\}$

VBD

Ans.

(1) $L = \{a^m b^n c^p d^q \mid \text{where } m + n = p + q\}$

Analysis :

(i) The language is fixed

(ii) All string start with a and end with d

(iii) In between string consist of b and c

(iv) The number of a and b is equal to number of c and d.

Algorithm :

(i) Push a and b to stack.

(ii) For c and d symbol pop a and b from the stack.

Transition :

$\delta(q_0, a, z_0) \to (q_0, az_0)$

$\delta(q_0, a, a) \to (q_0, aa)$

$\delta(q_0, b, a) \to (q_0, ba)$

$\delta(q_0, b, b) \to (q_0, bb)$

$\delta(q_0, c, b) \to (q_1, \epsilon)$

$\delta(q_1, c, b) \to (q_1, \epsilon)$

$\delta(q_1, c, a) \to (q_2, \epsilon)$

$\delta(q_2, c, a) \to (q_2, \epsilon)$

$\delta(q_2, d, b) \to (q_3, \epsilon)$

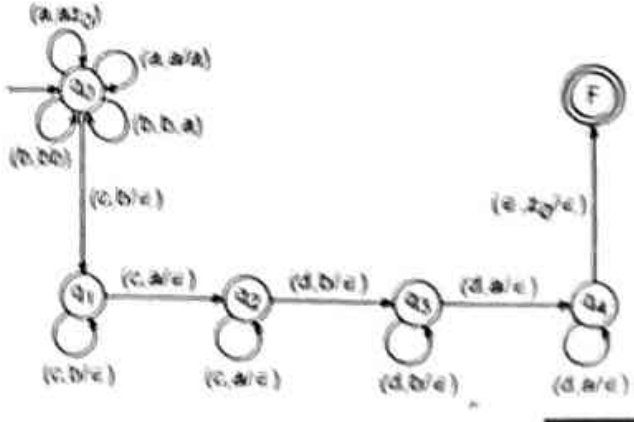
$\delta(q_3, d, b) \to (q_3, \epsilon)$

$\delta(q_3, d, a) \to (q_4, \epsilon)$

$\delta(q_4, d, a) \to (q_4, \epsilon)$

$\delta(q_4, \epsilon, z_0) \to (q_f, \epsilon)$

Transition diagram :



VBD

(2) $L = \{a^n b^m \mid n \le m + 3\}$

Analysis :

(i) The language is fixed.

(ii) All string start with a and end with b.

(iii) Number of a should be less than or equal to b.

Algorithm :

(i) Push 'a' on to the stack.

(ii) Pop 'a' when b is read.

Transition :

$\delta(q_0, a, z_0) \rightarrow (q_0, az_0)$

$\delta(q_0, a, a) \rightarrow (q_0, aa)$

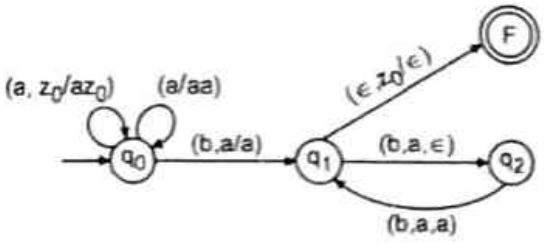
$\delta(q_0, b, a) \rightarrow (q_1, a)$

$\delta(q_1, b, a) \rightarrow (q_2, \in)$

$\delta(q_2, b, a) \rightarrow (q_1, a)$

$\delta(q_1, \in, z_0) \rightarrow (q_f, \in)$

Transition diagram :



Q.7.(b) Show that the context free languages are closed under the property of closure and union. **7M**

Ans. Context free language are closed closure. That means if L is a CFL, then L^* is always a CFL.

Since L is a CFL, there exists CFG

G = (V, T, P, S) generating L.

Let us construct a CFG, $G_1 = (V_1, T_1, P_1, S_1)$ using G as follows

$V_1 = V \cup \{S_1\}$

$T_1 = T$

$P_1 = P \cup \{S_1 \rightarrow S_1 S \mid \in\}$

$\therefore L(G_1) = L^*$

Context-free language are closed under union

That means if L_1 and L_2 are the CFL's then

$L_1 \cup L_2$ in always CFL.

Since L_1 and L_2 are CFL's there exists CFG's

$G_1 = (V_1, T_1, P_1, S_1)$ and

$G_2 = (V_2, T_2, P_2, S_2)$

generating L_1 and L_2 respectively.

Let us construct a CFG $G = (V, T, P, S)$ using G_1 and G_2 as follows

$V = V_1 \cup V_2 \cup \{S\}$

$T = T_1 \cup T_2$

$\therefore P = P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}$

L(G) will therefore contain those strings that are derivable from S_1, as well as derivable from S_2, hence $L(G) = L_1 \cup L_2$

Q.8.(a) State whether the following statements are true justify your answer. **6M**

(1) For every nondeterministic pushdown automata there exists a equivalent deterministic push down automata.

(2) Push down automata is more powerful than finite automata.

(3) **Context free language are closed under the property of intersection.**

Ans. **See P.4-6, Q.5, 6.**

(3) Context-free language are not closed under intersection :

Proof :

We know that the language $L = \{a^i b^i c^i \mid i \ge 1\}$ is not a CFL (by Pumping Lemma)

Consider the language

$L_1 = \{a^i b^i c^j \mid i, j \ge 1\}$

$L_2 = \{a^i b^j c^j \mid i, j \ge 1\}$

Both those language are CFL's because there exists CFG given below, generating L_1 and L_2 respectively CFG, for L_1 is ,

$S \rightarrow AB$

$A \rightarrow aAb \mid ab$

$B \rightarrow cB \mid c$

CFG for L_2 is :

$S \rightarrow AB$

$A \rightarrow aA \mid a$

$B \rightarrow bBc \mid bc$

We find that $L_1 \cap L_2 = \{a^i b^i c^i \mid i \geq 1\}$, which is not a CFG.

Therefore CFL are not closed under intersection.

Q.8.(b) Construct a push down automata for the following languages. **8M**

(1) $L = \{a^n b^m c^m \mid n \geq 1 \text{ and } m \geq 1\}$

(2) $L = \{a^n b^{2n} \mid n \geq 1\}$

Ans.

(1) $L = \{a^n b^m c^m \mid n \geq 1 \text{ and } m \geq 1\}$

Analysis :

(i) The language is fixed.

(ii) The string start with a and end with c.

(iii) The number of b and c are equal.

Algorithm :

(i) For all input a push 'a' onto the stack.

(ii) For $\in$ input pop 'a' from stack.

(iii) For all input b push 'b' onto the stack.

(iv) For all input 'c' pop b from the stack.

Transition :

(i) $\delta(q_0, a, z_0) \rightarrow (q_0, az_0)$

(ii) $\delta(q_0, a, a) \rightarrow (q_0, aa)$

(iii) $\delta(q_0, \in, a) \rightarrow (q_1, \wedge)$

(iv) $\delta(q_1, \in, a) \rightarrow (q_1, \wedge)$

(v) $\delta(q_1, \in, z_0) \rightarrow (q_2, z_0)$

(vi) $\delta(q_2, b, z_0) \rightarrow (q_3, bz_0)$

(vii) $\delta(q_3, b, b) \rightarrow (q_3, bb)$

(viii) $\delta(q_3, c, b) \rightarrow (q_4, \wedge)$

(ix) $\delta(q_4, c, b) \rightarrow (q_4, \wedge)$

(x) $\delta(q_4, \in, z_0) \rightarrow (q_f, \in)$

(2) **See P.4-9, Q.12.**

UNIT - V

(Total Marks : 26)

Q.9.(a) Give the formal definition of a Turing machine and linear bounded automata. **3M**

Ans. **See P.5-7, 34, Q.1, 40.**

Q.9.(b) Write a short note on universal Turing machine. **3M**

Ans. **See P.5-27, Q.26.**

Q.9.(c) Design a turing machine to obtain 2's complement of a binary number. **7M**

Ans. **See P.5-37, Q.45.**

Q.10.(a) Design a Turing machine to accept a language

$L = \{a^n b^n c^n \mid \text{where } n \geq 1\}$ **7M**

Ans. **See P.5-15, Q.10.**

Q.10.(b) List and explain the variants of a Turing machine. **6M**

Ans. **See P.5-21, Q.19.**

UNIT - VI

(Total Marks : 26)

Q.11.(a) Show that $f(x, y) = x + y$ is a primitive recursive function. **5M**

Ans. **See P.6-29, Q.45.**

Q.11.(b) Define the post correspondence problem and obtain solution for : **8M**

(1) $w_1 = 11;\ w_2 = 100;\ w_3 = 111$

$x_1 = 111, x_2 = 001, x_3 = 11$

(2) $w_1 = 00;\ w_2 = 001;\ w_3 = 1000$

$x_1 = 0, x_2 = 11, x_3 = 011$

Ans. **See P.6-15, Q.18.**

(1) $w_1 = 11;\ w_2 = 100;\ w_3 = 111$

$x_1 = 111;\ x_2 = 001;\ x_3 = 11$

i	List A w_i	List B x_i
1	11	111
2	100	001
3	111	11

Suppose that this instance of PCP has a solution

Let $i_1 = 1$

we write the string from list A above the corresponding string from list B.

i.e. 11

111

The next selection from list A begin with 1 i.e. with 1, 2 or 3.

Thus $i_2 = 2$ or $i_2 = 3$

$\therefore i_2 = 2$

i.e. 11100

111001

$\therefore i_2 = 1$

i.e. 1110011

111001111

$\therefore i_3 = 3$

i.e. 1110011111

11100111111

$\therefore i_4 = 1$

i.e. 1110011111

11100111111

Since the string from list B exceeds the string from list A

Thus according to above sequence of choices that generate compatible string, the string of list B is always longer.

Thus instance of PCP has no solution

(2) $w_1 = 00,\ w_2 = 001, w_3 = 1000$

$v_1 = 0, v_2 = 11, v_3 = 011$

i	List A	List B
1	00	0
2	001	11
3	1000	011

Suppose that this instance has a solution $i_1, i_2 \ldots i_k$

Let $i_1 = 1$

00 List A

0 List B

The next selection $i_2 = 3$

00100

0011

Not equal string

If begin with $i_1 = 2$. Thus both strings are not equal

i.e. 001

11

If $i_1 = 3$

Then both string are not beginning with same symbol

$\therefore$ PCP has no solution

Q.11.(c) Give Ackermann's function and obtain solution for A(2, 5) and A(3, 3) **4M**

Ans. Ackermann's function is defined by

$A(0, y) = y + 1$ ..... (1)

$A(x + 1, 0) = A(x, 1)$ ..... (2)

$A(x + 1, y + 1) = A(x, A(x + 1, y))$ ..... (3)

(i) $A(2, 5) = A(1 + 1, 4 + 1)$

$= A(1, A(2, 4))$

$= A(1, A(1 + 1, 3 + 1))$

$= A(1, A(1, A(2, 3)))$

$= A(1, A(1, A(1 + 1, 2 + 1)))$

$= A(1, A(1, A(1, A(2, 2))))$

$= A(1, A(1, A(1, A(1 + 1, 1 + 1))))$

$= A(1, A(1, A(1, A(2, 1))))$

$= A(1, A(1, A(1, A(1 + 1, 0 + 1))))$

$= A(1, A(1, A(1, A(2, 0))))$

$= A(1, A(1, A(1, A(0, 2))))$

$= A(1, A(1, A(1, 3)))$

$= A(1, A(1, A(0 + 1, 2 + 1)))$

$= A(1, A(1, A(0, A(1, 2))))$

$= A(1, A(1, A(0, A(0 + 1, 1 + 1))))$

$= A(1, A(1, A(0, A(0 + A(1, 1)))))$

$= A(1, A(1, A(0, A(0, A(0 + 1, 0 + 1)))))$

$= A(1, A(1, A(0, A(0, A(0, A(1, 0))))))$

$= A(1, A(1, A(0, A(0, A(0, 2)))))$

$= A(1, A(1, A(0, A(0, 3))))$

= A(1, A(1, A(0, 4)))

= A(1, A(1, 5))

= A(1, A(0 + 1, 4 + 1))

= A(1, A(0, A(1, 4)))

= A(1, A(0, A(0 + 1, 3 + 1)))

= A(1, A(0, A(0, A(1, 3))))

= A(1, A(0, A(0, A(0 + 1, 2 + 1))))

= A(1, A(0, A(0, A(0, A(1, 2)))))

= A(1, A(0, A(0, A(0, A(0 + 1, 1 + 1)))))

= A(1, A(0, A(0, A(0, A(0, A(1, 1))))))

= A(1, A(0, A(0, A(0, A(0, A(0 + 1, 0 + 1))))))

= A(1, A(0, A(0, A(0 , A(0 , A(0, A(1, 0)))))))

= A(1, A(0, A(0, A(0 , A(0 , A(0, 2))))))

= A(1, A(0, A(0, A(0 , A(0 , 3)))))

= A(1, A(0, A(0, A(0 , 4))))

= A(1, A(0, A(0, 5)))

= A(1, A (0, 6))

= A(1, 7)

= A(0 + 1, 6 + 1)

= A(0, A (1, 6))

= A(0, A (0 + 1, 5 + 1))

= A(0, A (0, A (1, 5)))

= A(0, A (0, A (0 + 1, 4 + 1)))

= A(0, A(0, A(0, A(1, 4))))

= A(0, A(0, A(0, A(0 + 1, 3 + 1))))

= A(0, A(0, A(0, A(0, A (1, 3)))))

= A(0, A(0, A(0, A(0, A(0 + 1, 2 + 1)))))

= A(0, A(0, A(0, A(0, A(0, A(1, 2))))))

= A(0, A(0, A(0, A(0, A(0, A(0 + 1, 1 + 1)))))

= A(0, A(0, A(0, A(0, A(0, A(0, A(1, 1)))))))

= A(0, A(0, A(0, A(0, A(0, A(0, A(1, 1)))))))

= A(0, A(0, A(0, A(0, A(0, A(0, A(0 + 1, 0 + 1)))))))

= A(0, A(0, A(0, A(0, A(0, A(0, A(0, A(1, 0)))))))))

= A(0, A(0, A(0, A(0, A(0, A(0, A(0, 5)))))))

= A(0, A(0, A(0, A(0, A(0, A(0, 5))))))

= A(0, A(0, A(0, A(0, A(0, 6)))))

= A(0, A(0, A(0, A(0, 7))))

= A(0, A(0, 8))

= 9

∴ A(2, 5) = 9

(iii) A (3, 3) = A(1 + 2, 1 + 2)

= A(2, A(3, 2))

= A(2, A(2 + 1, 1 + 1))

= A(2, A(2, A(3, 1)))

= A(2, A(2, A(2 + 1, 0 + 1)))

= A(2, A(2, A(2, A(3, 0))))

= A(2, A(2, A(2, 4)))

= A(2, A(2, A(1 + 1, 3 + 1)))

= A(2, A(2, A(1, A(2, 3))))

= A(2, A(2, A(1, A(1 + 1, 2 + 1))))

= A(2, A(2, A(1, A(1, A(2, 2)))))

= A(2, A(2, A(1, A(1, A(1 + 1, 1 + 1)))))

= A(2, A(2, A(1, A(1, A(1, A(2, 1))))))

= A(2, A(2, A(1, A(1, A(1, A(1 + 1, 0 + 1))))))

= A(2, A(2, A(1, A(1, A(1, A(1, A(2, 0)))))))

= A(2, A(2, A(1, A(1, A(1, A(1, 3))))))

= A(2, A(2, A(1, A(1, A(1, A(0 + 1, 2 + 1))))))

= A(2, A(2, A(1, A(1, A(1, A(0, A(1, 2))))))

= A(2, A(2, A(1, A(1, A(1, A(0, A(0 + 1, 1 + 1)))))))

= A(2, A(2, A(1, A(1, A(1, A(0, A(0, A(1, 1)))))))

= A(2, A(2, A(1, A(1, A(1, A(0, A(0, A(0 + 1, 0 + 1))))))))

= A(2, A(2, A(1, A(1, A(1, A(0, A(0, A(0, A(1, 0)))))))))

= A(2, A(2, A(1, A(1, A(1, A(0, A(0, A(0, 2)))))))))

= A(2, A(2, A(1, A(1, A(1, A(0, A(0, A(0, 3))))))))

= A(2, A(2, A(1, A(1, A(1, A(0, A(0, A(0, 4)))))))

= A(2, A(2, A(1, A(1, A(1, A(0, A(0, 5)))))))

= A(2, A(2, A(1, A(1, A(0 + 1, 4 + 1)))))

= A(2, A(2, A(1, A(1, A(0, A(1, 4))))))

= A(2, A(2, A(1, A(1, A(0, A(0 + 1, 3 + 1))))))

= A(2, A(2, A(1, A(1, A(0, A(0, A(1, 3)))))))

= A(2, A(2, A(1, A(1, A(0, A(0, A(0 + 1, 2 + 1)))))))

= A(2, A(2, A(1, A(1, A(0, A(0, A(0, A(1, 2))))))))

= A(2, A(2, A(1, A(1, A(0, A(0, A(0, A(0 + 1, 1 + 1)))))))

= A(2, A(2, A(1, A(1, A(0, A(0, A(0, A(0, A(1, 1)))))))

= A(2, A(2, A(1, A(1, A(0, A(0,

A(0, A(0, A(0 + 1, 0 + 1)))))))))

= A(2, A(2, A(1, A(1, A(0, A(0, A(0,

A(0, A(0, A(1, 0)))))))))

= A(2, A(2, A(1, A(1, A(0, A(0, A(0, A(0, A(0, 2)))))))))

= A(2, A(2, A(1, A(1, A(0, A(0, A(0, A(0, 3)))))))))

= A(2, A(2, A(1, A(1, A(0, A(0, A(0, 4)))))))

= A(2, A(2, A(1, A(1, A(0, A(0, 5)))))))

= A(2, A(2, A(1, A(1, A(0, 6)))))

= A(2, A(2, A(1, A(1, 7))))

= A(2, A(2, A(1, A(1, 7))))

= A(2, A(2, A(1, 9))) *[∵ A(1, 7) = 9]*

A(1, 9) = A(0 + 1, 8 + 1)

= A(0, A(1, 8))

= A(0, A(0 + 1, 7 + 1))

= A(0, A(0, A(1, 7)))

= A(0, A(0, A(0 + 1, 6 + 1)))

= A(0, A(0, A(0, A(1, 6))))

= A(0, A(0, A(0, A(0 + 1, 5 + 1))))

= A(0, A(0, A(0, A(0, A(1, 5)))))

= A(0, A(0, A(0, A(0, 7)))) *[∵ A(1, 5) = 7]*

= A(0, A(0, A(0, 8)))

= A(0, A(0, 9))

= A(0, 10)

= 11

∴ A(2, A(2, 11))

∴ A(2, 11) = A(1 + 1, 10 + 1)

= A(1, A(2, 10))

= A(1, A(1 + 1, 9 + 1))

= A(1, A(1, A(2, 9)))

= A(1, A(1, A(1 + 1, 8 + 1)))

= A(1, A(1, A(1, A(2, 8))))

= A(1, A(1, A(1, A(1 + 1, 7 + 1))))

= A(1, A(1, A(1, A(1, A(2, 7)))))

= A(1, A(1, A(1, A(1 + 1, 6 + 1)))))

= A(1, A(1, A(1, A(1, A(1 + 1, 5 + 1))))))

= A(1, A(1, A(1, A(1, A(1, A(2, 5)))))))

= A(1, A(1, A(1, A(1, A(1, A(1, 9))))))

= A(1, A(1, A(1, A(1, A(1, 11)))))

A(1, 11) = A(0 + 1, 10 + 1)

= A(0, A(1, 10))

= A(0, A(0 + 1, 9 + 1))

= A(0, A(0, A(1, 9)))

= A(0, A(0, A(0 + 1, 8 + 1)))

= A(0, A(0, A(0, A(1, 8))))

= A(0, A(0, A(0, A(0 + 1, 7 + 1))))

= A(0, A(0, A(0, A(0, A(1, 7)))))

= A(0, A(0, A(0, A(0, 9)))) *[∵ A(1, 7) = 9]*

= A(0, A(0, A(0, 10)))

= A(0, A(0, 11))

= A(0, 12)

= 13

∴ A(1, A(1, A(1, A(1, 13))))

∴ A(1, 13) = A(0 + 1, 12 + 1)

= A(0, A(1, 12))

= A(0, A(0 + 1, 11 + 1))

= A(0, A(0, A(1, 11)))

= A(0, A(0, 13))) *[∵ A(1, 11) = 13]*

= A(0, 14)

= 15

∴ A(1, A(1, A(1, 15)))

∴ A(1, 15) = A(0 + 1, 14 + 1)

= A(0, A(1, 14))

= A(0, A(0 + 1, 13 + 1))

= A(0, A(0, A(1, 13)))

= A(0, A(0, 15)) *[∵ A(1, 13) = 15]*

= A(0, 16)

= 17

A(1, A(1, 17))

A(1, 17) = A(0 + 1, 16 + 1)

= A(0, A(1, 16))

= A(0, A(0 + 1, 15 + 1))

= A(0, A(0, A(1, 15)))

= A(0, A(0, 17)) [∵ A(1, 15) = 17]

= A(0, 18)

= 19

A(1, 19) = A(0 + 1, 18 + 1)

= A(0, A(1, 18))

= A(0, A(0 + 1, 17 + 1))

= A(0, A(0, A(1, 17)))

= A(0, A(0, 19)) [A(1, 17) = 19]

= A(0, 20)

= 21

A(3, 3) = 21

Q.12.(a) Prove the following statements **8M**

(1) If L and $\overline{L}$ are both recursive enumerable language then both L and $\overline{L}$ are recursive

(2) Union of two recursively enumerable languages is a recursive language

Ans. See P.6-9, 8, Q.6. 5

Q.12.(b) Write short note on undecidability **8M**

Ans. See P.6-14, Q.16(a)

NOTES

WINTER EXAMINATION - 2017
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	28 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

WINTER EXAM.-2017
QUESTION PAPER

Q.1.(a) Explain pigeonhole principle **3M**

(b) Define the following terms with suitable example

(i) Prefix

(ii) Suffix

(iii) Substring

(iv) Subsequences **8M**

(c) Explain closure of a relation,

find R* for

R = {(1, 1), (1, 2), (2, 1), (2, 3), (3, 2)} **2M**

OR

Q.2.(a) Explain Chomsky Hierarchy in detail **6M**

(b) Using mathematical induction, prove that

$P(n) : 1.1! + 2.2! + 3.3! + \ldots + n.n!$

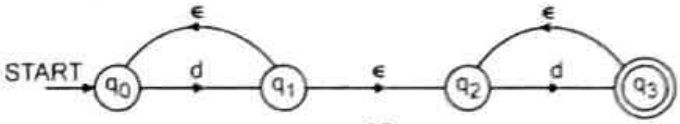
$= (n+1)! - 1, n > 1$ **7M**

Q.3.(a) Design a finite automata for accepting :

$$L = \left\{ w \;\middle|\; \begin{array}{c} w \in (a/b)^* \\ n(a)\ W \ MOD3 > n(b) W \ MOD3 \end{array} \right\}$$

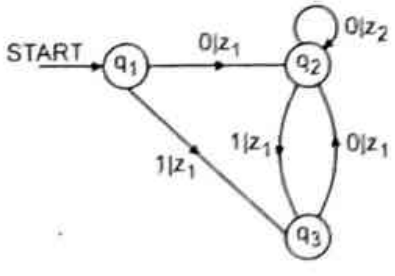
also write application of finite automata **7M**

(b) Obtain a deterministic finite automata for **6M**



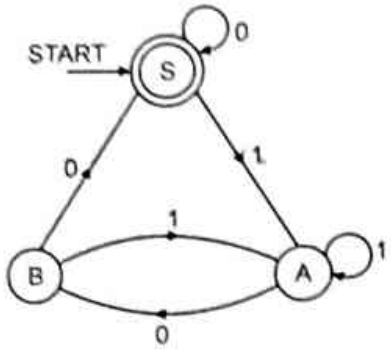
OR

Q.4.(a) Convert the given Mealy machine into equivalent Moore machine **7M**



(b) Design a Moore and Mealy machine over $\Sigma = \{a, b\}$ such that it will generate output EVEN if the number of a's are even and generate output ODD if number of a's are odd **6M**

Q.5.(a) Convert the given finite automata to equivalent regular expression **8M**



(b) Construct NFA with $\in$ - transitions equivalent to the given regular expression

$\boxed{R = 10 + (0 + 11)\,0^*1}$ **6M**

OR

Q.6.(a) Explain the procedure to convert right linear grammar to equivalent left linear grammar, convert the given RLG into LLG :

$S \rightarrow 01A|10$

$A \rightarrow 10A|10$ **8M**

(b) Convert the given grammar into GNF without renaming the grammar

$S \rightarrow AB$

$A \rightarrow BS \mid b$

$B \rightarrow SA \mid a$ **6M**

Q.7.(a) Design a non deterministic PDA for

$$L = \left\{ WW^R \;\middle|\; \begin{array}{c} W \in (a/b)^* \\ W^R \text{ is reverse of } W \end{array} \right\}$$

Also explain STACK execution with valid string **7M**

(b) Design a PDA for given CFG

$S \rightarrow XY$

$X \rightarrow AX|BX \mid a$

$Y \rightarrow YA \mid YB \mid a$

$A \rightarrow a$

$B \rightarrow b$ **7M**

OR

Q.8.(a) Design a PDA for

$$L = \left\{ W \;\middle|\; \begin{array}{l} W \in (a/b)^* \\ n_a(W) = n_b(W) \end{array} \right.$$

Where $n_a(W)$ → Number of a's in W

and $n_b(W)$ → Number of b's in W **7M**

(b) Convert the given PDA to CFG :

$\delta(q_0, a, z_0) \Rightarrow (q_0, xz_0)$

$\delta(q_0, a, x) \Rightarrow (q_0, xx)$

$\delta(q_0, b, x) \Rightarrow (q_1, \epsilon)$

$\delta(q_1, b, x) \Rightarrow (q_1, \epsilon)$

$\delta(q_1, \epsilon, z_0) \Rightarrow (q_1, \epsilon)$ **7M**

Q.9.(a) Design a Turing machine to perform $\frac{(n+1)}{2}$ where n is an unary number **7M**

(b) Explain different types of Turing machine **6M**

OR

Q.10.(a) Design a Turing machine that has

INPUT → #W# and generates

OUTPUT → #W#W

where $W = \{a, b\}^+$ **7M**

(b) Explain various properties of recursively enumerable language **6M**

Q.11.(a) What do you mean by primitive recursive functions? Show that

ADD (x, y) and SUB (x, y) are primitive recursive **7M**

(b) Write short note on LBA **6M**

OR

Q.12.(a) What is Ackermann's function? Calculate

A(1, 1) A(1, 2) A(2, 1) **7M**

(b) What is the significance of PCP? Solve the following using PCP

6M

i	wi	xi
1	0	(00)
2	01(00)	01
3	01	1

WINTER EXAM.-2017
SOLUTION
(Unit-Wise)

UNIT - I
(Total Marks : 26)

Q.1.(a) Explain pigeon hole principle **3M**

Ans. **See P.1-10, Q.16.**

Q.1.(b) Define the following terms with suitable example

(i) Prefix

(ii) Suffix

(iii) Substring.

(iv) Subsequences. **8M**

Ans. **See P.1-4, Q.2.**

(iii) Substring : A substring of a string S is a string S' that occurs in S.

e.g. For sequence abc we have

Substrings a, ab, abc, b, bc, c and the empty substring

(iv) Subsequences : A subsequence is a sequence that can be derived from another sequence by deleting some elements without changing the order of remaining elements

e.g. The sequence (A, B, D) is subsequence of (A, B, C, D, E, F) obtained after removal of elements C, E & F

Q.1.(c) Explain closure of a relation, find R* for

R = {(1, 1), (1, 2), (2, 1), (2, 3), (3, 2)}. **2M**

Ans. R = {(1, 1), (1, 2), (2, 1), (2, 3), (3, 2)}

$R^2 = R \circ R$ = {(1, 1), (1, 2), (2, 1), (2, 3), (3, 2)} ∘

{(1, 1), (1, 2), (2, 1), (2, 3), (3, 2)}

= {(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (3, 1), (3, 3)}

$R^3 = R^2 \circ R$ = {(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (3, 1), (3, 3)} ∘ {(1, 1), (1, 2), (2, 1), (2, 3), (3, 2)}

= {(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2)}

$R^4 = R^3 \circ R = \{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2),$

$(2, 3), (3, 1), (3, 2)\} \circ \{(1, 1),$

$(1, 2), (2, 1), (2, 3), (3, 2)\}$

$= \{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3),$

$(3, 1), (3, 2), (3, 3)\}$

$R^5 = R^4 \circ R = \{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2),$

$(2, 3), (3, 1), (3, 2)\} \circ \{(1, 1),$

$(1, 2), (2, 1), (2, 3), (3, 2)\}$

$= \{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2)\}$

$= R^3$

$R^6 = R^5 \circ R = R^3 \circ R = R^4$

$R^* = R \cup R^2 \cup R^3 \cup R^4$

$= \{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2),$

$(2, 3), (3, 1), (3, 2), (3, 3)\}$

$R^+ = R^* \cup \{(1, 1), (2, 2), (3, 3)\}$

$= \{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2),$

$(2, 3), (3, 1), (3, 2), (3, 3)\}$

Q.2.(a) Explain Chomsky Hierarchy in detail. **6M**

Ans. See P.1-11, Q.22.

Q.2.(b) Using Mathematical induction, prove that

Prove $1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$ **7M**

Ans. See P.1-8, Q.12.

UNIT - II

(Total Marks : 26)

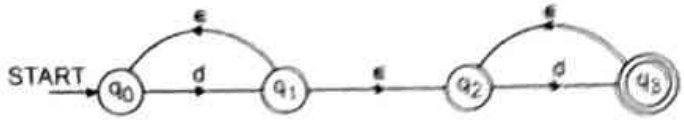
Q.3.(a) Design a finite automata for accepting

$$L = \left\{ w \;\middle|\; \begin{array}{c} w \in (a+b)^* \\ n(a) \bmod 3 > n(b) \bmod 3 \end{array} \right\}$$

also write applications of finite automata **7M**

Ans. See P.2-33, Q.48.

Q.3.(b) Obtain a deterministic finite automata for : **6M**



Ans. Transition table :

Q \ Σ	d	ϵ
→ q_0	q_1	-
q_1	-	q_0, q_2
q_2	q_3	-
q_3	-	q_2

ϵ - closure $(q_0) = \hat{\delta}(q_0, \epsilon) = \{q_0\}$

ϵ - closure $(q_1) = \hat{\delta}(q_1, \epsilon) = \{q_0, q_1, q_2\}$

ϵ - closure $(q_2) = \hat{\delta}(q_2, \epsilon) = \{q_2\}$

ϵ - closure $(q_3) = \hat{\delta}(q_3, \epsilon) = \{q_2, q_3\}$

Transition of NFA without ϵ - moves

$\hat{\delta}(q_0, d) = \epsilon$ - closure $\delta(\hat{\delta}(q_0, \epsilon), d))$

$= \epsilon$ - closure $(\delta(q_0, d))$

$= \epsilon$ - closure (q_1)

$= q_0, q_1, q_2$

$\hat{\delta}(q_1, d) = \epsilon$ - closure $\delta(\hat{\delta}(q_1, \epsilon), d))$

$= \epsilon$ - closure $(\delta(q_0, q_1, q_2), d)$

$= \epsilon$ - closure $(\delta(q_0, d), \delta(q_1, d), \delta(q_2, d))$

$= \epsilon$ - closure (q_1, q_3)

$= \epsilon$ - closure $(q_1) \cup \epsilon$ - closure (q_3)

$= q_0, q_1, q_2, q_3$

$\hat{\delta}(q_2, d) = \epsilon$ - closure $(\delta(\hat{\delta}(q_2, \epsilon), d))$

$= \epsilon$ - closure $(\delta(q_3, d))$

$= \epsilon$ - closure (q_2)

$= q_2, q_3$

$\hat{\delta}(q_3, d) = \epsilon$ - closure $(\delta(\hat{\delta}(q_3, \epsilon), d))$

$= \epsilon$ - closure $(\delta(q_2, q_3), d)$

$= \epsilon$ - closure $(\delta(q_2, d), \delta(q_3, d))$

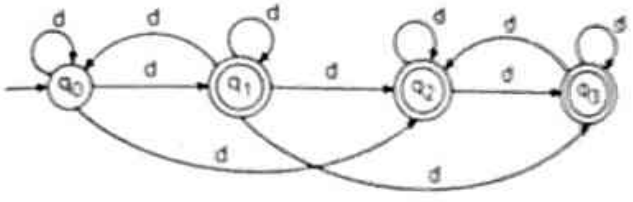
ϵ - closure (q_1)

$= q_1, q_2$

Transition Table for NFA without ϵ - moves

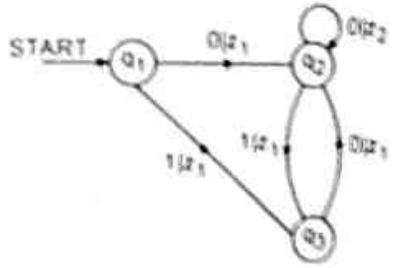
Σ / Q	d
$*q_0$	$\{q_0, q_1, q_2\}$
$*q_1$	$\{q_1, q_2, q_3\}$
$*q_2$	q_2, q_3
$*q_3$	q_3, q_3

Transition graph for NFA without ϵ - moves



Q.4.(a) Convert the given Mealy machine into equivalent Moore machine.

7M



Ans See P.3-51, Q.7(i).

Q.4.(b) Design a Moore and Mealy machine over $\Sigma = \{a, b\}$ such that it will generate output 'XYZ' if the number of a's are even and generate output 'ABC' if number of a's are odd.

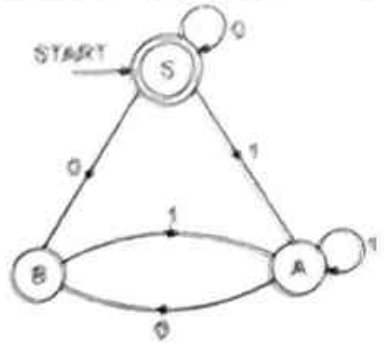
6M

Ans See P.3-11, Q.4(ii).

UNIT - III

(Total Marks : 28)

Q.5.(a) Convert the given finite automata to equivalent regular expression.



Ans See P.3-46, Q.6(i).

Q.5.(b) Construct NFA with ϵ - transitions equivalent to the given regular expression

$R = 10 + (0 + 11)0^*1$

6M

Ans See P.3-18, Q.22.

Q.6.(a) Explain the procedure to convert right linear grammar to equivalent left linear grammar, convert the given RLG into LLG.

$S \rightarrow 1B \mid 0D$

$A \rightarrow 1B \mid 0$

8M

Ans See P.PS-43, Q.6(ii).

Q.6.(b) Convert the given grammar into CNF without removing the grammar :

$S \rightarrow aB$

$A \rightarrow BS \mid b$

$B \rightarrow SA \mid a$

6M

Ans $S \rightarrow aB$

$A \rightarrow BS \mid b$

$B \rightarrow SA \mid a$

Now,

Here, $A \rightarrow b$ is already in CNF

$B \rightarrow a$ is already in CNF

(1) $S \rightarrow aB$

$S \rightarrow bB$ $[\because A \rightarrow b]$

(2) $A \rightarrow BS$

$A \rightarrow aS$ $[\because B \rightarrow a]$

(3) $B \rightarrow SA$

$B \rightarrow bSA$ $[\because S \rightarrow bB]$

Required GNF :

$S \rightarrow bB$

$A \rightarrow aS \mid b$

$B \rightarrow bBA \mid a$

UNIT - IV

(Total Marks : 28)

Q.7.(a) Design a non-deterministic PDA for :

$$L = \left\{ WW^R \;\middle|\; \begin{matrix} W \in (a/b)^* \\ W^R \text{is reverse of } W \end{matrix} \right.$$

Also explain STACK execution with valid string. **7M**

Ans. The given language L will accept the set of string W by the machine is combination of a's and b's and the string W^R is reverse of string W

Now let us assume that NPDA as

$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

where,

$Q = \{q_0, q_1, q_f\}$

$\Sigma = \{a, b\}$

$F = \{q_f\}$

The string either starts with a or start with b :

∴ Transition relation δ is defined as follows :

$\delta(q_0, a, Z_0) = (q_0, aZ_0)$ ....(1)

$\delta(q_0, b, Z_0) = (q_0, bZ_0)$ ....(2)

$\delta(q_0, a, a) = \{(q_0, aa), (q_1, \epsilon)\}$ ....(3)

$\delta(q_0, a, b) = (q_0, ab)$ ....(4)

$\delta(q_0, b, a) = (q_0, ba)$ ....(5)

$\delta(q_0, b, b) = \{(q_0, bb), (q_1, \epsilon)\}$ ....(6)

$\delta(q_1, a, a) = (q_1, \epsilon)$ ....(7)

$\delta(q_1, b, b) = (q_1, \epsilon)$ ....(8)

$\delta(q_0, \epsilon, Z_0) = (q_1, \epsilon)$ ....(9)

$\delta(q_1, \epsilon, Z_0) = (q_1, \epsilon)$ ....(10)

Now consider the string aabbaa which is in the from WW^R

where W = aab

Initial configuration $(q_0, aabbaa, Z_0)$

$(q_0, aabbaa, Z_0) \vdash (q_0, abbaa, aZ_0)$ ....(1)

$\vdash (q_0, bbaa, aa)$ ....(2)

$\vdash (q_0, baa, baa)$ ....(5)

$\vdash (q_1, aa, aa)$ ....(6)

$\vdash (q_1, a, a)$ ....(7)

$\vdash (q_1, \epsilon, Z_0)$ ....(7)

$\vdash (q_1, \epsilon, \epsilon)$

ϵ indicate that stack is empty, it is accepted by NPDA.

Q.7.(b) Design a PDA for given CFG :

$S \rightarrow XY$

$X \rightarrow AX|BX \mid a$

$Y \rightarrow YA \mid YB \mid a$

$A \rightarrow a$

$B \rightarrow b$ **7M**

Ans. **See P.4-25, Q.36.**

Q.8.(a) Design a PDA for :

$$L = \left\{ W \;\middle|\; \begin{matrix} W \in (a/b)^* \\ n_a(W) = n_b(W) \end{matrix} \right.$$

Where $n_a(W) \rightarrow$ Number of a's in W

and $n_b(W) \rightarrow$ Number of b's in W **7M**

Ans. **See P.4-16, Q.24.**

Q.8.(b) Convert the given PDA to CFG :

$\delta(q_0, a, z_0) \Rightarrow (q_0, xz_0)$

$\delta(q_0, a, x) \Rightarrow (q_0, xx)$

$\delta(q_0, b, x) \Rightarrow (q_1, \epsilon)$

$\delta(q_1, b, x) \Rightarrow (q_1, \epsilon)$

$\delta(q_1, \epsilon, z_0) \Rightarrow (q_1, \epsilon)$ **7M**

Ans. **See P.PS-61, Q.8(b).**

UNIT - V

(Total Marks : 26)

Q.9.(a) Design a Turing machine to perform $\frac{(n+1)}{2}$ where n is an uniary number **7M**

Ans. See P.5-43, Q.50.

[Note : Question is different but the answer is same as these are uniary number].

Q.9.(b) Explain different types of Turing machine. **6M**

Ans. See P.5-21, Q.19.

Q.10.(a) Design a Turing machine that has

INPUT : → #W# and generates

OUTPUT : → #W#W

where $W = \{a, b\}^*$ **7M**

Ans. See P.PS-42, Q.10(a).

Q.11.(b) Write short notes on LBA. **6M**

Ans. See P.5-34, Q.40.

UNIT - VI

(Total Marks : 26)

Q.10.(b) Explain various properties of recursively enumerable language. **6M**

Ans. See P.6-7, 8, 9, Q.2, 3, 4, 5, 6.

Q.11.(a) What do you mean by primitive recursive functions? Show that

ADD (x, y) and SUB (x, y) are primitive recursive **7M**

Ans. See P.6-28, 29, 31, Q.43, 45, 51.

Q.12.(a) What is Ackermann's function? Calculate

A(1, 1) A(1, 2) A(2, 1) **7M**

Ans. See P.6-38, Q.68.

Q.12.(b) What is the significance of PCP? Solve the following using PCP : **6M**

i	wi	xi
1	0	000
2	01000	01
3	01	1

Ans. See P.6-15, Q.18.

In this case PCP has a solution.

Let $m = 4$, $i_1 = 2$, $i_2 = 1$, $i_3 = 1$ and $i_4 = 3$

Then

$\omega_2 \omega_1 \omega_1 \omega_3 = x_2, x_1 x_1 x_3$

01 0 0 0 0 0 0 1 = 0 1 0 0 0 0 0 0 1

which is the solution of instance of PCP

NOTES

SUMMER EXAMINATION - 2018
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	26 M
Unit - V	28 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

SUMMER EXAM. - 2018
QUESTION PAPER

Q.1.(a) Explain the following in detail with example

(i) Type - 0 Grammar (ii) Type - 1 Grammar

(iii) Type - 2 Grammar (iv) Type - 3 Grammar **6M**

(b) What is string, prefix, proper prefix and proper suffix with examples **4M**

(c) Write short note on pigeonhole principle. State its applications **3M**

OR

Q.2.(a) Consider a relation R = {(a, b), (a, a), (b, b), (c, c), (d, d), (b, a)} on set A = {a, b, c, d}. Is relation R an equivalence relation. If so find equivalence classes **6M**

(b) Prove the following using method of induction.

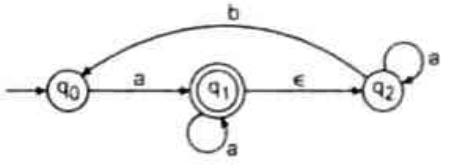
$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$ **7M**

Q.3.(a) Construct DFA which can accept the string those can have odd number of 0's and any number of 1's **7M**

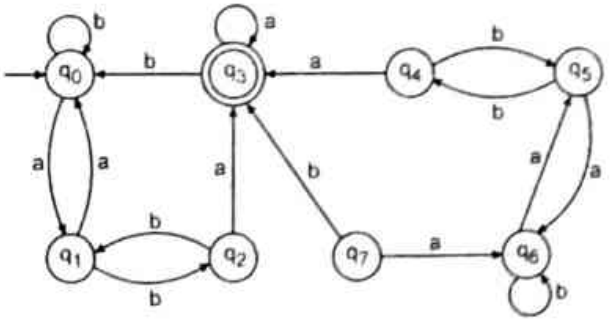
(b) Design a Moore machine for binary input sequence, if it ends in 101, output is 'A'. If it ends in 110 output is 'B'. Otherwise C **6M**

OR

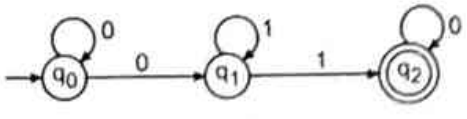
Q.4.(a) Convert the following NFA to equivalent DFA **6M**



(b) Minimize the following Automata. **7M**



Q.5.(a) Construct the regular expression for the automata given **6M**



(b) Construct context free Grammar for following. Where $(n \geq 0, m \geq 0, k \geq 0)$ **8M**

(i) $L = \{a^n b^m c^k \mid n = m \text{ or } m \leq k\}$

(ii) $L = \{a^n b^m \mid 2n \leq m \leq 3n\}$

(iii) $L = \{a^n b^m c^m d^n \mid m, n \geq 1\}$

(iv) $L = \{a^n b^{2n} \mid n \geq 1\}$.

OR

Q.6.(a) Remove all unit productions, all useless production all E-productions from the grammar given below **7M**

$S \rightarrow aA|aBB$

$A \rightarrow aaA|\epsilon$

$B \rightarrow bB|bbC$

$C \rightarrow B$

(b) Convert the following grammar into Greibach normal form **7M**

$S \rightarrow aSa|bBb$

$B \rightarrow abB|aaAa$

$A \rightarrow Aa|a$

Q.7.(a) Construct PDA for $L = \{a^{2n} b^n \mid n > 0\}$ **6M**

(b) Show that $L = \{a^n b^j \mid n \leq j^2\}$ is not a context free language. **3M**

(c) Explain the non-deterministic push down automata in detail. **4M**

OR

Q.8.(a) Convert the following CFG into PDA **7M**

$E \rightarrow aAB|d$

$A \rightarrow BA|a$

$B \rightarrow Ead|C$

(b) Explain the model of PDA and its acceptance by stack and acceptance by final state **6M**

Q.9.(a) Explain types of turing machine **5M**

(b) Construct a turing machine for **9M**

$f(a, b) = a - b$ if $a \geq b$

$= 0$ if $a < b$

OR

Q.10.(a) Explain the model of linear bounded automata **6M**

VBD

(b) Construct the turing machine for **9M**

$L = \{a^n b^n a^n b^n \mid n \geq 0\}$

Q.11.(a) Write a short note on **6M**

(i) Post correspondence problem

(ii) Primitive recursive function

(b) Define Ackermann's function

Compute A(1, 1), A(2, 1), A(2, 2) **7M**

OR

Q.12.(a) Define decidability and undecidability **4M**

(b) Explain the properties of recursively enumerable language. Give relation between recursive and recursive enumerable language **6M**

(c) Explain halting problem of turing machine in detail. **3M**

SUMMER EXAM. - 2018

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Explain the following in detail with example

(i) Type - 0 Grammar (ii) Type - 1 Grammar

(iii) Type - 2 Grammar (iv) Type - 3 Grammar **6M**

Ans. **See P.1-11, Q.22.**

Q.1.(b) What is string, prefix, proper prefix and proper suffix with examples **4M**

Ans. **See P.1-4, Q.2.**

Q.1.(c) Write short note on pigeonhole principle. State its applications **3M**

Ans. **See P.1-10, Q.16.**

Q.2.(a) Consider a relation $R = \{(a, b), (a, a), (b, b), (c, c), (d, d), (b, a)\}$ on set $A = \{a, b, c, d\}$. Is relation R an equivalence relation. If so find equivalence classes **6M**

Ans. **See P.PS-137, Q.1(c).**

Q.2.(b) Prove the following using method of induction

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$ **7M**

Ans. **See P.1-8, Q.11.**

UNIT - II

(Total Marks : 26)

Q.3.(a) Construct DFA which can accept the string those can have odd number of 0's and any number of 1's. **7M**

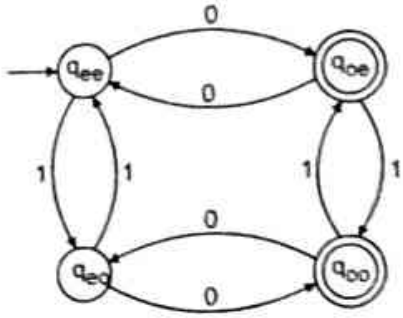
Ans. q_{ee} → even no. of 0's and even no. of 1's

q_{oe} → odd no. of 0's and even no. of 1's

q_{eo} → even no. of 0's and odd no. of 1's

q_{oo} → odd no. of 0's and odd no. of 1's

DFA : odd no. of 0's and any no. of 1's.



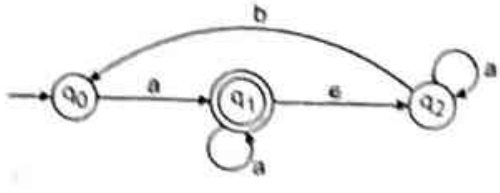
Transition table :

State \ Σ	0	1
→ q_{ee}	q_{oe}	q_{eo}
* q_{oe}	q_{ee}	q_{oo}
q_{eo}	q_{oo}	q_{ee}
* q_{oo}	q_{eo}	q_{oe}

Q.3.(b) Design a Moore machine for binary input sequence, if it ends in 101, output is 'A' if it ends in 110 output is 'B' Otherwise C **6M**

Ans. **See P.2-55, Q.77.**

Q.4.(a) Convert the following NFA to equivalent DFA. **6M**



Ans. Find ε closure :

$\hat{\delta}(q_0, \epsilon) = \epsilon$ - closure $(q_0) = q_0$

$\hat{\delta}(q_1, \epsilon) = \epsilon$ - closure $(q_1) = q_1, q_2$

$\hat{\delta}(q_2, \epsilon) = \epsilon$ - closure $(q_2) = q_2$

Transition of NFA without ϵ moves :

(1) $\hat{\delta}(q_0, a) = \epsilon$ - closure $(\delta(\hat{\delta}(q_0, \epsilon), a))$

$= \epsilon$ - closure $(\delta(q_0, a))$

$= \epsilon$ - closure (q_1)

$= q_1, q_2$

$\hat{\delta}(q_0, b) = \epsilon$ - closure $(\delta(\hat{\delta}(q_0, \epsilon), b))$

$= \epsilon$ - closure $(\delta(q_0, b))$

$= \epsilon$ - closure (ϵ)

$= \phi$

(2) $\hat{\delta}(q_1, a) = \epsilon$ - closure $(\delta(\hat{\delta}(q_1, \epsilon), a))$

$= \epsilon$ - closure $(\delta(q_1, q_2), a))$

$= \epsilon$ - closure $(\delta(q_1, a) \cup \delta(q_2, a))$

$= \epsilon$ - closure (q_1, q_2)

$= q_1, q_2$

$\hat{\delta}(q_1, b) = \epsilon$ - closure $(\delta(\hat{\delta}(q_1, \epsilon), b))$

$= \epsilon$ - closure $(\delta(q_1, q_2), b)$

$= \epsilon$ - closure $(\delta(q_1, b) \cup \delta(q_2, b))$

$= \epsilon$ - closure (q_0)

$= q_0$

(3) $\hat{\delta}(q_2, a) = \epsilon$ - closure $(\delta(\hat{\delta}(q_2, \epsilon), a))$

$= \epsilon$ - closure $(\delta(q_2, a))$

$= \epsilon$ - closure (q_2)

$= q_2$

$\hat{\delta}(q_2, b) = \epsilon$ - closure $(\delta(\hat{\delta}(q_2, \epsilon), b))$

$= \epsilon$ - closure $(\delta(q_2, b))$

$= \epsilon$ - closure (q_0)

$= q_0$

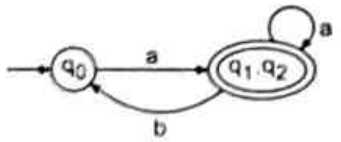
Transition Table for NFA :

State \ Σ	a	b
$\rightarrow q_0$	q_1, q_2	ϕ
$* q_1$	q_1, q_2	q_0
q_2	q_2	q_0

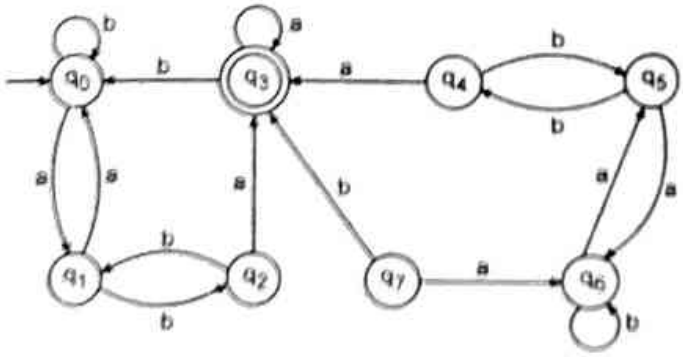
Transition Table for DFA :

State \ Σ	a	b
$\rightarrow q_0$	q_1, q_2	q
$* q_1, q_2$	q_1, q_2	q_0

Transition Diagram :



Q.4.(b) Minimize the following Automata **7M**

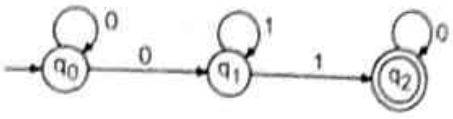


Ans. See P.2-41, Q.5B.

UNIT - III

(Total Marks : 28)

Q.5.(a) Construct the regular expression for the automata given. **6M**



Ans. Equations for all state

$q_0 = q_0 0 + \epsilon$

$q_1 = q_0 0 + 1q_1$

$q_2 = q_1 1 + q_2 0$

$q_0 = q_0 0 + \epsilon$

By Arden's theorem, we get

$q_0 = \epsilon 0^*$

$q_0 = 0^*$

$q_1 = 00^* + 1q1$

By Arden's theorem, we get

$q_1 = (00^*)1^*$

$q_1 = 00^*1^*$

$q_2 = 00^*1^*1 + q_2 0$

By Arden's theorem, we get

$q_2 = 00^*1^*10^*$

$R.E. = \underline{00}^*\underline{1}^*\underline{10}^*$

$= 0^*1^*0^*$

Q.5.(b) Construct context free grammar for following. Where $(n \geq 0, m \geq 0, k \geq 0)$. **8M**

(i) $L = \{a^n b^m c^k \mid n = m \text{ or } m \leq k\}$.

(ii) $L = \{a^n b^m \mid 2n \leq m \leq 3n\}$.

(iii) $L = \{a^n b^m c^m d^n \mid m, n \geq 1\}$.

(iv) $L = \{a^n b^{3n} \mid n \geq 1\}$.

Ans. See P.3 n.3, Q.1.10.

(i) $L = \{a^n b^m c^k \mid n = m \text{ or } m \leq k\}$.

It is generated by the grammar

$S \rightarrow aSbA \mid aAc$

$A \rightarrow cA \mid c$

(ii) $L = \{a^n b^m \mid 2n \leq m \leq 3n\}$.

Consider the language

$L = \{a^n b^m \mid 2n \leq m \leq 3n\}$

$L = L_1 \cup L_2$

Where

$L_1 = \{a^n b^m \mid 2n \leq m\}$

$L_2 = \{a^n b^m \mid m \leq 3n\}$

We construct L_1 first. The set of production for language L_1 are as follows.

$S \rightarrow AB$

$A \rightarrow aAbb \mid \epsilon$

$B \rightarrow aBb \mid b$

$P \rightarrow b$

Now we construct CFG for language L_2. The set of production for language L_2 are

$L_2 = \{a^n b^m \mid m \leq 3n\}$

$S \rightarrow C$

$C \rightarrow DEEF$

$D \rightarrow FDEEE \mid a$

$F \rightarrow a$

$E \rightarrow b$

Now, combining the set of production for language L_1 and L_2 we get CFG

$S \rightarrow AB \mid C$

$A \rightarrow aAbb \mid \epsilon$

$B \rightarrow aaBbP \mid \in$

$P \rightarrow b$

$C \rightarrow DEEE$

$D \rightarrow FDEEF \mid a$

$F \rightarrow a$

$E \rightarrow b$

(iv) $L = \{a^n b^{2n} \mid n \geq 1\}$.

CFG are

$S \rightarrow AB$

$A \rightarrow aA \mid a$

$B \rightarrow bbB \mid \in$

Q.6.(a) Remove all unit productions, all useless productions all $\in$-productions from the grammar given below **7M**

$S \rightarrow aA|aBB$

$A \rightarrow aaA| \in$

$B \rightarrow bBbbC$

$C \rightarrow B$

Ans. See P.3-79, Q.156.

Q.6.(b) Convert the following grammar into Greibach normal form **7M**

$S \rightarrow aSa|bBb$

$B \rightarrow abB|aaA|a$

$A \rightarrow Aa|a$

Ans. See P.PS-87, Q.4(a).

UNIT - IV

(Total Marks : 26)

Q.7.(a) Construct PDA for $L = \{a^{2n}b^n \mid n > 0\}$ **6M**

Ans. See P.4-17, Q.26.

Q.7.(b) Show that $L = \{a^n b^j n \leq j^2\}$ is not a context free language. **5M**

Ans. $L = \{a^n b^j n \leq j^2\}$

Assume for contradiction that L is a CFL. We apply the pumping lemma. Let m be the parameter of the pumping lemma we choose to pump the string $a^{m^2}b^m \in L$. We have that $a^{m^2}b^m = uvxyz$, with $|vxy| \leq m$, and $|vy| \geq 1$.

We examine all possible cases for the position of string vxy. First we note that the string v cannot span simultaneously both a^{m^2} and b^m, since if we pump up v (repeat v), the resulting string is not in the language (a's are mixed with b's). Therefore, it must be that v is either within a^{m^2} or within b^m. The same holds for y. Below are the rest of the cases. Notice that in all cases we obtain a contradiction and therefore for language is not context free. The most important case is (i).

(i) v is within a^{m^2} and y is within b^m

We have that $v = a^k$ and $y = b^l$, with $1 \leq k + l \leq m$

(Since $|vxy| \leq m$ and $|vy| \geq 1$)

Consider the case where $i \geq 1$ from the pumping lemma we have that $uv^i xy^i z \in L$. Therefore, $a^{m^2-k}b^{m+l} \in L$, and thus, it must be that $m^2 - k \leq (m-l)^2$. However, this is impossible since

$(m-l)^2 \leq (m-1)^2$ (since $l \geq 1$)

$= m^2 - 2m + 1$

$< m^2 - k$ (since $k \leq m$)

Consider now the case where $i = 0$. It must be that $k \geq 1$ (since $k + l \geq 1$) from the pumping lemma we have that $uv^2xy^2z \in L$. Therefore, $a^{m^2+kk}b^m \in L$, which is impossible since $m^2 + k \geq m^2$.

(ii) v and y are within a^{m^2}

If we pump up v and y (repeat them) we obtain a string of the form $a^{m^2+k}b^m$, with $k \geq 1$, which obviously is not in the language.

(iii) v and y are within b^m

If we pump down v and y (remove them) we obtain a string of the form $a^{m^2}b^{m-k}$, with $k \geq 1$, which obviously is not in the language.

Q.7.(c) Explain the non-deterministic push down automata in detail. **4M**

Ans. See P.4-6, Q.6.

Q.8.(a) Convert the following CFG into PDA **7M**

$E \rightarrow aAB|d$

$A \rightarrow BA|a$

$B \rightarrow Ed|c$

Ans. **See P.PS-126, Q.8(a)**

Q.8.(b) Explain the model of PDA and its acceptance by stack and acceptance by final state **6M**

Ans. **See P.4-5, 18, Q.2, 28.**

UNIT - V

(Total Marks : 28)

Q.9.(a) Explain types of turing machine **6M**

Ans. **See P.5-21, Q.19.**

Q.9.(b) Construct a turing machine for **7M**

$f(a, b) = a - b$ if $a \geq b$

$= 0$ if $a < b$

Ans. **See P.5-40, Q.43.**

Q.10.(a) Explain the model of linear bounded automata **6M**

Ans. **See P.5-34, Q.40.**

Q.10.(b) Construct the Turing machine for **9M**

$L = \{a^n b^n a^n b^n \mid n \geq 0\}$.

Ans. $L = \{a^n b^n a^n b^n \mid n \geq 0\}$

Transition table for T.M. :

Present state	a	b	X	Y	Z	B
$\rightarrow q_0$	q_1, X, R			q_0, Y, R	q_0, Z, R	q_0, B, R
q_1	q_1, a, R	q_2, Y, R		q_1, Y, R		
q_2	q_3, Z, R	q_2, b, R			q_2, Z, R	
q_3	q_3, a, R	q_4, Y, L		q_3, Y, R		
q_4	q_4, a, L	q_4, b, L	q_0, X, R	q_4, Y, L	q_4, Z, L	
q_5						

Simulation :

B c a b b a a b b B

$\rightarrow q_0$

X a b b a a b b B

q_1

X a b b a a b b B

q_1

X a Y b a a b b B

q_2

X a Y b a a b b B

q_3

X a Y b Z a b b B

q_3

X a Y b Z a b b B

q_3

X a Y b Z a Y b B

q_4

X a Y b Z a Y b B

q_0

X X Y b Z a Y b B

q_1

X X Y Y Z a Y b B

q_2

X X Y Y Z Z Y b B

q_0

X X Y Y Z Z Y Y B

q_4

X X Y Y Z Z Y Y B

q_1

X X Y Y Z Z Y Y B

q_5

UNIT - VI

(Total Marks : 26)

Q.11.(a) Write a short note on 6M

(i) Post correspondence problem

Ans. See P.6-15, Q.18.

(ii) Primitive recursive function

Ans. See P.6-28, Q.42, 44.

Q.11.(b) Define Ackermann's function

Compute A(1, 1), A(2, 1), A(2, 2). 7M

Ans. See P.6-38, Q.68.

Q.12.(a) Define decidability and undecidability. 4M

Ans. See P.6-13, 14, Q.15(a), 16(a).

Q.12.(b) Explain the properties of recursively enumerable language. Give relation between recursive and recursive enumerable language. 6M

Ans. See P.6-7, 8, 9, Q.2, 4, 5, 6, 1.

Q.12.(c) Explain halting problem of turing machine in detail. 3M

Ans. See P.6-19, Q.25.

WINTER EXAMINATION - 2017
B.E. IV SEM. (COMPUTER TECHNOLOGY) (CBS)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	28 M
Unit - II	26 M
Unit - III	26 M
Unit - IV	14 M
Unit - V	31 M
Unit - VI	35 M
Questions covered in VBD	160 M (100 %)

WINTER EXAM. - 2017
QUESTION PAPER

Q.1.(a) Define the following terms: Diagonalization, countability, decidability, relations and language **8M**

(b) If R = {(1, 1), (2, 2), (3, 1), (1, 4), (2, 4)} on a set A = {1, 2, 3, 4}. Obtain R^* and the reflexive transitive closure of R. **6M**

OR

Q.2.(a) Prove the following by principle of induction

(1) $1 + 2 + 3 + \ldots + n = \frac{n(n+1)}{2}$

(2) $1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n(n+1)(2n+1)}{6}$ **8M**

(b) Explain Pigeonhole principle with example **6M**

Q.3.(a) Construct minimized DFA equivalent to finite automata given below:

Where p is initial state q, s are final states. **8M**

	0	1
→ p	q, s	q
* q	r	q, r
r	s	p
*s	ϕ	p

(b) Differentiate between Moore machine and Mealy machine with example **5M**

OR

Q.4.(a) Design DFA which accepts binary string which starts with '0' and ends with '1' **6M**

(b) Design Mealy machine for incrementing given binary number by 1. **7M**

Q.5.(a) What are the steps for converting RLG into its LLG? Convert the following RLG into LLG:

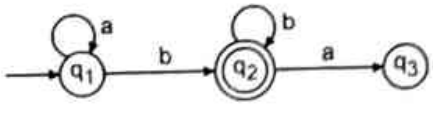
S → 0A | 1B

A → 0C | 1A | 0

B → 1B | 1A | 1

C → 0 | 0A **7M**

(b) Convert given FA into FE using Arden's theorem: **6M**



OR

Q.6.(a) Explain pumping Lemma and show that the language is non-regular using pumping Lemma.

$L = \{a^i b^i / i >= 1\}$ **7M**

(b) Obtain CNF and GNF for the grammar: **6M**

S → ABb | a

A → aaA | B

B → bAb

Q.7.(a) Construct PDA for the language: **7M**

$L = \{a^n b^{n+m} c^m / n, m >= 0\}$

(b) Construct PDA for the language: **7M**

$L = \{0^n 1^{2n} / n >= 1\}$

OR

Q.8.(a) Give mathematical definition of turing machine. Discuss various types of turing machines **7M**

(b) Design turing machine for the language: **7M**

$L = \{a^n b^n c^n / n >= 1\}$

Q.9. Design turing machine for the multiplication of two binary numbers. **13M**

OR

Q.10.(a) Define Ackermann's function and compute A (1, 1), A (2, 1) and A (2, 2). **7M**

(b) What is post correspondence problem? Prove that PCP with two list: **6M**

X = (01, 1, 1) and

Y = $(01^2, 10, 1^2)$ has no solution.

Q.11.(a) Define primitive recursive function. Show that the function f (x, y) = x * y is primitive recursive. **7M**

(b) "Every function is primitive recursive class is turing computable" Comment on it. **6M**

OR

Q.12. Write short notes on:

(a) Universal turning machine **4M**

(b) Recursively enumerable and recursive language **5M**

(c) Halting problem of turing machine **4M**

VBD

WINTER EXAM. - 2017

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 28)

Q.1.(a) Define the following terms : Diagonalization, countability, decidability, relations and language **8M**

Ans. See P.1-4, 5, 10, Q.3, 5, 18.

Q.1.(b) If R = {(1, 1), (2, 2), (3, 1), (1, 4), (2, 4)} on a set A = {1, 2, 3, 4} Obtain R^* and the reflexive transitive closure of R. **6M**

Ans. See P.PS-31, Q.1(b).

Q.2.(a) Prove the following by principle of induction :

(1) $1 + 2 + 3 + \ldots + n = \frac{n(n+1)}{2}$

(2) $1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n(n+1)(2n+1)}{6}$ **8M**

Ans. See P.1-7, 8, Q.10, 11.

Q.2.(b) Explain Pigeonhole principle with example **6M**

Ans. See P.1-10, Q.16.

UNIT - II

(Total Marks : 26)

Q.3.(a) Construct minimized DFA equivalent to finite automata given below where p is initial state, q, s are final states. **8M**

	0	1
→ p	q, s	q
* q	r	q, r
r	s	p
*s	ϕ	p

Ans. See P.2-17, Q.20.

Q.3.(b) Differentiate between Mouse machine and Mealy machine with example **8M**

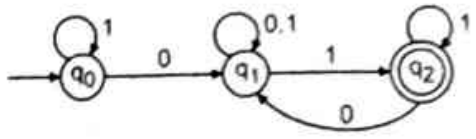
Ans. See P.2-49, Q.64.

Q.4.(a) Design DFA which accepts binary string which starts with '0' and ends with '1'. **6M**

Ans. R.E. is

$0(0 + 1)^* 1$

NFA for R.E. is



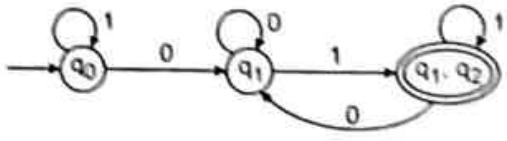
T.T. for NFA is

	0	1
→ q_0	q_1	q_0
q_1	q_1	q_1, q_2
* q_2	q_1	q_2

∴ T.T. for D.F.A is

	0	1
q_0	q_1	q_0
q_1	q_1	q_1, q_2
* $\{q_1 q_2\}$	q_1	q_1, q_2

∴ DFA is



Q.4.(b) Design Mealy machine for incrementing given binary number by 1. **7M**

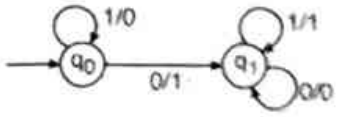
Ans. In this, we will read the binary number from LSB one bit at a time and we will replace each 1 by 0 until we get first 0. Once we get first 0 we will replace it by 1 and then keep remaining bits as it is.

e.g. 110011

\+ 1

110100

Mealy machine :



T.T. for Mealy machine :

Present state	Next state			
	Input 0		Input 1	
	State	Output	State	Output
$\rightarrow q_0$	q_1	1	q_0	0
q_1	q_1	0	q_1	1

UNIT - III

(Total Marks : 26)

Q.5.(a) What are the steps for converting RLG into its LLG? Convert the following RLG into LLG

$S \rightarrow 0A \mid 1B$

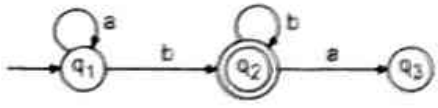
$A \rightarrow 0C \mid 1A \mid 0$

$B \rightarrow 1B \mid 1A \mid 1$

$C \rightarrow 0 \mid 0A$ **7M**

Ans. See P.3-41, 39, Q.51, 48.

Q.5.(b) Convert given FA into FE using Arden's theorem : **6M**



Ans. Equations for states are

$q_1 = q_1a + \epsilon$ ...(1)

$q_2 = q_1b + q_2b$ ...(2)

$q_3 = q_2a$ ...(3)

From (1),

By Arden's theorem

$q_1 = \epsilon a^* = a^*$ ...(4)

Put value of q_1 in equation (2), we get

$q_2 = a^*b + q_2b$

By Arden's theorem, we get

$q_2 = a^*bb^*$

Regular expression is

$q_2 = a^*bb^*$

Q.6.(a) Explain pumping Lemma and show that the language is non-regular using pumping Lemma

$L = \{a^ib^i / i >= 1\}$ **7M**

Ans. See P.3-54, Q.84.

Q.6.(b) Obtain CNF and GNF for the grammar **6M**

$S \rightarrow ABb \mid a$

$A \rightarrow aaA \mid B$

$B \rightarrow bAb$

Ans. See P.3-95, Q.187.

UNIT - IV

(Total Marks : 14)

Q.7.(a) Construct PDA for the language **7M**

$L = \{a^nb^{n+m}c^m / n, m >= 0\}$

Ans. See P.4-12, Q.16.

Q.7.(b) Construct PDA for the language **7M**

$L = \{(0^n1)^{2n} / n >= 1\}$

Ans. See P.4-9, Q.12.

UNIT - V

(Total Marks : 31)

Q.8.(a) Give mathematical definition of turing machine. Discuss various types of turing machines **7M**

Ans. See P.5-7, 21, Q.1, 19.

Q.8.(b) Design turing machine for the language **7M**

$L = \{a^nb^nc^n / n >= 1\}$

Ans. See P.5-15, Q.10

Q.9. Design turing machine for the multiplication of two binary numbers **15M**

Ans. See P.5-41, Q.49.

VBD

Q.12. Write short notes on

(a) Universal turning machine **4M**

Ans. See P.5-27, Q.26(A).

UNIT - VI

(Total Marks : 35)

Q.10.(a) Define Ackermann's function and compute A (1, 1), A (2, 1) and A (2, 2) **7M**

Ans. See P.6-38, Q.68.

Q.10.(b) What is post correspondence problem? Prove that PCP with two list **4M**

X = (01, 1, 1) and

Y = (01^2, 10, 1^2) has no solution

Ans. See P.6-15, 17, Q.18, 21(a).

Q.11.(a) Define primitive recursive function. Show that the function f (x, y) = x * y is primitive recursive **7M**

Ans. See P.6-28, 29, Q.42, 46.

Q.11.(b) "Every function is primitive recursive class is turing computable" Comment on it **6M**

Ans. See P.6-31, Q.52.

Q.12. Write short notes on

(b) Recursively enumerable and recursive language **5M**

Ans. See P.6-7, 8, 9, Q.1, 2, 4, 5, 6.

(c) Halting problem of turing machine **4M**

Ans. See P.6-19, Q.25.

NOTES

SUMMER EXAMINATION - 2018
B.E. IV SEM. (COMPUTER TECHNOLOGY) (CBS)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	28 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

SUMMER EXAM. - 2018
QUESTION PAPER

Q.1.(a) Prove the following by using principle of mathematical induction

(i) $1 + 2 + \quad + n = \frac{n^2 + n}{2}$

(ii) $1^2 + 2^2 + \quad + n^2 = \frac{n(n+1)(2n+1)}{6}$ **6M**

(b) Write short note on pigeon hole principle **3M**

(c) Define and explain transitive closure and reflexive transitive closure with example **4M**

OR

Q.2.(a) Consider a relation R = {(a, b), (a, a), (b, b), (c, c), (d, d), (b, a)} on set A = {a, b, c, d}, is relation R a equivalence relation? If so find equivalence classes **8M**

(b) Explain chomsky hierarchy of grammar in detail **8M**

Q.3.(a) Design DFA to accept the following strings over Σ = {a, b} **6M**

(a) String that have 'ab' at most two times

(b) String that do not have 'aba' as substring

(b) Minimize the given DFA **7M**

Σ / Q	0	1
→ A	B	D
B	A	D
C	B	E
D*	F	F
E	D	D
F*	F	F

OR

Q.4.(a) Differentiate between moore machine and mealy machine **8M**

(b) Design DFA which accepts the set of strings that either start with '01' or end with '10' over Σ = {0, 1} **5M**

(c) Design moore machine for binary input sequence such as it has a substring '101' the machine output 'A', if it has a substring '110', the machine output is 'B' otherwise output is 'C' **5M**

Q.5.(a) Convert following CFG into equivalent CNF form

S → AAD

A → aSS/1 **8M**

(b) Construct NFA with and without ∈ -transition equivalent to the following regular expression

$R_E = 10 + (0 + 11)0^*1$ **6M**

OR

Q.6.(a) Give the statement of pumping lemma and show that a language L = {ww/w is in {0, 1}*} is a non-regular language **7M**

(b) Convert the given CFG into equivalent CNF **4M**

S → abAB

A → bAB/∈

B → BAa/A/∈

(c) What is ambiguous grammar? Explain it with suitable example **3M**

Q.7.(a) Construct PDA equivalent to the following grammar

S → aAA

A → aS/bS/a

and show acceptance/rejection for the string 'aaabaaaa' **7M**

(b) Construct PDA for the following languages **7M**

(i) $L = \{a^m b^n c^p d^q$ / where $m + n = p + q\}$

(ii) $L = \{a^n b^m$ / where $n \le m + 3\}$

OR

Q.8.(a) Show that context free language are closed under the property of closure and union **7M**

(b) Construct PDA for the following language **7M**

$L = \{a^n b^{2n}$ / $n > 0\}$

Q.9.(a) Design TM to find 2's complement of a given number **7M**

(b) Design TM to recognize all palindrome over Σ = {a, b} **6M**

OR

Q.10.(a) Explain in detail about Universal TM **6M**

(b) Design TM to accept following language

$L = \{a^n b^n c^n$ / where $n \ge 1\}$ **7M**

Q.11.(a) Prove the following statements

(i) If L and $\bar{L}$ are both recursive enumerable languages then both L and $\bar{L}$ are recursive

(ii) Union of two recursively enumerable language is a recursive languages **8M**

(b) Give Ackermann's function and obtain solution for A (2, 1) and A (3, 3) **6M**

OR

Q.12.(a) Show that f(x, y) = x + y is a primitive recursive function **6M**

(b) What is PCP and modified PCP? Give one solution for the PCP for following sequences shown in table **7M**

i	A	B
1	0	000
2	010001	01
3	01	1

SUMMER EXAM. - 2018
SOLUTION
(Unit-Wise)

UNIT - I
(Total Marks : 26)

Q.1.(a) Prove the following by using principle of mathematical induction

(i) $1 + 2 + \ldots + n = \frac{n^2 + n}{2}$

(ii) $1^2 + 2^2 + \ldots + n^2 = \frac{n(n+1)(2n+1)}{6}$ **6M**

Ans. See P.1-7, 8, Q.10, 11.

Q.1.(b) Write short note on pigeon hole principle **5M**

Ans. See P.1-10, Q.16.

Q.1.(c) Define and explain transitive closure and reflexive transitive closure with example **4M**

Ans. See P.1-5, Q.4.

Q.2.(a) Consider a relation R = {(a, b), (a, a), (b, b), (c, c), (d, d), (b, d)} on set A = {a, b, c, d} is relation R a equivalence relation? If so find equivalence classes **5M**

Ans. See P.Ps-13*, Q.1(c).

Q.2.(b) Explain chomsky hierarchy of grammar in detail **8M**

Ans. See P.1-11, Q.22.

UNIT - II
(Total Marks : 26)

Q.3.(a) Design DFA to accept the following strings over Σ = {a, b} **6M**

(a) String that have 'ab' at most two times

(b) String that do not have 'aba' as substring

Ans. See P.PS-1.28, Q.3(b).

Q.3.(b) Minimize the given DFA **7M**

Σ / Q	0	1
→ A	B	D
B	A	D
C	B	E
D*	F	F
E	D	D
F*	F	F

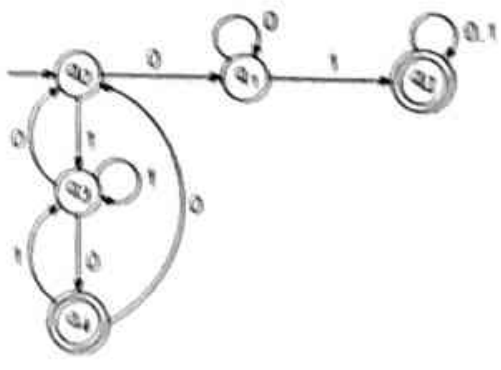
Ans. See P.PS-1.29, Q.4(a).

Q.4.(a) Differentiate between mealy machine and moore machine **5M**

Ans. See P.2-49, Q.64.

Q.4.(b) Design DFA which accepts the set of strings that either starts with '01' or end with '10' over Σ = {0, 1}. **8M**

Ans. DFA : Start with '01' or end with '10', Σ = {0, 1}



Transition table for DFA :

state \ Σ	0	1
→ q_0	q_1	q_3
q_1	q_1	q_2
* q_2	q_2	q_2
q_3	q_4	q_3
* q_4	q_0	q_3

Q.4.(c) Design moore machine for binary input sequence such as it has a substring '101' the machine output 'A', if it has a substring '110', the machine output is 'B' otherwise output is 'C' **5M**

Ans. See P.2-55, Q.77.

UNIT - III
(Total Marks : 28)

Q.5.(a) Convert following CFG into equivalent GNF form

$S \rightarrow AA/0$

$A \rightarrow SS/1$ **8M**

Ans. See P.3-89, Q.178.

Q.5.(b) Construct *NFA* with and without ∈ -transition equivalent to the following regular expression

$R_E = 10 + (0 + 11)\,0^*1$ **6M**

Ans. See P.3-18, Q.22.

Q.6.(a) Give the statement of pumping lemma and show that a language $L = \{ww/w \text{ is in } (0, 1)^*\}$ is a non-regular language **7M**

Ans. See P.3-55, Q.87.

Q.6.(b) Convert the given CFG into equivalent *CNF*

$S \rightarrow abAB$

$A \rightarrow bAB/\in$

$B \rightarrow BAa/A/a$ **4M**

Ans. See P.PS-141, Q.5(b).

Q.6.(c) What is ambiguous grammar ? Explain it with suitable example **3M**

Ans. See P.3-66, Q.120.

UNIT - IV
(Total Marks : 28)

Q.7.(a) Construct PDA equivalent to the following grammar

$S \rightarrow aAA$

$A \rightarrow aS/bS/a$

and show acceptance/rejection for the string 'aaabaaaaa' **7M**

Ans. See P.PS-132, Q.7(b).

Q.7.(b) Construct PDA for the following languages **7M**

(i) $L = \{a^m b^n c^r d^q / \text{where } m + n = p + q\}$

(ii) $L = \{a^n b^m / \text{where } n \le m + 3\}$

Ans. See P.PS-143, Q.7(a).

Q.8.(a) Show that context free language are closed under the property of closure and union **7M**

Ans. See P.PS-144, Q.7(b).

Q.8.(b) Construct PDA for the following language **7M**

$L = \{a^n b^{2n} / n > 0\}$

Ans. See P.4-9, Q.12.

UNIT - V
(Total Marks : 26)

Q.9.(a) Design TM to find 2's complement of a given number **7M**

Ans. See P.5-37, Q.45.

Q.9.(b) Design TM to recognize all palindrome over $\Sigma = \{a, b\}$ **6M**

Ans. See P.5-18, Q.15.

Q.10.(a) Explain in detail about Universal TM **6M**

Ans. See P.5-27, Q.26(a).

Q.10.(b) Design TM to accept following language

$L = \{a^n b^n c^n / \text{where } n \ge 1\}$ **7M**

Ans. See P.5-15, Q.10.

VBD

UNIT - VI

(Total Marks : 26)

Q.11.(a) Prove the following statements

(i) If L and $\bar{L}$ are both recursive enumerable language then both L and $\bar{L}$ are recursive

(ii) Union of two recursively enumerable languages is a recursive languages **6M**

Ans. **See P.6-8, Q.4, 5.**

Q.11.(b) Give Ackermann's function and obtain solution for A (2, 5) and A (3, 3) **6M**

Ans. **See P.PS-146, Q.11(c).**

Q.12.(a) Show that f(x, y) = x + y is a primitive recursive function **6M**

Ans. **See P.6-29, Q.45.**

Q.12.(b) What is PCP and modified PCP? Give one solution for the PCP for following sequences shown in table **7M**

i	A	B
1	0	000
2	010001	01
3	01	1

Ans. **See P.PS-133, 6-15, 18, Q.12(b), 18, 23.**

NOTES

WINTER EXAMINATION - 2018
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	28 M
Unit - III	26 M
Unit - IV	28 M
Unit - V	30 M
Unit - VI	22 M
Questions covered in VBD	160 M (100 %)

WINTER EXAM.-2018
QUESTION PAPER

Q.1.(a) Explain closure of a relation. Find R* for

R = {(1, 1) (1, 2) (2, 1) (2, 3) (3, 2)} **3M**

(b) Prove the following relation using principle of induction : **8M**

(i) $1.2.3 + 2.3.4 + \ldots + n(n+1)(n+2)$

$$= \frac{n(n+1)(n+2)(n+3)}{4}$$

(ii) $1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n(n+1)(2n+1)}{6}$

(c) What is countability? Explain **2M**

OR

Q.2.(a) Describe the concept of Pigeon - hole principle with example. **5M**

(b) Define the following any four : **8M**

(i) Transitive closure.

(ii) Reflexive transitive closure.

(iii) Prefix of string.

(iv) Suffix of string.

(v) Substring.

(vi) Subsequence.

Q.3.(a) Design a DFA to accept all the natural numbers divisible by 3. **6M**

(b) Construct a Mealy machine to find 2's complement of a given binary number. Assume that given binary number is presented from LSB to MSB. Also convert the resultant Mealy machine into its equivalent Moore machine. **8M**

OR

Q.4.(a) Convert the following NFA into its equivalent DFA : **6M**

Q/ Σ	0	1
→ p	p	p, q
*q	r	r
r	-	s
*s	s	s

(b) Construct a minimum state automaton equivalent to a given automation M whose transiton table is given by : **8M**

State/ Σ	a	b
→ q_0	q_0	q_3
q_1	q_2	q_5
q_2	q_3	q_4
(*q_3)	q_0	q_5
q_4	q_0	q_6
q_5	q_1	q_4
q_6	q_1	q_3

Q.5.(a) Reduce the following grammar : **5M**

S→aA / aBB

A→aaA/ ∈

B→bB/ bbC

C→B

(b) Obtain deterministic finite automata for the following regular expression : **4M**

(0 + 1)*10 (0 + 1)* + (0 + 1)*11(0 + 1)*

(c) Check whether the given grammar is ambiguous or not. **4M**

S→a / Sa / bSS / SbS

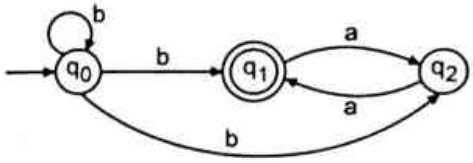
OR

Q.6.(a) Convert the following right linear grammar into left linear grammar : **7M**

S→ 01A /10

A→ 10A /10

(b) Construct a regular expression from the following finite automata : **6M**



Q.7.(a) Convert the CFG into PDA. **7M**

E→aAB/ d

A→BA / a

B→Ead / c

(b) Design a PDA for **7M**

L = {ww^R / w ∈ (a, b)*}

VBD

OR

Q.8.(a) Convert the given PDA to CFG : **7M**

$\delta(q_0, a, z_0) \rightarrow (q_0, x\, z_0)$

$\delta(q_0, a, x) \rightarrow (q_0, x\, x)$

$\delta(q_0, b, x) \rightarrow (q_1, \epsilon)$

$\delta(q_1, b, x) \rightarrow (q_1, \epsilon)$

$\delta(q_1, \epsilon, z_0) \rightarrow (q_1, \epsilon)$

(b) Using pumping lema, prove that language

$L = \{p^{i^2} / i \geq 1\}$ is not regular **7M**

Q.9.(a) Design a turing machine for the language $L = \{a^n b^m c^n / n, m \geq 1\}$ **6M**

(b) Design a TM to perform multiplication of two unary numbers **7M**

OR

Q.10.(a) Explain various types of turing machines. **6M**

(b) Design a turing machine to copy a string over $\Sigma = \{a, b\}^*$ **7M**

Q.11.(a) Explain post correspondence problem. Consider the post correspondence system described by the following lists :

A = {10, 01, 0, 100, 1}

B = {101, 100, 10, 0, 010}

Does this PCP have a resolution? **7M**

(b) Compute A(1, 1), A(1, 2), A(2, 1) using Ackermann function. **6M**

OR

Q.12. Write a short note on :

(i) Halting problem of turing machine.

(ii) Linear bounded automata.

(iii) Primitive recursive function. **13M**

WINTER EXAM. - 2018

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Explain closure of a relation. Find R^* for

$R = \{(1, 1) (1, 2) (2, 1) (2, 3) (3, 2)\}$. **3M**

Ans. See P.1-3, 5, Q.1, 4.

Given : $R = \{(1, 1), (1, 2), (2, 1), (2, 3), (3, 2)\}$

$R^2 = R \circ R = \{(1, 1), (1, 2), (2, 1) (2, 3) (3, 2)\}$

$\circ \{(1, 1) (1, 2) (2, 1) (2, 3) (3, 2)\}$

$= \{(1, 1) (1, 2) (1, 3) (2, 1) (2, 2) (3, 1) (3, 3)\}$

$R^3 = R^2 \circ R = \{(1, 1) (1, 2) (1, 3) (2, 1) (2, 2) (3, 1) (3, 3)\}$

$\circ \{(1, 1) (1, 2) (2, 1) (2, 3) (3, 2)\}$

$= \{(1, 1) (1, 2) (1, 3) (2, 1) (2, 2) (2, 3)$

$(3, 1) (3, 2)\}$

$R^4 = R^3 \circ R = \{(1, 1) (1, 2) (1, 3) (2, 1)$

$(2, 2) (2, 3) (3, 1) (3, 2)\} \circ \{(1, 1) (1, 2) (2, 1) (2, 3) (3, 2)\}$

$= \{(1, 1) (1, 2) (1, 3) (2, 1) (2, 2) (2, 3) (3, 1) (3, 2) (3, 3)\}$

$R^5 = R^4 \circ R = \{(1, 1) (1, 2) (1, 3) (2, 1) (2, 2) (2, 3) (3, 1)$

$(3, 2) (3, 3)\} \circ \{(1, 1) (1, 2) (2, 1) (2, 3) (3, 2)\}$

$= \{(1, 1) (1, 2) (1, 3) (2, 1) (2, 2) (2, 3) (3, 1) (3, 2) (3, 3)\}$

$R^5 = R^4$

$\therefore R^* = R \cup R^2 \cup R^3 \cup R^4$

$= \{(1, 1) (1, 2) (2, 1) (2, 3) (3, 2)\} \cup \{(1, 1) (1, 2) (1, 3) (2, 1)(2, 2)$

$(3, 1) (3, 3)\} \cup \{(1, 1) (1, 2) (1, 3) (2, 1) (2, 2) (2, 3) (3, 1) (3, 2)\}$

$\cup \{(1, 1) (1, 2) (1, 3) (2, 1) (2, 2) (2, 3), (3, 1) (3, 2) (3, 3)\}$

$R^* = \{(1, 1) (1, 2) (1, 3) (2, 1) (2, 2) (2, 3) (3, 1) (3, 2) (3, 3)\}$

$\therefore R^* = R^* \cup \{(1, 1) (2, 2) (3, 3)\}$

$= \{(1, 1) (1, 2) (1, 3) (2, 1) (2, 2) (2, 3)$

$(3, 1) (3, 2) (3, 3)\}$

Q.1.(b) Prove the following relation using principle of induction. **8M**

(i) $1.2.3 + 2.3.4 + \ldots + n(n + 1)(n + 2)$

$= \frac{n(n + 1)(n + 2)(n + 3)}{4}$

Ans. See P.1-9, Q.15.

(ii) $1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n(n+1)(2n+1)}{6}$

Ans. See P.PS-75, Q.1(a)(i).

Q.1.(c) What is countability? Explain. **2M**

Ans. See P.1-5, Q.5.

Q.2.(a) Describe the concept of Pigeon – hole principle with example. **5M**

Ans. See P.1-10, Q.16.

Q.2.(b) Define the following any four : **8M**

(i) Transitive closure.

(ii) Reflexive transitive closure.

(iii) Prefix of string.

(iv) Suffix of string.

(v) Substring.

(vi) Subsequence.

Ans. See P.1-4, 5, Q.12, 4.

(v) Substring : A substring is a contiguous sequence of a character within a string.

e.g. "The best of" is a substring of "It was the best of times".

(vi) Subsequence : A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

e.g. The sequence (A, B, D) is a subsequence of (A, B, C, D, E, F) obtained after removal of element C, E and F.

UNIT - II

(Total Marks : 28)

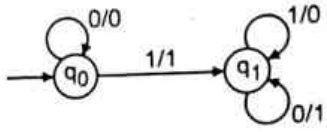
Q.3.(a) Design a DFA to accept all the natural numbers divisible by 3. **6M**

Ans. See P.2-32, Q.47.

Q.3.(b) Construct a Mealy machine to find 2's complement of a given binary number. Assume that given binary number is presented from LSB to MSB. Also convert the resultant Mealy machine into its equivalent Moore machine. **8M**

Ans. To find 2's complement of a binary number we assume that input is read from LSB to MSB. We will keep the binary number as it is until we read first 1, keep this 1 as it is and then change the remaining 1's by 0 and 0's by 1's.

Transition diagram for Mealy machine :



Transition table for mealy machine :

Present state	Next state			
	0		1	
	State	Output	State	Output
$\rightarrow q_0$	q_0	0	q_1	1
q_1	q_0	1	q_1	0

q_0 with output 0

q_1 which is associated with two output 0 and 1,

so we split q_1 into q_{10} and q_{11}. Now we will construct a transition table by introducing new state q_{10} and q_{11}.

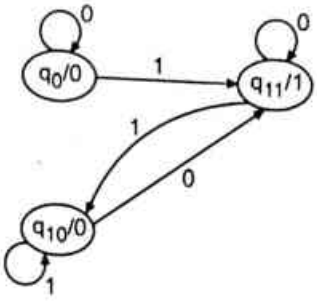
Transition table :

Present state	Next state			
	0		1	
	State	Output	State	Output
$\rightarrow q_0$	q_0	0	q_{11}	1
q_{10}	q_{11}	1	q_{10}	0
q_{11}	q_{11}	1	q_{10}	0

Thus, equivalent Moore machine is as follows :

Present state	Next state		Output
	0	1	
q_0	q_0	q_{11}	1
q_{10}	q_{11}	q_{10}	0
q_{11}	q_{11}	q_{10}	0

Transition diagram :



Q.4.(a) Convert the following NFA into its equivalent DFA : **6M**

Q/Σ	0	1
→p	p	p, q
*q	r	r
r	-	s
*s	s	s

Ans. See P.2-18, Q.21.

Q.4.(b) Construct a minimum state automaton equivalent to a given automaton M whose transition table is given by : **8M**

State/ Σ	a	b
→q_0	q_0	q_3
q_1	q_2	q_5
(q_2)	q_3	q_4
q_3	q_0	q_5
q_4	q_0	q_6
q_5	q_1	q_4
q_6	q_1	q_3

Ans.

State/ Σ	a	b
→q_0	q_0	q_3
q_1	q_2	q_5
(q_2)	q_3	q_4
q_3	q_0	q_5
q_4	q_0	q_6
q_5	q_1	q_4
q_6	q_1	q_3

Group 1 = $\{q_2\}$

Group 2 = $\{q_0, q_1, q_3, q_4, q_5, q_6\}$

Step 1 :

(i) $\{q_0, q_1\}$

$\therefore q_0 \neq q_1$

	a	b
q_0	q_0	q_3
q_1	q_2	q_5

Different group

(ii) $\{q_0, q_3\}$

$\therefore q_0 = q_3$

	a	b
q_0	q_0	q_3
q_3	q_0	q_5

(iii) $\{q_0, q_4\}$

$\therefore q_0 = q_4$

	a	b
q_0	q_0	q_3
q_4	q_0	q_6

(iv) $\{q_0, q_5\}$

$\therefore q_0 = q_5$

	a	b
q_0	q_0	q_3
q_5	q_1	q_4

(v) $\{q_0, q_6\}$

$\therefore q_0 = q_6$

	a	b
q_0	q_0	q_3
q_6	q_1	q_3

$\therefore$ New combination of group are

$\{q_2\}\{q_0, q_3, q_4, q_5, q_6\}\{q_1\}$

Step 2 :

(i) $\{q_0, q_3\}$

$\therefore q_0 = q_3$

	a	b
q_0	q_0	q_3
q_3	q_0	q_5

(ii) $\{q_0, q_4\}$

$\therefore q_0 = q_4$

	a	b
q_0	q_0	q_3
q_4	q_0	q_6

(iii) $\{q_0, q_5\}$

$\therefore q_0 \neq q_5$

	a	b
q_0	q_0	q_3
q_5	q_1	q_4

Different group

(iv) $\{q_0, q_6\}$

$\therefore q_0 \neq q_6$

	a	b
q_0	q_0	q_3
q_6	q_4	q_3

Different group

$\therefore$ New combination of group are

$\{q_2\}\{q_0, q_3, q_4\}\{q_5, q_6\}\{q_1\}$

Step 3 :

(i) $\{q_0, q_3\}$

$\therefore q_0 \neq q_3$

	a	b
q_0	q_0	q_3
q_3	q_0	q_5

Different group

(ii) $\{q_0, q_4\}$

$\therefore q_0 \neq q_4$

	a	b
q_0	q_0	q_3
q_4	q_0	q_6

Different group

$\therefore$ New combination of group are

$\{q_2\}\{q_0\}\{q_3, q_3\}\{q_5, q_6\}\{q_1\}$

Step 4 :

(i) $\{q_3, q_4\}$

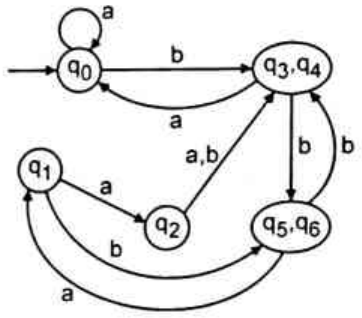
$\therefore q_3 = q_4$

	a	b
q_3	q_0	q_5
q_4	q_0	q_6

$\therefore$ Transition table :

Σ / State	a	b
$\rightarrow \{q_0\}$	q_0	$\{q_3, q_4\}$
$\{q_1\}$	q_2	$\{q_5, q_6\}$
$* \{q_2\}$	$\{q_3, q_4\}$	$\{q_3, q_4\}$
$\{q_3, q_4\}$	q_0	$\{q_5, q_6\}$
$\{q_5, q_6\}$	q_1	$\{q_3, q_4\}$

Transition graph :



UNIT - III

(Total Marks : 26)

Q.5.(a) Reduce the following grammar : **5M**

S→aA / aBB

A→aaA/ ∈

B→bB/ bbC

C→B

Ans. **See P.3-76, Q.148.**

Q.5.(b) Obtain deterministic finite automata for the following regular expression, **4M**

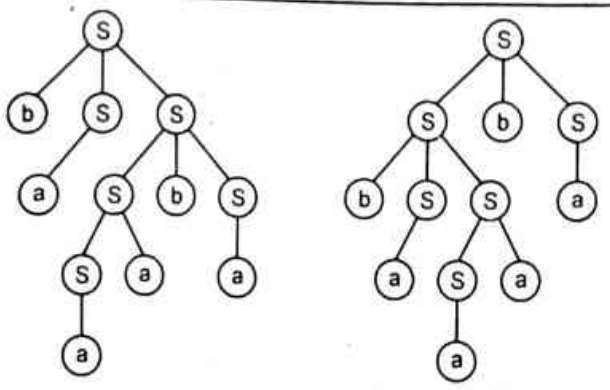
$(0 + 1)^*10\,(0 + 1)^* + (0 + 1)^*11(0 + 1)^*$

Ans. **See P.3-22, Q.29.**

Q.5.(c) Check whether the given grammar is ambiguous or not. **4M**

S→ a / Sa / bSS / SbS

Ans. Consider an input string "baaaba"



Since for an input string "baaaba", there exists two parse tree generating the string. Hence the given grammar is ambiguous.

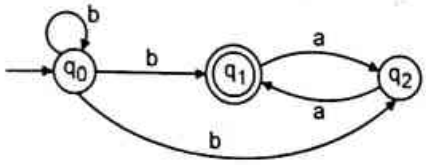
Q.6.(a) Convert the following right linear grammar into left linear grammar : **7M**

S→ 01A /10

A→10A /10

Ans. **See P.PS-41, Q.6(b).**

Q.6.(b) Construct a Regular expression from the following finite automata : **6M**



Ans. The equation for states are :

$q_0 = q_0b + \epsilon$ ...(1)

$q_1 = q_0b + q_2a$ ...(2)

$q_2 = q_0b + q_1a$ ...(3)

From equation (1)

$q_0 = q_0b + \epsilon$

By Arden theorem

$q_0 = \epsilon b^*$

$\therefore q_0 = b^*$ ...(4)

Put values of equation (3) to equation (2), we get

$q_1 = q_0b + (q_0b + q_1a)\,a$

$q_1 = q_0b + q_0\,ba + q_1aa$

Put values of equation (4), we get

$q_1 = b^*b + b^*ba + q_1\,aa$ ...(5)

By Arden theorem, we get

$q_1 = (b^*\,b + b^*ba)\,(aa)^*$

UNIT - IV

(Total Marks : 28)

Q.7.(a) Convert the CFG into PDA. **7M**

E→ aAB/ d

A→BA / a

B→Ead / c

Ans. **See P.PS-126, Q.8(a).**

Q.7.(b) Design a PDA for L = $\{ww^R / w \in \{a, b\}^*\}$ **7M**

Ans. **See P.4-15, Q.23.**

Q.8.(a) Convert the given PDA to CFG : **7M**

$\delta(q_0, a, z_0) \rightarrow (q_0, x\, z_0)$

$\delta(q_0, a, x) \rightarrow (q_0, x\, x)$

$\delta(q_0, b, x) \rightarrow (q_1, \epsilon)$

$\delta(q_1, b, x) \rightarrow (q_1, \epsilon)$

$\delta(q_1, \epsilon, z_0) \rightarrow (q_1, \epsilon)$

Ans. **See P.PS-61, Q.8(b).**

Q.8.(b) Using pumping lema, prove that language L = $\{a^{i^2} / i \geq 1\}$ is not regular **7M**

Ans. **See P.PS-77, Q.6(a).**

UNIT - V

(Total Marks : 30)

Q.9.(a) Design a turing machine for the language L = $\{a^n b^m c^n / n, m \geq 1\}$ **6M**

Ans. The language is fixed

Starts with a followed by b and end with c

Valid string

aa bbb cc

Algorithm :

(1) Assume q_0 is the initial state

(2) Replace the first symbol by a X. Change the state to q_1 and skip remaining a and b till symbol c arrives.

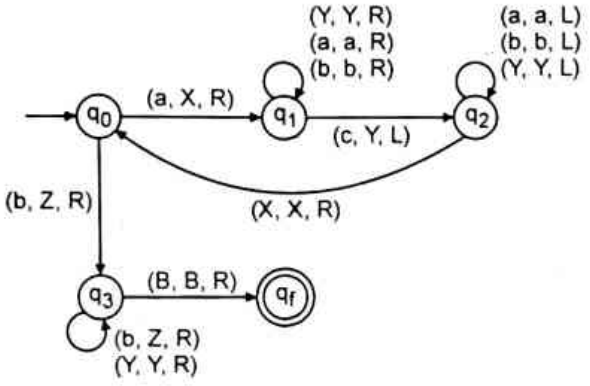
(3) On reading symbol c, replace it with Y and change the state to q_2 and moves towards left till we get X. On getting X, repeat the cycle.

(4) On reading the leftmost a, then replace b by Z and change the state to q_3. On q_3 state read all b and replace it by Z, move towards right till we get blank. On getting blank reach to final state

Transition table :

Present state	Input tape symbol						
	a	b	c	X	Y	Z	B
$\rightarrow q_0$	q_1, X, R	q_3, Z, R					
q_1	q_1, a, R	q_1, b, R	q_2, Y, L		q_1, Y, R		
q_2	q_2, a, L	q_2, b, L		q_0, X, R	q_2, Y, L		
q_3		q_3, Z, R			q_3, Y, R		q_R, B, R
q_f	-	-	-	-	-	-	-

Transition diagram :



Q.9.(b) Design a TM to perform multiplication of two unary numbers **7M**

Ans. **See P.PS-54, Q.10(b).**

Q.10.(a) Explain various types of turing machines. **6M**

Ans. **See P.5-21, Q.19.**

Q.10.(b) Design a turing machine to copy a string over $\Sigma = \{a, b\}^*$. **7M**

Ans. **See P.PS-88, Q.10(a).**

Q.12. Write a short note on :

(ii) Linear bounded automata. **4M**

Ans. **See P.5-34, Q.40.**

UNIT - VI

(Total Marks : 22)

Q.11.(a) Explain post correspondence problem. Consider the post correspondence system described by the following lists.

A = {10, 01, 0, 100, 1}

B = {101, 100, 10, 0, 010}

Does this PCP have a resolution? **7M**

Ans. **See P.6-15, 17, Q.18, 21(b).**

Q.11.(b) Compute A(1, 1), A(1, 2), A(2, 1) using Ackermann function **6M**

Ans. **See PS-54, Q.11(b).**

Q.12. Write a short note on :

(i) Halting problem of turing machine

(iii) Primitive recursive function. **9M**

Ans. **See P.6-19, 23, Q.25, 32.**

SUMMER EXAMINATION - 2019
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	26 M
Unit - V	34 M
Unit - VI	20 M
Questions covered in VBD	160 M (100 %)

SUMMER EXAM. - 2019
QUESTION PAPER

Q.1.(a) Explain Chomsky Hierarchy in detail. **6M**

(b) Prove the following using method of induction : **7M**

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

OR

Q.2.(a) Let R = {(1, 2), (2, 3), (3,1)} and A = {1, 2, 3}. Find reflexive, symmetric and transitive closure of R. **4M**

(b) Define : **3M**

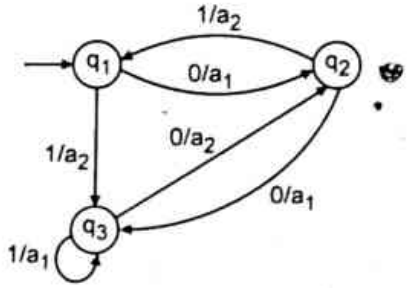
(i) Kleene closure

(ii) Positive closure with an example.

(c) Explain pigeon-hole principle in detail. **6M**

Q.3.(a) Construct a DFA over $\sum = \{0, 1\}$ for the "Language accepting 1100 or 1010 as a substring". **7M**

(b) Convert following mealy machine into equivalent Moore machine : **6M**



OR

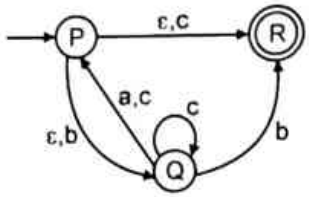
Q.4.(a) Construct DFA equivalent to :

$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0 \{q_2\})$

Where δ is defined by its state table **6M**

State	Input	
	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_2\}$
q_1	q_0	$\{q_1\}$
q_2	-	$\{q_0, q_1\}$

(b) Convert the NFA with ε -transition to NFA without ε -transition. **7M**



Q.5.(a) What is Regular Grammar? Find left linear and right linear grammar for the following regular expression : **8M**

(i) $(0 + 1)^* 00 (0 + 1)^*$

(ii) $0^* (1(0 + 1))^*$

(b) Convert the following grammar into CNF : **6M**

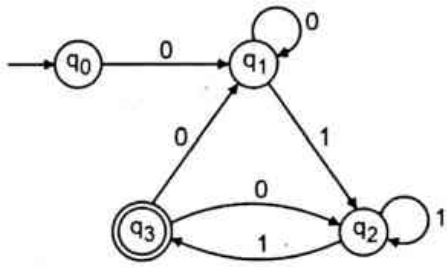
$S \rightarrow ABa$

$A \rightarrow aab$

$B \rightarrow Ab$

OR

Q.6.(a) Find Regular expression for following transition diagram : **7M**



(b) Explain closures properties of Regular set. **7M**

Q.7.(a) Design PDA for the language **7M**

$L = \{\omega \subset \omega^R \mid \omega \in (0 + 1)^*\}$

R : Reverse string.

(b) Explain the modal of PDA and its acceptance by stack and acceptance by final state. **6M**

OR

Q.8.(a) Explain pumping lemma theorem for context free language. **6M**

(b) Convert following CFG into PDA : **7M**

$E \rightarrow aAB \mid d$

$A \rightarrow BA \mid a$

$B \rightarrow Ead \mid c$

Q.9.(a) Design a Turing machine for the language

$L = \{WW^R \mid W \in (0 + 1)^*\}$ **8M**

R : Reverse string.

(b) Explain the modal of linear bounded automata. **6M**

OR

Q.10.(a) Explain Turing machine as transducers with example. **7M**

(b) Design a Turing machine that computes the function f (m, n) = m + n. **7M**

Q.11.(a) What is Ackermann's function? Calculate A (1, 1), A (1, 2), A (2, 1). **6M**

(b) Explain the properties of Recursively enumerable language. Give relation between recursive and recursive enumerable language. **7M**

OR

Q.12.(a) What is significance of PCP? Solve the following using PCP : **7M**

$A = \{b, bab^3, ba\}$, $B = \{b^3, ba, a\}$

(b) Write a short note on LBA. **6M**

SUMMER EXAM. - 2019

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Explain Chomsky Hierarchy in detail. **6M**

Ans. See P.1-11, Q.22.

Q.1.(b) Prove the following using method of induction :

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$ **7M**

Ans. See P.1-8, Q.11.

Q.2.(a) Let R = {(1, 2), (2, 3), (3,1)} and A = {1, 2, 3}. Find Reflexive, symmetric and transitive closure of R. **4M**

Ans. See P.PS-121, Q.2(c).

Q.2.(b) Define : **3M**

(i) Kleene closure

(ii) Positive closure with an example.

Ans. See P.1-11, Q.20.

Q.2.(c) Explain pigeon-hole principle in detail. **6M**

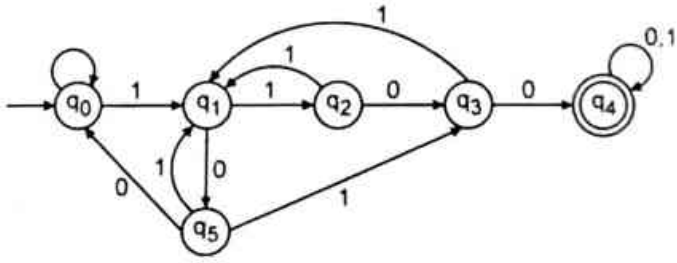
Ans. See P.1-10, Q.16.

UNIT - II

(Total Marks : 26)

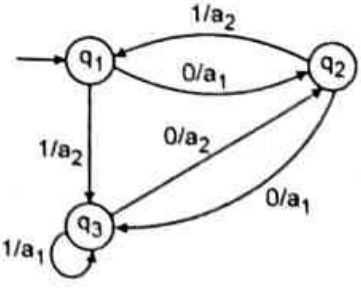
Q.3.(a) Construct a DFA over $\sum = \{0, 1\}$ for the "Language accepting 1100 or 1010 as a substring". **7M**

Ans. DFA :



Q \ Σ	0	1
q_0	q_0	q_1
q_1	q_5	q_2
q_2	q_3	q_1
q_3	q_4	q_1
(q_4)	q_4	q_4
q_5	q_0	q_1

Q.3.(b) Convert following mealy machine into equivalent Moore machine : **6M**



Ans. See P.PS-112, Q.4(a).

Q.4.(a) Construct DFA equivalent to :

$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$

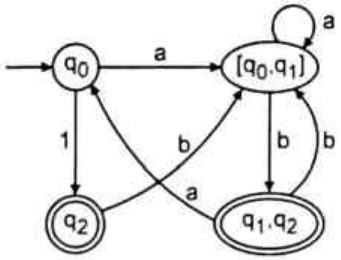
Where δ is defined by its state table. **6M**

State	Input	
	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_2\}$
q_1	q_0	q_1
q_2	-	$\{q_0, q_1\}$

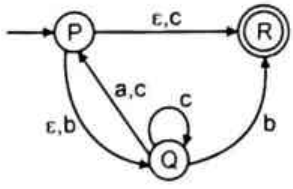
Ans.

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_2\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_1, q_2\}$
$*\{q_2\}$	-	$\{q_0, q_1\}$
$*\{q_1, q_2\}$	q_0	$\{q_0, q_1\}$

Transition diagram :



Q.4.(b) Convert the NFA with ε-transition to NFA without ε-transition. **7M**



Ans.

Σ / Q	a	b	c	∈
P	-	Q	R	R, Q
Q	P	R	P, Q	-
R	-	-	-	-

∈ - closure of state :

∈ - closure of (P) = {P, Q, R} = $\hat{\delta}$ (P, ∈)

∈ - closure of (Q) = {Q} = $\hat{\delta}$ (Q, ∈)

∈ - closure of (R) = {R} = $\hat{\delta}$ (R, ∈)

Transition of NFA without ∈ - moves

(1) $\hat{\delta}$ (P, a) = ∈ - closure (δ ($\hat{\delta}$ (P, ∈), a))

= ∈ - closure (δ (P, Q, R), a)

= ∈ - closure {δ (P, a) ∪ δ (Q, a) ∪ δ (R, a)}

= ∈ - closure {P}

= P, Q, R.

$\hat{\delta}$ (P, b) = ∈ - closure (δ ($\hat{\delta}$ (P, ∈), b))

= ∈ - closure (δ (P, Q, R), b)

= ∈ - closure {δ (P, b) ∪ δ (Q, b) ∪ δ (R, b)}

= ∈ - closure { Q ∪ R ∪ – }

= ∈ - closure {Q, R}

= Q, R.

$\hat{\delta}$ (P, c) = ∈ - closure (δ ($\hat{\delta}$ (P, ∈), c))

= ∈ - closure (δ (P, Q, R), c)

= ∈ - closure {δ (P, c) ∪ δ (Q, c) ∪ δ (R, c)}

= ∈ - closure {R ∪ (P, Q)}

= ∈ - closure {P, Q, R}

= P, Q, R.

(2) $\hat{\delta}$ (Q, a) = ∈ - closure (δ ($\hat{\delta}$ (Q, ∈), a))

= ∈ - closure (δ (Q, a))

= ∈ - closure {P}

= P, Q, R.

$\hat{\delta}$ (Q, b) = ∈ - closure (δ ($\hat{\delta}$ (Q, ∈), b))

= ∈ - closure (δ (Q, b))

= ∈ - closure {R}

= R.

$\hat{\delta}$ (Q, c) = ∈ - closure (δ ($\hat{\delta}$ (Q, ∈), c))

= ∈ - closure (δ (Q, c))

= ∈ - closure {P, Q}

= P, Q, R.

(3) $\hat{\delta}(R, a) = \epsilon$ - closure $(\delta(\hat{\delta}(R, \epsilon), a))$

$= \epsilon$ - closure $(\delta(R, a))$

$= \phi$

$\hat{\delta}(R, b) = \epsilon$ - closure $(\delta(\hat{\delta}(R, \epsilon), b))$

$= \epsilon$ - closure $(\delta(R, b))$

$= \phi$

$\hat{\delta}(R, c) = \epsilon$ - closure $(\delta(\hat{\delta}(R, \epsilon), c))$

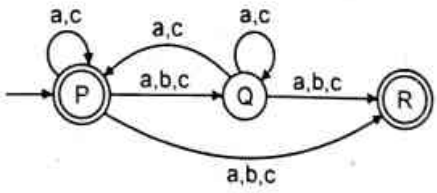
$= \epsilon$ - closure $(\delta(R, c))$

$= \phi$

∴ T. T.

Q \ Σ	a	b	c
*P	P, Q, R	Q, R	P, Q, R
Q	P, Q, R	R	P, Q, R
*R	ϕ	ϕ	ϕ

T. D.



UNIT - III

(Total Marks : 28)

Q.5.(a) What is Regular Grammar? Find left linear and right linear grammar for the following regular expression : **8M**

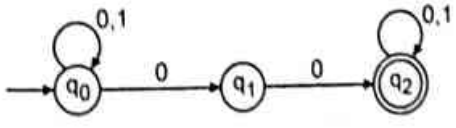
(i) $(0 + 1)^* 00 (0 + 1)^*$

(ii) $0^* (1(0 + 1))^*$

Ans. See P.3-36, Q.41.

(i) $(0 + 1)^* 00 (0 + 1)^*$

Transition graph :



Let $q_0 = A_0, q_1 = A_1, q_2 = A_2$

∴ $G = (\{A_0, A_1, A_2\}), \{0, 1\}, P, A_0)$

∴ Right linear grammar is

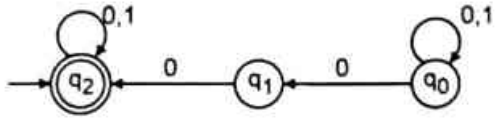
$A_0 \rightarrow 0A_0 \mid 1A_0 \mid 0A_1$

$A_1 \rightarrow 0A_2 \mid 0$

$A_2 \rightarrow 0A_2 \mid 1A_2 \mid 0 \mid 1$

Left linear grammar is :

Transition graph :



Let $q_0 = A_0, q_1 = A_1, q_2 = A_2$

∴ $G = (\{A_0, A_1, A_2\}), \{0, 1\}, P, A_0)$

∴ **LLG :**

$A_0 \rightarrow A_0 0 \mid A_0 1 \mid A_1 0$

$A_1 \rightarrow A_2 0 \mid 0$

$A_2 \rightarrow A_2 0 \mid A_2 1 \mid 0 \mid 1$

(ii) $0^* (1(0 + 1))^*$

$0^* = \{\epsilon, 0, 00, 000 \ldots\ldots\}$

$1(0 + 1) = (10 + 11)^*$

$(10 + 11)^* = \{\epsilon, 10, 11, 1011, 1110 \ldots\}$

∴ $0^* (1(0 + 1))^* = \{\epsilon, 010, 011, 01011, 01110, 0010\}$

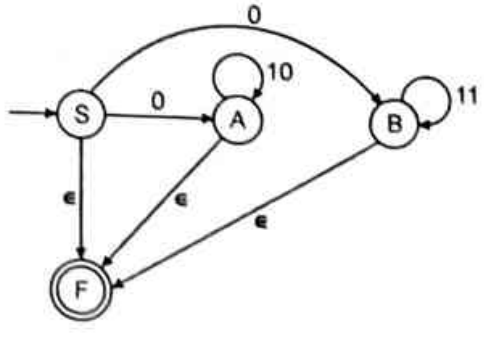
∴ RLG described is

$S \rightarrow 0AB \mid \epsilon$

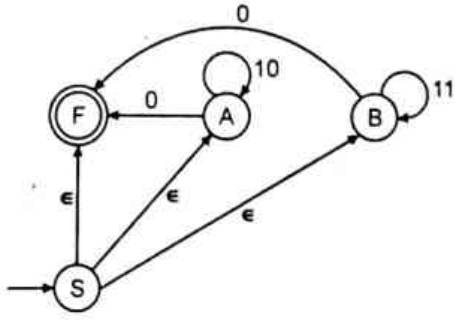
$A \rightarrow 10A \mid \epsilon$

$B \rightarrow 11B \mid \epsilon$

Transition graph for RLG



Exchange the position of initial and final state



LLG is

$S \rightarrow AB \mid \epsilon$

$A \rightarrow A10 \mid 0$

$B \rightarrow B11 \mid 0$

Q.5.(b) Convert the following grammar into CNF : **6M**

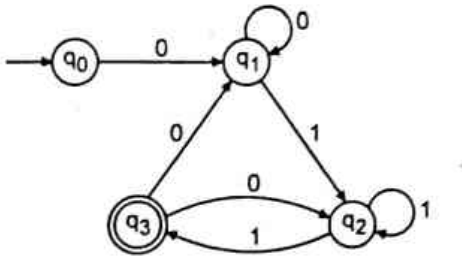
$S \rightarrow ABa$

$A \rightarrow aab$

$B \rightarrow Ab$

Ans. **See P.3-81, Q.161.**

Q.6.(a) Find Regular expression for following transition diagram : **7M**



Ans. Equation for all states

$q_0 = \epsilon$ ....(1)

$q_1 = q_0 0 + q_1 0 + q_3 0$ ....(2)

$q_2 = q_1 1 + q_2 1 + q_3 1$ ....(3)

$q_3 = q_2 0$ .....(4)

Put value of equation (1) in equation (2), we get

$q_1 = 0 + q_1 0 + q_3 0$ .....(5)

Put value of equation (4) in equation (5), we get

$q_1 = 0 + q_1 0 + q_2 00$ ....(6)

By Arden's theorem

$q_1 (0 + q_2 00) 0^*$ ...(7)

Put value of equation (7) in equation (3)

$q_2 = 00^* + q_2 000^* + q_2 1 + q_2 01$

$q_2 = 00^* + q_2 (000^* + 1 + 01)$

∴ By Arden theorem,

$q_2 = 00^* [00^* + 1 + 01]^*$

Put this value in equation (4)

$\therefore\ q_3 = 00^* [000^* + 1 + 01]^* 0$

Q.6.(b) Explain closures properties of Regular set. **7M**

Ans. **See P.3-59, Q.98.**

UNIT - IV

(Total Marks : 26)

Q.7.(a) Design PDA for the language **7M**

$L = \{\omega \subset \omega^R \mid \omega \in (0+1)^*\}$

R : Reverse string.

Ans. **See P.4-15, Q.22.**

Q.7.(b) Explain the modal of PDA and its acceptance by stack and acceptance by final state. **6M**

Ans. **See P.4-4, 5, 18, Q.1, 2, 28.**

Q.8.(a) Explain pumping lemma theorem for context free language. **6M**

Ans. **Pumping lemma for Context-Free Languages (CFL) :**

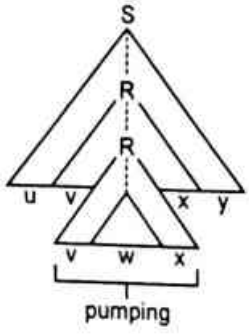
- Pumping lemma for CFL states that for any Context Free Language L, it is possible to find two substrings that can be 'pumped' any number of times and still be in the same language. For any language L, we break its strings into five parts and pump second and fourth substring.
- Pumping lemma, here also, is used as a tool to prove that a language is not CFL. Because, if any one string does not satisfy its conditions, then the language is not CFL.

  Thus, if L is a CFL, there exists an integer n, such that for all $x \in L$ with $|x| \geq n$, there exists u, v, w, x, $y \in \Sigma^*$, such that $x = uvwxy$ and

(1) $|vwx| \leq n$

(2) $|vx| \geq 1$

(3) for all $i \geq 0$: $uv^iwx^iy \in L$



For above example, 0^n1^n is CFL, as any string can be the result of pumping at two places, one for 0 and other for 1.

Let us prove, $L_{012} = \{0^n1^n2^n \mid n \geq 0\}$ is not context-free

Let us assume that L is context-free, then by pumping lemma, the above given rules follow :

Now, let $x \in L$ and $|x| \geq n$. So, by pumping lemma, there exists u, v, w, x, y such that (1) – (3) hold.

We show that for all u, v, w, x, y (1) – (3) do not hold.

If (1) and (2) hold then $x = 0^n1^n2^n = u\,v\,w\,x\,y$ with $|vwx| \leq n$ and $|vx| \geq 1$.

(1) Tells us that vwx does not contain both 0 and 2. Thus, either vwx has no 0's, or vwx has no 2's. Thus, we have two cases to consider.

Suppose vwx has no 0's. By (2), vx contains a 1 or a 2. Thus uwy has 'n' 0's and uwy either has less than 'n' 1's or has less than 'n' 2's.

But (3) tell us that $uwy = uv^0wx^0y \in L$.

So, uwy has an equal number of 0's, 1's and 2's gives us a contradiction. The case where vwx has no 2's is similar and also gives us a contradiction. Thus L is not context-free.

Q.8.(b) Convert following CFG into PDA : **7M**

$E \rightarrow aAB \mid d$

$A \rightarrow BA \mid a$

$B \rightarrow Ead \mid e$

Ans. **See P.PS-126, Q.8(a).**

UNIT - V

(Total Marks : 34)

Q.9.(a) Design a Turing machine for the language **8M**

$L = \{WW^R \mid W \in (0 + 1)^*\}$

R : Reverse string

Ans. **See P.5-18, Q.15.**

Q.9.(b) Explain the modal of linear bounded automata. **6M**

Ans. **See P.5-34, Q.40.**

Q.10.(a) Explain Turing machine as transducers with example. **7M**

Ans.

- Among the most general models of transducers that come to mind are probably those that allow more than one auxiliary work tape, unrestricted auxiliary work tapes, two-way input heads, inputs enclosed between end markers and acceptance anywhere in the inputs. A class of such models, called turning transducers, is introduced below :
- Each turning transducer M can be viewed as an abstract computing machine that consists of a finite-state control, an input tape, a read-only input head, m auxiliary work tapes for some $m \geq 0$, a read-write auxiliary work-tape head for each auxiliary work tape, an output tape and a write only output head. (See Fig.)

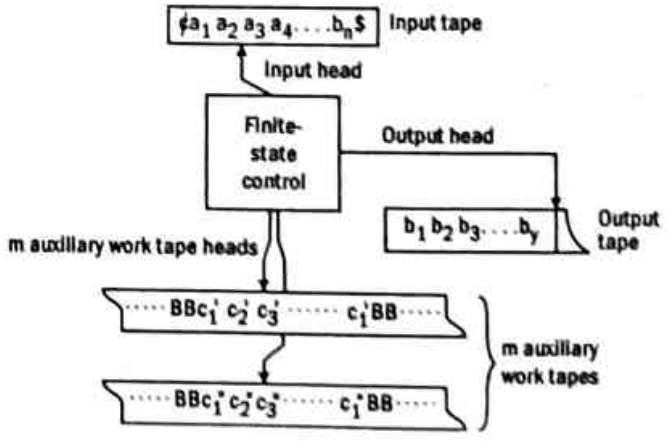


Fig. Schema of a turning transducer

- Each move of M is determined by the state of M, the symbol under the input head and the symbols under the heads of the auxiliary work tapes. Each move of M consists of changing the state of M, changing the symbol under the head of each auxiliary work tape, relocating

each head by at most one position in any direction and writing at most one symbol onto the output tape.

- Initially M is assumed to have its input $a_1 \ldots a_n$ stored on the input tape between a left end marker ¢ and a right end marker $. In addition, the input head is assumed to be located at the start of the input the auxiliary work tapes are assumed to contain just blank symbols B and the output tape is assumed to be empty

Q.10.(b) Design a Turing machine that computes the function

$f(m, n) = m + n$. **7M**

Ans. **See P.5-38, Q.47.**

Q.12.(b) Write a short note on LBA **6M**

Ans. **See P.5-34, Q.40.**

UNIT - VI

(Total Marks : 20)

Q.11.(a) What is Ackermann's function?

Calculate : A (1, 1), A (1, 2), A (2, 1) **6M**

Ans. **See P.6-38, Q.68.**

Q.11.(b) Explain the properties of Recursively enumerable language. Give relation between recursive and recursive enumerable language. **7M**

Ans. **See P.6-7, 8, 9, Q.2, 4, 5, 6.**

Q.12.(a) What is significance of PCP? Solve the following using PCP : **7M**

$A = \{b, bab^3, ba\}$, $B = \{b^3, ba, a\}$

Ans. **See P.6-16, Q.19.**

Notes :-

WINTER EXAMINATION - 2018
B.E. IV SEM. (COMPUTER TECHNOLOGY) (CBS)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	26 M
Unit - III	28 M
Unit - IV	28 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

WINTER EXAM. - 2018
QUESTION PAPER

Q.1.(a) Prove the following relations using principle of Mathematical induction : **8M**

(i) $1+4+7+-----+(3n-2)=\frac{n(3n-1)}{2}$

(ii) $1+2+-------+n=\frac{n^2+n}{2}$

(b) Define **5M**

(i) Reflexive relation

(ii) Symmetric relation

(iii) Transitive relation

(iv) Equivalence relation

OR

Q.2.(a) State and explain pigeon hole principle. **3M**

(b) Find R^* and R^+ if $R = \{(1, 2), (2, 3), (2, 4)\}$ be a relation in $\{1, 2, 3, 4\}$. **6M**

(c) Explain formal grammar. **4M**

Let grammar $G = \{(s, c), (a, b), p, s\}$ where

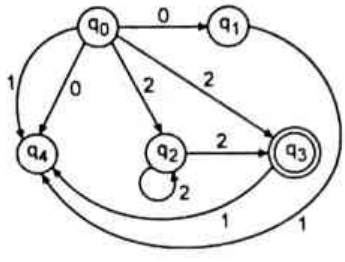
P consists of

$S \rightarrow aCa$

$C \rightarrow aCa \mid b$ find L (G).

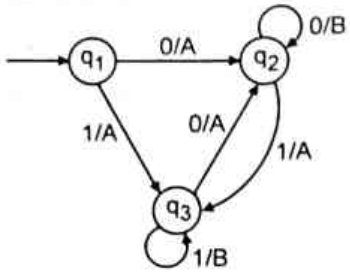
Q.3.(a) Design a Moore machine to generate 2's complement of a given binary number. **6M**

(b) Construct a deterministic finite automata equivalent to following NDFA : **7M**



OR

Q.4.(a) Construct a Moore machine equivalent to mealy machine given as : **6M**



(b) Construct a minimum state automate equivalent to DFA whose transition table is defined as : **7M**

State / Σ	a	b
→ q_0	q_1	q_2
q_1	q_4	q_3
q_2	q_4	q_3
(q_3)	q_5	q_6
(q_4)	q_7	q_6
q_5	q_3	q_6
q_6	q_6	q_6
q_7	q_4	q_6

Q.5.(a) Give regular expression for following statements : **4M**

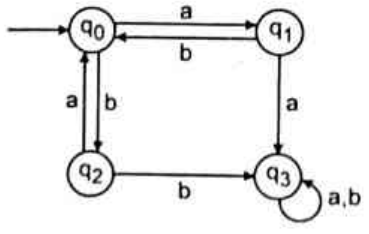
(i) The set of all string which consists of all words with exactly two a over {a, b}.

(ii) The set of all string of 0's and 1's ending in 00.

(iii) The set of all string of 0's and 1's beginning with 0 and ending with 1.

(iv) Any no. of 0's followed by any no of 1's followed by any no. of 2's over $\Sigma = [0, 1, 2]$.

(b) Construct a regular expression corresponding to state diagram given below : **6M**



(c) Find CFG for each of the language defined by following regular expression : **4M**

(i) ab^* (ii) $a^* b^*$

VBD

OR

Q.6.(a) Convert the following grammar in GNF : **9M**

S→AB

A→BS | a

B→SA | b

(b) Show the derivation steps and construct derivation tree for string aabbbb by using leftmost derivation and rightmost derivation with the grammar defined as : **5M**

S→AB | ∈

A→ aB

B→ Sb

Q.7.(a) Design PDA to accept following CFG : **7M**

S→ XY

X→AX | BX | a

Y→YA | YB | a

A→ a

B→ a

(b) Give formal definition of PDA with block diagram and construct a PDA for the following language:

$L = \{a^m b^n c^p d^q\}$, where $m + n = p + q$ **7M**

OR

Q.8.(a) Construct PDA for the language : **5M**

$L = \{a^n b^{3n} \mid n \geq 0\}$

(b) Construct a PDA - equivalent to the following context free grammar

S→ 0BB

B→ 0S | 1S | 0

Test whether 010^4 is in N(M) **5M**

(c) Explain closure properties of context free language **4M**

Q.9.(a) Construct a turing machine for proper subtraction of two integer, defined as :

$$m - n = \begin{cases} m - n, & m \geq n \\ 0, & \text{otherwise} \end{cases}$$ **8M**

(b) Describe linear bounded automata **5M**

OR

Q.10.(a) Design a turing machine that replaces all occurrence of '111' by '101' from sequence of 0's and 1's **6M**

(b) Explain different types of turing machine **7M**

Q.11.(a) What is primitive recursive function? Show that SUM (x, y) and SUB (x, y) are primitive recursive functions. **6M**

(b) Define Ackermann's function. Compute for A (1, 1) A (2, 1) and A (2, 2). **7M**

OR

Q.12.(a) Explain the properties of recursive and recursive enumerable language. **6M**

(b) Define decidable and undecidable language. **4M**

(c) What is μ -recursive function? Explain. **3M**

WINTER EXAM. - 2018

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Prove the following relations using principle of Mathematical induction : **8M**

(i) $1 + 4 + 7 + \text{- - - - } + (3n - 2) = \frac{n(3n-1)}{2}$

(ii) $1 + 2 + \text{- - - - - - } + n = \frac{n^2 + n}{2}$

Ans. **See P.PS-75, Q.1(a)(ii) and P.1-7, Q.10.**

Q.1.(b) Define : **5M**

(i) Reflexive relation.

(ii) Symmetric relation.

(iii) Transitive relation.

(iv) Equivalence relation.

Ans. **See P.1-4, 5 Q.3, 4.**

Q.2.(a) State and explain pigeon hole principle. **3M**

Ans. **See P.1-10, Q.16.**

Q.2.(b) Find R^* and R^+ if $R = \{(1, 2), (2, 3), (2, 4)\}$ be a relation in $\{1, 2, 3, 4\}$. **6M**

Ans. Given : $R = \{(1, 2), (2, 3) (2, 4)\}$

$R^2 = R \circ R$

$= \{(1,2)(2,3),(2,4)\} \circ \{(1,2),(2,3)(2,4)\}$

$= \{(1,3)(1,4)\}$

$\therefore R^3 = R \circ R^2$

$= \{(1,2),(2,3)(2,4)\} \circ \{(1,3)(1,4)\}$

$= \phi$

$\therefore R^* = R \cup R^2$

$= \{(1,2)(2,3),(2,4)\} \cup \{(1,3),(1,4)\}$

$= \{(1,2),(2,3),(2,4),(1,3),(1,4)\}$

$\therefore R^* = R^* \cup \{(1,1),(2,2),(3,3),(4,4)\}$

$= \{(1,2)(2,3),(2,4),(1,3)(1,4)\} \cup \{(1,1),$

$(2,2),(3,3),(4,4)\}$

$\therefore R^* = \{(1,2)(2,3)(2,4)(1,3)(1,4)(1,1)(2,2)(3,3)(4,4)\}$

Q.2.(c) Explain formal grammar.

Let grammar G = {(s, c), (a, b),p, s} where

P consists of **4M**

S→aCa

C→aCa | b find L (G).

Ans. A formal grammar defines a formal language, which is a set of finite length sequence of symbol that may be constructed by applying production rule to another sequence of symbol which initially contain start symbol

A formal grammar is a four tuples (V, T, P, S) where

V – is a finite set of symbols called as variables or non terminals

T – is a finite set of symbols called as terminals

P – is a finite set of rules called as production rule

S – is a a member of V called as start symbol

S → aCa

C → aCa | b

(1) S → aCa

S → aba

(2) S → aCa

S → aa Ca a

S → aabaa

(3) S → aCa

→ aaCaa

→ aaaCaaa

→ aaabaaa

$\therefore L(G) = \{a^n b a^n \mid n \geq 1\}$

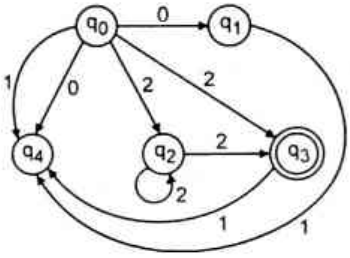
UNIT - II

(Total Marks : 26)

Q.3.(a) Design a Moore machine to generate 2's complement of a given binary number. **6M**

Ans. **See P.PS-60, Q.3.(a).**

Q.3.(b) Construct a deterministic finite automata equivalent to following NDFA : **7M**



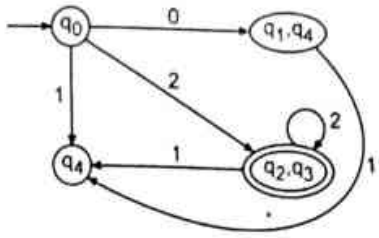
Ans. Transition table for NDFA

State	Input symbol		
	0	1	2
→q_0	$\{q_1, q_4\}$	q_4	$\{q_2, q_3\}$
q_1	–	q_4	–
q_2	ϕ	ϕ	q_2, q_3
*q_3	ϕ	q_4	ϕ
q_4	ϕ	ϕ	ϕ

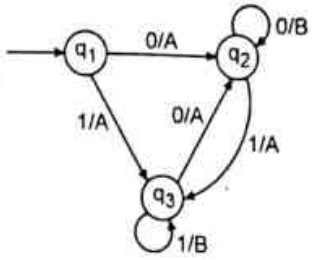
$\therefore$ Transition table for DFA

State	0	1	2
→q_0	$\{q_1, q_4\}$	q_4	$\{q_2, q_3\}$
$\{q_1, q_4\}$	ϕ	q_4	ϕ
*$\{q_1, q_4\}$	ϕ	q_4	$\{q_2, q_3\}$
q_4	ϕ	ϕ	ϕ

Transition diagram :



Q.4.(a) Construct a Moore machine equivalent to mealy machine given as : **6M**



Ans. See P.2-56, Q.78.

Q.4.(b) Construct a minimum state automate equivalent to DFA whose transition table is defined as : **7M**

State/ Σ	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_4	q_3
q_2	q_4	q_3
(q_3)	q_5	q_6
(q_4)	q_7	q_6
q_5	q_3	q_6
q_6	q_6	q_6
q_7	q_4	q_6

Ans. Group 1 = $\{q_3, q_4\}$

$\therefore$ Group 2 = $\{q_0, q_1, q_2, q_5, q_6, q_7\}$

Step 1 :

(i) $\{q_0, q_1\}$

$\therefore q_0 \neq q_1$

	a	b
q_0	q_1	q_2
q_1	q_4	q_3

↓

Different group

(ii) $\{q_0, q_2\}$

$\therefore q_0 \neq q_2$

	a	b
q_0	q_1	q_2
q_2	q_4	q_3

↓

Different group

(iii) $\{q_0, q_5\}$

$\therefore q_0 \neq q_5$

	a	b
q_0	q_1	q_2
q_5	q_3	q_6

↓

Different group

(iv) $\{q_0, q_6\}$

$\therefore q_0 \neq q_6$

	a	b
q_0	q_1	q_2
q_6	q_6	q_6

(v) $\{q_0, q_7\}$

$\therefore q_0 \neq q_7$

	a	b
q_0	q_1	q_2
q_7	q_4	q_6

↓

Different group

$\therefore$ New combination of group are

$\{q_3, q_4\}, \{q_0, q_6\}\{q_1, q_2, q_5, q_7\}$

Step 2 :

(i) $\{q_1, q_2\}$

$\therefore q_1 = q_2$

	a	b
q_1	q_4	q_3
q_2	q_4	q_3

(ii) $\{q_1, q_5\}$

$\therefore q_1 \neq q_5$

	a	b
q_1	q_4	q_3
q_5	q_3	q_6

↓

Different group

(iii) $\{q_1, q_7\}$

$\therefore q_1 \neq q_7$

	a	b
q_1	q_4	q_3
q_7	q_4	q_6

↓

Different group

∴ New combination of groups are

$\{q_3, q_4\}, \{q_0, q_6\}, \{q_1, q_2\}, \{q_5, q_7\}$

Step 3 :

(i) $\{q_5, q_7\}$

$\therefore q_5 = q_7$

	a	b
q_5	q_3	q_6
q_7	q_4	q_6

(ii) $\{q_0, q_6\}$

$\therefore q_0 \neq q_6$

	a	b
q_0	q_1	q_2
q_6	q_6	q_6

↓

Different group

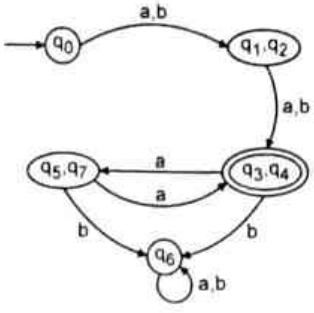
∴ New combinations of group are

$\{q_3, q_4\}, \{q_0\}, \{q_6\}, \{q_1, q_2\}, \{q_5, q_7\}$

∴ **Transition table :**

State	a	b
$\rightarrow q_0$	$\{q_1, q_2\}$	$\{q_1, q_2\}$
$\{q_1, q_2\}$	$\{q_3, q_4\}$	$\{q_3, q_4\}$
* $\{q_3, q_4\}$	$\{q_5, q_7\}$	$\{q_6\}$
$\{q_5, q_7\}$	$\{q_3, q_4\}$	q_6
$\{q_6\}$	q_6	q_6

Transition diagram :



UNIT - III

(Total Marks : 28)

Q.5.(a) Give regular expression for following statements : **4M**

(i) The set of all string which consists of all words with exactly two a over {a, b}.

(ii) The set of all string of 0's and 1's ending in 00.

(iii) The set of all string of 0's and 1's beginning with 0 and ending with 1.

(iv) Any no. of 0's followed by any no of 1's followed by any no. of 2's over $\Sigma = \{0, 1, 2\}$

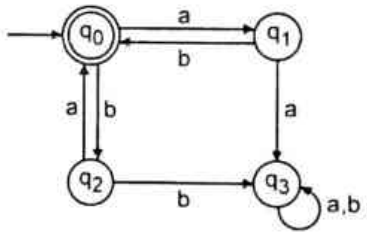
Ans (i) $b^* a b^* a b^*$

(ii) $(0 + 1)^* 00$

(iii) $0(0 + 1)^* 1$

(iv) $0^* 1^* 2^*$

Q.5.(b) Construct a regular expression corresponding to state diagram given below : **6M**



Ans. **See P.3-47, Q.68.**

Q.5.(c) Find CFG for each of the language defined by following regular expression : **4M**

(i) ab^* (ii) $a^* b^*$

Ans. **See P.3-63, Q.108.**

Q.6.(a) Convert the following grammar in GNF : **2M**

S→AB

A→BS | a

B→SA | b

Ans. **See P.3-88, Q.177.**

Q.6.(b) Show the derivation steps and construct derivation tree for string aabbbb by using leftmost derivation and rightmost derivation with the grammar defined as : **5M**

S→ AB | ε

A→ aB

B→ Sb

Ans. **See P.3-69, Q.129.**

UNIT - IV

(Total Marks : 28)

Q.7.(a) Design PDA to accept following CFG **7M**

S→ XY

X→AX | BX | a

Y→YA | YB | a

A→ a

B→ a

Ans. **See P.4-25, Q.36.**

Q.7.(b) Give formal definition of PDA with block diagram and construct a PDA for the following language

$L = \{a^m b^n c^p d^q\}$ where $m + n = p + q$ **7M**

Ans. **See P. PS-143, Q.7(a)(1)**

Q.8.(a) Construct PDA for the language.

$L = \{a^n b^{3n} \mid n \geq 0\}$ **5M**

Ans. The basic idea for designing this PDA is that all a's are pushed onto stack after scanning and when b's are scanned a group of three b's pop one a.

The required PDA is

$M = (\{q_0, q_1, q_2, q_3, q_f\}, (a, b),$

$(a, z_0), \delta, z_0, q_0, \{q_f\})$

where δ is defined as

(i) $\delta(q_0, \wedge, z_0) = (q_f, z_0)$ (case n = 0)

(ii) $\delta(q_0, a, z_0) = (q_0, a z_0)$

(iii) $\delta(q_0, a, a) = (q_0, aa)$

(iv) $\delta(q_0, b, a) = (q_1, a)$

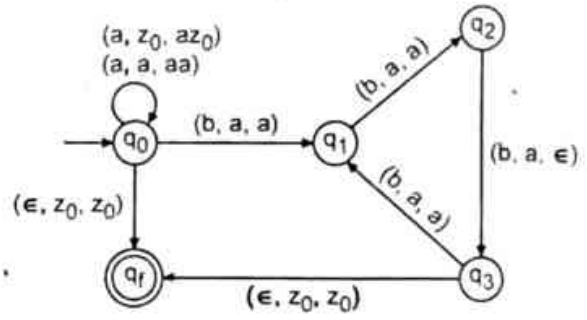
(v) $\delta(q_1, b, a) = (q_2, a)$

(vi) $\delta(q_2, b, a) = (q_3, \wedge)$

(vii) $\delta(q_3, b, a) = (q_1, a)$

(viii) $\delta(q_3, \epsilon, z_0) = (q_f, \epsilon)$

Transition diagram :



Q.8.(b) Construct a PDA - equivalent to the following context free grammar :

S→ 0BB

B→ 0S| 1S | 0

Test whether 010^4 is in N(M). **5M**

Ans. $M = (\{q\}, \{0, 1\}, \{S, B, 0, 1\}, \delta, q, S, f)$ where δ is defined by following rules :

(1) S →0BB

$\delta(q, 0, S) \rightarrow (q, BB)$ ...(1)

(2) B→ 0S

$\delta(q, 0, B) \rightarrow (q, S)$ ...(2)

(3) B→ 1S

$\delta(q, 1, B) \rightarrow (q, S)$ ...(3)

(4) B→ 0

$\therefore \delta(q, 0, B) \rightarrow (q, \epsilon)$ ...(4)

010^4

$(q, 010^4, S)$

⇓

$\vdash (q, 10^4, BB)$ ...by (1)

$\vdash (q, 0^4, SB)$ ...by (3)

$\vdash (q, 0^3, BBB)$ ...by (1)

$\vdash (q, 0^2, BB)$ ...by (4)

$\vdash (q, 0, B)$ ...by (4)

$\vdash (q, \epsilon)$

Q.8.(c) Explain closure properties of context free language. **4M**

Ans. See P.4-26, Q.40.

UNIT - V
(Total Marks : 26)

Q.9.(a) Construct a turing machine for proper subtraction of two integer, defined as

$$m - n = \begin{cases} m - n, & m \geq n \\ 0, & \text{otherwise} \end{cases}$$ **8M**

Ans. See P.5-40, Q.48.

Q.9.(b) Describe linear bounded automata **5M**

Ans. See P.5-34, Q.40.

Q.10.(a) Design a turing machine that replaces all occurrence of '111' by '101' from sequence of 0's and 1's. **6M**

Ans. See P.5-17, Q.13.

Q.10.(b) Explain different types of turing machine. **7M**

Ans. See P.5-21, Q.19.

UNIT - VI
(Total Marks : 26)

Q.11.(a) What is primitive recursive function? Show that SUM (x, y) and SUB (x, y) are primitive recursive function **6M**

Ans. See P.6-31, 28, 29, Q.51, 42, 45.

Q.11.(b) Define Ackermann's function. Compute for A (1, 1) A (2, 1) and A (2, 2) **7M**

Ans. See P.6-38, Q.68 and PS-54, Q.11(b).

Q.12.(a) Explain the properties of recursive and recursive enumerable language. **6M**

Ans. See P.6-7, Q.2.

Q.12.(b) Define decidable and undecidable language. **4M**

Ans. See P.6-13, 14, Q.15(a), 16(a).

Q.12.(c) What is μ-recursive function? Explain. **3M**

Ans. See P.6-26, Q.37.

Notes :-

SUMMER EXAMINATION - 2019
B.E. IV SEM. (COMPUTER TECHNOLOGY) (CBS)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	28 M
Unit - III	26 M
Unit - IV	26 M
Unit - V	28 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

SUMMER EXAM. - 2019
QUESTION PAPER

Q.1.(a) Explain following in detail with example : **6M**

(i) Type - 0 grammar.

(ii) Type - 1 grammar.

(iii) Type - 2 grammar.

(iv) Type - 3 grammar.

(b) Prove the following theorem by induction : **7M**

$$\sum_{i=1}^{n} \frac{1}{i(i+1)} = \frac{n}{n+1} \text{ is true for all } n \geq 0$$

OR

Q.2.(a) If R = {(1, 2), (2, 3), (1, 4), (4, 2), (3, 4)}, obtain R^+ and R^*. **7M**

(b) Explain following properties of relation with example : **6M**

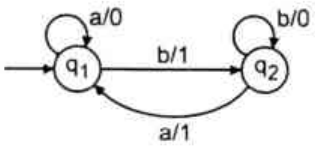
(i) Reflexive.

(ii) Antisymmetric.

(iii) Transitive.

Q.3.(a) Construct DFA to accept the set of all string such that every block of four consecutive symbols contain at least two 0's over {0, 1}*. **8M**

(b) Convert the following mealy machine into Moore machine : **6M**



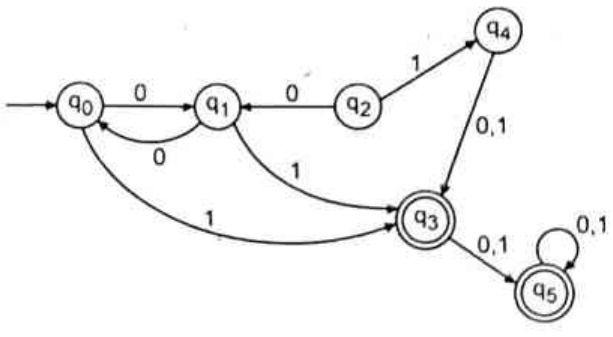
OR

Q.4.(a) Differentiate between : **7M**

(i) DFA and NFA.

(ii) Mealy and Moore machine.

(b) Construct the minimum state automata equivalent to following transition diagram shown below : **7M**

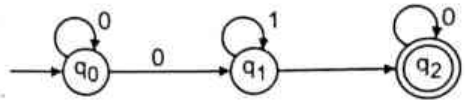


Q.5.(a) Construct left linear and right linear grammar for following expression $p = a^* b^* (a + b)^*$. **7M**

(b) Construct NFA with and without E-transition equivalent to following regular expression : $R = 10 + (0 + 11)\,0^* 1$ **6M**

OR

Q.6.(a) Construct regular expression for the following automata : **6M**



(b) Show that following language are not regular : **7M**

(i) $L = \{ a^p / p \text{ is a prime number} \}$.

(ii) $L = \{0^n 1^n \mid n \geq 1\}$.

Q.7.(a) Construct PDA that accept the language $L = \{a^n b^n c^n \mid n \geq 0\}$. **8M**

(b) Explain modal of PDA and its acceptance by stack and acceptance by final state. **5M**

OR

Q.8.(a) Construct PDA for the language $L = \{WCW^R / W \in \{a, b\}^*\}$, R : Reverse string. **8M**

(b) Show that content free language are closed under the property of closure and union. **5M**

Q.9.(a) Construct turing machine for : **8M**

$f(a, b) = a + b$, if $a > b$ and $f(a, b) = 0$, if $a < b$

(b) Explain halting problem in Turing machine. **6M**

OR

Q.10.(a) Design TM to find 2's complement of given number. **7M**

(b) Explain all types of Turing machine. **7M**

Q.11.(a) Show that $f(x, y) = x + y$ is primitive recursive function. **6M**

(b) Define Ackermann's function.

Compute A (1, 1), A (2, 1), A (2, 2). **7M**

OR

Q.12.(a) Explain the properties of recursively enumer-able language. Give relation between recursive and recursive enumerable language. **7M**

(b) Decide whether the following (A, B) pairs have a solution or not? If yes give a solution, if no why? **6M**

(i) $A = \{b, bab^3, ba\}$ $B = \{b^3, ba, a\}$.

(ii) $A = \{01, 1, 1\}$ $B = \{01^2, 10, 1^2\}$.

VBD

SUMMER EXAM. - 2019

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Explain following in detail with example **6M**

(i) Type - 0 grammar

(ii) Type - 1 grammar.

(iii) Type - 2 grammar

(iv) Type - 3 grammar

Ans. **See P.1-11, Q.22.**

Q.1.(b) Prove the following theorem by induction : **7M**

$$\sum_{i=1}^{n} \frac{1}{i(i+1)} = \frac{n}{n+1}$$ is true for all $n \geq 0$.

Ans. **See P.PS-67, Q.1(b).**

Q.2.(a) If R ={(1, 2), (2, 3), (1, 4), (4, 2), (3, 4)}, obtain R^+ and R^*. **7M**

Ans. **See P.PS-68, Q.2(b).**

Q.2.(b) Explain following properties of relation with example : **6M**

(i) Reflexive

(ii) Antisymmetric.

(iii) Transitive

Ans. **See P.1-5, Q.4.**

(ii) Antisymmetric :

An anti-symmetric relation is one in which for any ordered pair (x, y) in R, the order pair (y, x) must NOT be in R, unless x = y.

So far instance the binary relation R_1 = {(2, 2), (2, 4), (3, 2)} is anti-symmetric. But if we add the ordered pair (4, 2) to get R_2 = {(2, 2), (2, 4), (3, 2), (4, 2)} then R_2 is no longer anti-symmetric, because both (2, 4) and (4, 2) are in relation.

Informally, a relation is anti-symmetric if it has no ordered pair and its reverse in it (unless x = y).

[Antisymmetric means that the only way for both aRb and bRa to hold is if a = b]

UNIT - II

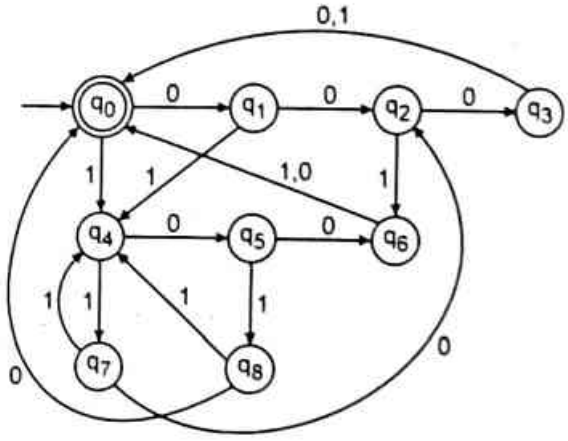
(Total Marks : 28)

Q.3.(a) **Construct DFA to accept the set of all string such that every block of four consecutive symbols contain at least two 0's over $(0, 1)^*$.** **8M**

Ans. Solution is a complete binary tree that works like a probability tree.

i.e. 0000, 0001, 0011, 0101, 1010, 1100, 1001, 0110.

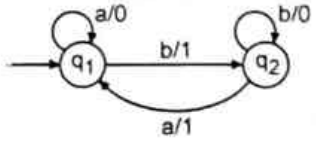
∴ Add 3 different states. At the last state if it was true it returns to the first original state.



T. T.

Q \ Σ	0	1
→*q_0	q_1	q_4
q_1	q_2	q_4
q_2	q_3	q_6
q_3	q_0	q_0
q_4	q_5	q_7
q_5	q_6	q_8
q_6	q_0	q_0
q_7	q_2	q_4
q_8	q_0	q_4

Q.3.(b) Convert the following Mealy machine into Moore machine : **6M**



Ans. Transition table for mealy machine :

Present state	a		b	
	State	Output	State	Output
$\rightarrow q_1$	q_1	0	q_2	1
q_2	q_1	1	q_2	0

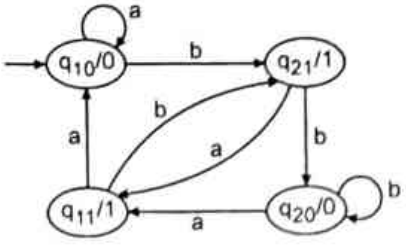
From the given table, q_1 and q_2 associate with two outputs 0 and 1 so split q_1 and q_2

Present state	a		b	
	State	Output	State	Output
$\rightarrow q_{10}$	q_{10}	0	q_{21}	1
q_{11}	q_{10}	0	q_{21}	1
q_{20}	q_{11}	1	q_{20}	0
q_{21}	q_{11}	1	q_{20}	0

Equivalent Moore machine table is

Present state	Next state		Output
	a	b	
q_{10}	q_{10}	q_{21}	0
q_{11}	q_{10}	q_{21}	1
q_{20}	q_{11}	q_{20}	0
q_{21}	q_{11}	q_{20}	1

Transition diagram :



Q.4.(a) Differentiate between **7M**

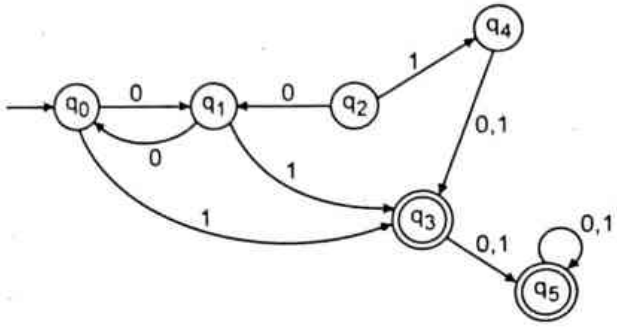
(i) DFA and NFA.

Ans. See P.2-13, Q.12.

(ii) Mealy and Moore machine.

Ans. See P.2-49, Q.64.

Q.4.(b) Construct the minimum state automata equivalent to following transition diagram shown below : **7M**



Ans. Transition table for the DFA :

State \ Σ	0	1
$\rightarrow * q_0$	q_1	q_3
q_1	q_0	q_3
q_2	q_1	q_4
$* q_3$	q_5	q_5
q_4	q_3	q_3
$* q_5$	q_5	q_5

Group 1 : $\{q_3, q_5\}$ = set of final state

Group 2 : $\{q_0, q_1, q_2, q_4\}$ = set of non final state.

Step 1 : $\{q_0, q_1, q_2, q_4\}$

(i) $\{q_0, q_1\}$

	0	1
q_0	q_1	q_3
q_1	q_0	q_3

$\therefore q_0 \equiv q_1$

(ii) $\{q_0, q_2\}$

	0	1
q_0	q_1	q_3
q_2	q_1	q_4

Different group

$\therefore q_1 \neq q_2$

(iii) $\{q_0, q_4\}$

	0	1
q_0	q_1	q_3
q_4	q_3	q_3

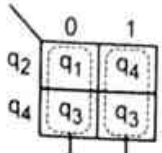
Different group

$\therefore q_0 \neq q_4$

New combination of groups are

$\{q_1, q_5\}, \{q_0, q_1\} \{q_2, q_4\}$

Step 2 : (i) $\{q_2, q_4\}$



Different group

$\therefore q_2 \neq q_4$

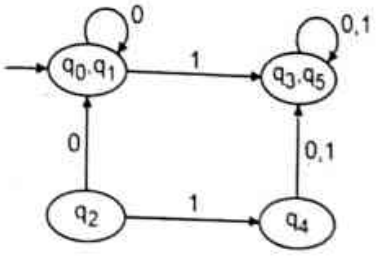
∴ Final combination of groups are

$\{q_3, q_5\}, \{q_0, q_1\} \{q_2\}, \{q_4\}$

Transition table :

State / Σ	0	1
→ $\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_3, q_5\}$
q_2	$\{q_0, q_1\}$	q_4
$\{q_3, q_5\}$	$\{q_3, q_5\}$	$\{q_3, q_5\}$
q_4	$\{q_3, q_5\}$	$\{q_3, q_5\}$

Minimized transition graph :



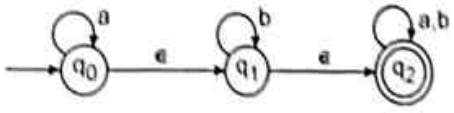
UNIT - III

(Total Marks : 26)

Q.5.(a) Construct left linear and right linear grammar for following expression p = a* b* (a + b)*. **7M**

Ans. P= a* b* (a + b)*

Transition graph :



Let $q_0 = A_0, q_1 = A_1$ and $q_2 = A_2$

$G = (\{A_0, A_1, A_2\}, \{a, b\}, P, A_0)$

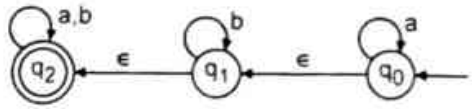
∴ Right linear grammar is

$A_0 \rightarrow a A_0 |$

$A_1 \rightarrow b A_1 | \epsilon$

$A_2 \rightarrow a A_2 | b A_2 | a | b$

∴ Transition graph for left linear grammar is



∴ Left linear grammar is

$A_0 \rightarrow A_0 a$

$A_1 \rightarrow A_1 b | \epsilon$

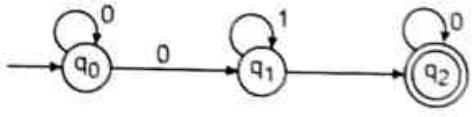
$A_2 \rightarrow A_2 a | A_2 b | a | b$

Q.5.(b) Construct NFA with and without E-transition equivalent to following regular expression :

$R = 10 + (0 + 11) 0^* 1$ **6M**

Ans. **See P.3-18, Q.22.**

Q.6.(a) Construct regular expression for the following automata : **6M**



Ans. Equation for all states

Note : Considering i/p ∈ for transition $q_1 \rightarrow q_2$

$q_0 = q_0 0 + \epsilon$ (1)

$q_1 = q_0 0 + q_1 1$ (2)

$q_2 = q_1 \epsilon + q_2 0$ (3)

$\therefore q_0 = q_0 0 + \epsilon$

By Arden's theorem, we get

$q_0 = \epsilon 0^*$

$q_0 = 0^*$ (4)

Put equations (4) value in equations (2), we get

$q_1 = 0^* 0 + q_1 1$

∴ By Arden's theorem, we get

$q_1 = 0^* 01^*$ (5)

Put value of equation (5) in equation (3), we get

$q_2 = 0^* 01^* + q_2 0$

∴ By Arden's theorem, we get

$q_2 = 0^* 01^* 0^*$

Q.6.(b) Show that following language are not regular **7M**

(i) L = { a^p /p is a prime number}

Ans. See P.3-53, Q.83.

(ii) L = {0^n 1^n | n ≥ 1}

Ans. See P.3-54, Q.84.

UNIT - IV
(Total Marks : 26)

Q.7.(a) Construct PDA that accept the language

L = {$a^n b^n c^n$ | n ≥ 0} . **8M**

Ans. L = {$a^n b^n c^n$ | n ≥ 0}

Analysis :

(1) The language is fixcel.

(2) It consists of equal number of a, b and c

(3) The string starts with a end with c

Valid string :

L = {a b c, aabbcc, ...}

Transition :

(1) $\delta(q_0, a, Z_0) = (q_0, aZ_0)$

(2) $\delta(q_0, a, q) = (q_0, aa)$

(3) $\delta(q_0, b, a) = (q_1, \wedge)$

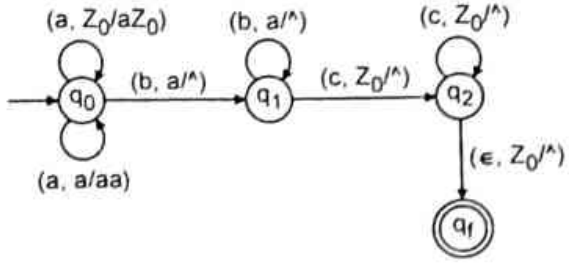
(4) $\delta(q_1, b, a) = (q_1, \wedge)$

(5) $\delta(q_1, c, Z_0) = (q_2, \wedge)$

(6) $\delta(q_2, c, Z_0) = (q_2, \wedge)$

(7) $\delta(q_2, \in, Z_0) = (q_f, \wedge)$

Transition diagram :



Q.7.(b) Explain modal of PDA and its acceptance by stack and acceptance by final state **5M**

Ans. See P.4-4, 5, 18, Q.1, 2, 28.

Q.8.(a) Construct PDA for the language **8M**

L = {WCW^R / W ∈ {a, b}*} , R : Reverse string

Ans. See P.4-15, Q.22.

Q.8.(b) Show that content free language are closed under the property of closure and union **5M**

Ans. See P.4-26, Q.40.

UNIT - V
(Total Marks : 28)

Q.9.(a) Construct turing machine for : **8M**

f (a, b) = a + b, if a > b and f (a, b) = 0, if a < b

Ans. See P.5-38, Q.47.

Q.9.(b) Explain halting problem in Turing machine. **6M**

Ans. See P.5-28, Q.27.

Q.10.(a) Design TM to find 2's complement of given number **7M**

Ans. See P.5-37, Q.45.

Q.10.(b) Explain all types of Turing machine. **7M**

Ans. See P.5-21, Q.19.

UNIT - VI
(Total Marks : 26)

Q.11.(a) Show that f (x, y) = x + y is primitive recursive function **6M**

Ans. See P.6-29, Q.45.

Q.11.(b) Define Ackermann's function.

Compute A (1, 1), A (2, 1), A (2, 2) **7M**

Ans. See P.6-38, Q.68.

Q.12.(a) Explain the properties of Recursively enumer-able language. Give Relation between recursive and recursive enumerable language **7M**

Ans. See P.6-7, 8, 9, Q.2, 4, 5, 6.

Q.12.(b) Decide whether the following (A, B) pairs have a solution or not? If yes give a solution, if no why? **6M**

(i) A = {b, bab^3, ba} B = {b^3, ba, a} .

Ans. See P.6-16, Q.19.

(ii) A = { 01, 1, 1} B = {01^2, 10, 1^2} .

Ans. See P.6-17, Q. 21(a).

WINTER EXAMINATION - 2019
B.E. IV SEM. (COMPUTER SCIENCE ENGG.) (CBS)
THEORETICAL FOUNDATIONS OF COMPUTER SCIENCE

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	28 M
Unit - III	26 M
Unit - IV	26 M
Unit - V	28 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

WINTER EXAM. - 2019
QUESTION PAPER

Q.1.(a) Explain Pigeon-Hole principle. **3M**

(b) With the help of mathematical induction, prove that: **8M**

(i) $1.1! + 2.2! + 3.3! + \ldots + 4.4! = (4+1)! - 1$

(ii) $1^2 + 2^2 + 3^2 + \ldots + 4^2 = \frac{4 \times (4+1)(24+1)}{6}$

(c) Which type of grammar is more powerful and why? **2M**

OR

Q.2.(a) Explain: **8M**

(1) Context free Grammar.

(2) Context sensitive Grammar and also justify why context free grammar is context sensitive but vice versa is not possible?

(b) Type 3 grammar is used in: **2M**

(a) Turing machine

(b) Finite automata.

(c) Push down automata.

(d) Linear bounded automata.

Explain in brief.

(c) Define the following terms with suitable example: **3M**

(i) Prefix

(ii) Suffix

(iii) Substring

Q.3.(a) Design a DFA for a string of decimal digits that are divisible by 3. **7M**

(b) Design a mealy machine to count number of occurrence of "aba" and convert the resultant machine into Moore machine $\Sigma = \{a, b\}$ **7M**

OR

Q.4.(a) State and explain applications of finite automata. **6M**

(b) Construct mealy machine to compute 2's complement of binary number. Also convert this machine to Moore machine. **8M**

Q.5.(a) Construct NFA with ϵ - transitions equivalent to the given Regular Expression. **7M**

$R = 10 + (0 + 11)0^*1$

(b) Construct context free Grammar for following: **6M**

(i) $L = \{a^n b^m c^m d^n / m, n \geq 1\}$

(ii) $L = \{a^n b^m / 2n \leq m \leq 3n\}$

OR

Q.6.(a) Find the minimum state DFA for following Regular Expression: **7M**

$ab^*(ab)^*(a + b)b^* + b$

(b) Check whether the given grammar is ambiguous or not. **6M**

$S \rightarrow a / Sa / bSS / SbS$

Q.7.(a) Explain the PDA and its acceptance by stack and acceptance by final state **7M**

(b) Prove that: $L = \{0^i 1^j / i > j\}$ is not context free language **6M**

OR

Q.8.(a) Design push down automata for: **8M**

$L = WW^R / W \in \{a, b\}^*\}$

(b) Differentiate between NPDA and DPDA. **5M**

Q.9.(a) Define the model of Turing Machine and explain it's types. **7M**

(b) Design Turing Machine for: **7M**

$L = \{a^n b^n c^n / n \geq 1\}$

OR

Q.10.(a) Design Turing machine to convert 111 into 101. **7M**

(b) Explain the concept of linear bounded automata. **7M**

Q.11.(a) What is post correspondence problem? Explain modified PCP. **6M**

(b) Solve using Ackermann function: **7M**

A(1, 1), A(2, 1), A(2, 2), A(2, 3)

OR

Q.12.(a) Explain: **6M**

(i) Halting problem.

(ii) Recursive function

(iii) Decidability and solvability.

(b) Explain the properties of Recursively Enumerable languages. **7M**

WINTER EXAM. - 2019

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Explain Pigeon-Hole principle **3M**

Ans. See P.1-10, Q.16.

Q.1.(b) With the help of mathematical induction, prove that: **8M**

(i) $1.1! + 2.2! + 3.3! + \ldots + 4.4! = (4+1)! - 1$

(ii) $1^2 + 2^2 + 3^2 + \ldots\ldots + 4^2 = \frac{4\times(4+1)(2\times4+1)}{6}$

Ans. (i) See P.1-8, Q.12.

(ii) $1^2 + 2^2 + 3^2 + \ldots\ldots + 4^2 = \frac{4\times(4+1)(2\times4+1)}{6}$

Basic step:

$\therefore$ For $4 = 1$

$\therefore$ L.H.S. $= 1^2 = 1$

$$\text{R.H.S.} = \frac{1\times(1+1)(2\times1+1)}{6} = \frac{2(3)}{6} = \frac{6}{6} = 1$$

$\therefore$ L.H.S. = R.H.S.

Induction hypothesis:

Assume that for some $4 \geq 0$

$$1^2 + 2^2 + \ldots\ldots + 4^2 = \frac{4(4+1)(2\times4+1)}{6} \quad \ldots$$

Induction step:

$\therefore 4 = 4 + 1$

$\therefore$ L.H.S. $1^2 + 2^2 + \ldots\ldots + 4^2 + (4+1)^2$

$\therefore$ L.H.S. $1^2 + 2^2 + 3^2 + \ldots\ldots + 4^2 + (4+1)^2$

$$\frac{4\times(4+1)(2\times4+1)}{6} + (4+1)^2$$

$$= \frac{4\times(4+1)(2\times4+1) + 6(5)^2}{6}$$

$$= \frac{4\times(5)(8+1)+150}{6}$$

$$= \frac{20\times9+150}{6} = \frac{330}{6} = 55 \quad \ldots (1)$$

$$\text{R.H.S.} = \frac{(4+1)(4+1+1)(2\times(4+1)+1)}{6}$$

$$= \frac{(5\times6)(2\times5+1)}{6}$$

$$= \frac{30\times11}{6} = \frac{330}{6} = 55 \quad \ldots (2)$$

From (1) & (2)

$\therefore$ L.H.S. = R.H.S.

Q.1.(c) Which type of grammar is more powerful and why? **2M**

Ans. Unrestricted grammars are more powerful. In fact they can generate the whole class of recursively enumerable (R.E.) language i.e. those that are accepted by turing machine.

Turing machines are more powerful than grammars in early to see since you can write a program to generate words from the grammar and any to program can be Turned into Turing machines.

Q.2.(a) Explain: **8M**

(1) Context free Grammar

(2) Context sensitive Grammar and also justify why context free grammar is context sensitive but vice versa is not possible?

Ans. See P.1-13, Q.22(2), (3) & P.PS-111, Q.2(a).

Q.2.(b) Type 3 grammar is used in: **2M**

(a) Turing machine

(b) Finite automata

(c) Push down automata

(d) Linear bounded automata

explain in brief.

Ans. Type 3 grammar is used in Finite Automata.

Type 3 grammar generated regular language. Type 3 grammars must have a single non-terminal on the left hand side and a right hand side consisting of a single terminal or single terminal followed by a single non terminal.

The production must be in the form $X \to a$ or $X \to aY$ where $X, Y \in V$ (Non terminal) and $a \in T$ (Terminal).

The rule $S \to \in$ is allowed if S does not appear on the right side of any rule.

Q.2.(c) Define the following terms with suitable example **3M**

(i) Prefix

(ii) Suffix

(iii) Substring

Ans. **See P.1-4, Q.2 & PS-153, Q.1(b).**

UNIT - II

(Total Marks : 28)

Q.3.(a) Design a DFA for a string of decimal digits that are divisible by 3. **7M**

Ans. **See P.2-32, Q.47.**

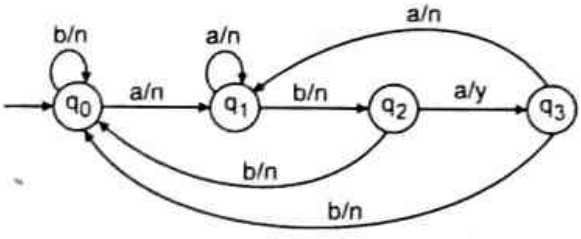
Q.3.(b) Design a mealy machine to count number of occurrence of "aba" and convert the resultant machine into Moore machine $\Sigma = \{a, b\}$. **7M**

Ans. Let us assume that the o/p alphabet

$\Delta = \{y, n\}$

'y' is yes will be output if string is accepted and 'n' is 'no' will be the output if string is not accepted.

∴ Transition diagram for mealy machine



Transition table for mealy machine

Present state	Next state			
	a		b	
	State	O/P	State	O/P
$\rightarrow q_0$	q_1	n	q_0	n
q_1	q_1	n	q_2	n
q_2	q_3	y	q_0	n
q_3	q_1	n	q_0	n

∴ o/p of state q_0 = n

o/p of state q_1 = n

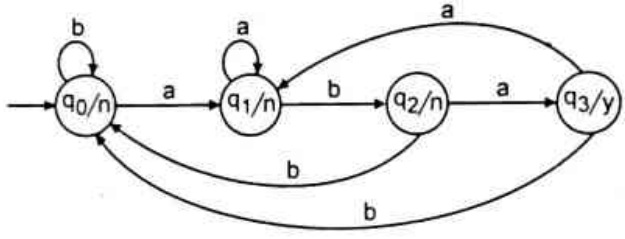
o/p of state q_2 = n

o/p of state q_3 = y

∴ Transition diagram for Moore machine

Present state	Next state		Output
	a	b	
$\rightarrow q_0$	q_1	q_0	n
q_1	q_1	q_2	n
q_2	q_3	q_0	n
q_3	q_1	q_0	y

Transition diagram for Moore machine is



Q.4.(a) State and explain applications of finite Automata. **6M**

Ans. **See P.2-8, 10, Q.1, 4.**

Q.4.(b) Construct mealy machine to compute 2's complement of binary number. Also convert this machine to Moore machine. **8M**

Ans. **See P.PS-182, Q.3(b).**

UNIT - III

(Total Marks : 26)

Q.5.(a) Construct NFA with ε - transitions equivalent to the given Regular Expression. **7M**

$R = 10 + (0 + 11)0^*1$

Ans. **See P.3-18, Q.22.**

Q.5.(b) Construct context free Grammar for following

(i) $L = \{a^n b^m c^m d^n / m, n \geq 1\}$

(ii) $L = \{a^n b^m / 2n \leq m \leq 3n\}$ **6M**

Ans. **See P.3-63, Q.110(ii) & PS-163, Q.5(b)(ii).**

Q.6.(a) Find the minimum state DFA for following Regular Expression **7M**

$ab^*(ab)^*(a+b)b^*+b$

Ans. **See P.PS-114, Q.6(a).**

Q.6.(b) Check whether the given grammar is ambiguous or not

$S \rightarrow a/Sa/bSS/SbS$ **6M**

Ans. **See P.PS-126, Q.6(a).**

UNIT - IV

(Total Marks : 26)

Q.7.(a) Explain the PDA and its acceptance by stack and acceptance by final state **7M**

Ans. **See P.4-4, 18, Q.1, 28.**

Q.7.(b) Prove that: $L = \{0^i 1^j / i > j\}$ is not context free language. **6M**

Ans. Suppose L be a context free language and n be the pumping lemma constant

Let $Z = 0^i 1^j$

Where $i > j$ and $|z| \geq n$

By pumping lemma Z can be written as-

$Z = u\,v\,w\,x\,y$ such that

$|vx| \geq 1$

$|u\,w\,x| \leq n$ and

For all $i \geq 0$ $uv^i wx^i y$ is in L

If v and x consists of only 0's than uwy (the string $uv^i wx^i y$ with $i = 0$) has n only 1 but less than n 0's, since $|vx| \geq 1$. This uwy is not of the from $0^i 1^j$ satisfying the condition $i > j$. This gives the contradiction

Similarly, we get contradiction in the case where v and x consist only 1's of v and x consist of only 1's then uwy (the strng $uv^i wx^i y$ with $i = \infty$) has n '0' but less than n 1's since $|vx| \geq 1$.

This uwy is not of the form $0^i 1^j$ satisfying the condition $i > j$. This gives contradiction. Hence we conduct that L is no context free language

Q.8.(a) Design push down automata for **8M**

$L = \{WW^R / W \in (a, b)^*\}$

Ans. **See P.4-15, Q.23.**

Q.8.(b) Differentiate between NPDA and DPDA **5M**

Ans. **See P.4-7, Q.7.**

UNIT - V

(Total Marks : 28)

Q.9.(a) Define the model of Turing Machine and explain it's types **7M**

Ans. **See P.5-7, 21, Q.1, 19.**

Q.9.(b) Design Turing Machine for **7M**

$L = \{a^n b^n c^n / n \geq 1\}$

Ans. **See P.5-15, Q.10.**

Q.10.(a) Design Turing machine to convert 111 into 101 **7M**

Ans. **See P.5-17, Q.13.**

Q.10.(b) Explain the concept of linear bounded automata **7M**

Ans. **See P.5-34, Q.40.**

UNIT - VI

(Total Marks : 26)

Q.11.(a) What is post correspondence problem? Explain modified PCP **6M**

Ans. **See P.6-15, 18, Q.18, 23.**

Q.11.(b) Solve using Ackermann's function

A(1, 1), A(2, 1), A(2, 2), A(2, 3) **7M**

Ans. **See P.6-38, Q.68 & PS-17, Q.12(a).**

Q.12.(a) Explain **6M**

(i) Halting problem

Ans. **See P.6-19, Q.25.**

(ii) Recursive function

Ans. **See P.PS-79, Q.12(b)(i).**

(iii) Decidability and solvability

Ans. **See P.6-13, Q.15(a) & PS-89, Q.11(b).**

Q.12.(b) Explain the properties of Recursively Enumerable languages **7M**

Ans. **See P.6-7, 8, 9, Q.2, 4, 5, 6.**

Notes:-

WINTER EXAMINATION - 2019
B.E. IV SEM. (COMPUTER TECHNOLOGY) (CBS)
THEORY OF COMPUTATION

Statistical Analysis

Unit Number	Marks
Unit - I	26 M
Unit - II	28 M
Unit - III	26 M
Unit - IV	28 M
Unit - V	26 M
Unit - VI	26 M
Questions covered in VBD	160 M (100 %)

WINTER EXAM. - 2019
QUESTION PAPER

Q.1.(a) Calculate R^* and R^+ if $R = \{(1, 1), (1, 2), (2, 3), (3, 1), (1, 4), (2, 4)\}$ on a set $A = \{1, 2, 3, 4\}$ **7M**

(b) Define following properties on a relation **3M**

(i) Reflexive relation

(ii) Transitive relation

(iii) Equivalence relation

(c) State and explain pigeon hole principle **3M**

OR

Q.2.(a) Solve the following relation using principle of mathematical induction **7M**

(i) $1.2.3 + 2.3.4 + \text{------} + n(n+1)(n+2) = n(n+1)(n+2)(n+3)/4$

(ii) $1 + 2 + \text{--------} + n = (n^2 + n)/2$

(b) Write a short note on Chomsky Hierarchy. **6M**

Q.3.(a) Construct a DFA equivalent to the NFA $m = (\{p, q, r, s\}, \{0, 1\}, s, p, \{s\})$ where s is given below: **8M**

Σ / State	0	1
→ p	p, q	p
q	r	r
r	s	ϕ
s*	s	s

(b) Differentiate between: **6M**

(i) NFA and DFA.

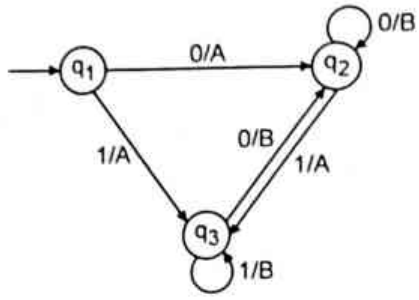
(ii) Moore and Mealy machine

OR

Q.4.(a) Minimize the given deterministic finite automata, state A is initial state and state D, F are final state **8M**

Present state	Next state	
	0	1
→ A	B	D
B	A	D
C	B	E
D*	F	F
E	D	D
F*	F	F

(b) Construct a Moore machine equivalent to nearly machine given as: **6M**



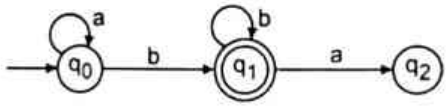
Q.5.(a) Change the following grammar into CNF

$S \rightarrow a\,AbB$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$ **3M**

(b) Construct RE from transition diagram given below: **7M**



(c) What is an ambiguous grammar? Show that the given grammar is an ambiguous grammar. **3M**

$S \rightarrow aSbS \mid b\,SaS \mid \epsilon$

OR

Q.6.(a) Construct right linear grammar accepting the regular language by left linear grammar given below: **6M**

$S \rightarrow Ca \mid Bb$

$A \rightarrow Bb$

$B \rightarrow Ba \mid b$

Also mention the steps.

(b) Find equivalent greibach normal form (GNF) representation for the following CFG. **7M**

$S \rightarrow AA \mid 0$

$A \rightarrow SS \mid 1$

Q.7.(a) Construct PDA for the following language **7M**

$L = \{0^n\,1^m 0^{m+n} \mid m, n > 1\}$

(b) Construct a PDA M, equivalent to the following context free grammar

$S \rightarrow 0BB$

$B \rightarrow 0S \mid 1S \mid 0$

Test weather 0110^4 is N (M) **7M**

OR

Q.8.(a) Construct an NPDA that accepts the language generated by the grammar.

And show the acceptance rejection of string

W = "aababab" **7M**

(b) Construct PDA for $h = \{wcw^R \mid w \in (a, b)^*$ and C in special input symbol **7M**

Q.9.(a) Design turing machine that replace all occurrences of 111 by 101 from sequences of 0's and 1's **7M**

(b) What are different type's of turing machine? Explain each types in brief **6M**

OR

Q.10.(a) Design a turing machine that computes a function f(m, n) = m + n i.e. addition of two integers. **7M**

(b) Design T.M. for language to find 2's complement of binary number. **6M**

Q.11.(a) Define Ackerman's function compute. **7M**

A (1, 1), A (2, 1), A (1, 2) and A (2, 2)

(b) Explain the properties of recursive enumerable language. **6M**

OR

Q.12.(a) Show that f(x, y) = x + y is a primitive recursive function. **6M**

(b) What do you mean by PCP? Decide whether the following (A, B) pair have a solution or not

A (b, bab^3, ba), B = (b^3, ba, a) **7M**

WINTER EXAM. - 2019

SOLUTION

(Unit-Wise)

UNIT - I

(Total Marks : 26)

Q.1.(a) Calculate R^* and R^+ if R = {1, 1), (1, 2), (2, 3), (3, 1), (1, 4), (2, 4)} on a set A = {1, 2, 3, 4} **7M**

Ans. See P.PS-10.3, Q.2(b).

Q.1.(b) Define following properties on a relation **3M**

(i) Reflexive relation

(ii) Transitive relation

(iii) Equivalence relation.

Ans. See P.1-5, Q.4.

Q.1.(c) State and explain pigeon hole principle **3M**

Ans. See P.1-10, Q.16.

Q.2.(a) Solve the following relation using principle of mathematical induction **7M**

(i) 1.2.3 + 2.3.4 + ——— + n (n + 1) (n + 2)

= n (n + 1) (n + 2) (n + 3) / 4

Ans. See P.1-9, Q.15.

(ii) $1 + 2 + \text{———} + n = (n^2 + n)/2$

Ans. See P.1-7, Q.10.

Q.2.(b) Write a short note on Chomsky Hierarchy **6M**

Ans. See P.1-11, Q.22.

UNIT - II

(Total Marks : 28)

Q.3.(a) Construct a DFA equivalent to the NFA m = ({p, q, r, s}, {0, 1}, s, p, {s}) where s is given below **8M**

State \ Σ	0	1
→ p	p, q	p
q	r	r
r	s	ϕ
s*	s	s

Ans. See P.2-15, Q.17.

Q.3.(b) Differentiate between **6M**

(i) NFA and DFA

(ii) Moore and Mealy machine

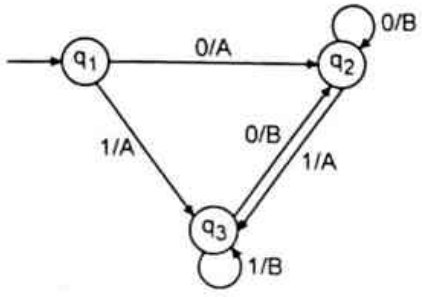
Ans. See P.2-13, 49, Q.12, 64.

Q.4.(a) Minimize the given deterministic finite automata, state A is initial state and state D, F are final state **8M**

Present state	Next state	
	0	1
→ A	B	D
B	A	D
C	B	E
D*	F	F
E	D	D
F*	F	F

Ans. **See P.PS-139, Q.4(a).**

Q.4.(b) Construct a Moore machine equivalent to Mealy machine given as : **6M**



Ans. **See P.2-56, Q.78.**

UNIT - III

(Total Marks : 26)

Q.5.(a) Change the following grammar into CNF: **3M**

S → aAbB

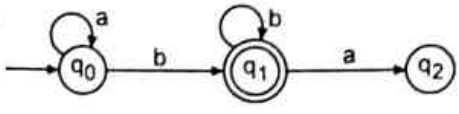
A → aA|a

B → bB|b

Ans. **See P.3-82, Q.164.**

Note: 'q' as 'a'

Q.5.(b) Construct RE from transition diagram given below: **7M**



Ans. **See P.PS-170, Q.5(b).**

Q.5.(c) What is an ambiguous grammar? Show that the given grammar is an ambiguous grammar. **3M**

S → aSbS | bSaS| ∈

Ans. **See P.3-66, 69, Q.120, 127.**

Q.6.(a) Construct right linear grammar accepting the regular language by left linear grammar given below: **6M**

S→ Ca | Bb

A→ Bb

B→ Ba | b

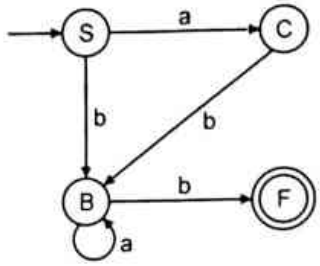
Also mention the steps.

Ans. S→ Ca | Bb

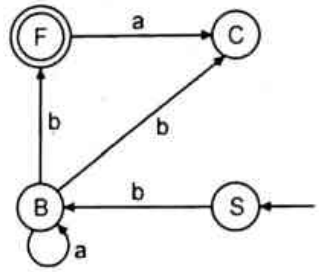
A→ Bb

B→ Ba | b

Transition graph too left linear grammar



Exchange the position of initial and final state and reverse the direction of all transitions.



∴ Right linear grammar is

S → bB

B → aB|bC|b

C → a

Q.6.(b) Find equivalent greibach normal form (GNF) representation for the following CFG. **7M**

S→AA | 0

A→ SS | 1

Ans. **See P.3-89, Q.178.**

UNIT - IV

(Total Marks : 28)

Q.7.(a) Construct PDA for the following language **7M**

$L = \{0^n 1^m 0^{m+n} \mid m, n > 1\}$

Ans. **See P.4-11, Q.14.**

Q.7.(b) Construct a PDA M, equivalent to the following context free grammar

$S \rightarrow 0BB$

$B \rightarrow 0S \mid 1S \mid 0$

Text weather 010^4 is N (M) **7M**

Ans. See P.PS-201, Q.8(b).

Q.8.(a) Construct on NPDA that accepts the language generated by the grammar: **7M**

And show the acceptance rejection of string

W = "aababab"

Ans. The grammar is not is GNF therefore, first we obtain an equivalent grammar M GNF.

$S \rightarrow aSSS/aCb$

$C_b \rightarrow b$

∴ Therefore NPDA equivalent to the given grammar is

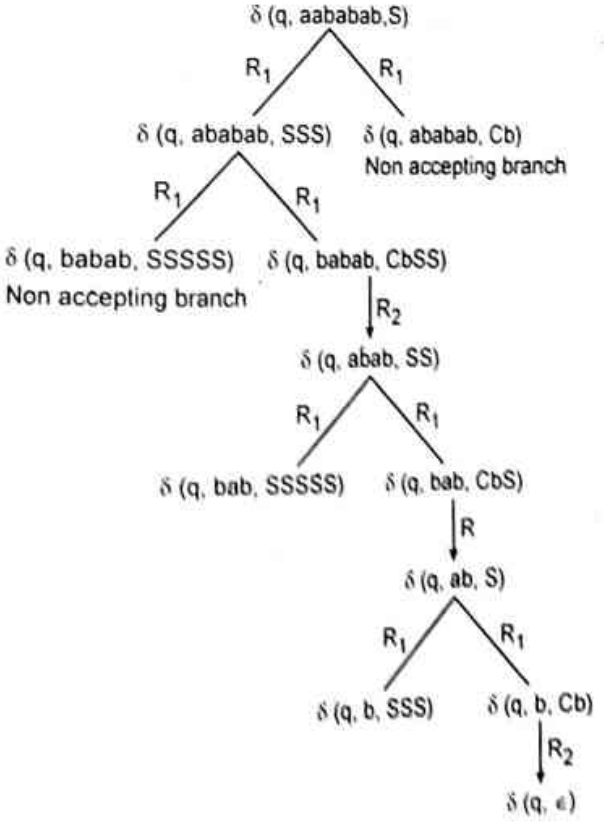
$M = (\{q\}, \{a, b\}, \{S, C_b\}, \delta, q, S, \phi)$

where δ is defined by following rules:

$\delta(q, a, S) = \{(q, SSS), (q, C_b)\}$ because $S \rightarrow aSSS$ and $S \rightarrow aCb$ ... (1)

$\delta(q, b, C_b) = \{(q, \in)\}$ because $C_b \rightarrow b$ ..... (2)

Let us check string "aababab"



Input accepted as stack is empty

Q.8.(b) Construct PDA for $h = \{wcw^R \mid w \in (a, b)^*$ and C in special input symbol **7M**

Ans. See P.4-15, Q.22.

UNIT - V

(Total Marks : 26)

Q.9.(a) Design turing machine that replace all occurrences of 111 by 101 from sequences of 0's and 1's. **7M**

Ans. See P.5-17, Q.13.

Q.9.(b) What are different type's of turing machine? Explain each types in brief. **6M**

Ans. See P.5-21, Q.19.

Q.10.(a) Design a turing machine that computes a function f(m, n) = m + n i.e. addition of two integers. **7M**

Ans. See P.5-38, Q.47.

Q.10.(b) Design T.M. for language to find 2's complement of binary number. **6M**

Ans. See P.5-37, Q.45.

UNIT - VI

(Total Marks : 26)

Q.11.(a) Define Ackerman's function compute

A (1, 1), A (2, 1), A (1, 2) and A (2, 2). **7M**

Ans. See P.6-38, Q.68 & P.PS-17, Q.12(a).

Q.11.(b) Explain the properties of recursive enumerable language. **6M**

Ans. See P.6-7, 8, 9, Q.2, 4, 5, 6.

Q.12.(a) Show that f(x, y) = x + y is a primitive recursive function **6M**

Ans. See P.6-29, Q.45.

Q.12.(b) What do you mean by PCP? Decide whether the following (A, B) pair have a solution or not.

$A (b, bab^3, ba), B = (b^3, ba, a)$ **7M**

Ans. See P.6-15, 16, Q.18, 19.

Notes:-

APPENDIX

INTRODUCTION TO THEORY OF AUTOMATON

Q.1. What is theory of Automaton?

Ans. Theory of Automaton:

- Automata theory (also known as Theory of Computation) is a theoretical branch of Computer Science and Mathematics, which mainly deals with the logic of computation with respect to simple machines, referred to as automata.
- Automata enables scientists to understand how machines compute the functions and solve problems. The main motivation behind developing automata theory was to develop methods to describe and analyze the dynamic behaviour of discrete systems.

  Automata is originated from the word "Automaton" which is closely related to "Automation".
- Now, let's understand the basic terminologies, which are important and frequently used in the Theory of Computation.

  Symbol :

  Symbol(often also called character) is the smallest building block, which can be any alphabet, letter, or picture.

  a, b, c, 0, 1, ...

  Alphabets (Σ) :

  Alphabets are a set of symbols, which are always finite.

  $\Sigma = \{0, 1\}$ is an alphabet of binary digits

  $\Sigma = \{0, 1, ..., 9\}$ is an alphabet of decimal digits

  $\Sigma = \{a, b, c\}$

  $\Sigma = \{A, B, C, ..., Z\}$

  String :

  A string is a finite sequence of symbols from some alphabet. A string is generally denoted as w and the length of a string is denoted as |w|

  Note: Σ^* is a set of all possible strings (often power set(need not be unique here or we can say multiset), of string, so this implies that language is a subset of Σ^*.

  An empty string is a string with zero occurrences of symbols, represented as ε.

  Number of Strings (of length 2)

  that can be generated over the alphabet {a, b} - - -

  aa

  a b

  b a

  bb

  Length of String |w| = 2

  Number of Strings = 4

  Conclusion:

  For alphabet {a, b} with length n, a number of strings can be generated = 2n.

Q.2. What are the advantages of Theory of Computation?

Ans. Advantages of Theory of Computation:

There are many advantages of the Theory of Computation. Some of them are listed below:

(i) Theory of Computation deals with how efficiently any algorithm would solve any computational problem. Also, abstract machines are introduced in the Computational theory, which is defined mathematically. Hence, the algorithms would not need to change every time any physical hardware gets enhanced.

(ii) There is a massive amount of work that has been made possible in the portion of NLP (Natural Language Processing) that involves the construction of FSMs (Finite State Machines), also known as FSA (Finite State Automata).

(iii) Theory of Computation has helped in many fields such as Cryptography, Design and Analysis of Algorithm, Quantum Calculation, Logic within Computer Science, Computational Difficulty, Randomness within Calculation and Correcting Errors in Codes.

(iv) A computational model can cope with complexity in ways that verbal arguments cannot, resulting in satisfactory answers for what would otherwise be ambiguous hand-wavy arguments. Furthermore, computational models can manage complexity at several levels of analysis, allowing data from various levels to be integrated and connected.

Q.3. What are the Applications of Automata?

Ans. **Applications of Automata:**

(1) **Finite Automata (FA):**

(i) For the designing of lexical analysis of a compiler.

(ii) For recognizing the pattern using regular expressions.

(iii) For the designing of the combination and sequential circuits using Mealy and Moore Machines.

(iv) Used in text editors.

(v) For the implementation of spell checkers.

(2) **Push Down Automata (PDA):**

(i) For designing the parsing phase of a compiler (Syntax Analysis).

(ii) For implementation of stack applications.

(iii) For evaluating the arithmetic expressions.

(iv) For solving the Tower of Hanoi Problem.

(3) **Linear Bounded Automata (LBA):**

(i) For implementation of genetic programming.

(ii) For the constructing syntactic parse trees for semantic analysis of the compiler.

(4) **Turing Machine (TM):**

(i) For solving any recursively enumerable problem.

(ii) For understanding complexity theory.

(iii) For the implementation of neural networks.

(iv) For the implementation of Robotics Applications.

(v) For the implementation of artificial intelligence.

REGULAR SET

Q.4. Define regular set. Explain it.

Ans. **Regular Set:**

A regular set is a set of strings for which there exists some finite automata accepting that set i.e. if R is a regular set then R = L (m) for some finite automata m. Similarly, if m is a finite automata then L (m) is always a regular set.

For example :

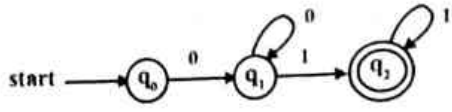


Fig. 1

Since there exists a finite automata the set L is a regular set.

There are many operations on languages that preserve regular sets, in the sense that the operations applied to regular sets results in regular sets. For example, the union of two regular sets is a regular sets. If L_1 and L_2 are two regular set, then L_1 & L_2 is also regular. Similarly, the concatenation of regular sets is a regular set and Kleene closure of regular sets is regular.

INTRODUCTION OF DCFL AND DPDA

Q.5. Show that DCFL's are closed under complementation operation. Also, explain the difficulties encountered in claiming the result.

Ans.

- Given a DPDA M if we interchange final and non-final states then resulting DPDA will accept the complement of L (M).
- There are two difficulties that complicate the above approach. The first difficulty is that the original DPDA might never move beyond some point on an input string, because on reading input w either it reaches an ID from which no move is possible or it makes an infinity of moves on ϵ-input and never uses another input symbol. In either case, the DPDA does not accept any input with w as a prefix and thus a DPDA accepting the complement should accept every string with prefix w. However, if we simply changed final and nonfinal states, the resulting DPDA still would not move beyond w and therefore would not accept strings with prefix w.
- The second difficulty is due to the fact that after seeing a sentence x, the DPDA may make several moves on ϵ input. The DPDA may be in the final states after some of these moves and in nonfinal states after others.
- In this case, interchanging the final and nonfinal states results in the DPDA still accepting x.

- DCFL's are closed under inverse homomorphism and intersection with a regular set.
- The DCFL's are not closed under homomorphism, union, concatenation or Kleen closure

Q.6. Prove that the complement of a DCFL is a DCFL.

Ans. Proof :

Let $M' = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be a DPDA.

Let $M' = (Q, \Sigma, \Gamma, \delta', q_0, Z_0, F')$

be a DPDA simulating M, where

$Q' = \{ [q, k] \mid q \text{ is in } Q \text{ and } k = 1, 2, \text{ or } 3 \}$

Let $F' = \{q, 3,\} \mid q \text{ in } Q\}$ and let

$$q_0 = \begin{cases} [q_0, 1] & \text{if } q_0 \text{ is in F;} \\ [q_0, 2] & \text{if } q_0 \text{ is not in F} \end{cases}$$

The purpose of k in [q, k] is to record, between true inputs, whether or not M has entered an accepting state. If M has entered an accepting state since the last true input, then k = 1. If M has not entered an accepting state since the last true input, then k =2. If k - 1 when M reads a true input symbol, then M' simulates the move of M and changes k to 1 or 2, depending on whether the new state M is or is not in F. If k = 2, M' first changes k to 3 and then simulates the move of M, changing k to 1 or 2, depending on whether the new state of M is or is not in F. Thus, δ' is defined as follows, for q and p in Q and a in Σ.

(1) If $\delta(q, \epsilon, Z) = (p, \gamma)$ the for k = 1 or 2

$\delta'(p, k,]q, Z) = ([p\ k'], \gamma)$

where k' = 1 if k = 1 or p is in F; otherwise k' = 2.

(2) If $\delta(q, a, Z) = (p, \gamma)$ for a in ϵ then

$\delta'(q, Z,]\epsilon, Z) = ([q\ 3], Z)$

and $\delta'([q, 1], a Z) = \delta'([q, 3], a, Z) = ([p, k], \gamma)$

where k =1 or 2 for p in F or not in F, respectively.

We claim that L (M) is the complement of L (M). Suppose that $a_1 a_2 \ldots a_n$ is in L (M). Then M enters an accepting state after using a_n as input. In that case, the second component of the state M will be 1 before it is possible for M to use a true input after a_n. Therefore M' does not accept (enter a state whose second component is 3) while a_n was the last true input used.

Q.7. Let M (Q, E, Γ, δ, q_0, Z_0, F) be a DPDA. Let B be any subset of Q Γ, that is, pairs of state and stack symbols.

Define $L = \{w \mid (q_0, w, Z_0) \overset{*}{\underset{M}{\vdash}} (q, \epsilon, Z\gamma \text{ for some } (q, Z) \text{ in } B\}$

then L is a DCFL.

Ans. Proof : We define a DPDA M, accepting L as follows.

$M' = (Q', \Sigma, \Gamma, \delta', q_0, Z_0, F')$

where in $Q' = \{q, q', q'' \mid q \ Q\}$ and

$F' = \{q'' \mid q \text{ in } Q\}$

M' makes the same moves as M, except M moves from an unprinted state to a singly primed state and then on ϵ–input, moves back to the corresponding unprimed state, either directly or through a doubly primed version. The latter case applies only if the pair of state and top symbol of the stack is in B.

Formally,

(1) If $\delta(q, a, Z) = (P, \gamma)$ then $\delta'(q, a, Z) = (P', \gamma)$

(2) $\delta'(q', \epsilon, Z)\ \delta(q, \epsilon, Z) = (q, Z)$ provided (q, Z) is not in B.

(3) $\delta'(q', \epsilon, Z) = (q'', Z)$ and $\delta(q'', \epsilon, Z) = (q, Z)$

if (q, Z) is in B.

Q.8. Prove that :

Let L be a DCFL and R a is regular set.

Then L/R is a DCFL.

Ans. Proof:

Let L = L (M) for M a DPDA that always scans its entire input.

Let R = L (A) for finite automaton A. Suppose

$M = (Q_M, \Sigma, \Gamma, \delta_M, q_0, Z_0, F_M)$ and $A = (Q_A, \Sigma, \delta_A, P_0, F_A)$

then let

$M' = (Q_M, \Sigma, \Gamma \times \Delta, \delta, q_0, [Z_0, \mu_0], F_M$

be π (M, A) the predicting machine for M and A. Let B be the subset of $Q_M \times (\Gamma \times \Delta)$ containing all $(q,[Z,\mu])$ such that (q, P_0) is in μ.

Then $L_1 = \{x \mid (q_0, x, [Z_0, \mu_0]) \overset{*}{\underset{M}{\vdash}} (q, \epsilon [Z, \mu] \gamma)$ and (q, P_0) is in μ $\}$ is a DCFL and

$L_1 = \{x \mid$ for some w in Σ^*, $(q_0, x, Z_0) \overset{*}{\underset{M}{\vdash}} (q, \epsilon Z \gamma')$ and $(q, w, Z \gamma) \overset{*}{\underset{M}{\vdash}} (S, \epsilon, \beta)$

where, S is in F_M. γ' is the first components of γ and $\delta_A (P_0, w)$ is in F_A $\}$

Equivalently,

$L_1 = \{x$ for some w in Σ^*, xw is in L (M) and w is in L (A)$\}$.

That is, $L_1 = L/R$. Thus L/R is a DCFL.

Q.9. If L is a DCFL, then MIN (L) & MAX (L) are DCFL's.

Ans. Let $M = (Q_M, \Sigma, \Gamma, \delta_M, q_0, Z_0, F_M)$ be DPDA that accepts L and always scans its entire input. Modify M to make no move in a final state. Then the resulting DPDA M_1 accepts MIN (L). In proof, if w is in MIN (L), then let

$$(q_0, w, Z_0) = I_0 \underset{M_1}{\vdash} I_1 \underset{M_1}{\vdash} \cdots \underset{M_1}{\vdash} I_m$$

be the sequence of ID's entered by M, where $I_m = (q, \epsilon, \gamma)$ for some γ and q is in F_M. Furthermore, since w is in MIN (L), none of $I_0, I_1, \ldots I_{m-1}$ has an accepting state. Thus (1) is also computation of M_1, so w is in L (M).

Conversely, if

$$(q_0, w, Z_0) = I_0 I_1 \underset{M_1}{\vdash} I_1 \underset{M_1}{\vdash} \cdots \underset{M_1}{\vdash} I_m \text{ is}$$

an accepting computation of M_1, then none of $I_0, I_1, \ldots I_{m-1}$ has an accepting state. Thus w is in MIN (L).

For MAX we must use the predicting machine. Let $A = (Q_A, \Sigma, \delta_A, P_0, F_A)$ be the simple FA of Fig. 34 accepting Σ^+. Let $M = (Q_M, \Sigma, \Gamma X \Delta, \delta, q_0, [Z_0, \mu_0], F_M)$ be π (M, A). Let $B = \{(q, [Z, \mu]) \mid q$ is in F_M and (q, P_0) is not in μ $\}$. Then $L_1 = \{x \mid (q_0, x, z_0) \mid \overset{*}{\underset{M}{\vdash}} (q, \epsilon, \gamma)$ For some q in F_M and for no $w \neq \epsilon$ does $(q, w, \gamma) \overset{*}{\underset{M}{\vdash}} (\Sigma, \epsilon, \beta)$ for S in F_M $\}$ is a DCFL. But L_1 = MAX (L), so MAX (L) is a DCFL.

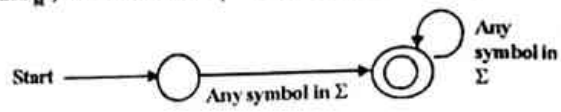


Fig. The automation A

DECISION PROPERTIES OF DCFL's

Q.10. If L is a DCFL and R is a regular set, then prove that following problems are decidable:

(i) Is L = R? (ii) Is $R \subseteq L$?

(iii) Is $\overline{L} = \phi$? (iv) Is $\overline{L}$ a CFL? (v) Is L regular?

Ans.

(i) L = R, if and only if $L_1 = (L \cap \overline{R}) \cup (\overline{L} \cap R)$ is empty. Since the DCFL's are effectively closed under complementation and intersection with a regular set and since the CFL's are effectively closed under union, L_1 is a CFL and emptiness for CFL's is decidable.

(ii) $R \subseteq L$, if and only if $\overline{L} \cap R = \phi$. Since $\overline{L} \cap R$ is a CFL, $\overline{L} \cap R = \phi$ is decidable.

(iii) Since the DCFL's are effectively closed under complementation, $\overline{L}$ is a DCFL and hence $\overline{L} = \phi$ is decidable.

(iv) The property $\overline{L}$ is a CFL is trivial for DCFL's and hence is decidable.

(v) Regularity for DCFL's is decidable.

UNDECIDABLE PROPERTIES OF DCFL's

Q.11. If L and L' are arbitrary DCFL's, then prove that the following problems are undecidable:

(a) Is $L \cap L' = \phi$?

(b) Is $L \subseteq L'$?

(c) Is $L \cap L'$ a DCFL?

(d) Is $L \cap L'$ a CFL?

(e) Is $L \cup L'$ a DCFL ?

Ans.

(a) This is undecidable whether $L(M) = \phi$. Since, DCFL's are closed under complement and $L \subseteq L'$, if and only if $L \cap L' = \phi$.

(b) This follows from (a)

To prove (c), (d) and (e), modify each TM M to make at least two moves before accepting. Then $L_1 \cap L_2$ is either a finite set or its not a CFL depending on whether L (M) is finite. Thus, decidability of (c) or (d) would imply decidability of finiteness for L (M), a known undecidable property. Since DCFL's are closed under complementation, deciding whether is $L \cup L'$ is a DCFL is equivalent to deciding if $L \cap L'$ is a DCFL.

Thus (e) follows from (c).

Q.12. If L is an arbitrary CFL and is undecidable then show whether L is a DCFL or not.

Ans. Let L is the CFL of invalid computations of an arbitrary TM M that makes at least two moves on every input. L is regular and hence a DCFL if and only if M accepts a finite set.

Q.13. Define : (a) Right sentential form

(b) Handle

(c) Viable prefix

(d) Augmented grammar.

Ans.

(a) Right Sentential Form :

Given a context free grammar G. If $S \rightarrow^* \alpha$, where α may contain nonterminals as well as terminals; then α is said to be a sentential form of G. A right sentential form is a sentential form G, that can be derived by rightmost derivation.

(b) Handle :

A handle of a right sentential form γ is a production $A \rightarrow \beta$ and position of β in γ, where the string β will be found and replaced by A to produce the previous right sentential form in the right most derivation of γ

i.e. if $S \rightarrow \alpha A \beta \rightarrow \alpha \gamma \beta$,

then $A \rightarrow \gamma$, is a handle of $\alpha \gamma \beta$ in the position following α.

Consider the grammar

$E \rightarrow E + E \mid E * E \mid id$

and the rightmost derivation

$E \rightarrow E + E \rightarrow E + E * E \rightarrow E + E * id$
$\rightarrow E + id * id \rightarrow id + id * id$

Given below are the handles of the sentential forms occurring in the above derivation.

Sentential form	Handle
id + id * id	$E \rightarrow id$, at the position preceding +
E + id* id	$E \rightarrow id$, at the position following +
E + E * id	$E \rightarrow id$, at the position following *
E+ E * E	$E \rightarrow E*E$, at the position following +
E + E	$E \rightarrow E+E$, at the position preceding the endmarker

(c) Viable Prefix :

Viable prefix of a right sentential form is that prefix of sentential form which may contain handle but no symbol to the right of the handle and therefore if a finite state machine recognizing viable prefixes of the right sentential forms is constructed it can be used to guide the selection of handle in the shift reduce parser.

(d) Augmented Grammar :

- To construct a DFA recognizing the viable prefixes we make use of augmented grammar, which is defined as follows.

  If G = (V, T, P, S) is a given grammar then the augmented grammar will be $G_1 (V \cup \{S_1\}, T, P \cup \{S_1 \rightarrow S\})$ i.e. we add a unit production $S_1 \rightarrow S$ to the grammar G and make S_1 to be the new start symbol, the resulting grammar will be an augmented grammar. The purpose of augmenting the grammar is to make it explicitly clear when to accept the string and stop parsing it will be when the parser is on the verge of carrying out reduction using $S_1 \rightarrow S$.

- Consider the transition $\delta (A \rightarrow \alpha, B \beta, \epsilon) = B \rightarrow \gamma$.

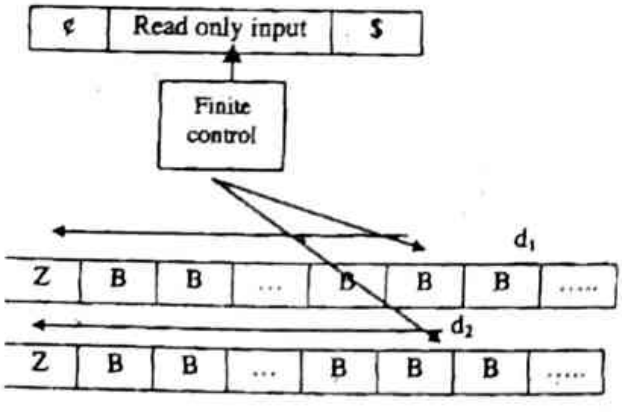
This transition is required because, if the current state is $A \to \alpha, B\beta$ that means we have not yet seen a string derivable from the non terminal β and $\beta \to \gamma$ being a production of the grammar, unless we see γ we will not get B, therefore we have walk on the path recognizing γ, which requires entering into the state identified $\beta \to \gamma$ without consuming any input symbol.

COUNTER MACHINE

Q.13. Write a short note on Counter Machines.

Ans. Counter Machines:

- Counter machines are off line. Turing machines whose storage tapes are semi-infinite and whose tape alphabets contain only two symbols, Z and B (blank). Symbol Z, which serves as a bottom of stack marker, appears initially on the cell scanned by the tape head and may never appear on any other cell. An integer i can be stored by moving the tape head i cells to the right of Z. A stored number can be incremented or decremented by moving the tape head right or left. We can test whether a number is zero by checking whether Z is scanned by the head, but whether two numbers are equal cannot be directly tested.
- Fig. shows a counter machine.



¢ and $ are used as end markers on the input. Here Z is the non-blank symbol on each type. A counter machine can be described by the state, the input tape contents, the position of the input head and the distance of the storage heads from the symbol Z (d_1 and d_2 as shown in) are called as counts on the tapes. The counter machine can really store a count on each tape and tell if that count is zero.

Q.14. Prove that a four-counter machine can simulate an arbitrary turing machine.

Ans.

- Let a stack have k-1 tape symbols, $Z_1, Z_2, \ldots, Z_{k-1}$. Then we can represent the stack $Z_{i_1}, Z_{i_2} \ldots Z_{i_m}$ uniquely by the count in base k

  $j = i_m + ki_{m-1} + k^2 i_{m-2} + \ldots + k^{m-1} i_1$.

  Those integers whose base-k representation contains the digit 0 do not represent the stack.
- Let us assume that the symbol Z_r is pushed onto the top of the stack $Z_{i_1}, Z_{i_2} \ldots Z_{i_m}$. The count associated with $Z_{i_1}, Z_{i_2} \ldots Z_{i_m}$ Z_r is $jk + r$. For getting this new count, the counter machine repeatedly moves the head of the first counter one cell to the left and the head of the second k cells to the right. When the head of the first counter reaches the nonblank symbol, the second counter will hold the count jk. Then r is added to the count.
- If Z_{i_m} i.e. the top symbol of the stack is popped, j should be replaced by $[j/k]$, the integer part of j/k. We repeatedly keep on decrementing the count on the first counter by k and adding one to the second count. When the first count is zero, the second count will be $[j/k]$. If the count j is stored on one counter, the four counter machine can copy j to another counter, computing j mod k in its finite control.

EQUIVALENCE OF CFL AND PDA

Q.15. Prove that if L is a context-free language, then there exists a PDA M such that L = N (M).

Ans. Proof : We assume that ϵ is not in L (G). The reader may modify the construction for the case where ϵ is in L (G). Let G = (V, T, P, S) be a context free grammar in Greibach normal form generating L. Let

$M = (\{q\}, T, V, \delta, q, S, \phi)$,

where δ (q, a A) contains (q, γ) whenever $A \to a\gamma$ is in P.

The PDA M simulates leftmost derivations of G. Since G is in Greibach normal form, each sentential form in a leftmost derivation consists of a string of terminals x followed by a string of variables

α M stores the suffix α of the left sentential form on its stack after processing the prefix x. Formally we show that $S \overset{*}{\Rightarrow} x\alpha$ by a leftmost derivation if and only if $(q, x, S) \overset{*}{\underset{M}{\vdash}} (q, \epsilon, \alpha)$ ——(1)

First we suppose that $(q, x, S) \overset{i}{\vdash} (q, \epsilon, \alpha)$ and show by induction on i that $S \overset{*}{\Rightarrow} x\alpha$. The basis, i = 0, is trivial since $x = \epsilon$ and $\alpha = S$. For the induction, suppose $i \geq 1$, and let x = ya. Consider the next-to-last step.

$(q, ya, S) \overset{i-1}{\vdash} (q, a, \beta) \vdash (q, \epsilon, \alpha)$ ——(2)

If we remove a from the end of the input string in the first ID's of the sequence we discover that $(q, y, S) \overset{i-1}{\vdash} (q, \epsilon, \beta)$ since a cannot affect on the moves of M until it is actually consumed from the input. By the inductive hypothesis $S \overset{*}{\Rightarrow} y\beta$ The move $(q, a, \beta) \vdash (q, \epsilon, \alpha)$ implies that $\beta = A\gamma$ for some A in V, $A \to a\eta$ is a production of G and $\alpha = \eta\gamma$. Hence

$S \overset{*}{\Rightarrow} y\beta \Rightarrow ya\eta\gamma = x\alpha$,

and we conclude the "if" portion of equation (1)

Now suppose that $S \overset{i}{\Rightarrow} x\alpha$ by a leftmost derivation. We show by induction on i that $(q, x, S) \overset{*}{\vdash} (q, \epsilon, \alpha)$. The basis, i = 0, is again trivial. Let $i \geq 1$ and suppose

$S \Rightarrow yA\gamma \Rightarrow ya\eta\gamma$,

where, x = ya and $\alpha = \eta\gamma$. By the inductive hypothesis $(q, y, S) \overset{*}{\vdash} (q, \epsilon, A\gamma)$ and thus $(q, ya, S) \vdash (q, a, A\gamma)$. Since $A \to a\eta$ is a production, it follows that $\delta(q, a, A)$ contains (q, η). Thus

$(q, x, S) \overset{*}{\vdash} (q, a, A\gamma) \vdash (q, \epsilon, \alpha)$,

and the "only if" portion of equation 1 follows.

To conclude the proof, we have only to note that with $\alpha = \epsilon$ says $S \overset{*}{\Rightarrow} x$ if and only if $(q, x, S) \overset{*}{\vdash} (q, \epsilon, \epsilon)$. That is, x is in L (G) if and only if x is in N (M).

Q.16. Prove that if L is in N (M) for some PDA M, then L is a context - free language.

Ans. Let M be the PDA $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, \phi)$

Let G = (V, Σ, P, S) be a context - free grammar where V is the set of objects of the form [q, A, P], q and p in Q, and A in Γ, plus the new symbol S. P is the set of productions

(1) $S \to [q_0, Z_0, q]$ for each q in Q.

(2) $[q, A, q_{m+1}] \to a\,[q_1, B_1, q_2]\,[q_2, B_2, q_3] \ldots [q_m, B_m, q_{m+1}]$

for each $q, q_1, q_2, \ldots q_{m+1}$ in Q, each a in $\Sigma \cup \{\epsilon\}$ and $A, B_1, B_2, \ldots B_m$ in Γ, such that $\delta(q, a, A)$ contains $(q_1, B_1B_2 \ldots B_m)$

(If m = 0, then the production is $[q, A, q_1] \to a$)

To understand the proof it helps to know that the variables and productions of G have been defined in such a way that a leftmost derivation in G of a sentence x is a simulation of the PDA M when fed the input x. In particular, the variables that appear in any step of a leftmost derivation in G correspond to the symbols on the stack of M at a time when M has seen as much of the input as the grammar has already generated. Put another way, the intention is that [q, A, p] derive x if and only if x causes M to erase an A from its stack by some sequence of moves beginning in state q and ending in state p.

To show that L (G) = N (M), we prove by induction on the number of steps in a derivation of G or number of moves of M that

$[q, A, p] \overset{*}{\underset{G}{\Rightarrow}} x$ if and only if $[q, x, A] \overset{*}{\underset{M}{\vdash}} (p, \epsilon, \epsilon)$ ——(1)

First we show by induction on i that if $[q, x, A] \overset{i}{\vdash} (p, \epsilon, \epsilon)$ then $[q, A, p] \overset{*}{\Rightarrow} x$. If i = 1, then $\delta(q, x, A)$ must contain (p, ϵ). Here x is ϵ or a single input symbol.

Thus $[q, A, p] \to x$ is a production of G.

Now suppose i > 1. Let x = ay and

$(q, ay, A) \vdash (q_1, y, B_1B_2 \ldots B_n) \overset{i-1}{\vdash} (p, \epsilon, \epsilon)$

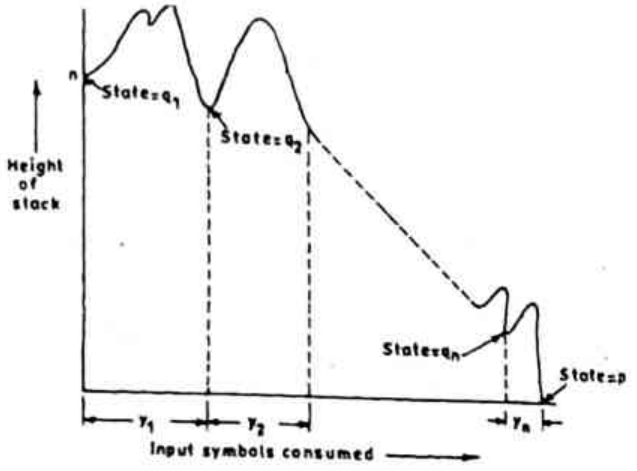


Fig. Height of stack as a function of input consumed.

Recalling the original move $(q, ay, A) \vdash (q_1, y, B_1 B_2 \ldots B_n)$, we know that

$[q, A, p] \Rightarrow a [q_1, B, q_2] [q_2, B_2, q_3] \ldots [q_n, B_n, q_{n+1}]$.

so $[q, A, p] \stackrel{*}{\Rightarrow} ay_1 y_2 \ldots y_n = x$

Now suppose $[q, A, p] \stackrel{i}{\Rightarrow} x$. We show by induction on that $(q, x, A) \vdash^* (p, \epsilon, \epsilon)$. The basis i = 1, is immediate, since $[q, A, p] \rightarrow x$ must be a production of G and therefore $\delta (q, x, A)$ must contain (p, ϵ). Note x is ϵ or in Σ here.

For induction, suppose

$[q, A, p] \Rightarrow a [q_1, B_1, q_2] \ldots [q_n, B_n, q_{n+1}] \stackrel{i-1}{\Rightarrow} x$

where, $q_{n+1} = p$. Then we may write $x = ax_1 x_2 \ldots x_n$, where $[q_j, B_j, q_{j+1}] \stackrel{*}{\Rightarrow} x_j$ for $1 \le j \le n$, with each derivation taking fewer than i steps. By the inductive hypothesis, $(q_j, x_j, B_j) \vdash (q_{j+1}, \epsilon, \epsilon)$ for $1 \le j \le n$. If we insert $B_{j+1} \ldots B_n$ at the bottom of each stack in the above sequence of ID's we see that

$$(q_j, x_j, B_j B_{j+1} \ldots B_n) \vdash^* (q_{j+1}, \epsilon, B_{j+1} \ldots B_n) \quad \text{---- (2)}$$

From the first step in the derivation of x from [q, A, p], we know that

$(q, x, A) \vdash (q_1, x_1 x_2 \ldots x_n, B_1 B_2 \ldots B_n)$

is a legal move of M, so from this move and eq. (2) for j = 1, 2, ..., n $(q, x, A) \vdash (p, \epsilon, \epsilon)$ follows.

The proof concludes with the observation that eq. (1) with $q = q_0$ and $A = Z_0$ says

$[q_0, Z_0, p] \stackrel{*}{\Rightarrow} x$, if and only if $(q_0, x, Z_0) \vdash^* (p, \epsilon, \epsilon)$

This observation, together with rule (1) of the construction of G, says that

$S \stackrel{*}{\Rightarrow} x$, if and only if $(q_0, x, Z_0) \vdash^* (p, \epsilon, \epsilon)$ for some state p.

That is, x is in L(G), if and only if x is in N (M).

Q.17. Let $M = (\{q_0, q_1\}, \{0, 1\}, \{X, Z_0\}, \delta, q_0, Z_0, \phi)$, where δ is given by

$\delta (q_0, 0, Z_0) = \{(q_0, XZ_0)\}$, $\delta (q_1, 1, X) = \{(q_1, \epsilon)\}$,

$\delta (q_0, 0, X) = \{(q_0, XX)\}$, $\delta (q_1, \epsilon, X) = \{(q_1, \epsilon)\}$,

$\delta (q_0, 0, Z_0) = \{(q_1, XZ_0)\}$, $\delta (q_1, 1, X) = \{(q_1, \epsilon)\}$,

Construct an equivalent CFG.

Ans. To construct a CFG G = (V, T, P, S) generating N (M) let

$V = \{[S, q_0, X, q_0], [q_0, X, q_1], [q_1, X, q_0], [q_1, X, q_1],$

$[q_0, Z_0, q_0], [q_0, Z_0, q_1], [q_1, Z_0, q_0], [q_1, Z_0, q_1]\}$,

and T = {0, 1}. To construct the set of productions easily, we must realize that some variables may not appear in any derivation starting from the symbol S. Thus we can save some effort if we start with the productions for S, the add productions only for those variables that appear on the right of some productions already in the set. The productions for S are

$S \rightarrow [q_0, Z_0, q_0]$

$S \rightarrow [q_0, Z_0, q_1]$

Next we add productions for the variable $[q_0, Z_0, q_0]$. These are

$[q_0, Z_0, q_0] \rightarrow 0 [q_0, X, q_0] [q_0, Z_0, q_0]$

$[q_0, Z_0, q_0] \rightarrow 0 [q_0, X, q_1] [q_1, Z_0, q_0]$

These productions are required by $\delta (q_0, 0, Z_0) = \{(q_0, XZ_0)\}$

Next, the production for $[q_0, Z_0, q_1]$ are

$[q_0, Z_0, q_1] \rightarrow 0 [q_0, X, q_0] [q_0, Z_0, q_1]$

$[q_0, Z_0, q_1] \to 0\ [q_0, X, q_1]\ [q_1, Z_0, q_1]$

These are also required by $\delta\ (q_0, 0, Z_0) = \{(q_0, XZ_0)\}$ The productions for the remaining variables and the relevant moves of PDA are.

(1) $[q_0, X, q_0] \to 0\ [q_0, X, q_0]\ [q_0, X, q_0]$

$[q_0, X, q_0] \to 0\ [q_0, X, q_1]\ [q_1, X, q_0]$

$[q_0, X, q_1] \to 0\ [q_0, X, q_0]\ [q_0, X, q_1]$

$[q_0, X, q_1] \to 0\ [q_0, X, q_1]\ [q_1, X, q_1]$

since $\delta\ (q_0, 0, X) = \{(q_0, XX)\}$

(2) $[q_0, X, q_1] \to 1$ since $\delta\ (q_0, 1, X) = \{(q_1, \epsilon)\}$.

(3) $[q_1, Z_0, q_1] \to \epsilon$ since $\delta\ (q_1, \epsilon, Z_0) = \{(q_1, \epsilon)\}$.

(4) $[q_1, X, q_1] \to \epsilon$ since $\delta\ (q_1, \epsilon, X) = \{(q_1, \epsilon)\}$.

(5) $[q_1, X, q_1] \to 1$ since $\delta\ (q_1, 1, X) = \{(q_1, \epsilon)\}$.

It should be noted that there are no productions for the variables $[q_1, X, q_0]$ and $[q_1, Z_0, q_0]$. As all productions for $[q_0, X, q_0]$ and $[q_0, Z_0, q_0]$ have $[q_1, X, q_0]$ or $[q_1, Z_0, q_0]$ on the right, no terminal string can be derived from $[q_0, X, q_0]$ or $[q_0, Z_0, q_0]$ either. Deleting all productions involving one of these four variables on either the right or left, we end up with the following productions.

$S \to [q_1, Z_0, q_1]$ $[q_1, Z_0, q_1] \to \epsilon$.

$[q_0, Z_0, q_1] \to 0\ [q_0, X, q_1]\ [q_1, Z_0, q_1]$ $[q_1, X, q_1] \to \epsilon$.

$[q_1, X, q_1] \to 0\ [q_0, X, q_1]\ [q_1, X, q_1]$, $[q_1, X, q_1] \to 1$

$[q_0, X, q_1] \to 1$

Q.18. Give a grammar for language N(M) where $M = (\{q_0, q_1\}, \{0, 1\}, \{Z_0, X\}, \delta, q_0, Z_0, \phi)$ and δ is given by

$\delta\ (q_0, 1, Z_0) = \{(q_0, X, Z_0)\}$,

$\delta\ (q_0, 1, X) = \{(q_0, X, X)\}$,

$\delta\ (q_0, 0, X) = \{(q_1, X)\}$,

$\delta\ (q_0, \epsilon, Z_0) = \{(q_1, \epsilon)\}$,

$\delta\ (q_1, 1, X) = \{(q_1, \epsilon)\}$,

$\delta\ (q_1, 0, Z_0) = (q_0, Z_0)$.

Ans. To construct CFG $G = (V, T, P, S)$ generating N (M) let

$V = \{S, [q_0, Z_0, q_0], [q_0, Z_0, q_1], [q_0, X, q_0], [q_0, X, q_1], [q_1, Z_0, q_1], [q_1, X, q_0], [q_1, X, q_1]\}$

We start with the productions for S, then add productions only for those variables that appear on the right of some productions already in the set. The production for S are

P_1 $S \to [q_0, Z_0, q_0]$

P_2 $S \to [q_0, Z_0, q_1]$

Next we add productions for the variable $[q_0, Z_0, q_0]$ which are required by $\delta\ (q_0, 1, Z_0) \Rightarrow \{(q_0, XZ_0)\}$

$P_3 \to [q_0, Z_0, q_0] \to 1\ [q_1, X, q_0]\ [q_0, Z_0, q_0]$

$P_4 \to [q_0, Z_0, q_0] \to 1\ [q_0, X, q_1]\ [q_1, Z_0, q_0]$

$P_5 \to [q_0, Z_0, q_1] \to 1\ [q_0, X, q_0]\ [q_0, Z_0, q_1]$

$P_6 \to [q_0, Z_0, q_1] \to 1\ [q_0, X, q_1]\ [q_1, Z_0, q_1]$

The productions required by

$\delta\ (q_0, 1, X) \to \{(q_0, X, X)\}$ are as follows

$P_7 \to [q_0, X, q_0] \to 1\ [q_0, X, q_0]\ [q_0, X, q_0]$

$P_8 \to [q_0, X, q_0] \to 1\ [q_0, X, q_1]\ [q_1, X, q_0]$

$P_9 \to [q_0, X, q_1] \to 1\ [q_0, X, q_0]\ [q_1, X, q_1]$

$P_{10} \to [q_0, X, q_1] \to 1\ [q_0, X, q_1]\ [q_1, X, q_1]$

The productions required by

$\delta\ (q_0, 0, X) \to \{(q_1, X)\}$ are

$P_{11} \to [q_0, X, q_0] \to 0\ [q_1, X, q_0]$

$P_{12} \to [q_0, X, q_1] \to 0\ [q_1, X, q_1]$

The production

$\delta\ (q_0, \epsilon, Z_0) \to \{(q_1, \epsilon)\}$ gives

$P_{13} \to [q_0, Z_0, q_0] \to \epsilon$

$\delta\ (q_1, 1, X) \to \{(q_1, \epsilon)\}$ gives

$P_{14} \to [q_1, X, q_1] \to 1$

$\delta\ (q_1, 0, Z_0) \to \{(q_0, Z_0)\}$ gives

$P_{15} \rightarrow [q_1, Z_0, q_0] \rightarrow 0\,[q_0, Z_0, q_0]$

$P_{16} \rightarrow [q_1, Z_0, q_1] \rightarrow 0\,[q_0, Z_0, q_1]$

$P_1 - P_{16}$ give the production in P. Hence G is the required CFG.

Q.19. Construct a PDA that is accepted by empty stack the language

$$L = \{a^m b^m c^n \mid m, n \geq 1\}$$

Hence determine the CFG that generates L.

Ans. The required PDA P is defined as follows

$P = (\{q_0, q_1\}, \{a, b, c\}, \{Z_0, Z_1\}, \delta, q_0, Z_0, q_1)$

Transition table for pushdown automata				
Move number	State	Input	Stack symbol	Move(s)
1	q_0	a	Z_0	$(q_0, Z_1 Z_0)$
2	q_0	a	Z_1	$(q_0, Z_1 Z_1)$
3	q_0	b	Z_1	(q_1, ϵ)
4	q_1	b	Z_1	(q_1, ϵ)
5	q_1	c	Z_0	(q_1, Z_0)
6	q_1	ϵ	Z_0	(q_1, ϵ)

Note that on reading a, we add Z_1 on the stack and on reading b we remove Z_1 and the state is changed. If the input is completely scanned and the stack symbol is Z_0, then it is removed by a ϵ-move.

Let us see the acceptability for the string aabbc. The sequence of moves in accepting the string is as follows

Move number	Resulting state	Unread Input	Stack
Initially	q_0	aabbc	Z_0
1	q_0	abbc	$Z_1 Z_0$
2	q_0	bbc	$Z_1 Z_1 Z_0$
3	q_1	bc	$Z_1 Z_0$
4	q_1	c	Z_0
5	q_1	ϵ	Z_0
6	q_1	—	—
accept			

(b) The transition function δ for pushdown automata

where δ is given by

$R_1 \rightarrow (q_0, a, Z_0) \rightarrow \{(q_0, Z_1, Z_0)\}$

$R_2 \rightarrow \delta(q_0, a, Z_1) \rightarrow \{(q_0, Z_1, Z_1)\}$

$R_3 \rightarrow \delta(q_0, b, Z_1) \rightarrow \{(q_1, \epsilon)\}$

$R_4 \rightarrow \delta(q_1, b, Z_1) \rightarrow \{(q_1, \epsilon)\}$

$R_5 \rightarrow \delta(q_1, C, Z_0) \rightarrow \{(q_1, Z_0)\}$

$R_6 \rightarrow \delta(q_1, \epsilon, Z_0) \rightarrow \{(q_1, \epsilon)\}$

To construct CFG G = (V, T, P, S)

Let

$V = \{S, [q_0, Z_0, q_0], [q_0, Z_0, q_1], [q_0, Z_1, q_0], [q_0, Z_1, q_1], [q_1, Z_0, q_0], [q_1, Z_0, q_1], [q_1, Z_1, q_0], [q_1, Z_1, q_1]\}$

The production in P are constructed as follows. The S-productions are

$P_1 = S \rightarrow [q_0, Z_0, q_0]$

$P_2 = S \rightarrow [q_0, Z_0, q_1]$

(1) $\delta(q_0, a, Z_0) = \{(q_0, Z_1, Z_0)\}$ gives

$P_3 = [q_0, Z_0, q_0] \rightarrow a\,[q_0, Z_1, q_0]\,[q_0, Z_0, q_0]$

$P_4 = [q_0, Z_0, q_0] \rightarrow a\,[q_0, Z_1, q_1]\,[q_1, Z_0, q_0]$

$P_5 = [q_0, Z_0, q_1] \rightarrow a\,[q_0, Z_1, q_0]\,[q_0, Z_0, q_1]$

$P_6 = [q_0, Z_0, q_1] \rightarrow a\,[q_0, Z_1, q_1]\,[q_1, Z_0, q_1]$

(2) $\delta(q_0, a, Z_1) = \{(q_0, Z_1 Z_1)\}$ gives

$P_7 = [q_0, Z_1, q_0] \rightarrow a\,[q_0, Z_1, q_0]\,[q_0, Z_1, q_0]$

$P_8 = [q_0, Z_1, q_0] \rightarrow a\,[q_0, Z_1, q_1]\,[q_1, Z_1, q_0]$

$P_9 = [q_0, Z_1, q_1] \rightarrow a\,[q_0, Z_1, q_0]\,[q_0, Z_1, q_1]$

$P_{10} = [q_0, Z_1, q_1] \rightarrow a\,[q_0, Z_1, q_1]\,[q_1, Z_1, q_1]$

(3) $P_{11} = (q_0, b, Z_1) \rightarrow b$ (since $\delta(q_0, b, Z_1) = \{q_1, \epsilon\}$

(4) $P_{12} = (q_1, b, Z_1) \rightarrow b$ (since $\delta(q_1, b, Z_1) = \{q_1, \epsilon\}$

(5) $\delta(q_1, C, Z_0) = \{(q_1, Z_0)\}$ gives

$P_{13} = [q_1, Z_0, q_0] \rightarrow C\,[q_1, Z_0, q_0]$

$P_{14} = [q_1, Z_0, q_1] \rightarrow C\,[q_1, Z_0, q_1]$

(6) $P_{15} = \delta(q_1, Z_0, q_1) \rightarrow \epsilon$ (since $\delta(q_1, \epsilon, Z_0) = \{q_1, \epsilon\}$

$P_1 - P_{15}$ give the productions in P. Hence G is the required CFG.

Q.20. Design a Non-deterministic Pushdown Automata (NPDA) for generating the string of type $b^n a^n$ where $n \geq 0$. Also write context free grammar CFG for it.

Ans. Non-deterministic Pushdown automata is defined as follows

$P = (\{q_0, q_1\}, \{a, b\}, (b, Z_0], \delta, q_0, Z_0, \phi)$

where δ is given by

$R_1 \rightarrow \delta(q_0, b, Z_0) = \{(q_0, b Z_0)\}$

$R_2 \rightarrow \delta(q_0, b, b) = \{(q_0, bb)\}$

$R_3 \rightarrow \delta(q_0, a, b) = \{(q_1, \epsilon)\}$

$R_4 \rightarrow \delta(q_1, a, b) = \{(q_1, \epsilon)\}$

$R_5 \rightarrow \delta(q_1, \epsilon, Z_0] = \{q_1, \epsilon\}$

R_1 is used to store b in the stack, if it is the first symbol of an input string. R_2 can be used to store a^n in the stack. When a is encountered for the first time in the input string, b is erased (in the stack) using R_3. Also NPDA makes the transition to state q_1. After processing the entire input string, if Z_0 remains in the stack, it can be erased using the ϵ move by R_5.

To construct CFG G = (V, T, P, S)

Let V = {S, $[q_0, Z_0, q_0]$ $[q_0, Z_0, q_1]$ $[q_1, Z_0, q_0]$ $[q_1, Z_0, q_1]$, $[q_0, b, Z_0]$, $[q_0, b, q_1]$, $[q_1, b, q_0]$ $[q_1, b, q_1]$ }

We start with production of for S then add production only for those variables that appear on the right of some production already in the set

The production for S are

$P_1 = S \rightarrow [q_0, Z_0, q_0]$

$P_2 = S \rightarrow [q_0, Z_0, q_1]$

(1) $\delta(q_0, b, Z_0) = (q_0, b, Z_0)\}$ gives

$P_3 = [q_0, Z_0, q_0] \rightarrow b\ [q_0, b, q_0]\ [q_0, Z_0, q_0]$

$P_4 = [q_0, Z_0, q_0] \rightarrow b\ [q_0, b, q_1]\ [q_1, Z_0, q_0]$

$P_5 = [q_0, Z_0, q_1] \rightarrow b\ [q_0, b, q_0]\ [q_0, Z_0, q_1]$

$P_6 = [q_0, Z_0, q_1] \rightarrow b\ [q_0, b, q_1]\ [q_1, Z_0, q_1]$

(2) $\delta(q_0, b, b) = \{(q_0, bb)\}$ gives

$P_7 = [q_0, b, q_0] \rightarrow b\ [q_0, b, q_0]\ [q_0, b, q_0]$

$P_8 = [q_0, b, q_0] \rightarrow b\ [q_0, b, q_1]\ [q_1, b, q_0]$

$P_9 = [q_0, b, q_1] \rightarrow b\ [q_0, b, q_0]\ [q_1, b, q_1]$

$P_{10} = [q_0, b, q_1] \rightarrow b\ [q_0, b, q_1]\ [q_1, b, q_1]$

(3) $P_{11} = [q_0, b, q_1] \rightarrow a$ (since $\delta(q_0, a, b) = \{q_1, \epsilon\}$

(4) $P_{12} = [q_1, b, q_1] \rightarrow a$ (since $\delta(q_1, a, b) = \{q_1, \epsilon\}$

(5) $P_{13} = [q_1, Z_0, q_1) \rightarrow \epsilon$ (since $\delta(q_1, \epsilon, Z_0) = \{q_1, \epsilon\}$

$P_1 - P_{13}$ gives the productions in P. Hence G is the required grammar

Q.21. Construct pushdown automata for the language $L = \{x c x^c \mid x \in (a, b)^*\}$. Hence determine CFG that generate L.

Ans. The pushdown automata P is defined as

$P = (\{q_0, q_1\}, \epsilon, \{a, b\}, \{Z_0, A, B\}, \delta, q_0, Z_0, q_1)$

Transition Table for Push Down Automata:

Move number	State	Input	Stack symbol	Move(s)
1	q_0	a	Z_0	(q_0, AZ_0)
2	q_0	b	Z_0	(q_0, BZ_0)
3	q_0	a	A	(q_0, AA)
4	q_0	b	A	(q_0, BA)
5	q_0	a	B	(q_0, AB)
6	q_0	b	B	(q_0, BB)
7	q_0	c	Z_0	(q_1, Z_0)
8	q_0	c	A	(q_1, A)
9	q_0	c	B	(q_1, B)
10	q_1	a	A	(q_1, Λ)
11	q_1	b	B	(q_1, Λ)
12	q_1	Λ	Z_0	(q_1, Λ)
(all other combinations)				none

To construct CFG G = (V, T, P, S) where V contains S and all objects of the form [p, X, q] where X is a stack and p and q can each be either q_0 or q_1. Production of the following types are contained in P

(0) $S \rightarrow [q_0, Z_0, q]$

(1) $[q_0, Z_0, q] \rightarrow a\ [q_0, A, p]\ [p, Z_0, q]$

(2) $[q_0, Z_0, q] \rightarrow b\ [q_0, B, p]\ [p, Z_0, q]$

(3) $[q_0, A, q] \rightarrow a\ [q_0, A, p]\ [p, A, q]$

(4) $[q_0, A, q] \rightarrow b\ [q_0, B, p]\ [p, A, q]$

(5) $[q_0, B, q] \rightarrow a\ [q_0, A, p]\ [p, B, q]$

(6) $[q_0, B, q] \rightarrow b\ [q_0, B, p]\ [p, B, q]$

(7) $[q_0, Z_0, q] \rightarrow c\ [q_1, Z_0, q]$

(8) $[q_0, A, q] \rightarrow c\ [q_1, A, q]$

(9) $[q_0, B, q] \rightarrow c\ [q_1, B, q]$

(10) $[q_1, A, q_1] \rightarrow a$

(11) $[q_1, B, q_1] \rightarrow b$

(12) $[q_1, Z_0, q_1] \rightarrow A$

Allowing all combinations of p and q gives 35 productions in all.

Consider the string bacab. The PDA accepts it by the sequence of moves

$(q_0, bacab, Z_0) \vdash (q_0, acab, BZ_0)$

$\vdash (q_0, cab, ABZ_0)$

$\vdash (q_1, ab, ABZ_0)$

$\vdash (q_1, b, BZ_0)$

$\vdash (q_1, \wedge, Z_0)$

$\vdash (q_1, \wedge, \wedge)$

The corresponding leftmost derivation in the grammar is

$S \Rightarrow [q_0, Z_0, q_1]$

$\Rightarrow b\ [q_0, B, q_1]\ [q_1, Z_0, q_1]$

$\Rightarrow ba\ [q_0, A, q_1]\ [q_1, B, q_1]\ [q_1, Z_0, q_1]$

$\Rightarrow bac\ [q_1, A, q_1]\ [q_1, B, q_1]\ [q_1, Z_0, q_1]$

$\Rightarrow baca\ [q_1, B, q_1]\ [q_1, Z_0, q_1]$

$\Rightarrow bacab\ [q_1, Z_0, q_1]$

$\Rightarrow$ bacab

INTERCONVERSION

Q.77. Explain the procedure of converting a CFG into equivalent PDA with suitable example.

OR If L is a context free language then construct a PDA A accepting L empty store i.e. L = N (A)

Ans. We construct the PDA A by making use of productions in G.

Construction of A - Let L = L (G) , where G = (V, T, P, S) is a context free grammar. We construct a PDA A as follows -

$A = (\{q\}, \Sigma, V \cup T, \delta, q, S, \phi)$

where δ is defined by the following rules -

$R_1 - \delta (q, \wedge, \wedge) = \{(q, \alpha)\ A\ \alpha \text{ is in } P\}$

$R_2 - \delta (q, a, a) = \{(q, \wedge)$ for every a in Σ

The construction can be explained as follows. The pushdown symbols in A are variables and terminals. If the PDA reads a variable A on the top of the stack it makes a -move by placing the R.H.S. of any A-production (after erasing -A). If the PDA reads a terminal a on the top of the stack and if it match with the current input symbol, then the PDA erases a. The PDA halts in other cases.

If $w \in L (G)$ is obtained by a leftmost derivation.

$S \Rightarrow u_1A_1\alpha_1 \Rightarrow u_1u_2A_2\alpha_2\alpha_1 \Rightarrow \ldots \Rightarrow w$,

then A can empty the stack on application of input string w. The first move of a is by a $\wedge$ - move corresponding to $S \rightarrow u_1A_1\alpha_1$. The PDA erases S and stores $u_1A_1\alpha_1$. Then using R_2 the PDA erases the symbols in u_1 by processing a prefix of w. Now, the topmost symbol in the stack is A_1. Once again by applying the $\wedge$-move corresponding to $A_1 \rightarrow u_2A_2\alpha_2$, the PDA erases A_2 and stores $u_2A_2\alpha_2$ above α_1. Proceeding in this way, the PDA empties the stack by processing the entire string w. Let us take an example

$S \rightarrow aB \mid bA$

$S \rightarrow a \mid aS \mid bAA$

$B \rightarrow b \mid bS \mid aBB$

Solution. The required PDA A can be defined as follows -

$A = (\{q\}, \{a, b\}, \{S, A, B, a, b\}, \delta, q, S, \phi)$

where δ is defined by the following rules -

$R_1 - \delta(q, \wedge, S) = \{(q, aB), (q, bA)\}$

$R_2 - \delta(q, \wedge, A) = \{(q, a), (q, aS), (q, bAA)\}$

$R_3 - \delta(q, \wedge, B) = \{(q, b), (q, bS), (q, aBB)\}$

$R_4 - \delta(q, a, a) = \{(q, \wedge)\}$.

$R_5 - \delta(a, b, b) = \{(q, \wedge)\}$

The language accepted by grammar is

$$L = \{w \in \{a, b\}^* \mid n_a(w) = n_b(w)\}$$

Let us see the sequence of moves in processing the string baab

$(q, baab, S) \vdash (q, baab, bA)$ by rule R_2

$\vdash (q, aab, A)$ by rule R_5

$\vdash (q, aab, aS)$ by rule R_2

$\vdash (q, ab, S)$ by rule R_4

$\vdash (q, ab, aB)$ by rule R_1

$\vdash (q, b, B)$ by rule R_4

$\vdash (q, b, b)$ by rule R_5

$\vdash (q, \wedge, \wedge)$ by rule R_4

Thus, $baab \in N(A)$

Hence, A is the required PDA.

Notes:-

Due to Covid - 19

Summer - 2020 to Winter - 2021

exams were conducted online by

the respective Colleges.

Notes:-

VBD

COMPUTER SCIENCE ENGINEERING

Based on New Syllabus

Our Most Useful Titles for B.Tech. IV Sem. (RTMNU)

VBD

DISCRETE MATHEMATICS AND GRAPH THEORY

Solved Question Bank
+
University Paper Solutions

VBD

VBD

SYSTEM PROGRAMMING

Solved Question Bank
+
University Paper Solutions

VBD

VBD

DATABASE MANAGEMENT SYSTEM

WITH VIVA VOCE

Solved Question Bank
+
University Paper Solutions

VBD

VBD

DATA STRUCTURE AND PROGRAM DESIGN

WITH VIVA VOCE

Solved Question Bank
+
University Paper Solutions

VBD

VBD

COMPUTER NETWORKS

Solved Question Bank
+
University Paper Solutions

VBD

VBD

THEORY OF COMPUTATION

Solved Question Bank
+
University Paper Solutions

VBD

VBD PUBLICATIONS

- Sales Office -

Plot No. 236, Near Ram Coolers, Besides Ganesh Temple, Singada Market,
Navi Shukrawari, Mahal, Nagpur-18
Ph. : 0712-2722235, 2722236, 7620266004

For online purchases, visit WWW.VBDPUBLICATIONS.COM